

Introducción a la Programación Numérica: punto flotante y algunas aplicaciones

Miguel Angel Norzagaray Cosío

17 de enero de 2013

Preámbulo

En 1993 impartí por primera vez un curso de Métodos Numéricos para estudiantes de Ingeniería en Sistemas Computacionales, en la Escuela Superior de Cómputo (ESCOM) del Instituto Politécnico Nacional, 3 años después de haberlo cursado yo mismo. Me di cuenta que el temario era en esencia el mismo que cursé con el nombre de Análisis Numérico en la Licenciatura en Matemáticas de la Universidad de Sonora, pese a la diferencia de perfiles profesional. Igual pasa con carreras como Ingeniería Civil, Electrónica y otras donde aún se cursa esa asignatura¹.

Una de las características más distintivas de los profesionales de la computación es la programación de computadoras y eso debiera causar distinción en algunos programas de asignatura como los mencionados en el párrafo anterior. Los cursos de Análisis o Métodos Numéricos no explotan la ventaja de conocimientos de programación de los estudiantes. Simplemente se continúa impartiendo los problemas de matemáticas acostumbrados con los algoritmos de siempre.

Prefiero llamarlos *Programación Numérica*.

Creo firmemente que, en las carreras de computación, el curso de Programación Numérica debiera impartirse con un contenido que haga mejor uso de las habilidades que el estudiante está desarrollando y que además lo prepare para hacer utilizar con mayor conciencia su herramienta principal de trabajo: la computadora. Los contenidos actuales tendrían que ser modificados, determinando lo que sí es indispensable para cada perfil, pero ese es tema para otra ocasión.

En perfiles de computación de corte más científico, en carreras donde se requiera programar simulaciones por computadora o realizar cálculos numéricos intensos y sensibles, un curso optativo sobre programación científica sería de gran ayuda y algunos de los tópicos incluidos aquí serían indispensables, principalmente los de la primera parte.

Aún si se logra (o no) resolver el problema de fondo, que es la adecuada influencia de la asignatura en la formación profesional, faltaría asegurar que se tienen suficientes referencias en español con que se apoye el curso con el

¹El curso de Análisis o Métodos Numéricos ya ha desaparecido de algunas carreras de ingeniería.

nuevo contenido² o que al menos sea útil para los interesados en programación numérica seria. Mucho material se encuentra disperso en libros (algunos difíciles de conseguir), revistas y páginas web. La traducción de Alejandro Casares del libro de Michel Overton [299] es la única excepción.

Era una intención de hacer varios años escribir este material, cuando tenía una idea superficial de todo lo que había al respecto. La solicitud del Consejo Técnico del área Interdisciplinaria de Ciencias del Mar de la UABCS de que escribiera una monografía durante mi sabático en la UNISON aceleró las cosas. El apoyo recibido en la UNISON, para dar los cursos que quise, así como encontrar materiales como el casi extinto libro de Traub en su biblioteca fue de gran importancia.

Debo insistir: se cambie o no la idea de esos cursos, este texto intenta llenar un vacío en las referencias en español. No está originalmente pensado como un texto que deba seguirse capítulo tras capítulo, pero puede seguirse como tal. También permite introducirse a una mayor comprensión de la manera en que las computadoras realizan las operaciones aritméticas en punto flotante y las técnicas de programación más adecuadas. En ese sentido, puede ser útil como material de consulta a los cursos tradicionales, optativos, tesis o incluso de cursos de posgrado, sin dejar de ser una referencia para todo desarrollador profesional.

No se incluye el nivel más bajo, que es el de diseño de circuitos para realizar las operaciones aritméticas básicas. Quien lo desee puede consultar referencias especializadas como [170, 222] sobre algoritmos aritméticos para computadora para información detallada.

Mientras investigaba y organizaba los materiales, me di cuenta del enorme mundo de ideas que hay por considerar al hacer programas numéricos, por lo que me vi obligado, por honestidad, a agregar la palabra *Introducción* al título. Se requiere mucho más trabajo para incluir todo tipo de temas importantes y que estén actualizados. Cada vez se me hace más extraño que tantos resultados útiles con más de 25 años (en algunos casos casi medio siglo) no aparezcan en los libros de texto, principalmente los que son sencillos desde el punto de vista teórico.

Con la situación actual de la norma de punto flotante (recién votada en mayo de 2008), este documento presenta el estado actual de la aritmética de punto flotante y su uso, con todo y sus vacíos y deficiencias. Dentro de una década habrá que seguir de cerca las modificaciones que se le hagan a la norma para actualizar el texto.

En los preliminares matemáticos no pude evitar incluir métricas, funciones de Lipschitz y otros temas pues sí sirven al lector no matemático a acostumbrarse a esa nueva terminología. De esa manera, leer un libro o artículo especializados no será tan complicado como cuando sólo dispone de los conceptos de un libro típico de métodos numéricos para ingenieros. La presentación intenta ser más

²La triste realidad es que la mayoría de los estudiantes mexicanos de nivel superior se resisten a leer materiales en inglés.

intuitiva que formal en casi todo el texto, pero sin olvidar un sustento teórico suficiente.

Toda la primera parte se concentra en la aritmética del punto flotante y su adecuada aplicación práctica tanto para las normas que regulan su uso en el diseño de lenguajes, procesadores y compiladores, como su uso en programas y rutinas sencillas, como la norma de un vector. La intención de un capítulo dedicado a la aritmética de intervalos queda latente, espero escribirlo.

La segunda parte se forma por ejemplos concretos de aplicaciones de la aritmética de punto flotante. Desde mi punto de vista, los ejemplos son suficientemente diversos como para ofrecer un panorama amplio de lo que implica la programación numérica. Lo difícil fue decidir qué ejemplos no incluir. El tiempo invertido en lectura de referencias y principalmente la programación de experimentos numéricos no permite que en poco tiempo se abarque más.

Me hubiera gustado incluir capítulos, al menos introductorios, para álgebra lineal, integración o diferenciación, pero quedan como tarea para futuras ediciones. Lo que me interesa más es no repetir lo que aparece en los libros típicos de Análisis o Métodos Numéricos. El tema de raíces de ecuaciones es el que sí aparece en todos, pero no con la extensión que presento aquí, ni en lo teórico ni en lo práctico, por la inclusión de resultados como los de Sharkovsky y Kantorovich o los detalles de la programación de bisección o Newton. Hubo necesidad de separarlo en 2 capítulos, uno para polinomios y otro para funciones en general.

Sirva entonces como material de texto o referencia complementaria para estudiantes y profesores de programación o análisis numérico, principalmente en carreras de computación.

No me queda más que agradecer a todos aquellos que han colaborado con comentarios, observaciones e incluso preguntas no consideradas cuya respuesta merece atención. Particularmente debo agradecer a José Manuel Gutiérrez, matemático de la Universidad de La Rioja especialista en métodos iterativos (particularmente el de Newton) que leyó de buena gana todo el borrador, encontrando errores menores y mayores. También a Omar Baqueiro, exalumno de la UABCS quien me envió muchas observaciones cuando estaba terminando su doctorado en Inglaterra.

Índice general

Preámbulo	III
Índice de figuras	XVI
Índice de tablas	XVIII
Índice de códigos	XXI
Simbología	XXIII
I Fundamentos de programación numérica	1
Introducción	3
Computadoras y problemas numéricos	3
Proceso típico del error en la computadora	4
Contenido de este texto	5
Enseñanza de la Programación Numérica	7
1. Aritmética de Punto Flotante	9
1.1. Motivación histórica	9
1.1.1. Antes de las computadoras	9
1.1.2. Primeras computadoras	10
1.1.3. Primeros análisis de los errores de redondeo	11
1.2. Representación de números reales	14
1.2.1. Notación científica	15
1.2.2. Punto flotante	19
1.3. El Sistema de Números de Punto Flotante	20

1.3.1.	Selección de la base β	22
1.3.2.	Observaciones iniciales sobre $\mathbb{IF}$	23
1.3.3.	Valores importantes y propiedades del redondeo	26
1.3.4.	Distribución de los Números de Punto Flotante	29
1.3.5.	Números subnormales	31
1.4.	Propiedades de $\mathbb{R}$ y $\mathbb{IF}$	33
1.4.1.	Propiedades de los reales extendidos	33
1.4.2.	Propiedades de $\mathbb{IF}$	35
1.4.3.	Dígito de guardia	37
1.4.4.	Resultados teóricos útiles	39
2.	Norma 754 de IEEE	43
2.1.	Proceso histórico	43
2.2.	Contenido de la norma (y observaciones)	47
2.2.1.	Propósitos	47
2.2.2.	Formatos definidos	48
2.2.3.	Precisión al cambiar entre bases	54
2.2.4.	Modos de redondeo	55
2.2.5.	Operaciones	60
2.2.6.	Infinito, NaNs y bit de signo	69
2.2.7.	Excepciones	73
2.3.	Anexos de la norma	74
2.4.	Lo que faltó en la norma	76
2.4.1.	Aspectos técnicos	76
2.4.2.	Aspectos educativos	78
3.	Programando la norma	81
3.1.	Antes de la norma	81
3.1.1.	La historia inicial	81
3.1.2.	Primeras técnicas	83
3.1.3.	Transformaciones libres de errores	84
3.1.4.	Cambio de fórmulas	87
3.1.5.	Hipotenusa	90
3.1.6.	División compleja	93
3.2.	Excepciones numéricas	95

3.2.1.	Los primeros pasos	96
3.2.2.	Detección de excepciones numéricas	102
3.2.3.	Omitiendo excepciones	109
3.2.4.	Levantando banderas	111
3.2.5.	Dos problemas famosos	112
3.3.	Después de la norma	112
3.3.1.	Evaluando si se cumple	113
3.3.2.	Programación básica	118
3.3.3.	Identificando valores especiales	120
3.3.4.	Decodificando NPF	121
3.3.5.	Más sobre Transformaciones Libres de Error	124
3.3.6.	Más sobre la Hipotenusa	128
3.3.7.	La función 2^n	132
3.3.8.	Más de la división compleja	134
3.3.9.	Seleccionando el lenguaje correcto	137
II	Aplicaciones de la Programación Numérica	141
	Introducción a las aplicaciones	143
4.	Sumatorias	145
4.1.	Suma de arreglos o conjuntos de números	146
4.1.1.	Interfaces para los algoritmos de suma	148
4.2.	Algoritmos de suma recursiva	149
4.3.	Sumas problemáticas	150
4.4.	Algoritmos básicos	151
4.4.1.	La suma en acumuladores separados	151
4.4.2.	Suma en orden de magnitud creciente.	154
4.4.3.	Variantes de Demmel y Hida	154
4.4.4.	Suma por inserción o de cascada	156
4.4.5.	Cota de error de los métodos anteriores	156
4.5.	Método de Pichat	157
4.6.	Suma compensada o de Kahan	158
4.7.	Destilación de Anderson	159
4.8.	Suma con doble destilación o de Priest	161

4.9. Método de Rump-Ogita-Oishi	162
4.9.1. La idea de Zielke y Drygalla	163
4.9.2. Algoritmo de ROO	163
4.10. Algoritmo de Eisinberg y Fedele	166
4.10.1. Nueva representación para los NPF	168
4.10.2. La suma de NPF en $\widehat{\mathbb{F}}$	169
5. Raíces de ecuaciones I: funciones en general	173
5.1. Sobre el problema de raíces en general	174
5.1.1. Organización del capítulo	175
5.1.2. Presentación del problema	175
5.1.3. Geometría de las raíces	177
5.1.4. Método gráfico	178
5.1.5. Métodos no abordados en este libro.	180
5.2. Conceptos y fundamentos teóricos	180
5.2.1. Convergencia	181
5.2.2. Etapas típicas de la aproximación de raíces	184
5.2.3. Puntos fijos	185
5.2.4. Selección de x_0	190
5.2.5. Sobre los criterios de paro	196
5.2.6. Comprobación final y refinamiento	199
5.2.7. Qué reportar	200
5.2.8. Más raíces	201
5.2.9. Funciones de prueba	202
5.3. Métodos que encierran las raíces	202
5.3.1. Método de bisección	202
5.3.2. Método de la secante	209
5.3.3. Método de falsa posición o regula falsa	210
5.3.4. Método de Müller	212
5.3.5. Método de Ridder	214
5.3.6. Método de Dekker	215
5.3.7. Método de Brent	216
5.3.8. Método de Crenshaw	220
5.3.9. Método de Sharma-Goyal	222
5.3.10. Método de Neta	222

5.4.	Métodos que usan la geometría de la función	223
5.4.1.	Método de Newton-Raphson	223
5.4.2.	Variantes de Newton	232
5.4.3.	Aceleración de Aitken	236
5.4.4.	Método de Steffensen	237
5.4.5.	Steffensen modificado	239
5.4.6.	Método de Halley	241
5.4.7.	Variantes con cuadratura	244
5.4.8.	Aceleración convexa - Super Halley	246
5.5.	Convergencias superiores	247
5.5.1.	Iteración de Chebyshev	247
5.5.2.	Fórmula de Schröder	248
5.5.3.	Householder	249
6.	Raíces de ecuaciones II: polinomios	251
6.1.	Sobre los polinomios y sus raíces	251
6.1.1.	Organización del capítulo	252
6.1.2.	Presentación del problema	252
6.1.3.	Teorema Fundamental del álgebra	253
6.1.4.	Etapas típicas en la búsqueda de raíces de polinomios	255
6.1.5.	El método gráfico: Lill	256
6.1.6.	Métodos polinomiales no abordados en este libro.	256
6.2.	Evaluación de polinomios y otras operaciones	256
6.2.1.	Evaluación directa	257
6.2.2.	Representación de polinomios en lenguaje C	257
6.2.3.	Método de Horner y variantes	258
6.2.4.	División de polinomios	267
6.2.5.	Máximo común divisor de dos polinomios	267
6.3.	Conceptos y fundamentos teóricos	268
6.3.1.	Definiciones, notación y resultados básicos	268
6.3.2.	Acotación y aislamiento de raíces en general	269
6.3.3.	Reducción de grado	271
6.3.4.	Cota de Cauchy	274
6.3.5.	Cota de Knuth	275
6.3.6.	Teorema de Sturm	275

6.3.7.	Aislamiento de raíces	277
6.3.8.	Estimación de punto	277
6.3.9.	Reducción de grado	278
6.3.10.	Variedad peyorativa	279
6.4.	Probando que la aproximación de raíces es exacta	279
6.4.1.	La prueba exhaustiva	281
6.4.2.	El método de prueba más sencillo	282
6.4.3.	Polinomios de prueba para grados 2 y n	283
6.4.4.	Polinomios aleatorios	289
6.5.	Polinomios de grado menor	290
6.5.1.	Fórmula general de segundo grado	290
6.5.2.	Interfaz para la ecuación general	291
6.5.3.	Problemas potenciales de la interfaz	293
6.5.4.	Resolviendo los problemas	294
6.5.5.	Polinomios de grado 3 y 4	295
6.5.6.	Polinomios de grado 5 o mayor	297
6.6.	Métodos que aproximan una raíz a la vez	297
6.6.1.	Método de Lin-Bairstow	297
6.6.2.	Metodo de Bernoulli	299
6.6.3.	Método de Duran-Kerner	302
6.6.4.	Jenkins-Traub	302
6.7.	Métodos que aproximan todas las raíces al mismo tiempo	304
6.7.1.	Método de Graeffe	304
6.7.2.	Método de Aberth-Ehrlich	305
6.7.3.	Método de Laguerre	306
6.7.4.	Método de la Matriz Adjunta	308
7.	Programación de funciones: e^x	311
7.1.	Fundamentos de programación de funciones	312
7.1.1.	Fórmulas y propiedades	313
7.1.2.	Reducción de rango	314
7.1.3.	Tipos de aproximaciones polinomiales	320
7.1.4.	Apoximación por tablas	321
7.1.5.	Pruebas de exactitud	323
7.2.	Propiedades básicas de e^x	324

7.3. Aproximación de e^x con series de potencias	325
7.4. Reducciones básicas de rango	328
7.4.1. Reducciones sencillas	328
7.4.2. Reducciones más sofisticadas	331
7.5. Uso básico de tablas	333
7.6. Probando la rutina	335
7.6.1. Monotonicidad	335

III Apéndices 337

A. Conceptos matemáticos 339

A.1. Conceptos numéricos básicos	339
A.1.1. Error absoluto y relativo	340
A.1.2. Precisión y exactitud	343
A.1.3. Estabilidad y condicionamiento	344
A.2. Notación asintótica	349
A.3. Números complejos	350
A.3.1. Operaciones básicas	351
A.3.2. Otras Propiedades y definiciones	352
A.3.3. Consideraciones de programación	352
A.4. Vectores y matrices	353
A.4.1. Operaciones y propiedades básicas	354
A.4.2. Consideraciones de programación	357
A.5. Polinomios de Taylor	358
A.5.1. Interpretación geométrica	360
A.5.2. Consideraciones de programación	362
A.6. Distancias, normas y otros conceptos	363
A.6.1. Espacios normados	363
A.6.2. Espacios de Banach	366
A.6.3. Convexidad	367
A.7. Cambio de base	368
A.7.1. Implicaciones	370
A.8. Fracciones continuas	371
A.9. Ejercicios	371

B. Otras aritméticas	373
B.1. Aritmética de Punto fijo	373
B.1.1. Pros y contras	374
B.1.2. Usos del Punto Fijo	375
B.2. Sistema logarítmico	376
B.3. Sistema exponencial	377
B.4. Aritmética de intervalos	378
B.4.1. Cuidados al usar aritmética de intervalos	379
B.4.2. Librerías y otras herramientas	380
B.5. Aritmética de cifras significativas	381
B.6. Aritmética de redundancia	383
 Bibliografía	 385
 Índice alfabético	 415

Índice de figuras

1.1. Distribución de $\mathbb{F}$.	29
1.2. Wobbling o tambaleo.	31
1.3. El conjunto $\mathbb{F}$ con los números desnormalizados.	32
2.1. Formato binario en precisión simple.	49
2.2. Tablas de conversión de decimal a dpd y viceversa.	53
2.3. Potencias de 2 y 10.	55
2.4. Redondeo al más cercano (TiesToEven).	62
2.5. La función $(\cos(x) - 1)/x$.	71
4.1. Suma compensada.	146
5.1. Gráfica de $f(x) = x^2 - \frac{e^x}{2}$.	178
5.2. Función con dos raíces aparentes.	179
5.3. Raíz con multiplicidad.	179
5.4. Gráfica de una función de apariencia inocente.	207
5.5. El acercamiento a las raíces aclara la situación.	208
5.6. Aproximaciones del método de Dekker.	216
5.7. Interpolación cuadrática inversa.	218
5.8. Método de Newton: el punto de cruce de la recta tangente es la siguiente aproximación.	224
5.9. Comportamiento de una raíz múltiple.	230
5.10. Problemas de Newton-Raphson al acercarse a una raíz múltiple.	239
5.11. Comparación gráfica de los métodos de Newton y Halley.	243
5.12. Aceleración convexa de Newton.	246
6.1. Raíces acotadas por la cadena de Sturm.	277

6.2. Gráfica de $p(x) = 987x^2 - 1220x + 377$	280
7.1. Diferencia entre la función exponencial y el polinomio de Taylor.	322
7.2. La función exponencial cerca de 0.	336
A.1. Precisión y exactitud.	344
A.2. El área bajo la curva $y_n(x)$ va disminuyendo en $[0, 1]$	347
A.3. Backward error y forward error.	348
A.4. Gráfica de $f(x) = \frac{e^x}{5} - 3 \sin x^2$ y su aproximación con $p(x)$	361
A.5. Gráfica de $f - p$ en dos intervalos distintos.	361
A.6. La función convexa define un conjunto convexo.	368
B.1. El error relativo del Wobbling o tambaleo.	377
B.2. Casos de comparación de intervalos.	380

Índice de tablas

1.1. Números en punto flotante.	19
2.1. Valores de los parámetros de punto flotante para números binarios.	48
2.2. Interpretación del formato binario en precisión simple. Ver la ecuación (2.1).	49
2.3. Interpretación del formato binario en precisión doble.	50
2.4. Interpretación del formato binario en precisión cuádruple.	50
2.5. Algunos valores límites aproximados de $\mathbb{F}$	51
2.6. Conversión directa de base 10 a binario con bcd.	52
2.7. Diferencia entre el redondeo empírico con el TiesToEven.	57
2.8. Algunas expresiones con resultados especiales de $\mathbb{F}$	70
3.1. Ejemplos de de cambio de fórmulas para evaluar con seguridad.	89
3.2. Excepciones indicadas por la norma 754 de IEEE con su resultado correcto.	103
3.3. Funciones para manejo de excepciones en C	104
3.4. Macros que puede responder fpclass.	105
3.5. Representación de excepciones numéricas para POSIX.	108
3.6. Bits del registro MXCSR para manejo de excepciones.	110
3.7. Algunas instrucciones de punto flotante equivalentes en C.	121
3.8. Algunas funciones numéricas básicas desde lenguaje C (c99).	123
3.9. Casos por considerar de 2^n	132
5.1. Raíces con multiplicidad y las derivadas.	176
5.2. Órdenes de convergencia más comunes.	183
5.3. Multiplicidad vs. orden de convergencia.	183
5.4. Iteraciones de punto fijo.	188

5.5. Posibles valores iniciales para buscar raíces.	192
5.6. Tabla comparativa de las iteraciones de Bisección y Newton-Raphson.	229
5.7. Comportamiento de Bisección y Newton-Raphson ante una raíz múltiple.	229
5.8. Iteraciones necesarias con multiplicidad fija y calculada.	234
5.9. Comparación de Newton y Steffensen ante una raíz múltiple. . .	238
5.10. Iteraciones del método de Steffensen con $(x-1)(x-10)^5$ comenzando en 10.35, con un error relativo de 10^{-15}	238
5.11. Comparación de Steffensen con la modificación del autor. En este caso la mejora es evidente.	240
5.12. Comparación de Newton, Steffensen y la modificación del autor para diversos valores iniciales.	240
6.1. Evaluaciones de $p_n(x_k)$, donde se ven varios efectos del redondeo.	262
6.2. Cadena de Sturm de p (ver texto).	276
7.1. Los rangos válidos para evaluar la función e^x en $\mathbb{F}$, considerando precisión doble.	325
A.1. Aproximaciones sucesivas de $\sqrt{2}$	340
A.2. Aproximaciones sucesivas de 3^{-n} con un método inestable y precisión simple.	346
B.1. Números en notación de punto fijo.	374

Índice de códigos

1.1. Calculando ε_M (épsilon de máquina).	27
2.1. Comprobando si hay diferencia con los valores intermedios	63
3.1. Algoritmo FastTwoSum para duplicar la precisión al sumar.	85
3.2. Macro para duplicar la precisión al sumar cantidades con precisión ya duplicada.	86
3.3. Macro mejorada para duplicar la precisión al sumar.	86
3.4. Algoritmo TwoSum de Knuth.	87
3.5. Manera correcta de calcular $e^x - 1$.	89
3.6. Método de Blue para calcular la hipotenusa.	92
3.7. Método Moler-Morrison para calcular la hipotenusa. El resultado quedará en la variable p .	92
3.8. Método Stewart para dividir números complejos.	95
3.9. Ejemplo de uso de las funciones de la norma: lectura de banderas y redondeo direccionado.	106
3.10. Usando excepciones con Fortran	107
3.11. Instalando un manejador de excepciones en (algunas versiones de) Fortran.	107
3.12. Levantando banderas y determinando la arquitectura en tiempo de compilación.	111
3.13. Levantando banderas sin conocer la arquitectura.	111
3.14. Calculando el valor de la base β .	114
3.15. Calculando la precisión.	115
3.16. Conversión de apuntadores de doble a entero en lenguaje C.	120
3.18. Unión para decodificar variables de punto flotante.	121
3.19. Otra unión para decodificar variables de punto flotante en simple precisión.	122
3.20. Campos de bits para decodificar variables de punto flotante.	122

3.21. Algoritmo PriestSum que considera redondeo fiel.	125
3.22. Algoritmo TresFMA para representar fma como la suma de 3 números.	126
3.23. Algoritmo ulp para obtener la unidad en la última posición del valor x . La constante S toma el valor $2^{-p} + 2^{-2p+1}$	126
3.24. Algoritmo SumCmplx sumar dos números complejos $S = x + y$, $x = a + ib$ y $y = c + id$. Ver el texto para más detalles.	128
3.25. La hipotenusa en Fortran 2003.	129
3.26. Algoritmo Norm2D para calcular la hipotenusa $\sqrt{x^2 + y^2}$. Ver el texto para más detalles.	131
3.27. Calculando 2^n	132
3.28. Método de Kahan para división compleja.	136
3.29. Método de Priest para división compleja.	138
4.1. Algoritmo normal de suma.	150
4.2. Algoritmo de suma de Wolfe, con dos acumuladores.	152
4.3. Algoritmo de suma con múltiples acumuladores.	153
4.4. Algoritmo de suma por inserción o cascada.	156
4.5. Método de suma de Pichat.	157
4.6. Suma compensada o de Kahan.	158
4.7. Variante de suma compensada.	159
4.8. Algoritmo de reducción de Anderson.	160
4.9. Algoritmo de destilación simple.	161
4.10. Doble destilación o de Priest.	162
4.11. Algoritmo de separación de ROO para vectores.	165
4.12. Cálculo del valor M para acotar n	166
4.13. Transformación de a_i en (τ_1, τ_2, a'_i)	167
4.14. Suma exacta de dos números (técnica de Dekker).	167
5.1. Esqueleto de una función muy inocente que itera ϕ hasta alcanzar un punto fijo con alguna tolerancia eps	199
5.2. Método de bisección para aproximar ceros de ecuaciones (versión simple).	204
5.3. Método de bisección para aproximar ceros de ecuaciones (versión no tan simple, pero aún incompleta).	205
5.4. Cálculo de un ϵ adecuado.	206
5.5. Método de Brent para aproximar ceros de ecuaciones.	219
5.6. Función que controla la convergencia en el método de Crenshaw.	221

5.7. Esquema general del método de Crenshaw.	221
5.8. Método de Newton-Raphson para aproximar ceros de ecuaciones.	228
6.1. Estructura para representar polinomios.	257
6.2. Creando un polinomio.	258
6.3. Método de Horner para evaluar polinomios.	259
6.4. Método de Horner-Adams para evaluar polinomios con cota del error.	264
6.5. Método de Horner para evaluar la derivada de un polinomio.	265
6.6. Realizando una división sintética.	278
6.7. Intento de prueba exhaustiva para el cálculo de raíces de la ecuación cuadrática.	282
6.8. Calculando las raíces de una ecuación cuadrática.	292
6.9. Ejemplo de uso de la función <code>EcGral</code>	293
7.1. Versión deficiente de la función exponencial.	326
7.2. Versión un poco más eficiente de la función exponencial.	327
7.3. Separando la función exponencial en sus partes fraccionaria y entera.	329
7.4. Método sencillo y eficientemente para calcular x^n	329
7.5. La función exponencial con la técnica de Hull. Se han suprimido las comprobaciones de los límites.	331
7.6. La función exponencial usando una tabla uniformemente distribuida.	334
A.1. Ejemplo de uso de números complejos en C++.	353
A.2. Creando una matriz de $m \times n$ en tiempo de ejecución, en len.	358

Simbología

Los siguientes son símbolos (y acrónimos) que aparecen con frecuencia en el texto. Algunos se usan con un único significado, como $\mathbb{F}$, pero otros pueden usarse de dos maneras, como el caso de m , utilizado para la mantisa o para la multiplicidad de una raíz. La página indica el lugar donde se define, si es que ocurre.

Varios de los símbolos aparecen con el mismo significado que en la literatura convencional. En el caso que diversos autores utilizan notación distinta se ha seleccionado la más común o la que es más consistente con el resto del texto. Por ejemplo, $[x]$ se usa para la parte entera en algunos textos, pero aquí se usa para representar un intervalo al que pertenece x .

No es del todo sencillo determinar aquello que el lector requiere, pero se espera que esta colección sea suficiente para facilitar la lectura fluida. Cuando no hay referencia a una página se trata de un símbolo sin definición necesaria.

Sigla	Significado	Pág.
APF	Aritmética de Punto Flotante	—
NPF	Número de Punto Flotante	—
NaN	Not a Number, cantidades que no se pueden calcular	20

Símbolo	Significado	Pág.
β	La base del sistema numérico	15
$\mathbb{C}$	El conjunto de los números complejos	364
ε_M	La épsilon de máquina	26
e	El exponente (principalmente en la primera parte), pero también la base de los logaritmos naturales	15
$e_{\text{máx}}, e_{\text{mín}}$	Límites del exponente para los números de punto flotante.	20
η	Menor número subnormal, $\eta = \beta^{1-e_{\text{mín}}-p}$	32
$\mathcal{F}$	La inversa de f , es decir, f^{-1}	217
$\mathbb{F}$	El conjunto de los números de punto flotante	20
$\mathbb{F}_p$	El conjunto de los NPF con precisión p	20
f, f'	Una función y su derivada	—
$f^{(n)}$	La derivada n -ésima de f	—
$\text{fl}(x)$	El NPF más cercano al número real x	27
$\text{fl}_{\Delta}(x)$ y $\text{fl}_{\nabla}(x)$	Números de punto flotante más próximos a x	28
$L_f(x)$	La convexidad logarítmica de f en x	367
m	La mantisa, pero también la multiplicidad de una raíz (en el capítulo 6)	15
μ	La unidad de redondeo, $\mu = \frac{1}{2}\varepsilon_M$	27
$\odot$	Cualquiera de las operaciones $+, -, \times, \div$	—
p	Normalmente un polinomio o la precisión	20
$\mathbb{P}^n$	El conjunto de los polinomios de grado n con coeficientes en $\mathbb{R}$	268
$\mathbb{R}$	El conjunto de los números reales	—
$\mathbb{U}$	Números subnormales	31
ufp	Unidad en la primera posición	16
ulp	Unidad en la última posición	16
$u(x)$	La corrección de Newton $\frac{f(x)}{f'(x)}$	224
$\bar{x}_i$	El promedio de los valores x_i	—
$\underline{x}$ y $\bar{x}$	Números de punto flotante que rodean a x en aritmética de intervalos	378
$[x]$	El intervalo que contiene a x , es decir, $[\underline{x}, \bar{x}]$. También la diferencia dividida de orden 0, $[x] = f(x)$.	378
$[x_k, \dots, x_{k+j}]$	La diferencia dividida de orden k	212

Parte I

Fundamentos de programación numérica

Introducción

Ya son varias décadas desde que la humanidad comenzó a utilizar computadoras, al menos del tipo que hoy se conocen, para hacer cálculos científicos y financieros. Desde los primeros intentos de cálculo fue claro que no se tendrían los resultados matemáticamente correctos. Aunque desde el principio se diseñaron formas de resolver los problemas de falta de precisión, aún después de muchas décadas es muy común que los profesionales de la computación y áreas afines como matemáticas, física y algunas ingenierías ignoren la magnitud de los problemas que pueden surgir.

Es un problema nada fácil la construcción de un modelo matemático que representa un fenómeno físico o financiero. Tras su construcción y validación, obtener resultados mediocres no es agradable. Lo verdaderamente perturbador es que la deficiente precisión y exactitud no solo no permita resultados correctos sino que en ocasiones la magnitud de los errores es excesiva. En el peor de los casos, no es fácil predecir el tamaño del error.

Precisión no es lo mismo que exactitud, según se define más adelante.

Computadoras y problemas numéricos

Los cálculos numéricos con computadora pueden acarrear una gran cantidad de errores de redondeo. Es imposible evitarlos pero se pueden programar las operaciones con cuidado para minimizar los errores.

Este tema es comunmente considerado como demasiado especializado y hasta “esotérico” por algunos profesionales de la computación y suele pensarse que para los sistemas comunes y corrientes no se necesita. Sin embargo, sólo hay que considerar que por algún motivo:

- casi todo lenguaje de programación dispone de tipos para punto flotante,
- todo tipo de computadoras incluye aceleradores para cálculos de punto flotante,
- todos los compiladores incluyen instrucciones y opciones especiales para punto flotante,

- todo microprocesador incluye excepciones de punto flotante que respeta el sistema operativo y
- cuando se sume una nómina, cuadre un proceso contable o se amorticen costos a largo plazo, no siempre estarán correctos los centavos.

Es entonces un recurso cuyas generalidades deben ser conocidas por todo analista, diseñador y programador de sistemas y los detalles finos por quienes abordan la nada sencilla tarea de hacer rutinas matemáticas robustas.

Una computadora no analiza, simplemente ejecuta lo que se le dice y los resultados de un mismo algoritmo pueden variar aún en condiciones muy similares. La intuición no es una buena herramienta en este tipo de sistemas. Como la aritmética nativa de la computadora comete errores, es necesario poder medirlos para controlarlos, prevenirlos y evitar al máximo su propagación.

Los errores que se consideran en diversas áreas de aplicación son de diferente magnitud, ya sea el investigador que experimenta con algoritmos de alta precisión en los límites de la capacidad del hardware o el técnico que va al campo a hacer mediciones en condiciones poco o nada controladas. El analista numérico espera que los errores de punto flotante sean pequeños y aún así se preocupa, como bien presenta Stewart en su reporte [354], por los errores de las variables reales, deficiencia de lectura de campo o equipo con poca precisión, que ocasionan gran diferencia en la salida de un programa.

Los errores en las cantidades iniciales del usuario, por lectura deficiente o lo que sea, no se pueden controlar y se propagarán posiblemente creciendo poco a poco en el mejor de los casos. Por otra parte, hay programas donde no hay entrada de datos, sino solo un proceso matemático (como una simulación) que origina una respuesta, como en algunas simulaciones. Aún en este segundo caso, lo errores de redondeo y conversión deben poder medirse.

Proceso típico del error en la computadora

El usuario promedio confía *a priori* en las computadoras y calculadoras. De manera simplista, regularmente ocurre lo siguiente.

1. El usuario captura un número exacto x , generalmente en base 10.
2. La computadora almacena x con su formato interno y redondea si no existe representación exacta³. La base interna es generalmente 2.
3. Ya no hay x sino la aproximación $x + \delta$. Se espera que δ sea muy pequeña.

El usuario normalmente cree que la computadora sí almacena sus datos originales.

³La excepción es cuando el número es un racional que tiene representación exacta en binario y es correctamente transformado por el compilador, o bien, cuando el programador experto escribe a mano tal representación binaria.

4. La computadora hace operaciones (inexactas) con $x + \delta$ y obtiene el resultado $f(x + \delta)$.
5. Para presentar el resultado al usuario, convierte $f(x + \delta)$ del formato interno a la salida que el usuario lee, redondeando de ser necesario.
6. El usuario lee el resultado $f(x + \delta) + \epsilon$, causado por el redondeo en la conversión.

Obsérvese que en algunos casos, la representación puede ser exacta. Si x es un entero menor que 1000 y el programa lo multiplica por dos, no habrá error alguno. Pero por lo general, los problemas incluyen números fraccionarios para los que es común que no exista representación exacta en base 2. Allí comienzan los problemas, en el paso 2, ya que se ocasiona lo que nos dice el paso 3: x se convierte en $x + \delta$, próximo a x pero distinto. En algunas aplicaciones, esta diferencia es tolerable, en otras no.

Por ejemplo, en algunas computadoras, el .34 se almacena internamente como .3400000035762786865234375 y cuando se le presenta al usuario, se redondea de nuevo a .34, lo que significa que el usuario no ve realmente las cantidades con las que está trabajando⁴. En este caso no hay problema aparente, pero cuando la computadora comienza a hacer cálculos y más cálculos partiendo de las aproximaciones, el resultado, aún redondeado, puede ser incorrecto. ¿Qué tanto? Esa es la pregunta importante y requiere de análisis y diseño cuidadosos. ¿WYSIWYG?

De manera más sencilla, cuando un cálculo termina en una cantidad como 21.00000000001 es fácil suponer que el último dígito es incorrecto. Pero si el número es 3.394762376385945, no es obvio saber hasta dónde se puede confiar.

En algunos textos de programación, se indica a los principiantes que las variables flotantes de precisión simple son más rápidas pero menos precisas, comparadas con las de doble precisión. La sorpresa es cuando el estudiante diligente hace una prueba y se percata de que sí son menos precisas, pero no más rápidas en muchas ocasiones, pese a que muchos libros de texto y manuales de programación así lo afirman.

Lo mejor para el programador es interesarse en comprender las fuentes del error y su correcto uso en el desarrollo de programas. Decidirse por aprender sólo unas cuantas recetas para resolver los errores más comunes es práctico pero riesgoso. En el caso de estudiantes, lo mejor es cuidar la parte formativa y comenzar con las bases.

Contenido de este texto

Con este texto ambas cosas son posibles, aprender recetas o su fundamento. El lector decidirá el que le es más útil.

⁴El cómputo numérico no es un sistema WYSIWYG (*what you see is what you get*, lo que ves es lo que tienes).

En los preliminares matemáticos, aritmética de punto flotante y el estudio de la norma de IEEE está toda la información necesaria para entender ejemplos y programar rutinas con cuidado. Se recomienda especial atención a la aritmética de punto flotante, donde se estudian las propiedades del conjunto de números que la computadora utiliza para representar a los números reales.

Un tema que ha quedado fuera es el de la aritmética de precisión arbitraria (idealmente infinita), que ofrece otras posibilidades y que es empleada por algunos paquetes comerciales y bibliotecas de funciones. Su uso tiene un alto costo computacional, pero es necesario en algunas circunstancias, lo que queda bien presentado en [134].

Toda la exposición supone que se trabaja con escalares, aunque es posible desarrollar la teoría para vectores. Cuando el tema lo amerite, se comentarán las extensiones que complementen el caso escalar, ya sea para números complejos o para espacios vectoriales.

En la segunda parte se presentan aplicaciones de la programación numérica que ilustran el uso y efectos de la aritmética de punto flotante. Primeramente el caso de la suma de números de punto flotante para pasar a la aproximación de raíces de polinomios y ecuaciones en general y terminar con una introducción a la programación de funciones elementales, donde se toma el caso de la función exponencial como ejemplo.

Los programas y códigos que se presentana lo largo del texto fueron, en su mayoría compilados, utilizando gcc 4.5.2 en su instalación MinGW, utilizando CodeBlocks como ambiente de desarrollo. En muchos casos se probó utilizando otras herramientas, en particular Dev-C, Borland C e IDL. Los códigos están escritos en C y no C++ debido a que en C++ la sobrecarga de operadores implícita impide asegurar el redondeo correcto en los cálculos intermedios de una expresión aritmética.

Aún cuando la versión oficial de C es la de 1999, como dice el documento ISO/IEC 9899:TC2, elaborado conjuntamente por ISO⁵ e IEC⁶, la mayor parte de los compiladores no la implementa en su totalidad. Lo mismo ocurre con otros lenguajes. El problema es que los equipos de desarrolladores usan el estándar de cada lenguaje como especificación de requisitos, por lo que las correcciones o modificaciones al estándar pueden ser difíciles de tomar en cuenta.

Por ejemplo, el grupo de trabajo que desarrolla la excelente colección de compiladores de GNU (gcc) advierte en el manual de uso que los lineamientos especificados por la norma internacional de aritmética de punto flotante no están incluidos en su totalidad. Además, indican en la introducción del manual que se basan el documento que especifica el lenguaje C en la versión anterior, que es la de 1989, conocida como C89 o C90, publicada en 1990, ratificada por ISO/IEC ese mismo año, con correcciones publicadas en 1996. Este grupo de trabajo agrega algunos lineamientos de c99 como extensión de gcc, seleccionables como opciones del compilador, pero de manera parcial.

⁵International Organization for Standardization.

⁶International Electrotechnical Commission.

El texto se desarrolló en L^AT_EX, utilizandola distribución MikTeX 2.6, editado con TexnicCenter. La herramienta JabRef 2.3.1 [187] fue de gran ayuda para el manejo de las referencias. Casi todas las gráficas fueron elaboradas en Scientific WorkPlace, utilizando el motor de Maple, salvo las que fueron producidas por hojas de cálculo como Microsoft Excel o Calc de OpenOffice.

Este es una panorama muy amplio de programación numérica que pretende informar y sensibilizar principalmente a estudiantes sobre los cuidados, dificultades y soluciones al desarrollar programas donde la aritmética de punto flotante sea importante.

De cualquier manera, ya es ventaja tener reunida toda esta información en un sólo documento. El lector podrá encontrar la misma información, pero distribuida en libros, artículos de revistas y no publicados y una gran cantidad de páginas web. La mayor parte se encuentra en inglés, lo que ocasiona problemas muchos estudiantes.

Enseñanza de la Programación Numérica

En este punto no se pretende profundizar. Este no es un texto sobre didáctica de la programación. Lo único se hace es incapié en ideas básicas, que permitan al lector interesado orientar más los esfuerzos propios o ajenos por adquirir habilidad para desarrollar programas numéricos.

Aprender a programar requiere esfuerzo similar al de otras actividades profesionales. Programar es una labor intelectual que comparte muchas características con varias disciplinas científicas. Se requiere abstracción, memoria, creatividad, orden e incluso buen gusto. Sin embargo posee también características propias, comenzando con el hecho de que se realiza como una artesanía. Los métodos formales para desarrollar sistemas ha dado frutos, pero distan mucho de ser de uso generalizado. La humanidad lleva siglos dando clases de matemáticas y otras disciplinas, pero solo unas cuantas décadas enseñando a programar. Utilizar las mismas técnicas solo por inercia no necesariamente es lo correcto.

Para desarrollar programas numéricos se requieren las mismas habilidades que para programar en general: especificar, diseñar, codificar, depurar y probar. Lo único que se agrega es el conocimiento de los fundamentos de la operación del punto flotante y de los recursos que el entorno numérico ofrece (o debiera ofrecer) al programador.

De eso trata este libro: de que el programador sea sensible a los riesgos de hacer programar numéricos sin cuidado y de cómo tener ese cuidado necesario para reducir los fallos.

Por lo tanto, todas las habilidades como programador debieran ser practicadas con ejemplos de problemas numéricos que garanticen la comprensión de las bases teóricas y de los recursos que la norma numérica pone al alcance. Con esto, todo profesional de computación, en particular los de orientación científica,

podrán seleccionar entorno y herramientas adecuadas para lograr la exactitud y precisión deseadas de los resultados.

Capítulo 1

Aritmética de Punto Flotante

It makes me nervous to fly on airplane, since I know they are designed using floating point arithmetic.

A. Householder.

Además de presentar los fundamentos teóricos de la Aritmética de Punto Flotante (APF) y las propiedades de los Números de Punto Flotante (NPF), en este capítulo se da un panorama introductorio de los problemas a los que se enfrentaron los primeros diseñadores de computadoras que utilizaron programas para hacer cálculos numéricos.

1.1. Motivación histórica

El lector no interesado en detalles históricos sobre el desarrollo de la aritmética de punto flotante puede saltarse esta sección¹ y pasar directamente a la sección 1.2. En muchas áreas científicas es imposible encontrar los orígenes exactos de las bases fundamentales, no así en cómputo numérico, por lo que se le dedica un poco de atención.

1.1.1. Antes de las computadoras

Se pueden encontrar en la historia de la ciencia una gran cantidad de progresos teóricos y técnicos que fueron fundamentales en el desarrollo de las compu-

¹Los diversos documentos citados en esta sección son de lo más interesante que el autor leyó para la elaboración de este libro.

tadoras modernas. No se pueden olvidar las calculadoras mecánicas como la pascalina, ideada por Blais Pascal en 1645 para realizar operaciones aritméticas con acarreo, calculadora que dio inicio a la fabricación de instrumentos similares, cada vez más complejos, por toda Europa.



Alan Turing, (de 1912 a 1954), matemático británico, creador del modelo teórico de computadora moderna, más conocido por descifrar códigos militares alemanes.

De igual forma, el desarrollo del álgebra booleana, las tarjetas perforadas como entrada y salida de datos, las impresoras, los tubos de vacío, relevadores y flip-flops abrieron el camino para construir máquinas cada vez más sofisticadas, llegando a la posibilidad de tener máquinas con memoria para almacenar información y utilizarla con operaciones en secuencia, escritas en tarjetas perforadas.

El trabajo que dejó ver el potencial de las calculadoras fue [371], sobre números computables, de Turing. Deja claro que se pueden desarrollar máquinas para no sólo hacer operaciones aritméticas, sino para ejecutar conjuntos de instrucciones y poder hacer cálculos mucho más elaborados. Este trabajo fue de gran influencia en el desarrollo de algoritmos y computadoras.

Con todo y los progresos anteriores, es bastante difícil ponerse en la situación de los primeros científicos que emplearon computadoras para hacer cálculos numéricos, a mediados del siglo XX. Muchas generaciones de científicos acostumbrados a la exactitud de las matemáticas, principalmente con los infinitos números reales y complejos enfrentándose a un dispositivo electromecánico en el que irremediamente había que tener precisión limitada. En ese entonces nadie tenía idea de las implicaciones de la propagación de errores al realizar una serie de cálculos, aunque se sabía que con registros limitados por el tamaño, las computadoras no podrían almacenar números reales.

1.1.2. Primeras computadoras

Pronto los científicos que diseñaban computadoras se encontraron con el problema de cómo representar números reales en la computadora. Inevitablemente el rango estaba limitado, así como la cantidad de cifras de los números, pero ante la infinidad de maneras de escribir el mismo número, considerando bases distintas y escalas, había que analizar con calma. Hasta los números enteros quedan restringidos a un rango finito fijo. Sí era claro que distinguir entre 2 valores de pulsos eléctricos hacía muy fácil emplear sistema binario.

Konrad Zuse (1910-1995), científico alemán diseñador de las primeras computadoras.

El primero en diseñar computadoras en el sentido moderno fue Zuse. Ya había diseñado una aritmética binaria de punto flotante con muchos de los elementos actuales para sus computadores, desde la Z1 en 1936 hasta la Z3 en 1941, pero los documentos fueron destruidos. Z3 ya empleaba 22 bits y aritmética de punto flotante, 8 para exponente y 14 para mantisa, lo que le permitía un rango de $\pm 2^{-63}$ y $\pm 2^{63}$, además de manejo de excepciones. Esto además del primer lenguaje de programación, diseñado en 1945: el Plankalkül. Zuse reconstruyó sus computadoras después de 1960. Se dice que estos esfuerzos, aislados (y destruidos) por la guerra no tuvieron influencia definitiva en el desarrollo de las computadoras modernas.

Durante la segunda guerra mundial, la construcción de computadoras (y toda la teoría relacionada) se consideraban secretos militares. Estados Unidos de Norteamérica e Inglaterra tenían sus propios desarrollos. Es esta época surge la famosa ENIAC, computadora que no guardaba programas en memoria, sino que el cableado tenía que recomponerse para otro cálculo distinto², trabajo de John William Mauchly y J. Presper Eckert, comenzada en 1943 y terminada en 1946. Trabajaba con aritmética decimal con punto fijo, con rango de $10^{-10} - 1$ a $1 - 10^{-10}$.

La gigantesca ENIAC era programada por un equipo de 6 mujeres, que suplieron a las 80 matemáticas (injustamente llamadas sub-profesionales) que elaboraban tablas de balística para los artilleros del ejército. Se dice que los operadores se la pasaban la mayor parte del tiempo viendo cuál de los 17,468 tubos de vacío (bulbos) estaba fallando.

Mauchly y Eckert hablaron de su diseño a von Neumann en 1944 y sobre mejoras posibles a ENIAC, ya planeando un diseño posterior. Derivado de esto, en 1945 von Neumann escribe [382], la propuesta para la construcción de la computadora EDVAC, que fue muy difundida, controvertida pues no aparece crédito alguno, pero no hay duda de que no eran las intenciones del gran científico. Por ello se le llama arquitectura de von Neumann.

John von Neumann (1903 - 1957), científico húngaro considerado el último de los grandes genios. Hizo importantes contribuciones en matemáticas, física, economía y computación.



Es interesante que von Neumann estima en 10^{-8} la exactitud necesaria en cálculos de ecuaciones diferenciales, que equivale a 2^{-27} en binario. EDVAC usaría aritmética binaria con 30 bits, con números de punto fijo entre -1 y 1 . Esto requiere cuidados especiales para que ninguna operación aritmética de resultados fuera del rango especificado. Los controles del escalamiento necesario pueden verse en [66]. El redondeo se realizaba con las terminaciones 00, 01, 10 y 11 pasando a 0, 1, 1 y 1 respectivamente, lo que no es óptimo.

El mismo año (1945) en Inglaterra, Turing escribe la propuesta de la computadora ACE (Automatic Computing Engine) en [369], con información muy detallada sobre la electrónica, a diferencia de [382] de von Neumann. La ACE sería terminada hasta 1950, aunque no la propuesta exacta de Turing, por lo que se llamó Pilot ACE. Disponía de tipo binario de punto flotante, con rango desde 10^{-70} a 10^{86} , incluyendo negativos. Turing afirma que el error en el último bit equivale a 2.5 o 5 partes en 10^{10} , es decir, un error relativo máximo de 2^{-31} , en la representación de números reales.

1.1.3. Primeros análisis de los errores de redondeo

Ni von Neumann ni Turing (ni el resto de sus colaboradores) pudieron disponer de las computadoras fabricadas para hacer experimentos numéricos, pero sí hicieron análisis de los resultados que se podían esperar. Cabe decir que en esta época, se tenía muy poca confianza de que se pudiera resolver con compu-

²La idea de los programas en memoria se desarrolló mientras se construía ENIAC, pero al ser necesaria como proyecto militar, se prefirió terminar el proyecto como estaba diseñado.

tadora sistemas de tamaño medianos. Esta opinión estaba apoyada por análisis muy burdos, que presentaban errores numéricos exagerados.

Por lo mismo, aquellos que aplicaban eliminación gaussiana con pivoteo a sistemas de ecuaciones lineales de $n \times n$ terminaban muy sorprendidos de la exactitud de las soluciones, contra lo incorrecto de los pronósticos teóricos, que indicaban un factor de error de 4^n . Estas aproximaciones burdas, así como algunas formas de prevenirlos, ya las reportaba Hotelling en [171], publicado en 1943³.

Surgen así surgen los primeros dos trabajos conocidos que introducen el tema de los errores de redondeo al hacer cálculos con computadoras. Ambas publicaciones analizan problemas de álgebra lineal, pues sus métodos de solución implican operaciones finitas, no trascendentes, en especial los métodos directos, por lo que los errores de redondeo dominan sobre otro tipo de errores, como la exactitud de los datos de entrada o lo idealizado de los modelos con respecto a la realidad.

Von Neumann

En 1947 nace el análisis numérico moderno, en opinión del autor.

El primero de estos dos trabajos seminales es [380], publicado en 1947, de von Neumann y Goldstine, en el que usan el problema de la inversión de matrices. En este extenso trabajo (79 páginas) el análisis lo hacen para una arquitectura como la EDVAC, con aritmética de punto fijo, con números entre -1 y 1 . Estudian la naturaleza de las operaciones aritméticas para encontrar cotas de error útiles para el problema de la inversión de matrices. La parte que interesa es la inicial y se sintetiza a continuación.

Primero definen los números digitales $\bar{x}$ como secuencias de s bits con un signo ϵ al inicio, como

$$\bar{x} = \epsilon(\alpha_1, \alpha_2, \dots, \alpha_s) \quad (1.1)$$

$$= \epsilon \sum_{k=1}^s \beta^{-k} \alpha_k \quad (1.2)$$

donde cada α_i es un dígito en base β , preferentemente 2 o 10, según el artículo.

Además, definen las *pseudo-operaciones* $\bar{x} \pm \bar{y}$, $\bar{x} \times \bar{y}$ y $\bar{x} \div \bar{y}$ como las contrapartes inexactas (por redondeo) de las operaciones aritméticas exactas $x \pm y$, xy y x/y . Dan preferencia a estimaciones probabilistas contra las exactas pues su cota para el error en los cálculos exactos es demasiado grande. Especial cuidado tienen al analizar el producto

$$\sum_{i=1}^m \bar{x}_i \bar{y}_i \quad (1.3)$$

³Se suponía que para resolver un sistema de $n \times n$ se requería guardar hasta $n \log 4$ cifras en los cálculos intermedios. Un sistema de 20 incógnitas necesitaría de 28 cifras significativas para cálculos intermedios.

pues aparece constantemente en operaciones vectoriales, matriciales y estadísticas.

Si $\bar{x}$ y $\bar{y}$ tienen s dígitos, entonces su producto tendrá $2s$, por lo que para almacenarlo habrá que cancelar la mitad de las cifras. Von Newumann y Goldstine presentan la idea de hacer el cálculo de la ecuación (1.3) usando un registro intermedio con precisión de $2s$ para acumular la suma y redondear al final. Si esta idea se emplea, entonces el error final no es

$$\left| \sum_{i=1}^m \bar{x}_i \bar{y}_i - \sum_{i=1}^m \bar{x}_i \times \bar{y}_i \right| \leq \frac{m\beta^{-s}}{2} \quad (1.4)$$

sino

$$\left| \sum_{i=1}^m \bar{x}_i \bar{y}_i - \sum_{i=1}^m \bar{x}_i \times \bar{y}_i \right| \leq \frac{\beta^{-s}}{2} \quad (1.5)$$

y la exactitud mejor es clara por el factor m . Luego de analizar el problema de inversión de matrices, terminan concluyendo que se puede tener una precisión de hasta 12 decimales en sistemas de 150×150 . Este es uno de los primeros cálculos documentados trascendentes para el análisis numérico.

Este trabajo fue continuado y se publicó una segunda parte en [381], en la que se aborda el problema desde un punto de vista estadístico, con cotas de error menos pesimistas (elevan el tamaño del sistema a 400 en vez de 150).

No se puede dejar de mencionar el artículo [172] de Alston Housholder, quien retoma los resultados de [380, 381] para estudiar la generación de errores en las computadoras digitales, encontrando cotas exactas para diversas operaciones elementales y compuestas⁴.

Turing

El segundo importante trabajo inicial se publicó al año siguiente⁵, de Turing [370], en el que de nuevo toma otro problema de álgebra lineal y analiza los errores de redondeo al resolver, con diferentes técnicas, sistemas de ecuaciones lineales. Este trabajo lo elaboró Turing un año después de resolver un sistema de ecuaciones lineales con 18 incógnitas. Se utilizó eliminación gaussiana con pivoteo total y todo el equipo estaba en espera de resultados desastrosos, cosa que no ocurrió, sorprendiendo a todos.

De manera sintética, esta investigación de Turing deja claro que las cotas que Hotelling presenta en [171] son muy pesimistas y hace referencia al trabajo de von Newmann, que dice tener resultados similares. Prueba que los errores de redondeo no necesariamente crecen de manera exponencial. Además de analizar los errores, presenta el número de condición de una matriz, el método de

Hotelling sólo analizó el caso de base 10. Decía que se requerían 48 decimales para garantizar un decimal correcto en un sistema de 78×78 .

⁴Housholder es honesto y aclara que muchos de los análisis que expone ya eran conocidos, pero que no habían sido publicados y lo hace para tener referencias y mostrar cómo se puede hacer análisis de errores, para transformar el arte del cómputo en una ciencia.

⁵Aunque fue presentado en noviembre de 1947

descomposición LU y el refinamiento iterativo para pulir la solución encontrada, mismos que son adjudicados a Turing por Wilkinson (que trabajaba para Turing) en [390], publicado el mismo año que [370].

El enfoque empleado por Turing es distinto al de von Neumann. No hace su análisis a partir de la representación específica de los números en una computadora, se basa en el término de error E que resulta de restar a la matriz identidad I el producto de la matriz original A por su inversa, A^{-1} calculada con errores de redondeo, es decir

$$E = I - AA^{-1} \quad (1.6)$$

y el error del sistema de ecuaciones lo pone en términos del número de condición de la matriz A , siempre considerando una cantidad fija de decimales en cada número.

Según Turing, la razón de calcular la inversa es que de otra manera se ve muy difícil encontrar el error en la solución de un sistema de ecuaciones. Por eso muestra una manera de refinar la inversa. El resto del artículo describe el error esperado de diversos métodos de solución de sistemas de ecuaciones.

Así, Turing y von Neumann dieron inicio al análisis de errores de redondeo, desde puntos de vista distintos. Lo que sucedió en los años siguientes fueron los análisis y propuestas de sistemas numéricos con características diversas, según cada fabricante construía sus computadoras, algunos con punto flotante, otros con punto fijo, normalizando cantidades, probando base distintas y cada quien con su propio tamaño de palabra (bits de la representación).

En 1949 se termina la construcción de la computadora EDSAC, en la universidad de Manchester. Primer computadora completamente funcional, utilizando la arquitectura de von Neumann. Mayo 6 de 1949 se considera el inicio de la computación moderna, al ejecutarse por primera vez un programa almacenado en memoria, con entrada y salida en cinta de papel. La Pilot Ace ejecutó su primer programa un año después.



James Hardy Wilkinson, (1919-1986). De los científicos más importantes en el análisis numérico, fundador del moderno análisis de errores.

Análisis formales de los errores relativos se comenzaron a calcular, como el de Casey [61] en 1959, para el caso de punto flotante. Esta nueva rama de las matemáticas, comparativamente con el resto, se consolidó sistemáticamente en el trabajo de Wilkinson [395], aunque también Forsythe [128], Golub y Van Loan [248] son referencia excelente.

En el mundo de la computación, aún se discutían las bondades de lo analógico y lo digital (ver [266]) y poco a poco los matemáticos ponían su atención a este nuevo tema de errores de redondeo.

1.2. Representación de números reales

Representar números reales en la computadora conlleva la limitante del espacio. Números con fracción infinita (como π o $\sqrt{2}$) deben ser representados con

aproximaciones finitas, lo que ocasiona un error inevitable. Primeramente, debe emplearse una forma base de escribir los números y esta es la notación científica.

1.2.1. Notación científica

Al escribir números de magnitudes muy diversas, es costumbre utilizar una notación que sea homogénea y fácil de entender, la llamada *notación científica*. La idea es representar al número real x con la terna de números (m, b, e) , llamados mantisa, base y exponente, como

$$x = m \times b^e$$

de tal forma que $1 \leq m < b$, con $b > 1$ y e es un número entero. Es común que se emplee $b = 10$, la base decimal. El ejemplo siguiente es ilustrativo.

Ejemplo 1. *Se muestran a continuación varios números y su notación científica correspondiente en base 10.*

$$\begin{aligned} 600000000000000000 &= 6.0 \times 10^{18} \\ 23\,000\,000\,000\,000\,000\,000 &= 2.3 \times 10^{19} \\ .00000000000000000012 &= 1.2 \times 10^{-20} \\ 1.0 &= 1 \times 10^0 \end{aligned}$$

Cuando se utiliza un sistema numérico con una base distinta a 10, se acostumbra indicarlo con un subíndice⁶, de tal manera que usando las bases 10, 16, 8 y 2 respectivamente, se escribe $15 = f_{16} = 17_8 = 1111_2$. Como el sistema en base 2 se usará muy frecuentemente, se omitirá el subíndice excepto cuando pueda ser confuso. La base puede ser cualquier número $\beta > 1$.

También es común que se utilice notación científica para redondear algunas cantidades o simplemente por ahorro de espacio, por ejemplo en el siguiente caso: $20000000000000000000.0000000000000000001 \approx 2.0 \times 10^{21}$, donde la cantidad 10^{-20} se ha suprimido (por eso se usa el símbolo $\approx$ y no $=$).

¿El redondeo es con fines estéticos o prácticos?

Como se ve, un número en notación científica en base 10 se forma con un número real en el intervalo $[1, 10)$ y una potencia 10. De manera más general, es de la forma

$$d_0.d_1d_2\dots d_p \times 10^e = d_0 \times 10^e + d_1 \times 10^{e-1} + \dots + d_p \times 10^{e-p} \quad (1.7)$$

con la notación llamada *expandida*, donde d_0 debe ser distinto de cero y cada d_i cumple con $0 \leq d_i < 10$, para $i > 0$. A la cadena de dígitos $d_0.d_1d_2\dots d_p$ se le llama mantisa. e es el exponente. Cualquier cantidad que se pueda descomponer como la suma finita de potencias de 10 no tendrá problemas de representación, pero cantidades como $1/3$ o π son un caso distinto, requiriéndose una infinidad de términos. Con las restricciones anteriores, la notación es única para cada número.

⁶También se usa $\mathbf{fh} = \mathbf{xf} = 1111\mathbf{b} = 1111\mathbf{B}$ para el 15 en hexadecimal y binario, pero en este texto se utilizará la convención del subíndice.

Ejemplo 2. *Considérense los números:*

$$\begin{aligned} 3.1416 \times 10^0 &= 3 \times 10^0 + 1 \times 10^{-1} + 4 \times 10^{-2} + 1 \times 10^{-3} + 6 \times 10^{-4} \\ 7.56 \times 10^{12} &= 7 \times 10^{12} + 5 \times 10^{11} + 6 \times 10^{10}. \end{aligned}$$

En computación, la notación corta $d_0.d_1d_2\dots d_p \times 10^e$ se prefiere a la expandida, más utilizada en textos de aritmética.

La cantidad p de decimales después del punto es comunmente referida como *precisión* y se fija en vez de considerar números arbitrariamente precisos como $d_0.d_1d_2\dots d_pd_{p+1}\dots \times 10^e$ pues es la mejor manera de modelar la representación limitada de las computadoras.

El *exponente* e es un número entero (positivo o negativo) finito, aunque de nuevo en las computadoras tiene restricciones: hay valores mínimos y máximos posibles.

Unidad en la última posición - ulp

Obsérvese que dos números que difieran sólo en el último dígito, tienen una diferencia que es múltiplo de 10^{e-p} . A esta cantidad genéricamente se la llama ulp por sus siglas en inglés *units in last position*.

Acuñado en 1960 por Kahan, en [194] para referirse a la exactitud de la librería matemática de las IBM 7090 y 7094.

Ejemplo 3. *Considérense las dos aproximaciones distintas del número real π , cada una con precisión distinta.*

$$\begin{array}{rcl} & 3.14159265358 & 3.1415926535897 \\ - & 3.14159265359 & - 3.1415926535898 \\ \hline & 0.00000000001 & 0.0000000000001 \\ & = -10^{-11} & = -10^{-13} \end{array}$$

En ambos casos se 1 ulp de la aproximación, cada uno relativo a su propia precisión, lo que significa magnitudes distintas. Debe ser claro que se NO se habla de ulp(π), sino del ulp de las aproximaciones.

De manera análoga, se define *ufp* como la *unidad en la primera posición*. Su valor es $\text{ufp}(x) = 10^{\lfloor \log_{10} |x| \rfloor}$.

Además de Kahan, Goldberg [140], Harrison [160], Markstein [256] y Muller [281] dieron definiciones similares (no equivalentes) entre 1991 y 2006. En 2005, Muller publicó un interesante reporte de investigación [280] en el que compara las diversas definiciones e investiga algunas propiedades. Muchas de las observaciones resultan útiles al momento de desarrollar programas que utilicen ulp como medida de exactitud o precisión. Otra forma interesante de definición aparece en [236].

La definición aceptada (la de Kahan⁷ es la que aparece en la norma de IEEE) determina que, dado $x \in \mathbb{R}$, la $\text{ulp}(x)$ es la distancia entre los dos números de

⁷Modificada ligeramente por completitud.

punto flotante finitos más cercanos a x . El problema de esta definición (y la razón de las otras alternativas) es que no es del todo clara alrededor de las potencias de la base empleada. Más adelante se detallará esto.

Con este nuevo concepto, puede definirse el de *cifras significativas*⁸. Se dice que deben diferir en menos de .5 ulp para tener p dígitos significativos en común, suponiendo que p es la cantidad de cifras utilizadas. La precisión p es un parámetro importante en las computadoras pues los números disponen de espacio limitado al ser almacenados en registros o memoria, por lo que la cantidad de cifras es esencial.

También es posible que el número que acompaña a la potencia no esté en el intervalo $[1, 10)$, con lo que tenemos números como 23.1×10^5 o $.012 \times 10^9$. Se llama entonces *notación científica desnormalizada* y no se utilizará en este texto.

Cuando en la ecuación (1.7) se tiene que $d_0 = 0$ y el primer dígito después del punto decimal es, en este caso un dígito del 1 al 9, entonces se tiene la *notación científica normalizada*. En este caso, la mantisa es un valor en el intervalo $[1/10, 1)$. La ventaja es que la representación de cada número es única, mientras que con la desnormalizada se pueden tener varias representaciones distintas del mismo número.

Ejemplo 4. El número $.231 \times 10^7$ puede representarse en notación desnormalizada como

$$23.1 \times 10^5 = 2.31 \times 10^6 = .00231 \times 10^9 = 231 \times 10^4.$$

En notación científica normalizada es 0.231×10^7 .

Notación científica en base 2

Como las computadoras representan toda su información con ceros y unos, llamados *bits*, es normal que el sistema numérico base 10 se sustituya con el de base 2. Los conceptos del apartado anterior siguen siendo los mismos, sólo las restricciones a los dígitos empleados en la mantisa cambian. Por ejemplo, la notación científica normalizada ahora es

Hay 10 tipos de personas: las que no entienden el sistema binario y las que sí lo hacen.

$$x = 0.d_1d_2d_3\dots d_p \times 2^e \quad (1.8)$$

donde d_1 es distinto de cero y cada otro d_i cumple con $0 \leq d_i < 2$. Así, se puede escribir

$$0.d_1d_2\dots d_p \times 2^e = d_1 \times 2^e + d_2 \times 2^{e-1} + \dots + d_p \times 2^{e-(p-1)}$$

y es sencillo representar cualquier cantidad que resulte de la suma de potencias de 2.

⁸Véase también la sección A.1.1 en la página 341, particularmente el ejemplo 51.

Ejemplo 5. *Considérese el número decimal 4.75 y su descomposición como la suma de potencias de 2.*

$$\begin{aligned} 4.75 &= 4 + .5 + .25 \\ &= 2^2 + 2^{-1} + 2^{-2} \\ &= 100.11_2 \\ &= .10011 \times 2^3 \quad (\text{normalizado}). \end{aligned}$$

El problema es con las cantidades que no son resultado de la suma de potencias de 2, como el $1/10$, el 0.34 o $1/5$. En estos casos, la representación en base dos requiere de una infinidad de términos, a diferencia de la notación en base 10.

La observación anterior es importante, pues implica que no hay una base numérica que tenga representación finita para todos los números reales. Para más detalles de cambios de base, se recomienda al lector consultar la sección A.7 en la página 368.

Regresando a la base 2, si $x = 0.d_1d_2d_3\dots d_p \times 2^e$, entonces se tiene que $1 \text{ ulp}(x) = d_p \times 2^{e-(p-1)}$. El número $x + 2^{e-p}$ difiere en $.5 \text{ ulp}$ de x :

$$\begin{aligned} x &= 0.d_1d_2d_3\dots d_p \times 2^e \\ x + 2^{e-p} &= 0.d_1d_2d_3\dots d_p \mathbf{1} \times 2^e \\ &= x + 1 \times 2^{e-p} \end{aligned}$$

y el último 1 equivale a $\frac{1}{2}d_p \times 2^{e-(p-1)}$, que es la última cifra de x . La medida $.5 \text{ ulp}$ es de gran importancia pues es el error máximo que se espera en las *operaciones elementales* ($+$, $-$, $\times$, $\div$, $\sqrt{}$) con una computadora, luego del redondeo final. La meta que se busca al programar rutinas matemáticas es que todo resultado tenga una diferencia menor que $.5 \text{ ulp}$ de lo matemáticamente correcto. Esto ya ocurre para la mayoría de las funciones elementales (como $\sin$, $\cos$, $\exp$ y otras), con la excepción de la potencia: x^y .

Una observación importante es que, por la forma de la de notación científica normalizada, en base 2 todo número comienza con el dígito 1 después del punto.

Ejemplo 6. *Obsérvense por ejemplo las siguientes cantidades escritas en dicha notación.*

$$\begin{aligned} &.1 \times 2^{12} \\ &.10001101 \times 2^{-9} \\ &.1111110101 \times 2^0 \\ &.1100110011 \times 2^{-4} \end{aligned} \tag{1.9}$$

Esta característica es importante porque al almacenar un número en la computadora, este primer bit que siempre es 1 no requiere ser guardado. De esta manera, se ahorra un bit en la representación y se le llama *bit implícito*⁹.

Este es precisamente el ulp o unidad en la primera posición, donde lo que importa claramente no es el 1, sino el exponente que lo acompaña.

⁹En algunos textos le llaman bit fantasma o perdido.

A diferencia del Gulp, G=giga, acuñado con sarcasmo por Fred Gustavson refiriéndose a librerías matemáticas con errores enormes.

$$\begin{aligned}
123.456 &= .123456 \times 1000 \\
&= 123456 \times \frac{1}{1000} \\
&= 1.23456 \times 100.
\end{aligned}$$

Tabla 1.1: Números en punto flotante.

1.2.2. Punto flotante

Los números de *punto flotante* son todos aquellos en los que se selecciona dónde poner el punto, agregando un factor a la representación para que el significado numérico sea el mismo. Por ejemplo en la tabla 1.1.

De esta manera, el punto “flota” de una posición a otra.

Por convención la posición del punto se selecciona para mantener la parte entera dentro de algún rango predefinido. Como es más complicado que la notación punto fijo¹⁰, muchas computadoras antiguas realizaban las operaciones aritmética de punto flotante usando rutinas de la biblioteca de funciones, usando punto fijo con el procesador¹¹.

En la actualidad, el punto flotante es el de mayor uso. El punto fijo se emplea en sistemas donde no se requiere gran precisión y el ahorro de espacio y tiempo es significativo.

Desde el inicio de las computadoras, hubo dos opciones adicionales, la normalizada y la no normalizada o desnormalizada. En una computadora que usa base 2 los números de punto flotante tienen la forma

$$x = m \times 2^e \quad (1.10)$$

donde e es un exponente en un rango fijo y m es la mantisa. Si la mantisa se restringe

$$\frac{1}{2} \leq m < 1 \quad \text{o bien} \quad -\frac{1}{2} > m \geq -1$$

entonces en sistema es normalizado. En general se prefiere la mantisa en el intervalo $\beta^{-1} \leq |m| < 1$, para cualquier base β . De otra forma, si solo se cumple que $|m| \leq 1$, entonces el sistema es desnormalizado.

En las primeras computadoras no era claro cuál era mejor y se construyeron con uno u otro sistema. En [380], el trabajo pionero de 1947, von Neumann y Goldstine analizan los errores de redondeo considerando sólo el sistema desnormalizado. El sistema normalizado era comunmente acompañado por un entero que indicaba los dígitos significativos correctos, luego de los redondeos de las

¹⁰Ver la sección B.1.

¹¹La publicación en 1964 del libro *Rounding Errors in Algebraic Processes* [395] de Wilkinson, terminó por decantar la balanza en favor del punto flotante. Este libro reunió de manera sobresaliente la experiencia de muchos años de trabajo programando computadoras, analizando y resolviendo los errores que se producían.

operaciones. En el desnormalizado, los dígitos ceros al inicio de la mantisa indicaban la precisión, además de que es menos probable que los resultados queden fuera de rango.

En la década de 1950, se hicieron numerosas investigaciones para determinar qué representación era más eficiente. Digna de mencionarse es el trabajo [191] de John Carr III, de 1957, donde deja ver con claridad las ventajas del sistema normalizado, aunque muestra un ejemplo (una serie de potencias) donde no hay diferencia. Otras investigaciones sobre el sistema desnormalizado y sistemas mixtos son [14, 115, 155, 383].

Las investigaciones mostraron que el sistema normalizado ofrece ventajas en cuanto a precisión, aunque requiere de más tiempo de cómputo. El aumento de velocidad de los procesadores permitió que este sistema fuera preferido.

1.3. El Sistema de Números de Punto Flotante

Se presenta a continuación el conjunto de los números de punto flotante (NPF) de manera parcial, indicando los valores normales y especiales¹². Un poco más adelante, con la justificación adecuada, se presentarán (y agregarán) los llamados números subnormales (en la sección 1.3.4).

Se denota con $\mathbb{F}$ al conjunto (finito) de números racionales que tienen representación exacta con una computadora, llamados *números de punto flotante*:

$$\mathbb{F} = \{(-1)^s \times m \times \beta^e\} \cup \overbrace{\{-0, 0, -\infty, +\infty, \text{NaN}\}}^{\text{Valores especiales}} \quad (1.11)$$

donde los parámetros son

- s el *signo*,
- β la *base* numérica seleccionada, comunmente 2, 4, 10 o 16,
- p la *precisión* o cantidad de dígitos empleados, comunmente 24, 53, 64 o 112,
- m la *mantisa*, un número en el rango $\frac{1}{\beta} \leq |m| < 1 - \beta^{-p}$, representado con p dígitos base β y
- e el *exponente* en el rango $e_{\min} \leq e \leq e_{\max}$.

Sin no se le dan valores a los parámetros, no se define ningún conjunto específico, por lo que este no es un conjunto único de valores. Si se cambia el valor de la precisión o los límites para el exponente, se obtienen conjuntos distintos, por lo que es más correcto llamarle Sistema de Números de Punto Flotante. Esto modela el recurso de las computadoras de tener tipos de variables numéricas

¹²Donald Knuth da una presentación axiomática más formal en [219].

La base 2 es la más utilizada. La base 16 poco a poco se ha dejado de usar.

con precisión distinta. Por eso hay quien se refiere mejor a $\mathbb{F}(\beta, p, e_{\min}, e_{\max})$ especificando los parámetros del sistema o al menos la precisión, lo que se indica con $\mathbb{F}_p$. En el resto del texto, se especificarán los parámetros sólo de ser necesario, refiriéndose siempre sólo a $\mathbb{F}$ en el contexto dado.

Así, pueden definirse sistemas que vayan en aumento de rango y precisión, como ya lo propuso Reinsch en 1979, en [318], donde da las características generales de estos sistemas, al menos las que espera un analista numérico.

Reinsch explica la necesidad de que el sistema sea monotónico, es decir, cada sistema más preciso debe, necesariamente, contener al previo: $\mathbb{F}_1 \subset \mathbb{F}_2 \subset \mathbb{F}_3 \cdots$ y tantos como subsistemas tenga el ambiente de programación. Claro, si la base se cambia, la contención no ocurrirá.

$$x \in \mathbb{F}_i \Rightarrow x \in \mathbb{F}_{i+1}$$

Entre las características deseables están cosas como

- si $x \in \mathbb{F}$, entonces $-x \in \mathbb{F}$;
- se prefiere que $e_{\min} + e_{\max} \approx 0$;
- $x \in \mathbb{F} \Rightarrow 1/x \in \mathbb{F}$ (pertenencia de los recíprocos)

y varias más. La pertenencia de los recíprocos es algo que es imposible pues siempre habrá números sin representación exacta (sea cual sea la base).

Trabajos como el de Reinsch en un sentido similar de crítica a los diseñadores y fabricantes de computadoras son [52, 53, 68, 118, 316]. El espíritu de esa época era otro y queda claro cuando se recomendaba a los programadores *regresar al trabajo, dejar los intentos de remodelar el hardware y aceptar la realidad*. Sin duda, la crítica más ácida la hace Dunham en [97], donde pone claro que las prioridades en computación están marcadas por el dinero de los inversionistas y la mercadotecnia.

Extensión teórica

Las restricciones numéricas de los exponentes se ponen para modelar correctamente las limitaciones prácticas de la computadora. En algunas ocasiones, para facilitar el análisis de algoritmos, es conveniente emplear el conjunto extendido de números de punto flotante que resulta de eliminar tales restricciones.

De esta forma, se puede definir el conjunto

$$\mathbb{F}_X = \{(-1)^s \times m \times \beta^e\} \cup \overbrace{\{-0, 0, \text{NaN}\}}^{\text{Valores especiales}} \quad (1.12)$$

donde los parámetros son como en el apartado inmediato anterior.

Otra forma de abordar el análisis sin definir el sistema numérico (1.12) es agregar a las hipótesis la frase “supóngase que no ocurre underflow ni overflow”, que es lo más acostumbrado. Sin embargo, en forma velada lo que se hace es definir $\mathbb{F}_X$ como el conjunto sobre el que se trabajará. Ejemplo de este uso puede verse en [284] y otros trabajos de los mismos autores (Boldo y Muller).

1.3.1. Selección de la base β

Es obvio que ya sea con base 2, 10 o cualquier otra, es matemáticamente posible representar cualquier número real, pero se requiere de una infinidad de cifras. Con la precisión restringida en una computadora, no es posible representar el conjunto de los números reales. No se usa 2 como base definitiva pues 10 sí es importante en algunas aplicaciones, así que β sigue representando la base numérica. Al cambiar de base se degrada la precisión sólo si la exactitud de los cálculos empeora.

En [382], von Newmann argumenta que la base 2 es preferible para la unidad aritmética de la computadora y que la base 10 es necesaria para el dispositivo externo de datos. Por ello la necesidad de rutinas de conversión desde los inicios de la computación¹³. Von Newmann reconoce que, aunque la base 10 requiere muchas menos operaciones, la necesidad de consultar tablas para cada operación aritmética hace su uso incosteable y presenta algunas de las primeras maneras de hacer eficiente el uso del sistema binario, además de justificar la necesidad de tener al menos 8 decimales después del punto para problemas de ecuaciones diferenciales (algo así como 27 dígitos binarios).

En su artículo “Finger or Fists” (dedos o puños), de Buchholz en 1959 [55], hay una de los primeros estudios comparativos publicados sobre la eficiencia de las bases 2 y 10, que no deja mucho qué agregar en favor de la base 10, dejándola como segunda opción, pero necesaria, en especial para aplicaciones monetarias, como años después sería evidente. Buchholz indica que para cada dígito decimal se requieren $\log_2 10 \approx 3.322$ dígitos binarios.

En 1960, se publica el artículo [304] en el que C. Perry muestra un algoritmo compacto para la conversión de números de punto flotante entre dos bases, aplicando fracciones continuas.

Poco después, en 1967, Bennett Goldberg presenta su artículo [142] en el que muestra un argumento irrefutable: como $10^8 < 2^{27}$, entonces es posible representar un número decimal de 8 dígitos con 27 bits, pero que se requieren 28 bits para representar todos los números de 8 dígitos decimales. Esto se debe a que en general si $10^a < 2^{b-1}$, entonces b bits son suficientes para una precisión de a decimales. Esta es una aplicación práctica del Teorema de Cambio de Base de Matula, quien presentó en 1967 el artículo [260] y posteriormente otros trabajos similares ([261, 262, 264]).

Otra referencia interesante es [54], de Brown y Richman en 1969, que justifican formalmente las bondades de la base 2 y la relación con el tamaño de la palabra y los bits de signo¹⁴.

¹³Es en este reporte donde von Newmann muestra su idea de punto fijo, manteniendo puros números entre -1 y 1 , además de puntualizar detalles del redondeo de resultados, un tanto inocentes para nuestra época.

¹⁴En cualquier caso, la conversión de los números del usuario al formato interno de la computadora y viceversa estaba limitado por el teorema de cambio de base (véase la sección A.7 o el artículo de Matula [261]).

Base dos para propósito general es lo correcto, pero algunas aplicaciones particulares se benefician con el uso de otros valores¹⁵. Incluso la base 4 resulta útil, como lo muestra Nikmehr en [292], así como muchos otros autores. La base 8 también ha sido estudiada por Nanarelli en [285]. Estos nuevos esquemas obtienen ganancias en reducción de tiempo de cómputo y consumo de electricidad, a costa de un razonable incremento del área de circuito. En ocasiones, acompañan esta base con notación en punto fijo en vez de flotante.

En el caso de arquitecturas específicas, como los FPGA (Field Programmable Gate Array, por sus siglas en inglés), para los que se han planteado diversas bases, como en [62], en 2005. Ahí, los autores muestran un sumador de punto flotante en base 16 que emplea 30 % menos área de procesador, con la misma precisión.

Más detalles sobre la aplicación de estos conceptos se verán en la sección 2.2.3, en la página 54, al presentar la norma de IEEE.

1.3.2. Observaciones iniciales sobre $\mathbb{F}$

Por el rango de la mantisa, se trata de un sistema normalizado¹⁶. NaN representa el resultado de operaciones numéricas inválidas o no definidas en una computadora, como $\sqrt{-2}$, $\log(-5)$, $\cos(\infty)$ o $\infty - \infty$. Por otra parte, los símbolos $\pm\infty$ no tienen el significado matemático convencional, sino que se emplean para cualquier número tan grande que quede fuera del rango de $\mathbb{F}$ después del redondeo.

Números representados

Es claro que el conjunto de los números reales $\mathbb{R}$ no puede representarse en una computadora, así que es bueno que todo programador tenga clara la idea de los números que sí se pueden representar. Con esta notación, el menor número es el $\beta^{e_{\min}}$ y el mayor el $\beta^{e_{\max}}(1 - \beta^{-p})$ ya que $1 - \beta^{-p}$ representa el mayor valor posible para la mantisa.

En un sistema equilibrado, como lo dice Reinsch en [318], debe cumplirse que $e_{\min} = -e_{\max}$, por lo que al multiplicar el mayor número por el menor se debe obtener 1. En el caso del sistema definido, la diferencia es tan sólo de $\text{ulp}(1)$ pues

$$\beta^{e_{\min}} \times \beta^{e_{\max}}(1 - \beta^{-p}) = 1 - \beta^{-p}$$

es decir, se cumple $e_{\min} \approx -e_{\max}$, que no ocurriría en sistemas donde se daba preferencia al rango sobre la densidad o viceversa.

¹⁵Pese a tanta argumentación, hubo autores que se pronunciaban por la base 10 como la principal, como Hull en [175].

¹⁶La definición de los NPF no es única. Es común ver que la mantisa se presenta como entero, por lo que los números toman la forma $m \times \beta^{e-p}$, pero es equivalente.

Ejemplo 7. En un sistema numérico en base 2 donde se tengan 3 dígitos de precisión y un rango de exponentes entre -2 y 2 , tendríamos números de la forma $\pm.d_1d_2d_3 \times 2^p$. El menor es $2^{-2} = .1 \times 2^{-2} = .25$, mientras que el mayor es el $2^2(1 - 2^{-3}) = .111 \times 2^2 = 3.5$.

$$.25 \times 3.5 = 1 - .125 = .875$$

A partir de estos valores se pueden calcular los límites o el intervalo real al que pertenecen los NPF:

$$\mathbb{F} \subsetneq [-\beta^{e_{\max}}(2 - 2^{1-p}), +\beta^{e_{\max}}(2 - 2^{1-p})] \quad (1.13)$$

Se usa $\subsetneq$ porque en ese intervalo existe una infinidad de números reales que no pertenecen a $\mathbb{F}$. Ni siquiera están todos los enteros, pues para valores grandes del exponente, la precisión es insuficiente para representarlos.

Ejemplo 8. Si

$$1 = 0.x_1x_2\dots x_p \times \beta^e \quad (1.14)$$

es la representación de 1, entonces se puede multiplicar por β^p y se desplaza toda la mantisa, lo que queda indicado en el exponente:

$$\begin{aligned} 1 \times \beta^p &= 0.x_1x_2\dots x_p \times \beta^e \times \beta^p \\ &= 0.x_1x_2\dots x_p \times \beta^{e+p} \end{aligned}$$

y si a este número se le suma 1, con su representación en (1.14) el resultado es la duplicación de la mantisa

$$0.x_1x_2\dots x_px_1x_2\dots x_p \times \beta^{e+p}$$

por lo que la mantisa requiere de $2p$ cifras para su representación y se deduce que $\beta^p + 1$ es un entero que no pertenece a $\mathbb{F}$.

Por ese mismo motivo, además de la imposibilidad de representar expansiones decimales infinitas, tampoco están todos los números racionales. Por último, ningún número irracional pertenece a $\mathbb{F}$, como es de esperarse.

Representación del cero

A partir de la definición es posible observar que el número cero (0) no se puede representar debido a las restricciones de la mantisa. Este inconveniente se resuelve agregando directamente el 0 al conjunto $\mathbb{F}$, como definición.

Esta observación surgió desde el inicio y los fabricantes de computadoras. Ni Neumann ni Turing lo comentan en sus diseños y debieron obviar la representación. El primer autor que aborda este problema es Gram [154], en 1964. Gram¹⁷

¹⁷Gram analiza las repercusiones de la representación del cero y los posibles efectos en diversos problemas. Es interesante que aborda el problema de la cuadratura de Romberg, aplicando el algoritmo de suma compensada de Kahan, pero sin referencias, excepto a [276], de Ole Moller.

presenta lo que llama representación civil y científica del cero, todo enfocado al lenguaje ALGOL, con un sistema numérico de punto flotante en base 2, con 29 dígitos de precisión y exponentes en el rango de $-512 \leq e < 512$, es decir, $\mathbb{F}(2, 29, -512, 511)$.

Al parecer, es la primera ocasión en que se propone que el resultado de las operaciones debe depender del origen y secuencia de los cálculos para mantener cierta congruencia.

En la sección 2.2.2 se ve más sobre la representación del cero.

Overflow y underflow

Todo número fuera del intervalo mostrado en (1.13) quedará representado por $\pm\infty$, salvo posibles redondeos. Más específicamente, cuando el resultado de una operación resulta en un exponente mayor a $e_{\max}$, el resultado será $\pm\infty$, dependiendo del signo s de la mantisa y se dice que ocurre un *overflow* (desborde).

En algunos lenguajes de programación, en variables de tipo entero, al sumar 1 al entero mayor se obtiene el entero menor, por la notación complemento a 2 empleada. No así en las variables de punto flotante. Si por el contrario, la operación genera un exponente menor que $e_{\min}$ el resultado se hace 0 y se dice que ocurre un *underflow*.

Obsérvese que si se usan los p dígitos de precisión en la notación normalizada, entonces $m\beta^p$ es un número entero. Además, se cumple que

$$-\beta^p < m\beta^p < \beta^p \quad (1.15)$$

lo que es escalamiento, por lo que en algunas ocasiones es posible simular operaciones haciendo uso de variables enteras.

Ejemplo 9. En base 10, considerando mantisa de 4 dígitos,

$$.1234 \times 10^4 = 1234$$

y cumple que

$$-10^4 < 1234 < 10^4.$$

Para ser realistas con la representación de las computadoras, sólo falta considerar el bit implícito (ver página 18) en la notación de $\mathbb{F}$ y los números subnormales, pero se hará poco más adelante.

Por completitud, se define $\text{ulp}(\text{NaN}) = \text{NaN}$, $\text{ulp}(\infty) = \beta^{e_{\max}}(1 - \beta^{-p})$.

Operaciones y pérdida de precisión

Como ya se ilustró en el ejemplo 8, es posible que las operaciones entre NPF produzcan resultados intermedios que no pertenecen a $\mathbb{F}$. De manera más

formal, sean $x = 0.x_1x_2\dots x_p \times \beta^{e_x}$ y $y = 0.y_1y_2\dots y_p \times \beta^{e_y}$ elementos de $\mathbb{F}$ y sea $e_x - e_y = p + 1$, es decir, $|x| > |y|$.

Para realizar la operación $x + y$ se requiere poner a y con el mismo exponente de x , lo que se conoce como *normalización*¹⁸. El resultado requiere una mantisa de $2p$ dígitos en base β pues de lo contrario hay pérdida de cifras significativas.

$$\begin{aligned}
 x + y &= 0.x_1x_2\dots x_p \times \beta^{e_x} + 0.y_1y_2\dots y_p \times \beta^{e_y} = \\
 &= 0.x_1x_2\dots x_p \times \beta^{e_x} + \overbrace{0.000\dots 0}^{p \text{ ceros}} y_1y_2\dots y_p \times \beta^{e_x} \\
 &= 0.x_1x_2\dots x_p y_1y_2\dots y_p \times \beta^{e_x} \\
 &= 0.x_1x_2\dots x'_p \times \beta^{e_x} \quad \text{¡Desapareció } y!
 \end{aligned} \tag{1.16}$$

y el valor x'_p es posiblemente afectado por el redondeo. Aún sin redondeo, no hay garantía de que $x + y = x$. Ejemplos similares pueden inventarse para cada operación elemental. Esa es una muestra de los problemas de exactitud del sistema $\mathbb{F}$, cuyas particularidades quedan definidas al seleccionar el valor de los parámetros β , p , $e_{\min}$ y $e_{\max}$.

Esto implica que las computadoras tienen diferente arquitectura.

Si dos computadoras se fabrican con valores distintos de los parámetros, el mismo programa no dará los mismos resultados como consecuencia de las diferencias. Esto llegó a ocasionar grandes problemas de portabilidad de código y de dificultad de elaborar librerías matemáticas robustas en el inicio de la programación de computadoras como comenta Kahan en [197]. Aunque el problema persiste pero ya se dispone de convenciones que facilitan las cosas, pero aún hay mucho que hacer. El mayor problema sigue siendo la desinformación, la ignorancia en el tema de diseñadores de lenguajes, compiladores y procesadores.

1.3.3. Valores importantes y propiedades del redondeo

La distancia entre 1 y el siguiente NPF (escrito como $\bar{1}$ con la notación convencional) es llamada *épsilon de máquina* y se define y denota como

$$\varepsilon_M = \beta^{1-p}. \tag{1.17}$$

En general, el espacio entre el NPF x y el siguiente ($\bar{x}$) es al menos $\frac{\varepsilon_M}{\beta} |x|$ y a lo más $\varepsilon_M |x|$, salvo para los valores especiales.

La cota máxima no es difícil de ver ya que si x tiene la menor mantisa ocurre $x = .1 \times \beta^e$ y la distancia al siguiente número de punto flotante es

$$x\beta^{1-p} = .\overbrace{000\dots 0}^{p-1 \text{ ceros}} 1 \times \beta^{e+1-p}$$

que es precisamente $\text{ulp}(x)$.

Es sencillo calcular ε_M en un programa, como lo muestra el código 1.1. Al

¹⁸El procesador primero verifica que ningún operando sea un valor especial, en cuyo caso se da otro tratamiento.

Código 1.1: Calculando ε_M (épsilon de máquina).

```

1 double e = 1.0;
2 while ( 1+e != 1 )
3     e = e/2;
4 e = e*2;

```

final, luego de la ejecución de la línea 4, el valor de `e` será ε_M para el tipo de dato de la variable `e`. El mismo método puede usarse en prácticamente todos los lenguajes procedimentales¹⁹. En algunos ambientes de programación, esta es una constante ya definida. Por ejemplo, en Matlab es **eps**.

A partir de ε_M es posible determinar la base numérica empleada en la representación interna por la computadora pues

$$\beta = \frac{1 + \varepsilon_M}{1 - .5\varepsilon_M}. \quad (1.18)$$

Si al NPF más pequeño se le divide entre $1 - .5\varepsilon_M$ se obtiene una buena cota para determinar qué tan significativo es el underflow.

A la cantidad

$$\mu = \frac{1}{2}\varepsilon_M \quad (1.19)$$

se le llama *unidad de redondeo* y tiene gran importancia en todo análisis de errores. $\mu = 0.5 \times \text{ulp}(1.0)$

Redondeo y truncamiento

Cada número real $x \in \mathbb{R}$ tiene al menos un representante en $\mathbb{F}$ que se obtiene mediante la función $\text{fl} : \mathbb{R} \rightarrow \mathbb{F}$, como $\text{fl}(x) \in \mathbb{F}$. Se dice por extensión (y economía) que $\text{fl}(\mathbb{R}) = \mathbb{F}$.

Es fácil encontrar ejemplos que prueban que la función fl es inyectiva pero no sobreyectiva. De hecho, la inversa $\text{fl}^{-1} : \mathbb{F} \rightarrow \mathbb{R}$ no es función, pues una infinidad de valores reales tiene la misma representación por la precisión limitada de $\mathbb{F}$. A fl la clasifica Kulisch como un semimorfismo en [229, 230] por la estructura algebraica de $\mathbb{F}$.

El número real x y su representante en $\mathbb{F}$ asociado $\text{fl}(x)$ deben diferir a lo más en 1 ulp, por definición. La transformación de x a $\text{fl}(x)$ se llama *redondeo* y está definida como

$$\text{fl}(x) = x(1 + \delta) \quad (1.20)$$

donde $|\delta| < \mu$. Esta propiedad (y notación) ha sido empleada desde los primeros años del análisis de errores (véase [395], de 1964), generalmente atribuida a

¹⁹Más adelante se presentará otra forma de calcular ε_M .

Wilkinson y es de valor fundamental en análisis numérico²⁰.

Al despejar δ de (1.20) se obtiene el error relativo

$$\delta = \frac{\text{fl}(x) - x}{x}$$

y la cota para el error absoluto es

$$|x - \text{fl}(x)| \leq \frac{1}{2} \beta^{e_{\min} - p + 1} \quad (1.21)$$

siempre que se respete el redondeo al más cercano.

Otra manera de interpretar la función fl es con la propiedad

$$|\text{fl}(a \odot b) - (a \odot b)| \leq \varepsilon_M |a \odot b| \quad (1.22)$$

derivada directamente de la ecuación (1.20). El símbolo $\odot$ representa cualquiera de las operaciones aritméticas $(+, -, \times, \div)$. En la literatura, algunos autores distinguen la suma de reales con $x + y$ y la suma de flotantes como $\text{fl}(x + y) = x \oplus y$, pero en este libro se prefiere la primera.

El concepto de redondeo es central en programación numérica.

Cuando un número real puede ser representado con dos NPF con el mismo error, es decir, que queda a la misma distancia de los dos NPF más cercanos, el representante es seleccionando dependiendo de la regla de redondeo que se utilice, pudiendo ser siempre al mayor, siempre al menor, hacia aquel cuyo último dígito es 0, que es lo mismo que pedir que sea par, en el caso de $\beta = 2$, u otra regla.

Si no se selecciona el número más cercano y simplemente se descartan las cifras excedentes dejando sólo las p del formato, entonces el proceso se llama *truncamiento*, que posee un error relativo mayor (el doble). El error relativo para el truncamiento está acotado por ε_M y no por μ . Las primeras computadoras utilizaron este esquema. El error absoluto del truncamiento cumple con

$$|x - \text{fl}(x)| < \beta^{e_{\min} - p + 1} \quad (1.23)$$

y se puede comparar su magnitud con la del error relativo (1.21).

Los dos números en $\mathbb{F}$ más cercanos a $x \in \mathbb{R}$ son

$$\text{fl}_{\Delta}(x) = \min\{f \in \mathbb{F} \text{ tal que } f \geq x\} \quad (1.24)$$

$$\text{fl}_{\nabla}(x) = \max\{f \in \mathbb{F} \text{ tal que } f \leq x\} \quad (1.25)$$

por lo que $x \in [\text{fl}_{\nabla}(x), \text{fl}_{\Delta}(x)]$.

Además del redondeo al más cercano o redondeo por truncamiento, se puede tomar la decisión de direccionar voluntariamente el redondeo, es decir siempre emplear $\text{fl}_{\Delta}(x)$ o bien representar a x con $\text{fl}_{\nabla}(x)$.

²⁰Wilkinson ya la había empleado antes en [391, 392] y en [393] comenta que este enfoque fue utilizado primero por Givens en [136] de 1954. Givens hace referencia al trabajo de von Neumann y Godlstone [381] de 1951.

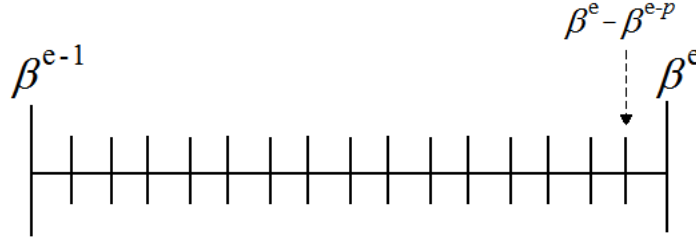


Figura 1.1: Distribución de IF. Los NPF entre 2 potencias consecutivas de β están uniformemente distribuidos, por lo que el error relativo va aumentando en ese intervalo. Siempre son β^p mantisas.

Como se ve, $[\text{fl}_\nabla(x), \text{fl}_\Delta(x)]$ es el intervalo más pequeño que contiene x . Se prefiere escribir

$$x \in [\underline{x}, \bar{x}]$$

que es más acostumbrada en aritmética de intervalos (sección B.4) e incluso $x \in [\lfloor x \rfloor_p, \lceil x \rceil_p]$ en algunos textos, y se agrega la precisión p como subíndice para no causar confusión con las funciones piso y techo. Si x es representado por alguno de estos dos valores, se dice que se ha realizado un *redondeo fiel*.

Las reglas de monotonicidad se cumplen, es decir, si $a \in \mathbb{IF}$, $b \in \mathbb{IF}$ y $x \in \mathbb{R}$ entonces

$$a \leq r \leq b \quad \Rightarrow \quad a \leq \text{fl}(r) \leq b$$

Es fácil ver que $\text{fl}(-x) = -\text{fl}(x)$, o económicamente, $\mathbb{IF} = -\mathbb{IF}$.

1.3.4. Distribución de los Números de Punto Flotante

A partir de la definición de $\mathbb{IF}$ (página 20) se ve que los mismos valores de mantisa m , desde $1/\beta$ hasta $1 - \beta^{-p}$, se repiten al pasar a otro exponente, representando otros números.

Para un exponente fijo e , la menor mantisa representa al número

$$\frac{1}{\beta} \times \beta^e = \beta^{e-1}$$

y la mayor mantisa al

$$(1 - \beta^{-p})\beta^e = \beta^e - \beta^{e-p}$$

que queda exactamente a la izquierda de β^e , la siguiente potencia, es decir, $\beta^e - \beta^{e-p}$ es el NPF exactamente menor que β^e . Esto se ilustra en la figura 1.1.

Esto significa que aunque las potencias de β se van separando, la cantidad de representantes entre cada par de potencias se mantiene constante: son β^p .

En el intervalo $[\beta^{e-1}, \beta^e]$, el error absoluto máximo (su distancia) entre dos NPF es el mismo (vale $(\beta^e - \beta^{e-1})/\beta^p$) entre cada par de potencias de β y se duplica al pasar a la siguiente potencia.

Este vale $(\beta^e - \beta^{e-1})/\beta^p$, es decir,

$$\begin{aligned} \frac{\beta^{e_{\min}+1} - \beta^{e_{\min}}}{\beta^p} & \text{mínimo y} \\ \frac{\beta^{e_{\max}} - \beta^{e_{\max}-1}}{\beta^p} & \text{máximo.} \end{aligned}$$

Ejemplo 10. Para la precisión simple de una computadora, con 32 bits para la representación, los valores mínimo y máximo del error absoluto son respectivamente 7×10^{-46} y 5×10^{30} , de manera aproximada.

Con el error relativo ocurre algo distinto. Para toda x , sin importar entre qué potencias de β^{e-1} y β^e se encuentre, se tiene que

$$\left| \frac{x - \text{fl}(x)}{x} \right| \approx \frac{1}{2} \beta^{1-p} \quad (1.26)$$

por lo que el error relativo es múltiplo de la base numérica empleada y no depende del valor de x .

Ejemplo 11. De nuevo para el caso de la precisión simple de una computadora convencional, este valor es siempre 1.192×10^{-7} , de manera aproximada.

Esto significa que en un sistema binario, el error relativo en la representación de números reales se duplica al pasar a de una potencia de la base a la siguiente. Por ello, se prefiere utilizar la base 2 para reducir este fenómeno, conocido como *wobbling* (tambaleo), que consiste en el comportamiento no suave del error relativo en la representación de x con $\text{fl}(x)$. El error puede ser 0 (para las potencias de 2) o puede ser tan grande como la unidad de redondeo (en el caso del mínimo NPF mayor que x).

En la figura 1.2, el wobbling (1.26) se ve en las marcas grandes que representan potencias de β , pues la distancia a la marca de la izquierda es la mitad de la distancia a la marca de la derecha. Los NPF que representan números reales con menor error relativo para cada potencia son lo que tienen mantisa de mayor valor.

De la misma figura 1.2 puede verse por qué la definición de *ulp* tiene problemas alrededor de las potencias de la base, pues su tamaño se duplica. (La figura es un tanto engañosa pues se ha dibujado pensando en $\beta = 2$, pero es imposible hacerlo para un valor arbitrario de la base.)

Hay estudios estadísticos sobre la distribución de logaritmica que el conjunto $\mathbb{F}$ posee y las desviaciones con respecto a la distribución de la suma de de estos números. Un ejemplo es el artículo de Scheidt y Schelin, [335]²¹.

²¹Con el tiempo, se restó importancia a los estudios estadísticos para la APF [201].

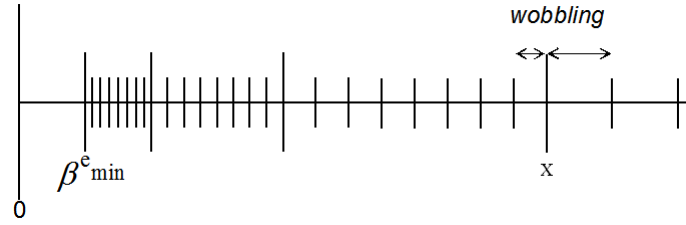


Figura 1.2: Wobbling o tambaleo. Consiste en la reducción abrupta del error relativo en la aproximación de x con $\text{fl}(x)$.

1.3.5. Números subnormales

Observando de nuevo la figura 1.2, es evidente un vacío entre el 0 y $\beta^{e_{\min}}$. Ese intervalo no tiene representantes en $\mathbb{F}$ tal y como se definió en 1.11 pues cualquier resultado con exponente menor a $e_{\min}$ sufre un underflow. Ese intervalo real $(0, \beta^{e_{\min}})$ no posee representantes en $\mathbb{F}$, ocasionando problemas de precisión y dificultando el análisis de errores.

Ejemplo 12. Como caso representativo de los problemas graves que pueden ocurrir, la comparación *if* ($x==y$) no es equivalente a *if* ($x-y == 0$) si se mantiene ese vacío alrededor del 0.

Cualquier número en este intervalo se representará con 0 o con $\beta^{e_{\min}}$, con un error tan grande como $\frac{1}{2}\beta^{e_{\min}}$. Eso significa que el número real $\frac{1}{2}\beta^{e_{\min}}$ se representará con un porcentaje de error de 100 %.

Desde antes de 1980 se tomaron medidas al respecto. La solución, que con los años resultó ser la mejor, fue la de agregar a $\mathbb{F}$ el conjunto de *números subnormales*²²

El coprocesador 8087 de Intel fue el primero que los usó.

$$\mathbb{U} = \{(-1)^s \times m \times \beta^{e-p}\} \quad (1.27)$$

donde ahora se relaja la restricción de la mantisa y $0 < m < \frac{1}{\beta}$, lo que significa que los elementos de $\mathbb{U}$ no están en notación científica normalizada. Esto permite que al intervalo $(0, \beta^{e_{\min}})$ se agreguen $\beta^p - 1$ valores igualmente distribuidos (sólo la combinaciones formada por puros 0s no se cuenta). La distancia entre estos números es la misma que la distancia entre los NPF del intervalo $[\beta^{e_{\min}}, \beta^{e_{\min}+1}]$.

A esto se le llama *underflow gradual*. Entre otras cosas, esto significa que no hay bit implícito en los números subnormales. Como la distancia entre ellos es fija (vale $\beta^{e_{\min}}/(\beta^p - 1)$), el error relativo aumenta conforme los valores se acercan a 0.

Desbordamiento gradual

Ahora el conjunto $\mathbb{F}$ está completo y luce como la figura 1.3. La región sombreada representa los números subnormales. Además de rellenar ese espacio vacío, los subnormales tienen la ventaja de no sufrir de wobbling. Ahora que ya está completo el conjunto de los NPF $\mathbb{F}$, la instrucción *if*($x==y$) ya es equivalente a *if*($x-y==0$).

²²Llamados originalmente desnormalizados.

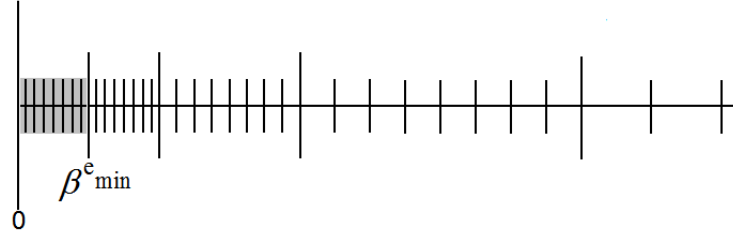


Figura 1.3: El conjunto $\mathbb{U}$ queda representado por la región sombreada que rodea al 0, aunque aquí sólo se muestra el eje positivo.

El menor número subnormal es referido por muchos autores como $\eta = \beta^{1-e_{\min}-p}$. Aparece menos frecuentemente que μ y ε_M , pero resulta útil cuando se analizan algoritmos con posibles underflows.

Para todo $x \in \mathbb{F}$, debe cumplirse que si $a = (x - \text{ulp}(x))$ y $b = (x + \text{ulp}(x))$, entonces se cumple que

$$\text{ó } a < x \leq b \quad \text{ó } a \leq x < b. \quad (1.28)$$

Esta regla aplica incluso para los valores especiales en la definición (1.11). Aunque esto aparente tener fines exclusivamente teóricos, las señales de Operación Inválida de todo programa pueden tener sus orígenes en la falta de cuidado al no tener en cuenta propiedades tan simples como (1.28).

Costo de los subnormales

Los números del conjunto $\mathbb{U}$, al llenar el abrupto vacío alrededor del cero, resuelven muchos problemas, pero esto es a costa de un sistema numérico más complejo de realizar. El principal problema de los números subnormales es que requieren circuitos adicionales en la unidad de aritmética del procesador. La opción de programarlos con software y no en con circuitos, resulta en un desempeño realmente lento, pero fue una de las primeras opciones para los fabricantes de computadoras.

Una observación inicial en contra es con respecto a los inversos. Como se mencionó en las características deseables del sistema los inversos deben pertenecer a $\mathbb{F}$, es decir, $x \in \mathbb{F} \Rightarrow 1/x \in \mathbb{F}$ de tal forma que hay siempre un elemento que cancela a otro. Pero, como al respetar $e_{\min} \approx e_{\max}$, al extender los números representados por debajo de $\beta^{e_{\min}}$ estos ya no tendrán inverso en $\mathbb{F}$. De esta forma, el recíproco de cualquier subnormal ocasionará overflow.

En los inicios, los detalles de las diversas soluciones eran secretos entre las compañías, por lo que no salió mucho en la literatura, como afirma Hauser en [162], sino hasta que los mejores caminos ya eran bien conocidos. Por otra parte, hubo muchas compañías fabricantes de procesadores o compiladores que

no consideraron a los subnormales como una opción, apoyándose mejor en las críticas en contra de la existencia de $\mathbb{U}$, como [121], donde Fraley comenta múltiples problemas, como excepciones y depuración más difíciles.

Trabajos fuertes en el otro sentido fueron [78], de Coonen, donde muestra la manera como los subnormales entran con facilidad en al conjunto de los NPF. También [90], donde Demmel demuestra que de muchos esquemas posibles para tratar el underflow, los subnormales son superiores, incluso con precisión extra para cálculos intermedios.

Con el tiempo, se publicaron esquemas eficientes que permitían que los números subnormales es codificaran en los procesadores. Ejemplo de esto es [340], de Schwartz, donde presenta algoritmos para las operaciones básicas con números subnormales.

Sin embargo, el mayor problema, con subnormales o sin ellos, era ocasionado por la falta de homogeneidad en los detalles del sistema de punto flotante disponible en cada computadora, tema que será abordado en el siguiente capítulo.



James Weldon Demmel (1953 -), matemático norteamericano bien conocido por sus trabajos en el desarrollo de las librerías LAPACK y otras muchas rutinas de alto rendimiento.

1.4. Propiedades de $\mathbb{R}$ y $\mathbb{F}$

El conjunto de los números reales cumple con propiedades que se definen sobre las operaciones básicas y se estudian desde los niveles escolares más básicos. En resumen es un campo ordenado, denso y no numerable²³, un grupo conmutativo con respecto a suma y multiplicación. Se enumeran a continuación las propiedades básicas de $\mathbb{R}$ con fines de comparación con $\mathbb{F}$. Por ello se considera al conjunto de los reales extendidos $\mathbb{R} = \mathbb{R} \cup \{-\infty, \infty\}$. Todo usuario de computadoras desearía que estas propiedades se cumplieran con los NPF.

Principalmente los programadores.

1.4.1. Propiedades de los reales extendidos

Sean x , y y z números reales cualquiera.

Propiedades de la suma

1. **Cerradura.** La suma $x + y$, también es real.
2. **Identidad.** Hay un único número, llamado cero (0) con la propiedad de que $x + 0 = 0 + x = x$.
3. **Inversos.** Para cada numero x , existe un único número denotado por $-x$, tal que $x + (-x) = (-x) + x = 0$.
4. **Asociatividad.** Se cumple $(x + y) + z = x + (y + z)$.
5. **Conmutatividad.** Se cumple $x + y = y + x$.

²³Consultar cualquier libro de álgebra o cálculo.

Propiedades de la multiplicación

6. **Cerradura.** El producto xy , también es real.

7. **Identidad.** Hay un único número, llamado uno (1) tal que para todo x se cumple $1x = x1 = x$.

8. **Inversos.** Para cada x , existe un único número, denotado con x^{-1} , tal que $xx^{-1} = x^{-1}x = 1$.

9. **Asociatividad.** Se cumple $(xy)z = x(yz)$.

10. **Conmutatividad.** Se cumple $xy = yx$.

Además, la suma y la multiplicación cumplen con otro axioma:

11. **Distributividad.** Se cumple $x(y + z) = xy + xz$.

Los axiomas anteriores definen un *campo*.

Propiedades de orden

Un campo es totalmente ordenado si se definen las relaciones $<$, $=$ y $>$ con las propiedades:

12. **Tricotomía.** Se cumple una y sólo una de las condiciones: $x < y$, $x > y$ o $x = y$.

13. **Reflexividad.** Si $x < y$, entonces $y > x$ y si $x = y$, entonces $y = x$.

14. **Transitividad.** Si $x < y$ y $y < z$ entonces $x < z$. Si $x = y$ y $y = z$ entonces $x = z$.

Relaciones de orden y las operaciones de grupo.

15. **Invarianza a traslaciones.** $x + z < y + z \iff x < y$. Además, $x + z = y + z \iff x = y$.

16. **Invarianza a escalas.** $xz < yz \iff x < y$. Además, $xz = yz \iff x = y$.

Propiedad de Densidad.

17. **Densidad.** Si $x < y$ entonces existe z tal que $x < z < y$.

Otra característica propiedad de los números reales es que son no numerables. son infinitamente muchos más que los números racionales. Su tamaño (cardinalidad) escapa del sentido común.

Con las sólidas propiedades anteriores se han desarrollado la gran mayoría de teorías matemáticas. Remover alguna propiedad puede tener repercusiones inesperadas.

1.4.2. Propiedades de $\mathbb{F}$

Las propiedades de $\mathbb{F}$ y sus operaciones es lo que se conoce como Aritmética de Punto Flotante.

Siendo $\mathbb{F}$ un subconjunto de los números reales, sería deseable que cumpliera con propiedades similares pero no es posible. A partir de la definición 1.11 se sabe que $\mathbb{F}$ es un conjunto finito. Con más exactitud que antes, $\mathbb{F}$ es un subconjunto de los números racionales, no incluye ningún número irracional.

Por los parámetros de $\mathbb{F}$ se puede deducir que NO se cumplen varias propiedades de los números reales: 1, 4, 6, 9, 11, 15 y 17 sin incluir los valores especiales, donde NaN tampoco cumple 12, 13, 14 y 16. Las propiedades 2, 3 y 8 se cumplen parcialmente. Es un buen ejercicio encontrar contraejemplos.

Por su significado, NaN no está ordenado, lo que implica que si x es NaN, las relaciones $x < y$, $x = y$ y $x > y$ son todas falsas. La única correcta es $x \neq x$ (x distinto de x).

Cambia por completo el panorama de la matemática que se puede desarrollar con $\mathbb{F}$ pues en general, sólo 7 propiedades se cumplen de alguna manera útil. La desigualdad del triángulo o la de Cauchy-Schwartz, con tantas aplicaciones, son otros ejemplos de propiedades no válida para los NPF. La misma regla de tres, de uso tan extendido ya no se puede aplicar en el caso general.

Como contraparte, disponer de ∞ hace que la división entre cero sea una operación válida en $\mathbb{F}$, aunque genera otras indefiniciones, como $\infty - \infty$, ∞/∞ o $\infty \times 0$. Este símbolo NO representa lo indefinido.

Para poder analizar y cuidar los errores en procesos numéricos, se necesita que para todas las operaciones elementales ($+$, $-$, $\times$, $\div$) se cumpla

$$\text{fl}(x \odot y) = (x \odot y)(1 + \delta), \quad |\delta| \leq \mu \quad (1.29)$$

como ya se dijo y $\odot$ es cualquiera de las operaciones elementales²⁴. Este es el modelo de aritmética de punto flotante (APF) más ampliamente utilizado, aunque no es el único posible.

Sean x y y dos números reales en el rango normal de $\mathbb{F}$. Su suma es

$$\begin{aligned} \text{fl}(x) + \text{fl}(y) &= x(1 + \delta_x) + y(1 + \delta_y) \\ &= x + y + x\delta_x + y\delta_y \\ &\leq x + y + x\delta + y\delta, \quad \delta = \max(|\delta_x|, |\delta_y|) \leq \mu \\ &= x + y + (x + y)\delta \\ &= (x + y)(1 + \delta) \\ &= \text{fl}(x + y) \end{aligned}$$

y de esta manera, el error relativo δ de la suma cumple con las características de los NPF. También podría utilizarse la descripción más exacta $x = 0.x_1x_2\dots x_p \times \beta^{e_x}$, pero como se vio, no es necesario.

²⁴En la práctica también se agrega $\sqrt{}$

Una propiedad útil que se puede aplicar en otras operaciones es que $\text{fl}(-x) = -\text{fl}(x)$, así como $\mu^2 \approx 0$, que se aplica en el caso del producto.

Si el resultado de la operación binaria cualquiera $x \odot y$ es exactamente un NPF, entonces se cumple (o debe cumplirse) que $\text{fl}_{\nabla}(x \odot y) = x \odot y = \text{fl}_{\Delta}(x \odot y)$.

Otra forma práctica de denotar el error relativo que no causa confusiones es escribir $\delta_{\odot}$ y similarmente para otras operaciones. De esta manera, en vez de escribir $\text{fl}(x + y) = (x + y)(1 + \delta)$ se escribe de manera compacta como δ_{x+y} .

Ejemplo 13. Sin tratarse ahora de una operación elemental, supongamos el mismo error en la aproximación de x y y :

$$\begin{aligned} \text{fl}(xy\sqrt{x}) &= xy\sqrt{x}(1 + \delta)(1 + \delta)(1 + \delta_{\sqrt{x}}) \\ &= xy\sqrt{x}(1 + \delta)^2(1 + \delta_{\sqrt{x}}) \\ &\approx xy\sqrt{x}(1 + 2\delta + \delta_{\sqrt{x}}) \end{aligned}$$

pues los términos cuadráticos se desprecian por $\mu^2 \approx 0$. Entonces, $\delta_{xy\sqrt{x}} = 2\delta + \delta_{\sqrt{x}}$.

Es evidente que esta compacta notación no causa confusiones. Knuth la utiliza en [219] y también otros más recientes como Hull en [181].

No es difícil encontrar ejemplos numéricos desagradables en $\mathbb{F}$, como ya se vio en la suma $x + y$ en la página 26. Si se calcula el promedio de dos números como $m = (x + y)/2$, pueden ocurrir varias cosas:

- $x + y$ resulta en overflow, aún cuando $m \in [x, y]$, por lo que no se puede realizar el cálculo.
- Si se intenta $x/2 + y/2$ y los x y y son ambas el número subnormal más pequeño, los cocientes resultan en underflow, por lo que $m = 0$ y no $m = x = y$, que es lo exacto.
- En una base distinta a 2, el resultado puede estar fuera del intervalo $[x, y]$.

Ejemplo 14. En base 10 con mantisa de 4 decimales, sean $x = .1 \times 10^1$ y $y = .9999 \times 10^0$. Su promedio es

$$\begin{aligned} m &= \frac{1}{2}(x + y) \\ &= \frac{1}{2}(.1 \times 10^1 + .9999 \times 10^0) \\ &= \frac{1}{2}(.1 \times 10^1 + .09999 \times 10^1) \\ &= \frac{1}{2}(.1 + .09999) \times 10^1 \\ &= \frac{1}{2} \times .1999 \times 10^1 \\ &= .9995 \times 10^0 \end{aligned}$$

que queda fuera de $[.9999, .1]$, si se redondea por truncamiento. Aún si se redondea al más cercano, el resultado queda

$$\begin{aligned} m &= \frac{1}{2}(.1 + .09999) \times 10^1 \\ &= \frac{1}{2} \times .19999 \times 10^1 \\ &= .99995 \times 10^0 \\ &= .1 \times 10^1 \end{aligned}$$

que es un extremo del intervalo y no el punto medio, pero claro, el conjunto de NPF determinado por $\beta = 10$ y $p = 4$ no se puede representar a .99995.

1.4.3. Dígito de guardia

El caso de la operación resta merece especial atención. Un ejemplo para explicar por qué y presentar el remedio.

Ejemplo 15. Sea un sistema numérico en base 2, con precisión $p = 4$. Se tiene $x = .1000 \times 2^1$ y $y = .1111 \times 2^0$. Para calcular su resta, primero se igualan los exponentes a 2^1 y se restan las mantisas:

$$\begin{array}{r} .1000 \\ - .\underline{0111} \\ \hline .00001 \end{array}$$

El resultado se normaliza sin necesidad de redondeo (en este caso), $x - y = .1 \times 2^{-3}$.

Para poder realizar la resta es necesario que se use la mantisa intermedia .01111 que requiere $p + 1$ dígitos. Este dígito extra temporal es conocido como *dígito de guardia* o *bit de guardia* y es obligatorio para que el error continúe siendo menor que μ .

Si no se utiliza el dígito de guardia, al igualar los exponentes se truncaría el último dígito de y , quedando la resta del ejemplo como

$$\begin{array}{r} .1000 \\ - .\underline{0111} \\ \hline .0001 \end{array}$$

y el error es el doble (apenas se cumple $|\delta| \leq 2\mu$). Si se redondea en vez de truncar, la resta es 0.

En 1977 se publicó [149], de Goodman y Feldstein, donde se analiza el efecto del bit de guardia y el proceso de normalización en, para aritmética normalizada. Los mismos autores ya habían estudiado el problema desde antes, publicando resultados de redondeo en [148].

En la actualidad, son pocas las computadoras que carecen de dígito de guardia (como ocurrió con algunos de los primeros modelos de las supercomputadoras Cray²⁵) e incurrir en resultados con errores relativos inesperadamente altos.

Cancelación catastrófica

Antes de definir el concepto, de nuevo un ejemplo ilustrará con eficiencia.

Ejemplo 16. De nuevo con la base $\beta \geq 2$, sea $x = .1$ y $y = x - \beta^{-p} = .0111\dots 1$ (p dígitos). La diferencia exacta $x - y$ es β^{-p} , pero si no hay dígito de guardia, el último bit de y se corta, por lo que la diferencia es β^{1-p} . El error relativo es

$$\begin{aligned} \frac{\beta^{1-p} - \beta^{-p}}{\beta^{-p}} &= \frac{\beta^{-p}(\beta - 1)}{\beta^{-p}} \\ &= \beta - 1 \end{aligned}$$

lo que es un error relativo enorme: en base 2 el error es tan grande como el resultado y si $\beta = 10$, ¡sería 9 veces mayor!



La necesidad del dígito de guardia es evidente, para evitar lo que se conoce como *cancelación catastrófica*. El nombre suena alarmante, pero es más por el hecho de que es la operación aritmética que mayor error relativo puede alcanzar si se realiza mal, cosa que es de esperarse que ningún fabricante de computadoras realice actualmente.

Siegfried M. Rump (1955 -), científico alemán, con importantes desarrollos en la APF, tanto en la teoría como en algoritmos. Muy conocido por sus trabajos en aritmética de intervalos.

Ejemplo 17. Un ejemplo notable de cancelación catastrófica lo presentó Rump en [325]. Se trata de evaluar la expresión

$$z = 333.75b^6 + a^2(11a^2b^2 - b^6 - 121b^4 - 2) \quad (1.30)$$

para los valores $a = 77617$ y $b = 33096$. Este ejemplo patológico fue reescrito en 2002 para la norma 754 en [249]. Seis años más tarde se publicó [85], donde es analizado con detalle. Del análisis (y experimentación) quedan claras varias cosas:

1. Aún cuando los resultados de usar distinta precisión ($p = 24, 53, 113$) son congruentes, pueden estar mal.
2. Este fenómeno no solo ilustra cancelación catastrófica, sino resultados distintos en distintos sistemas de cómputo.
3. Un programador promedio no se dará cuenta del error si no compara con otro ambiente numérico.

²⁵La Cray 1 en particular fue una queja constante de los analistas numéricos, pero incluso las X-MP y C90 no se escaparon de críticas, no solo por dígitos de guardia, sino por su modo de redondeo. Esto queda de manifiesto en el artículo [21] de Bailey. El interés comercial por la velocidad a costa de todo tuvo sus perjuicios.

La razón se hace obvia reescribiendo (1.30) como

$$\begin{aligned}y &= 333.75 b^6 + a^2(11a^2b^2 - b^6 - 121b^4 - 2) \\x &= 5.5b^8 \\z &= x + y + \frac{a}{2b}\end{aligned}$$

y el problema resulta de que

$$\begin{aligned}y &= -7917111340668961361101134701524942850 \\x &= +7917111340668961361101134701524942848\end{aligned}$$

que son cantidades que tienen 35 dígitos en común, con diferencia de los 2 menos significativos. Cualquier sistema numérico que no permita almacenar esa mantisa tendrá problemas pues $x + y$ será considerado como cero, y el resultado sólo como el cociente $a/(2b) \approx 1.1726 \dots$

Tal vez se pueda esperar la cancelación catastrófica en los procesadores gráficos (GPU), donde la precisión queda en segundo término pues se requiere que todas las transformaciones geométricas (típicamente en 3D) ocurran a la más alta velocidad posible. La APF de las tarjetas gráficas es menos exacta pero mucho más veloz, usando valores bajos de p .

Una herramienta reciente la propone Michell Lam en [233], que detecta de manera automática cancelaciones catastróficas. Lo que hace es verificar los exponentes binarios de los operandos y del resultado. Si el exponente del resultado del resultado es menor que el máximo exponente entre los operandos, entonces ha ocurrido una cancelación. El nivel de catástrofe se mide con

Esto debiera estar en las herramientas de trabajo de los depuradores modernos.

$$priority = \max(e_1, e_2) - e_r$$

donde e_1 y e_2 son los exponentes de los operandos y e_r el del resultado.

El análisis ignorará cualquier cancelación debajo de cierto umbral. Lam y su equipo, usan 10 bits como umbral, algo así como 3 dígitos decimales. Su herramienta detecta y reporta (en un archivo log) todos los eventos que cumplen con la restricción.

Este interesante prototipo lo desarrollaron utilizando la interfaz de programación Dyninst API, que es parte del proyecto Paradyn, de la Universidad de Maryland.

1.4.4. Resultados teóricos útiles

Hay varios resultados importantes relacionados con el dígito de guardia, para cualquier β que se utilice. Garantizan que bajo ciertas condiciones, la suma y resta se calculan con exactitud. Resultados adicionales y métodos para extender la exactitud con restricciones más flexibles se verán en el capítulo 3.

Proposición 1 (Ferguson, [112]). *Si $x = m_x \times \beta^{e_x}$ y $y = m_y \times \beta^{e_y}$ son dos NPF para los que el exponente de $x \pm y \leq \min(e_x, e_y)$, y la suma o diferencia se calcula con $p + 1$ dígitos de precisión intermedia, entonces la suma o diferencia $x \pm y$ da un resultado exacto, siempre que no ocurra underflow.*

Ferguson obtiene este resultado por el análisis de la reducción de rango en la evaluación de funciones, materia del capítulo 7. El siguiente, llamado por Kahan Teorema de la Cancelación exacta en [201], es un corolario del resultado anterior, debido a Sterbenz.

Proposición 2 (Sterbenz, [352]). *Si $y/2 \leq x \leq 2y$ y hay al menos $p + 1$ dígitos de precisión intermedia, entonces $x - y$ da un resultado exacto.*

Siempre hay un bit extra en los procesadores modernos, así que esa restricción es con fines de un enunciado completo. El resultado anterior lo generalizó Nievergelt en [291]. Es interesante que se garantiza la diferencia exacta, cuando hay tanto libro de métodos numéricos que la satanizan en exceso. Es cierto, bajo ciertas condiciones, la resta causa la cancelación catastrófica, pero en muchos casos, está garantizado un resultado exacto en $\mathbb{F}$. El resultado anterior depende sólo de la representación de los números y no es una característica del procesado.

Ejemplo 18. *Si $y = 1000$, entonces $x - y$ será exacto para todo número de punto flotante $x \in [500, 2000]$, por lo que sólo afuera de ese rango se esperan problemas de redondeo.*

Esta es una propiedad de la representación flotante y no de la aritmética de los procesadores en sí. Por ello ha habido intentos de extender este resultado a otras aritméticas y verificadores formales, como en [160] o [38], donde prueban que el resultado es independiente de la base.

Extendiendo el resultado de Sterbenz, en [242] se demuestra la siguiente proposición.

Proposición 3. *Sean a y b números de $\mathbb{F}$ con $k + l$ bits en la mantisa y $l \geq 0$. Si se cumple que*

$$\frac{b}{2} \leq \frac{b}{1 + 2^{-l}} \leq a \leq b(1 + 2^{-l}) \leq 2b \quad (1.31)$$

entonces $a - b \in \mathbb{F}$, es decir, $a - b$ tiene representación exacta con k bits en la mantisa.

Al igual que el resultado de Sterbenz, la anterior proposición es válida para bases mayores a 2. Para completar el resultado de Sterbenz y garantizar que incluso los números subnormales pueden dar resultados exactos en ciertas condiciones, se presenta el siguiente resultado.

Proposición 4 (Hauser, [162]). *Si $\text{fl}(x + y) \in \mathbb{U}$ entonces $x + y = \text{fl}(x + y)$ con exactitud.*

Otra manera de verlo es que, si hay underflow gradual y la suma de dos números es menor que $\beta^{e_{\min}}$, entonces el menor número flotante normalizado, entonces su suma es exacta. La razón deriva de que todo número es múltiplo del número subnormal más pequeño, $\eta = \beta^{1-e_{\min}-p}$.

El bit de guardia no fue suficiente en la práctica para redondear con eficiencia, por lo que tuvo que utilizarse una extensión de esa idea. En 1973, Yohe mostró en [401] que dos bits de guardia son suficientes para resolver la mayoría de los casos, pero esto se presentará con la norma de punto flotante, en los modos de redondeo, en la sección 2.2.4.

Así como los resultados anteriores garantizan exactitud para suma y resta en ciertos casos, hay otros que garantizan exactitud de producto. Uno de los primeros fueron los trabajos de Goodman y Feldstein sobre productos en [148] y posteriormente Goodman para el caso de punto fijo en [147]. En ambos casos se analizan tanto redondeo como truncamiento, pero desde un punto de vista más estadístico, lo que en la actualidad ha perdido interés, pero no dejan de ser buenos ejemplos de análisis. El lector interesado puede consultar [282], pág. 132, para análisis más actualizados.

Existen muchos otros resultados teóricos interesantes, aunque se refieren a sistemas no normalizados, a representación en punto fijo, como los de Bareiss en [23] o generalizaciones a bases numéricas distintas a 2 y 10, como en [36], por lo que no se presentan aquí. Para más referencias consultar la sección B.1, página 373.

Capítulo 2

Norma 754 de IEEE

Almost all programming languages persist in harmful practices inherited from 1950's superstitions about floating-point arithmetic.

William Kahan, octubre de 2010.

Se presenta a continuación una revisión rápida de la norma de punto flotante vigente a partir de agosto 2008, comenzando con las condiciones que propiciaron su desarrollo. No tiene sentido transcribir las más de 50 páginas del documento oficial, pero se extraen y analizan las características que más pueden interesa a los programadores.

2.1. Proceso histórico

El lector no interesado en el desarrollo histórico de la norma, puede pasarse directamente a la sección 2.2, aunque el autor recomienda ampliamente.

Debido a la diferencia de valores en los parámetros del sistema numérico (1.11), así como los criterios de redondeo, operaciones con números especiales y uso de excepciones, academias e industrias de la computación vieron la necesidad de definir una manera de trabajar efectiva y consistente. Al inicio de los 80s, había más de 20 formatos distintos para NPF en las computadoras existentes.

Uno de los primeras referencias donde se comparan diversas APF es [383], aunque el trabajo sobre APF no normalizadas de Ashenurst y Metropolis [14] es todavía anterior y abogan por el uso de formatos no normalizados, presentando las bondades en cuanto a la propagación de errores y la eficiencia temporal¹.

Los códigos portables tenían que ser muy extensos para considerar las múltiples y diferentes arquitecturas.

¹La rapidez con que se realizan los cálculos.

En [155], Gray analiza los formatos normalizados y no normalizados, agregando lo que fue llamado índice de significancia, que era una cuenta de cifras significativas, esquema que pronto dejó de ser utilizado.

Posteriormente, en 1964, Ashenhurst presentó en [13] los criterios de ajuste para la evaluación de funciones de una variable con parámetros no normalizados. La decisión de la base β para la representación interna fue crítica y muy discutida en esa época, en la que los números 2 y 10 eran los prospectos más fuertes.



William Velvel Kahan (1933 -), considerado por muchos como el padre de la APF. Matemático canadiense, ganador de la medalla Turing por sus trabajos en el desarrollo de la norma de PF. Personaje de la mayor importancia en cómputo numérico.

La entrevista a William Kahan² revela mucho de la historia detrás de esta norma, como la indiferencia de los fabricantes de grandes computadoras hipnotizados por la velocidad o la interesante lucha de la compañía Dec (véanse [302], que presenta la otra propuesta y en particular [301] para la APF de VAX) en contra del underflow gradual incluido en la primera propuesta K-C-S (Kahan-Coonen-Stone) de 1979. Las otras propuestas de norma fueron la de Payne-Strecker y Fraley-Walther, ambas de 1980, así como la de Brown en [52]. Es altamente recomendable leer todo lo que ha publicado Kahan en su página web.

La necesidad imperiosa de normar la forma de hacer cálculos numéricos con computadoras fue plasmada por Kahan con gran claridad y muchos detalles en [197] “Why do we need a floating-point arithmetic standard?”³ en 1981. Los primeros pasos ya los había dado durante su trabajo con la APF de IBM y las calculadoras HP.

Por esos años, algunos alegaban que la propuesta era más bien un diseño de software y no una norma, por lo que IEEE no tendría por qué dedicarle tanto esfuerzo. El proceso de desarrollo de la norma, comienza en Intel en 1976, en un intento por lograr que cada procesador que dijera Intel por fuera, no calculara disparates por dentro. La propuesta fue formalizada en [207] y sus implicaciones a los lenguajes de alto nivel fueron analizadas por Richard Fateman en [108], principalmente los aspectos referentes a Fortran, donde propone soluciones parciales a algunos problemas.

Dos documentos adicionales que ilustran y hacen ver los beneficios y problemas remanentes son el de Kahan [202], de observaciones a la primera norma de 1985 y el artículo multicitado de Goldberg “Lo que todo científico de la computación debe saber sobre APF” [140], que presenta realmente lo que todo programador debe saber al respecto, basado en un curso que Kahan dio en Sun Microsystems en 1988. Posteriormente, en 2000, Sun agregó un anexo (de Doug Priest) que aclaraba algunas cosas más, disponible en su “Numerical Computation Guide” [355]. En 2006 se publicó algo que podría llamarse “lo que todo estudiante debe saber sobre APF” de Chuck Allison ([7]), y hay otras versiones similares para programadores en Java (2003) y para desarrolladores de agentes móviles (2007).

Ante el interés de una norma para otras bases diferentes a 2, debido principalmente a la existencia de computadoras en base 8, 10 y 16, surgió la norma

²www.cs.berkeley.edu/~wkahan/ieee754status/754story.html enero de 2008.

³¿Por qué necesitamos una norma para aritmética de punto flotante?

Lectura recomendada para todo estudiante de esta área y afines.

854, cuyo primer borrador inicial se presenta en 1985 en [74]. Se aprobó en 1987. En 1991, se publica la “Language Compatible Arithmetic Standard” (LCAS, ISO/IEC 10967:1991), propuesta por Mary Payne y Brian Wichmann, que fue duramente criticada por Kahan en [200] donde comenta que “está tan llena de errores que el mundo de la computación debe rehazarla”, con argumentos de todo tipo; su lectura vale la pena. Al año siguiente, el grupo de trabajo de software numérico de IFIP⁴ publicó otra crítica en [82] en la misma dirección que la opinión de Kahan.

Fue precisamente la IFIP quien en 1978 publicó [118] donde se propone una estandarización de nombres y definiciones necesarias para la fácil portabilidad de códigos y algoritmos. El artículo se escribió a partir del cuarto borrador que había sido ampliamente difundido entre programadores y analistas numéricos. Muchas de las sugerencias fueron adoptadas por la norma de IEEE, hayan o no sido desarrolladas independientemente.

Tras aprobarse y publicarse en 1985 la primera norma, transcurrieron años en los que la industria comenzó a adoptarla al menos parcialmente. La obligación moral no era suficiente para forzar a la industria. En las revisiones de algunos lenguajes de programación, se comenzaron a incluir indicaciones de la norma y poco a poco han sido respetadas por los fabricantes de compiladores. La norma no la cumplía ninguno en su totalidad. Mucha experiencia se ganó y surgieron trabajos como [31] de Blackford, en 1997, en el que se muestran los problemas del cómputo numérico heterogéneo. Blackford y sus colegas describen la experiencia en el desarrollo de las bibliotecas de funciones ScaLAPACK y la PVM de NAG.

Las normas de IEEE tienen una vigencia de 15 años, por lo que en 1999 se inició la revisión de la norma, circulando durante los trabajos el borrador conocido como IEEE-754R, cuya última versión pública fue aprobada en octubre 9 de 2006. ¡Se tardaron 8 años en la revisión!

Los objetivos de la revisión fueron muy claros:

- Mezclar las normas 754 (punto flotante base 2) con la 854 (punto flotante base ≥ 4), emitida en 1986.
- Reducir las opciones de implementación.
- Incluir y unificar formatos de datos externos.
- Resolver las ambigüedades de las normas anteriores.
- Estandarizar la operación fma (fused multiply-add), $x + y \times z$ con un solo redondeo final. fma ya se usaba desde antes, como se muestra en la página 64
- Incluir la cuádruple precisión (128 bits).
- Estandarizar algunas funciones elementales ($\sqrt{}$ ya se había incluido en 1985).

⁴International Federation for Information Processing, creada en 1960 por UNESCO. El grupo de análisis numérico de IFIP (WG 2.5) se reunió por primera vez en 1975, en Oxford.

Lo anterior, evitando invalidar la operación de ninguna computadora que respetara razonablemente la norma de 1985 y que siguiera en uso. Todos los detalles pueden consultarse directamente en el documento [77], también conocido como IEC 60559:1989 para la versión de 1985, cuyo borrador final está disponible en internet⁵.

Luego de año y medio de trabajo en el que ocurrieron 8 procesos de votación, finalmente se aprobó el 12 de junio de 2008 y se publicó el 29 de agosto del mismo año. Los cambios con respecto al borrador de octubre de 2006 son sustanciales en detalle pero en general el documento contiene los mismos apartados.

La norma indica la forma como debe implementarse un sistema de punto flotante, por software, hardware o alguna combinación de ambos. De hecho, se especifica que el soporte para punto flotante deben darlo en conjunto el lenguaje, el compilador y el procesador, sin indicar cuál cada característica.

Es importante enfatizar que la norma no garantiza respuestas correctas, como ya se vio que es imposible, pero sí respuestas consistentes de un sistema a otro. Un error común es pensar que la norma es para que la cumpla el hardware y que los programadores no deben preocuparse. Usar bien la norma en un programa puede significar los detalles con que se construye el mejor software.

Como el proceso de revisión de la norma ha sido completamente público, diseñadores de lenguajes y fabricantes de procesadores y compiladores han ido incluyendo en sus productos algunos de los lineamientos definidos, por lo que en este momento disponemos de sistemas de cómputo que cumplen parcialmente alguna de las características. Ninguno la cumple del todo, aunque hay excelentes esfuerzos, como el de la versión C99 del lenguaje C o Fortran 2003, aprobados por ANSI.

Sin embargo, ni c99 ni Fortran 2003 incluyen tipos decimales oficialmente aún. El grupo de trabajo WG14 de ANSI está elaborando la propuesta TR 24732 sobre las extensiones para variables decimales.

Basic y PL/I usan base 10, en C# existe un tipo nativo que utiliza 128 bits para variables flotantes base 10 y Java dispone de la clase `java.math.BigDecimal` pero sin soporte de hardware, por lo que es unas 100 veces más lenta, pero útil para aplicaciones financieras.

El grupo de trabajo WG14 del ANSI c99⁶ se encuentra trabajando desde 2006 en una extensión para el c99 que permita utilizar variables para base 10. Los documentos base son la revisión de la norma 754 y el ANSI C99. Se espera que en los próximos años, se pueda verificar que un compilador considera los tipos decimales de la norma si define la macro `_STDC_DEC_FP_`.

ISO define una serie de estándares adicionales para aritmética de computadora en el documento ISO/IEC-10967 (IEC 60559), de 2005 que en general es compatible con el de IEEE. Vale la pena su revisión por quien quiera adentrarse aún más a los detalles finos de la especificación de requisitos numéricos. A

⁵www.validlab.com/754R/ o grouper.ieee.org/groups/754/

⁶JTC1/SC22/WG14 con exactitud.

diferencia de las normas de IEEE, no contiene detalles como la representación numérica, bandera de inexacto, algunos modos de redondeo o NaNs, dejando más libertades (¿o riesgos?) que IEEE.

2.2. Contenido de la norma (y observaciones)

La norma tiene todos los elementos de una típica norma de IEEE, distinguiendo aquello que no pertenece a la norma sino que aparece para aclarar o informar.

En algunos casos, la información incluida es sólo sugerida, como aquello en lo que el equipo de trabajo no se puso de acuerdo.

2.2.1. Propósitos

La norma tiene como propósito general poner orden en el desempeño del cómputo de punto flotante (binario y decimal), para obtener resultados que sean independientes de si se procesan en hardware, software o una combinación.

El usuario debe tener control sobre todos los parámetros para:

1. Facilitar la portabilidad de programas.
2. Permitir que usuarios no expertos desarrollen programas seguros y sofisticados.
3. Permitir que usuarios expertos desarrollen programas robustos, eficientes y portables.
4. Dar facilidades para el diagnóstico de problemas en tiempo de ejecución, manejo de excepciones sencillo y aritmética de intervalos adecuada.
5. Propiciar la programación de funciones elementales, APF de precisión múltiple y fácil cómputo simbólico.

La norma incluye anexos que determinan posibles caminos a futuro, así como recomendaciones generales y sugerencias para fabricantes y desarrolladores.

La norma *especifica*:

- Formatos para datos en punto flotante binario y decimal, para cómputo y almacenamiento.
- Conversiones entre enteros y formatos de punto flotante.
- Conversiones entre los diversos formatos.
- Excepciones y su manejo, incluyendo datos que no son números.
- Lo que es obligatorio, lo que se espera, lo que se permite y lo que se recomienda.

Aunque se especifican las características mínimas de las conversiones, la norma no dice cómo deben hacerse, por lo que la decisión queda a cada fabricante de compiladores.

Base	2			10	
Tamaño	32	64	128	64	128
p	24	53	113	16	34
$e_{\text{máx}}$	127	1023	16383	384	6144
$e_{\text{mín}}$	-126	-1022	-16382	-383	-6143

Tabla 2.1: Valores de los parámetros de punto flotante para números binarios. El tamaño y la precisión se indican en cantidad de bits, mientras que los exponentes por su rango.

La norma NO *especifica*:

- Formatos internos o externos para números enteros o secuencias de caracteres.
- Cómo convertir entre bases.
- Interpretación del signo o mantisa de NaN.
- Cuántos bits extras se deben usar en los cálculos intermedios.
- Qué parte del ambiente de programación debe proporcionar cada característica.

2.2.2. Formatos definidos

Los números de punto flotante se dividen en dos grupos, para cómputo o para almacenamiento. En ambos casos se dispone de representación en base 2 y 10. Para el formato interno, hay precisión simple, doble y cuádruple, mientras que el formato para almacenamiento sólo dispone de una precisión:

	Base 2	Base 10
Cómputo	32, 64 y 128	64 y 128
Almacenamiento	16	34

En la propuesta de extensión para el ANSI c99, se incluyen tipos decimales de 32, 64 y 128 bits, agregando una precisión no indicada en la norma.

Todos los formatos se especifican con 4 parámetros básicos: base, precisión, exponente máximo y exponente mínimo. Los valores correspondientes a los formatos de cómputo se presentan en la tabla 2.1. Para los valores de almacenamiento, referirse directamente a la norma (ver [77]).

Con estos parámetros, se representan números de la forma

$$x = (-1)^s \times m \times \beta^e \quad (2.1)$$

donde los campos corresponden a los indicados en la expresión (1.11), página 20. La mantisa m es un número representado con una cadena de dígitos de la forma $0.d_1d_2d_3\dots d_p$, con $0 \leq d_i < \beta$, por lo que $0 \leq m < 1$.

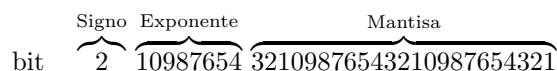


Figura 2.1: Formato binario en precisión simple. Nótese cómo los bits se cuentan de derecha a izquierda.

e	m	Valor de x
255	$\neq 0$	NaN
255	$= 0$	$(-1)^s \times \infty$
$0 < e < 255$	cualquiera	$(-1)^s \times 2^{e-127} \times (1.m)$ (normales)
0	$\neq 0$	$(-1)^s \times 2^{e-126} \times (0.m)$ (subnormales)
0	$= 0$	$(-1)^s \times 0$

Tabla 2.2: Interpretación del formato binario en precisión simple. Ver la ecuación (2.1).

Cualquier formato debe poder representar las cantidades $-\infty$, ∞ , una señal NaN y un NaN silencioso (o qNaN). Este último señala una operación inválida que no requiere de tratamiento forzoso y puede dejarse pasar.

Formatos base 2

En el caso de los números de *precisión simple*, el formato es de 32 bits, reservando el primero para el signo (s), 8 para exponente (e) y 23 para mantisa (m). En cada campo, los bits más significativos están a la izquierda. Para la computadora, el formato es el indicado en la figura 2.1, donde los números indican la significancia de los bits. El valor representado x en la fórmula (2.1) se calcula según las definiciones de la tabla 2.2.

En el caso de los números normales, se usa el bit implícito y los subnormales permiten el underflow gradual. Los ejemplos siguientes ilustran el uso del formato binario de precisión simple.

Ejemplo 19. *El número 1.0 queda representado por*

0 01111111 000000000000000000000000

En este caso, los parámetros toman los valores $m = 0$ y $e = 127 - 127 = 0$, por lo que $(-1)^0 \times 2^0 \times 1.0 = 1.0$.

Ejemplo 20. *El número 2.0, es*

0 10000000 000000000000000000000000

Ahora, los valores son $m = 0$ y $e = 128 - 127 = 1$, por lo que $(-1)^0 \times 2^1 \times 1.0 = 2.0$.

e	m	Valor de x
2047	$\neq 0$	NaN
2047	$= 0$	$(-1)^s \times \infty$
$0 < e < 2047$	cualquiera	$(-1)^s \times 2^{e-1023} \times (1.m)$
0	$\neq 0$	$(-1)^s \times 2^{e-1024} \times (0.m)$
0	$= 0$	$(-1)^s \times 0$

Tabla 2.3: Interpretación del formato binario en precisión doble.

e	m	Valor de x
16383	$\neq 0$	NaN
16383	$= 0$	$(-1)^s \times \infty$
$0 < e < 16383$	cualquiera	$(-1)^s \times 2^{e-8191} \times (1.m)$
0	$\neq 0$	$(-1)^s \times 2^{e-8190} \times (0.m)$
0	$= 0$	$(-1)^s \times 0$

Tabla 2.4: Interpretación del formato binario en precisión cuádruple.

Ejemplo 21. ¿Qué número representa 0 10000000 10010010000111111011010?
En este caso,

$$m = .570\,796\,370\,506\,286\,621\,093\,75$$

$$e = 1$$

por lo que

$$(-1)^0 \times 2^1 \times 1.570\,796\,370\,506\,286\,621\,093\,75 = 3.141\,592\,741\,012\,573\,242\,187\,5$$

que es una aproximación al valor exacto π . Obsérvese que esta no es la mejor aproximación posible (falla en el séptimo decimal), pero es el formato de precisión simple.

Otras cantidades fáciles de representar son

0 1111110 11111111111111111111	Máximo número normal
0 0000001 00000000000000000000	Mínimo número normal
0 0000000 11111111111111111111	Máximo subnormal
0 0000000 00000000000000000001	Mínimo subnormal.

Todos los formatos en base dos tienen la misma forma de representación, excepto que los valores de los parámetros (tabla 2.1) son distintos. Para precisión **doble** y **cuádruple** son las tablas 2.3 y 2.4 respectivamente.

Se pueden calcular los valores mínimos y máximos para los formatos, tanto normales como subnormales y se presentan de manera aproximada en la tabla 2.5. No todos los compiladores o procesadores disponen del tipo de dato para precisión cuádruple. De hecho es más común que se disponga del tipo de dato de precisión extendida (80 bits). Pero sí existen varios disponibles en forma gratuita en internet con los que se puede trabajar (Ch, Dev-C++ y otros).

Precisión	Simple	Doble	Cuádruple
Mayor normal	3.402×10^{38}	1.797×10^{308}	1.189×10^{4932}
Menor normal	1.175×10^{-38}	2.225×10^{-308}	3.362×10^{-4932}
Menor subnormal	1.401×10^{-45}	4.94×10^{-324}	6.475×10^{-4966}
ε_M	1.192×10^{-7}	2.22×10^{-16}	1.926×10^{-34}
μ	5.96×10^{-8}	1.11×10^{-16}	9.629×10^{-35}

Tabla 2.5: Algunos valores límites aproximados de $\mathbb{F}$.

Formatos base 10

Pese a que la base 2 tiene ventajas sobre las otras, en muchas aplicaciones donde el ser humano está en contacto constante con los números (como en los sistemas financieros) no resulta tan conveniente. Ya en 1986 la norma 854 describía las formas de operación para bases distintas a la 2, incluyendo el 10. Ante la ventaja de la base 10 para cantidades donde la interacción humana es constante⁷, durante la revisión de la norma se decidió incluir tanto la base 2 como la base 10 como las únicas aceptables.

Como los números con que el usuario trabaja tienen representación exacta en base 10, es de esperar se que al hacer operaciones sin salirse de esta base se evite el problema de convertir con inexactitud a base dos, hacer los cálculos y reconvertir con otra inexactitud a base 10. Uno de los mayores promotores y desarrolladores de esta base es Mike Cowlishaw, de IBM, quien ha colaborado intensamente en el desarrollo del formato y la aritmética decimal, en particular la Especificación Aritmética Decimal de IBM (ver [84] de 2003, aunque la versión mas reciente es de 2007). Para mayor ilustración del trabajo decimal, el lector interesado revisar las librerías decNumber para ANSI C y la BigDecimal class de Java.

Como es de esperarse por el fenómeno (1.26) en la página 30, el wobbling tiene mayor impacto en base 10 que en base 2, pero este inconveniente es compensado por la exactitud de la representación de las cantidades de interés, como las monetarias. No es recomendable en cálculos meramente científicos, por lo que no se presenta con todo el detalle de la representación en base 2, sino sólo las generalidades.

Desde hace muchas décadas, se ha utilizado la representación binaria para números decimales y se desarrollaron gran cantidad de variantes, que cumplieran con alguna necesidad o restricción. La más simple es donde cada dígito decimal es representado por su conversión natural en binario:

Utilizada por muchos compiladores.

De esta manera, el número 16743 puede representarse como

Representación: $\overbrace{0001}^1 \overbrace{0110}^6 \overbrace{0111}^7 \overbrace{0100}^4 \overbrace{0101}^3$.

⁷Las calculadoras de mano son un buen ejemplo.

0	1	2	3	4	5	6	7	8	9
0000	0001	0010	0011	0100	0101	0110	0111	1000	1001

Tabla 2.6: Conversión directa de base 10 a binario con bcd.

Esta codificación se conoce como *decimal codificado en binario* (bcd, por sus siglas en inglés). Como no es la única forma de conversión, esta es referida como bcd8421. Algunas ventajas son:

- 1. Convertir la cadena numérica de entrada al formato interno es directo.
- 2. El redondeo es trivial.
- 3. Escalar por potencias de 10 es muy sencillo.
- 4. Alinear cantidades con el punto decimal es fácil.

Algunos compiladores disponen de este tipo de datos para variables tanto enteras como flotantes. Si se utiliza este método con notación científica normalizada, entonces hay que guardar el signo, la mantisa y el exponente de la base 10. Como el exponente es un entero, se usa la notación complemento a 2 convencional, que es el método más ampliamente utilizado para representar cantidades enteras en las computadoras modernas, por sus cómodas propiedades aritméticas.

Notación complemento a 2 y bcd Con n bits, existen 2^n configuraciones. Los positivos y el cero se identifican porque el bit más significativo es 0. Los negativos se calculan restando a 2^n el valor absoluto del número.

Ejemplo 22. Si se utilizan 8 bits, el 54 es 00110110_2 . El -54 se calcula invirtiendo los bits y se suma 1:

$$\begin{array}{lcl} 54 = 00110110_2 & & \\ 11001001_2 & \text{se invierte cada bit y} & \\ -54 = 11001010_2 & \text{se suma 1.} & \end{array}$$

Para la conversión contraria, se toma el $-54 = 11001010_2$, se le resta 1 y se invierten los bits.

El problema de la notación en bcd es que las representaciones binarias del 9 al 15 no se utilizan, es decir, las que faltan en la tabla 2.6.

Dígito	9	10	11	12	13	14	15
Binario	1001	1010	1011	1100	1101	1110	1111

Si se utiliza bcd para codificar los 1000 números del 0 al 999, se usarían en total 12 bits para cada número (4 por dígito). La observación de que con sólo 10 bits hay $2^{10} = 1024$ representaciones deja ver que es posible ahorrarse 2 bits cada 3 dígitos decimales.

Y sobran 24 representaciones.

aei	pqr	stu	v	wxy	vwkst	abcd	efgh	ijklm
000	bcd	fgh	0	jkm	0...	0pqr	0stu	0wxy
001	bcd	fgh	1	00m	100..	0pqr	0stu	100y
010	bcd	jkh	1	01m	101..	0pqr	100u	0sty
100	jkd	fgh	1	10m	110..	100r	0stu	0pqy
110	jkd	00h	1	11m	11100	100r	100u	0pqy
101	fgd	01h	1	11m	11101	100r	0pqu	100y
011	bcd	10h	1	11m	11110	0pqr	100u	100y
111	00d	11h	1	11m	11111	100r	100u	100y

a. Conversión de decimal a dpd.

b. Conversión de dpd a decimal.

Figura 2.2: Tablas de conversión de decimal a dpd y viceversa.

Del hecho anterior surge la idea de codificar el formato decimal de una manera compacta. La primera versión data de 1971, por Tien Chi-Chen e Irving T. Ho, [364], publicada con varios refinamientos. La versión utilizada en IEEE754 es llamada *Densely Packed Decimal* (DPD), presentada en [83] por Cowlshaw, que tiene la ventaja de utilizar de manera óptima no solo 10 bits para 3 dígitos decimales, sino 7 bits para dos dígitos y 4 bits para 1 dígito, este último codificado como el bcd convencional.

Así, la norma define el *delet* como un grupo de 10 bits que representan 3 dígitos decimales. La mantisa del formato flotante en base 10 se forma por varios delets. La representación es óptima pues si se trata de 5 dígitos decimales, se usan dos bytes: 10 bits para 3 y luego 7 más para los 2 restantes. El bit restante se usa en un campo que se combina con el del exponente.

Y se le llama campo de combinación.

La codificación trabaja de la siguiente manera. Si los tres dígitos decimales en código bcd son los 12 bits $B=(abcd)(efgh)(ijklm)$, entonces, cada bit de la representación dpd se construye a partir de la tabla 2.2a.

De esta manera se forma $D=(pqr)(stu)(v)(wxy)$. También aquí el último bit de cada representación se mantiene constante. Quedan aún 24 de las posibles 1024 combinaciones para futuras extensiones⁸. Para decodificar, se parte de D y se obtiene de nuevo B utilizando la tabla 2.2b.

Ejemplo 23. Para convertir el número $B=437$. En bcd se representa con

$$B = \overset{4}{0100} \overset{3}{0011} \overset{7}{0111}$$

Considerando la tabla de codificación, se ve que $aei=000$ (el primer bit de cada dígito), por lo que corresponde al primer renglón. Entonces, D se forma como 100 011 0 111.

Para decodificarlo, como $v=0$ en la tabla de decodificación, corresponde al primer renglón de nuevo, sin importar el valor de $wkst$. Y de nuevo se obtiene B .

⁸La norma no especifica cómo se deben usar.

Restando aún más ortogonalidad a la norma. La representación fue uno de tantos motivos de debate intenso y el resultado fue que la norma incluye tanto representación en bcd como en dpd, como se explica en el apartado 5.5.c incisos 1 y 2 de la norma.

En general la representación de un número de punto flotante tiene la forma

$$\text{bit} \quad \underbrace{1 \text{ bit}}_{\text{Signo}} \underbrace{w + 5 \text{ bits}}_{\text{Exponente}} \underbrace{T = J \times 10 \text{ bits, } J \text{ declets.}}_{\text{Mantisa}}$$

El campo del exponente, G, es de longitud variable, llamado *campo de combinación*. Mide al menos 5 bits, para los valores especiales de 1.11:

G	Significado
11111	NaN
11110	$\pm\infty$
00000	
10000	± 0
01000	

Si G tiene otra combinación de bits, entonces se trata de un número finito con declets válidos como los de las tablas 2.2a y 2.2b. Como es natural, la mantisa se forma con la concatenación de los bits de los declets y posiblemente alguno del campo de combinación. Aún en números finitos, los primeros bits de $G = g_1g_2\dots g_{w+5}$ determinan la forma de codificación.

Es recomendable utilizar códigos probados en vez de capturar las tablas 2.2a y 2.2b.

El lector interesado por el resto de los detalles de la codificación base 10 debe remitirse a [77] o a la patente 6437715 de Cowlishaw, en USA, donde se dan recomendaciones de implementación.

2.2.3. Precisión al cambiar entre bases

Toda conversión de base entre formatos con precisión finita queda limitada por los alcances del Teorema de Cambio de Base, de Matula, publicado en [260–262]. El lector puede consultar la sección A.7 en la página 368 para más detalles.

No es difícil determinar los valores mínimos de la precisión binaria y decimal para garantizar que la conversión decimal a binario y viceversa conserve el último bit correcto. Supóngase (como hasta ahora) que se tiene p dígitos binarios de precisión y P dígitos decimales. Sea el número real $x \in [2^b, 2^{b+1}]$ y $x \in [10^D, 10^{D+1}]$. Úsese de referencia la figura 2.3.

El error de redondeo máximo del formato binario es 2^{b+1-p} y el del formato decimal es 10^{D+1-P} , es decir, la distancia de x con los NPF vecinos más cercanos. Las conversiones de binario a decimal y decimal a binario tendrán un error de redondeo máximo de $E_D = 5 \times 10^{D-P}$ y $E_b = 2^{b-p}$ respectivamente. La conversión de $\text{fl}(x)$ de binario a decimal no tendrá pérdidas (mayores a E_D)

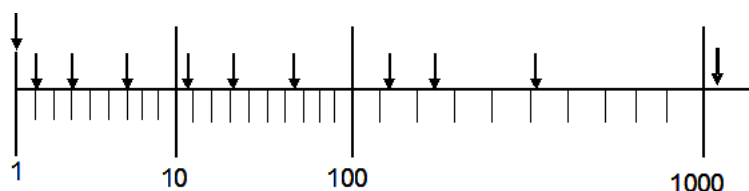


Figura 2.3: Potencias de 2 y 10. Las flechas muestran la posición aproximada de las potencias de 2.

si la suma de los errores es menor que la distancia entre los NPF del formato binario, es decir,

$$E_D + E_b < 2^{b+1-p}.$$

Esto se cumple cuando $P > D + 1 - (b + 1 - p) \log 2$.

Como se supone que $10^D \leq 2^{b+1}$, entonces $D \leq (b + 1) \log 2$, por lo que se obtiene

$$P > 1 + p \log 2.$$

Si $p = 53$ como en precisión doble, entonces con $P = 17$ es suficiente. El proceso en el otro sentido, es decir, la conversión de $\text{fl}(x)$ de decimal a binario, requiere que $E_D + E_b < 10^{D+1-P}$ y se obtiene que para garantizar 53 dígitos binarios correctos se requieren 15 dígitos decimales como mínimo. Dígitos decimales extra ya no estarán correctamente redondeados ($E_b \leq 2^{b-p}$) con 53 bits.

De igual manera se puede investigar para el caso de la cuádruple precisión, $\log_{10} 2^{113} \approx 34.0163$ con 113 dígitos binarios, que se corresponden con 34 dígitos decimales.

Estos cálculos y otros similares determinan los parámetros de los diversos formatos para cumplir con la demanda exigida.

Problemas inesperados pueden surgir al requerir comparar datos con base distinta, uno en base 2 y otro en base 10. Aún no es posible confiar en que los compiladores harán el trabajo adecuado. Lo común es que el compilador convierte un tipo en el otro para poder comparar, dando a lo más una advertencia de que se hizo un redondeo. Lo mejor es seguir las recomendaciones de [48]. El primer paso es comparar exponentes. Los autores presentan varias ideas para resolver eficientemente este problema.

Si los compiladores supieran cómo redondear, la vida sería más fácil.

2.2.4. Modos de redondeo

El modo de redondeo es un parámetro implícito en la norma. El ambiente de programación debe permitir modificar el modo en tiempo de ejecución, de compilación o de depuración, preferentemente los tres, así como el manejo de

excepciones correspondiente⁹. Este parámetro afecta a todas las operaciones inexactas.

Según la norma, todas las operaciones deben realizarse como si se calculara un resultado correcto hasta el infinito y sin cotas, para después ajustarlo a su formato final mediante el modo de redondeo, que puede ser de dos tipos¹⁰. Supóngase que se tiene un resultado infinitamente exacto x y que los NPF más cercanos son $\underline{x}$ y $\bar{x}$, de manera que $\underline{x} \leq x \leq \bar{x}$.

Como en aritmética de intervalos.

- Redondeo al más cercano:
 - TiesToEven. x se convierte en el NPF más cercano. Si están a la misma distancia, se usa aquel cuyo último bit sea 0.
 - TiesToAway. x se convierte en el NPF más cercano. Si están a la misma distancia, se usa aquel con mayor magnitud.
- Redondeo direccionado:
 - TowardPositive. x se convierte en $\bar{x}$.
 - TowardNegative. x se convierte en $\underline{x}$.
 - TowardZero. x se convierte en el NPF más cercano no mayor en magnitud.

Todos estos modos de redondeo ya habían sido presentados en [401], en 1973 por Michel Yohe, junto con los algoritmos para llevarlos a cabo y las pruebas de su correctitud.

El modo de redondeo por default es siempre el redondeo al número más cercano (TiesToEven). Si los dos más cercanos están igualmente cerca, se preferirá aquel que tenga su bit menos significativo igual a cero, lo que se conoce como el *algoritmo del banquero*. La justificación es muy clara en base 10. El ejemplo a continuación deja ver que el sesgo se reduce.

La idea data de 1894, de Jules Tannery, el matemático francés.

Ejemplo 24. Si se va a redondear a dos decimales los números 2.005, 2.015, 2.025, 2.035, entonces redondear empíricamente sumando .005 sesga las cantidades hacia arriba.

La mejor decisión es restar cuando el dígito decimal anterior es par (como en 2.005 y 2.025) y sumar cuando sea impar (como en 2.015 y 2.035). Véase la tabla 2.7.

La utilidad mayor de los modos de redondeo direccionados, a parte de la obvia aplicación en aritmética de intervalos, tiene fundamento en que normalmente los errores de cálculo de las computadoras se deben a unos cuantos errores críticos de redondeo más que a legiones de errores numéricos de todo tipo distribuidos uniformemente en todo el código. El caso evidente son los cálculos alrededor de singularidades, como $\frac{1}{1-x}$ cerca de $x = 1$.

Debe suponerse esta regla con sus debidas excepciones.

⁹Esto es particularmente importante en cuando se trata de aritmética de intervalos.

¹⁰El modo de redondeo es mucho más crítico de lo que aparenta a primera vista. Impacta en el diseño de hardware, de compiladores, depuradores, análisis de algoritmos numéricos, diagnóstico de anomalías, programación de funciones elementales y mucho más.

		Redondeo empírico	TiesToEven
	2.005	2.01	2
	2.015	2.02	2.02
	2.025	2.03	2.02
	2.035	2.04	2.03
Suma:	8.08	8.1	8.07
Error	absoluto	0.02	0.01
	relativo	0.00247525	0.00123762

Tabla 2.7: Diferencia entre el redondeo empírico con el TiesToEven. Se muestra el caso de la suma, con los errores absoluto y relativo.

Aislar un bloque de código sospechoso y ejecutarlo con modos de redondeo distinto puede llevar a un diagnóstico más eficiente del problema. Al mismo tiempo, se debe estar atento a la reorganización del código por parte de la etapa de optimización del compilador, para evitar cambios de semántica numérica.

Diseñar buenas pruebas es normalmente más difícil que diseñar bien las rutinas que se quieren probar.

Las primeras experiencias con lo que en su momento era la propuesta de norma, hizo que los fabricantes desarrollaran esquemas eficientes para realizar el redondeo. El primer bit ya fue presentado en la página 37 al ver las propiedades de los NPF IF: el bit de guardia. En la representación, este se coloca al lado del bit menos significativo de la mantisa, como es de esperarse.

Estudios estadísticos pioneros son [228]¹¹, donde Kuki y Cody analizaron el desempeño de los bits de guardia con bases 2 y 16 y diversos modos de redondeo; y [212], donde Kanedo y Liu calculan el error relativo de la suma utilizando dígitos de guardia. En ambos trabajos, que datan de 1973, se justifica su necesidad.

Se han hecho varios análisis posteriores de los modos de redondeo, donde diversos algoritmos se han diseñado buscando eficiencia, al tiempo de cumplir los requisitos de la norma. En [106] se analizan 3 distintos métodos de redondeo utilizados por distintas arquitecturas. En el reporte [314], se propone redondear en dos etapas, primero construyendo una tabla de redondeos y luego con un esquema predictivo. En todos los casos, esto impacta directamente en el diseño del hardware.

Bit de redondeo

El bit de guardia asegura que la resta no incurra en errores relativos altos, pero no es suficiente para asegurar un redondeo que garantice un error máximo de .5 ulp. Para esto, se agrega un bit más, que afina el redondeo y es conocido como *bit de redondeo*. Este se coloca al lado del bit de guardia, en la parte menos significativa.

¹¹ Antes había sólo modelos probabilísticos que modelaban la propagación de errores.

En la práctica, cada fabricante decide qué hacer. Con el formato de doble precisión se tiene una mantisa de 52 bits y uno implícito (el *ufp*). Al inicio del segundo milenio de nuestra era, las diferencias son aún notables: en vez de usar 52+2 bits, Intel dice usar 80+2 bits en sus registros flotantes, aunque 64 son los utilizados por la mantisa, así que trabaja con 64+2 para redondeo. Por su parte, algunos procesadores de IBM, como el del servidor z990, usan 56+4 bits. Sparc sí usa los 52+2 que la norma indica.

Sticky bit

La diferencia de bits de redondeo daría problemas de portabilidad, de no ser por un mecanismo adicional que termina por asegurar que todos redondeen como si redondearan a partir del resultado infinitamente correcto.

El bit queda “pegado”.

Para indicar que durante cierta operación hay más bits residuales que los que se pueden guardar en el formato numérico destino, se agrega un tercer bit llamado *sticky bit*¹². Una vez que el sticky se prende, ya no puede apagarse pues es un indicador de que ha ocurrido una pérdida. Claro, esto es mientras se están haciendo cálculos en los registros, pues una vez regresado el valor a la memoria, sus 52+1 bits redondeados es lo único que prevalece. Aunque ya estaba comenzando a ser utilizado, este concepto fue publicado en [227].

Con este tercer bit ya no se necesita tener una precisión intermedia de una gran cantidad de bits. Uno sólo sirve para representar toda mantisa descartada durante la normalización: es el OR lógico de los bits que se pierden.

Ejemplo 25. *Supóngase que se tiene un número en precisión simple cuyo formato indicando el bit implícito y los tres bits adicionales es*

0 0000001 (1)110000000000000000000010 000

El campo del exponente es 1, por lo que al dividir entre dos exponente se hace 0, usando la representación de los subnormales. Al ir dividiendo entre 2 sucesivamente (cada fila de la tabla) se obtiene la secuencia

Valor inicial	<i>0 0000001 (1)110000000000000000000010 000</i>
/2	<i>0 0000000 (0)111000000000000000000001 000</i>
/2	<i>0 0000000 (0)011100000000000000000000 100</i>
/2	<i>0 0000000 (0)001110000000000000000000 010</i>
/2	<i>0 0000000 (0)000111000000000000000000 001</i>
/2	<i>0 0000000 (0)000011100000000000000000 001</i>
×2	<i>0 0000000 (0)000111000000000000000000 001</i>
⋮	⋮

y el sticky (en negrita) no debe apagarse pues es la señal de que se perdieron bits.

¹²Alejandro Casares le llama bit pegajoso, en la traducción [299] del libro de Michael Overton [298]

En el caso del redondeo al más cercano, se toman las siguientes decisiones:

Si el dígito de redondeo es ...	entonces se resuelve ...
$< \beta/2$	truncando
$> \beta/2$	sumando 1 ulp (redondeo hacia arriba)
$= \beta/2$	redondeando al más cercano

Sólo en el caso del redondeo al más cercano los bits de guardia, redondeo o pegajoso pueden resolver de otra manera. No hay duda de que la existencia de estos bits resuelve muchos problemas de exactitud, pero su generación y evaluación significa un retraso en los cálculos numéricos, particularmente de los multiplicadores. Hacer el OR lógico de los bits que quedan fuera de la precisión no es una técnica adecuada debido a su mal desempeño temporal.

En algunas referencias no se distingue entre estos 3 bits y en conjunto se les llama bits de guardia, aunque esto pueda llegar a confundir con el primero de estos. Estos tres bits, junto con otros se colocan en los *registros de estado y control*, del procesador. También se incluyen bits para indicar el modo de redondeo empleado y las banderas de excepción.

El tema sigue siendo de gran importancia para los diseñadores de procesadores. Aún en 2009, Gök y Özbilen publicaron [139], donde evaluaron diversos métodos para la generación del sticky bit y proponen una técnica muy veloz.

La bolsa de valores de Vancouver Desde enero de 1982, la bolsa de valores de Vancouver, Canadá, realizaba alrededor de 3000 transacciones diarias, partiendo de un índice de precios con valor 1000. Operaban con cuatro decimales y truncaban a tres en cada ocasión. Esto resultaban en la pérdida de unos 20 puntos diarios en el índice de precios. En noviembre 25 de 1983, el índice de precios estaba en 524.811 puntos pero el mercado lucía muy bien, por lo que decidieron revisar. Después de tres semanas de trabajo los consultores pudieron recalcular y corregir los 22 meses de errores de redondeo, comenzando el siguiente lunes en 1098.892 puntos, 574.081 sobre el valor anterior.

Revisaron en busca de un error ¡casi dos años después!

Algo más sobre los bits

En los primeros años de la computación, mucho se investigó sobre la distribución estadística de los dígitos al inicio y al final de las cantidades, para ver si cumplían con alguna regla que permitiera mejores diseños de hardware y software. Ejemplos claros son [44, 111, 113, 148, 164, 182, 306, 315, 368, 373], que utilizaron ideas de los estudios estadísticos de distribución de dígitos. Muchos se interesaron cuando Benford presentó su Ley de los Números Anormales en [26].

Ley que va en contra del sentido común.

Desde el principio fue claro que operaciones atómicas que requerían muchos bits adicionales para garantizar un resultado correcto en los cálculos tenían que ser optimizadas para lograr tanto exactitud como poco tiempo de ejecución. Tales optimizaciones llevaron al diseño de las ideas que ahora conocemos como

los dígitos de guardia, que se presentaron en artículos como [69, 149, 185, 212, 228, 358, 395], particularmente el bit pegajos (sticky) en [227], donde Kuck muestra en 1977 que un sólo bit, además de los de redondeo, es suficiente para simular precisión extra en el caso de la resta, especialmente si se usa truncamiento.

Sin el sticky bit, con 4 bits de guardia se redondea correctamente el 70 % de los casos, con 5 bits se alcanza el 84 % de redondeos correctos. La aproximación al 100 % es asintótica. Wilkinson ya había probado en [395] que tener g bits de guardia mejora la exactitud en 2^{-g} .

2.2.5. Operaciones

La norma define más de 100 operaciones, que pueden ser clasificadas por tipo y excepción que generan o por la relación de los operandos y el resultado. Como ya se comentó, la idea fundamental es que cada cálculo debe realizarse suponiendo un resultado intermedio con precisión infinita y sin cotas, para luego redondear al formato destino.

En general los orientados a objetos. Para los fanáticos del C++, Java y otros lenguajes similares, la sobrecarga de operadores imposibilita que el tipo de dato esperado influya la selección del operador, dependiendo sólo de los operandos inmediatos, que es contrario al requisito de la norma. Variables atómicas debieran considerarse como intervalos si aparecen en expresiones mezcladas con intervalos, pero no ocurre así. Por ello se prefiere C, tal como lo diseñaron Kernigham y Ritchie, donde la evaluación de expresiones es más segura.

Los interesados en los detalles de todas las operaciones deberán referirse a [77]. Se presentan a continuación sólo 3 operaciones relevantes y útiles para el resto de este texto.

nextUp, nextDown y nextAfter

Estas operaciones permiten ir al siguiente NPF en la dirección indicada. Sea $x \in \mathbb{R}$. Se definen

nextUp(x) el menor NPF mayor que x ,
nextDown(x) el mayor NPF menor que x y
NextAfter el siguiente NPF de x en la dirección de y .

NextAfter puede programarse fácilmente a partir de las otras dos funciones. Algunas definiciones exigidas y propiedades que se cumplen son las siguientes.

1. **nextDown**(x) = -**nextUp**(- x)
2. Si x es el número de menor magnitud en su precisión y negativo, entonces **nextUp**(x) = -0.
3. **nextUp**(± 0) es el número positivo de menor magnitud.
4. **nextUp**(∞) = ∞
5. **nextDown**($-\infty$) = $-\infty$
6. **nextUp**(NaN) = **nextDown**(NaN) = **NextAfter**(NaN) = NaN

y lo mismo se cumple si el argumento es NaN.

Si estas operaciones están disponibles en el ambiente de programación, entonces permiten calcular la precisión de la máquina ε_M de una manera más sencilla que el código 1.1 de la página 27. Solo se necesita evaluar `nextUp(1)`-1 o `nextafter(1,2)`-1.

Estas operaciones también permiten calcular fácilmente $\text{ulp}(x)$ y otros valores útiles.

Conversión de formato externo a interno y viceversa

Se le llama *formato externo* a todo forma de codificación de NPF distinta a la especificada en la norma, como los valores intermedios luego de la lectura de números desde el teclado, archivos o cualquier otra forma de entrada.

La norma 754 define conversiones entre todos las combinaciones de formatos, incluyendo entre decimales y binarios (excepto con formatos externos) aunque sin decir cómo deben realizarse. Tampoco especifica la conversión de cadenas de caracteres a formatos internos, como cuando el usuario captura números flotantes y la computadora realiza la conversión. A continuación se ilustran los problemas que enfrentan estas operaciones.

Ninguna empresa está obligada a revelar su tecnología

Hay dos motivos por los que un número real no puede ser representado: imposibilidad matemática o insuficiencia de precisión. El primero se ilustra en el siguiente ejemplo.

Ejemplo 26. *El inocente número decimal $1/10 = 0.1$. Aún teniendo una representación decimal finita, su representación binaria es infinita.*

$$\begin{aligned} 10^{-1} &= (0.0000110011001100110011\dots)_2 \\ &= (0.\widehat{000011})_2 \end{aligned}$$

Esto significa que se encuentra exactamente entre dos números de punto flotante y puede ser representado por cualquiera de estos con la misma precisión¹³.

Otro ejemplo ilustrará el problema de la falta de precisión.

Ejemplo 27. *Supóngase que se trabaja con formato de simple precisión. Al sumar $2^0 + 2^{-23}$, el 2^0 ocupa el bit implícito mientras que el 2^{-23} ocupa el bit menos significativo de la mantisa de 23 bits. En notación binaria tenemos*

$$\begin{aligned} b &= 2^0 + 2^{-23} \\ &= (1.00000000000000000000001)_2 \end{aligned}$$

que utiliza toda la mantisa del formato de simple precisión. Al realizar la operación

$$\begin{aligned} c &= 2^0 + 2^{-24} \\ &= 1.000000000000000000000001 \end{aligned}$$

¹³Está en el intervalo $[0.099999999999999984, 0.100000000000000002]$.

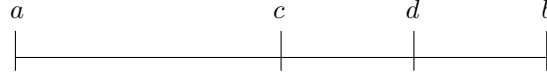


Figura 2.4: Redondeo al más cercano (TiesToEven).

deja el último bit fuera de la mantisa y el resultado debe redondearse a $a = 1$.

El símbolo $|$ indica cuando la mantisa del formato termina, todo dígito posterior queda fuera de la mantisa. Esto es debido a que como $2^0 + 2^{-24}$ queda exactamente entre a y b , el redondeo es equivalente al truncamiento (roundTiesToEven) y a es el NPF cuyo último bit es cero. Pero cuando se suma

$$\begin{aligned} d &= 2^0 + 2^{-24} + 2^{-25} \\ &= 1.000000000000000000000000|11 \end{aligned}$$

el resultado d se redondea a b pues $c < d < b$. Esto se ilustra en la figura 2.4. donde

$$\begin{aligned} a &= 1.000000000000000000000000 & (2.2) \\ c &= 1.000000000000000000000000|1 \\ d &= 1.000000000000000000000000|11 \\ b &= 1.000000000000000000000001 \end{aligned}$$

Considerando arquitectura Intel, sumarle 2^{-63} a c resulta en el redondeo a b pues $c + 2^{-63}$ es mayor que c . Una potencia más pequeña aún $2^q < 2^{-63}$, es decir, $q < -63$, ya no logra ese redondeo pues la mantisa de los registros de punto flotante de Intel es de 64 bits, por lo que $\text{fl}(c + 2^q) = \text{fl}(c)$. Y de la misma forma, si la suma queda entre a y c , el redondeo dará a .

El sticky bit resolverá este problema sólo si las operaciones están suficientemente cercanas en el código como para mantener todos los valores en registros flotantes, disponiendo de bits adicionales. Si la variable se copia de la memoria a un registro, los tres bits adicionales son cero (son banderas del CPU).

Aunque estas cantidades tengan representación exacta en el formato de IEEE, al imprimirlas en pantalla deben convertirse a base 10 y su representación no resulta el mismo número que en base 2 en todos los casos. Por ejemplo, $1 + 2^{-7} = 1.0078125$ ocupa toda la mantisa de la simple precisión, pero $1 + 2^{-8} = 1.0039062|5$ requiere una posición decimal adicional.

Mientras el valor de una variable de punto flotante exista en algún registro del procesador, disfrutará de todos los bits disponibles. Pero cuando el registro se necesita para otra cosa el valor de la variable se copia a memoria con la precisión que indica el tipo de dato, redondeando de ser necesario.

Código 2.1: Comprobando si hay diferencia con los valores intermedios

```

1 volatile double x = 3./7.;
2 if ( x != 3./7. )
3     puts( 'Formato intermedio mayor' );

```

Una forma sencilla de verificar la diferencia con el formato intermedio es usando el código 2.1. Es claro que no es la única, pero suficientemente sencilla como para incluirse como prueba del ambiente numérico. Las variables tipo `volatile` toman su valor de la memoria todo el tiempo, jamás de su copia existente en algún registro, por más actualizada que esté.

En algunos casos, funciones como `printf` leen directamente del registro flotante del procesador, por lo que aún cuando el tipo sea de simple precisión, la conversión a base 10 utilizará los 64 bits de mantisa del registro y no los 24 del formato. De esa manera, $1 + 2^{-23}$ aparecerá como 1.00000011920928955|078125, quedando sólo 7.8125×10^{-19} fuera de la conversión. El ejemplo siguiente analiza ¿Error tolerable? este problema un poco más.

Ejemplo 28. *El problema es sencillo de entender. Considérese el formato de doble precisión. La suma $s = 1 + 2^{-1} + 2^{-24} + 2^{-52}$ tiene representación exacta en $\mathbb{F}$, pero su representación en base 10 es*

1.5000000596046449974352299250313080847263336181640625

por lo que no es posible realizar una conversión a base 10 correcta a partir de la representación binaria exacta, así que se redondea. En este caso, s es un número que no tiene conversión de binario a decimal exacta.

Similarmente, la suma $S = 1 + 2^{-1} + 2^{-24} + 2^{-53}$ no cabe en el formato de doble precisión de IEEE, por lo que la representación es inexacta y por supuesto la conversión al decimal exacto:

1.50000005960464488641292746251565404236316680908203125

tampoco es posible. No ha sido difícil encontrar estos ejemplos adversos, como puede apreciarse.

Mucho se ha estudiado al respecto sobre cómo hacer la conversión correctamente tanto en un sentido como en el otro desde los inicios de la computación. Por ejemplo en 1962, Lynch en [252] y Lake en [232], o Goldberg en 1967 [142], quien observó de que con m dígitos binarios y n dígitos decimales la conversión en un sentido y otro es exacta (con redondeo) si $2^{m-1} > 10^n$. En [262], 1968, David Matula analiza las condiciones del formato interno para la conversión exacta (cuando es posible) o al menos la conversión redondeada con 1 ulp correctamente redondeado.

Estos y otros trabajos fueron la base para que Clinger [65], Steele y White [159] diseñaran en 1990 algoritmos que logran la conversión exacta a la hora de leer o imprimir valores numéricos, salvo redondeos, utilizando la precisión recomendada por la norma IEEE754 de 1985. Ese mismo año, los laboratorios Bell publicaron un manuscrito [130] de David Gay donde se revisan los trabajos de Clinger y Steele, aportando algunas mejoras, que incluyen el uso de sólo doble precisión, excluyendo la necesidad de precisión extendida¹⁴.

Al lector interesado le conviene revisar el trabajo de 1996 de Burguer y Dybvig [58]¹⁵, en el que se quejan de la poca eficiencia temporal del algoritmo de Steele y White y presentan una mejora que reduce 70 veces el tiempo de cómputo, haciéndolo un algoritmo no sólo elegante, sino práctico. Los dos artículos [58, 159] trabajan con formato de punto fijo y flotante.

Fused multiply-add (fma)

Un producto seguido de una suma son una de las parejas de operaciones consecutivas que con más frecuencia aparecen en todo tipo de cálculos numéricos. Aquí se hace referencia a la evaluación de la expresión

$$a + b \times c \quad (2.3)$$

que aparece en cálculos tan básicos como frecuentes: producto de vectores, al evaluar polinomios, operaciones con matrices, cuadraturas, estadística, etcétera.

En la APF convencional, $a + b \times c$ requiere dos redondeos, por lo que el programador obtiene $\text{fl}(a + \text{fl}(b \times c))$ en vez de $\text{fl}(a + b \times c)$. El riesgo consiste en que al hacer *doble redondeo* se puede llegar al resultado incorrecto, además de un error final mayor en la evaluación. Esto ocurre cuando se implementan mal algunas operaciones básicas, sin respetar la norma.

Siendo claros, casi toda operación conlleva dos redondeos que pueden ser inofensivos, primero cuando el cálculo se hace en los registros del procesador y el resultado se redondea y luego cuando el resultado se pasa de un registro al formato destino final en la memoria RAM.

Ejemplo 29. Si el resultado de una evaluación se almacenará en una variable de simple precisión como en las línea de código en lenguaje C

```
float f;
f = 3.3+4.5;
```

donde la suma se realiza por ejemplo primero con 80+2 bits¹⁶ y se redondea en un registro flotante de 80 bits, para después ajustar al formato destino de 32 bits de la variable `f` redondeando por segunda ocasión.

¹⁴Actualmente, la norma no tiene precisión extendida, pues de la doble se pasa a la cuádruple.

¹⁵Raffaello Giuliotti le hizo modificaciones en 2006.

¹⁶Los bits extra, que pueden ser más dependiendo del procesador, o los bits de guardia en el caso de la resta.

El ejemplo anterior es un caso inevitable, e ilustra los fenómenos que en ocasiones escapan a la comprensión del programador promedio. En [283], se hace un análisis más detallado de los problemas del doble redondeo y la manera como afecta aritmética flotante moderna. Es siguiente ejemplo detalla más el problema.

Ejemplo 30. *Para comprender mejor el peligro del doble redondeo, supóngase el caso de un sistema de punto flotante hipotético con $\beta = 2$ y $p = 3$ bits de precisión: si el resultado exacto en binario de una operación es $x = .100101$, el redondeo al más cercano es $.101$ por los motivos descritos en la sección 2.2.4, página 56 con un error relativo de*

$$\begin{aligned} \frac{|.100101 - .101|}{.100101} &= \frac{|.578\,125 - .5625|}{.578\,125} \\ &= .027027\dots \end{aligned}$$

que es menos del 3 %. Si se realiza doble redondeo, primero a 4 bits en un formato intermedio y luego a los tres bits finales, se obtiene como resultado $.100$ en vez de $.101$. La razón es que, cuando se redondea primero a 4 bits, el valor x inicial está en el centro del intervalo $[.10010, .10011]$, por lo que se prefiere el número con el último bit en 0. Al redondear $.1001$ a 3 bits, por la misma regla se obtiene $.100$ con un error relativo de

$$\begin{aligned} \frac{|.100101 - .100|}{.100101} &= \frac{|.578\,125 - .5|}{.578\,125} \\ &= .135135\dots \end{aligned}$$

que es más del 13 %. Obsérvese que en cada redondeo, el número inicial estaba exactamente en medio de las dos representaciones más cercanas. Por ello, utilizar variables en una sola precisión es mejor y más fácil de analizar en la mayoría de los casos, principalmente para los programadores que NO pueden controlar el redondeo de variables intermedias, normalmente declarados con precisión mayor (por el tamaño extra de los registros flotantes más los bits extras).

El doble redondeo se presenta cuando

1. se usa una precisión intermedia distinta a la precisión del formato destino,
2. al utilizar operandos escalados para los que el resultado final será un sub-normal o
3. cuando hay conversiones de binario a decimal y viceversa.

En la mayoría de los casos, está bien determinado cuándo el doble redondeo es inofensivo, como lo estudia Figueroa en [114], donde presenta la cantidad de bits necesarios en los cálculos intermedio para evitar este problema, al menos en las operaciones elementales.

Es fácil calcular el error en la expresión $a + b \times c$:

Proposición 5. *El error máximo al evaluar la expresión (2.3) es de 2 unidades de redondeo, es decir,*

$$\text{fl}(a + \text{fl}(b \times c)) = (a + b \times c)(1 + 2\delta) \quad (2.4)$$

con $\delta < \mu$.

Demostración.

$$\begin{aligned}
 \text{fl}(a + \text{fl}(b \times c)) &= \text{fl}(a + (b \times c)(1 + \delta_1)) \\
 &= \text{fl}(a + b \times c + \delta_1(b \times c)) \\
 &= (a + b \times c + \delta_1(b \times c))(1 + \delta_a) \\
 &= a + b \times c + \delta_1(b \times c) + \delta_a(a + b \times c + \delta_1(b \times c)) \\
 &= a + b \times c + \delta_a a + \delta_1(b \times c) + \delta_a(b \times c) + \delta_a \delta_1(b \times c)
 \end{aligned}$$

y si $\delta = \max\{|\delta_1|, |\delta_a|\}$, entonces se obtiene

$$\begin{aligned}
 \text{fl}(a + \text{fl}(b \times c)) &\leq a + b \times c + \delta a + 2\delta(b \times c) \\
 &< a + b \times c + 2\delta(a + b \times c) \\
 &= (a + b \times c)(1 + 2\delta)
 \end{aligned}$$

suponiendo que $\delta_a \delta_1 \approx 0$. □

Sin embargo, la cota anterior puede mejorarse de dos maneras:

1. considerando el underflow gradual y
2. evaluando la expresión (2.3) de manera atómica, es decir, con un sólo redondeo.

Para el primer caso, en [162], Hauser mejora el resultado (2.4) considerando underflow gradual. La cota exacta se muestra en la siguiente

Proposición 6. *Sea un sistema numérico $\mathbb{F}$ con underflow gradual y a, b y c números de punto flotantes no ceros. Además, sea a un número con mantisa normalizada ($a \in \mathbb{F}$ pero $a \notin \mathbb{U}$). Entonces,*

$$\text{fl}(a + \text{fl}(b \times c)) = (a + (bc \times \rho)) \times \sigma \quad (2.5)$$

y se cumplen las desigualdades

$$\frac{1 - 2\delta}{1 - \delta} \leq \rho \leq \frac{1}{1 - \delta} \quad (2.6)$$

$$1 - \frac{3}{2}\delta < \sigma < \frac{1}{1 - \frac{3}{2}\delta} \quad (2.7)$$

donde δ se define como en la expresión (1.20), que es el error relativo por redondeo en $\mathbb{F}$.

Es claro que ρ es la perturbación en el producto y σ la de la suma. Hauser agrega que si no ocurre underflow, entonces es posible probar las cotas más exactas

$$1 - \delta \leq \rho \leq 1 + \delta \quad (2.8)$$

$$1 - \delta \leq \sigma \leq 1 + \delta \quad (2.9)$$

lo que suena lógico considerando la identidad

$$\frac{1 - 2\delta}{1 - \delta} = 1 - \delta - \delta^2 - \delta^3 - \dots$$

que indica una perturbación ligera de $1 - \delta$.

De no tener underflow gradual, entonces las cotas se duplican en magnitud y se llega al resultado de la proposición 5. La prueba de la proposición 6 está en [162] (Teorema 3.4.3, en el apéndice).

Por los anteriores motivos, surge la idea de evaluar la expresión (2.3) de manera atómica, con un sólo redondeo al final y se define así una de las operaciones de mayor importancia de la norma.

$$\text{fma}(a, b, c) = a + b \times c.$$

Esta operación *fused multiply-add*, es novedad en la norma, pues en la de 1985 no aparece, pero no lo es en la práctica¹⁷.

También *fmac*.

La operación *fma* fue utilizada inicialmente en DSP (procesadores digitales de señales por sus siglas en inglés), en 1999 fue agregada como rutina estándar en el lenguaje c99 por ANSI y considerada en la revisión de la norma flotante de 1985¹⁸.

La ventaja de la operación *fma* especificada por la norma, además de una implementación más veloz, es que se obtiene

$$\text{fma}(a, b, c) = (a + b \times c)(1 + \delta) \quad (2.10)$$

con $|\delta| < \mu$ porque sólo se realiza un redondeo. Adicionalmente, la operación atómica *fma* posee otras ventajas:

- Se reduce su tiempo de cómputo al ser una operación atómica y tener alto grado de paralelización, explotando bien el pipeline de los procesadores modernos¹⁹, a diferencia de sumar y multiplicar por separado (con redondeos también separados). Para detalles consúltese [222].
- Procesos como el producto de matrices y vectores acumulan la mitad de errores de redondeo (ver [290] donde se calcula con exactitud hasta el penúltimo dígito).
- Es sencillo extenderlo para calcular $\pm a \pm b \times c$.

¹⁷La arquitectura VAX de la compañía Dec ya la incluía en 1977, como parte de la instrucción POLY, que evaluaba polinomios con la técnica de Horner

¹⁸*fma* es la pieza central en el conjunto de instrucciones de punto flotante de algunos procesadores, como el Itanium, RS6000 y PowerPC.

¹⁹Los exponentes de a y de $b \times c$ se ajustan al mismo tiempo, así que no hay esperas, sólo un poco más de espacio (registros suficientemente grandes) para garantizar la normalización de exponentes.

- La suma y la multiplicación se programan como casos particulares: $a+1 \times c$ y $0 + b \times c$ y se reduce el espacio necesario en el microprocesador.
- La programación de diversas funciones del ambiente numérico se facilita, como NextUp, $\text{ulp}(x)$ y similares (ver [284]).
- Se facilita la simulación de mayor precisión con software, como se muestra en [243], donde se detallan las ideas tras la excelente biblioteca de funciones BLAS.
- Permite calcular cocientes y raíces cuadradas más eficientemente. Por ejemplo, hay instrucciones que aproximan con 8 bits de precisión ambas operaciones. A partir de allí, se refina con unas cuantas aplicaciones de Newton-Raphson con fma .

Para la división de a/b , se parte de una aproximación (por tabla) de $1/b$ y se considera $f(x) = b - 1/x$, iterando con

$$\begin{aligned} e_i &= 1 - bx_i && \text{el error en la aproximación} \\ x_{i+1} &= x_i + e_i x_i && \text{la siguiente aproximación.} \end{aligned}$$

Ahora ya se puede multiplicar $a \times (1/b)$. Todas las operaciones redondeadas al más cercano. El residuo puede calcularse fácilmente como $a - b \times (a/b)$, siempre y cuando se respete el orden de las operaciones.

Para la raíz, de nuevo se aplica Newton-Raphson a la función $f(x) = 1/x^2 - a$, con una buena aproximación de $1/\sqrt{a}$ y se itera

$$\begin{aligned} e_i &= \frac{1}{2} - \frac{ax_i^2}{2} \\ x_{i+1} &= x_i + e_i x_i \end{aligned}$$

y este caso, al igual que en la división, se llega en pocas iteraciones al NPF correctamente redondeado.

Estos ejemplos se detallan y demuestran en [80, 81] publicados altruistamente por Intel. El trabajo de Diran Sarafyan ya veía posibilidades similares [333].

Por otra parte, peraciones como $a \times b + c \times d$ no resultan tan obvias pues en general $\text{fl}(\text{fl}(a \times b) + c \times d)$ es distinto de $\text{fl}(a \times b + \text{fl}(c \times d))$ en $\mathbb{F}$ y el compilador no puede tomar la mejor decisión fácilmente, requiere ayuda. No es fácil dar esa ayuda, como explica Kahan en [203]. Esto ocurre cuando se implementa con fma el producto de un número complejo por su conjugado. Matemáticamente la parte imaginaria es cero, pero no en $\mathbb{F}$ ²⁰.

Otro caso similar es al calcular el discriminante $b^2 - 4ac$ en la ecuación cuadrática. Con fma se pierde la relación válida en $\mathbb{F}$ de $\text{fl}(b^2) > \text{fl}(4ac)$ cuando $b^2 > 4ac$, ocasionando problemas en vez de resolverlos.

²⁰En el procesador PowerPC, deshabilitaban fma para compilar algunas versiones de programas como MatLab.

Otro ejemplo bien conocido es la hipotenusa de un triángulo, que se estudia con mayor detalle en el siguiente capítulo.

La definición de c99 sí permite activar o desactivar `fma` por bloques de instrucciones, por lo que el programador tiene el control para decidir cómo hacer la evaluación de la expresión. Desafortunadamente esta característica está disponible sólo parcialmente en algunos compiladores.

Más recientemente, en 2005, Boldo y Muller en [284]²¹ presentaron la forma de calcular varias funciones básicas del ambiente numérico (`ulp`, `nextAfter` y otras) haciendo uso de `fma`. Además, muestran que si el redondeo es al más cercano, entonces es posible representar el resultado de $\text{fma}(a, b, c)$ de manera exacta en $\mathbb{F}$ como la suma de dos números de punto flotante, acotando el error. Condiciones más generales y una prueba formal la ofrecieron hasta 2011, en [35], que vale la pena por el lujo de detalles.

Diversas optimizaciones de `fma` se han desarrollado para todo tipo de arquitecturas, dependiendo de la necesidad de menor tiempo, espacio o consume de energía, tanto para CPUs como para GPUs. Excelente ejemplo (y resumen) de esto es el trabajo [129] de Galal y Horowitz.

2.2.6. Infinito, NaNs y bit de signo

En este apartado, íntimamente relacionado con el de excepciones, la norma define las circunstancias que deberán dar como resultado $\pm\infty$ y los dos tipos de NaN, así como el significado y usos del bit de signo.

Un aspecto importante es la *propagación* de los valores especiales. Esto significa que si se evalúa la expresión

$$\cos(e^{\frac{1}{xy}})$$

y el producto xy causan overflow, entonces su resultado correcto es ∞ , por lo que el cociente $\frac{1}{xy}$ debe resultar en 0, así que el resultado final es $\cos(1)$. Por supuesto, el programador debe poder enterarse de que ocurrió el overflow, pero el procesador (o intérprete) no debe tomarse la atribución de detener la ejecución del programa. Más que otra cosa, este apartado de la norma trata de casos especiales, como es de esperarse al operar con los valores especiales de $\mathbb{F}$.

Infinitos

En matemáticas, infinito representa lo indeterminado, no es un resultado numérico. Sin embargo, al hacer operaciones numéricas con computadoras, es conveniente darle otro significado que sea útil para la automatización de cálculos.

Por ello, $\pm\infty$ es un resultado válido si se usa conforme la norma lo indica. Claro, como representa a todo número en $\mathbb{F}$ cuyo exponente es mayor que $e_{\text{máx}}$, el CPU debe levantar la bandera de *operación inexacta*, además de la de overflow.

La excepción . . .	se presubstituye con . . .
Operación inválida	NaN
División entre cero	$\pm\infty$ (con el signo adecuado)
Overflow	$\pm\infty$
Underflow	0
Inexacto	redondeo u overflow

Tabla 2.8: Algunas expresiones con resultados especiales de $\mathbb{IF}$.

En el caso más común, $\pm\infty$ es el resultado exacto con operandos válidos cuando ocurre una división entre ± 0 , no es un error numérico.

$$\begin{aligned}x/0 &= \infty \\ x/(-0) &= -\infty\end{aligned}$$

Si los(el) operandos son inválidos y el resultado de la operación se almacenará en una variable, la variable debe resultar en NaN. En este caso tenemos las operaciones como

- 1. $\infty \times 0$,
- 2. $\text{fma}(0, \infty, a)$, o
- 3. $\text{fma}(\infty, 0, a)$ y
- 4. $\infty - \infty$

para algún valor $a \in \mathbb{IF}$ normal.

Otra característica importante es la *presubstitución*. Consiste en sustituir un valor o un procedimiento para evitar o corregir los problemas de las excepciones (ver el siguiente apartado). De esta manera, se espera que no sea necesario tomar medidas agresivas como interrumpir el programa en ejecución. Cada excepción evitada por anticipado, tiene un valor posible para la presubstitución determinado por la norma y se muestran en la tabla 2.8.

A estos resultados, el programador puede agregar los suyos propios, dependiendo de la aplicación. El más obvio es la sustitución de $\pm\infty$ por el número de mayor magnitud representable. La presubstitución permite al programador ignorar las excepciones o posponer su detección para mejores lugares del código, o simplemente pausar la ejecución durante la depuración. Son el único método completamente portable para manejar excepciones, pero si no se implementan correctamente pueden perjudicar seriamente el desempeño del programa.

Esto debiera enseñarse en todo curso de programación.

Ejemplo 31. Si durante un proceso iterativo se va a calcular el cociente de dos números como parte de una condición como

```
if ( fabs(a/b) < eps )
```

entonces tal vez se está esperando que la relación vaya decreciendo. Si al inicio de las iteraciones alguna (o ambas) variables tomara como valor a ∞ , por ejemplo leída como el resultado de la evaluación de otra función, como $\mathbf{a=IniIzq(x)}$, entonces el programador puede prevenir situaciones excepcionales como $1/\infty$ o

²¹El autor recomienda ampliamente este artículo, que es muy ilustrativo de las posibilidades que ofrece la aritmética de punto flotante, bien empleada, para programadores.

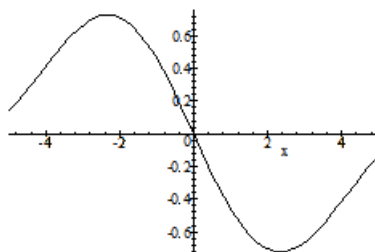


Figura 2.5: La función $(\cos(x) - 1)/x$. Su evaluación en cero puede ser problemática, pero la presubstitución resuelve adecuadamente.

∞/∞ substituyéndolas automáticamente con 0 o 1 respectivamente, con los elementos que provea el lenguaje de programación empleado.

Para mediados de 2006, ningún lenguaje de programación, salvo c99 y Fortran2003, ofrece esta posibilidad que indica la norma, pero siempre es posible simularla con instrucciones como

```
if ( isInfinito(b) )
    Cociente = 0;
```

También puede impedirse que el cociente a/b se evalúe como ∞ alterando ligeramente los valores de a , b o ambos, dentro de los límites válidos de la aplicación, posiblemente con las funciones `nextUp` para mayor cuidado. Esta técnica la utilizan en la librería BLAS para el cálculo de eigenvalores de matrices de gran tamaño ($> 20000 \times 20000$).

La presubstitución permite evaluar límites que son imposibles con las operaciones convencionales en $\mathbb{F}$, como $(\cos(x) - 1)/x$ pues el denominador puede ocasionar división entre cero, aunque la función tiene una gráfica bastante simple en 0. Ver figura 2.5.

Cuando la operación y valores lo ameriten, se puede presubstituir con NaN. En todos los casos, las banderas correspondientes deben levantarse (ver el apartado de excepciones). Para más detalles e ideas, ver las notas de Kahan [206].

NaNs (Not a Number)

NaN es el resultado correcto para una operación inválida, imposible de calcular con límites o porque cualquier otro valor (incluyendo 0 e ∞) puede resultar más confuso. Se definen dos tipos de NaN.

1. qNaN (NaN silencioso) controlado por el usuario, adecuado para diagnóstico retrospectivo.
2. sNaN (NaN de alarma) es una alarma que avisa de operaciones inválidas que requieren forzosamente de tratamiento, es decir, interrumpen el cálculo al no poderse propagar, como como $\sqrt{-1}$, a^{NaN} , $\sin(\infty)$, $0 \times \infty$ y otras similares.

Estos dos resultados se diferencian en la codificación de la tabla 2.2 porque el primer bit (el menos significativo) de la mantisa es 1 para el qNaN y es cero en el otro caso. Si se trata de formato decimal, entonces qNaN posee el bit 6 del exponente (el campo de combinación G) apagado.

La comparación con NaN siempre es falsa, por lo que instrucciones como

```
if ( x != x )
    return Operacion.Invalida;
```

sirven para saber si x es NaN. El programador deberá tener cuidado de comprobar que el compilador que utiliza no cambia esta línea al optimizar sin precauciones, pues parecería que siempre es falsa en aritmética convencional. Por fortuna, muchos lenguajes definen funciones como `isnan` o `isinf` para identificar estos y otros valores. En ANSI C estos se declaran en el archivo de cabecera `math.h`.

Bit de signo

Las reglas generales de los signos que sigue la norma son

positivo+positivo = positivo
negativo+negativo = negativo.

Se define que $x - y = x + (-y)$ y la operación $x + x = x - (-x)$ debe preservar el signo de x aún cuando sea cero, es decir, sólo $(-0) + (-0)$ y $(-0) - (+0)$ dan -0 .

Cuando en un programa debe decidirse saltar una discontinuidad, como en $\frac{1}{x}$ al acercarse a 0, pasar de un lado a otro de la discontinuidad es riesgoso. En caso de que el signo del punto de cruce por la discontinuidad se necesite, entonces se requiere algo más de información para conocer en qué extremo se está.

Esta información extra es el signo del 0. Esto ofrece la posibilidad de diferenciar entre $f(-0)$ y $f(0)$ cuando sea válido en $\mathbb{IF}$ (lo que depende de las propiedades de continuidad de f). El caso obvio es $1/a$.

Ejemplo 32. *Si se está evaluando $1/x$ para valores negativos, cada vez más pequeños en magnitud, entonces es posible llegar al punto en el que el cociente se calcule siendo x un número subnormal, por lo que se obtendrá $-\infty$ de manera correcta.*

Por otra parte, si no se pasa por los valores subnormales y x se hace tan pequeña que resulta en -0 (por venir del lado izquierdo del eje), entonces el resultado que se espera de $1/(-0)$ es $-\infty$ y no ∞ , como podría ocurrir en caso de no hacer diferencia por el signo del cero.

En el sentido del ejemplo anterior, debe ser posible evaluar una función desde $-\infty$ a -0 y luego desde 0 hasta $+\infty$. Si una computadora no respeta este mandato de la norma es seguro que graficará mal gran cantidad de funciones con asíntota en cero, además de funciones complejas con polos.

También se define (por continuidad y propagación) que $\sqrt{-0} = -0$. El estándar no interpreta el bit de signo cuando se trata de NaNs, en ningún caso.

2.2.7. Excepciones

Una *excepción* (numérica²²) es un evento que ocurre cuando una operación no posee un resultado adecuado para toda aplicación razonable. Tal operación debe hacer que el CPU emita una señal y levante la bandera correspondiente. Es importante considerar que las palabras evento, excepción o señal pueden tener significados distintos en diversos ambientes de programación.

Un resultado no útil para todo tipo de aplicaciones.

Hay quien lo define (como en Java) sólo como el evento que interrumpe el flujo normal de un programa, pero esto es limitativo y supone que las excepciones son errores.

Basándose en los primeros borradores de propuesta de la norma, Eggers y otros²³ publicaron en 1978 un estudio proponiendo la forma de manejar excepciones en [101].

Tradicionalmente se piensa que las excepciones son errores detectados por el procesador y que lo único que resta por hacer es detener los cálculos y resolver de alguna manera, recuperando lo que se pueda, generalmente con la supervisión del usuario. Nada más alejado de la realidad.

$x/0$ no está definido en matemáticas, pero sí en computación.

En la norma IEEE754 se especifican cinco excepciones y la manera correcta de manejar, además de los recursos que el programador debe disponer para hacer resolver adecuadamente en cada aplicación. Las excepciones son

- Operación inválida.
- División entre cero
- Overflow
- Underflow
- Inexacto

y la norma indica lo que de manera obligatoria un lenguaje debiera poner a disposición de los programadores.

²²Excepciones como fin de archivo o fallos de memoria no se tratan aquí.

²³En este artículo, los autores arremeten contra el underflow gradual, que es la propuesta de Payne, una de las autoras y explican la manera de manejar excepciones en el caso de que no haya números subnormales. Claro, la propuesta no prosperó.

La especificación se da en términos de las características que deben cumplir las operaciones y operandos para señalar una excepción. Al ocurrir una excepción, debe enviarse la señal que corresponda y las *banderas de estado* cambiarse adecuadamente. Estas forman parte del registro de control y estado de punto flotante. Por las limitaciones de la APF, toda operación que redondee el resultado debe enviar una señal y levantar la bandera de inexacto.

Los valores correctos ante las excepciones numéricas están señalados claramente en la norma IEEE754 y fueron ya indicados en la tabla 2.8. Estos valores fueron seleccionados con todo cuidado para que los programadores pudieran posponer su manejo a partes adecuadas del código o ignorarlos por completo. Por eso, la bandera levantada por la excepción correspondiente se baja sólo de manera explícita con la instrucción adecuada. Tales instrucciones pueden evaluar el estado de la bandera, guardar su estado o restaurarlo.

Levantar banderas es un efecto colateral de una sentencia de código, lo que NO es conveniente, desde el punto de vista de los expertos en la teoría de lenguajes de programación. Lo cierto es que la computación está plagada de efectos colaterales: tan solo medítese sobre el manejo de archivos o sincronización de procesos paralelos.

Los lenguajes deben definir medios para leer y modificar los valores de las banderas del procesador. Esto depende de la forma como el fabricante del procesador codifica su registro de control. Más sobre su programación se verá en el siguiente capítulo.

2.3. Anexos de la norma

Consisten en información general, razonamientos y recomendaciones para diseñadores y fabricantes de hardware y software, pero realmente es en lo que NO se pudieron poner de acuerdo pero consideraron importante.

- A **Bibliografía.** Documentos en los que se apoyó el trabajo, así como referencias que explican y justifican con mas detalles algunos aspectos.
- B **Evaluación de expresiones.** Expone lineamientos sobre cómo debieran evaluarse, optimizarse y asignarse las expresiones en general. Los compiladores que cambian expresiones como $e=((a+b)-a)-b$ por $e=0$ no respetan la norma pues no permiten rescatar errores de redondeo. El uso de banderas de excepción tampoco tiene una secuencia obvia para los compiladores.
- C **Ajuste de tamaño** (widento). Sugerencias para cuidar el tamaño de los formatos intermedios y finales para permitir el redondeo controlado por el programador. Es importante para evitar problemas como el doble redondeo. 31 de las operaciones incluidas en la norma requieren de este cuidado.
- D **Funciones trascendentes elementales.** Se enlistan las funciones elementales mínimas que un programador debiera tener disponible²⁴. Son públicos los métodos

²⁴La función x^y no se incluye. En 2006 todavía faltaba determinar los peores casos de x^n . En mayo de 2007, Kornerup, Lefevre y Muller ya lo publicaron en [224], al menos para base 2 y suponiendo fina, para todos los modos de redondeo.

saveFlags, restoreFlag, ...

No existe ningún lenguaje de programación que sea ortogonal, sin efectos colaterales (y al mismo tiempo útil).

Es una lástima que x^n no se incluya en esa lista aún.

que garantizan funciones elementales correctamente redondeadas (para todos los modos de redondeo), aunque a veces en dominios limitados, por lo que debiera ser posible disponer de librerías matemáticas exactas hasta .5 ulp. Se trata de garantizar, en la medida de lo posible, preservar propiedades como la monotonicidad y simetría.

E Modos alternativos de excepciones. Se recomiendan las formas como los lenguajes debieran permitir al programador definir sus propias excepciones, así como su respuesta, tanto para un programa completo, módulos individuales e incluso subrutinas o funciones.

F Operaciones básicas con arreglos. Debido a que operaciones como la norma de un vector pueden resultar en un overflow por los cálculos intermedios ($\|\mathbf{x}\| = \sqrt{x_1^2 + \dots + x_n^2}$ y para alguna i , x_i^2 mayor que el máximo NPF en $\mathbb{F}$), se enlistan las funciones que debieran poder calcularse evitando este fenómeno. Son solo 3: $\|\mathbf{x}\|$, $\mathbf{x} \cdot \mathbf{y}$ y $\prod (x_i - y_i)$.

G Lineamientos de depuración. Describe las facilidades que debe tener un programador para depurar programas numéricos y encontrar errores, así como motivos de sensibilidad numérica y excepciones. Esto resulta en sugerencias para desarrolladores de depuradores y la posibilidad de disponer (algún día) de mayores controles en tiempo de depuración.

Estos controles incluyen actividades en tiempo de depuración como:

- Cambiar el modo de redondeo, para detectar programas muy sensibles por mal condicionamiento de problemas²⁵.
- Alterar el manejo de excepciones, aún cuando no se disponga del código fuente.
- Pausar el programa cuando ocurra una excepciones específicas.
- Prender o apagar las banderas de excepción en cualquier momento.
- Guardar el historial de excepciones por subprograma para su análisis posterior, con sus respectivas banderas e historia. Esto al menos en cada NaN producido.
- Detectar las 24 cadenas de bits en dpd (ver 2.2.2) que no representan a ninguna combinación de números en los formatos de base 10.
- Detectar regiones de memoria (variables) sin valores iniciales establecidos.
- La posibilidad de que cada variable que entra a la pila del programa levante una bandera de NaN para que quede la referencia de su origen, aún teniendo un valor correcto.

Por ejemplo al momento de declararla.

Todo programador de rutinas científicas desearía disponer de un compilador así y todo indica que lo seguirá deseando por mucho tiempo.

Otros anexos que se discutieron y no se incluyeron fueron sobre características de lenguajes, incompatibilidad con la norma previa, procesamiento de sub-normales con hardware y otros formatos numéricos posibles (incluyendo tipos de 256 bits que tal vez aparezcan en la próxima revisión).

²⁵Sí es posible confirmar alta sensibilidad al redondeo pero su exposición es accidental. Un esquema parecido es el de repetir los cálculos, permitiendo que programas donde los errores de redondeo se compensan beneficiosa pero aleatoriamente, queden expuestos, detectando sensibilidad. El cómputo repetido con modo de redondeo aleatorio no es fácil de programar bien y es lento: nada se compara con un buen análisis matemático del redondeo.

2.4. Lo que faltó en la norma

Pese a todo el trabajo detrás de la norma y a los excelentes científicos e ingenieros que participaron, han quedado vacíos, algunos ocasionadas posiblemente por intereses comerciales, pero otros por divergencias académicas auténticas. De los borradores del trabajo se sabe que en muchas cosas el debate fue intenso y a veces no llegaron a ningún acuerdo. Ejemplos de anomalías que aún requieren solución y otras en las que el programador puede hacer algo al respecto se presentan en el trabajo de Bailey [20].

2.4.1. Aspectos técnicos

Sin pretender encontrar y exponer todos los vacíos de la norma, sí hay algunos importantes para quienes hacen programas numéricos. Las siguientes son consideraciones generales, incluidas las del autor. Dentro de las cosas que no se detallan o faltaron:

1. Decidirse por un solo formato decimal. Ante la posibilidad de formatos bcd y dpd para formato decimal, solo queda esperar que la portabilidad de software (programas y datos) no tenga problemas serios. Aún no teniendo los, esto implica mayor cantidad de código en alguna parte, ya sea el compilador, el procesador y, en el peor de los casos, el programador final.
2. Para el formato decimal, debiera haber una forma de indicarle al programador que una operación o una variable tiene una de las 24 combinaciones de bits no considerados en el dpd. Si las excepciones no se habilitan, ¿cuál será el resultado si se propaga una de estas combinaciones? No es obvio que su decodificación debiera resultar NaN. La recomendación es verificar la bandera de estado después de cada operación sospechosa, lo que podría complicar los códigos. El resultado es que la presubstitución con fines de propagación deja de ser eficiente.
3. El anexo sobre funciones elementales redondeadas correctamente hasta el último bit debiera ser obligatorio, más aún siendo públicas las técnicas para hacerlo. La única excepción es la función x^y (incluyendo el caso particular x^n), para las que aún no se tienen implementaciones exactas, por desconocerse los peores casos.
4. El anexo sobre depuración debiera ser obligatorio. Los fabricantes de procesadores o depuradores no incluirán algo que es opcional. Siempre habrá algo pendiente que tenga mayor prioridad sobre lo que simplemente se recomienda. Actualmente, los depuradores permiten ver los valores de los registros del procesador, pero no actividades deseables como detener la ejecución hasta que se levanta la bandera de inexacto o exclusivamente la de underflow.

Podría ser útil disponer en tiempo de depuración de una variable alterna en base 2.

Todavía menos cuando lo obligatorio lo incluyen parcialmente.

5. Cuando no se dispone de un depurador como el deseado, algunas anomalías de redondeo pasan de unas versiones a otras de procesador y compiladores, posiblemente cambiando el síntoma anómalo. Sin herramientas eficaces para su detección, muchas persistirán, con resultados inesperados. Sin depuradores eficientes, han sido décadas de experiencia con punto flotante base 2, lo que ha conseguido la valiosa información de cómo hacer las cosas. Con el creciente uso de los tipos decimales, ¿cuánto tiempo se necesitará?
6. La norma no dice que si se define una variable decimal en `dpd`, la inspección de su valor durante la depuración debe mostrar tanto la codificación interna como el valor decodificado, aunque sería de esperar que todo compilador que incluye tipos decimales ofrezca esta característica que no implica muchos problemas. Sería muy agradable que con pasar el mouse sobre la variable se mostraran ambos valores.
7. Como el manejo alternativo de excepciones (las definidas por el usuario) no es obligatorio, sino sólo aparece como información en un anexo, el programador no puede confiar que su programa será portable, pues el manejo de excepciones es diferente en distintas arquitecturas. Significa que el programador no puede confiar en expresiones como $e^{-\frac{1}{x}}$, donde si $x = 0$ el cociente da infinito y la exponencial es 0.
8. No se indica qué ocurrirá al dispararse dos excepciones a la vez, como inexacto y underflow. Si por el ambiente de programación, una de ellas obliga un salto o una trampa, no es claro si el compilador debe indicarlo o si el depurador permitirá seleccionar el camino por seguir.
9. No es fácil regular la sintaxis de los predicados que detecten o controlen las excepciones, pues eso depende de la semántica de cada lenguaje, así como sus modos y estilo. El problema verdadero es que aún siendo el mismo lenguaje, al pasar a otras arquitecturas las llamadas pueden ser distintas y cada fabricante defiende sus propios esquemas. En estos casos, la portabilidad es inversamente proporcional a la legibilidad de los códigos, que es primordial para efectos de mantenimiento.
10. En ambientes multitarea (multihilo), que se espera que dominen el mundo de los procesadores de aquí en adelante, el proceso hijo hereda del padre parámetros de precisión, modo de redondeo y manejadores de excepción, pero no se define si también hereda las banderas de estado o si se reinician en todas en cero para este nuevo hilo (que es lo que habría que esperar: el nuevo hilo no ha realizado ninguna operación)²⁶. Esto fue discutido desde 1992 por Goldberg en [141].
11. La observación anterior debiera extenderse al uso de la función `fork()`, es decir, es claro que algunos atributos deben heredarse a los procesos

²⁶En el caso de los `uthreads`, lo mejor sería compartir el estado de PF del verdadero thread del que penden.

hijos, pero no necesariamente todos. El programador es quien debe tomar la decisión.

12. Posiblemente queda fuera del contexto inicial y propósito de la norma, pero la programación de procesos sincronizados debe analizarse con mayor cuidado.
13. Valdrían la pena análisis como el que Evgenija Popova publicó en 1995 [308], que si bien concluye con buenas perspectivas de las normas 754 y 854, también pone de manifiesto sus inconsistencias formales (algebraicamente).

Mucho de lo anterior fue discutido. Cuando un tema fue enviado como opcional o informativo (es lo mismo para efectos prácticos) a un anexo, los participantes en la reforma fallaron en ponerse de acuerdo, condenando a toda la comunidad de programadores y analistas numéricos a realizar análisis de punto flotante cada vez más complicados (lo sencillo poco a poco se cubre) sin disponer de herramientas que orienten y diagnostiquen con eficiencia, dando información amplia y útil sobre las características de las anomalías.

2.4.2. Aspectos educativos

Otro aspecto problemático que no depende de la norma, cae en la educación. Por su parte, los profesionales de la computación reciben (o debieran hacerlo) en algún momento instrucción sobre programación numérica, al menos generalidades. Esto con el objetivo de una orientación mínima que les permita saber cuándo vale la pena preocuparse al desarrollar sistemas. En México, la ANIEI²⁷ define perfiles profesionales en los que se han determinado una proporción de conocimientos diferentes, pero sin especificar detalles.

¿Cuánto le cuesta a una empresa el retraso al buscar errores numéricos en los módulos financieros de un sistema con fecha límite de entrega que no podrá cumplirse? ¿Qué tan frecuentemente ocurre esto? Sin respuestas a estas y otras preguntas, la decisión de qué tanto enseñar sólo puede ser por apreciaciones personales o de grupo.

Por otra parte, los profesionales de otras ramas (químicos, economistas, físicos-matemáticos, geólogos, etcétera) hacen uso de paquetería para obtener resultados. En la mayoría de los casos no tienen problemas numéricos graves pues utilizan menos precisión que la que aporta la computadora. En el caso de desarrollar sus propias rutinas, un problema sería la inestabilidad, pero es mayor cuando se realiza investigación y se usan modelos matemáticos un poco más complejos. Eso sin mencionar el asombro que le causa al programador no profesional el mensaje NaN en sus resultados.

Los recaudadores de impuestos no perdonan ni un centavo y es cuando los errores numéricos pegan donde más duele.

²⁷Asociación Nacional de Instituciones de Educación en Informática

Es el caso de los actuarios, estadísticos y otros investigadores que incluso usan simulaciones por computadora. Es fácil detectar anomalías cuando un resultado diverge mucho, pero cuando todo parece estar bien y los resultados tienen la mitad de las cifras correctas, tal vez ya no es tan sencillo darse cuenta. ¿Hasta dónde se le debe advertir a estos profesionales? Esa es una discusión para otro lugar.

* * *

Pese a todo lo anterior, esta sigue siendo una de las normas con grandes beneficios y de uso muy extendido. Quizás la siguiente revisión, dentro de 15 años, con la experiencia creciente de los programadores, características de nuevos lenguajes y procesadores novedosos, permita que el siguiente comité se ponga de acuerdo más fácilmente. Desde otro punto de vista, ya no es posible pensar en no disponer de una norma, aún cuando sea parcialmente respetada por el mundo de la computación.

El éxito en la solución de muchos problemas numéricos de la computación moderna es un hecho transitorio, como lo muestra el libro de Bo Einarsson [103], donde se trata el tema del incremento de los problemas críticos de la exactitud esperada en las nuevas y futuras computadoras. Habrá de seguirse revisando esta norma conforme evolucionen lenguajes, procesadores, compiladores y la computación en general, pues con los modelos de cómputo actuales no se dispondrá de precisión infinita jamás.

Capítulo 3

Programando la norma

Cuánta gloria es - y cuánto miedo
- ser una excepción.

Louis Charles Alfred de Musset.

Apoyarse en la norma es útil, pero durante las décadas en que no la había se hicieron gran cantidad de buenos cálculos y se desarrolló experiencia valiosa. Mucha de esa experiencia sigue siendo útil¹ y se complementa con los lineamientos que siguen cada vez más fabricantes de hardware y software. En honor a eso, se presentan primero algunas ideas generales de cómo minimizar los problemas de exactitud de los cálculos numéricos en una computadora, para luego pasar al uso de la norma.

Una sección completa se ha dejado para el manejo de excepciones, debido a tres factores:

1. el poco peso que se les da en la formación de profesionales,
2. su gran incidencia en los errores de diseño de programas y
3. el enorme desconocimiento de su manejo, particularmente numérico.

En los ejemplos de programación, se usará pseudocódigo, lenguajes C y Fortran. Como se comentó con anterioridad, lenguajes populares como C++ o Java no ofrecen garantías numéricas suficientes (ver [208]).

3.1. Antes de la norma

3.1.1. La historia inicial

Desde los inicios del cómputo numérico, las limitaciones de la computadora fueron compensadas con el ingenio de muchos científicos. Se desarrollaron lo

¹Pese a que la mayoría no ha llegado a las aulas de las escuelas de computación.

que en su momento fueron trucos y que pronto pasaron a ser técnicas estándar, para compensar las deficiencias de exactitud, principalmente al tratar de escribir código que funcionara lo más parecido posible entre las diversas arquitecturas.

En todo tipo de problemas, sencillos o complejos, surgieron este tipo de ideas. Desde sumar hasta calcular eigenvectores de matrices muy grandes, ecuaciones diferenciales o simulaciones. Se dice que las primeras experiencias en la programación con números de punto flotante fueron de Wilkinson y su equipo. Cuando trabajaba en el National Physical Laboratory de Inglaterra, trabajó con Alan Turing en el diseño de la computadora ACE² al tiempo de estudiar (y desarrollar) análisis numérico³. Turing le solicitó en 1946 que programara un conjunto de rutinas para trabajar en APF con la versión 5 de la ACE, que inicialmente operaba en punto fijo. Estas fueron las primeras rutinas de punto flotante, posteriormente pulidas por otros miembros del equipo.

Según Wilkinson, Turing fue de los primeros que concientizaron en la importancia de la velocidad de los cálculos, obligando a pulir la APF, con lo que ganaron experiencia antes de tener construida la primera computadora. Allí progresó el análisis de error para la APF, en 1955-56, que posteriormente se publicaría en [395, 396], dos obras monumentales de lectura obligada para los interesados.

Un ejemplo notable de esfuerzo de cómputo se publicó en 1956, donde Brown, Carr y otros presentan [51], en el que analizan la solución de un problema de 28 ecuaciones simultáneas (parciales y diferenciales), integradas sobre un largo periodo de tiempo. Muchas medidas de seguridad y detección de errores fueron empleadas, algunas de ellas realizadas a mano durante las 70 horas de la ejecución del programa de 4000 líneas de código, en su primer corrida. Fue reiniciado 4 veces y el software detuvo la ejecución 10 veces de manera automática por errores de diverso tipo. Los autores recomiendan muchas medidas de prevención que faciliten la labor de los programadores.

Esa experiencia que ganaron cada uno de los equipos que construyeron las primeras computadoras, fue revelado cuando ya habían pasado las secuelas de la segunda guerra mundial y no se consideraban secretos militares. A finales de los 50 se comenzaron a conocer una batería amplia de técnicas de cálculo numérico.

Los lenguajes que indicaban la precisión en la declaración de variables, como Pascal, Ada o Turing ([177]), fueron una idea que no terminó bien. Por otra parte, quienes desarrollaba compiladores, es decir, traductores de lenguajes de alto nivel a ensamblador también desarrollaro mucha experiencia, que ala fecha sigue en aumento, por ejemplo [117], de Floyd en 1961. En esos años se daba mayor énfasis a reducir la cantidad de operaciones. En ese mismo sentido, los trabajos [12, 213, 332, 341, 386], de años previos, un tanto más orientados a la complación. Base de muchos de estos trabajos fue [214], de Kantorovich.

²Trabajó en 1948 en la Pilot-ACE, una vez que Turing abandonó NPL, dejando a la cabeza a Wilkinson. La Pilot-ACE era rápida y pequeña para su época, terminada en 1950. Trabajaba a 1MHz y podía resolver un sistema de 10 ecuaciones lineales en 1 segundo. No disponía de multiplicación, por lo que se usaba una rutina de sumas sucesivas.

³Al lado de expertos en el área como Goodwin, Fox, Olver y Clenshaw.

3.1.2. Primeras técnicas

Uno de los primeros ejemplos de método de cálculo para reducir problemas fue la diferencia de cantidades casi iguales, calculada como

$$a - b = \frac{a}{2} - b + \frac{a}{2} \quad (3.1)$$

para evitar la cancelación de cifras significativas.

Pequeños detalles como estos se fueron juntando, como la evaluación de $((0.5D0-x)+0.5D0)$ en vez de $(1.0D0-x)$ recomendado en las computadoras S/360 de IBM.

El ingenio dio pie a transformaciones de expresiones y reescritura de fórmulas para minimizar no sólo errores de redondeo, sino el antiguo overflow, que abortaba los cálculos. Actualmente, las precauciones de la norma permiten no necesitar de técnicas como la fórmula (3.1) que en su momento fueron de gran utilidad.

Un ejemplo que causa impresión es que en algunas computadoras era preferible escribir `if ( p*1 == 0 )` en vez de `if ( p==0 )` no sólo porque no se usaban números subnormales, sino por la deficiente representación de los números más pequeños⁴. Se podía escribir $p \times 1$ o bien $p/1$.

Existen trabajos para el análisis de errores automatizado, notablemente los de Webb Miller, de 1975 y 1978 [273, 274], en los que diseña rutinas en Fortran para perturbar sistemas de ecuaciones en busca de inestabilidades numéricas. Estos análisis se apoyaban en trabajos previos no automáticos, como [25, 179, 234, 235, 244, 372], que analizan la propagación de errores de redondeo y eficiencia general de algoritmos. Para ello se emplean herramientas de todo tipo, desde grafos hasta derivadas parciales, pasando por estadística y álgebra lineal.

Librerías matemáticas y otros recursos se hicieron del dominio público, ampliando las posibilidades de cómputo numérico confiable. En 1975, William James Cody presentó la FunPack en [70], una colección de funciones especiales programadas en Fortran, cuidadosamente diseñadas para ser robustas y eficientes en distintas plataformas.

Con la experiencia de FunPack, Cody publicó un excelente libro en 1980 [76] sobre la programación de funciones elementales, así como su artículo de lineamientos de programación [71], base para la librería ELEFUNT de funciones elementales. Estas y otras fuentes forman un aglutinado de ideas no sólo de programación, sino de métodos para probar lo correcto de las rutinas. Cody posteriormente publicó la versión para variables complejas CELEFUNT [73]. Ambas librerías están programadas en Fortran, disponibles en NetLib.

⁴Dekker y Bus comentaban en 1975 [60] que sería preferible comparar contra una cantidad muy pequeña, que es la idea actual de ε_M y justifican el uso de operaciones dependientes de la arquitectura lamentándose de que no exista algún acuerdo internacional al respecto.

3.1.3. Transformaciones libres de errores

Entre los ejemplos notables está el formato de *precisión múltiple* (consúltense [156, 184]) que consiste en emplear dos variables para representar un número en más alta precisión (el doble y el triple respectivamente, en esa época, 84 y 132 bits, para algunas computadoras⁵). En estos trabajos se extendía la mantisa sin aumentar el rango del exponente, ganando sólo precisión.

Ejemplo 33. *De manera ficticia, sin hacer referencia a ninguna arquitectura en particular, considérese el número real*

$$x = .11122233344455566677788899$$

requiere una mantisa de 26 decimales, pero puede representarse como la suma de $x_h + x_l$ como

$$\begin{aligned} x_h &= .11122233344455 \\ x_l &= .666777888999 \times 10^{-3} \end{aligned}$$

que requieren menos de 15 cada uno y $x_l < \text{ulp}(x_h)$.

Esta técnica, también llamada *precisión aumentada* en [47], es empleada en muchas librerías de funciones elementales para garantizar un error de .5 ulp con el doble de la precisión máxima. Aunque la norma no considera esta representación, sí garantiza que tanto x_l como x_h van a estar correctos hasta .5 ulp y con eso se garantiza que la suma (no evaluada) también lo estará. Técnicas que usan esta idea existen desde hace mucho tiempo, cada una con diversas propiedades.

La primera es la que publicó Dekker en 1971 en [89], para cierto tipo de APF de esa época, restringidas a una precisión par ($p = 2k$). Las técnicas, presentadas originalmente en Algol, aún son útiles.

Dekker indica que la suma (posiblemente inexacta)

$$s = \text{fl}(a + b)$$

(o bien, $s \approx a + b$) se calcula en dos partes como

$$s + t = a + b$$

y puede programarse en lenguaje C como el código 3.1. Este importante algoritmo (llamado **FastTwoSum**) será utilizado de nuevo más adelante.

⁵En [156] presentan la forma de realizar las 4 operaciones aritméticas básicas con registros de 48 bits. Dos registros se empleaban para un total de 96, y quitando los 12 de exponente quedaban 84 para la mantisa. Claro, las operaciones propuestas no incluían aún detalles posteriores como números subnormales o el sticky bit. En [184], Ikibe extiende la misma técnica empleada por Gregory para 84 bits, mostrando la forma de operar para mantisas de 132 bits ($36 + 48 + 48$), empleando 3 palabras en vez de 2. Este esquema se empleo en la Control Data 1604, del Centro de Cómputo de la Universidad de Texas.

Si el lector alega que esto no es numérico, sino simbólico, tiene toda la razón.

Código 3.1: Algoritmo **FastTwoSum** para duplicar la precisión al sumar.

```

1  double FastTwoSum(double a, double b, double *t)
2  {
3      double r, S;

5      if ( a<b ) {
6          r=a; a=b; b=r;
7      }
8      S  = a+b;
9      r  = S-a;
10     *t = b-r;
11     return S;
12 }
```

En este primer algoritmo, la suma $S+t$ es exacta y corresponde a $a + b$. A esto se le conoce como *transformación libre de error*. El mayor inconveniente del algoritmo FastTwoSum es que requiere determinar cuál de los dos números es mayor, por lo que su desempeño queda comprometido, pues interrumpe el pipeline de los procesadores modernos.

Ejemplo 34. La siguiente es la salida de un programa que emplea **FastTwoSum** para sumar 3 parejas de números en doble precisión. Se agrega el valor de r para comparar.

Salida del programa FastTwoSum

```
1/3 + 1/10 = 0.4333333333333335 -2.7755575615628914e-017
r=0.10000000000000003
```

```
sqrt(2) + 1 = 2.4142135623730949 + 2.2204460492503131e-016
r=0.99999999999999978
```

```
1.3333333333333333e+017 + 1e+017 = 2.3333333333333331e+017 + 16
r=99999999999999984
```

Dekker, presenta una extensión para cuando se dispone ya de cantidades con precisión múltiple $x_h + x_l$ y $y_h + y_l$. Esto se ilustra en el código 3.2, programado como macro sólo para ilustrar otra técnica posible⁶. En este caso, la suma ya no puede ser exacta, pero se garantiza un error máximo de

$$4.94 \times 10^{-32}(|x_h + x_l| + |y_h + y_l|).$$

Estas técnicas sólo cubren suma, resta y multiplicación (la división siempre es más difícil) y son empleadas en la IBM Accurate Mathematical Library de 2001. Estos trabajos fueron iniciados más de 10 años antes, como presentan

⁶Dekker presenta un algoritmo para multiplicar dos números x y y ya separados, donde el error queda acotado por $|xy - (r_1 + r_2)| \leq \frac{7}{2}\beta^{e_{\min} - p + 1}$.

Código 3.2: Macro para duplicar la precisión al sumar cantidades con precisión ya duplicada.

```

1 #define SUMA2(x,xx,y,yy,z,zz,r,s) \
2 { double r, s; \
3   r=(x)+(y); \
4   s=(ABS(x)>ABS(y)) ? (((((x)-r)+(y))+(yy))+(xx)) : \
5                       (((((y)-r)+(x))+(xx))+(yy)); \
6   z=r+s; zz=(r-z)+s; \
7 }
```

Código 3.3: Macro mejorada para duplicar la precisión al sumar.

```

1 #define SUMAMEJ(x,xx,y,yy,z,zz,r,s) \
2 { extended r,q,s; \
3   r=(x)+(y); \
4   q=(x)-r; \
5   s=((((q+(y))+(x)-(q+r)))+(xx))+(yy) \
6   z=r+s; zz=(r-z)+s; \
7 }
```

Gal y Bachelis en [16], garantizando 1 ulp correctamente redondeado con una seguridad del 99.7% en el caso de las funciones elementales.

Hay otras formas de garantizar .5 ulp con el doble de la precisión máxima en la suma sin suponer que el error de redondeo es menor que .5 ulp ni que la precisión es par, según presentó Linnainmaa, en 1981 [245], generalizando el resultado de Dekker [89]. En este caso, la transformación libre de error para la suma se muestra en el código 3.3.

El algoritmo del código 3.3 es similar al de Dekker, pero con garantías teóricas adicionales, en particular porque se apoya en la primera norma de IEEE, que en ese momento estaba en discusión.

Estas técnicas de programación se extendieron para incluir división y multiplicación. Para ello, desde el inicio había que separar cada variable en dos partes, lo que ya había mostrado Dekker en [89] como $x = x_h + x_l$, presentando una técnica propia, variante de una anterior de Veltkamp en 1968 en reportes técnicos para programación en Algol. Linnainmaa lo formaliza y prueba que la técnica de Dekker es más general de lo pensado. Las operaciones son

$$\begin{aligned}
 t &= \text{fl}(x(1 + \beta^{p-k})) \\
 x_h &= \text{fl}(t - (t - x)) \\
 x_l &= \text{fl}(x - x_h)
 \end{aligned} \tag{3.2}$$

donde p es la precisión, como de costumbre, k es un entero con $1 \leq k < p$, x_h tiene k dígitos y x_l los restantes $p - k$. Lo anterior suponiendo que el redondeo es al más cercano. En su artículo, Linnainmaa presenta rutinas para multiplicar variables ya separadas.

Por sugerencia de Kahan.

Código 3.4: Algoritmo **TwoSum** de Knuth.

```

1  double TwoSum(double x, double y, double *t)
2  {
3      double S, a, b, da, db;

5      S = x+y;
6      a = S - y;
7      b = S - x;
8      da = x - a;
9      db = x - b;
10     *t = da + db;
11 }

```

Como es de esperarse, estas técnicas hacían poco a poco los códigos un tanto menos naturales, de lectura muy lenta (y a veces incomprensibles) para todos los programadores no familiarizados con la aritmética del punto flotante.

Sin considerar esta precisión extendida usando dos variables, Donald Knuth presenta otro algoritmo que no requiere conocer cuál operando es mayor, en el segundo tomo de su famosa serie de libros [219]. El algoritmo requiere 6 operaciones en vez de 3, pero el ahorro por no preguntar `if ( x<y )` suele ser mayor que 3 operaciones en los procesadores modernos. Este resultado también lo presentó Moller en [276] y será mencionado como algoritmo **TwoSum** (código 3.4).

El algoritmo sólo supone que no ocurre overflow o underflow, pero Boldo probó en [39] que el underflow no perjudica el resultado. En [223], los autores muestran que este algoritmo es óptimo en el sentido de las operaciones y del aprovechamiento de las arquitecturas modernas.

El comportamiento de estas funciones (**Fast2Sum** y **TwoSum**) pueden dar resultados incorrectos (en el valor del término correctivo t) en el caso de que el procesador realice doble redondeo. Por ejemplo, para el algoritmo **Fast2Sum**, si la suma es $a + b = S + t$, el valor intermedio de r (ver código 3.1) es correcto, pero t será exacto si $S = a + b$ se calcula sin errores. De otra forma, sólo se puede garantizar que $t = \text{fl}(a + b - S)$. Para más detalles, ver [283], donde además se analiza la separación de un valor en dos variables (3.2) ante el doble redondeo.

Los anteriores algoritmos suponen el caso de redondeo al más cercano, que es el default para la norma numérica. Si esto no se cumple, no hay garantía de los resultados. El caso de disponer de un ambiente numérico que respeta la norma 754 de IEEE, en particular el redondeo fiel, se trata en la sección 3.3.5.

3.1.4. Cambio de fórmulas

El descubrimiento de la *cancelación catastrófica* y otros fenómenos desagradables en la APF, motivó el replanteamiento de fórmulas comunes como la del



Donald Ervin Knuth (1938 -), científico norteamericano. Uno de los mayores expertos en análisis de algoritmos, creador del lenguaje TeX y de su serie multicitada “El Arte de Programar Computadoras”.

discriminante de la ecuación cuadrática

$$\frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

para evitar el problema de que la magnitud de b^2 sea parecida a la de $4ac$, o que $b^2 \gg 4ac$, ocasionando la diferencia de cantidades casi iguales en el numerador. En la sección 6.5 se analiza este caso detalladamente. A continuación se presentan otros ejemplos.

Ejemplo 35. *La fórmula de Herón para calcular el área de un triángulo a partir de sus lados es*

$$A = \sqrt{s(s-a)(s-b)(s-c)} \quad (3.3)$$

donde $s = (a + b + c)/2$ es el semiperímetro. Si uno de los lados es sumamente pequeño, ocurrirá una cancelación que puede ocasionar un error relativo grande. En 1986 Kahan propuso reorganizar las aristas para que $a \geq b \geq c$ y emplear la fórmula

$$A = \frac{1}{4} \sqrt{(a + (b + c))(c - (a - b))(c + (a - b))(a + (b - c))} \quad (3.4)$$

para garantizar un error más pequeño. El lector ya debe ser sensible a la importancia de respetar los paréntesis.

No solo es posible cuidar problemas de redondeo como la cancelación de cifras en el discriminante o (3.4), en algunos casos también es posible reducir problemas de estabilidad.

Muchas otras operaciones más complicadas que estas fueron desarrolladas, así como sugerencias de cómo evaluar o reescribir expresiones matemáticas. La tabla 3.1 muestra algunos ejemplos famosos⁷. El problema de redondeo o cancelación no ocurre en todo el dominio, sino en cierto punto crítico, cerca de una asíntota o por cancelación catastrófica. Es de esperarse que en este momento el lector pueda determinarlos.

En este caso, la tabla 3.1 sólo muestra algunos ejemplos de manipulaciones algebraicas que se ha considerado que son numéricamente más seguros. De algunas de estas hay más expresiones posibles, códigos eficientes o rutinas disponibles en la librería del compilador por lo que poco a poco se usan menos. Como dice el pie de la tabla, el uso de la nueva fórmula se sugiere cuando el argumento se acerca a cierto valor crítico.

Ejemplo 36. *Un caso donde el código estable puede ser tan simple como eficiente es la versión de Kahan para $e^x - 1$, mostrada en el código 3.5.*

Este procedimiento simple tiene gran cuidado y garantiza un resultado correcto. La división reduce los errores generados por $e^x - 1$ y $\ln(e^x - 1)$. El punto en las constantes 1 sirve para forzar al compilador a que las considere como flotantes desde el momento de compilar.

⁷Busque el lector dónde fallan las expresiones de la primera columna.

Para calcular . . . es mejor escribir . . .	
$1 - \cos(x)$	$2 \operatorname{sen}(x/2)^2$
$\frac{1}{\cos(1-x)}$	$\frac{2}{\operatorname{sen} \sqrt{x/2}}$
$e^x - 1$	$\tanh(x/2)(e^x + 1)$
$\ln(x + 1)$	$2 \arctan(\frac{x}{x+2})$
$\tanh(x)$	$\frac{e^{2x} - 1}{e^{2x} + 2}$
$\tanh^{-1}(x)$	$\ln(\frac{2x}{1-x} + 1)/2$
$\sqrt{1-x} - 1$	$\frac{x}{\sqrt{1-x} + 1}$
$1 - \sqrt{1-x}$	$\frac{x}{\sqrt{1-x} + 1}$
$\sqrt{x^2 + 1} - 1$	$\frac{x^2}{\sqrt{x^2 + 1} + 1}$
$\sqrt{x + \delta} - \sqrt{x}$	$\frac{\delta}{\sqrt{x + \delta} + \sqrt{x}}$

Tabla 3.1: Ejemplos de de cambio de fórmulas para evaluar con seguridad. Estas deben usarse con cuidado pues es sólo para la cercanía a ciertos valores críticos.

Código 3.5: Manera correcta de calcular $e^x - 1$.

```

1 double expm1(double x)
2 {
3     double u = exp(x);
4     if (u == 1.)
5         return 1.;
6     return (u-1.)/ln(u);
7 }
```

3.1.5. Hipotenusa

La hipotenusa de un triángulo se calcula con

$$c = \sqrt{a^2 + b^2} \quad (3.5)$$

donde a y b son los catetos. Es un caso particular⁸ de la *norma* de un vector en $\mathbb{R}^n$ para $n = 2$. Por ellos se dice a veces hipotenusa y en otras norma en 2D.

Otra aplicación frecuente es el *módulo* del número complejo $z = a + ib$, definido como $|z| = z\bar{z}$, con $\bar{z} = a - ib$, llamado el *conjugado complejo* de z . El número imaginario i se define como de costumbre: $i^2 = -1$. El módulo se calcula entonces como $|z| = a^2 + b^2$.

Un intento directo e inocente de programar la hipotenusa es la función

```
double Hipotenusa(double a, double b)
{
    return sqrt(a*a+b*b);
}
```

Los valores intermedios son positivos y pueden ocasionar underflow u overflow, ya sea a^2 o b^2 y la suma puede ocasionar overflow. En caso de no ocurrir overflows, la fórmula directa (3.5) es bastante precisa, con un error relativo entre 2^{1-p} y 2^{-2p} .

La solución típica para evitar overflow es *escalar* con:

$$c = s \sqrt{\left(\frac{a}{s}\right)^2 + \left(\frac{b}{s}\right)^2} \quad (3.6)$$

y el valor de s se selecciona de tal forma que se minimicen errores de redondeo. Lo tradicional es hacer $s = \max(|a|, |b|)$, sin importar el valor y la fórmula queda

$$c = s \sqrt{1 + \left(\frac{\min(|a|, |b|)}{s}\right)^2}$$

que es equivalente al planteamiento de Friedman en [124] de 1967.

La otra opción es imponer a s el mínimo valor para el que $\left(\frac{a}{s}\right)^2 + \left(\frac{b}{s}\right)^2$ no causa overflow y que sea potencia de 2, idea de Kahan no publicada y que aparece también en [269]. Si L es el mayor número en el formato usado, entonces el valor mínimo de s se aproxima buscando la siguiente potencia de 2 mayor que

$$\begin{aligned} s &> \frac{1}{L} \sqrt{a^2 + b^2} \\ &= \sqrt{\left(\frac{a}{L}\right)^2 + \left(\frac{b}{L}\right)^2} \end{aligned}$$

⁸Y también de la Ley de los Cosenos $c = a^2 + b^2 - 2ab \cos C$ para triángulos no rectángulos.

y se recalcula (3.6). Este esquema sigue siendo un tanto inocente pues evita el overflow pero promueve el underflow, además de ser demasiados cálculos para encontrar c .

Como puede verse, no es tan sencillo calcular una hipotenusa. Por fortuna, casi todos los compiladores disponen de una función de librería `hypot` que se espera que esté bien optimizada por un equipo de expertos.

Lo que se busca es un valor s para poder calcular $(sa)^2 + (sb)^2$, con riesgos reducidos por el escalamiento. Si ambos a y b son muy grandes s será un valor pequeño y viceversa. Pero no debe ser tan pequeño o grande que ocasione problemas de precisión y mucho menos overflow. Además, si s es una potencia de β no se ocasionarán problemas de redondeo.

Los NPF máximo y mínimo son

$$\begin{aligned} \text{Mayor: } R &= \beta^{e_{\text{máx}}}(1 - \beta^{-p}) \\ \text{Menor: } r &= \beta^{e_{\text{mín}}}. \end{aligned} \quad (3.7)$$

El menor número es realmente el subnormal β^{e+1-p} pero puede ocasionar problemas de exactitud, por lo que se prefiere aquí el mínimo normal.

En 1978, James Blue, de los Laboratorios Bell, publicó [32], donde presenta una forma que cuida estos aspectos para evitar problemas de overflow o underflow. Lo que hace es calcular las constantes t , T , s y S a partir de las definiciones (3.7) de la siguiente manera.

$$\begin{aligned} t &= \beta^{\lceil (e_{\text{mín}}-1)/2 \rceil} \approx 1.491 \times 10^{-154} \\ T &= \beta^{\lfloor (e_{\text{máx}}-p+1)/2 \rfloor} \approx 1.998 \times 10^{146} \\ s &= \beta^{\lfloor (e_{\text{mín}}-1)/2 \rfloor} \approx 7.458 \times 10^{-155} \\ S &= \beta^{\lceil (e_{\text{máx}}-p+1)/2 \rceil} \approx 1.998 \times 10^{146} \end{aligned}$$

Los valores aproximados corresponden al formato de doble precisión. Estos darían problemas de overflow o underflow al elevarse al cuadrado. Si a o b se salen del rango $[t, T]$, se requiere escalar, cada uno independientemente. Si alguno queda en ese rango, se puede calcular su cuadrado sin problemas.

El procedimiento escala cada valor independientemente. En el segmento de código 3.6 se usan 3 acumuladores: `cm` para escalar underflows, `c` cuando no se requiere escalar y `cM` para minimizar overflows. En su artículo, Blue demuestra que el algoritmo anterior evita underflows y que el overflow ocurrirá sólo cuando $\sqrt{a^2 + b^2} > R$.

Otro interesante procedimiento se debe a Cleve Moler y Donald Morrison, presentado en [275]. El procedimiento es iterativo, sin necesitar calcular la raíz cuadrada. En esencia es el método de aproximación de raíces de Halley, que se estudia con más detalle en el capítulo 5 de la segunda parte.

La $\sqrt{}$ es más costosa que la división.

En el código 3.7, que presenta el método de Moler–Morrison, cada iteración va aumentando M y disminuyendo m . El método de Halley tiene convergencia

Código 3.6: Método de Blue para calcular la hipotenusa.

```

1  double HipotenusaBlue(double a, double b)
2  {
3      double c, cm, cM, xm, xM;

4
5      a = fabs(a);
6      b = fabs(b);
7      c = cm = cM = 0.0;
8      if ( a>T )
9          cM = (a/S)*(a/S);
10     else if ( a<t )
11         cm = (a/s)*(a/s);
12     else
13         c = a*a;
14     if ( b>T )
15         cM += (b/S)*(b/S);
16     else if ( b<t )
17         cm += (b/s)*(b/s);
18     else
19         c += b*b;
20     if ( cM>0.0 ) {
21         if ( sqrt(cM) > R/S )
22             return R*R; /* Overflow inevitable */
23     if ( c>0.0 ) {
24         xm = min(sqrt(c), S*sqrt(cM));
25         xM = max(sqrt(c), S*sqrt(cM));
26     } else
27         c = S*sqrt(cM);
28     } else if ( cm > 0 )
29         if ( c>0.0 ) {
30             xm = min(sqrt(c), s*sqrt(cm));
31             xM = max(sqrt(c), s*sqrt(cm));
32         } else
33             c = s*sqrt(cm);
34     if ( xm < sqrt(eps)*xM )
35         c = xM;
36     else
37         c = xM*sqrt( 1 + (xm/xM)*(xm/xM) );
38     return c;
39 }

```

Código 3.7: Método Moler-Morrison para calcular la hipotenusa. El resultado quedará en la variable p.

```

1  M=max(|a|,|b|);
2  m=min(|a|,|b|);
3  while ( fabs(m) > eps ) {
4      r = M/m;
5      r *= r;
6      s = r/(r+4);
7      M = M + 2*s*M;
8      m = s*m;
9  }

```

cúbica, lo que significa que cada iteración triplica la cantidad de cifras correctas. Es más lento que el de Blue (por ser iterativo y no directo) pero muy preciso, pues con 2 iteraciones se obtienen hasta 6 decimales correctos y con 3 ya son 20 decimales o pocos menos. Por ello, 4 iteraciones fijas son suficientes para garantizar un resultado correcto en doble precisión.

Según los autores de [275], otras ventajas del código 3.7 son:

1. No ocurre overflow o underflow si el resultado es un número normal en $\mathbb{F}$,
2. Se puede usar para calcular $\sqrt{a^2 - b^2}$ cambiando la instrucción `r *= r` por `r *= -r`.
3. Como el algoritmo calcula $M\sqrt{1+r}$, se puede modificar para calcular la raíz cuadrada de un número.

3.1.6. División compleja

Otro ejemplo muy ilustrativo es el cociente de dos números complejos, pues existen muchos problemas potenciales. Sean los números complejos $z = a + ib$ y $w = c + id$. El cociente complejo $z/w = x + iy$ se define como

$$\begin{aligned}\frac{z}{w} &= \frac{a + ib}{c + id} \\ &= \frac{ac + bd}{c^2 + d^2} + i \frac{bc - ad}{c^2 + d^2}\end{aligned}$$

y puede ocurrir overflow si $|w| > \sqrt{\beta} \beta^{\frac{\varepsilon_{\max}}{2}}$, con β la base del sistema numérico $\mathbb{F}$ empleado.

Con $|w|$ debe entenderse el módulo del número complejo.

Dos conceptos de error relativo

Como los números complejos se forman con dos cantidades, el concepto de error, particularmente el relativo debe reconsiderarse. Sea $z = a + ib$ un número complejo y $\hat{z} = \text{fl}(a) + i \text{fl}(b)$ y su aproximación. Las dos opciones que se pueden emplear son las expresiones

$$\left(\frac{|\text{fl}(a) - a|}{|a|}, \frac{|\text{fl}(b) - b|}{|b|} \right) \quad (3.8)$$

$$\frac{|\hat{z} - z|}{|z|} \quad (3.9)$$

donde la primera es por los errores relativos de las componentes y la segunda considera el error relativo del módulo (o norma). Tener una cota para (3.8) implica tener una cota para (3.9). La implicación en el otro sentido es falsa, es decir, se puede tener $|\hat{z} - z| < \varepsilon_M |z|$ y ocurrir que $\frac{|\text{fl}(a) - a|}{|a|}$ sea arbitrariamente grande. El lector interesado puede consultar [168, 282] para detalles adicionales o la demostración de este hecho.

A partir de estas definiciones se puede decidir qué criterio emplear para evaluar la eficiencia de las operaciones con números complejos en $\mathbb{F}$.

Método de Smith

En 1962, Robert L. Smith publicó un método [350] para escalar la división y evitar el overflow, que procede según las ecuaciones

$$\frac{a+ib}{c+id} = \begin{cases} \frac{a+b(d/c)}{c+d(d/c)} + i \frac{b-a(d/c)}{c+d(d/c)} & \text{si } |d| < |c| \\ \frac{a(c/d)+b}{c(c/d)+d} + i \frac{b(c/d)-a}{c(c/d)+d} & \text{si } |d| \geq |c|. \end{cases}$$

Aún con los cuidados de Smith, tanto overflow como underflow pueden ocurrir cerca de los límites de los NPF, además de requerir una división extra para el factor d/c o c/d . Esta técnica se ha utilizado desde entonces, incluso para aritmética de precisión arbitraria, como en [349] en 1998. Por otra parte, averiguar en qué caso se está tiene el costo de extra de interrumpir el pipeline de los procesadores.

Método de Stewart

Los problemas remanentes fueron analizados por Stewart en 1985, en [353], logrando evitar la mayoría, al hacer operaciones cuidadosas. Stewart planteó el algoritmo que se presenta en el código 3.8. El resultado queda en las variables x (la parte real) y y (la parte imaginaria).

Lo que hace es calcular con mucho cuidado expresiones como $b(d/c)$ asociando de la manera más segura posible. Se logran eliminar mayor cantidad de problemas, pero aún quedan algunos. Siendo una variante del cálculo de Smith, aún puede ocurrir overflow o underflow, además de la posibilidad de que el cálculo resulte en NaN.

The Art of Scientific Computing.

En el famoso libro “Numerical Recipes in C”, [311], se presentan sugerencias generales para la multiplicación, módulo, división y raíz cuadrada de números complejos, aunque mucho menos detalladas y referenciadas que aquí. Al abarcar tanto, este famoso libro⁹ ha sido criticado por diversos expertos en algunas áreas por la falta de profundidad. De cualquier manera, es un buen compendio de técnicas tanto de matemáticas como de programación.

* * *

Otros ejemplos interesantes pueden consultarse en [317], donde Reid recomienda una forma de programar rutinas básicas para manejo portable de números de punto flotante, basándose en las definiciones de normativas de IFIP en [118]. Funciones no tan básicas pero igualmente interesantes aparecieron en [53], donde Brown y Feldman proponen parámetros básicos para los

⁹La versión inicial fue en Fortran y resultó de gran apoyo para programadores inexpertos. Actualmente no es complicado encontrar serias críticas a los códigos que lo acompañan.

Código 3.8: Método Stewart para dividir números complejos.

```

1  F = false;
2  if ( |d|>=|c| ) {
3      Intercambiar(c,d);
4      Intercambiar(a,b);
5      F = true;
6  }
7  s = 1/c;
8  t = 1/(c+d*(d*s));
9  if ( |d|>=|s| )
10     d = s;
11  if ( |b|>=|s| )
12     x = t*(a + s*(b*d));
13  else if ( |b|>=|d| )
14     x = t*(a + b*(s*d));
15  else
16     x = t*(a + d*(s*b));
17  if ( |a|>=|s| )
18     y = t*(b - s*(a*d));
19  else if ( |a|>=|d| )
20     y = t*(b - a*(s*d));
21  else
22     y = t*(b - d*(s*a));
23  if ( F == true )
24     b = -b;

```

NPF, en la forma de funciones básicas, ejemplificando su uso con el logaritmo, exponencial y escalamiento de vectores.

Se requiere mucho espacio para poder presentar una colección que intente ser completa de todas las técnicas que durante décadas los programadores han ideado para reducir problemas numéricos, pero es mejor comprender los fundamentos para poder entenderlas y generar las propias.

3.2. Excepciones numéricas

Este tema compite fuertemente con el de aritmética de punto flotante entre los menos entendidos por los programadores. La falta de una definición exacta, objetiva y formal es el inicio de los malos entendidos. Su conceptualización maduró poco a poco, a la par del diseño de gran número de lenguajes.

El manejo adecuado de excepciones, en general, es fundamental para poder desarrollar sistemas confiables y que entreguen resultados correctos. En el caso de sistemas numéricos, se espera al menos que los resultados estén en márgenes de error dentro de las estimaciones teóricas. En los inicios de la computación, las excepciones fueron tratadas como errores (fatales), pero actualmente es claro que deben ser parte integral en casi todo sistema serio. Esta sección motiva el epígrafe al inicio del capítulo.

3.2.1. Los primeros pasos

El lector no interesado en los aspectos históricos puede pasarse directamente a la sección 3.2.2, página 102.

Las excepciones, llamadas primero condiciones excepcionales, fueron conocidas desde los inicios de la programación de computadoras y lo primero que se hizo ante una situación inesperada (o indeseable) fue suspender la ejecución del proceso. Posiblemente las referencias que más han sido citadas son [144–146], de Goodenough, aunque la referencia más antigua encontrada por el autor es [293] de 1968, sobre el control de excepciones en lenguaje PL/1. Goodenough define las excepciones como operaciones que requieren de la atención del invocador, lo que da idea de que su manejo estaba lejos de automatizarse y mucho menos de ser transparente. En todos los casos, la definición es poco precisa, pudiéndose referir a eventos de todo tipo que se salen del flujo normal del programa.

Programación estructurada vs. saltos

Es de notarse que en esta época (en la década de 1970) surgió el paradigma de la programación estructurada, cuya prioridad era la escritura de programas muy ordenados y que eliminaran el uso de saltos, como el `goto` de diversos lenguajes, instrucción de lo más común en esa época. Esto estaba encaminado a codificar de una manera que fuera fácil de leer para los programadores e incluso se llegó a escribir el Teorema del Programa Estructurado, donde se demuestra que todo `goto` puede ser sustituido por estructuras de control adecuadas, aunque a costa de banderas extra y líneas de código para evaluarlas (véase [33]).



Edsger Wybe Dijkstra (1930 - 2002), programador holandés. Fundó muchas de las bases de la programación de computadoras y definió muchos de los términos que actualmente se usan. Muy larga la lista de logros de este pionero de la programación.

En 1968, el ya entonces muy respetado científico Dijkstra, escribió un artículo corto [95] cuyo título fue decidido por el editor para que saliera como una comunicación rápida. Fue titulado “Go To Statement Considered Harmful” (sentencia `goto` considerada peligrosa) por el editor (Niklaus Wirth), tergiversando un poco la intención real de Dijkstra¹⁰. En efecto, Dijkstra sugiere abolir la sentencia `goto` de los lenguajes de alto nivel, esto en una época en que había ensamblador y lenguajes de alto nivel, sin puntos intermedios. Como resultado, se satanizó la sentencia `goto` y se comenzaron a diseñar lenguajes con estructuras de control que hicieran innecesario el uso del salto. Pese a todos estos esfuerzos, la sentencia siguió apareciendo en todos los lenguajes procedimentales.

Se había llegado a una época que esperaba que computadoras más poderosas facilitaran la labor de los programadores, pero lo que llegó fue la *crisis de los años sesentas*, donde la programación era cada vez más complicada, contrario a lo esperado. ¿Por qué era así? Muchos estuvieron felices de haber encontrado el *capo* de las sentencias culpables de tantos problemas, pero sólo fue un chivo expiatorio. Para 1970, muchos programadores creían que era más eficiente el uso de `goto` en vez de una llamada a un procedimiento o función tan sólo por el exceso de operaciones que involucraba manejar la pila del proceso (por la

¹⁰Ese artículo ha sido citado en más de 260 trabajos diversos.

Al igual que las variables globales, los apuntadores, la aritmética de punto flotante, interrupciones, semáforos y otros tantos conceptos con doble filo.

aritmética de apuntadores). Steele deja claro que esto es falso en [351] y que en general se debe a un deficiente diseño de lenguaje, las llamadas a subrutinas no tienen por que ser más lentas.

Donald Knuth refiere que Dijkstra, en un comunicado personal de 1973, le comentó que sus pensamientos no eran dogmáticos con respecto al uso del `goto` y que le incomodaba la manera como otros lo estaban convirtiendo en un asunto casi religioso. En respuesta a esto y con la intención de llamar la atención lo suficiente (cosa que logró), Knuth escribió [218], donde presenta pros y contras de la sentencia en cuestión, además de analizar con gran detalle las referencias sobre el tema (que impresiona por los esfuerzos fundamentalistas de eliminar esa sentencia de los lenguajes de programación). Lo que hace Knuth en su artículo es mostrar muchos ejemplos donde el uso de `goto` perjudica el desempeño de una rutina y otros tantos donde su uso mejora tal desempeño. Así, deja al lector la conclusión.

A favor y en contra de las excepciones

Estando las cosas de esa manera, es de esperarse que mecanismos que manejaran excepciones donde se pretende dar salidas alternas a una función, no fueran bien vistos por la comunidad de programadores, pues rompen abruptamente con el flujo (normal) de un programa y, lo que es peor, tienen efectos colaterales. Si el `goto` interrumpe la secuencia lógica de instrucciones, la excepción, además, levanta banderas o envía señales que tal vez no sean empleadas.

Fueron llamados códigos spaghetti. Luego surgirían los códigos ravioli y lasagna.

Sí hubo voces que se pronunciaron a favor de los saltos para permitir al programa recuperarse de la manera más honerosa posible, como [169], de Hill, donde analiza qué hacer con y sin efectos colaterales, pronunciándose por saltar a una rutina que maneje cada problema. En un intento por reunificar criterios, Goodenough, Levin y otros autores muestran diversas maneras de estructurar las excepciones en [146, 241, 247, 400] y mantener ordenados los códigos. Sus sugerencias fueron seguidas de múltiples formas por los diseñadores de lenguajes.

Para inicios de 1980 ya había muchos formalismos que trataban a las excepciones como una abstracción que debía axiomatizarse. Ejemplo de esto son los trabajos [251, 399], pero fallan al diferenciar errores de excepciones. Además, la variedad de características entre distintos lenguajes hace muy difícil una axiomatización uniforme y, por tanto, útil en la práctica.

El caso contrario queda bien ejemplificado por el reporte técnico [29], de 1983, en el que se Black se manifiesta abiertamente en contra de la existencia de excepciones en los lenguajes de programación, además de ofrecer un análisis detallado (excelente resumen) de las diversas definiciones de excepción en la literatura de la época y del proceso histórico que el manejo de excepciones vivió desde los inicios de la computación hasta 1983.

“Los errores deben ser corregidos, no manejados.”

Black alega¹¹, elocuéntemente, que una pila vacía, una archivo no encontrado

¹¹El análisis tan detallado de Black pareciera haber llegado a oídos sordos, por la cantidad

o una división entre cero son situaciones donde los argumentos (el dominio) de la operación (función) no debe verse como un conjunto incompleto, sino que lo que se requiere es completar el conjunto posible de resultados (rango). Así, una división entre cero no dará problemas si hay un valor numérico asociado (el ∞ de la norma); la función `Pop` no tendrá problemas si se considera un elemento llamado `PILA_VACIA`.

El concepto madura

Programas cada vez más sofisticados se desarrollaron, con un uso aún diverso de formas de controlar las excepciones¹². Uno de los trabajos más referenciados, que ya consideraban las implicaciones de la norma numérica es [162], de Hauser, publicado en 1996, donde el autor recomienda dar soporte a todas las características requeridas por la norma. Este detallado artículo (36 páginas) pone de ejemplo a la librería `LINPACK`, cuyos excelentes resultados son, en parte, debidos al buen manejo de excepciones como lo manifiesta Demmel en [94]. Comentarios similares aparecen en [178], pero haciendo uso de un lenguaje similar a `CLU`, ya en desuso, diseñado por Liskov y sus estudiantes, dando el crédito de las ideas a Goodenough.

El lector interesado no debe dejar de revisar [56, 286, 331, 375, 387], excelentes referencias para conceptos generales y puntos de vista diversos sobre los problemas que enfrenta el manejo de excepciones. En estos trabajos se ve cómo maduraron estos conceptos y otros dentro de la ingeniería de software, quedando claro que las excepciones no eran un aspecto secundario, sino primario en el desarrollo de sistemas, especialmente los de gran tamaño.

Hay toda una corriente de investigadores de ingeniería de software que proponen separar, de diversas maneras el manejo de excepciones del flujo normal del sistema, como se analiza en la introducción de [342]¹³, donde Shah incluso propone el nuevo perfil profesional de *Ingeniero en Excepciones*, como especialidad dentro del desarrollo de sistemas. En este trabajo, queda claro que el típico desarrollador profesional tiende a dedicarle mucho esfuerzo al flujo normal del sistema, a cumplir con los requisitos especificados, mientras que dedica poco tiempo (con cierto desden) a las maneras como su programa se recuperará de cada fallo posible.

Problemas numéricos particulares resueltos eficientemente con excepciones son [181] de Hull y [198] de Kahan. Las notas de clase [206] son otro excelente ejemplo de manejo de la operación ∞/∞ , donde de manera muy elegante se continúan los cálculos de eigenvectores ortogonales substituyendo $\infty/\infty = 1$.

de recursos que aparecieron (y permanecen) en los lenguajes de programación, pero en la actualidad, muchos de los elementos de los lenguajes que respetan la norma numérica lo hacen en el sentido de [29], aunque no todos son de su autoría. Por otra parte, conceptos como los de *manejo de catástrofes*, nunca se han empleado.

¹²El cómputo en paralelo ya se utilizaba y las excepciones ahí requerían cuidados especiales, como se ve en [186] o más recientemente en [226].

¹³Presentado en el 4th International Workshop on Exception Handling.

Kahan presenta, en caso de no haber presubstitución, dos alternativas obvias, dos alternativas poco obvias y una nada obvia, excelentes ejemplos de análisis, astucia y buen empleo del ambiente numérico. Lectura obligada.

Para lenguajes particulares, ir a la definición (ANSI) de los lenguajes es de gran ayuda, además de referencias específicas. Para lenguaje C, [131, 286], además de [321] donde Roberts presenta una manera de incluir bloques try-catch para C, utilizando macros.

Para C++, está [336] e indudablemente el libro [220] de Koenig y Stroustrup. Además, [183, 388] para Fortran, [41, 322] para Ada y [238, 294, 402] para Java.

Excepciones y la norma numérica

El manejo de excepciones propuesto en la norma 754 de IEEE, en su versión de 1985, fue resultado de la experiencia ganada en las primeras décadas de la computación. Dentro de los conceptos fundamentales está la propagación de excepciones, que se refiere continuar los cálculos con las acciones adecuadas.

En 1979 ya se estaba trabajando en la propuesta de la norma, como puede verse en [210], donde comentan que las ideas iniciales partieron de la propuesta de Intel de 1977. Kahan comenta que el estándar propuesto no es un conjunto de conceptos nuevos y radicales, sino la integración de buenas ideas de muchas personas. En [210] ya se presentan los elementos principales de la norma que se aprobó en 1985, particularmente las excepciones recomendadas. Además, presentan ejemplos de códigos para resolver problemas sencillos (y unos no tanto) que hacen uso correcto de las recomendaciones de la norma.

El primer trabajo que critica la propuesta el tema es [101], también de 1979 en el que los autores (Eggers, Leonard y Payne) presentan una alternativa sencilla a la propuesta de norma numérica de Kahan y su equipo, demasiado compleja según ellos. Argumentan que la mayor diferencia es en el manejo de excepciones, pero además se niegan a la idea de los números desnormalizados, que dicen es la primera área de desacuerdo, entre otras cosas, por tratarse de aritmética de punto fijo. Aunque aceptan que para secuencias de sumas y restas es adecuado, lo consideran innecesario y estorboso cuando hay productos y divisiones. El resto del artículo abordan el problema de manejar underflow, overflow y problemas de memoria, pero no tocan operaciones inválidas y las otras excepciones.

Desmoralizados, en vez de desnormalizados, dice cómica e irónicamente el resumen en la página de la ACM (Digital Library).

En 1988, Hull y otros [178] presentaron lineamientos generales para el manejo de excepciones, que reducen a un mínimo las excepciones predefinidas (incluyendo algunas no numéricas), sin limitar las que el programador pudiera agregar¹⁴.

En su publicación de 1996, John Hauser [162] expone con gran claridad las maneras como los programadores pagan la falta de cuidado de los fabricantes de procesadores y compiladores en cuanto a las características de un manejo de

¹⁴Además presentan excepciones no numéricas como el fin de archivo inesperado, que no se tratarán en este libro.

excepciones efectivo. Con su análisis, respalda como mejor solución los lineamientos de la norma 754, así como de LIA de IEC (véase la sección 2.1).

Hauser sintetizó claramente las 4 posibles soluciones ante cualquier caso de underflow/overflow:

1. **Fuerza bruta:** reevaluar con un formato de mayor precisión o precisión múltiple.
2. **Escalando** la operación, por ejemplo dividiendo entre una constante y al final multiplicar para reescalar de nuevo. En ocasiones se pueden hacer varios intentos de escala. Los mejores factores son las potencias de β , ya que solo alteran el exponente, sin tocar la mantisa, excepto en los subnormales.
3. **Presubstituir** con ∞ o 0. Si en una operación como $10 + 1/(x+y)$, si $x+y = \infty$, entonces se puede sustituir sin riesgos a $x+y$ por el mayor número posible en su formato y terminar la operación, ya que aún así el redondeo resultará en 10 y no se perderá tiempo en la excepción de overflow, sólo levantando la bandera de inexacto.
4. Si es underflow, usar **underflow gradual**. Esto es respetado por la mayoría de los procesadores.

Todo programador debe ver las excepciones como una oportunidad para hacer más cálculos, no para detenerlos. Como dice Kahan:

“Las excepciones son errores sólo cuando se manejan mal.”

Las excepciones son una de las 3 formas de modificar la ejecución de un programa (las otras dos son las interrupciones de software y hardware), lo que puede ocurrir por un fallo (como división entre cero), una trampa (como overflow) o abortar la ejecución (problemas severos).

Fault, trap, abort.

Excepciones y malos entendidos

Pese a que la norma existe desde 1985 y que c99 (no C++) y Fortran2003 la cumplen parcialmente, ninguno otro lenguaje o compilador permite a los programadores un manejo de excepciones consistente y correcto numéricamente, comenzando con el significado que le dan al propio concepto de excepción.

Por ejemplo, en C++ y Java se le llama así a la *trampa* (trap) o transferencia de control ante una situación no considerada por el usuario, incluyendo las no numéricas. La *excepción* es la respuesta a la situación y el salto la única manera de resolverlo (principalmente en el caso de Java). Esta connotación de *atrapar* la excepción la hace ver como un fallo alarmante y no es la mejor solución en cómputo numérico. Tan sólo piense el lector en la (muy propensa a errores) estructura **try-throw-catch-finally**, que suspende el flujo normal de una

El salto es una medida desesperada la mayoría de las veces.

simple presubstitución. La entrada a un `catch` puede aún tener varios razones que pasarán inadvertidas si no se consideran las banderas de excepción.

En la tesis de doctorado de Westley Weimer [387], se presentan 800 errores por mal manejo de excepciones (de todo tipo), localizados en 4 millones de líneas de código en Java en software. En la tesis se propone una técnica de análisis automatizada que descubre estos errores. Es claro que el programador no le dedica el mismo tiempo a las excepciones que a los casos normales. Adicionalmente, vale la pena considerar la herramienta COJAC¹⁵, una pequeña aplicación de línea de comandos, con un plugin disponible para el IDE Eclipse, que permite detectar y manejar algunas excepciones numéricas, no todas.

Es un buen inicio.

En lenguaje Ada, la respuesta natural a un error numérico es abortar los cálculos, incluso ante un simple overflow. En la versión de 1983, existía confusión entre `Numeric_Error` y `Constraint_Error`, por lo que la brillante decisión en 1995 fue ¡eliminar `Numeric_Error`! Esto elimina a Ada (y similares) de los lenguajes adecuados para cómputo numérico confiable, aunque nadie duda de sus otras bondades.

Y ahora no se sabe qué pasó.

En Basic y Ada, la instrucción `On Error goto ...` tienen la misma débil e incorrecta base. Lo mismo pasa con el `longjmp` de C, que debe emplearse cuando realmente se amerita y no para cálculos numéricos.

Eso cambia el significado que tienen las excepciones en la norma de ser eventos posibles que requieren atención especial. La respuesta debe ser la presubstitución y levantar la bandera correspondiente, no dar un salto sin preguntar nada. Es claro que el salto es solo una de tantas posibles acciones ante una excepción, principalmente cuando se está depurando.

No sólo se trata de lenguajes mal diseñados, en ocasiones el problema es la política de las compañías. Los compiladores de Microsoft (C/C++) antes y ahora el famoso .NET, incumplen diversos requisitos de la norma:

1. Han usado la bandera de operación inválida de los procesadores exclusivamente para detectar la pila llena (FP stack overflow), por lo que no está disponible para su uso según la norma.
2. Por default se desactivan las excepciones, en un afán de darle al compilador flexibilidad para optimizar.
3. Algunas plataformas de Windows NT convierten los números subnormales en cero.
4. Sólo considera 4 modos de redondeo.
5. No incluyen muchas de las operaciones de la norma, como `frexp`.

El débil argumento fue que casi todo el tiempo la bandera estaría apagada.

Aún en 2010, con la norma numérica muy difundida y ya aprobada, el soporte técnico en línea de Microsoft Office indica que algunos errores como $1 * (.5 - .4 - 1) \neq 0$ se deben no a Excel o Works, sino al requerimiento de la norma de IEEE de almacenar números en binario. Esto tergiversa y mal informa (mal educa) al usuario. En todo caso, debieran tener opción de hacer uso de aritmética decimal y no hecharle la culpa a la norma.

¡Y recomiendan usar la función `ROUND` o reducir la cantidad de decimales desplegados!

¹⁵<https://code.google.com/p/cojac/>.

Rápidas pero inexactas.

Por su parte, el manual de C/C++ de las computadoras Cray no niega su entusiasmo por la velocidad a costa de todo, lo que incluye no manejo de excepciones. De esta forma, una simple división entre cero puede abortar un programa en ejecución. Claro, no es de extrañarse, considerando que ni siquiera se emplea la codificación de números de punto flotante indicada por la norma. Cosa curiosa es que existe la opción de compilación `-h -fp0` para acercarse un poco a la norma de IEEE. En su favor se puede decir que las nuevas máquinas Cray sí se están apegando a lo que la norma especifica, pero el autor no ha podido comprobar esto.

El concepto de excepciones numéricas ha madurado mucho, en especial las numéricas, y hoy en día es sencillo controlarlas. Esta es una de las mayores aportaciones de la norma. Las excepciones numéricas no tienen por qué interrumpir la secuencia de instrucciones, sino continuar la ejecución al tiempo de levantar las banderas que indican qué suceso ocurrió.

3.2.2. Detección de excepciones numéricas

Las banderas sí requieren ser revisadas luego de cada operación sospechosa, para actuar adecuadamente y permitir la propagación de valores especiales hasta donde sea necesario. Cada plataforma (lenguaje-compilador-sistema operativo) tiene sus propias políticas al respecto, eso sin considerar un mundo en lento tránsito de 32 a 64 bits, por lo que a continuación se tratan aspectos generales y pocos casos particulares.

La norma numérica especifica, como propuestas de lo que el ambiente numérico debe proporcionar, una serie de acciones que permiten detectar (y actuar en consecuencia) cuando el resultado de una operación no sea representable con un número normalizado finito. Son 5 tipos de excepciones, tal como se muestra en la tabla 3.2.

Una vez que una operación levanta una bandera, esta permanecerá levantada hasta que el usuario la baje (borre) intencionalmente. No hay manera que el procesador o el sistema operativo determinen que ya pasó suficiente tiempo como para apagarlas de manera automática. Esto puede causar confusión cuando se detecta un problema, pues hay que determinar en qué momento preciso fueron levantadas, una ayuda que ningún depurador posee aún. Más de esto en la sección 2.4.1.

La norma recomienda, sin obligar, que el usuario pueda escribir manejadores específicos para cada excepción, atrapándola y saltando al código encargado de corregir o dar proceso adicional a las operaciones. Esto para casos muy particulares, pues la presubstitución por sí sola es capaz de remediar muchos casos.

La receta típica que se usa en la programación numérica moderna tiene un diseño general como el siguiente:

1. Hacer los cálculos directos, con un algoritmo adecuado.

Nombre	Causas	Resultado
Operación inválida	$\infty - \infty$, $0 \times \infty$, ∞/∞ , $0/0$, $\sqrt{-1}$, $x \bmod 0$, $\infty \bmod x$, $x <=> NaN$, o conversión (binaria-decimal) imposible	qNaN, excepto si la operación sea una comparación o conversión no permitida
División entre cero	$x/0$ con $ x < \infty$ y $x \neq 0$	$\pm\infty$
Overflow	Resultado normalizado más grande que el formato destino	$\pm\infty$
Underflow	Resultado muy pequeño para representarlo en el formato destino	Subnormal o ± 0.0
Inexacto	La mantisa del resultado x es mayor que la del formato destino	$\text{fl}(x)$

Tabla 3.2: Excepciones indicadas por la norma 754 de IEEE con su resultado correcto. Se muestran en orden decreciente de importancia. Cada una levanta también la bandera de Inexacto.

2. Revisar las banderas de excepción. Si nada grave ocurrió, entonces continuar.
3. En otro caso, repetir los cálculos empleando más recursos: precisión extra, un algoritmo muy seguro aunque sea costoso o con ambos recursos.

Si luego del paso 3 aún siguen mal las cosas, sólo queda emplear cómputo simbólico, pero eso es tema para otra ocasión.

El ambiente de programación debe permitir que el programador lea y modifique las banderas de estado y excepción del procesador, para lo cual cada lenguaje y sistema operativo debe proveer los medios adecuados. Los lenguajes C¹⁶ y Fortran¹⁷ de ISO, son los más completos y apegados a la norma, por lo que se utilizan para ejemplificar. Para más detalles, el usuario debe consultar los manuales correspondientes de lenguaje y compilador.

Los programadores suelen encontrar sorpresas agradables y desagradables en los detalles de los voluminosos manuales oficiales.

Llamadas de `fenv.c`

En lenguaje C, mucho del trabajo con el ambiente de punto flotante se realiza mediante una estructura `fenv_t`, declarada en el archivo de cabecera `fenv.h` y que contiene algunos campos útiles para el programador (y otros tantos reservados para uso futuro). Los valores de las banderas se codifican en un número entero

¹⁶www.open-std.org/jtc1sc22wg14

¹⁷www.nag.co.uk/sc22wg5

Llamada	Propósito
<code>fegetenv</code>	Lee en una estructura <code>fenv_t</code> el estado del ambiente de punto flotante.
<code>feholdexcept</code>	Igual que la anterior, pero pone en cero todas las banderas.
<code>fesetenv</code>	Coloca (restaura) el estado del ambiente numérico.
<code>feupdateenv</code>	Como el anterior, pero respeta las banderas que se encuentran levantadas.
<code>fegetround</code>	Determina el modo de redondeo actual.
<code>fesetround</code>	Establece un modo de redondeo.
<code>fetestexcept</code>	Revisa si cierta bandera(s) está(n) levantada(s).
<code>fegetexceptflag</code>	Lee en una variable todas las banderas de estado.
<code>feclearexcept</code>	Borra las banderas de excepción que su argumento lo indica.
<code>feraiseexcept</code>	Levanta la bandera que su argumento indica.
<code>fesetexceptflag</code>	Establece un conjunto de banderas de excepción.

Tabla 3.3: Funciones para manejo de excepciones en C. Permiten controlar las banderas, los modos de redondeo y el ambiente numérico.

simple. La tabla 3.3 muestra el conjunto completo de las llamadas definidas para el ANSI C, varias de las cuales emplean `fenv_t`.

En otros sistema operativo, donde es común que el procesador sea distinto a Intel, es frecuente que las llamadas tengan otro nombre. En SunOS (mejor conocido como Solaris), en sus versiones modernas (mantenidas por Oracle) el prefijo `fe` se cambia por `fp` y los nombres se modifican: `fpgetround`, `fpsetmask` o `fpsetsticky`.

El archivo `fenv.h` también incluye las macros con valores de las banderas de excepción. Es de esperarse que todo lenguaje que pretenda hacer cálculos numéricos con eficiencia disponga de llamadas de alto nivel para controlar las banderas.

Cada segmento de código que requiere acceso a las banderas de punto flotante, requiere que se compile indicando con la directiva de preprocesamiento `pragma STDC FENV_ACCES ON`, pues de lo contrario el resultado de algunas funciones declaradas en `fenv.h` no está definido. No es correcto activar la directiva al inicio de la función `main` y confiar que se cumple para el resto del programa. El efecto sigue hasta encontrar la misma directiva con el valor `OFF` o el fin del archivo que se compila, exclusivamente, por lo que se debe usar este recurso en cada lugar donde el acceso a las banderas se requiera. La mejor práctica es cerrar con `OFF` cuando ya no es necesaria la lectura de banderas.

De la misma manera, la directiva `pragma STDC FP_CONTRACT OFF` impide que el compilador modifique expresiones durante la conversión a código objeto, hasta que se encuentra la directiva con el valor `OFF` o el fin de archivo.

Etiqueta en C	Etiqueta en Fortran	Clase numérica
_FPCLASS_SNAN	FOR_K_FP_SNAN	Señal de Not a Number
_FPCLASS_QNAN	FOR_K_FP_QNAN	Not a Number silencioso
_FPCLASS_NINF	FOR_K_FP_NEG_INF	Infinito negativo
_FPCLASS_PINF	FOR_K_FP_POS_INF	Infinito positivo
_FPCLASS_NN	FOR_K_FP_NEG_NORM	Normal negativo
_FPCLASS_ND	FOR_K_FP_POS_NORM	Subnormal negativo
_FPCLASS_NZ	FOR_K_FP_NEG_ZERO	Cero negativo
_FPCLASS_PZ	FOR_K_FP_POS_ZERO	Cero positivo
_FPCLASS_PD	FOR_K_FP_NEG_DENORM	Subnormal positivo
_FPCLASS_PN	FOR_K_FP_POS_DENORM	Normal positivo

Tabla 3.4: Macros que puede responder `fpclass`. La misma llamada es utilizada en Fortran, pero las macros son distintas.

A manera de ejemplo del uso de las operaciones de la norma en `fenv.h` y `math.h` se presenta el código 3.9. En el primer bloque se causa un underflow al dividir al menor número subnormal entre 2. El segundo bloque evalúa .1 cambiando el modo de redondeo.

La salida del programa es la siguiente:

```

Underflow provocado:
Operacion subnormal
Underflow
Operacion inexacta
Modo de redondeo:
0.099999999999999992, 0.100000000000000001

```

Con estos elementos, es posible dar gran versatilidad a los programas y probar fenómenos indeseables en los cálculos, modificando el modo de redondeo y leyendo banderas de excepción.

Otras llamadas útiles del ANSI

Los recursos definidos en `fenv.h` no son los únicos que el programador tiene de su lado para ejercer control sobre las excepciones. En los archivos de cabecera `math.h`, `float.h`, `ieeefp.h`¹⁸ o `fpclass.h` (dependiendo del compilador) pueden encontrarse funciones de la familia `fpclass`, que establece la categoría a la que pertenece cualquier número.

En MinGW, la llamada para determinar la categoría de valores (clase) a la que pertenece la variable `x` es `Clase = _fpclass(x)`. Ligeros cambios son de esperarse para otros sistemas, como `fpclass_d` para linux y similares. Los resultados posibles se especifican en la tabla 3.4.

¹⁸Este archivo aparece en FreeBSD (OpenBSD, NetBSD) y cambia muchas cosas en las llamadas y macros del ambiente flotante.

Código 3.9: Ejemplo de uso de las funciones de la norma: lectura de banderas y redondeo direccionado.

```

1  #include <stdio.h>
2  #include <math.h>
3  #include <fenv.h>

5  int main()
6  {
7      int fstat;
8      int roundIni;
9      double x;

11     #pragma STDC FENV_ACCESS ON
12     feclearexcept(FE_ALL_EXCEPT);
13     puts("\t Underflow provocado:");
14     x=nextafter(0,1);
15     x = x/2;
16     fstat = fetestexcept(FE_ALL_EXCEPT);
17     if (fstat & FE_INVALID ) puts("Operacion invalida");
18     if (fstat & FE_DENORMAL ) puts("Operacion subnormal");
19     if (fstat & FE_DIVBYZERO) puts("Division entre 0");
20     if (fstat & FE_OVERFLOW ) puts("Overflow");
21     if (fstat & FE_UNDERFLOW) puts("Underflow");
22     if (fstat & FE_INEXACT  ) puts("Operacion inexacta");

25     puts("\tModo de redondeo:");
26     roundIni = fegetround();
27     if ( fesetround(FEDOWNWARD) )
28         puts("No se pudo cambiar el modo de redondeo");
29     x = 10.0;
30     printf("%.17g, ", 1/x);
31     if ( fesetround(FEUPWARD) )
32         puts("No se pudo cambiar el modo de redondeo");
33     printf("%.17g\n", 1/x);

35     fesetround(roundIni);
36     #pragma STDC FENV_ACCESS OFF

38     return 0;
39 }

```

Código 3.10: Usando excepciones con Fortran

```

1 Discriminante = sqrt(b**2 - 4*a*c)
2 CALL IEEE_GET_FLAGS(OUT_OF_RANGE, flags)
3 if ( any(flags) ) then
4     ...

```

Código 3.11: Instalando un manejador de excepciones en (algunas versiones de) Fortran.

```

1 EXTERNAL Manejador
2     real :: x = 1.0, y = 0.0
3     ieee_handler ( 'set', 'division', Manejador )
4     z = x/y
5 END

7 integer FUNCTION Manejador(sig,code,context)
8     integer sig, code, context(5)
9     CALL abort()
10 END

```

Excepciones en Fortran

La especificación ISO de Fortran (2003 y sus trabajos de 2008), aclara que las facilidades proporcionadas por los módulos intrínsecos IEEE_EXCEPTIONS, IEEE_ARITHMETIC y IEEE_FEATURES dependen del procesador con que se trabaje.

Fortran cuenta con maneras de evaluar y modificar el ambiente numérico. Por ejemplo, la función `fpclass` existe en Fortran, por lo que buena parte de los problemas pueden detectarse. Los valores que regresa están también en la tabla 3.4.

En Fortran se tiene acceso a las banderas del procesador con la función IEEE_GET_FLAGS. Por ejemplo en el código 3.10, donde se hacen los cálculos sin preocuparse y luego se averigua si algo malo ocurrió, revisando la variable `FLAGS`.

Este método es transparente para el programador. Las instrucciones son sencillas y elegantes. Un ejemplo más completo es el código 3.25, página 129, para el cálculo de la hipotenusa.

Es ideal tener códigos que se leen fácil.

En el caso de que la intención sea atrapar una excepción, algunos compiladores (Oracle, Intel) ofrecen la función `ieee_handler` (que también existe para C), que instala un manejador de excepción. La norma lo recomienda, pero no es una llamada definida oficialmente en Fortran (ni en GFortran). Se ejemplifica en el código 3.11, donde no se da tratamiento alguno, sólo se aborta.

Si se desea determinar qué excepciones están habilitadas, la siguiente línea lo hace.

Valor	Significado
FPE_INTDIV	Integer divide by zero
FPE_INTOVF	Integer overflow
FPE_FLTDIV	Floating-point divide by zero
FPE_FLTOVF	Floating-point overflow
FPE_FLTUND	Floating-point underflow
FPE_FLTRES	Floating-point inexact result
FPE_FLTINV	Invalid floating-point operation
FPE_FLTSUB	Subscript out of range

Tabla 3.5: Representación de excepciones numéricas para POSIX.

```
i = ieee_flags("get", "exception", in, out)
```

Algunos compiladores permiten conocer el lugar en el código donde ocurrieron las excepciones, como `-qflttrap` y `-qsigtrap` del compilador IBM XL o `-ftrap=excep` de Sun, pero esto no está indicado en la norma.

Excepciones en POSIX

La X viene de UNIX que es la plataforma original donde surgieron en 1988.

POSIX (Portable Operating System Interface) es una API (Interfaz de Programación de Aplicaciones) común para gran cantidad de sistemas operativos derivados de UNIX (Linux, AIX, Solaris, HP-UX y muchos más), definido como un conjunto de normas de IEEE. Para Windows, hay un API completo llamado Cywin, que es de código abierto, mantenido por GNU (y mucho mejor que la versión minimalista de Microsoft, el Microsoft POSIX subsystem, sin threads ni sockets).

La manera de detectar una excepción numérica es con la señal SIGFPE, que normalmente es sólo una constante simbólica declarada en el archivo de cabecera `signal.h` de cada sistema operativo.

A la variable de sistema `errno` se le asigna el error que ocurrió más recientemente. Los valores que representan problemas numéricos son EDOM, cuando es un valor inválido para el dominio o ERANGE cuando el resultado es mayor al tipo de dato.

Una vez recibida la señal SIGFPE, sólo se sabe que algún problema numérico ocurrió. Para determinar cuál fue el problema específico se requiere revisar el `si_code`, dentro de una estructura `sig_info_t`. Los valores de ese campo deben estar definidos en el archivo `siginfo.h` y se muestran en la tabla 3.5.

Una vez determinado el motivo de la señal, se puede emplear la llamada `sa_sigaction`, que es un apuntador a la función que manejará el problema, en caso de ser necesario. El apuntador se define usando la función `sigaction`, cuyo prototipo es

```
int sigaction(int sig, const struct sigaction *act,
              struct sigaction *oact);
```

El argumento `act` apunta al manejador designado y `oact` al manejador previo (muchas veces `NULL`).

En POSIX es frecuente enmascarar señales que se deben bloquear mientras se atiende una excepción, pero no aplica para los problemas numéricos.

El programador debe crear la subrutina o función que se encargará de manejar cada excepción. En C, lo que se hace es utilizar la función `signal` como:

```
signal(SIGFPE, Manejador);
```

y `Manejador` es la rutina que será llamada cuando se de la señal `SIGFPE`. Cada compilador y sistema operativo define los detalles. Además, ANSI define variables externas como `errno` de lenguaje C, que indican el tipo de error, cuyo valor se actualiza cada operación.

En el año 2000, Koopman y DeVale sometieron a prueba 233 llamadas a sistema y funciones del POSIX en 15 sistemas operativos distintos. La mayor parte de los problemas no son numéricos, pero los problemas de punto flotante sí aparecen en el estudio, particularmente por overflow entero y flotante o conversión incorrecta. FreeBSD es el único sistema donde lo numérico sobresale debido a que ahí se insiste en utilizar `SIGFPE` para indicar una amplia gama de errores. Los detalles los reportaron en [221].

3.2.3. Omitiendo excepciones

Algo que no define el estándar `c99` es la manera como se activan o desactivan las excepciones, pero puede hacerse en algunos procesadores utilizando la estructura `feenv_t`, cuyo campo `_mxcsr` permite bloquear excepciones numéricas específicas. Esto es posible en el caso de los procesadores SIMD (Single Instruction Multiple Data), en particular para SSE (Streaming SIMD Extensions), disponible desde 1999. El registro de control `MXCSR` permite monitorear y controlar el estado de las instrucciones.

El registro `MXCSR` consta de 32 bits, aunque sólo del 0 al 15 están definidos. Los bits del 7 al 12, mostrados en la tabla 3.6 deben ser cero, indicando que todas las excepciones deben ser identificadas. Levantar algún bit atrapa la excepción y continúa la ejecución de las instrucciones del programa con normalidad sin hacer caso de problemas, lo que es un arma de doble filo: acelera los procesos pero puede causar resultados inválidos.

Los otros bits del `MXCSR` permiten considerar los subnormales como cero, leer o establecer el modo de redondeo y leer las excepciones que han ocurrido (con los bits no enmascarables). No todos los procesadores permiten que el usuario modifique todas las banderas, por lo que debe tenerse cuidado. Un fallo general de protección puede ocurrir si se modifica un bit reservado.

Para desactivar la excepción de operación inválida (el caso más drástico) se requiere algo como

Mnemónico	Bit	Propósito
PM	12	Operación inexacta
UM	11	Underflow
OM	10	Overflow
ZM	9	División entre cero
DM	8	Resultado subnormal
IM	7	Operación inválida

Tabla 3.6: Bits del registro MXCSR. Permiten controlar las banderas, los modos de redondeo y el ambiente numérico.

```
struct fenv_t Mi_env;
fegetenv(&Mi_env);
Mi_env.__mxcsr |= (1 << 7);
fesetenv(&Mi_env);
```

y en este segmento de código sólo falta verificar los valores de retorno de las funciones `fgetenv` y `fsetenv`, para tener certeza de que se realizaron con éxito (ambas deben regresar cero).

Si se trabaja con el API WIN32, otra posibilidad en el muy utilizado MinGW es emplear la llamada `_controlfp(BitsNew, Mask)`. Es similar a la llamada `_control87`, pero esta última siempre pone en cero el bit de resultado subnormal. Otras llamadas de esta familia son `_clearfp` y `_statusfp`, cuyos nombres hablan por sí mismos.

A diferencia del lenguaje C, Fortran sí dispone de maneras directas de deshabilitar excepciones. Por ejemplo, la llamada que desactiva la excepción de Operación inválida es la siguiente.

```
CALL IEEE_SET_HALTING_MODE(IEEE_INVALID, .false.)
```

La deshabilitación puede ser en el tiempo de compilación en el caso de algunos compiladores. Por ejemplo, en el XL Fortran de IBM o Intel, se dispone de las opciones `-fpeX` (`/fpe:X` en Windows), donde X puede ir de 0 a 3. X=0 detiene el programa ante cualquier excepción de punto flotante y los subnormales se van a 0. X=3 usa la presubstitución en todos los casos y maneja subnormales como en la norma.

En POSIX, si la aplicación no es demandante numéricamente, como es el caso en indicadores estadísticos generales, conversión de imágenes, procesos gráficos y otros, se dispone de formas de omitir señales, como `signal(SIGFPE, SIG_IGN)` que ignoran toda señal¹⁹. Incluso opciones de compilación como `-mieee` para anular la verificación.

Extensiones en GNU La Biblioteca GNU para lenguaje C define una macro llamada `FE_NOMASK_ENV` que representa un entorno donde toda excepción lanza-

¹⁹Realmente direccionan toda señal de punto flotante a un manejador vacío.

da provoca una trampa. Tal macro estará definida sólo si `_GNU_SOURCE` también está definida, lo que se averigua con un `#ifdef`.

En algunas versiones, la biblioteca de funciones de GNU C incluye funciones para atrapar excepciones, que permiten poner la bandera respectiva para que la excepción sea o no considerada. Las llamadas son `feenableexcept` y `fedisableexcept`. Por otra parte, `fegetexcept` consulta el estado de las banderas. Ninguna de estas llamadas es del c99, lo que resta portabilidad.

3.2.4. Levantando banderas

Si un programador está desarrollando rutinas de bajo nivel, en algunas ocasiones debe realizar operaciones que consisten en cierto manejo simbólico de los números. Cuando se puede anticipar que el resultado de una operación será un número especial, es necesario que el código lleve a cabo el protocolo completo y levante la bandera de excepción que corresponde. Hay dos maneras principales de hacer esto.

Por ejemplo, si se está calculando la función entera $1/n!$, entonces se ocasiona overflow de $n!$ en precisión doble si $n > 170$, por lo que el resultado correcto del cociente debe ser 0 por las propiedades de propagación de los valores especiales. Esta operación debe levantar la bandera de Overflow. Si la variable `UnderflowFlag` ya tiene el bit correcto prendido, entonces en el código es necesario agregar el segmento de código que se muestra en el código 3.12.

Código 3.12: Levantando banderas y determinando la arquitectura en tiempo de compilación.

```
1  if ( n>170 ) {
2    #ifdef 80x86
3      _status87( UnderflowFlag );
4    #else /* Unix por default */
5      fpsetsticky( UnderflowFlag );
6    #endif
7    return 0.0;
8  }
```

Otra técnica sencilla es hacer una operación que asegure que la bandera de excepción correcta será levantada, independientemente de la plataforma y resultado. En el ejemplo anterior, antes de regresar cero (que no levanta ninguna bandera) se puede calcular `MAXDBL*MAXDBL`, lo que ocasionará un overflow con toda seguridad, evitando levantar la bandera desde el código y preocuparse por determinar la arquitectura. Esto se muestra en el código 3.13.

Sencilla y elegante, considerando que a la fecha aún hay más de 10 arquitecturas distintas.

Código 3.13: Levantando banderas sin conocer la arquitectura.

```
1  if ( n>170 )
2    return (1/DBLMAX)*(1/DBLMAX);
```

Con el código 3.13 se asegura que el procesador se encargará de levantar las banderas de inexacto y underflow, lanzando la señal correspondiente. `DBLMAX` está definido en `math.h`.

El ejemplo de $1/n!$ es minimalista si se considera que con $n = 171$ a 178 se obtienen valores subnormales correctos²⁰, además de que a la variable externa **errno**, que provee el sistema, se le debe asignar el valor que corresponda.

En el artículo [134], los autores analizan la expresión

$$173746 \sin(10^{22}) + 94228 \log(17.1) - 78487e^{0.42}$$

y ven las diversas maneras de calcularla, incluso utilizando precisión arbitraria²¹. Vale la pena experimentar por su propia cuenta.

3.2.5. Dos problemas famosos

Cohete Ariane 5 En el lanzamiento del cohete Ariane 5, un sensor reportó una aceleración tan fuerte que ocurrió un overflow al convertir de flotante a entero. En vez del manejo esperado (levantar las banderas de overflow e inexacto y regresar el resultado) se activó el sistema de diagnóstico (que quedó activado por error), lanzando información para depuración posterior a una región de memoria en uso. El resultado pudo simplemente ser ignorado por el sistema de control de los motores y continuado con la trayectoria normal, en vez de considerar que era necesario un brusco cambio de dirección. ¿Y si un problema similar ocurriera con el sistema que activa los explosivos que evitan que caiga en una región habitada?

El error costo 500 millones de dólares, mas otros 7000 del desarrollo.

Navío Yorktown El otro caso es el del navío de combate Yorktown, cuyo sistema se quedó esperando el reinicio de la computadora ante una división entre cero calculada por un campo en blanco (por error) en una base de datos. De seguirse la norma, se hubiera calculado ∞ y continuado con etapas correctivas, en vez de navegar en círculos de manera incontrolada hasta que la tripulación logró reiniciar la computadora. ¿Y si hubiera ocurrido en un puerto, o en combate?

El costo pudo ser en vidas humanas.

Muchos otros problemas pueden ser consultados en muchas fuentes. Varios más viene en el primer capítulo de [282], de Muller, Lefevre y otros siete autores especializados en diversas áreas de aritmética de punto flotante. El primer capítulo está disponible en la página web de Muller.

3.3. Después de la norma

Comenzando con el 8087 de Intel. Antes de aprobarse, la norma 754 ya comenzaba a ser utilizada tanto por programadores (los más) como por fabricantes de procesadores y diseñadores de lenguajes.

²⁰Lo mejor sería precalcularlos y guardarlos en un arreglo, pues sólo son 7 valores.

²¹Utilizando para el experimento la librería MPFR de GCC.

Si el procesador, lenguaje y compilador cumplen con la norma, entonces cualquier programador no especializado pudiera desarrollar programas moderadamente complejos y obtener resultados cuya exactitud estará limitada sólo por la APF. Esto sin necesidad de conocer la norma en detalle y sin ser un especialista en análisis numérico.

3.3.1. Evaluando si se cumple

Si se desea hacer uso de las bondades de la norma, lo mejor sería poder evaluar si procesador, lenguaje y compilador cumplen con los requisitos mínimos. Los algoritmos pueden ser verificados más fácilmente, pero probar que el dispositivo físico sigue cumpliendo con los requisitos es otra cosa. En los primeros años de su desarrollo, las baterías de prueba y simulaciones fueron la forma de asegurar que lo planteado en papel se cumplía en el procesador y esa metodología sigue siendo fundamental para todos los fabricantes.

Verificación formal

Existen propuestas de pruebas formales en las que se busca una demostración matemática de que un programa es correcto. Al realizarse y aprobarse por un ambiente en particular, no se dejan dudas sobre la confianza que se puede tener, pero son más laboriosas. Poco después de la publicación de la norma en 1985, Barret publicó [24] donde aplica el lenguaje Z en la verificación de la norma. Aunque Barret anuncia que analiza sólo operaciones no excepcionales, la lectura de su trabajo deja ver que considera ∞ y NaN como la norma lo indica, pero no revisó más allá de operaciones aritméticas y lógicas elementales. Aún así, es un excelente ejemplo.

Se debe mencionar también el trabajo [308], de Popova, donde revisa el formalismo necesario para un correcto estudio de la norma y analiza con todo detalle todas las posibles combinaciones de operaciones entre valores especiales. Otra referencia muy completa es el trabajo de Kern y GreenStreet [216], sobre verificación de diseño de hardware en general. Describen una vasta cantidad de técnicas, desde la abstracción y el refinamiento hasta la prueba automática de teoremas, pasando por lógicas temporales, lenguajes regulares y otras cosas, sin olvidarse de demostrar sus limitaciones. Las 197 referencias y 71 páginas del artículo reflejan el estudio a fondo sobre el tema.

La especificación formal es una técnica aún difícil de aplicar en sistemas grandes, pero útil para rutinas y programas pequeños.

Trabajos sobre pruebas formales de operaciones particulares pueden servir de referencia para iniciar un estudio más específico, como [188] sobre **fma**, [211] sobre la multiplicación, [265] sobre la división o [346] donde se estudia la verificación de programas numéricos considerando cómputo paralelo. Sobre construcción segura de fórmulas, el trabajo de Brillout [45] es el que presenta un algoritmo para aproximar iterativamente una fórmula compleja e incluso detecta defectos en la especificación, pudiendo incluso generar contraejemplos.

Código 3.14: Calculando el valor de la base β .

```

1  a=1.0;
2  while ( ((a+1)-a)-1 == 0 )
3      a = 2*a;
4  Base = 1.0;
5  while ( ((a+Base)-a)-Base != 0 )
6      Base = Base+1;

```

Algunos fabricantes publican los trabajos de verificación formal y pruebas prácticas de los algoritmos empleados en sus procesadores y de los procesadores mismos, comparando las cotas teóricas con las prácticas, como [296] de Intel o [330] de AMD.

La revisión reciente más amplia de este tema es [398], de Woodcock y varios autores más. En este trabajo se habla del análisis y verificación de cualquier parte del ciclo de desarrollo. En 36 páginas hacen un resumen excelente del estado del arte, además de lo variado de sus 150 referencias.

Verificación práctica

En la verificación formal se parte del diseño de los sistemas, por lo que sólo es posible hacerla si se tiene toda la documentación del ambiente numérico. Por ello, su aplicación es limitada, aunque el aumento de sistemas de código abierto ha facilitado algo de este trabajo.

Desde hace muchas décadas ya hay programas para evaluar las características de la APF de las computadoras. Son más prácticas que los trabajos de verificación formal y generalmente los programas pueden servir para varias computadoras, comparando parámetros detectados o resultados de operaciones.

Una de las primeras referencias²² es el trabajo de Redish y Ward de 1971 [316], en el que plantean que es de esperarse que la siguiente generación de lenguajes tengan algunas facilidades comunes normadas. Mientras tanto, era necesario ir pensando a futuro en los requisitos que debían exigirse a los lenguajes. Proponen algunas llamadas de alto nivel para separar las variables de punto flotante en sus componentes básicos, como en la definición 1.11, de la página 20 (**frexp** y **ldexp** de la norma).

Michael A. Malcolm fue de los primeros que desarrollaron programas y rutinas para manipular NPF. Las primeras rutinas las desarrolló en Fortran (véase [255]) y posteriormente se aplicaron en otros programas. Básicamente eran subrutinas que calculaban los parámetros β , p y si el CPU aplica redondeo o truncaiento, que era de lo más relevante en esa época.

La base β puede ser calculada con el código 3.14. Teniendo la base, la precisión p se calcula con el código 3.15.

²²El autor no ha encontrado ninguna anterior.

Cosa que no ocurrió.

Código 3.15: Calculando la precisión.

```

1  a=1.0;
2  do {
3      Prec = Prec + 1;
4      a = a*Base;
5      b = a + 1;
6  } while ( b-a == 1 );

```

Un error de programación flotante cometido por Malcolm en su programa fue corregido por Gentleman y Marovich en [132]. Ese error es el típico ejemplo de problemas que a la fecha existen, aunque sin causar daños tan graves. Si los cálculos intermedios se realizaban con una precisión distinta al del formato destino, los parámetros eran incorrectos. ¿Qué precisión se debe reportar, la interna (mayor) o la del formato? En una computadora Honeywell 6000, el programa de Malcolm incorrectamente reportó todo mal: $\beta = 0$, $p = \infty$ y redondeo detectado por truncamiento. Reescribiendo en lenguaje C (el original está en Fortran), las condiciones de la forma

```
if ( a+b != a )...
```

debían cambiarse por

```
if ( (a+b)-b > 0 )...
```

para corregir el problema y persistía aún en algunas computadoras con aritmética deficiente.

En [245], en 1981, aparece (y se demuestra) un nuevo procedimiento atribuido a Kahan para calcular la base y la precisión. Para obtener β y p , primero se calculan

$$U = \text{fl} \left(3 \times \left(\frac{4}{3} - 1 \right) - 1 \right)$$

$$R = \text{fl} \left(\left(\frac{U}{2} + 1 \right) - 1 \right)$$

si $R \neq 0$ se hace $U = R$

$$u = \text{fl} \left(3 \times \left(\frac{2}{3} - .5 \right) - .5 \right)$$

$$r = \text{fl} \left(\left(\frac{u}{2} + .5 \right) - .5 \right)$$

si $r \neq 0$ se hace $u = r$. Ahora ya se pueden calcular

$$\beta = \text{fl} \left(\frac{U}{u} \right)$$

$$p = \left\lfloor \text{fl} \left(\frac{-\log(u)}{\log(\beta)} + .5 \right) \right\rfloor$$

¡Más mal imposible!

con la ventaja adicional de ser métodos directos y no iterativos.

Mientras se discutía la norma, en 1983 Kahan programó *Paranoia* en Basic, un programa que permite determinar algunos de los parámetros de la APF, como la base, precisión, modo de redondeo, existencia del bit de guardia y operaciones correctamente redondeadas. Ha sido traducido a otros lenguajes y está disponible en NetLib²³. En 2004 se publicó una versión de *paranoia* para sistemas empotrados (embedded systems) por Les Hatton en [161]. El mismo año, Hillesland y Lastra presentaron una versión de *Paranoia* para evaluar los procesadores de las tarjetas gráficas (GPU) en [193].

Además de encontrar la base y la precisión de dos maneras distintas para comprobar, *paranoia* detecta el modo de redondeo, límites de exponentes, valores límites, épsilon de máquina (ε_M), si se realiza underflow gradual, evaluación de subexpresiones con precisión extra, el sticky bit, manejo de excepciones y otros más. Adicionalmente, busca anomalías aritméticas en las operaciones elementales.

El autor recomienda ampliamente descargar el código, compilarlo y ejecutarlo. La mejor documentación es la salida misma del programa. Aún habiendo sido programado hace tantos años, su vigencia es de admirarse.

Basado en sus trabajos de 1980, Cody escribió el programa *Machar* como parte de la librería para probar funciones elementales ELEFUNT y en 1988 lo reescribió acorde a la norma de IEEE [72] en [72] para reportar parámetros similares a los de *paranoia*. A diferencia de *paranoia*, no busca anomalías en operaciones elementales.

En 2002, Nelson Beebe reescribió el programa *Machar* en Java para usarlo la nueva librería `java.lang.Math` que extiende las funciones matemáticas para una mejor programación numérica. Con este mismo objetivo surgió el lenguaje Borneo (a propuesta de Kahan y Joseph Darcy) como una extensión de la semántica de punto flotante de Java, para resolver todas las deficiencias numéricas y apegarse a la norma. Estos problemas los analiza Kahan en [208] y se puede adelantar que no posee mecanismos para redireccionar el redondeo en tiempo de ejecución.

En 1996, Kahan propuso otra forma de evaluar exclusivamente la precisión en [202]. Lo hace resolviendo una ecuación cuadrática con una serie de coeficientes que cada vez requieren una mantisa mayor, lo que propone como medida estándar de precisión. El código original es para MatLab, como muchos de Kahan.

Herramientas para probar el ambiente de numérico

Los códigos de *Paranoia* y *Machar* se consiguen fácilmente en Internet. A parte de estos, otros tantos se han diseñado para evaluar la eficiencia numérica del ambiente de programación. Algunos de estos se muestran a continuación.

²³<http://www.netlib.org/> Además, hay una versión llamada `esparanoia.c`, donde Lee Hatton eliminó la interactividad y agregó algunas extensiones menores.

De lo más instructivo que hay para programadores novatos y expertos.

A la fecha no existe ninguna implementación de Borneo que el autor conozca.

- UCBTEST, que es un conjunto de programas para probar algunos casos difíciles de la norma, es decir, operaciones cuyo resultado queda muy cerca o en medio de dos números representables en $\mathbb{F}$. Además de Paranoia, incluye UCBmultest, UCBdivtest y UCBsqrttest, que generan casos de prueba problemáticos, en el sentido de producir resultados muy cerca del punto medio de dos NPF vecinos. UCBTest prueba funciones elementales. Esta batería de prueba fue motivada por Kahan luego de su curso de 1988 en Sun Microsystems.
- TestBase, programas para verificar la conversión entre formato binario interno y cadenas decimales, para muchos sistemas, considerando todos los modos de redondeo de IEEE, incluyendo modos especiales (que ya no se usan).
- TestFloat es un conjunto de pruebas de John Hauser que utiliza la librería de gran calidad SoftFloat²⁴ de IEEE que emula con software las operaciones aritméticas para compararlas con las del hardware. Una discrepancia indicará anomalías de algún tipo.
- NAG Floating-Point Validation package, del Numerical Algorithms Group, uno de los grupos de desarrollo de software numérico mas consistentes. Algunos notables investigadores, como Cody, consideran NAG superior en muchos aspectos a IMSL.
- SRTEST es un programa pequeño de Kahan para probar la correctitud de los algoritmos de división de los procesadores. La idea original es detectar errores de división mucho antes de lo que se espera que el uso cotidiano logre.
- MPCHECK es un programa escrito por Revol, Péliissier y Zimmermann y disponible en la página web www.loria.fr/~zimmerma/free/. Lo que hace es calcular el porcentaje de redondeos incorrectos para diversas arquitecturas, en casos difíciles de resolver para muchas funciones.
- IeeeCC754 verifica que la precisión y los rangos sean los correspondientes a las normas 754 y 854. Para más detalles consultar [376,377]. Está basado en partes en el programa inicial de Jerome Coonen FPTEST²⁵.
- CADNA (Control of Accuracy and Debugging for Numerical Applications), que es una extensión para C/C++ y Fortran. Ofrece una perspectiva distinta. Pretende medir la exactitud de los cálculos por la propagación de errores de redondeo, detectar inestabilidades numéricas, la sensibilidad de los decisores (branching) y la precisión de los cálculos intermedios. Lo que hace es ir repitiendo los cálculos en diversos modos de redondeo y llevar estimación de todos los valores (calculados como el promedio de los

Suena a aritmética de intervalos.

²⁴<http://www.jhauser.usarithmicSoftFloat.html>

²⁵Se puede consultar la página www.win.ua.ac.be/~cant/ieecc754.html, donde hay más detalles.

resultados). Las variables de los códigos por revisar tienen que ser declaradas como estocásticas, definidas para cada tipo. Por ejemplo `float_st` en vez de `float`.

- Nelson Beebe ha incluido en su página²⁶ una colección amplia de software para probar características numéricas, además de notas y ligas a software interesante para todo programador.

Si el lector diligente descarga se pone a probar las diversas herramientas, debe considerar que operaciones inexactas pueden ser el resultado de opciones de compilación incorrectas o un compilador defectuoso. Por lo mismo, resultados exactos a la primera pueden estar escondiendo hardware o software defectuoso. Se recomienda proceder poco a poco.

En particular cuando se trata de detalles finos de la APF.

Aún no se puede confiar por completo en programar con la norma. De hecho, la situación no ha cambiado mucho desde 1997, cuando el estudio de Dennis Verschaeren y otros [378] reporta qué tanto cumplen la norma diversos ambientes de programación, en el caso de C/C++ y Fortran. Por ello es necesario poder evaluar cada ambiente de desarrollo, en particular aquellos con los que el programador no está familiarizado.

3.3.2. Programación básica

Conciendo la norma y habiendo comprobado hasta dónde el ambiente de programación la respeta, lo más evidente es hacer uso tanto del formato como de sus propiedades. Por ejemplo, si $\text{fl}(x) = m \times \beta^e$, funciones como $\log(x)$ pudieran simplificarse como

$$\log(m \times \beta^e) = \log(m) + e \log(\beta).$$

Esto es lo obvio, pero hay otras operaciones donde las cosas no son tan agradables, como x^y . En ese caso, qué hacer luego de

$$x^y = (m_1 \times \beta^{e_1})^{m \times \beta^e} = m_1^{m \times \beta^e} \times \beta^{e_1 \times m \times \beta^e}$$

ya no es tan obvio.

Gran cantidad de librerías matemáticas se han desarrollado en diversos lenguajes populares como Java o C++, así como en Fortran o en C. Este último tiene la ventaja de disponer de recursos para manipular el bajo nivel con facilidad, tanto direcciones de memoria como los bits de manera directa con operadores nativos, por lo que se usará más comunmente en los ejemplos subsecuentes.

Planteamientos para extender la precisión y garantizar errores relativos menores en las operaciones al estilo de Dekker y Linnainmaa [89, 245] siguen siendo

²⁶ <http://www.math.utah.edu/~beebe/software/ieee/>.

útiles, pero ya hay versiones más modernas como la de Bayle [19], programada en Fortran (apoyada por [18]) y utilizada por la excelente librería BLAS (ver [243]²⁷).

En 2006, Martell presentó el trabajo [257], donde analiza de manera muy formal, la semántica de los lenguajes para propiciar transformaciones (basadas en la semántica) en los programas que benefician la exactitud de los cálculos. Las transformaciones planteadas producen resultados más cercanos a los que se obtendrían si se usara aritmética exacta. Un importante trabajo previo sobre este tema es [34], de 1991.

Algo así sería deseable dentro de las optimizaciones numéricas de los compiladores, incluso [138].

Sin la intención de adentrarse en los detalles de aplicaciones muy amplias, reservadas para la segunda parte de este documento, se ejemplifican algunos casos sencillos.

Cambio de fórmulas y algo más

Después de la norma, no han dejado de surgir técnicas y estrategias generales para minimizar errores. Desde construcciones sofisticadas de fórmulas hasta análisis de la semántica de los lenguajes. Un ejemplo interesante que perturba sistemáticamente el código, incluso estudia la (in)estabilidad numérica y el número de condición es el de Erhel en [104], de 1993. La definición de número de condición es cambiada y, claro, supone continuidad (al menos local), algo complicado en $\mathbb{F}$, pero siempre se parte de ahí. En el artículo, un conjunto de valores $x = (x_1, x_2, \dots, x_n)$ es perturbado con la regla

$$x_i = x_i(1 + e_i\alpha)$$

donde α es el tamaño de la perturbación y e_i es una variable aleatoria.

Uno de los mejores compendios de transformaciones, posteriores a la norma, la publicaron en 1994 Bacon, Graham y Sharp [17], en 76 páginas (poco más de 8 páginas son puras referencias). Ahí incluyen técnicas de cómputo secuencial y paralelo. Muchas de estas, ya se estaban practicando en la etapa de optimización de los compiladores.

Un reciente artículo que analiza los cambios de expresiones es [259], donde Martel propone una semántica basada en transformaciones para minimizar los errores. Esta no se limita sólo a el cambio de fórmulas, sino a la modificación segura de estructuras de control.

Un paso más lejos se da en [360], de 2010, los autores hace uso de un sistemático cambio de expresiones para analizar estadísticamente la estabilidad de sistemas numéricos. Las transformaciones de código se hacen como perturbaciones aleatorias (método de Monte Carlo). Perturban no sólo expresiones, sino valores numéricos (en los últimos bits) para comparar resultados y detectar anomalías. En [357], Sumner presenta mejoras muy interesantes a este procedimiento.

²⁷En particular el código de la función `BLAS_dsum_x`.

3.3.3. Identificando valores especiales

Ya algunos recursos de programación fueron presentados en la sección 3.2 sobre manejo de excepciones, por ejemplo con la función `fpclass`. En este apartado se complementa esta información.

Las comprobaciones de que los números sean finitos y no especiales debiera poder hacerse desde el lenguaje con instrucciones booleanas como `isNaN(x)`, `isSubnormal(x)`, `isFinite(x)` o `isInfinite(x)`. Incluso debiera existir la función booleana `is754()` que regrese cierto si el ambiente opera conforme a la norma. Cuando no se dispone de esto, se puede comprobar de otras maneras, como `if ( x != x )`, que es cierto sólo cuando `x` es NaN.

Ojo con los compiladores que cambian esto a falso.

La siguiente es una técnica muy utilizada. En lenguaje C, es posible hacer la conversión de apuntador de doble a entero como en el código 3.16.

Código 3.16: Conversión de apuntadores de doble a entero en lenguaje C.

```
1 int *p;
2 float x;
3 p = (int *)&x;
```

Después del cast²⁸ en la línea 3, la variable `p` tiene la dirección en memoria de la variable `x` y la ve como entero, por lo que es posible hacer operaciones como si se tratara de un entero normal. A partir de la conversión, para saber si `x < 0`, basta ver si el bit de signo está prendido, con la instrucción:

Código 3.17: Verificando si el bit de signo está prendido.

```
4 if ( *p & 0x10000000 )
```

El único inconveniente es la posibilidad de que el compilador utilizado asigne sólo 2 bytes al entero, en vez de 4 que es lo que se necesita para empalmarse con los 4 bytes del float (en precisión simple). Ya casi todo compilador usa enteros de 4 bytes. Supóngase que se ha definido la macro

```
#define F12Int(f) ( * ((int *)&f) )
```

y que `f` es una variable tipo `float` (precisión simple del lenguaje C). La tabla 3.7 muestra otras posibilidades de comprobación.

El programador debe cuidar el uso de los ejemplos de la tabla y otros esquemas similares dependiendo del tipo de arquitectura. Las computadoras BigEndian colocan las posiciones de memoria en orden decreciente de la significancia de bytes, en sentido contrario de las LittleEndian. Normalmente las arquitecturas x86 son LittleEndian y otras como RS6000 son BigEndian. Esto significa que un código portable debiera verificar primero en qué arquitectura se compila para seleccionar el orden adecuado de las referencias a memoria.

²⁸Cast es la conversión de un tipo de dato a otro.

Se puede usar ...	en vez de ...
<code>if (F12Int(f) <= 0)</code>	<code>if (f<=0.0)</code>
<code>if (F12Int(f) > 0)</code>	<code>if (f>0)</code>
<code>if (F12Int(f) >= 0x404000000)</code>	<code>if (f>=3.0)</code>
<code>if (F12Int(a-b) < 0x80000000)</code>	<code>if (a<b)</code>
<code>F12Int(f) ^= 0x80000000}</code>	<code>f = -f</code>
<code>F12Int(f) &= 0x7fffffff}</code>	<code>f = fabs(f)</code>
<code>if (F12Int(f) == 0x80000000)</code>	<code>if (f == -0.0)</code>
<code>if (F12Int(f) 0x7fffffff == 0x7fffffff)</code>	<code>if (isInfinity(f))</code>

Tabla 3.7: Algunas instrucciones de punto flotante equivalentes en C. Estas conversiones son más rápidas que las del procesador pero no consideran a los números especiales. Si f es ∞ o NaN el resultado será incorrecto. La macro `F12Int` se supone definida como en el texto.

Ejemplo 37. Si una variable entera de dos bytes toma el valor en hexadecimal `0xff00`, ocho unos seguidos de ocho ceros, en una máquina *LittleEndian* las celdas de memoria estarán dispuestas primero el 255 y luego el 0, mientras que en una *BigEndian* será alrevés. Un flotante f con cuatro bytes tendrá representación $b_1b_2b_3b_4$ en una *LittleEndian* y $b_4b_3b_2b_1$ en la otra.

Por otra parte, las operaciones de la tabla 3.7 son más rápidas que las del procesador, pero no consideran los valores especiales. Si f es NaN, el resultado puede ser inválido). El procesador considera todos los casos y regresa el valor correspondiente, que es lo correcto. Las versiones de la tabla se deben usar si se está seguro de que los valores no son especiales.

Como ejemplo de cómo identificar el valor ∞ en muchos lenguajes, consúltese Infinity en la página web <http://rosettacode.org>. Es fácil apreciar cómo en unos lenguajes, los diseñadores se preocuparon por el ambiente numérico, mientras que en otros no pensados para cálculos numéricos los programadores requieren de mucha pericia.

3.3.4. Decodificando NPF

Otra técnica que se emplea con frecuencia es utilizar las uniones de C, pues como los datos comparten memoria, se pueden encimar flotantes con enteros o caracteres para tener acceso a los bits del flotante.

Usando un arreglo de tantos bytes como el tamaño del tipo de PF, por ejemplo para la precisión simple (4 bytes).

Código 3.18: Unión para decodificar variables de punto flotante.

```

1 union {
2     unsigned char Byte[4];
3     float f;
4 } U;
```

Si a **f** se le da un valor, se pueden leer los bits del campo **Byte** para decodificar y ver el formato. Por ejemplo el bit de signo se revisa con

```
5      if ( U.Byte[3] & 0x80 )
```

Luego del bit de signo, siguen 8 de exponente, 7 bits en **Byte[3]** y uno en **Byte[2]** (el más alto), por lo que el valor del exponente puede obtenerse con

```
6      e = ( ( U.Byte[3] & 0x7f ) << 8 ) |
7          ( U.Byte[2] & 0x80 )
```

El valor como entero de la mantisa (sin el bit implícito) puede extraerse con

```
8      e = ( ( U.Byte[2] & 0x7f ) << 16 ) |
9          (U.Byte[1] <<8 ) | U.Byte[0]
```

Si los enteros miden 4 bytes (la mayoría lo son), entonces es sencillo con

Código 3.19: Otra unión para decodificar variables de punto flotante en simple precisión.

```
1  union {
2      unsigned int B;
3      float f;
4  } U;
```

De nuevo, el exponente se obtiene con **U.B & 0x7f800000**. Este método es más compacto, pero es deseable que el lector pueda usar ambas posibilidades. En caso de que **f** sea de doble precisión, puede utilizarse el primer método (código 3.18), pero por fortuna algunos compiladores incluyen enteros de 8 bytes (tipo **long long**).

Usar campos de bits también puede resultar atractivo:

Código 3.20: Campos de bits para decodificar variables de punto flotante.

```
1  union {
2      struct {
3          unsigned Signo:1,
4              Exp:8,
5              Mantisa:23;
6      } B;
7      float f;
8  } U;
```

La forma de acceder a los campos es más sencilla, pero con el inconveniente de la poca portabilidad de los campos de bits, comparada con las dos técnicas anteriores, además del posible desperdicio de memoria en compiladores ineficientes. Para extraer exponente y mantisa se escribe el código adicional.

```
9  U.f=4.17;
10 e = U.B.Exp;
11 m = U.B.Mantisa;
```

Llamada	Efecto
<code>n=ceil(x)</code>	redondea hacia arriba (al menor entero mayor que x)
<code>n=floor(x)</code>	redondea hacia abajo (el mayor entero menor que x)
<code>r=fmod(x,y)</code>	el residuo de x/y
<code>n=modf(x, &f)</code>	separa la parte entera ^a de fracción normalizada de $x=n+f$
<code>m=frexp(x,&e)</code>	extrae la mantisa y el exponente de x
<code>x=ldexp(m,e)</code>	calcula $m \times 2^e$
<code>e=logb(x)</code>	extrae el exponente de x
<code>x=scalb(m,e)</code>	calcula $m \times \beta^e$ ^b

^aLa parte entera se declara como flotante

^bSi $\beta = 2$ es equivalente a `ldexp`.

Tabla 3.8: Llamadas en C de la norma de punto flotante. La lista completa se puede consultar en la norma. Estas son sólo sugerencias, por lo que no todos los lenguajes tienen tales primitivas.

En lenguajes que no tengan acceso a los bits de una variable mediante operadores nativos, siempre es posible hacer las mismas operaciones lógicas a partir de operaciones aritméticas. Obsérvese que se decodifican los bits del formato indicado por la norma, sin tener acceso a todos los bits del registro flotante.

Para apoyar el uso de la norma, lenguajes como C disponen de rutinas de alto nivel que facilitan las cosas. Por ejemplo en C, algunas llamadas útiles se presentan en la tabla 3.8, la mayoría incluidas en el apartado de operaciones de la norma.

Partiendo de las llamadas de la tabla 3.8, es más sencillo hacer uso de la norma, ya que los códigos son más portables y legibles. Funciones como `scalb`, `ldexp`, `frexp` y otras, que ya habían sido sugeridas en [53], resultan de gran utilidad.

Librerías comerciales y gratuitas incluyen funciones apegadas a la norma para el caso de que el compilador empleado no las provea. Por ejemplo, la librería científica `gsl` del proyecto GNU²⁹ incluye, entre muchas otras cosas, rutinas para la programación eficiente de punto flotante, en particular para identificar valores especiales, determinar y establecer precisión, modo de redondeo y manejo de excepciones. La librería lee la variable de ambiente `GSL_IEEE_MODE`, por lo que es fácil modificar los modos sin cambiar nada en el código fuente.

GNU Scientific Library.

²⁹El autor recomienda ampliamente al lector descargar la librería de internet y comenzar a utilizarla. Esta escrita por completo en ANSI C, por lo que no debiera haber problema de usarla con ningún buen compilador. Claro, si se utiliza con el compilador `gcc` del proyecto GNU hay mayores garantías.

3.3.5. Más sobre Transformaciones Libres de Error

Ya en la sección 3.1.3 se vieron las técnicas que diseñaron los primeros científicos de la computación para ganar precisión sobre la limitada de los primeros procesadores, al momento de realizar operaciones básicas como suma o multiplicación. Una vez con la norma a favor, otros logros fueron posibles.

Considerando que las transformaciones libres de errores están basadas en resultados intermedios internos del procesador, como al aplicar la técnica de Dekker en (3.2), es necesario tener garantías de estos valores *correctivos*. En el caso de la suma, donde $x + y$ se representa con $S + e$, el *valor correctivo*³⁰ $e = (S - x) - y$ se obtiene de redondeo fiel en el procesador. Esta técnica es cada vez más ampliamente usada.

Estas cantidades correctivas tan pequeñas corren el riesgo de sufrir underflow, por lo que se requieren garantías teóricas. Estas fueron dadas por Boldo y Daumas en [37] para las operaciones básicas suma, resta, multiplicación, división y raíz cuadrada. Esto acaba con el constante requisito (o al menos lo reduce al máximo) de muchos autores desde 1965, donde se suponía que no ocurría underflow (y por supuesto overflow) para garantizar algún resultado.

En [37], se garantizan los resultados, siempre que el resultado sea un número (no una de las cantidades especiales) y estos términos se calculan según las fórmulas exactas

$$(x + y) - \text{fl}(x + y) \quad (3.10)$$

$$(x - y) - \text{fl}(x + y) \quad (3.11)$$

$$(x \times y) - \text{fl}(x \times y) \quad (3.12)$$

$$(x/y) - \text{fl}(x/y) \quad (3.13)$$

$$\sqrt{x} - (\text{fl}(\sqrt{x}))^2 \quad (3.14)$$

con lo que se da garantía de resultados indispensables para lograr la máxima precisión posible.

Los algoritmos **FastTwoSum** y **TwoSum** requieren que el redondeo sea al más cercano. En su tesis de doctorado, Priest relaja esta restricción, suponiendo sólo redondeo fiel, es decir, que $\text{fl}(x)$ sea uno de los dos NPF más cercanos a x : $\text{fl}(x) \in \{\underline{x}, \bar{x}\}$. De nuevo, el algoritmo de Priest calcula $S + t = x + y$ y se presenta en el código 3.21.

Por otra parte, contar con la función `fma` ofrece claras ventajas para algunas operaciones. Por ejemplo, el producto $a \times b = x + y$ se reduce a las dos instrucciones

$$x = a \times b \quad (3.15)$$

$$y = \text{fma}(a, b, -x) \quad (3.16)$$

³⁰La suma no evaluada $S + e$ es llamada *expansión*, por algunos autores, pero aún no es nomenclatura estándar, al igual que los términos correctivos.

Código 3.21: Algoritmo **PriestSum** que considera redondeo fiel.

```

1  double SUMAPriest(double x, double y,
2                      double *t)
3  {
4      double S,e,g,h,f;
5      if ( fabs(x)<fabs(y) ) {
6          double r=x ;
7          x=y; y=r;
8      }
9      S = x+y;
10     e = s-x;
11     g = s-e;
12     h = g-x;
13     f = y-h;
14     *t = f-e;
15     if ( d+e != f ) {
16         S = x;
17         *t = y;
18     }
19     return S
20 }
```

y sea este el algoritmo llamado **TwoProdFMA**.

Suponiendo la aritmética de la norma, es posible representar fma como la suma de 3 números (dos son insuficientes). Un interesante algoritmo, debido a Boldo y Muller en [35], logra esto y se muestra en el código 3.22. Se calcula $\text{fma}(a, b, c) = x + y + z$ exactamente, cumpliéndose además que

$$x = \text{fl}(a \times b + c) \quad (3.17)$$

$$|y + z| \leq \frac{1}{2} \text{ulp}(x) \quad (3.18)$$

y los autores agregan algunas propiedades más, garantizando su funcionamiento incluso en el caso de underflow.

Los lectores interesados en estos temas pueden encontrar mayor información en [35, 151, 290, 312, 326] e incluso [150, 289, 312] para redondeo a cero o truncamiento, de utilidad en algunas aplicaciones específicas (como en los procesadores CELL de los Playstations).

La tesis de Priest es una verdadera cornucopia para APF.

Particularmente ilustrativo es [284], de Boldo y Muller, donde presentan (ocho) funciones numéricas útiles que se programan fácilmente partiendo de la existencia de fma . Entre ellas $\text{ulp}(x)$ (de dos maneras), así como $\underline{x}$ y $\overline{x}$.

El algoritmo para $\text{ulp}(x)$ se presenta en el código 3.23. En este caso, se supone que la constante S tiene el valor $S = 2^{-p} + 2^{-2p+1}$, con p la precisión del sistema $\mathbb{F}$ empleado. Es claro que S pertenece a $\mathbb{F}$.

Lo que calcula la función ulp del código 3.23 es la distancia entre x y el

Código 3.22: Algoritmo **TresFMA** para representar fma como la suma de 3 números.

```

1  double TresFMA(double a, double b, double c,
2                      double *y, double *z)
3  {
4      double x, u1, u2, d1, e1, e2;

6      x = fma(a, b, c);
7      u1 = TwoProdFMA(a, b, &u2);
8      d1 = TwoSum(c, u2, z);
9      e1 = TwoSum(u1, d1, &e2);
10     y = (e1-x)+e2;
11     return S;
12 }
```

Código 3.23: Algoritmo **ulp** para obtener la unidad en la última posición del valor **x**. La constante **S** toma el valor $2^{-p} + 2^{-2p+1}$.

```

1  double ulp(double x)
2  {
3      double y;

5      y = fma(S, x, x);
6      return fabs(y-x);
7  }
```

siguiente número de punto flotante, es decir, se usa el hecho que

$$\text{ulp}(x) = \bar{x} - |x|$$

por lo que la función debe usarse con las reservas del caso, pues también puede definirse $\text{ulp}(x) = |x| - \underline{x}$ y el resultado diferirá alrededor de las potencias de 2 ya que³¹

$$x - \underline{x} = \frac{1}{2} (\bar{x} - x).$$

Extendiendo la idea de separar un resultado en dos variables para duplicar la precisión, Lauter en [236] representa valores con 3 variables, de tal manera que ahora se tiene $x_l + x_m + x_h$ y los bits de estas variables no se traslapan, es decir,

$$|x_m| < \text{ulp}(|x_h|) \quad \text{y} \tag{3.19}$$

$$|x_l| < \text{ulp}(|x_m|). \tag{3.20}$$

El reporte [236] incluye muchas combinaciones de operaciones, con pruebas garantizan su correctitud, además de cotas de error, redondeos y las conversiones entre formatos sencillos, dobles y triples.

³¹Para más detalles, la sección 1.2.1 (página 16).

La extensión más reciente es de Hida, presentada en [166], donde cada número se puede representar con cuatro cantidades como $x = x_0 + x_1 + x_2 + x_3$. En este caso, se necesita que

$$|x_{i+1}| \leq \frac{1}{2} \text{ulp}(x_i)$$

para i de 0 a 2. Así, x_0 es la aproximación de x y los restantes son la ganancia en precisión. Cada valor separado como esta suma debe ser renormalizado para garantizar que no exista traslape de bits. Esto se hace con un algoritmo que presentó Priest en su tesis [312].

En [166] se presentan algoritmos para operaciones entre formatos cuádruples y combinado. El problema de los algoritmos presentados por Hida es que lo hace sin demostración formal, por lo que hay posibilidades de traslapes de bits que perjudiquen los resultados. En su favor, Hida comenta que los resultados de las pruebas son muy similares a los de la biblioteca de funciones MPFUN y mucho más rápidas. Ideas similares aparecen en [345], de Shewchuk, con un poco más de formalidad.

Los ejemplos en el artículo de Shewchuk son muy ilustrativos.

Aritmética compleja

Las transformaciones libres de error se han extendido a números complejos (ver sección A.3, pág. 350). Así como $\mathbb{F}$ representa al conjunto $\mathbb{R}$, con $\mathbb{F} + i\mathbb{F}$ se representa a $\mathbb{C}$. Recuerdese que hay dos formas de medir la precisión, mostrada en las ecuaciones 3.8 y 3.9 y se prefiere la primera.

Cálculos precisos con números complejos ya se habían revisado desde el inicio de la computación, pero en años recientes se tiene algoritmos que buscan que hasta el último bit esté bien o al menos correctamente redondeado. Primeramente, se debe mencionarse el trabajo de Brent, [42], que acota el error de la multiplicación compleja, además de hacer algo de historia sobre el problema.

Brent comenta que ya Olver en [297] había encontrado una cota relativa de $\sqrt{16/3}$. En [42] se demuestra que $\sqrt{5}$ es una mejor cota³². El error relativo es realmente menor que $\frac{1}{2}\beta^{1-p}\sqrt{5}$.

Los primeros algoritmos para operar con complejos, con transformaciones libres de error aparecen en [153], de Graillat y Menisser, publicados ambos en 2007. Sólo incluyen suma y producto. La suma se presenta en el algoritmo 3.24. Se supone que $x, y \in \mathbb{F} + i\mathbb{F}$, $x = a + ib$, $y = c + id$ y se calcula $S + e = x + y$, con $S = s_1 + is_2$ y $e = e_1 + ie_2$. Graillat garantiza en su artículo que el término de error $e \leq \varepsilon_M |S|$.

Para la función `SumCmplx` es posible definir tipos complejos con el tipo de dato `complex`, que es una estructura con dos campos, la parte real y la parte imaginaria, lo que puede significar mejoras en el código.

³²Esta cota ya era conocida por Olver, pero fue un resultado que esperó 20 años para ser publicado.

Código 3.24: Algoritmo **SumCmplx** sumar dos números complejos $S = x + y$, $x = a + ib$ y $y = c + id$. Ver el texto para más detalles.

```

1  double SumCmplx(double a, double b, double c,
2                      double d, double *s2,
3                      double *e1, double *e2)
4  {
5      double x, u1, u2, d1, e1, e2;

7      s1 = TwoSum(a, c, &e1);
8      *s2 = TwoSum(b, d, &e2);

10     return s1;
11 }
```

En el caso del producto (que requiere 80 flops), el resultado se descompone como la suma de 4 valores, el primero es el producto y los otros 3 los términos de corrección, que siguen siendo equivalentes a la cota de $\sqrt{5}$.

Además, los mismos autores realizan la evaluación de polinomios complejos en [152], que se presentará en el capítulo de raíces de polinomios.

3.3.6. Más sobre la Hipotenusa

En la sección 3.1.5 se presentaron diversas técnicas para calcular la hipotenusa $c = \sqrt{a^2 + b^2}$, antes de la norma. Se aplicaba la técnica aritmética de escalar para evitar overflow. Con la norma, primero es conveniente revisar los valores especiales, pues podría ocurrir que los valores de a o b son NaN o ∞ y regresar el valor correspondiente.

Como de costumbre, la experiencia anterior a la norma es útil. Los problemas de overflow y underflow fueron resueltos con los esquemas ya presentados. Sólo falta remediar la garantía de tener el último bit correctamente redondeado.

La raíz cuadrada se incluye entre las operaciones elementales de la norma, por lo que su error relativo es menor que μ , lo que garantiza un error menor que $.5\text{ulp}$. Falta garantizar lo mismo para $a^2 + b^2$. Si $a \geq b$, puede medirse

$$h = ((a^2 + b^2) - a^2) - b^2$$

para detectar el error h . Entonces el cálculo será realmente

$$c = \sqrt{a^2 + b^2 + h}.$$

Si el resultado final c se eleva al cuadrado, la diferencia

$$g = (a^2 + b^2) - c^2$$

incluye el error de redondeo de la raíz cuadrada y es cota superior de h . El forward error es mayor que el backward error por pocas μ . Esto significa que el cálculo es estable.

Código 3.25: La hipotenusa en Fortran 2003.

```

1  real function hipot(x, y)
2  use, Intrinsic :: ieee_arithmetic
3  real x,y
4  real scaled_x, scaled_y, scaled_result
5  logical, dimension(2) :: flags
6  type (ieee_flag_type), parameter, dimension(2) :: &
7      out_of_range = (/ ieee_overflow, ieee_underflow /)
8  intrinsic sqrt, abs, exponent, max, digits, scale
9      hipot = sqrt( x**2 + y**2 )
10     call ieee_get_flag(out_of_range, flags)
11     if ( any(flags) ) then
12         call ieee_set_flag(out_of_range, .false.)
13         if ( x==0.0 .OR. y==0.0 ) then
14             hipot = abs(x) + abs(y)
15         else if ( 2*abs(exponent(x)-exponent(y)) >
16                 digits(x)+1 ) then
17             hipot = max( abs(x), abs(y) )
18         else
19             scaled_x = scale( x, -exponent(x) )
20             scaled_y = scale( y, -exponent(x) )
21             scaled_result = sqrt( scaled_x**2 + scaled_y**2 )
22             hipot = scale( scaled_result, exponent(x) )
23         end if
24     end if
25 end function hipot

```

Hipotenusa en la biblioteca MPFR

Un buen ejemplo de cálculo es el empleado por algunas librerías que usan precisión arbitraria, como la MPFR en Fortran (véase [120]).

En los documentos sobre Fortran 2003, la misma introducción incluye el código de una función que calcula la hipotenusa, como ejemplo del uso de las facilidades, reproducido con cambios menores en el código 3.25. Esta idea se publicó en [180] (junto con algoritmos para otras (importantes) operaciones con números complejos), de Hull y son un buen ejemplo de manejo de excepciones como lo indica la norma numérica.

Primero se intenta un cálculo directo y rápido (línea 9), verificando que ninguna bandera se levantó (líneas 10 y 11). De haber ocurrido, se repite el cálculo de una manera más lenta pero segura (líneas 13–22). Este es un ejemplo poco común en el sentido de que, en los cálculos numéricos, la mayoría de los casos, las situaciones excepcionales son más comunes de lo que se quisiera. Por ello lo normal sería poner el caso directo (como la línea 9 del código 3.25) al final, luego del manejo de los casos especiales.

Para garantizar la máxima precisión, primero es necesario calcular $a^2 + b^2$ con la máxima precisión posible.

Como $a^2 + b^2$ tiene 3 redondeos es tentador emplear `fma` para reducirlos a dos, pero hay dos formas de aplicarla: `fl(fl( $a^2$ ) +  $b \times b$ )` o `fl( $a \times a$  + fl( $b^2$ ))`, es

Recuérdese que esta NO es una decisión que el compilador deba tomar.

decir, $\text{fma}(\mathbf{a}*\mathbf{a}, \mathbf{b}, \mathbf{b})$ o $\text{fma}(\mathbf{b}*\mathbf{b}, \mathbf{a}, \mathbf{a})$. Esta es una mala idea, un uso demasiado inocente de fma y NO debe emplearse.

Una manera natural de aumentar la precisión de los cálculos se logra si se aplica precisión aumentada, como la técnica de Dekker, página 86, tanto a a como a b . Supóngase que $a \geq b$. Considerando variables de precisión doble, se tiene

$$\begin{aligned} a &= a_h + a_l \\ b &= b_h + b_l \end{aligned}$$

y $a_h = a$ con los 32 bits más bajos en 0 para precisión doble, similarmente con b_h . Como

$$\begin{aligned} (a_h + a_l)^2 &= a_h^2 + 2a_h a_l + a_l^2 \\ (b_h + b_l)^2 &= b_h^2 + 2b_h b_l + b_l^2 \end{aligned}$$

entonces, hay que analizar las posibilidades. Al elevarse al cuadrado, las cantidades requieren una mantisa mayor, por lo que la suma dejará afuera los bits más bajos, que están en

$$\begin{aligned} a^2 + b^2 &= a_h^2 + (b^2 + 2a_h a_l + a_l^2) \\ &= a_h^2 + (b^2 + x_l(x + x_h)) \end{aligned}$$

es decir, es más conveniente sumar los términos más pequeños primero y dejar a lo último a a_h^2 . Esta técnica es la que emplean en la librería de SUN.

Método con precisión expandida

Otra idea muy novedosa es la empleada en el trabajo [47] de 2011, presentado en el XX Symposium en Aritmética de Computadoras, donde se muestra una forma óptima de calcular la norma en 2D con redondeo fiel. Lo primero que hacen los autores es garantizar que la raíz cuadrada se pueda calcular con mayor precisión.

Para calcular la raíz cuadrada con mayor precisión lo que se hace es representarla como la suma de dos cantidades, es decir,

$$\sqrt{x} = R + e \tag{3.21}$$

y lo único que se requiere es hacer

1. $R = \text{fl}(\sqrt{x})$
2. $t = x - R^2$, calculado con un redondeo como $\text{fma}(R, -R, x)$
3. $e = t/(2R)$

y el algoritmo, llamado **SQRText**, se basa en la expansión de Taylor de $\sqrt{x}$. Lo que se demuestra en [47] es que, con este algoritmo, se cumplen las condiciones

$$|\sqrt{x} - (R + e)| < 2^{-p-1} \text{ulp}(R) \tag{3.22}$$

$$|\sqrt{x} - (R + e)| < 2^{-2p} R \tag{3.23}$$

Código 3.26: Algoritmo **Norm2D** para calcular la hipotenusa $\sqrt{x^2 + y^2}$. Ver el texto para más detalles.

```

1  double Norma2D(double x, double y
2                      double *r1)
3  {
4      double t;

6      if ( fabs(x) < fabs(y) ) {
7          t=x; tx=y; y=t;
8      }

10     bx = logb(x);
11     x = scaleb(x,-bx);
12     y = scaleb(y,-bx);
13     sx = TwoProdFMA(x,x,&px);
14     sy = TwoProdFMA(y,y,&py);
15     sh = Fast2Sum(sx,sy,&ps);
16     sl = (px+py)+ps;
17     shp = Fast2Sum(sh,sl,&slp);
18     r1 = sqrt(shp);
19     t = fma(r1,-r1,shp);
20     r2 = t/(2*r1);
21     c = slp/(2*shp);
22     r3 = fma(r1,c,r2);
23     r1p = scaleb(r1,bx);
24     r3p = scaleb(r3,bx);
25     rh = Fast2Sum(r1p,r3p,&r1);

27     return rh;
28 }

```

por lo que se concluye que e en (3.21) es una mejor aproximación del error $\sqrt{x} - R$. Este error es equivalente a calcular (asintóticamente) con el doble de precisión.

Una vez garantizada la mayor precisión de la raíz cuadrada, los autores de [47]³³ presentan dos algoritmos para calcular la norma 2D (hipotenusa), el primero ligeramente más rápido y el segundo ligeramente más preciso. Se incluye aquí sólo el segundo, considerando que las operaciones de la norma **logB** y **scaleB** están disponibles. El algoritmo se muestra en el código 3.26.

Los algoritmos **FastTwoSum** y **TwoProdFMA** aparecen en las páginas 85 y 124. Además del algoritmo, los autores comentan cuándo el redondeo es fiel, así como acotan la distancia entre $\sqrt{x^2 + y^2}$ y un punto central entre dos números de punto flotante, que es imprescindible para el redondeo correcto.

³³Honor a quien honor merece: Nicolas Brisebarre, Mioara Joldes, Peter Kornerup, Érik Martin-Dorel y Jean-Michel Muller.

Caso	Resultado
$1023 < n$	∞ (overflow)
$-1022 \leq n \leq 1023$	Número normal
$-1074 \leq n < -1022$	Número subnormal
$n < -1074$	Cero (underflow)

Tabla 3.9: Casos por considerar de 2^n . Esto cubre todas las posibilidades, asegurando una respuesta confiable de la función. Como el exponente es un entero, no hay peligro de recibir algún valor especial.

3.3.7. La función 2^n

Este es otro buen ejemplo para el uso de la norma. Por supuesto que lo más práctico es utilizar la función `ldexp(1,n)`³⁴, mandatada por la norma numérica, pero es ilustrativo ver una de tantas maneras como puede programarse, buscando legibilidad más que eficiencia, para ilustrar la forma de aplicar la norma numérica. Supóngase el caso de la precisión doble.

Lo más evidente e ineficiente en este caso sería un ciclo que multiplicara $2 \times 2 \times 2 \times \dots \times 2$, $n - 1$. Es una mala solución ya que

$$2^{34} = 2^2(((2^2)^2)^2)^2$$

requiere sólo 6 productos, a diferencia de 33. La codificación del formato flotante de las potencias de 2 facilita aún más las cosas.

Usando la idea del código 3.18 en la página 121, para superponer en memoria números de punto flotante con bytes, se puede proceder como sigue para la doble precisión.

Código 3.27: Calculando 2^n .

```
1 union {
2     unsigned char b[8];
3     double d;
4 } u;
```

En la unión `u` los 8 bytes del arreglo `b` comparten los 8 mismos bytes de la variable `d`. Ahora se deben considerar todos los casos posibles de resultados, que se presentan en la tabla 3.9.

Lo más conveniente desde el principio es dejar en ceros todos los bits del valor que se va a regresar para proceder a prender los que sean necesarios.

```
5 u.d = 0; /* Para poner a cero todos los bits */
```

El primero y último casos (overflow y underflow) requieren una señal y codificar el resultado, que no es difícil. Para el overflow se requiere construir la

³⁴El autor sugiere leer el código de la función `ldexp` de algún compilador OpenSource.

representación del ∞ , prendiendo todos los bits del exponente y apagando el resto.

```

6  if ( n > 1023 ) {
7      u.b[7] = 0x7f; /* Respetando el bit del signo */
8      u.b[6] = 0xf0;
9      raise(SIGFPE);
10     errno = ERANGE;
11     return u.d;
12 }

```

Si no se está usando POSIX, como en el código presentado, lo correcto es levantar la bandera de overflow usando `feraiseexcept(FE_OVERFLOW)`. Más sencillo resulta si se puede confiar en que la plataforma respeta la norma, haciendo

```

if ( n > 1023 ) {
    u.d = HUGE_VAL*HUGE_VAL;
    return u.d;
}

```

Para el underflow hay que regresar 0.0, por lo que es suficiente con

```

13 if ( n < -1074 ) {
14     raise(SIGFPE);
15     return u.d;
16 }

```

si se usa POSIX. O usar `feraiseexcept(FE_UNDERFLOW)` en otro caso.

Considerando que para los números normales el formato de las potencias de 2 sólo tienen un bit prendido en la mantisa (el implícito), cambiando sólo el exponente, habría que modificar los bits para obtener de manera directa nuestro resultado.

Como al exponente del formato de doble precisión se le resta 1023 al codificarse, primero se debe obtener el valor final del exponente en la representación. Si se va a calcular 2^n , en la codificación se tendrá $n + 1023$ como exponente, por lo que el número que se guardará en los bits que corresponden al exponente es $e = n + 1023$.

En doble precisión, los bits que ocupa el exponente son los 7 menos significativos de `u.b[7]` y los 4 más significativos de `u.b[6]`, por lo que el caso normal queda

```

17 Exp = n + 1023;
18 u.b[7] = Exp >> 4;
19 u.b[6] = ( Exp & 0xf ) << 4;
20 return u.d;

```

El caso subnormal debe tratarse distinto. El exponente será cero y en la mantisa habrá un bit prendido, por lo que se deben calcular el byte y el bit que le corresponden, a partir del valor del exponente, calculado como `Exp + bits de la mantisa`, 52 en este caso. Sean `Bit` y `Byte` dos variables enteras.

```

21 Pos = n+1023+52;
22 Byte = Pos/8;
23 Bit = Pos%8;
24 if ( Bit )
25     Bit = 1<<(Bit-1);
26 else {
27     Bit = 1<<7;
28     Byte -= 1;
29 }
30 u.b[ Byte ] |= Bit;
31 return u.d;

```

El valor de `Bit` debe ajustarse para tome un valor de 1 a 8 y no de 0 a 7. En el caso de `Bit==0` se debe ajustar también hacia el byte anterior. La penúltima línea prende el bit adecuado.

Si se busca mejor desempeño, el código puede reescribirse, pero de esta manera permite ver un manejo más evidente de uso del formato. Las operaciones más costosas son la división y el módulo, que en ensamblador se reducirían a una división pues el módulo se obtiene del acarreo.

3.3.8. Más de la división compleja

Retomando el problema de la división compleja, sección 3.1.6, el problema es calcular $x + iy = (a + ib)/(c + id)$. Las técnicas previas fueron el escalamiento de Smith (pag. 94) para evitar overflow y los cuidados de Stewart para evitar algunos underflow y overflows remanentes.

Solución en la librería BLAS

Finalmente, en 2000 casi todos los problemas fueron eliminados en la librería BLAS, véase Li en [243], escalando con mayor cuidado que el procedimiento de Smith, obteniendo un error relativo de apenas unas cuantas ulp, a costa de más cómputo. Escala tanto el numerador como el denominador.

Este método considera las cotas de overflow y underflow de la norma. Sean R y r como en (3.7), página 91, el mayor y menor NPF. Primero se calcula

$$m = \max(|a|, |b|) \quad y \\ n = \max(|c|, |d|).$$

En caso de que $m > R/16$, se escala con limpieza: $a = a/16$ y $b = b/16$ sin tocar la mantisa, sólo el exponente. En caso de $m < k \times rN/\varepsilon_M$, con ε_M el épsilon de máquina y k un entero pequeño potencia de 2 (puede ser de nuevo 16), se escala $a = a \times k/\varepsilon_M^2$ y $b = b \times k/\varepsilon_M^2$.

Se repite el proceso de escala con el otro número complejo $c + id$. Ahora, los 4 coeficientes están en el intervalo $(k \times r/\varepsilon, R/16)$. Se aplica el método de Smith y se resultado se escala con las operaciones contrarias.

Solución de Kahan

Uno de los mejores procedimientos, por directo, que elimina todos los problemas además de considerar los números especiales de la definición (1.11), página 20 es el de Kahan, en [199]³⁵ de 1987, donde presubstituye con software a falta de soporte del lenguaje. Se basa en la fórmula

$$\begin{aligned} x + iy &= \frac{a + ib}{c + id} \\ &= (a + ib) \frac{c - id}{c^2 + d^2}. \end{aligned} \quad (3.24)$$

Una función con la idea de Kahan se presenta en el código 3.28. Se calcula $m + in = (u + iv)/(x + iy)$.

Las funciones `fegetenv` y `fesetenv` regresan cero si cumplen su función con éxito, pero aún si fallan salvando y restaurando el ambiente de punto flotante, el cociente complejo se puede calcular. Pura profilaxis.

Claro que habría que confiar en que el producto de complejos en la antepenúltima línea ocurre como es de esperarse. La llamada `copysign(x, y)` está especificada en la norma y copia a `x` el signo de `y`. Las banderas se restauran luego de la llamada a la multiplicación para continuar con el mismo contexto numérico previo, evitando cambios producidos por la rutina `Multiplicar`. Según Kahan, la rutina no produce overflows ni underflows innecesarios y preserva la identidad

$$\left| \frac{a + ib}{c + id} \right| = \frac{|a + ib|}{|c + id|}$$

incluso en los casos ∞ y 0. El caso de los subnormales queda cubierto automáticamente. Este procedimiento y otros tantos son los que emplean las calculadoras HP, donde Kahan trabajó como consultor.

El problema de la anterior rutina es que es lenta, si se piensa utilizar en gran demanda, como grandes matrices con entradas complejas. En su favor habla el hecho de que la división se realiza poco, pero sí hay ocasiones donde su uso es inevitable. El costo de tantas llamadas a la función `scalb`.

Método de Priest

Ante esto, Priest diseñó un método publicado en 2004 en [313] que se basa también en la fórmula (3.24) usando la idea básica de

$$\begin{aligned} r &= 1/(c^2 + d^2) \\ x &= (ac + bd)/r \\ y &= (bc - ad)/r \end{aligned}$$

³⁵Artículo separado en 3 partes, el algoritmo de división (y muchos otros) está al inicio del tercero. Cualquiera que se interese par la programación de funciones analíticas debe consultar esta referencia de Kahan, condensada en un sólo documento en su página web.

Código 3.28: Método de Kahan para división compleja.

```

1  /* Calcula m+in = (u+iv)/(x+iy) */
2  double DivComplex(double u, double v,
3                    double x, double y, double *n)
4  {
5      double ;
6      fenv_t P_Env;

8      if ( isnan(u) || isnan(v) ||
9          isnan(x) || isnan(y) )
10         return u*v*x*y;

12     fegetenv(&FP_Env);

14     L = logb(MaxDouble)/2-1
15     h = logb(max{|c|,|d|})

17     K = L - entero(h+.5)
18     c = scalb(d,K)
19     d = scalb(d,K)

21     J = L - entero(h+.5)
22     a = scalb(a,J)
23     b = scalb(b,J)
24     d = x*x + y*y
25     if (d==Infinito or d==0 ) {
26         if ( |x|==d ) x = copysign(1,x)
27         if ( y != 0 ) y = copysign(1,y)
28     }
29     m+in = Multiplicar(a+ib,c+id)
30     fesetenv(&FP_Env);
31     return ( scalb(m/d, K-J) + i scalb(n/d,K-J) )
32 }

```

Es claro que los primeros problemas pueden surgir al calcular r , por lo que se necesitará escalar adecuadamente con algún factor s y calcular $r = (sc)^2 + (sd)^2$. Lo único que se requiere es que $\max(|sc|, |sd|)$ esté suficientemente cerca de 1 para calcular r y su recíproco con seguridad. El detalle fino del método de Priest radica en la selección de s y el orden de evaluación que se selecciona.

Priest cuidó todas las posibilidades, garantizando que sólo ocurre overflow o underflow cuando es realmente inevitable aritméticamente.

Una versión NO completa se presenta en el código 3.29, donde sólo falta manejar los casos especiales de valores ∞ y NaN, pero es de esperarse que no sea problema para el lector extenderla. Revísese [313] para el código completo. Se supone que los enteros son de 4 bytes y que existe el tipo `long long`.

Este código se ejecuta con precisión y velocidad, con cotas casi ideales de exactitud.

Técnica de Thomas y Tydeman

Otra idea interesante y sencilla, aplicable a muchas otras situaciones, es la de Jim Thomas y Fred Tydeman, que usaron el control de banderas de excepción de c99.

Primero guardan y limpian las banderas de excepción. Calculan el divisor $c^2 + d^2$ de manera directa y revisan de inmediato las banderas. Estas indican qué ocurrió y de ser necesario se vuelve a hacer el cálculo reescalando, siempre con potencias de la base β .

Esta técnica no puede emplearse en lenguajes donde no se puedan leer las banderas de excepción indicadas por la norma. En ese caso es inevitable buscar un buen factor de escala previo al cálculo.

Comenzando con los compiladores de Microsoft.

En 2012, en un artículo no publicado [189], analizan el problema de la división compleja, desde el punto de vista del error en los componentes (3.8). Esto lo hacen asumiendo base $\beta = 2$ y que existe la operación `fma` en el ambiente numérico. Utilizando el algoritmo compensado para determinantes de 2×2 de Kahan, presentan nuevos algoritmos de división, con errores relativos de $5\varepsilon_M + 13\varepsilon_M^2$ y $\frac{9}{2}\varepsilon_M + 9\varepsilon_M^2$. Es otra buena fuente de ideas para calcular $ab + cd$ con mucha exactitud.

Ejemplos más extensos se presentan en la segunda parte de este texto.

3.3.9. Seleccionando el lenguaje correcto

No siempre es una tarea sencilla decidir el lenguaje que se utilizará para desarrollar una aplicación. El estudiante o programador que conocen un solo lenguaje (y a veces un único compilador) deciden de inmediato, pero quien utiliza diversos lenguajes, dialectos, compiladores, arquitecturas o sistemas operativos, debe pensar con más calma.

Código 3.29: Método de Priest para división compleja.

```

1 void DivCompleja(double *v, const double *z,
2                 const double *w)
3 {
4     union {
5         long long int i;
6         double d;
7     } aa, bb, cc, dd, ss;
8     double a, b, c, d, t;
9     int ha, hb, hc, hd, hz, hw, hs;

10
11     /* Componentes de z y w */
12     a = z[0];
13     b = z[1];
14     c = w[0];
15     d = w[1];
16     /* Extraer los 32 bits mas altos para
17                                     estimar |z| and |w| */
18     aa.d = a;
19     bb.d = b;
20     ha = (aa.i >> 32) & 0x7fffffff;
21     hb = (bb.i >> 32) & 0x7fffffff;
22     hz = (ha > hb)? ha : hb;
23     cc.d = c;
24     dd.d = d;
25     hc = (cc.i >> 32) & 0x7fffffff;
26     hd = (dd.i >> 32) & 0x7fffffff;
27     hw = (hc > hd)? hc : hd;

28
29     /* |z| < 2-909 and 2-215 <= |w| < 2114 */
30     if (hz < 0x07200000 && hw >= 0x32800000 &&
31         hw < 0x47100000)
32         hs = (((0x47100000 - hw) >> 1) & 0xfff00000) +
33             0x3ff00000;
34     else
35         hs = (((hw >> 2) - hw) + 0x6fd7ffff) & 0xfff00000;
36     ss.i = (long long int) hs << 32;
37     /* Se escalan c y d, y se calcula el cociente */
38     c *= ss.d;
39     d *= ss.d;
40     t = 1.0 / (c * c + d * d);
41     c *= ss.d;
42     d *= ss.d;
43     v[0] = (a * c + b * d) * t;
44     v[1] = (b * c - a * d) * t;
45 }

```

En el mundo moderno del desarrollo de software, la integración de soluciones es el camino que se acostumbra seguir por diversos motivos, principalmente el tiempo. Posiblemente uno o ninguno de los módulos del sistema requiera cálculos numéricos correctamente redondeados, por lo que quienes diseñen o especifican los requisitos del software deberá responder a la pregunta *¿Se requieren cálculos numéricos con gran precisión en alguna parte del sistema?* La mayor parte de las veces, la respuesta será negativa y a veces dada a la ligera.

A veces, hasta un sistema financiero donde sólo se requiere de cuidar dos decimales después del punto (los centavos) llega a dar sorpresas desagradables. La inexperiencia suele tener costos altos en estos casos.

Lo que interesa aquí es el camino que se debe seguir cuando la respuesta es afirmativa. Muchos lineamientos y recomendaciones tan útiles como interesantes se incluyen en el libro editado por Bo Einarsson [103], con exposiciones clasificadas en 3 partes: errores en cómputo numérico, herramientas de diagnóstico y tecnología existente para mejorar software numérico.

Se presentan a continuación algunos aspectos generales³⁶.

Lenguajes

Actualmente (diciembre de 2012) no existe ningún lenguaje cuyo soporte sea completo, pero hay algunos que se acercan más a los requisitos de la norma.

Como se dijo al presentar la norma de IEEE, el soporte numérico no es atribución exclusiva del lenguaje, sino que es compartida con el compilador, librerías y procesador. A referirse a un lenguaje también se hace referencia a un conjunto de rutinas determinadas en la definición misma del lenguaje. Estas pueden variar en desempeño y seguridad dependiendo de la versión de cada compilador, por lo que se necesario leer la referencia que el fabricante escribe, para no asumir atributos falsos.

Hay que estar atentos a las notas de la versión y al *What's New* de cada compilador.

Cuando se trata de módulos con cálculos numéricos críticos o intensivos, los aspectos son los que más importan al seleccionar el lenguaje son: espacio, tiempo y confiabilidad. El espacio es especialmente importante en software empujado y se cuida eliminando cuidadosamente todo contenido no necesario en la versión final, como las tablas de símbolos para depuración y funciones de diagnóstico en la etapa de pruebas.

El tiempo de cómputo y la confiabilidad son un compromiso mutuo que debe analizarse con cuidado. Fortran 77 era notablemente más rápido que las versiones modernas, pero ahora se dispone de prestaciones mayores, por lo que casi todo lenguaje de alto nivel dispone de compiladores que generan código tan óptimo como es posible.

Si se desea generar código con alta legibilidad, lenguajes orientados a objetos son un buen remedio, salvo la sobrecarga de operadores intrínseca que

³⁶Muchos detalles y sugerencias están distribuidas a lo largo del texto.

puede perjudicar el redondeo (ver sección 2.2.5). Da mayor confianza escribir cada parte de un programa en el lenguaje adecuado y ligando código objeto o llamando a programas ejecutables por separado, comunicados por entrada y salida a archivos compartidos.

Como recomendación general, es mejor descartar lenguajes que no fueron diseñados pensando en cómputo numérico.

En el caso de lenguajes especializados, hay varias buenas opciones. Lo que hay que decidir es con qué compañía se desea trabajar.

Librerías

Cada compilador está acompañado de librerías, más correctamente llamadas bibliotecas de funciones, que incluye métodos y rutinas que se ligan en tiempo de compilación, automáticamente o a solicitud del programador.

Adicionalmente, existen librerías de funciones especializadas que pueden ser utilizadas independientemente del compilador y en ocasiones sin importar el lenguaje. Tal es el caso de IMSL, NAG, BLAS o LAPACK.

Parte II

Aplicaciones de la Programación Numérica

Introducción a las aplicaciones

Presentar todas las aplicaciones importantes de programación numérica es imposible a menos que se escriba una enciclopedia sobre el tema, donde se reserve uno o varios volúmenes para cada aplicación o familia de aplicaciones. Lo que sí es posible es mostrar con cierto nivel de detalle algunas que permitan ejemplificar la utilidad de la base teórica y práctica de la aritmética de punto flotante y su norma oficial con su correspondiente manejo de excepciones, extendiendo las ideas del capítulo 3.

Los tres temas se han seleccionado por la sencillez de su planteamiento, eficiente ilustración de la teoría, utilidad práctica y diversidad. Aunque se espera que esta segunda parte sea autocontenida, es necesario que el lector esté familiarizado con la mayoría de los conceptos matemáticos y de computación que se utilizarán, como programación y estructuras de datos, además de álgebra y cálculo, ya utilizados en la primera parte.

Aunque no se presentan todos los detalles posibles en los tres tópicos, sí se analizan los suficientes para orientar a estudiantes y programadores profesionales sobre maneras diversas de resolver problemas básicos, de gran utilidad en la práctica.

Los códigos que se presentan son de carácter ilustrativo y sólo en algunos casos se incluyen versiones completas y eficientes de los algoritmos. Detalles como asegurar la portabilidad hacen crecer indeseablemente los códigos, por lo que sólo se indican versiones con ANSI C, salvo unas cuantas excepciones, al igual que en la primera parte.

Primeramente se caracterizan los procesos de suma de NPF en términos de la propagación de errores de redondeo. Sumar es un proceso sencillo que se realiza con eficiencia en los procesadores modernos, pero cuando la cantidad de sumandos crece, la suma de errores de redondeo puede ser mayor de lo que puede creer el programador promedio. Por ello se han desarrollado diversas maneras de reducir este fenómeno.

En segundo lugar se presenta un estudio sobre diversas técnicas para aproximar las raíces de polinomios y de ecuaciones en general, en ambos casos para

una sola variable. Este es un problema que desde hace muchos siglos ha causado interés y alrededor del cual hay gran cantidad de teoría y métodos. Se conocen cientos de algoritmos y métodos iterativos distintos y cada año aparecen novedades en la literatura especializada, aunque en los cursos de Métodos o Análisis Numérico, siguen estudiándose unos pocos (y los mismos, que se cuentan con la mano).

En un inicio, todo el material de raíces de ecuaciones estaba en un sólo capítulo, que creció tanto que lo de polinomios se convirtió en un capítulo a parte. Aunque los métodos generales también pueden aplicarse para polinomios, se ha preferido separado, de tal forma que en el capítulo de polinomios no se analiza con detalle la teoría de punto fijo, sino sólo lo necesario para la presentación de algunos métodos.

Por último se aborda el tema de la programación de funciones elementales, donde se presentan las técnicas generales que se aplican y el estudio de el caso particular de la función exponencial. Esta es una de las funciones que la norma de IEEE [77] define dentro de las que deben estar disponibles en todo ambiente de programación de punto flotante correctamente redondeadas. Las técnicas empleadas en e^x pueden aplicarse a otras funciones y rutinas en general, por lo que se espera que el lector encuentre útiles algunas de las ideas.

Cada uno de los temas se presentan desde una perspectiva histórica, es decir, la manera en que la teoría ha evolucionado, influenciándose de otros avances. Algunos métodos que ya no se utilizan para lo que fueron pensados, están basados en una idea que sí es útil para problemas de otro tipo.

Se indican referencias suficientes para los interesados en revisar con más detalle cada tema. La bibliografía es amplia en los tres problemas, principalmente en la solución de ecuaciones no lineales, donde la exposición es notablemente mayor pues el problema es mucho más antiguo.

Quedan pendientes temas tan importantes como solución de sistemas de ecuaciones lineales, problemas de eigenvectores, optimización, integración, ecuaciones diferenciales o elemento finito, pero había que tomar una decisión³⁷. El autor espera que la selección presente sea adecuada para dar un panorama general, sensibilizar e informar al programador promedio sobre las posibilidades y limitaciones de la Programación Numérica.

³⁷Textos clásicos como [57, 63, 133, 217] pueden servir como buena introducción.

Capítulo 4

Sumatorias

La suma una de las operaciones elementales definidas para la aritmética de punto flotante. Su error relativo está acotado por la unidad de redondeo μ :

$$fl(a + b) = (a + b)(1 + \delta), \quad |\delta| < \mu. \quad (4.1)$$

Como debe recordarse, ya Dekker mostr¹ que la suma de dos números puede calcularse de manera exacta si se utilizan dos variables y las transformaciones libre de errores, como en la ecuación (3.2). Si a y b son elementos de $\mathbb{F}$ y la suma (4.1) es $s = fl(a + b)$, entonces el error

$$e = (a + b) - s$$

sirve para representar la suma exacta¹, también conocida como una compensada:

$$a + b = s + e. \quad (4.2)$$

El algoritmo **FastTwoSum** de Dekker fue presentado en el código 3.1 (página 85).

Como es importante comprender bien esto, se ilustra con la figura 4.1, en donde $t - b = e$. En caso de que a y b tengan signo contrario, se trata de una resta, pero s y e tendrán el mismo signo. Esta idea fue descubierta desde 1951, cuando se aplicó en problemas ecuaciones diferenciales, área donde la estabilidad siempre es crítica.

Si δ cumple $s = a + b + \delta$ entonces $|\delta| \leq \mu_{\text{ufp}}(a + b) \leq \mu_{\text{ufp}}(s) \leq \mu|s|$ es un refinamiento de (4.1).

Como el error $e = ((a + b) - a) - b$ es algebraicamente cero, un compilador que simplifique la expresión estará cometiendo un serio error de punto flotante. En lenguaje C esta es una prohibición explícita para los compiladores.

¹Esto es válido si el modo de redondeo es al más cercano.

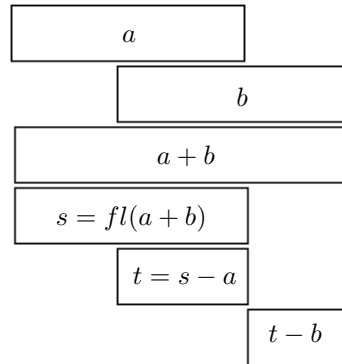


Figura 4.1: Suma compensada. Se supone que las variables a y b han sido normalizadas, utilizan toda su mantisa y que $a > b$, por lo que los bits menos significativos de b se pierden.

En [219], Knuth presentó el algoritmo **TwoSum** (código 3.4) que calcula la suma $a + b$ de otra forma, sin importar cuál es mayor y se reproduce de nuevo:

$$\begin{aligned}
 x &= fl(a + b) \\
 z &= fl(x - a) \\
 y &= fl((a - (x - z)) + (b - z))
 \end{aligned} \tag{4.3}$$

y $a + b = x + y$, válido incluso en presencia de underflow y evita la penalización del pipeline de los procesadores modernos del algoritmo de Dekker. Para más detalles en la suma de dos números o la representación con más de una cifra consultar las secciones 3.1 y 3.3.

4.1. Suma de arreglos o conjuntos de números

Hasta raro es encontrar programas grandes donde no se requiera hacer alguna suma.



Lo que se estudia en este capítulo son los errores de redondeo acumulados en la suma de un conjunto de números de punto flotante. Es algo que de manera rutinaria se incluye en casi todo tipo de programas, al calcular un promedio, resultados finales de un torneo, desempeño total de un evento y muchos más. Lo ejemplos están al alcance de la mano.

Aún cuando no se requiera la suma como valor final, sí se requiere como cálculo intermedio en muchas situaciones, como en graficación, integración, operaciones con matrices, nóminas, simulaciones y una lista interminable.

En la práctica, el conjunto que se suma siempre es finito pero no necesariamente determinado. En el caso de un arreglo se sabe por anticipado cuántos elementos son, pero no en el caso de una serie (como un polinomio de Taylor), donde se requiere sumar hasta cumplir con cierto criterio. Incluso es posible que se tengan que sumar los elementos de un arreglo hasta que se cumpla alguna condición.

En esencia el problema por resolver se plantea como

$$S = \sum_{i=1}^n a_i \quad (4.4)$$

donde cada $a_i \in \mathbb{F}$ es un NPF. Como cada suma puede acarrear un error de redondeo por (4.1), se espera que el resultado final exacto S difiera de $\text{fl}(S)$, pues se usan formatos y operaciones nativas de los procesadores modernos y no multiprecisión emulada en software.

Los problemas de redondeo en la suma de NPF fueron reconocidos por Wilkinson desde los inicios del cómputo numérico, como comenta él mismo en [395]. Wilkinson demostró que la suma de n números similares en magnitud con x puede tener un error de hasta $n^2 10^{-p}$, donde p es la cantidad de posiciones en la mantisa, como de costumbre. Ese es el error máximo, pero el error más probable fue calculado en 1996 por Mikov en [272] como $n^{1.5} 10^{-p}$.

El problema resultó no ser tan trivial como debió parecer al principio, por lo que aún a principios del siglo XXI se publican trabajos presentando nuevas formas de reducir los problemas de falta de precisión al efectuar sumas. 

Hay dos recursos básicos que se deben considerar al diseñar casi todo algoritmo: el tiempo y el espacio. Adicionalmente, aquí interesa lo eficiente del algoritmo, medido en términos de su precisión, exactitud, estabilidad y condicionamiento, conceptos todos definidos en la primera parte. La suma es un proceso bien condicionado pues cambios pequeños en los datos de entrada deben producir cambios pequeños en la suma final, aunque sí hay casos patológicos donde la suma da resultados muy alejados del resultado correcto.

El número de condición de la suma se define como

$$\text{cond}(A) = \frac{\sum_{i=1}^n |a_i|}{\left| \sum_{i=1}^n a_i \right|} \quad (4.5)$$

y una cota para algunos algoritmos es $\mu^2 \times \text{cond}(A)$, aunque un buen algoritmo debiera no depender de $\text{cond}(A)$.

La estabilidad, precisión y exactitud dependen del procedimiento empleado para sumar los datos y es lo que se analizará en las siguientes secciones, en las que se presenta diversos algoritmos de suma, desde el método más evidente de ir sumando los números en el orden que se presentan hasta la versión cuya cota teórica garantiza la máxima exactitud posible, bajo ciertos supuestos del ambiente de programación.

Lo que se intenta es reducir el error absoluto final, representado por 

$$e_S = \left| \text{fl} \left(\sum_{i=1}^n a_i \right) - \sum_{i=1}^n a_i \right|. \quad (4.6)$$

Si $n\mu \leq 1$ y $\max\{|a_i|\} \leq$  entonces

$$e_S \leq \frac{n(n-1)}{2} \mu T. \quad \text{} \quad (4.7)$$

que es una cota para (4.6) cualquiera que sea el orden en el que se sumen las a_i e incluso cuando nT causa overflow.

Esto es de gran importancia la
momento de diseñar un programa.

Cuando un programador utiliza funciones de alguna librería gratuita o comercial, usa objetos de una clase programada por otros o se integra soluciones pegando diversas rutinas en general, hay una pregunta importante: ¿qué tanta precisión y exactitud se requiere en el sistema o bloque que se está desarrollando? La respuesta pudiera indicar que el error relativo tolerable es suficientemente grande como para no preocuparse por la minucia numérica. Este caso se tiene al promediar edades de personas o calificaciones o procesos gráficos.



Pero si la respuesta es que se requiere la máxima precisión posible, como el caso aplicaciones científicas, simulaciones por computadora, amortización a largo plazo, o sistemas financieros donde los redondeos pueden ser perjudiciales, ¿qué garantía se tiene del desempeño si se desconoce cómo se realiza algo tan sencillo como la suma de arreglos? Un desarrollador competente debe poder enfrentar este caso sin problemas, al menos sabiendo que hay métodos para reducir los errores.

En ambas preguntas se requiere conocer la cantidad de elementos que se van a procesar. Un grupo de 30 estudiantes o 500 derechohabientes de una clínica son poco trabajo. Pero el inventario de una gran empresa, estadísticas de encuestas de salida durante elecciones, muestreos automatizados durante una simulación a largo plazo, donde puede haber cientos de miles de elementos o hasta millones, requiere de un procesamiento más cuidadoso de la información.

Al menos los más comunmente ci-
tados.

Casi cada nuevo artículo sobre procesos de suma incluye un estudio sobre los métodos anteriores. A la fecha siguen siendo pocos métodos comparados con los empleados para aproximar las raíces de ecuaciones, por lo que es posible considerarlos casi todos. Ya Higham dedicó un capítulo de su libro [168] a la suma, basado en su artículo [167]. También puede consultarse el artículo [267] de John Michael McNamee, que incluye el código de dos algoritmos de suma bastante eficientes.

4.1.1. Interfaces para los algoritmos de suma

Ojo: $S \equiv fl(S)$ pues S es la suma
exacta.

En este capítulo, A es el arreglo (o conjunto) con n elementos a_i , S o **Suma** es la variable donde se acumula la suma y e es el error absoluto. Se supondrá en general que se usa precisión doble. Es posible adelantar que actualmente no hay ningún algoritmo que garantice la cota perfecta para una cantidad arbitraria de sumandos, pero si para conjuntos de gran tamaño.

Lo que un programador desea tener es una función que trabaje de manera transparente. Lo más sencillo es simplemente enviarle el conjunto A como arreglo y que la función regrese la suma, en algo que tendría el prototipo

```
double Sumar(double const A[] , int n);
```

donde el arreglo se declara **const** para evitar que sea modificado dentro de la función. El valor n se requiere para conocer la cantidad de elementos. El programador utilizará la función como caja negra, sin saber lo que ocurre adentro. Por ejemplo, el segmento de código siguiente.

```
    CapturarCalificaciones( Calif , n);
    S = SumarCalificaciones( Calif , n);
    Promedio = S/n;
```

Lo que no está previsto en esta interfaz es qué hacer en el caso de que la suma sea ∞ o NaN, que son posibles valores de retorno de la función **Suma**. Lo correcto sería verificar el valor de S luego de la llamada a **Suma** o bien leer las banderas del procesador. La forma más sencilla es comprobar con `if ( isnan(S) )` y `if ( isinfinity(S) )` y tomar la medida adecuada.

En el caso de saber que el conjunto A tiene un número de condición alto, entonces es prudente disponer de una función que utilice parámetros como el error relativo máximo esperado (determinado por el programador) o el número de condición (calculado por la función).

Por otra parte, puede ser que por el tipo de aplicación que se está programando sea preferible que la suma se realice con la mayor exactitud posible a costa del tiempo empleado o al revés, se requiere la suma en el menor tiempo posible, quedando la exactitud en segundo término.

También es posible que, para favorecer el desempeño de un algoritmo, sea preferible que la función sí modifique el arreglo A a diferencia de lo indicado en la primera interfaz. De cualquier manera, el programador puede sacar copia del arreglo antes de la llamada a la función suma, si considera conveniente tener la información original para después.

Una librería completa requiere una función suma con varios parámetros que servirían para que internamente sea llamada la función adecuada, dependiendo de los mismos. En librería de este tipo tal vez sea preferible tener una estructura donde se incluya el arreglo, su tamaño, número de condición, la suma, error relativo, si se puede modificar y otros valores².

4.2. Algoritmos de suma recursiva

La suma directa es lo primero que se utilizó por ser el método natural. Consiste en el código 4.1.

Como las líneas 5 y 6 se realizarán de manera ininterrumpida, se espera que la variable **Suma** siempre esté en un registro, por lo que cuenta con los bits de

²Una clase con métodos sería agradable si la sobrecarga de operadores implícita en todo lenguaje orientado a objetos no causara problemas de redondeo (ver página 60).

Código 4.1: Algoritmo normal de suma.

```

1  double SumaNormal(double const *A, int n)
2  {
3      double Suma=0;

5      for ( i=0; i<n ; i++ )
6          Suma += A[i];

8      return Suma;
9  }

```

guardia, redondeo, el sticky y posiblemente bits extra. Aún con esa ayuda, el código 4.1 tendrá problemas de precisión.

El algoritmo básico es recursivo pues puede plantearse como

$$S_1 = a_1 \quad (4.8)$$

$$S_{i+1} = S_i + a_{i+1} \quad (4.9)$$

$$S = S_n \quad (4.10)$$

por lo que también se le conoce con ese nombre: *suma recursiva*. La complejidad temporal y espacial es $O(n)$ ya que recorre el arreglo de n elementos, utilizando espacio extra constante $O(1)$ por la variable **Suma**, que no es significativo.

En algunos lenguajes se tiene rutinas que realizan la suma, pero internamente deben aplicar algún algoritmo como 4.1 o algo más eficiente. Para comprender los problemas potenciales de una suma se presentan dos casos patológicos a continuación.

4.3. Sumas problemáticas

El algoritmo básico presentado en el código 4.1 falla gravemente cuando se presentan arreglos con un acomodo mal intencionado. Por ejemplo, como la unidad de redondeo μ no altera al 1, es decir, $1 + \mu = 1$, sumando directamente el arreglo

$$A = \{1, \mu, \mu\} \quad (4.11)$$

se obtendría 1 pues al suma $1 + \mu$ se obtiene 1 y al sumar el tercer elemento se repite el fenómeno. Sin embargo, $1 + 2\mu = 1 + \varepsilon_M \neq 1$. Con sumar en sentido contrario sería suficiente para evitar este problema.

Otro arreglo con problemas involucra el mayor número de punto flotante $M = \beta^{e_{\max}}(1 - \beta^{-p})$. Por ejemplo, la suma del arreglo

$$A = \{M, M/2, -M\} \quad (4.12)$$

debe dar $M/2$ pues se cancelan el primero con el último elemento, pero como ocurre un overflow al sumar $M/2$ a M , el resultado final es ∞ . De nuevo un reordenamiento de los sumandos eliminaría el problema.

Parece natural que el orden debe ser de menor a mayor en magnitud, pero si N es el menor NPF para el que ocurre que $N+1=N$, entonces la suma del arreglo

$$A = \{1, N, -N\} \quad (4.13)$$

resultará en 0 en vez de 1. En este caso, sumar en orden decreciente resulta ser lo correcto.

Tanto (4.11) como (4.12) y (4.13) son casos patológicos, pero los mismos fenómenos ocurren a menor escala en programas cotidianos. El problema es que normalmente se desconoce la escala en la que ocurren.

4.4. Algoritmos básicos

Los primeros algoritmos son sencillos y hasta intuitivos de alguna manera. Han sido planteados sin autoría fácil de decidir y resultan casos particulares de un algoritmo más general. Incluso la suma recursiva es también un caso particular.

4.4.1. La suma en acumuladores separados

Esta fue la primera idea reportada en la que se intentaba minimizar los problemas de redondeo. Fue publicada en [397] por Jack Wolfe en 1964. La idea es simplemente utilizar varias variables para llevar sumas parciales separando los datos por rangos.

En ese entonces, Wolfe recomendaba utilizar una variable para todos los valores entre 0 y 9.99999, otra para valores entre 10 y 99.999999 y así sucesivamente. Si un acumulador llega al límite mínimo del siguiente rango, se suma al acumulador de ese rango y el acumulador se hace cero.

La cantidad de acumuladores depende de la aplicación y el mínimo es dos, que son los que se utilizan en el código 4.2³.

Incluso no representa ningún problema enviar a la función la cantidad de acumuladores que se deben usar o hacer una estimación de la cantidad adecuada dependiendo de un análisis de los datos, pero no es necesario. Este algoritmo se usa en raras ocasiones, pero significa una idea sencilla para usarse en diversas situaciones. Independientemente de la cantidad de acumuladores, este algoritmo recorre una sola vez el arreglo, por lo que su complejidad temporal es $O(n)$ y su complejidad espacial es $O(n + k)$, con k la cantidad de acumuladores.

En 1971, Malcolm propuso una extensión de la idea de Wolfe en [254]. La idea es separar cada elemento del arreglo en la suma de números cuya representación tenga los últimos bits apagados. Esto asegura que ningún número requiere toda

³El código 4.2 es una versión minimalista pues no considera que si $S1 > Sep$, se debe hacer $S2+=S1$ y $S1=0$.

Código 4.2: Algoritmo de suma de Wolfe, con dos acumuladores.

```

1  double SumaAcumuladores(double const *A, int n, double Sep)
2  {
3      double S1=0, S2=0;

5      for ( i=0; i<n ; i++ )
6          if ( A[i] < Sep )
7              S1 += A[i];
8          else
9              S2 += A[i];
10     return S1+S2;
11 }

```

la mantisa y de esta forma la etapa de normalización y renormalización no afectan la precisión.

El arreglo de tamaño n se convierte en un arreglo de al menos $2n$, dependiendo en cuantas partes se separe cada elemento a_i , lo que puede hacerse con técnicas como las de Dekker o Linnainma (ver la página 86) o manipulando directamente la representación de los NPF⁴. Una vez separados los números, se aplica el algoritmo de Wolfe. Lo ideal es tener al menos tantos acumuladores como partes en que se separe cada valor, lo que implica un aumento de la complejidad espacial, pero poco significativo.

Implementaciones modernas del algoritmo de Malcolm extraen el exponente de la representación de IEEE del valor a_i , como en el código 3.18 y le aplican alguna operación para convertirlo en el índice del acumulador que le corresponde a ese valor. La operación más común es una división, pues de esta manera, varios exponentes consecutivos comparten el mismo acumulador, como se espera. En [267], esta versión resultó más eficiente que algoritmos más sofisticados.

Si acumuladores y datos miden igual no hay beneficio.

3 bits en vez de 2 servirían para agrupar 8 exponentes, lo que podría ser adecuado para arreglos grandes.

En el código 4.3 se supone 64 acumuladores con precisión doble para un arreglo de números en precisión simple. De las líneas 13 a la 17, los números a_i son clasificados por el rango de su exponente, al recorrer los bits de la representación el total de bits de mantisa+2 para agrupar 4 exponentes. Eso decide a qué acumulador se agrega cada elemento, convertido a precisión doble. En el ciclo de las líneas 21 y 22 se suman los acumuladores.

En las líneas 24 y 25 se obtiene el exponente de la suma y en la 26 se extrae la suma doble del acumulador correspondiente. Se suman de nuevo los acumuladores y se hace la corrección final en la línea 30.

Al código anterior habría que agregarle revisiones adicionales para evitar overflow en el arreglo de acumuladores en el caso de que n sea muy grande. Lo más sencillo es pasar lo del acumulador en riesgo a un acumulador mayor⁵ con la corrección correspondiente. Algo que no agrada es que la complejidad temporal

⁴Por supuesto, en esa época Malcolm no utilizó la representación interna porque era demasiado dependiente de la arquitectura.

⁵Al acumulador que correspondería si lo acumulado fuera el siguiente a_i .

Código 4.3: Algoritmo de suma con múltiples acumuladores.

```
1 float SumAcum(float A[], int n)
2 {
3     union {
4         int n;
5         float f;
6     } U;
7     float sum_float;
8     double Acum[64], sum_double, doublex, corr;
9     int iexp, i;

11    for ( i=0 ; i<64 ; i++)
12        Acum[i]=0.0;
13    for ( i=0 ; i<n ; i++) {
14        U.f = fabs(A[i]);
15        iexp = U.n>>25;
16        Acum[iexp] += (double)A[i];
17    }

19    sum_double = 0.0;
20    for ( i=0 ; i<64 ; i++)
21        sum_double += Acum[63-i];
22    sum_float = (float)sum_double;

24    U.f = fabs(sum_float);
25    iexp = U.n>>25;
26    Acum[iexp] -= sum_double;
27    corr = 0.0;
28    for ( i=0; i<64; i++)
29        corr += Acum[63-i];
30    sum_double += corr;

32    return (float)sum_double;
33 }
```

del algoritmo depende del rango de los exponentes de a_i .

4.4.2. Suma en orden de magnitud creciente.

Para evitar el fenómeno de (4.11), lo correcto y más sencillo es ordenar el arreglo en magnitud creciente, para comenzar a sumar los números de menor magnitud en valor absoluto primero. Implica ordenar los datos y luego aplicar el algoritmo recursivo. Ya Wilkinson había observado que en ese orden, el error (4.6) era menor. La complejidad temporal debe incluir el ordenamiento, con lo que se eleva a $O(n \log n)$ para ordenamientos como quick sort.

Una modalidad parecida es ordenarlos en el sentido contrario, pero el error por redondeo puede ser mayor. Este orden sólo funciona en casos como el arreglo (4.13). En general, se sabe que el orden decreciente es mejor cuando la suma es mucho menor que la magnitud de los elementos de arreglo, es decir, cuando hay mucha cancelación. En cualquier otro caso es mejor el orden creciente.

También se ha experimentado combinando esta técnica con la de acumuladores. Para programarlo, lo único que se requiere es agregar una instrucción al código 4.1, el ordenamiento del arreglo antes del ciclo de la línea 4, por lo que no se presenta el listado.

Cualquier algoritmo de ordenamiento es bueno, de preferencia uno que no demerite el desempeño demasiado.

En el aspecto del mejor orden, Robertazzi y Schwartz publicaron [320], en el que determinan la influencia del orden en la exactitud del resultado. Calculan el error promedio final en términos de la media y la varianza de los datos, suponiendo distribuciones uniforme y exponencial para los órdenes y algoritmo básicos (5 en total).

El estudio anterior es interesante pues el error esperado es menor que las cotas típicas, que siempre miden lo peor que puede pasar. Entre otras cosas, revela lo que es de esperarse: el orden creciente es el que menos errores ocasiona y el orden decreciente es 2.6 veces mayor.

4.4.3. Variantes de Demmel y Hida

En 2002, Demmel y Hida presentan en un reporte técnico [91] una variante donde ordenan el arreglo en orden decreciente del exponente de la representación de IEEE, suponiendo que la suma parcial S_i posee más bits extra que cada a_i , es decir, hay registros con precisión extra. Posteriormente publicaron [92, 93] con mejoras, otros detalles y aplicaciones. En la práctica esto sí aplica por la arquitectura de los procesadores en cuanto a los registros flotantes.

La variante trabaja independientemente de $\text{cond}(A)$, acotando el error como

$$e_S \leq 1.5 \text{ulp}(S)$$

aunque si $S = 0$, la suma es exacta siempre.

Supóngase que p es la cantidad de bits de la mantisa, considerando el bit implícito y que P es la mantisa del registro flotante. Demmel y Hida demuestran

Se supone que $P > p$.

que si el redondeo es al más cercano y si hay undeflow gradual, entonces es posible sumar un conjunto de hasta

$$n \leq \left\lfloor \frac{2^{P-p}}{1 - 2^{-p}} \right\rfloor + 1 \quad (4.14)$$

números garantizando una suma correctamente redondeada hasta 1.5 ulp⁶. Si la cota se rebasa, puede perderse toda la exactitud, lo que significa un error relativo de hasta 1.

Si se utiliza como acumulador una variable de precisión doble y se suman datos en precisión simple, es posible sumar hasta

$$\frac{2^{53-24}}{1 - 2^{-24}} \approx 5.368\,709\,44 \times 10^8$$

elementos. En el caso de un acumulador de precisión cuádruple, se pueden sumar hasta

$$\frac{2^{113-53}}{1 - 2^{-53}} \approx 1.152\,921\,505 \times 10^{18}$$

números de precisión doble. En ambos casos se garantiza un error relativo bajo.

En el caso de confiar sólo en los registros de la computadora, para datos en precisión doble en Intel, con registros flotantes donde la mantisa utiliza 64 bits, se podrían sumar hasta

$$\frac{2^{64-53}}{1 - 2^{-53}} = 2048$$

números con la misma garantía.

Adicionalmente, Demmel y Hida presentan dos variantes que ahorran tiempo en el ordenamiento de las a_i pues es lo que más tiempo consume. Como los exponentes son muchos menos que el total de elementos, pues cada exponente puede agrupar muchos números, es de considerarse utilizar ordenamientos con complejidad temporal lineal, como el ordenamiento por conteo, que sólo requiere un poco más de espacio. De tratarse de precisión doble, el rango de los exponentes es de poco menos de 2000. La primera variante consiste en ordenar sin considerar los bits menos significativos del exponente, lo que resulta en un orden parcial.

La otra variante la aplican al algoritmo de la sección 4.4.1 de acumuladores. La cantidad óptima de acumuladores depende de la cantidad de bits del exponente, E , pues se utiliza la idea de Malcolm (como en el último párrafo de la sección 4.4.1). En el caso de un acumulador por exponente, la cota (4.14) se reduce a

$$n \leq 2^{P-p-E} \quad (4.15)$$

lo que es bastante pesimista.

⁶En su publicación dan diversas cotas dependiendo de la relación entre p y P .

Código 4.4: Algoritmo de suma por inserción o cascada.

```

1  double SumaCascada(double *A, int n)
2  {
3      double S=0;

5      while ( n>1 ) {
6          for ( i=0; i<n/2 ; i++ )
7              A[i] = A[2*i]+A[2*i+1];
8          n/=2;
9      }
10     return A[0];
11 }

```

Por ello también es posible reducir la cantidad de acumuladores a $N = \lceil P/p \rceil$ y ordenar los números que corresponden a cada acumulador para aplicar el algoritmo original (con los a_i de cada acumulador en orden), lo que aumenta la cota (4.15).

4.4.4. Suma por inserción o de cascada

En 1970, Peter Linz propuso en [246] un algoritmo en el que los elementos del arreglo se suman por parejas, produciendo al final de la primera etapa un arreglo con la mitad de los elementos. El proceso se repite hasta que queda un arreglo de un solo elemento, que es la suma total.

Si se supone que el tamaño del arreglo es una potencia de dos por simplicidad, el código 4.4 implementa el algoritmo.

En este caso el arreglo no se recibe como `const` como en (4.1) porque es modificado. Cada iteración del ciclo `while` recorre el arreglo hasta la mitad de la iteración anterior.

Como es fácil de ver, este algoritmo puede preferirse en el caso de contar con arquitectura paralela pues el tiempo de cómputo se reduce de $O(n)$ a $O(\log n)$.

4.4.5. Cota de error de los métodos anteriores

En todos los algoritmos anteriores se realizan de una u otra forma las $n - 1$ sumas individuales requeridas, por lo que en cada ocasión que se repite el paso (4.9) y ocurre

$$\text{fl}(S_i + a_{i+1}) = (S_i + a_{i+1})(1 + \delta_i), \quad |\delta_i| < \mu.$$

por lo que la cota del error relativo en la suma final s es

$$e_S \leq (n - 1)\mu \sum_{i=1}^n a_i \quad (4.16)$$

Código 4.5: Método de suma de Pichat.

```

1  double SumaPichat(double const *A, int n, double Tol)
2  {
3      double Suma, Sa, SumaErr;
4      double *Err;

6      /* Memoria para el arreglo de errores */
7      if ( !(Err = (double *)malloc(n*sizeof(double)) ) )
8          return NaN; /* No hay nada mejor */

10     Suma=A[0];
11     for ( i=1; i<n ; i++ ) {
12         Sa = Suma;
13         Suma += A[i];
14         Err[i-1] = (Sa-Suma)+A[i];
15     }
16     SumaErr = SumaPichat(Err,n,Tol);
17     if ( fabs(SumaErr)<Tol )
18         Suma += SumaErr;

20     return Suma;
21 }

```

como máximo pesimista. Una cota más exacta la presenta Higham en [167] y es

$$e_S \leq \frac{n\mu}{1-n\mu} \sum_{i=1}^n |a_i|. \quad (4.17)$$

Algunos autores sugieren utilizar esta cantidad para decidir si se requiere un algoritmo más preciso. Si $n\mu/(1-n\mu) \sum_{i=1}^n |a_i| > \epsilon|S|$ para la tolerancia ϵ entonces aplicar un algoritmo más preciso.

Este es uno de los parámetros que pueden manejarse en la interfaz.

Los tres algoritmos anteriores no mejoran notablemente el procedimiento normal de suma, con excepción de las variantes de Demmel y Hida.

Es necesario aclarar que las cotas para el error relativo son pesimistas en el sentido de que es lo peor que puede ocurrir, pero no lo que se espera en el caso medio.

4.5. Método de Pichat

En 1972, Pichat publicó en [305] un algoritmo que refina iterativamente hasta reducir el error debajo de un umbral seleccionado. El algoritmo puede implementarse como en el código 4.5, que almacena el error de cada iteración en un arreglo, para posteriormente sumarlos y comprobar que el error total es menor que cierta tolerancia. De no serlo, se suman los errores con el mismo algoritmo y luego se restan de la suma final.

Código 4.6: Suma compensada o de Kahan.

```

1  double SumaKahan(double const *A, int n)
2  {
3      double Suma=0, Err=0, sa, t;

5      for ( i=0; i<n ; i++ ) {
6          s = Suma;
7          t = A[i]+Err;
8          Suma = s+t;
9          Err = (s-Suma)+t;
10     }
11     Suma = Suma+Err;
12     return Suma;
13 }

```

El algoritmo original de Pichat indica que la suma de errores debe repetirse recursivamente hasta no superar la tolerancia, a diferencia del código 4.5 donde se realiza una sólo vez. De una manera coloquial, el algoritmo junta la “rebaba numérica” para que al final pueda agregarse de nuevo. Su complejidad temporal depende de $\text{cond}(A)$, lo que determina la cantidad de llamadas recursivas. El mínimo peor caso es $O(4n)$ temporal y $O(2n)$ espacial, por el arreglo de errores.

Aunque es más lento que todos los anteriores, es fácilmente paralelizable. Tiene mejor precisión pero la idea básica (4.2) (ver figura 4.1) la utiliza de mejor manera el siguiente algoritmo.

4.6. Suma compensada o de Kahan

William Kahan publicó en 1965 en [195] un algoritmo de *suma compensada* en el que cada iteración se conserva el error para intentar sumarlo en la siguiente iteración. La intención original de Kahan era poder simular precisión doble⁷, en particular en Fortran⁸. El algoritmo de suma compensada se presenta en el código 4.6.

Un procesador moderno cuenta con suficientes registros flotantes como para almacenar cada variable del ciclo en alguno, por lo que se dispone de los bits suficientes para garantizar un buen desempeño numérico. Claro, realiza 4 pasos cada iteración, por lo que invierte $4 \times (n - 1)$ pasos en total.

Retomando la simbología de la figura 4.1, lo que ocurre en la línea 7 del código 4.6 es que el error e se suma a la cantidad b , que representa el siguiente término del arreglo.

En el planteamiento original de Kahan no aparece la línea 11 del código 4.6.

⁷En esa época había computadoras que truncaban o redondeaban antes de normalizar por lo que el algoritmo no funcionaba en ellas.

⁸Incluso comenta que “la precisión doble pronto será universalmente aceptable como un sustituto para la ingenuidad en la solución de problemas numéricos”.

Código 4.7: Variante de suma compensada.

```

1   if ( A[i] < Suma ) {
2       s = Suma;
3       t = A[i]+Err;
4   } else {
5       s = A[i];
6       t = Suma+Err;
7   }
8   Suma = s+t;
9   Err = (s-Suma)+t;

```

Este sencillo algoritmo logra un error relativo muy bajo de

$$e_s = (2\mu + n\mu^2)|S|.$$

La cota puede mejorarse si se considera que en cada iteración es posible sumar el error de manera selectiva, a la suma parcial o al sumando siguiente, dependiendo de la magnitud.

Años después se intentó emplear el algoritmo de suma recursiva para recuperar todos estos errores en lugar de sumarlos en cada iteración, pero el error relativo aumenta a $(2\mu + n^2\mu^2)|S|$ si la cantidad de datos n es pequeña ($n\mu \leq .1$).

Uno de los problemas de la suma compensada es que agrega el error de la iteración anterior al nuevo término a_i , pues en general se espera que la suma parcial sea mayor que cada término siguiente. Como a_i puede ser mayor que la suma parcial S_i , a la que el error ya no pudo ser agregado, entonces con mayor razón no se sumará a ese nuevo término. Parece mejor idea un segundo intento de sumar el error a la suma parcial en espera de que los bits extra rescaten algunos de los bits perdidos. Eso significa un el cuerpo del ciclo en el código 4.6 podría cambiarse por el código 4.7.

Lo obvio es que las operaciones aumentan debido a la bifurcación del **if**, que es poco recomendable si se desea explotar el pipeline de los procesadores.

4.7. Destilación de Anderson

En 1999 se publicó una idea de I. J. Anderson en [11] que utiliza la idea de representar con dos variables la suma exacta, como en 4.2 (página 145), sólo que aquí se trata de $a - b = s - e$. El algoritmo de Anderson es el siguiente:

1. Ordenar los números en orden decreciente en valor absoluto.
2. Encontrar los primeros dos que tengan signo opuesto. Sean a y b .
3. Eliminarlos de la lista.
4. Reescribirlos como en la ecuación 4.2.
5. Agregarlos al conjunto de números ordenados en las posiciones que corresponda.

6. Repetir desde el paso 2 hasta tener puros números con el mismo signo.
7. Aplicar el algoritmo de Kahan.

Este algoritmo puede ser útil cuando el conjunto de números está mal condicionado y es propenso a cancelaciones graves, como el arreglo (4.13). Al llegar al último paso, el conjunto A tiene número de condición 1 pues

$$\sum_{i=1}^n |a_i| = \left| \sum_{i=1}^n a_i \right|$$

y su suma es exactamente la misma que el conjunto original. Con esto, el algoritmo es completamente estable.

En el caso de que a y b difieran enormemente en magnitud, es preferible separarlos con algo como el código 4.8 que se conoce como *reducción*. La línea 7 no sirve si $\beta \neq 2$. Entonces s y e se regresan a la lista inicial y se continúa con el algoritmo de manera normal.

Código 4.8: Algoritmo de reducción de Anderson.

```

1      s = a;
2      while ( (a-s) != a ) {
3          e = s;
4          s = s/2;
5      }
6      s = a-e;
7      e = a-s;
```

Según el análisis de Anderson, este algoritmo tiene una complejidad temporal de $O(n^2) + O(n \log n)$ debido al ordenamiento inicial y a las posibles aplicaciones recurrentes antes de tener un conjunto A con elementos del mismo signo.

Aprovechando que el paso 4 produce un valor muy pequeño, Anderson propone una variante de su algoritmo.

1. Separar A en dos conjuntos según su signo: positivos ($\mathcal{P}$) y negativos ($\mathcal{N}$), ambos en orden decreciente.
2. Retirar el primer elemento de cada conjunto y calcular su suma como (4.2) $s + e$.
3. Agregar s a $\mathcal{P}$ o $\mathcal{N}$ según corresponda y w a un conjunto inicialmente vacío $\mathcal{E}$.
4. Repetir pasos 2 y 3 hasta que $\mathcal{P}$ o $\mathcal{N}$ esté vacío.
5. Los elementos de $\mathcal{E}$ se agregan a $\mathcal{P}$ o $\mathcal{N}$ según corresponda.
6. Se calcula la suma (con el algoritmo de Kahan) de $\mathcal{P}$ y $\mathcal{N}$. Se calcula el número de condición $|(S_{\mathcal{P}} + S_{\mathcal{N}})/(S_{\mathcal{P}} - S_{\mathcal{N}})|$. Si no es 1 se repite desde el paso 2.
7. La suma final se calcula con el algoritmo de Kahan.

Código 4.9: Algoritmo de destilación simple.

```

1  /* Repetir el ciclo for hasta que
2      $A[i] \leq 2^{-P} |A[i+1]|$  para toda  $i$  */
3  for (i=0 ; i<n-1 ; i++) {
4     s = A[i] + A[i+1];
5     if ( fabs(A[i]) > fabs(A[i+1]) )
6         A[i] = (A[i]-s) + A[i+1];
7     else
8         A[i] = (A[i+1]-s) + A[i];
9     A[i+1] = s;
10 }
```

Para evitar problemas de ciclado, Anderson recomienda no separar la suma $a + b$ entre dos cantidades tan dispares como s y e , por lo que transformaciones libres de error como la de Dekker (3.2, pag. 86) se pueden emplear.

Este algoritmo obtiene mejor precisión que el de suma compensada, según los experimentos en [11].

4.8. Suma con doble destilación o de Priest

Un algoritmo de suma que alcanza casi la cota perfecta es el de Priest en su tesis de doctorado [312], conocido como *doble destilación*, derivado a partir del de Kahan.

Para comprenderlo mejor, la idea de destilación puede verse como en el segmento de código 4.9, que es lo que se aplica en el algoritmo de Anderson.

El ciclo debe repetirse hasta que $a_i \leq 2^P |a_{i+1}|$ para toda i . Al término de la destilación, el arreglo original ha quedado sustituido por otro arreglo cuya suma es igual a la del original, pero que requiere menos bits en promedio para las a_i . El algoritmo tal cual tiene complejidad cuadrática, pero puede reducirse a $O(n \log n)$, según Priest en [312]⁹.

La idea de Priest es aplicar la compensación de Kahan sobre sí misma, como se muestra en el código 4.10. La idea de la suma compensada se aplica tres veces, en la misma forma (líneas 11-12, 14-15 y 17-19).

En el código 4.10, las variables **A**, **n**, **S** y **e** siguen teniendo el mismo significado. Las nuevas variables son para poder aplicar la compensación de Kahan sobre sí misma dos veces más¹⁰. La idea de regresar **S+e** no parece mala, tal como cuando Kahan agregó una línea similar a su planteamiento de 1964, pero debe recordarse que si la suma S ocupa toda su mantisa, $\text{ulp}(e) < \text{ulp}(S)$.

La complejidad temporal está dominada por el ordenamiento inicial, por lo que es de $O(n \log n)$.

⁹Una posible interpretación es pensar que el código 4.9 destila como el ordenamiento de burbuja, mientras que la idea de Priest aplica merge sort.

¹⁰Por lo que se dice que debiera llamarse *triple compensación*.

Código 4.10: Doble destilación o de Priest.

```

1  double SumaPriest(double const A[], int n)
2  {
3      int i;
4      double S, e, r, y, a, t, b, c;

6      S=0;
7      e = 0;
8      for (i=1; i<n; i++) {
9          r = A[i];

11         y = r + e;
12         a = r - ( y - e );

14         t = S + y;
15         b = y - ( t - S );

17         c = a + b;
18         S = t + c;
19         e = c - (S - t);
20     }
21     return S;
22 }

```

Según demuestra Priest en su tesis, en el caso de que se cumplan las condiciones de la norma de IEEE, el error relativo estará acotado por $2\mu|S|$, que es apenas el doble de la cota ideal, independientemente del número de condición del arreglo. El error se mantiene siempre que la cantidad de elementos esté acotada por

$$n \leq \beta^{p-3}. \quad (4.18)$$

En doble precisión, (4.18) implica que se pueden sumar hasta $2^{49} \approx 5.629\,499 \times 10^{14}$ manteniendo ese error relativo.

Demmel y Hida utilizan este algoritmo dentro de una de sus variantes de destilación. La cuarta de ellas de la sección 4.4.3 se aplica para reducir el tamaño del arreglo inicial y luego aplicar el método de Priest. Adicionalmente, sugieren cómo combinar sus variantes con la doble destilación para lograr el mejor resultado, dependiendo del valor de n .

Un algoritmo tan preciso como el de Priest ha logrado buenos resultados en diversas áreas de computación pues además de estable es sencillo e independiente del número de condición de A .

4.9. Método de Rump-Ogita-Oishi

El método Rump-Ogita-Oishi (ROO) se basa en la separación de valores con transformaciones libres de error pero con mayor cuidado. Antes de presentarlo es conveniente analizar su procedencia.

4.9.1. La idea de Zielke y Drygalla

En 2003, Zielke y Drygalla presentaron en [404] un algoritmo que recurre una vez más a la idea de separar los elementos a_i , pero tomando en cuenta el $\max |a_i|$, de tal manera que para sumandos pequeños, la parte mayor sea cero. La transformación libre de error debe cumplir que la suma de las partes mayores se realice sin error, lo que se repite (aplicando el mismo algoritmo una y otra vez) hasta que las partes menores son cero. Cuando esto ocurre, la suma parcial de partes mayores S_j es igual a la suma final S . Las partes superpuestas de las sumas parciales se eliminan agregándoles como acarreo en orden creciente para terminar sumando las sumas parciales en orden decreciente.

Con más detalle: primero se calcula el entero positivo k tal que $\max a_i < 2^k$ y una M tal que $n < 2^M$. Siendo p la precisión, se extraen $p - M$ bits de a_i , desde la posición k hasta $k - (p - M) + 1$, y se colocan en q_i (las partes mayores) y a un vector de acarreo. Se agrega q_i a la suma parcial S_1 , que es exacta por la selección de M .

Se continúa extrayendo los $p - M$ siguientes bits de las a_i , desde el $k - (p - M)$ hasta $k - 2(p - M) + 1$ y se agregan a S_2 . El proceso continúa hasta que las partes remanentes son cero. Debe ser claro que $\sum a_i = \sum S_j$. Los traslapes de S_j se eliminan agregándolos a S'_j con bits de acarreo en orden creciente. Ahora $\sum a_i = \sum S'_j$ y los bits no se traslapan pues forman una representación binaria exacta del resultado. Por último, las S'_j se suman en orden decreciente.

El algoritmo de Zielke y Drygalla adolece de varias cosas. En las 100 páginas de su artículo, se presenta un código de solo 7 líneas en MatLab. No se presenta un análisis y tampoco el caso de overflow¹¹. Por último, debido a un escalamiento deficiente, el rango de las a_i está muy limitado.

4.9.2. Algoritmo de ROO

Lo anterior resulta en una buena idea, pero mal concluida y presentada. Aprovechando esto, Rump, Ogita y Oishi publicaron¹², en 2007 el artículo en dos partes [328, 329]¹³ en los que subsanan todas las deficiencias teóricas y prácticas de Zielke y Drygalla.

En la primera parte, presentan un robusto algoritmo que calcula $\text{fl}(S)$ correctamente redondeado que se adapta al número de condición de A y que no

¹¹Rump, Ogita y Oishi presentan un arreglo de sólo 3 elementos con el que ese código no funciona, causando overflow, como en el caso del arreglo (4.12).

¹²Basados en [327], un reporte técnico de 2005.

¹³Una de las mayores bondades de los artículos de Rump, Ogita y Oishi es la excelente investigación de los trabajos relacionados con el tema de suma de NPF. Aunque comentan que [168, 243] son excelentes trabajos (lo que el autor no duda), la introducción de [328] es superior, lo que se nota en las 50 referencias sobre el tema. Además, lo riguroso, detallado y extenso de su exposición (60 páginas) lo hace adecuado como material de estudio para todo analista numérico. Una introducción más sencilla es su artículo anterior [295].

depende del rango de los exponentes de las a_i . No requiere de operaciones especiales ni supone precisión extra intermedia.

La segunda parte es particularmente útil para aritmética de intervalos pues incluye algoritmos con redondeo direccionado. Entre otras cosas, incluye un algoritmo especial para calcular el signo de S sin evaluar la suma final.

La mejora sobre el algoritmo de Zielke y Drygalla es indudable, tanto con la fundamentación teórica como en la implementación. Aplican la cantidad óptima de transformaciones libres de error, mejoran el escalamiento y evitan el acarreo y sumas parciales innecesarias. Con todo eso, logran la cota perfecta: un error relativo de μ . La complejidad de su algoritmo es el logaritmo de $\text{cond}(A)$.

Lo central en el algoritmo ROO es la separación de cada a_i con transformaciones libres de error. Como se presentó en (3.2), la técnica de Dekker es simplemente

$$\begin{aligned} x &= \text{fl}(a + b) \\ q &= \text{fl}(x - a) \\ y &= \text{fl}(b - q) \end{aligned} \tag{4.19}$$

para transformar $a + b$ en $x + y$ y resulta más eficiente que la de Knuth (4.3). El algoritmo ROO transforma sin errores el vector de valores a_i en dos NPF τ_1, τ_2 y otro vector con valores a'_i , de tal manera que $\sum a_i = \tau_1 + \tau_2 + \sum a'_i$ exactamente. La suma S se redondea correctamente como $\text{fl}(S) = \text{fl}(\tau_1 + (\tau_2 + \sum a'_i))$.

Para esto es necesario una separación como (4.19) que no requiera que $|a| \geq |b|$. El único requisito es que no ocurra que el menor bit no cero de a sea menor que el bit menos significativo de b . POO demuestran que si (4.19) se aplica, entonces $x + y = a + b$ y $x = \text{fl}(a + b)$ como se sabe, pero además

$$|y| \leq \mu \text{ufp}(a + b) \leq \mu \text{ufp}(x) \tag{4.20}$$

$$q = \text{fl}(x - a) = x - a \tag{4.21}$$

$$y = \text{fl}(b - q) = b - q \tag{4.22}$$

lo que significa que las diferencias (4.21) y (4.22) también son exactas. La separación de ROO para cada a_i es la siguiente. Sea σ una potencia de 2 no menor que $\max\{a_i\}$.

$$\begin{aligned} q &= \text{fl}((\sigma + a_i) - \sigma) \\ a'_i &= \text{fl}(a_i - q) \end{aligned} \tag{4.23}$$

La transformación (4.23) separa a_i en dos partes que dependen de σ , a diferencia de (3.2, página 86) en que se separa dependiendo del exponente. Eso significa que la parte mayor puede tener todos los p bits de la mantisa. La separación no es simétrica en el sentido de que $-a_i$ puede separarse distinto que a_i , debido al valor fijo $\sigma > 0$.

Código 4.11: Algoritmo de separación de ROO para vectores.

```

1 void ExtraerVector(double const *A, int n, double Sigma,
2                   double *Ap, double *Tau)
3 {
4     int i;
5     double q;

6
7     *Tau = 0.0;
8     for ( i=0; i<n ; i++ ) {
9         ExtractEscalar(A+i, Sigma, Ap+i, *q);
10        *Tau = *Tau + q;
11    }
12 }
```

Aunque la idea original de (4.23), llamada **ExtractEscalar** por los autores, ya era conocida, el análisis de ROO es nuevo y revela que si $|a_i| \leq 2^{-M}\sigma$ para algún número natural M , entonces¹⁴

$$a_i = q + a'_i \quad (4.24)$$

$$|a'_i| \leq \mu\sigma \quad (4.25)$$

$$|q| \leq 2^{-M}\sigma. \quad (4.26)$$

La separación 4.23 se aplica a vectores, lo que resulta en el código 4.11. En este caso, se ha preferido enviar la dirección de los vectores A' y q y no preocuparse por la memoria.

La función **ExtractEscalar** no debiera ser problema para el lector. Según observan ROO, el código 4.11 es naturalmente paralelizable, lo que lo hace adecuado para explotar el pipeline de los procesadores modernos. Hasta este punto, se tiene la transformación libre de error $\sum a_i = \tau + \sum a'_i$.

ROO demuestran que si $\sigma \in \mathbb{F}$, $\sigma = 2^k \in \mathbb{F}$ para algún entero k , $n < 2^M$ para $M \in \mathbb{F}$ y $\max\{a_i\} \leq 2^{-M}\sigma$, entonces

$$\sum a_i = \tau + \sum a'_i \quad (4.27)$$

$$\max\{a'_i\} \leq \mu\sigma \quad (4.28)$$

$$|\tau| \leq (1 - 2^{-M})\sigma \quad (4.29)$$

y la limitación $n < 2^M$ se requiere para que $|\text{fl}(\tau + \text{fl}(\sum a'_i))| \leq \sigma$ se cumpla. Sobre los valores de σ y M no se ha supuesto nada excepto su relación.

El mejor valor posible para M es $\lceil \log_2(n+1) \rceil$, pero para evitar el logaritmo binario se puede calcular con el código 4.12, que busca la siguiente potencia mayor que x . En algunos lenguajes, particularmente los de orientación científi-

¹⁴El planteamiento de ROO fija la base numérica en $\beta = 2$, sin suponer nunca la posibilidad de aritmética decimal. Todos sus ejemplos son considerando precisión doble.

Código 4.12: Cálculo del valor M para acotar n .

```

1  double CalcularM(double x)
2  {
3      double y, P;

5      if ( x==0 )
6          return NaN;
7      y = x/MU;
8      /* Revisar la bandera de overflow */
9      P = fabs((x+y)-y);
10     if ( P==0.0 )
11         P = fabs(x);
12     return L;
13 }

```

ca, no es difícil encontrar funciones de biblioteca que ya realizan este tipo de cálculos¹⁵.

El código 4.12 trabaja bien mientras no ocurra overflow, para lo que habría que agregar algunas instrucciones más. Con el underflow no hay problema. La constante MU puede calcularse como $(\text{nextafter}(1,2) - 1)/2$ si el compilador cumple con la norma. De no disponer de las operaciones recomendadas, se puede usar el código 1.1 de la página 27, eliminando la última línea¹⁶.

El algoritmo trabaja utilizando la transformación libre de error, que se aplica con la función **ExtractVector** y modificando el valor σ mientras que la suma parcial sea menor que $2^{2M}\mu\sigma$, que es la cota óptima para detener las iteraciones. Es en esencia lo que hace el código 4.13.

La función **SumaExacta** es simplemente la técnica de Dekker 4.19, que se presenta en el código 4.14.

Con el código 4.13, lo único que habría que hacer es hacer la llamada a la función **Transform** y luego calcular $t1+(t2+\text{Suma}(A))$, con lo que el resultado estará correctamente redondeado hasta el último bit, lo que significa un error de .5 ulp. Primer algoritmo con la cota perfecta.

El algoritmo, tiene complejidad $O(4m + n)$, donde m es la cantidad de repeticiones del ciclo **do--while**.

[283] sección 5.

4.10. Algoritmo de Eisinberg y Fedele

En 2007 se publicó [109], en el que Eisinberg y Fedele presenta un algoritmo que, a costa de cambiar la representación de los NPF, obtiene un error relativo

¹⁵Lenguajes científicos como Matlab o Mathematica ofrecen estas y otras facilidades.

¹⁶El código 1.1 calcula el épsilon de máquina ε_M y no $\mu = \varepsilon_M/2$.

Código 4.13: Transformación de a_i en (τ_1, τ_2, a'_i) .

```

1 void Transform(double A[], int n, double *Sigma
2               double *t1, double *t2)
3 {
4     double max_ai;

6     max_ai = maxAbsArr(A,n);
7     if ( max_ai == 0 ) {
8         *t1 = *t2 = *Sigma = 0.0;
9         return 0;
10    }
11    M = SigPot2(n+2);
12    Sigma_t = Pot2(M)*SigPot2(max_ai);
13    t_t = 0.0;
14    do {
15        t = t_t;
16        Sigma = Sigma_t;
17        ExtraerVector(A,n,Sigma,A,&Tau);
18        t_t = t + Tau;
19        if ( t_t == 0.0 ) {
20            Transform(A,n,Sigma,&Tau1,&Tau2);
21            return;
22        }
23        Sigma_t = Pot2(M)*MU*Sigma;
24    } while ( fabs(t_t) < Pot2(2*M)*MU*Sigma );
25    SumaExacta(t1,t2,t,Tau);
26 }

```

Código 4.14: Suma exacta de dos números (técnica de Dekker).

```

1 /* a+b se convierte exactamente a x+y */
2 void SumaExacta(double a, double b,
3                double *x, double *y)
4 {
5     double t;

7     *x = a+b;
8     t = x-a;
9     *y = b-q;
10 }

```

muy bajo en aplicaciones prácticas, según muestran los autores. No se requiere algún ordenamiento particular de A y es independiente de $\text{cond}(A)$.

A diferencia de los métodos anteriores, donde se reordena A , se separan los valores a_i o se evalúan los errores en cada suma para agregarse al resultado, la idea de Eisenberg y Fedele es modificar la representación de las a_i antes de sumar, para garantizar que la suma sea correcta sin necesidad de operaciones como las mencionadas.

El método aplica dos pasos:

1. Cada a_i es transformada a una nueva representación numérica equivalente, llamada $\widehat{\mathbb{F}}$. En ese conjunto $\widehat{\mathbb{F}}$, los números se representan como una combinación lineal de potencias de β , donde los coeficientes son enteros de $\mathbb{F}$.
2. Los números se suman de manera directa, ya que no hay error de redondeo al sumar enteros o potencias de β . En caso de que sean demasiados números, se recurre al algoritmo de Priest como apoyo en la última etapa.

Lo interesante del método de Eisenberg y Fedele es que la conversión se puede hacer rápidamente y luego sólo se trata de sumar enteros y multiplicar por potencias de β , lo que no consume tiempo.

4.10.1. Nueva representación para los NPF

Excluyendo por lo pronto los valores especiales

En la representación de los NPF se tiene que

$$x = (-1)^s \times m \times \beta^{-p}$$

$$e = \lfloor \log_\beta |x| \rfloor + 1.$$

donde $\mathbb{F}$ queda determinado por un conjunto de parámetros (ver página 20), que es equivalente a

$$x = (-1)^s \sum_{j=1}^q x_j \beta^{e-E(j)} \quad (4.30)$$

donde

$$x_j = \sum_{k=E(j-1)+1}^{E(j)} d_k \beta^{E(j)-k} \quad (4.31)$$

$$E(0) = 0 \quad (4.32)$$

$$E(j) = E(j-1) + \alpha_j \quad (4.33)$$

los q números α_j suman p (la precisión contando el bit implícito), que determinan en cuántos términos se separará x . Los d_k son los dígitos binarios 0 o 1. En [109] se demuestra la equivalencia entre las dos representaciones, lo que se logra simplemente sustituyendo una en otra.

La forma de encontrar las cantidades x_j es mediante las fórmulas

$$x_n = \frac{x}{\pm\beta^e} = \sum_{k=1}^p d_k \beta^{-k} \quad (4.34)$$

$$x_j = \left\lfloor \beta^{E(j)} x_n - \sum_{i=j}^{j-1} \beta^{E(j)-E(i)} x_j \right\rfloor. \quad (4.35)$$

La conversión de $\mathbb{F}$ a $\widehat{\mathbb{F}}$ es casi directa. Como este método es muy distinto a los anteriores, vale la pena un pequeño ejemplo de conversión de formato.

Ejemplo Sea $x = \text{fl}(\pi) = 3.141\,592\,653\,589\,7932$ en doble precisión. Si se separa en 3 partes habría que dividir la mantisa de 53 bits en 17+18+18, que son los valores de las α_j . Con estos valores y las ecuaciones (4.32) y (4.33) se calculan las $E(j) = \{0, 17, 35, 53\}$.

Ahora ya se pueden calcular los x_j a partir de las fórmulas (4.34) y (4.35) de la siguiente manera:

$$\begin{aligned} x_n &= \frac{\text{fl}(\pi)}{\beta^e} = \frac{\text{fl}(\pi)}{2^2} = .785\,398\,163\,397\,4483 \\ x_1 &= \lfloor 2^{17} \times x_n \rfloor = 102\,943 \\ x_2 &= \lfloor 2^{35} \times x_n - 2^{35-17} \times 102\,943 \rfloor = 185\,617 \\ x_3 &= \lfloor 2^{53} \times x_n - (2^{53-17} \times 102\,943 + 2^{53-35} \times 185\,617) \rfloor = 115\,44. \end{aligned}$$

De esta forma, $\text{fl}(\pi) = 102\,943 \times 2^{2-17} + 185\,617 \times 2^{2-35} + 115\,44 \times 2^{2-53}$ según la ecuación (4.30). De la misma manera, $\text{fl}(\sqrt{2}) = 92\,681 \times 2^{-16} + 235\,935 \times 2^{-34} + 211\,916 \times 2^{-52}$ o $\text{fl}(\sqrt{2}) = 474\,531\,32 \times 2^{-25} + 109\,001\,677 \times 2^{-52}$.

El algoritmo que convierte de $\mathbb{F}$ a $\widehat{\mathbb{F}}$ es sencillo a partir del anterior ejemplo. La conversión en el sentido contrario es muy simple: solo falta normalizar cada término del formato y sumar. Como $\sum \alpha_j = p$ no hay pérdida de bits. A partir de la conversión de formato, el proceso de suma comienza.

4.10.2. La suma de NPF en $\widehat{\mathbb{F}}$

Cada a_i del arreglo que se desea sumar se ha reescrito en otro formato, como suma de enteros por potencias de β como en la ecuación (4.30), o equivalentemente

$$a_i = (-1)^s \beta^{e(i)} \sum_{j=1}^q x_j \beta^{-E(j)}.$$

De todos los exponentes de β en su conjunto (para todas las a_i), se localiza el menor, sea $e_{\min} = \min\{e - E(j)\}$. Todos los exponentes se pueden reescribir

como la suma de e_{min} más una cantidad γ_i . Ahora cada valor a_i se reescribe como

$$a_i = (-1)^s \beta^{e_{min} + \gamma_i} \sum_{j=1}^q x_j \beta^{-E(j)}$$

por lo que la suma puede reescribirse de nuevo como

$$S = \sum_{j=1}^q f_j \quad (4.36)$$

donde

$$f_j = \beta^{e_{min} - E_j} \sum_{i=1}^n \text{sgn}(a_i) x_j \beta^{\gamma(i)}.$$

La suma (4.36) es una suma de q enteros, por lo que su error es menor que la suma de los n enteros originales si $q < n$, que es lo esperado. Lo óptimo es que $q = 2$, lo que implica separar cada a_i en como la suma de sólo dos enteros, cada uno multiplicado por una potencia de la base. La ventaja es que el error relativo de la suma de dos NPF es la cota ideal para la suma de cualquier cantidad de números.

Esto será posible mientras se disponga de un registro donde se pueda almacenar el entero f_j con todos sus bits. El entero máximo en un registro con t bits es $2^t - 1$, lo que limita la cantidad o magnitud de los elementos que se sumarán. En este caso, $t = p$ pues sólo los bits de la mantisa se puede utilizar. Como x_j está acotada por

$$\max\{|x_j|\} \leq \beta^{\alpha_j} - 1$$

y en el peor caso $\gamma(i) = e_{max} - e_{min}$, entonces debe cumplirse que

$$n \beta^{e_{max} - e_{min}} |x_j|_{max} \leq 2^p - 1. \quad (4.37)$$

Con $q = 2$, la ecuación (4.36) queda escrita como

$$S = \beta^{e_{min} - E_1} \sum_{i=1}^n \text{sgn}(a_i) x_1 \beta^{\gamma(i)} + \beta^{e_{min} - E_2} \sum_{i=1}^n \text{sgn}(a_i) x_2 \beta^{\gamma(i)} \quad (4.38)$$

que será exacta si la transformación de formato se ha realizado correctamente y si

$$n \leq \frac{\beta^p - 1}{\beta^{e_{max} - e_{min}}} \min \left\{ \frac{1}{\beta^{\alpha_1} - 1}, \frac{1}{\beta^{\alpha_2} - 1} \right\} \quad (4.39)$$

que se obtiene por la ecuación (4.37).

Para el formato de precisión doble, la cota es de 67 108 865 números. La cota de n requiere mayor atención. Si $q = 2$, entonces se pueden seleccionar $\alpha_1 = \lfloor p/2 \rfloor$ y $\alpha_2 = \lceil p/2 \rceil$ sin pérdida de generalidad, por lo que $\alpha_1 \leq \alpha_2$. Para que $n \geq 1$ se requiere que

$$\beta^p - 1 \geq \beta^{e_{max} - e_{min}} (\beta^{\alpha_2} - 1)$$

o equivalentemente

$$e_{max} - e_{min} \leq \alpha_2 = \lceil p/2 \rceil. \quad (4.40)$$

Si $e_{max} - e_{min} > \alpha_2$, el rango de los valores a_i impide una suma exacta, pero el resultado será $\text{fl}(S)$. La complejidad temporal es básicamente $O(2n)$ debido a que primero se recorre el arreglo para convertir a la nueva notación y luego se realiza la suma.

Si el valor de n excede su cota pero se cumple (4.40) ocurre un overflow y el tratamiento es distinto. Los sumandos de (4.38) son exactos pero la suma excede el entero máximo. Eisenberg y Fedele representan la suma en términos del módulo 2^p como

El caso en que la precisión p se queda corta.

$$\sum_{i=1}^n f_1(i) = \Delta_1 2^p + R_1 \quad (4.41)$$

$$\sum_{i=1}^n f_2(i) = \Delta_2 2^p + R_2 \quad (4.42)$$

por lo que (4.38) se escribe como

$$S = \Delta_1 2^{e_{min} - E(1) + p} + R_1 2^{e_{min} - E(1)} + \Delta_2 2^{e_{min} - E(2) + p} + R_2 2^{e_{min} - E(2)}. \quad (4.43)$$

Los cuatro números de (4.43) no contienen errores mientras se cumpla $e_{max} - e_{min} \leq \alpha_2$ y pueden sumarse con el algoritmo de Priest, con un error relativo de 2μ , el doble de cuando n no excede su cota. Sigue siendo muy rápido pues sólo se suman cuatro números en vez de n .

* * *

Pese a la cota del error perfecta de los últimos dos algoritmos, el problema seguirá estudiándose pues hay otras restricciones que son necesarias dependiendo de la aplicación. A veces se preferirá un método más veloz con un error relativo máximo predefinido o un algoritmo de ordenamiento cuyo código sea de tamaño más reducido para incluirlo en algún circuito.

Además, el ordenamiento de números en base 10 puede ofrecer más alternativas, por ejemplo, habría que estudiar un posible adaptación del método de Eisenberg.

Capítulo 5

Raíces de ecuaciones I: funciones en general

Nothing at all takes place in the universe in which some rule of maximum or minimum does not appear.

Euler.

Obtener la raíz de una ecuación de cualquier tipo es uno de los temas de gran interés desde hace muchos siglos y aún en esta época es un tema de primera importancia tanto para la teoría como para muchas aplicaciones.

A reserva de presentar el problema de manera formal, se puede decir que dada una expresión matemática con una sola variable, se trata de encontrar el (los) valor(es) de la variable que hace que la expresión valga cero. Si la expresión es $3e^x - 4$, hay que encontrar el valor x que hace que se cumpla $3e^x - 4 = 0$. Este es un problema sencillo y simplemente se despeja la variable x , encontrando que ese valor es $x = \ln(4/3)$. Pero si la ecuación es $e^x + x^2 = 0$ entonces despejar ya no es posible y es necesario emplear otros medios.

Los métodos para encontrar raíces de ecuaciones se abordan en prácticamente todos los cursos básicos de análisis o métodos numéricos, por lo que el lector encontrará fácilmente gran cantidad de información. Algunos los presentan desde un enfoque matemático adecuado para los primeros semestres de una carrera superior, donde se prueban las condiciones de convergencia y presentan los términos de error, pero sin muchos de los detalles que en la práctica son importantes. Otros los presentan principalmente como recetas, incluyendo su código, olvidando de nuevo incluir o explicar los detalles finos de programación, en especial lo referente a la APF.

Al buscar en otro libro, siempre se desea encontrar algo distinto, no lo mismo.

La intención es no repetir lo que se encuentra con mayor facilidad en los libros de análisis/métodos numéricos típicos, sino revisar las implicaciones de la APF en los detalles de programación y presentar una amplia variedad de métodos, sin olvidar su fundamento matemático. Basta con comparar el contenido de algunos de los textos que con mayor recurrencia se utilizan en estos cursos para darse cuenta de la repetición de temas: Burden [57], Chapra [63], Kincaid [217] y Gerald [133], todos traducciones de su original en inglés.

5.1. Sobre el problema de raíces en general

Rara vez ocurre que la solución de un problema real termine al calcular las raíces de una ecuación. Lo que ocurre con más frecuencia es que calcular las raíces sea un paso intermedio dentro de un proceso mayor. Por ello no sólo la precisión es un objetivo importante, sino posiblemente la eficiencia temporal.

A menos que se trate de un algoritmo fijo que se desea probar, una robusta rutina de búsqueda de raíces necesita de

1. dos o más métodos para buscar la raíz, que se complementen,
2. un algoritmo seguro para el caso de un mínimo local cuando se tenga sospecha (o seguridad) de ello y
3. una manera de ir recordando el dominio de f que se va descubriendo, al alternar (lo mínimo posible) entre los dos casos anteriores, posiblemente llevando memoria de algunas cosas o de todo.

En este capítulo se da énfasis a la variedad de métodos de búsqueda, pero se incluyen los esquemas generales que permiten la construcción de una rutina robusta.

Son cientos de métodos conocidos para la aproximación de raíces¹ por lo que no se presenta más que una selección variada e ilustrativa, determinada por los siguientes motivos.

- Métodos básicos, que sin ser necesariamente eficientes, introducen conceptos útiles.
- Métodos de importancia teórica y/o histórica.
- Métodos robustos, generalmente preferidos en las librerías comerciales de software.
- Combinaciones de métodos que garantizan convergencia, a costa de eficiencia.
- Métodos que han sido muy analizados en la literatura.

¹Basta consultar el libro de Traub [367] *Iterative Methods for the Solution of Equations* para darse una idea.

- Extensiones teóricas importantes, como los que garantizan que en uno o dos pasos rebasan la precisión numérica nativa de una computadora convencional

5.1.1. Organización del capítulo

Este capítulo consta de 3 partes principales. Primero, tras analizar la geometría de las raíces de ecuaciones y su relación con el conjunto de números de punto flotante $\mathbb{F}$, se presentan los conceptos y resultados teóricos que dan sustento formal a los métodos iterativos de aproximación.

El primer grupo de métodos que se presenta y analiza parte del método de bisección, el más sencillo de todos. Bisección da pie para mostrar muchos otros métodos que son variantes o mejoras del de bisección o que emplean la misma idea de alguna manera.

El segundo grupo de métodos parten del de Newton-Raphson y se dice que hacen uso de la geometría de la función por el hecho de emplear al menos la primera derivada. Luego de analizar sus problemas y mejoras se revisan métodos que ofrecen mayor velocidad de convergencia, es decir, presumen llegar a la raíz en menos iteraciones.

El problema de las raíces de ecuaciones puede ser abordado de manera muy formal, con conceptos de análisis funcional, topología o teoría de la medida. Mientras que esto es imprescindible en investigación científica, en la práctica los programadores requieren algoritmos claros, con cotas de error calculables. La presentación de este tema, aún siendo orientada para desarrolladores de software, se extiende un poco más allá de las matemáticas de los números reales, para ofrecer mayor información a los lectores que desean ir un poco más allá y conocer el sustento de cada método.

5.1.2. Presentación del problema

Es necesario plantear de manera precisa el problema.

Definición 1. Sea una función $f : \mathbb{R} \rightarrow \mathbb{R}$. A un valor $r \in \mathbb{R}$ se le llama raíz o cero de f si cumple con

$$f(r) = 0. \quad (5.1)$$

El problema consiste en encontrar tal valor r que sea raíz de f . Puede ocurrir que f tenga varias raíces y es posible que se necesiten todas, algunas o una en particular, a veces la mayor, la menor o la que cumpla cierto criterio.

Definición 2. Se dice que la raíz r tiene multiplicidad m si f puede escribirse como

$$f(x) = (x - r)^m g(x) \quad (5.2)$$

donde $g(r) \neq 0$, es decir, r no es raíz de g .

$x =$	1.0	2
$f(x) = x^3 - 5x^2 + 8x - 4 =$	0.0	0.0
$f'(x) = 3x^2 - 10x + 8 =$	-1.0	0.0
$f''(x) = 6x - 10 =$	-4.0	2.0

Tabla 5.1: Función $f(x) = (x-1)(x-2)^2$ con dos raíces de multiplicidad distinta. La derivación sólo preserva las raíces múltiples.

Cuando una raíz r tiene multiplicidad m , entonces ocurre que

$$f'(x) = f''(x) = \dots = f^{(m-1)}(x) = 0$$

y $f^{(m)}(x) \neq 0$. Esto se ilustra en el siguiente ejemplo.

Ejemplo 38. La función polinomial $f(x) = (x-1)(x-2)^2$ tiene 2 raíces. Una sencilla y otra múltiple, con multiplicidad 2. La evaluación de la función y sus primeras derivadas en cada raíz se muestra en la tabla 5.1.

En el caso particular (y común) de que $m = 1$, se dice que r es una *raíz simple*. En ocasiones, si se sabe que f es invertible en $[a, b]$, entonces significa que r es un *cero simple* y para la inversa de f se cumple que $f^{-1}(0) = r$.

Otra manera útil de caracterizar la multiplicidad es utilizando la cantidad

$$u(x) = \frac{f(x)}{f'(x)} \quad (5.3)$$

Smale llama a u longitud del vector de Newton y Traub lo llama función f normalizada.

conocida como *corrección de Newton*, que es de gran utilidad para definir y analizar métodos de aproximación de raíces, además de simplificar la notación. Si r es un cero simple, entonces

$$\lim_{x \rightarrow r} \left(\frac{u(x)}{x - r} \right) = 1 \quad (5.4)$$

y si r tiene multiplicidad m , entonces

$$\lim_{x \rightarrow r} \left(\frac{u(x)}{x - r} \right) = \frac{1}{m} \quad (5.5)$$

aunque es claro que estas cantidades no se pueden calcular pues r no es conocida.

La definición de raíz dada, así como el problema de encontrarla, han sido planteados para el conjunto de los números reales, pero al utilizar una computadora es claro que se trabajará con $\mathbb{IF}$ y no con $\mathbb{IR}$, así que se puede dar una definición más adecuada.

Definición 3. Un número $x \in \mathbb{IF}$ es raíz de la función si se cumple que

$$|\text{fl}(f(x))| \leq \epsilon \quad (5.6)$$

con $\epsilon > 0$ y se dice que x aproxima a r .

Este es el criterio más realista. El valor de ϵ se selecciona de tamaño que responda a las necesidades del problema que se está resolviendo. Se dice que ϵ es la tolerancia de fallo aceptable en la aproximación. Tolerancias aceptables pueden ser valores arbitrarios o dependientes de la computadora.

En el caso ideal, la aproximación x será tal que $\text{fl}(r) = x$, pero diversas fuentes de error pueden ocasionar que $\text{fl}(r) \neq x$ y aún se cumpla (5.6).

5.1.3. Geometría de las raíces

¿Cómo cruzan las ecuaciones el eje de las abscisas? En el caso de la matemática continua con exactitud infinitamente precisa no hay duda: exactamente en un punto r donde la función f es cero y alrededor toma valores positivos o negativos. Pero como la computadora no trabaja con $\mathbb{R}$ sino con $\mathbb{F}$, estamos en un caso discreto, con números no distribuidos uniformemente, sino logarítmicamente.

No puede esperarse que en todos los casos se cumpla que para algún valor $x \in \mathbb{F}$ la ecuación sea $f(x) = 0$ exactamente, aunque sí puede ocurrir para ciertas ecuaciones y raíces. Lo que se espera es que para algún valor $x \in \mathbb{F}$ ocurra $\text{fl}(f(x)) = 0$ y que ese valor sea $x = r(1 + \delta)$, con $\delta < \varepsilon_M$. Vale la pena considerar estos detalles un poco más para comprender mejor la exactitud y precisión que se puede exigir a cada método de aproximación de raíces.

Supóngase primero el caso más simple. Una raíz simple aislada (suficientemente alejada de otras raíces). En el caso ideal ocurrirá que, siendo $r \in \mathbb{F}$ un NPF, $f(r) = 0$ y que en cualquier otro punto $x \neq r$ en una vecindad cercana de r ocurra que $f(x) \neq 0$.

En ese caso ocurrirá que al evaluar f en los NPF próximos a r su valor será distinto de cero. Este es un caso discreto ya que en medio de x y $\text{nextUp}(x)$ no existe ningún otro punto. Se espera que $|f(\text{nextUp}(x))| \geq \eta$, donde η es el menor número subnormal (véase la página 32), lo que dependerá de parámetros internos y externos al problema, como

1. el sistema $\mathbb{F}(\beta, p, e_{\min}, e_{\max})$ empleado,
2. la pendiente o inclinación de f , es decir, del valor de $f'(r)$,
3. el número de condición de f ,
4. la estabilidad del algoritmo empleado para evaluar f ,
5. la precisión interna (intermedia) de los cálculos y
6. que sí ocurra la fortuna de este caso ideal.

En el caso de que $f'(r) < 0.5$ (y las otras condiciones enlistadas) es de esperarse que también $f(\text{nextUp}(x)) = 0$ o $f(\text{nextDown}(x)) = 0$.

Estrictamente se puede decir que $\text{nextUp}(x)$ es también una raíz, pero se sabe que es por la precisión limitada de la computadora. Cabe la pregunta de cuál es la raíz correcta, r o su vecino que también se evalúa como cero. Tolerar esta imprecisión es la costumbre, pero ya se vio que la evaluación con precisión mayor puede dar con la respuesta correcta, a costa de mayor esfuerzo.

La raíz $\text{nextUp}(x)$ es artificialmente producida por las imperfecciones nativas de la APF.

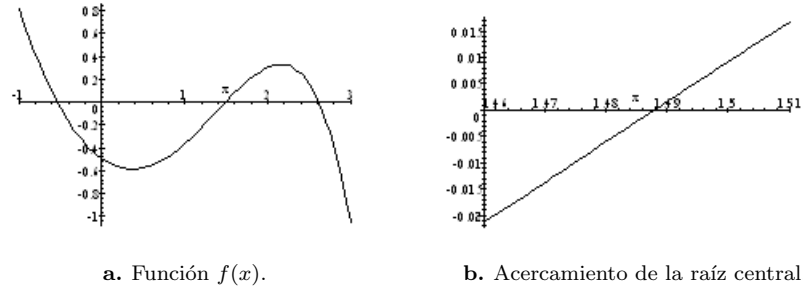


Figura 5.1: Gráfica de $f(x) = x^2 - \frac{e^x}{2}$ y acercamiento en una de las raíces.

Hay dos casos complicados con características similares: *raíces múltiples* y *raíces muy cercanas*. Si f posee m raíces en medio de dos NPF, las raíces se aproximarán con alguno de los dos, por lo que parecerán a lo más dos de ellas, con multiplicidad > 1 . En este caso la precisión no alcanza para distinguir unas raíces de otras.

Cuando una función tiene una raíz múltiple su gráfica es característica. En una vecindad de r la función es prácticamente 0. Si dos o más NPF consecutivos se evalúan como cero en f , difícilmente se podrá distinguir si son dos raíces o es un problema de precisión numérica.

5.1.4. Método gráfico

La forma más sencilla de encontrar raíces es el método gráfico.

Ejemplo 39. Por ejemplo, $f(x) = x^2 - \frac{e^x}{2}$. Su gráfica se presenta en la figura 5.1a.

Es posible intentar graficar cada uno de los cruces para aproximar visualmente el resultado. Por ejemplo, sólo el cruce del intervalo $[1.46, 1.51]$, como la figura 5.1b y hay una mejor aproximación.

Este método también es útil para saber si la función tiene un cambio de signo en la raíz o si sólo hace contacto tangencialmente con el eje x , lo cual es de gran importancia al momento de decidir el método que se empleará, ya que las raíces con multiplicidad dan muchos problemas a algunos métodos, además de aquellos que sólo trabajan cuando ocurre en cambio de signo.

Problemas del método gráfico

Además de no funcionar con raíces complejas, el método gráfico tiene otros problemas adicionales. Primero es poco exacto. Principalmente si la raíz se nece-

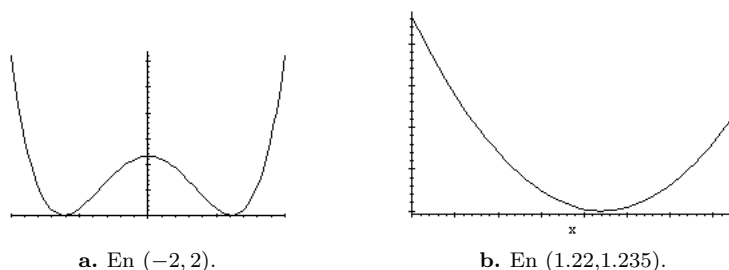


Figura 5.2: Función con dos raíces aparentes. Un acercamiento deja ver la realidad.

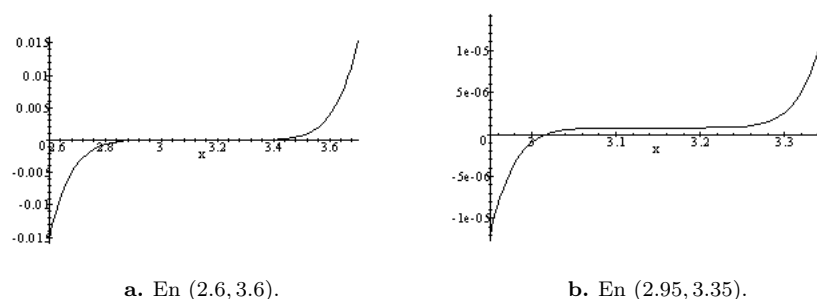


Figura 5.3: Raíz con multiplicidad. Representan un verdadero problema por su sensibilidad. El graficador empleado (Maple) se equivoca con la verdadera raíz ($r = 3.15$).

sita con muchas cifras de precisión. En segundo término, pueden ocurrir ciertos engaños visuales.

Ejemplo 40. $x^4 - 3.02x^2 + 2.3101$ es el polinomio de la gráfica 5.2. Aparentemente hace contacto en dos puntos (gráfica 5.2a), pero graficando la función en un intervalo pequeño centrado en uno de los contactos, se ve que no es así (gráfica 5.2b).

Ejemplo 41. Considérese ahora el polinomio $x^7 - 22.05x^6 + 208.3725x^5 - 1093.955625x^4 + 3445.96021875x^3 - 6512.8648134375x^2 + 6838.508054109375x - 3077.32862434921875$, cuya gráfica en el intervalo $[2.6, 3.6]$ es la de la figura 5.3.

De la figura 5.3a no es claro que 3.15 sea la raíz, aunque sólo por la forma se espera que sea múltiple. Más aún, aunque los cálculos son precisos y que las raíces se obtienen con exactitud, la gráfica en $[2.95, 3.35]$ se muestra en la figura 5.3b y la raíz pareciera estar cerca 3.02, lo que no corresponde con la función. Esta gráfica puede resultar distinta si se calcula con otra computadora

*u otro graficador*².

Una tercera razón de que el método gráfico es limitado, es que depende por completo de la interacción con el usuario. Si bien gana al momento de ser un recurso didáctico, pierde por completo al no poder considerarse como un módulo automatizado. Por demás queda comentar que su desempeño está limitado por el software graficador y la pericia de quien lo opera.

Por todos los motivos anteriores, la graficación no es el mejor método para localizar raíces. Su uso se restringe a tener una idea inicial de la posición de las raíces reales, en caso de que existan, pues puede haber sólo raíces complejas. Si obtener las raíces de un polinomio es una etapa intermedia en un programa de cómputo, este método no tiene nada que hacer.

5.1.5. Métodos no abordados en este libro.

Inevitablemente hubo que dejar material fuera de este libro. Para comenzar, no se incluye el caso de funciones analíticas, es decir, funciones definidas sobre el conjunto de los números complejos, que pueden tratarse como funciones de dos variables, la parte real y la parte imaginaria. La excepción a esto se hace en el capítulo siguiente, dedicado a polinomios, donde de manera natural el plano complejo debe abordarse.

Tampoco se abordan métodos para encontrar raíces de funciones muy específicas, donde es necesario utilizar técnicas más complejas, por ejemplo ecuaciones diferenciales o transformadas de Prüfer, lo que significa un grado mucho mayor de especialización (véanse por ejemplo [137, 307]).

De manera natural, muchos métodos se extienden y aplican a sistemas de ecuaciones con varias variables, pero no se aborda este caso, que cae de lleno en el área de optimización no lineal. Sólo en pocos casos se dan referencias en esa dirección.

La idea es complementar cursos de Métodos Numéricos, no de Investigación de operaciones.

5.2. Conceptos y fundamentos teóricos

Existe una gran cantidad de algoritmos para aproximar raíces de polinomios y de ecuaciones en general. Algunos son de propósito general, otros especializados en cierto tipo de raíces y otros provienen de combinar métodos sencillos. Su eficiencia varía y están igualmente limitados por la precisión numérica de la computadora. Se presenta a continuación un concepto con el que se califica a los métodos de aproximación.

²En este caso se usó un motor de Maple.

5.2.1. Convergencia

Además de la precisión y estabilidad con que un algoritmo numérico debe trabajar, hay varios conceptos igualmente importantes que se presentan en esta sección.

Con complejidad temporal se hace referencia qué tan rápido se acerca un algoritmo a la solución. No hay duda de que es un tema importante, pero desafortunadamente en muchos textos se hace demasiado (o único) énfasis en esto³, como si fuera más importante que la precisión o la estabilidad. La velocidad de convergencia debe verse como una medida más de la eficiencia de todo algoritmo.

Definición 4. Se dice que una sucesión $\{x_i\}_{i=0}^{\infty}$ cuyo límite es r tiene orden de convergencia k si se cumple que

$$|x_{n+1} - r| \leq C |x_n - r|^k \quad (5.7)$$

para alguna constante $0 < C < 1$, llamada constante asintótica del error.

A las x_i se acostumbra llamarlas iteradas. El orden de convergencia cumple con

$$\lim_{n \rightarrow \infty} \frac{x_{n+1} - r}{(x_n - r)^k} = C \quad (5.8)$$

que es la manera como lo definen muchos autores. Si x_{i-1} , x_i y x_{i+1} son aproximaciones sucesivas cada vez más cercanas a la raíz r , entonces el valor de k también puede aproximarse como

$$k \approx \frac{\ln |x_{i+1} - r|}{\ln |x_i - r|} \quad (5.9)$$

Algunos autores además reportan la convergencia en términos de la longitud del intervalo $[a_n, b_n]$ que contiene a x_n , o incluso como

$$k = \liminf \left\{ (-\ln(|x_n - r|))^{1/n} \right\}$$

pero la expresión (5.8) es más común. El problema de todas las expresiones anteriores es que requieren conocer a r , que es el valor que se busca. Si el orden k se requiere evaluar durante la búsqueda de la raíz (lo que es común para ir evaluando si hay problemas), entonces se emplea

$$x_{n+1} - r \approx x_{n+1} - x_n \quad (5.10)$$

como una burda pero útil aproximación, válida en el sentido que ambos términos de error se van aproximando a cero al mismo ritmo.

³Este es un fenómeno social. Un síntoma más familiar es la forma de vender (o comprar) una computadora, pues preocupa su velocidad y tamaño de memoria, nadie pregunta si cumple o no con las especificaciones de la norma de punto flotante. Hasta algunas empresas que realizan comparaciones entre equipo (benchmarks) adolecen de este mal.

En [205], Kahan incluye lo que define como tasa de convergencia, distinguiéndola del orden de convergencia, como

$$k = \liminf \left\{ \frac{-\ln(|x_n - r|)}{n} \right\} \quad (5.11)$$

valor que se espera que siempre sea positivo.

Un algoritmo de aproximación emplea un punto de inicio x_0 y cada iteración genera una aproximación mejor a la raíz de la función f que interesa. En lo sucesivo, emplearemos la notación ϕ o $x_{i+1} = \phi(x_i)$ para referirnos al algoritmo o esquema iterativo que produce tales aproximaciones o iteradas. Tal método no es único. De hecho hay una infinidad de esquemas iterativos, pero sólo interesan aquellos que logran convergencia.

Aunque ϕ sea un método iterativo con alto orden de convergencia, su desempeño en la práctica puede verse limitado por requerir mucho esfuerzo para evaluarlo. Esto motiva las siguientes definiciones, tomadas de Traub [367].

Definición 5. Si el esquema iterativo emplea W evaluaciones de f o sus derivadas, en uno o más puntos, se dice que W es el uso informacional de ϕ .

Definición 6. Se llama índice de eficiencia del esquema iterativo ϕ , denotado por E_ϕ , a

$$E_\phi = \frac{k}{W} \quad (5.12)$$

con k el orden de convergencia y W el uso informacional. Otra definición alternativa del índice de eficiencia es

$$E_\phi = k^{1/W}. \quad (5.13)$$

Tanto (5.12) como (5.13) fueron diseñadas para facilitar su uso algebraico al momento de analizar la convergencia de algoritmos. Ninguna de las dos hace uso del costo por evaluar f o sus derivadas ni de la complejidad de las operaciones para evaluar ϕ .

Una manera de obtener esquemas iterativos de alta convergencia es utilizar el siguiente resultado, también en [367].

Proposición 7. Si ϕ_1 y ϕ_2 son dos esquemas iterativos con órdenes de convergencia k_1 y k_2 , entonces el orden de convergencia del esquema iterativo de la composición

$$\phi(x) = \phi_1(\phi_2(x)) \quad (5.14)$$

es $k_1 \times k_2$.

Los órdenes de convergencia más comunes son los de la tabla 5.2.

A manera de ejemplo, varios algoritmos con órdenes de convergencia 1.4142, 1.4892, 1.5537, 1.5874 y 1.618 se presentan en [6]. La velocidad de convergencia

Valor de k	Convergencia
1	Lineal
$1 < k < 2$	Superlineal
2	Cuadrática
3	Cúbica

Tabla 5.2: Órdenes de convergencia más comunes. En la práctica lo superlineal es lo que se consigue con mayor frecuencia.

$\phi(x)$	Multiplicidad de r		
	$m = 1$	$m = 2$	$m = 3$
$x - u(x)$	2	1	1
$x - m u(x)$	2	2	1
$x - \frac{u(x)}{u'(x)}$	2	2	2

Tabla 5.3: Multiplicidad vs. orden de convergencia. Se ilustra el caso de 3 esquemas iterativos comunes. La iteración $x - u(x)$ es el famoso método de Newton-Raphson, que se estudiará con detalle más adelante.

as dignas de tomarse en superlineal es la que normalmente se espera obtener en la práctica, como afirma Kahan en [205].

La velocidad de convergencia importa para la precisión pues entre menos operaciones se requieran, menos errores numéricos se introducirán, aunque por lo general, a mayor k , aumenta la complejidad de las fórmulas. Existen algoritmos con orden de convergencia superior, como cúbica, cuártica y más, pero en general no tienen aplicaciones prácticas, salvo distinguidas excepciones. Algunos se presentan más adelante.

El orden de convergencia no es una medida absoluta pues el mismo método iterativo puede tener un orden de convergencia distinto para raíces con multiplicidad distinta. Para raíces simples se obtiene el mayor orden de convergencia. A mayor multiplicidad, menor orden de convergencia. Por eso es posible hablar de orden de convergencia k para cierto(s) valor(es) de multiplicidad. Esto se ilustra en la tabla 5.3.

Convergencia local y global

La mayor parte de los métodos de aproximación estudian el caso de la convergencia en la cercanía de una raíz r . Por lo tanto las condiciones para la búsqueda de r se dan de manera sólo localmente. A esto se le llama *convergencia local*, lo que significa que si el punto inicial x_0 está muy alejado de r no hay garantía de que se logre la convergencia.

En el caso afortunado de tener condiciones muy fuertes, es posible garantizar la *convergencia global*, lo que significa que no es necesario que x_0 esté cercano a

r para garantizar su convergencia. En la práctica difícilmente se tiene este caso.

Los métodos estudiados en este libro son mayoritariamente de convergencia local.

5.2.2. Etapas típicas de la aproximación de raíces

Todo el que ha programado métodos de aproximación de raíces ha sido testigo de que, con algunas excepciones, la aproximación se lleva a cabo en tres etapas.

1. **Bamboleo lejano.** Ocurre al inicio, cuando se parte de la primera aproximación x_0 y se comienzan a dar los primeros pasos. Esto depende de dos factores: la geometría de la función f y de si x_0 es suficientemente próximo a r . Si los dos factores están a favor, entonces es posible que se salga de esta etapa desde la primera iteración. En caso contrario, si la función es complicada o x_0 está demasiado alejado de r , entonces la convergencia puede fallar si el método iterativo no está adaptado para este caso. Para muchos métodos existen cotas de la distancia $|x_0 - r|$ en la que se garantiza llegar a la raíz.
2. **Acercamiento.** En esta etapa, las aproximaciones sucesivas se acercan (se espera rápidamente) a la raíz, sin tropiezos. Es el caso más estudiado en los libros de análisis numérico.
3. **Bamboleo cercano.** Las iteraciones están tan cerca de la raíz que las iteraciones pierden precisión. Este fenómeno puede ocurrir por problemas *extrínsecos* al método o *intrínsecos*.

El principal problema extrínseco es la precisión limitada del sistema $\mathbb{F}$ disponible, es decir, que se exige una precisión mayor de la que es posible alcanzar.

Los problemas intrínsecos pueden ser la inestabilidad del método numérico empleado para producir las x_i , el condicionamiento de f e irregularidades locales de f . La figura 5.10 es muestra de esto. Es la etapa en la que se debe tomar la decisión de detener el método iterativo y anunciar la existencia o no de la raíz.

La segunda etapa es la más segura y que menos tiempo requiere en la aproximación de raíces, además de ser más sencilla de analizar (por lo que aparece en todos los libros).

La primera y tercera etapas requieren que las rutinas de búsqueda programadas sea versátil y disponga de varias herramientas (métodos alternos y heurísticas) para lidiar con las distintas posibilidades (raíces múltiples, discontinuidades, mínimos locales, asíntotas, ...). Para la tercera etapa, pasar a un método de refinamiento puede ser conveniente.

Lo primero sería determinar en qué etapa se está y regular el tamaño del paso de cada iteración. Pasos grandes si se está lejos y cada vez más finos conforme se aproxima a r . A esto se le dedica más atención en este libro.

5.2.3. Puntos fijos

El esquema de punto fijo de los métodos iterativos no debe confundirse con la *notación* en punto fijo que aún se emplea para representar números en algunos procesadores. En este capítulo se debe entender por punto fijo lo siguiente.

Definición 7. *Dada una función $\phi : \mathbb{R} \rightarrow \mathbb{R}$, se dice que x es un punto fijo de ϕ si $\phi(x) = x$.*

Este es un tema de gran interés en matemáticas desde hace más de un siglo⁴. Dado el problema de resolver la ecuación (5.6), es decir, encontrar r tal que sea raíz de f , lo que se necesita es encontrar una función ϕ a partir de f que cumpla con

$$\phi(r) = r \quad (5.15)$$

o más precisamente

$$|\text{fl}(\phi(r) - r)| < \epsilon \quad (5.16)$$

para el caso de $\phi : \mathbb{F} \rightarrow \mathbb{F}$. Se dice que localizar r resuelve a la función f .

Encontrar la función ϕ adecuada a partir de f no siempre es posible. Por otra parte, ϕ no es única por lo que se han desarrollado diversas técnicas (restricciones) para encontrar la más adecuada.

En su artículo [337], Schroder define el orden de convergencia k haciendo uso del esquema iterativo ϕ de la siguiente manera.

Proposición 8. *Sea $\{x_i\}$ una sucesión que converge a la raíz r y $e_i = |x_i - r|$. Sea ϕ el proceso iterativo que genera las x_i , con orden de convergencia k . Entonces se cumplen*

$$\phi(r) = r \quad (5.17)$$

$$\phi^{(k)}(r) \neq 0 \quad (5.18)$$

$$\phi^{(j)}(r) = 0 \quad (5.19)$$

para $j = 1, 2, \dots, k-1$. Es claro que esto sólo es válido para esquemas iterativos con k derivadas continuas.

Además, se cumple que

$$\lim_{i \rightarrow \infty} \frac{e_{i+1}}{e_i^k} \rightarrow \frac{\phi^{(k)}(r)}{k!} \quad (5.20)$$

⁴Es un concepto central en sistemas dinámicos, alrededor del cual hay gran cantidad de resultados, desde puramente teóricos hasta control, pasando por ecuaciones diferenciales y sistemas lineales.

El resultado anterior se deriva de la aproximación de Taylor

$$x_{i+1} = \phi(x_i) = r + \phi'(r)(x_i - r) + \cdots + \frac{\phi^{(k)}(\zeta)}{k!}(x_i - r)^k \quad (5.21)$$

y ζ es un número real entre x_i y r .

Al momento de depurar es cuando no se agradece haber confiado en corazonadas.

Aunque todo lo anterior sirve para definir lo que son las raíces, su convergencia y los esquemas iterativos de aproximación, nada garantiza su existencia, así como no se garantiza que toda sucesión de valores x_i generada por ϕ converge a r , ni en $\mathbb{R}$ ni en $\mathbb{F}$. Desarrollar programas sin una base teórica completa que sustente los resultados esperados no es conveniente. Las garantías adicionales se presentan en los llamados teoremas de punto fijo⁵.

Hay muchos teoremas que garantizan la existencia y unicidad en cierto dominio restringido. Muchos tienen aplicación directa en el problema de las raíces, dando ideas de aproximación, cotas de convergencia o garantías de algún tipo. Otros teoremas de punto fijo se plantean para espacios mucho más abstractos, fuera del objetivo de este libro. Para los fines de este capítulo, las versiones sobre los números reales son suficientes.

Un primer resultado teórico (no de punto fijo) de gran ayuda es el siguiente.

Proposición 9 (Teorema del Valor Intermedio). *Sea $f : [a, b] \rightarrow [a, b]$ una función real continua. Si d es un valor entre $f(a)$ y $f(b)$, entonces existe un valor $c \in [a, b]$ tal que $f(c) = d$.*

El caso que interesa más es cuando $d = 0$, pues es la definición de raíz y el resultado particular se conoce como el *Teorema de Bolzano*. En ese caso, $f(a)$ y $f(b)$ tienen signo contrario y hay al menos un punto de cruce en el eje x .

Es importante observar que la validez del Teorema del Valor Intermedio⁶ depende de que $\mathbb{R}$ es denso. El mismo resultado no es válido si f se define sobre el conjunto de los número racionales pues una función como $f(x) = x^2 - 2$ cruza el eje en $\sqrt{2}$ que no es un número racional. Esto es importante porque la computadora no trabaja con $\mathbb{R}$, sino con $\mathbb{F}$.



Luitzen Egbertus Jan Brouwer, (1881 - 1966), matemático holandés pionero de la topología que demostró gran cantidad de resultados importantes. Padre le intuicionismo.

En la práctica, el teorema anterior no garantiza la existencia de una raíz. Esto debido a que la condición de continuidad es difícil de comprobar (a menos que se deje al usuario la responsabilidad de garantizarla). Piense el lector en el caso de una asíntota vertical.

El primer respaldo fuerte de la existencia de raíces (5.15) es un importante teorema de Brouwer.

Proposición 10 (Teorema de Punto Fijo de Brouwer). *Si $\phi : [a, b] \rightarrow [a, b]$ y ϕ es continua, entonces existe al menos un punto $x \in [a, b]$ que cumple con $x = \phi(x)$.*

⁵La existencia de cientos de teoremas de puntos fijos o el *Journal of Fixed Point Theory and Applications*, apenas si hace honor a la importancia de este sencillo concepto.

⁶Fue probado por Bolzano y Cauchy a principios del siglo XIX, cuando formalizaban el análisis de funciones. No debe confundirse este teorema con el del valor medio.

Para demostrarlo basta definir $h(x) = \phi(x) - x$ y aplicar el teorema del valor intermedio (particularmente el de Bolzano). Este sencillo y elegante resultado fue formulado en 1912 para el caso más general de bolas cerradas $\overline{B}_r(x)$ en $\mathbb{R}^n$ y Schauder lo generalizó a subconjuntos convexos compactos en espacios de Banach. Posteriormente Kakutani lo generalizó en 1959, relajando la condición de continuidad, lo que es conveniente para aritmética de intervalos.

Es muy interesante que si se generaliza el Teorema del Valor Intermedio para esferas unitarias en n dimensiones⁷, entonces el Teorema de Brouwer resulta una consecuencia, como probó Su en [356].

Para probar que el punto fijo r es único se puede imponer una restricción adicional al método iterativo ϕ : la contracción (ver el apéndice de Fundamentos Matemáticos). La contracción es una poderosa herramienta que incluso garantiza continuidad en las funciones reales (y en general en espacios métricos).

Proposición 11 (Teorema de Punto Fijo de Banach). *Si $\phi : [a, b] \rightarrow [a, b]$ y ϕ es contractiva, es decir,*

$$|\phi(x) - \phi(y)| \leq L |x - y| \quad (5.22)$$

con $x, y \in [a, b]$ y $0 \leq L < 1$. Entonces existe un único punto fijo en $[a, b]$.

Este resultado también es conocido como Teorema del Mapeo Contractivo. Para probarlo se supone la existencia de dos puntos fijos r_1 y r_2 distintos y aplicar (5.22) para llegar a una contradicción. Obsérvese que no se pide continuidad a ϕ , la contracción ya la garantiza.

Los dos fuertes teoremas anteriores (en su versión reducida para $\mathbb{R}$) garantizan que tal punto fijo r existe y que es único. Lo que falta es garantizar que el esquema iterativo ϕ converge a r .

Proposición 12. *Sea $\phi : [a, b] \rightarrow [a, b]$ contractiva, con constante de Lipschitz $L < 1$. La sucesión x_i generada con $x_{i+1} = \phi(x_i)$ a partir de un punto inicial $x_0 \in [a, b]$ converge al punto fijo r de ϕ en $[a, b]$.*

A esto se llega pues, $|x_{i+1} - r| \leq L^i |x_i - r|$ y como $L < 1$, pues $L^i \rightarrow 0$.

El valor L es la constante de Lipschitz (A.49) (ver página 367) y puede aproximarse con la derivada ya que si se despeja L se obtiene

$$L \leq \frac{|\phi(x_{i+1}) - \phi(x_i)|}{|x_{i+1} - x_i|} \approx f'(x_{i+1})$$

por lo que la sucesión generada por la iteración (5.15) converge si existe una constante positiva $K < 1$ tal que se cumpla la condición

$$|f'(x)| < K \quad (5.23)$$

⁷Si $f : S^n \rightarrow \mathbb{R}^n$ es continua, con $S^n = \{x \in \mathbb{R}^n / \|x\| = 1\}$, entonces existe un par de puntos en S^n exactamente contrarios (antipodales) que f mapea al mismo punto en $\mathbb{R}^n$. Este resultado es conocido como el Teorema de Borsuk-Ulam. Esto implica que sobre el ecuador terrestre, siempre hay dos puntos opuestos con la misma temperatura.

Iteración	$\phi_1(x) = \sqrt{5 - e^x}$	$\phi_2(x) = \ln(5 - x^2)$
1	1.0000000	1.0000000
2	1.5105357	1.3862944
3	0.6861804	1.1243411
4	1.7360545	1.3179773
5	NaN	1.1826274
6	—	1.2813206
7	—	1.2114104
$\vdots$	—	$\vdots$
100	—	1.24114275839960

Tabla 5.4: Iteraciones de punto fijo. En ambos casos se comenzó a iterar desde $x_0 = 1$. Aunque $x = \ln(5 - x^2)$ converge lo hace con mucha lentitud.

en una vecindad de la raíz que se busca. Si el punto inicial x_0 está en tal vecindad, la convergencia local está asegurada. Esta es la condición que exigen algunos métodos de aproximación.

Los tres resultados previos garantizan que ϕ converge a su único punto fijo, aunque no se asegura que lo haga rápidamente, lo que en la práctica es importante. Sirva el ejemplo siguiente para comprender los 3 resultados anteriores.

Ejemplo 42. Si la ecuación es $x^2 + e^x = 5$, la idea es despejar x de alguna manera conseguir una forma iterativa como (5.15). Hay dos posibilidades: $x = \sqrt{5 - e^x}$ o $x = \ln(5 - x^2)$. Partiendo de algún valor inicial, se itera para conseguir las aproximaciones que se muestran en la tabla 5.4.

Con la iteración ϕ_1 no hay convergencia pues $\phi'_1 > 1$ alrededor de la raíz, mientras que ϕ_2 converge ya que $\phi'_1 < 1$ en la misma vecindad. Sin embargo, el punto inicial no puede alejarse demasiado pues $\phi'_1(x) > 1$ para valores $x < 1 - \sqrt{6}$ y $x > 1 + \sqrt{6}$.

Del ejemplo anterior también se desprende que entre menor sea el valor de la derivada, la constante de Lipschitz será menor, por lo que la contracción provocará una convergencia más rápida. Por otra parte, como

$$|x_p - r| \leq \frac{L^p}{1 - L} |x_1 - x_0|$$

para algún valor p , entonces si L es muy cercana a 1 la convergencia será muy lenta.

Las tres proposiciones 10, 11 y 12 son resultados que dan seguridad teórica, pero su uso en programas que busquen raíces es muy limitado. Lo mismo para gran cantidad de resultados teóricos sobre convergencia local, comunmente con apariencia de globales. La proposición 12 pareciera la de mayor utilidad, pero requiere estar evaluando algún intervalo que contenga a la raíz en cada iteración

para comprobar que disminuye su tamaño. No es claro si es más eficiente seguir iterando en busca de la raíz o estar preguntando cada iteración por la inclusión de la misma. Por otra parte, la contracción en un intervalo muy grande es una condición suficiente, pero no necesaria.

Por fortuna para los programadores, sí existe una condición necesaria y suficiente, además de combinatoria, que permite garantizar la convergencia. Esta aparece en la referencia más antigua y llena de ayuda tanto teórica como práctica: las notas [205] de Kahan, sobre búsqueda de raíces. Escritas originalmente en 1986, con resultados de investigaciones de 1975, esas notas de clase han sido modificadas poco a poco durante tres décadas y son de lectura obligada para los interesados en el tema.

Las condiciones mencionadas son las del teorema de no intercambio de Sharkovsky [343] de 1964, una versión simplificada se plantea a continuación. ¡Oro puro!

Proposición 13 (Teorema de No Intercambio de Sharkovsky). *Sea el esquema iterativo $\phi : [a, b] \subset \mathbb{R} \rightarrow [a, b]$ una función del intervalo cerrado $[a, b]$ en sí mismo. Entonces el esquema iterativo $x_{i+1} = \phi(x_i)$ converge al punto fijo $r \in [a, b]$ partiendo de cualquier $x_0 \in [a, b]$ si y sólo si NO existen x y $y \in [a, b]$ distintos tales que ϕ los intercambie, es decir, $\phi(\phi(x)) = x$ si y sólo si $\phi(x) = x$.*

Las siguientes condiciones son equivalentes a la **condición de no intercambio** (y son parte del enunciado del teorema original en [343]):

1. **No separación:** No existe $x \in [a, b]$ que esté estrictamente entre $\phi(x)$ y $\phi(\phi(x))$, es decir, si $(x - \phi(x))(x - \phi(\phi(x))) \leq 0$ entonces $x = \phi(x)$.
2. **No camino:** Dados $x, y \in [a, b]$, si $\phi(x) \leq x \leq y \leq \phi(y)$ entonces $\phi(x) = x = y = \phi(y)$.
3. **Un solo lado:** Si $x_1 \neq \phi(x_0)$ y $\phi(x_0) \neq x_0$ en $[a, b]$, entonces todas las iteraciones siguientes $x_{n+1} = \phi(x_n)$ también difieren de x_0 y están en el mismo lado.

Según Fateman [107], el resultado anterior⁸ fue descubierto por Kahan en 1976. Además, Fateman presenta un algoritmo con el que verifica la condición de manera muy simple para funciones racionales. El trabajo está muy enfocado a polinomios, pero aún así la idea es de gran utilidad.

Es importante notar el Teorema de Sharkovsky no demuestra que la sucesión de puntos generados con $x_{n+1} = \phi(x_n)$ converge para todo $x_0 \in [a, b]$. De no cumplirse la condición de no intercambio, entonces la sucesión converge al menos a dos límites (puntos fijos) distintos. Esta observación la hace Kahan en [205].

El Teorema de No Intercambio de Sharkovsky debiera presentarse en todo libro de métodos numéricos que hable sobre puntos fijos o raíces de ecuaciones.

⁸Caso particular del Teorema de Sharkovsky, de gran importancia en sistemas dinámicos discretos. El teorema de no intercambio es el más sencillo de una familia de relaciones de puntos fijos.

La equivalencia de la condición de un solo lado con el no intercambio es útil en rutinas que aproximan raíces, como se muestra en el siguiente apartado, mientras que buscar que un proceso iterativo es una contracción puede ser igual de complejo que encontrar la raíz.

Para completar con un criterio de convergencia adecuados, Kahan mostró el siguiente resultado en [205].

Proposición 14 (Lema de la Tierra de Nadie). *Si una sucesión de un sólo lado (en el sentido de Sharkovsky) no es monótonica, entonces puede ser separada en dos subsucesiones disjuntas. Una de ellas asciende monótonamente a un límite nunca mayor que el límite al cual la otra subsucesión desciende monótonamente. Si ambos límites son distintos, el espacio entre ellos no tiene ningún elemento de alguna de las subsucesiones.*

El lema de Tierra de Nadie también es de gran utilidad práctica pues no es raro que, al irse aproximando a la raíz, alguna x_{i+1} se pase al otro extremo de r . Esto paso excesivo de una iteración a otra no necesariamente es malo, puede ser motivado por la geometría local de la función o por las limitaciones de $\mathbb{F}$ (básicamente debidos a redondeos). Pasarse de un lado al otro de la raíz no es grave mientras el proceso las x_i se aproximen a r cada vez más.

Los trabajos teóricos van mucho más allá de las intenciones de este libro. Por ejemplo, en [367], se estudian diversas sucesiones de iteraciones

$$E_s = \{\phi_0, \phi_1, \dots\} \quad (5.24)$$

de tal manera que el método iterativo ϕ_i tiene orden de convergencia i . La forma de las iteraciones se da de manera explícita. Según Traub, estudios como este aparecen en al menos 12 referencias⁹ previas a su trabajo de 1964.

5.2.4. Selección de x_0

Todos los algoritmos de búsqueda de raíces suponen que el usuario seleccionará el punto de inicio x_0 adecuado para lograr la convergencia. Esto no siempre se cumple, pues no todos los usuarios conocen suficiente cómo generar tal valor. Por otra parte, es dejarle al usuario gran responsabilidad si sólo cuenta con f . Si todo funciona como en los libros, se parte de x_0 y en pocas iteraciones se llegará a la raíz r . Pero, ¿qué tal si x_0 está demasiado alejada de r o si f es discontinua cerca de r ?

La experiencia deja ver que entre mejor sea la selección de x_0 , menos iteraciones serán necesarias para la convergencia.

Algunas Heurísticas posibles

Buscar un cero cuando no se tiene una aproximación inicial es algo que requiere fuerza bruta o heurísticas. La fuerza bruta no es deseable pues $\mathbb{F}$ tiene

⁹La mayoría muy difíciles de conseguir.

demasiados valores donde probar, aunque puede hacerse con algo de astucia.

Por otra parte, si no se tiene x_0 es posible que tampoco se tenga el dominio de f , por lo que primero es necesario encontrar valores donde f pueda evaluarse y dar como resultado números no especiales de $\mathbb{F}$. Como se dijo en la primera sección de este capítulo, al tiempo de buscar la raíz debe irse determinando el dominio de f .

La primera heurística reportada en la bibliografía se encuentra en [122], de Frank, quien en 1958 menciona que la computadora UNIVAC 1103 ya empleaba tales ideas. En esencia, se trata de encontrar 3 puntos donde sea posible evaluar f , para aplicar el método de Muller (sección 5.3.4), presentado en [279], originalmente sólo para polinomios, pero generalizado sin problemas en [122].

Se dice que era un esquema seguro para la mayoría de los casos.

Los valores que resultan obvios son -1 , 0 y 1 . Frank menciona que otra forma es partir de una aproximación inicial x_0 (o una raíz encontrada) y perturbarla para obtener los otros dos valores.

Disponer de 3 valores posee varias virtudes:

1. Se puede aplicar el método de Muller [279] (que será estudiado más adelante) para encontrar una nueva aproximación. Además, hay 3 posibilidades para x_0 o dos posibilidades para métodos iterativos que emplean dos aproximaciones sucesivas para calcular la siguiente.
2. Se puede aproximar la derivada f' .
3. Se puede aproximar la segunda derivada f'' , lo que permite conocer la convexidad. Además de ser un criterio para decidir si se está en un mínimo o máximo local.

Además de los valores sugeridos por Frank, hay varios valores adicionales que se pueden emplear, considerando el conjunto $\mathbb{F}$. Primeramente, el intervalo de búsqueda inicial debe ser $[a, b] = [-\infty, \infty]$. Evaluar f en alguno de los extremos de este intervalo resultará en valores que pueden dar información útil sobre las raíces, aunque no es definitiva.

Ejemplo 43. Si la función es $f(x) = x(1+e^{-x^2})$ entonces se obtendrá $f(-\infty) = -\infty$ y $f(\infty) = \infty$, lo que denota un cambio de signo. El cambio de signo da una esperanza de raíz, no seguridad. Para ello habría que garantizar continuidad, lo que no es nada sencillo para una computadora.

Ejemplo 44. La función

$$f(x) = \frac{x-1}{x-2}$$

posee una asíntota horizontal en 1, por lo que se cumple que $\lim_{x \rightarrow \infty} f(x) = 1$, pero al evaluar $f(\infty)$ en la computadora, surge el valor intermedio ∞/∞ , lo que resulta en NaN.

Del ejemplo 44 se desprende que obtener NaN no implica estar fuera del dominio y que la evaluación en ∞ puede no aportar información útil. La propagación de resultados, de tanta utilidad, puede ser engañosa, por lo que debe ser interpretada con cuidado.

Valor	Razón
∞	Aún con la posibilidad de overflow, puede dar información útil sobre el signo.
MAXDBL	El mayor número en doble precisión (o la empleada), lo que incluye los mayores números en las precisiones flotantes menores.
$\sqrt{x}$ y $x/2$	Con x el valor anterior.
β	Este y los siguientes valores resultan naturales para todo sistema $\mathbb{F}$.
1	
ε_M	
0	

Tabla 5.5: Posibles valores iniciales para buscar raíces. Se pueden emplear los mismos valores con signo negativo. alguna selección más adecuada se puede hacer dependiendo de la naturaleza del problema.

De mayor a menor (sólo positivos, pues la simetría aplica), los valores de $\mathbb{F}$ donde se puede evaluar para probar son:

A los valores anteriores se les puede perturbar para obtener 3 cantidades a partir de cualquiera de estos, de esta forma, si x es el valor en revisión, entonces $x \pm h$ son dos valores que encierran x . Tal perturbación h puede ser pequeña, mediana o grande, todas relativas al valor perturbado.

- Una *perturbación pequeña* se puede obtener modificando los últimos bits de x , por ejemplo $x \pm c \text{ulp}(x)$ con c un valor como 2 o 4. De manera equivalente, se puede calcular $x \pm c x \varepsilon_M$.
- Una *perturbación intermedia* se obtiene aumentando el valor de c hasta $p/2$, la mitad de la precisión del tipo de dato empleado.
- Una *perturbación grande* puede darse simplemente haciendo $h = x/2$ o algún valor similar o simplemente como $2x$ y $x/2$.

Estas perturbaciones no aplican en el caso del cero. Ahí es mejor un criterio absoluto.

Los anteriores valores son sugerencias para una búsqueda general, útiles si no conocemos la naturaleza del problema que se resuelve. En casos particulares, se tendrán límites impuestos por el modelo empleado, como el ancho de un cauce de agua, la velocidad del vehículo, un ángulo, el peso de una estructura y cualquier otro. Ahí será más fácil buscar un valor inicial porque el dominio ya está restringido.

Test alpha

Existe una rama de estudio llamada teoría de estimación de punto, iniciada por Steve Smale, que proporciona seguridad en la convergencia de algoritmos que buscan raíces de ecuaciones, usando sólo información de f en punto inicial x_0 . Todo valor del que se parta en un proceso iterativo de punto fijo que lleve a la raíz se denomina *cero aproximado*.

La primer condición para decidir si x_0 es un cero aproximado la proporcionó Smale en [348]. Este criterio es conocido como el *Test Alpha* y asegura que para alguna constante α_0 , si se cumple que

Y dio inicio a una nueva área de conocimiento.

$$\alpha(f, x) = |u(x)| \max_{k \geq 1} \left\{ \sqrt[k-1]{\left| \frac{f^{(k)}(x)}{k!f'(x)} \right|} \right\} < \alpha_0 \quad (5.25)$$

con $u(x)$ la corrección de Newton (5.3), entonces x es un cero aproximado de f . Wang encontró en [384] que $\alpha_0 = 3 - 2\sqrt{2} \approx 0.1715$ es una mejor cota.

La fórmula (5.25) resulta poco práctica pues se requiere conocer todas las derivadas hasta $f^{(k)}(x)$, lo que en la práctica es casi imposible, excepto para ciertas funciones. Sin embargo, Smale demostró en [347] que si f es un polinomio y $\phi_d(x) = \sum_{i=0}^d x^i$, entonces

$$\sqrt[k-1]{\left| \frac{f^{(k)}(x)}{k!f'(x)} \right|} \leq \frac{\|f\|}{|f'|} \frac{(\phi_d'(x))^2}{\phi_d(x)}$$

por lo que la cota ya se puede estimar en un programa. Claro, retringido para polinomios. El hecho de que toda función es aproximada con polinomios ofrece sólo cierta esperanza, pero ninguna garantía teórica aún.

Casos particulares

Para ciertas funciones particulares sí existen estimaciones de punto inicial confiables. Es el caso de funciones elementales como $\sqrt{x}$ o en general $x^{1/p}$ y otras, incluyendo la operación básica $\div$, la más compleja de esa familia. Mucho se ha escrito al respecto desde 1960, cuando ya imperaba la necesidad de bibliotecas de funciones matemáticas robustas y precisas en toda computadora.

Hacer todo con sumas, restas y multiplicaciones es lo ideal.

Muchas de tales referencias son buenos ejemplos para la sección 3.1. Como ejemplo, se presenta a continuación el caso particular de la función $\sqrt{x}$ para números reales no negativos. Esto está en la frontera entre los temas de este capítulo y el de programación de funciones, por lo que algunos conceptos (como reducción de rango) se tratan superficialmente.

$\sqrt{\cdot} : \mathbb{F}^+ \rightarrow \mathbb{F}$

El algoritmo básico para aproximar la raíz cuadrada es muy antiguo. Aparece descrito en el famoso libro de Al-Khwarizmi y lo documenta también Herón de Alejandría, quién podría haberlo heredado de la antigua Babilonia. De nuevo

Referencias más antiguas ya no se puede.

parten de un valor cercano al que se desea e iterativamente se va aproximando, con el método de Newton-Raphson, que apareció más de dos mil años después.

La idea del método es obtener la raíz de la ecuación $x^2 - a$, por lo que tal raíz será $r = \sqrt{a}$. Lo que se hace es partir de un punto inicial x_0 y aplicar el esquema iterativo

$$x_{n+1} = \frac{1}{2} \left(x_n + \frac{a}{x_n} \right) \quad (5.26)$$

hasta que se cumpla algún criterio adecuado, como $x_n^2 - a^2 < \varepsilon_M$ o algún otro seleccionado. ¿Cómo seleccionar x_0 ?

En el mundo de la computación, las primeras referencias de $\sqrt{}$ datan de la década de 1960, como [67, 105, 253, 277]. Sea

$$S(x) = \text{fl}(\sqrt{x}) \quad (5.27)$$

la aproximación de la raíz cuadrada. Los primeros estudios intentan minimizar el error máximo

$$\max_{x \in [a, b]} \left| \log \frac{S(x)}{\sqrt{x}} \right|$$

o el error relativo

$$\max_{x \in [a, b]} \left| \frac{S(x) - \sqrt{x}}{\sqrt{x}} \right|$$

y se sabía que la cantidad de iteraciones necesarias para alcanzar cierta precisión no dependía del valor inicial.

Se puede pensar que se requiere una cantidad infinita de puntos iniciales distribuidos por todo el dominio infinito de la raíz cuadrada (todos los reales no negativos), pero si se aplica el hecho de que

$$\sqrt{x} = 2^k \sqrt{2^{-k}x} \quad (5.28)$$

entonces se reduce el dominio a un pequeño intervalo. En las primeras computadoras, se probaron intervalos como $[\frac{1}{16}, 1]$, $[\frac{1}{4}, 1]$ o $[\frac{1}{2}, 1]$, que equivale a la mantisa de un número de punto flotante en computadoras con distinta base.

Tradicionalmente, el punto inicial se aproximaba con una función polinomial o racional como

$$x_0 = c_1 + c_2x \quad (5.29)$$

$$x_0 = c_1 + \frac{c_2}{x + c_3} \quad (5.30)$$

$$x_0 = c_1 + c_2x + \frac{c_3}{x + c_4} \quad (5.31)$$

Ejemplo 45. Con el polinomio

$$y = 2.5416391 - 4.8375280/(x + 2.1372553)$$

se determinaba el punto inicial para una mantisa en $[\frac{1}{2}, 1]$, con un error relativo máximo de 0.0003228.

Con el intervalo bien determinado, ahora es posible hacer subdivisiones y determinar el mejor punto inicial para cada una. A esto se agregan fórmulas explícitas para determinar el valor inicial óptimo dependiendo de una cantidad fija de iteraciones. En [339], 1994, presentan tales fórmulas para el caso de $1/a$.

Los tratamientos generales abordan el problema de las raíces de la función

$$f(x) = x^q - a \quad (5.32)$$

y la raíz que se busca es $r = a^{1/q}$, de donde se deriva el esquema iterativo

$$x_{n+1} = \frac{x_n}{q} \left(q - 1 + \frac{a}{x_n^q} \right) \quad (5.33)$$

y los casos que interesan son para los valores de q son: -1 para $r = 1/a$, 2 para $r = \sqrt{a}$ y -2 para $r = \frac{1}{\sqrt{a}}$.

En la actualidad lo común es tomar una cantidad fija de bits del valor $a - 1$, siendo $r = \sqrt{a}$ la raíz deseada, que sirven de índice para determinar el subintervalo $[a_{\min}, a_{\max}]$ de que se trata, que equivale a $r \in [\sqrt{a_{\min}}, \sqrt{a_{\max}}]$ por monotonicidad. Todo número que caiga en ese subintervalo tendrá la misma x_0 . La intención es minimizar el error, es decir, localizar x_0 tal que

$$\min_{a \in [a_{\min}, a_{\max}]} |r - x_0|$$

y en el caso de una función monótona como la raíz cuadrada, lo típico es tomar $x_0 = (a_{\min} + a_{\max})/2$, pero no es lo óptimo. Se requiere suponer que habrá n iteraciones, por lo que es necesario estimar $|r - x_n|$ a partir de $|r - x_0|$.

Este problema fue resuelto por Kornerup y Muller en [225]. Lo que se obtuvo fue la ecuación polinomial de tercer grado

$$\begin{aligned} a_{\max}^{1 - \frac{1}{2^{n-1}}} \left(\frac{3}{a_{\min}} - (x_0 - a_{\min}) \frac{q+1}{a_{\min}^2} (x_0 - a_{\min})^2 \right) \\ = \\ a_{\min}^{1 - \frac{1}{2^{n-1}}} \left(\frac{3}{a_{\max}} - (x_0 - a_{\max}) \frac{q+1}{a_{\max}^2} (x_0 - a_{\max})^2 \right) \end{aligned} \quad (5.34)$$

que se puede resolver numéricamente para luego obtener la raíz en cada intervalo

$$[\sqrt{a_{\min}}, \sqrt{a_{\max}}]$$

con lo que se encuentran los puntos iniciales que se necesitan. Un ejemplo permitirá comprender mejor esta idea.

Ejemplo 46. Sea $[a_{\min}, a_{\max}] = [\frac{5}{8}, \frac{6}{8}]$ la subdivisión de interés, buscando el punto inicial óptimo para dos iteraciones ($n=2$).

5.2.5. Sobre los criterios de paro

Si no se está diseñado rutinas que localicen de manera global todos los ceros de una función, sino sólo rutinas confiables que logren encontrar cierta raíz r a partir de una primera aproximación x_0 cercana (o tal vez no tanto), entonces se espera obtener una de dos posibles respuestas:

1. la raíz r (más bien la mejor aproximación posible) o
2. el diagnóstico de que cerca de x_0 no hay raíz, si es posible con alguna justificación, como una asíntota o un mínimo local.

En el primer caso de encontrar la raíz, la condición (5.6) debe cumplirse. El valor ϵ debe seleccionarse con mucho cuidado, por los errores de redondeo (ver [4]).

El segundo caso requiere más trabajo y los posibles diagnósticos son menos precisos.

Caso esperado

En el caso ideal, el esquema iterativo ϕ localizará un valor x suficientemente cercano a r y se cumplirá la condición (5.6) y posiblemente $\text{fl}(f(x)) = 0$ luego de redondeo. Es claro que se debe considerar cierta tolerancia, en especial si ϕ o f son de evaluación muy complicada.

Si r es la verdadera raíz que se aproxima con el valor x , como $|f(x)| < \epsilon$, entonces posibles valores de ϵ son:

$$\epsilon = 10^{-q} \quad (5.35)$$

$$\epsilon = \varepsilon_M \quad (5.36)$$

$$\epsilon = \sqrt{\varepsilon_M} \quad (5.37)$$

$$\epsilon = \text{ulp}(x) \quad (5.38)$$

$$\epsilon = \beta^{e_{\min}} \quad (5.39)$$

$$\epsilon = \eta \quad (5.40)$$

valores que fueron definidos en el capítulo de Aritmética de Punto Flotante. Estas cotas son muy estrictas, especialmente (5.39) y (5.40), para muchas aplicaciones y es común que se prefiera medir en términos de la cantidad de decimales correctos (5.35). El criterio (5.37) es propuesto en [385], pero puede resultar demasiado laxo, pues equivale a usar la mitad de la precisión nativa. Útil si la precisión no es algo prioritario.

Aún cuando la magnitud de $f(x)$ sea muy pequeña, es necesario que también ocurra que x sea muy próximo a r , por lo que se espera que $|x - r|$ sea un valor pequeño. De no ser así, es posible que se tenga el caso de una raíz múltiple y hay que decidir la mejor aproximación de r con más cuidado.

Considerando lo anterior, las evaluaciones

$$|x_{n+1} - x_n| < \varepsilon_1 \quad \text{y} \quad (5.41)$$

$$|f(x_{n+1})| < \varepsilon_2 \quad (5.42)$$

son determinantes pues al cumplirse alguna de ellas se detienen las iteraciones con la confianza de que x_{n+1} es la mejor aproximación posible de r , ante las limitaciones de la precisión de las computadoras y el hecho de que es preferible el error relativo que el absoluto.

Los valores de ε_i son difíciles de determinar eficientemente por anticipado. Debieran depender de la función y la raíz que se desean calcular, pero también del número de condición de ϕ y su estabilidad numérica. Si no se tiene esta información o se encuentra alguna manera de calcularla en tiempo de ejecución, el programador está obligado a usar cotas poco justas.

En el caso de que la rápida respuesta de la rutina sea crítica, es posible que el criterio (5.42) sea suficiente y la aproximación se regrese si hacer mayores averiguaciones. Si no es así, comprobar y refinar la respuesta es lo mejor.

Por otra parte, si el criterio (5.41) indica que dos iteraciones están muy cercanas, es posible que la raíz esté muy próxima, pero puede significar que la convergencia comienza a hacerse muy lenta y no que se está muy cerca, por lo que la interpretación es complicada si no se usa además el criterio (5.42).

El apartado anterior da una idea más formal de criterio de paro¹⁰: cuando la condición de un solo lado del teorema de Sharkovsky no se satisface. Lo mejor que se puede hacer es vigilar la sucesión de intervalos que contienen a las aproximaciones sucesivas. Aún cuando un proceso iterativo no genere intervalos, estos pueden formarse y actualizarse cada paso utilizando cada nueva aproximación, al tiempo de registrar la dirección de convergencia.

Más que un criterio de paro, se trata de una estrategia de iteración.

Casos anómalos

Considérese ahora que cerca de x_0 no hay raíz. La idea de (5.41) es detenerse cuando ya no hay un avance significativo en las aproximaciones, esperando estar cerca de la raíz. A falta de poder determinar $|r - x_{n+1}|$, que en condiciones normales siempre se menor que $|x_{n+1} - x_n|$. Eso es falso en el caso de funciones con derivada muy pequeña en valor absoluto cerca de la raíz, como cuando se trata de una raíz múltiple. Por ello se prefiere omitir este criterio de paro, excepto en el caso de cambio de signo.

Un caso derivado del anterior, pero más afortunado, es cuando se tiene un intervalo que rodea la supuesta raíz. Hay dos posibilidades:

1. Cambio de signo: en cuyo caso se puede estar en el caso de una *asíntota vertical* con cambio de signo, como la función del ejemplo 44, con una asíntota en $x = 2$.

¹⁰La forma de pensar no es sólo buscar condiciones de convergencia, sino en decidir qué más hacer además de iterar cuando una iteración parece no converger.

Sea la sucesión de intervalos $[a_n, b_n]$ que encierran cada vez más una raíz de la función f que tiene cambio de signo, es decir, $f(a_i)f(b_i) < 0$ para cada iteración i . El criterio más simple para detectar una asíntota es

$$\left| \frac{b_n - a_n}{f(m)} \right| < \epsilon \quad (5.43)$$

donde ϵ es una tolerancia es y $m = (a_n + b_n)/2$. Si no se trata de una asíntota, entonces el cociente de (5.43) se mantiene acotado.

2. Sin cambio de signo: raíz múltiple o tangencial.

Otra forma mucho menos deseable de terminar las iteraciones es llegar al máximo permitido, es decir, ya han sido demasiadas iteraciones. Por ejemplo, si en 50 iteraciones no se ha alcanzado alguno de los criterios (5.41) y (5.42)). Pero, ¿Por qué no detenerse a las 30? ¿O a las 20? Después de todo, hay métodos muy rápidos cuando convergen.

El criterio de máximo número de iteraciones es muy cuestionable y *no se recomienda*. Tal vez con una iteración más y se hubiera llegado a la raíz. Además no es equivalente a tiempo de procesador iterando medido en ticks del reloj por dos motivos distintos: 1- ϕ puede tardarse más en evaluarse que otras esquemas iterativos y 2- el tiempo de evaluación puede ser distinto para la misma función ϕ en iteraciones diferentes.

Lo anterior no significa que no se da la convergencia, sino que ocurre con demasiada lentitud, lo que deja otro criterio de paro adicional: cuando se detecta que la convergencia no ocurrirá. Esto normalmente se evalúa cuando repentinamente las aproximaciones se alejan de la vecindad en que se inició o un crecimiento abrupto de $|f(x_{i+1})|$, como en la vecindad de una asíntota. En ese caso, la sucesión x_i diverge o puede ocurrir que converge a otra raíz que no se esperaba.

Los intervalos deben ser cerrados, por lo que se definen como $[x_i, x_j]$. Se espera que toda iterada posterior x_k , con $k > \max\{i, j\}$, quede dentro del intervalo, como dice la condición de un solo lado. De inicio puede pensarse que el primer intervalo es $[-\infty, \infty]$, en cuyo caso la raíz está encerrada, pero no es raro que el intervalo inicial no incluya la raíz en algunos métodos.

Toda rutina debe considerar todas las posibles patologías de f , como discontinuidades o asíntotas (polos) principalmente cuando la convergencia no se logra.

La solución para cada caso debe diseñarse cuando se tenga una manera de diagnosticar primeramente el problema. En algunas ocasiones detener las iteraciones y regresar algún mensaje de no convergencia puede ser adecuado, pero en otras es necesario extrapolar aplicando algún método de aceleración o aplicar una transformación alterna más segura o rápida.

Los problemas mayores serán cuando la función es discontinua. En ese caso, el intervalo puede encerrar una asíntota, lo que complica las cosas. Antes de la norma de punto flotante, se acostumbraba que si la evaluación de ϕ resultaba

Código 5.1: Esqueleto de una función muy inocente que itera ϕ hasta alcanzar un punto fijo con alguna tolerancia **eps**.

```

1 double PuntoFijo(double x, double (*phi)(double),
2               double eps)
3 {
4     while ( fabs((*phi)(x))>eps )
5         x = (*phi)(x);
6
7     return x;
8 }

```

en una operación inválida se interrumpían los cálculos, pero hoy en día se deben aplicar las reglas de propagación de valores especiales.

Sólo el caso de cuando $\phi(x_i) = \text{NaN}$ debe interrumpir los cálculos, entendiéndose que el resultado sale del dominio. Una función que (incorrectamente) confía en que siempre hay convergencia se presenta en el código 5.1.

La aproximación inicial se recibe en **x** y se comienza a iterar hasta que $\phi(x) < \epsilon$. La función ϕ (**phi** en el código) se recibe como apuntador a función. Normalmente es lo que se prefiere en la interfaz al buscar una raíz, aunque en todos los esquemas iterativos de este capítulo se supondrá que todas las funciones definidas en el ámbito de cada método, por lo que no se usan directamente.

Una función más robusta implicaría llevar memoria de las iteraciones inmediatas anteriores para poder darse cuenta de que las aproximaciones se estacionan o alejan demasiado. En ese caso, faltaría algún parámetro adicional con el que se indique bajo qué condiciones regresó la función, si con éxito o fracaso y ayuda adicional, por ejemplo el intervalo donde se debe buscar la raíz: de salirse del intervalo se detienen las iteraciones. Regresar NaN es otra posibilidad.

Todo lo anterior será necesario mientras no exista un sustento teórico que garantice condiciones útiles en la práctica con las que se evite que un proceso iterativo divague alrededor de una raíz.

Los dos métodos de búsqueda de raíces más básicos son el de bisección y el de Newton-Raphson. De estos derivan una gran cantidad de métodos y por ello se discuten con mayor detalle que otros.

Aparecen en todos los cursos y libros de Métodos numéricos.

5.2.6. Comprobación final y refinamiento

Una vez encontrada la mejor aproximación, entonces se cumple $|f(x_n)| < \epsilon$ y seguramente, si no hay multiplicidad, la iterada x_n estará muy próxima a x_{n-1} . Es el momento de refinar, de pulir la aproximación más reciente.

Además, cuando f es como en la gráfica 5.3, entonces se cumple (5.42) pero no (5.41) y es posible que algo más de computación sea requerido.

El esquema iterativo ϕ empleado posee su propio índice de eficiencia. En caso de tratarse de un método que realiza muchos cálculos cada iteración, vale

la pena dar algunos pasos más con otro método que sea mucho más simple, con la intención de introducir menos errores de redondeo en el paso final.

Si el método ϕ empleado ya es simple, con índice de eficiencia alto, otra posibilidad es emplear una versión especial de ϕ , aplicando transformaciones libres de error.

Una revisión rápida en la vecindad de r no está mal, por ejemplo evaluando en los números de punto flotante vecinos. Esto permite no sólo comprobar si r es una buena raíz, sino detectar anomalías

5.2.7. Qué reportar

Es claro que el principal objetivo de una rutina que busca raíces es regresar r , el valor tal que $f(r) = 0$. Antes de preguntarse sobre lo que debe reportar una rutina es mejor determinar qué se espera como resultado. Esto es un compromiso entre el programador y el usuario de la rutina, quién debe ser capaz de comprender e interpretar lo que la rutina obtuvo.

La búsqueda de raíz puede terminar con éxito o con fracaso y cada caso requiere información distinta. En cualquiera de los dos casos, es adecuado que el reporte quede dentro de una estructura con toda la información que puede ser útil con respecto a la raíz o a la razón de no encontrarla.

En caso de encontrar la raíz r

En primer término se requiere tener como respuesta es el valor r , la raíz. Es deseable tener también:

- el valor de $f(r)$,
- el intervalo más pequeño que encierra la raíz (seguramente el de la última iteración) o la distancia de las iteradas más recientes $|x_{i+1} - x_i|$,
- qué tipo de raíz es (simple o múltiple),
- su multiplicidad estimada y
- el orden de convergencia estimado.

Si la rutina es un paso intermedio dentro de una serie de cálculos lo único que debe regresar es la raíz para continuar los procesos y no entorpecerlos. Regresar sólo la raíz es limitado si se considera toda la información útil de que dispone la rutina de búsqueda.

Lo mejor es guardar toda la información obtenida en una estructura que el usuario determine lo que le es útil.

En caso de NO encontrar la raíz

La misma estructura donde se reporta la raíz se puede emplear, con las banderas adecuadas, para dar información sobre una búsqueda no exitosa. No

se puede hablar de fracaso porque puede ser útil asegurarse de que en cierto intervalo no hay raíz y no encontrarla es el éxito.

Además de la mejor aproximación x_n , también es útil:

- $f(x_n)$, que puede ser un mínimo local,
- la bandera de que NO es una raíz,
- información adicional como sospecha de discontinuidad o asíntota.

Mucho más elaborado (y consumidor de recursos) sería localizar el dominio de la función.

Es casi imposible elaborar una rutina que localice ceros de cualquier función, como una que cause overflow en casi todo $\mathbb{F}$ excepto para un pequeño intervalo donde hay un cero. Ante una función así, parece que el único remedio es recorrer todos los valores posibles e ir evaluando la función hasta agotar $\mathbb{F}$. Esto sólo es posible en el caso de un proceso completamente fuera de línea, pues es muy tardado. Si es el único camino, se sugiere utilizar la menor precisión flotante disponible.

La fuerza bruta en su máxima esplendor.

5.2.8. Más raíces

No es raro que una rutina de aproximación de raíces sólo busque la mayor raíz, la menor, la que esté más cerca de cierto valor o algún otro criterio. Sin muchas averiguaciones, una rutina puede reportar el primer valor que cumpla con el criterio 5.6.

¿Y si varios números de $\mathbb{F}$ consecutivos cumplen con el criterio? Este es el caso de gráficas como la de la figura 5.3a. Si sólo son pocos valores, pudiera regresarse un arreglo, pero también el intervalo completo.

Ejemplo 47. La función $f(x) = (x-1)(x-10)^5$ tiene un valor absoluto menor que ε_M en el intervalo $[9.9996, 10.0004]$. Eso no garantiza que la evaluación cumpla esa misma restricción por motivos de redondeo. Por otra parte, al buscar la raíz puede estimarse la multiplicidad.

Una idea útil, practicada desde la década de 1950 por Forsythe¹¹ es reducir la función f conforme se vayan encontrando raíces. Si r_i son las raíces ya encontradas, con $i = 1, \dots, k$, entonces se obtiene se reduce f como

$$f_r(x) = \frac{f(x)}{\prod_{i=1}^k (x - x_i)} \quad (5.44)$$

cuidando siempre que no se evalúe en ninguna raíz. A esta técnica sólo falta agregarle la multiplicidad de cada raíz, pues de otra forma no trabaja. Las raíces y su multiplicidad deben ser almacenadas de alguna manera adecuada al algoritmo (y lenguaje) empleado. Sin embargo, ese esquema fue probado con raíces de multiplicidad mayor a 6 y no falló, según Frank en [122].

Esta técnica es la natural en el caso de raíces de polinomios.

¹¹La única referencia encontrada por el autor es un comentario en la segunda hoja de [122].

Lo ideal es localizar las raíces considerando su valor absoluto: de la menor a la mayor. De esta manera, los problemas de redondeo acumulados no afectan a raíces que requieren precisión mayor.

5.2.9. Funciones de prueba

Casi todo artículo que propone un nuevo algoritmo o esquema iterativo incluye un conjunto de funciones con las que se hicieron pruebas, unos más y otros menos. Incluso estas funciones se reciclan y los autores emplean ejemplos ya probados en otros artículos, para poder comparar mejor el nuevo algoritmo con los anteriores. Un caso frecuente es el polinomio $x^3 - 4x^2 - 10$.

El autor se dio a la tarea de recopilar funciones de muchos de estos artículos para conformar una batería de prueba grande (más de 80 funciones) y poder hacer muchos tipos distintos de pruebas. Las funciones elaboradas por el autor (más de 20) están diseñadas para darle problemas a los diversos algoritmos. Muchas de estas no tiene raíz alguna, sólo mínimos locales o discontinuidades.

Por ser extensa, la lista de funciones no se incluye aquí, pero está disponible para el lector interesado, comunicándose con el autor a manc@uabcs.mx. Tanto en formato $\text{T}_{\text{E}}\text{X}$ como el código en lenguaje C.

5.3. Métodos que encierran las raíces

5.3.1. Método de bisección

Este es un método útil en muchas situaciones, aunque su convergencia no es muy alta¹².

Si f es una función continua en el intervalo $[a, b]$ y si $f(a)f(b) < 0$, entonces f debe tener un cero en (a, b) por el teorema del valor medio. Se calcula el punto medio $x = \frac{1}{2}(a + b)$ y se revisa si $f(a)f(x) < 0$. Si es así, se hace $b = x$, de lo contrario, se hace $a = x$ y el proceso se repite. Hay dos criterios de paro importantes:

1. $|f(x)| < \varepsilon$, con $\varepsilon > 0$ un número que representa nuestra tolerancia o error aceptable. Este criterio surge de la definición misma de raíz pues lo que se busca es un número que cumpla con $f(x) = 0$.
2. $|b - a| < \delta$, con $\delta > 0$ otro número pequeño que determina el tamaño del menor intervalo donde buscaremos a la raíz. Eso significa que obtendremos

¹²Esta anotación de convergencia “lenta” es principalmente de carácter teórico. Con la velocidad de las computadoras modernas y la eficiencia de los compiladores, al aproximar un cero de una función las iteraciones terminan antes de levantar el dedo del ¡ENTER! o del botón del mouse. El problema es cuando se requieren muchos miles de aproximaciones dentro de un proceso de cómputo mayor. Además, no hay criterios automáticos para calcular los parámetros iniciales en el caso general. De otra manera, este sigue siendo un método robusto.

una aproximación a la raíz verdadera r , en el intervalo $(r - \frac{\delta}{2}, r + \frac{\delta}{2})$. Al mismo tiempo, este criterio fija la cantidad mínima de iteraciones que se necesitan, pues se puede calcular considerando que

$$\frac{|b - a|}{2^n} < \delta$$

y despejando se obtiene que n debe ser tal que

$$n > \left\lceil \log_2 \left(\frac{|b - a|}{\delta} \right) \right\rceil.$$

En los dos criterios, ε y δ no son necesariamente distintos. Pueden estar determinados por un error relativo para el intervalo inicial.

Orden de convergencia de la bisección

Si $[x_n]$ es la sucesión de aproximaciones a la raíz exacta r , entonces se cumple

$$|x_{n+1} - r| \leq \frac{1}{2} |x_n - r|$$

por lo que el orden de convergencia es lineal.

No es un mal orden de convergencia si consideramos la velocidad de las computadoras modernas, pero no termina de agradar que la sucesión de aproximaciones $\{x_i\}$ no se acerca a r en cada iteración. Por ejemplo, si la raíz es $r = 5$ y se comienza en el intervalo $[1, 10]$, $x_0 = 5.5$ y el siguiente intervalo será $[1, 5.5]$. La primera aproximación quedó a .5 de distancia de r . En la segunda iteración se hace $x_1 = (1 + 5.5)/2 = 3.25$, con lo que la segunda aproximación queda a 1.75 de distancia de la raíz.

En algunas aplicaciones esto es más serio que la convergencia lineal ya que se usa el criterio de continuar iterando mientras mejore la aproximación no se puede aplicar.

Aspectos de la programación

El código 5.2 muestra directamente el algoritmo presentado. Se supone que f es la función cuya raíz r interesa y que se sabe está en el intervalo $[a, b]$, donde cambia de signo. Si no hay cambio de signo el algoritmo no se puede aplicar.

La condición de la línea 4 es para comprobar que el cambio de signo se cumple. Este intento es muy legible pero tiene muchas cosas que criticarle, como el exceso de evaluaciones¹³ de la función f o el hecho de la precisión fija 10^{-13} .

¹³La cantidad de evaluaciones de la función de interés durante cada iteración es un parámetro importante al comparar los métodos iterativos. La convergencia en la mitad de las iteraciones puede perder sentido si se realizan más del doble de evaluaciones.

Código 5.2: Método de bisección para aproximar ceros de ecuaciones (versión simple).

```

1  double Bisec(double a, double b)
2  {
3      double r;
4      if ( f(a)*f(b) > 0 )
5          return 0;
6      while ( 1 ) {
7          r = (a+b)/2;
8          if ( fabs(f(r))<1e-13 )
9              break;
10         if ( f(a)*f(r)<0 )
11             b=r;
12         else
13             a=r;
14     }
15     return r;
16 }

```

¿Y si la raíz que se aproxima está en el intervalo $[0, 10^{-15}]$? ¿O en $[10^{11}, 10^{12}]$? La precisión fija no resulta adecuada.

En el caso de que por descuido se envíe un intervalo donde no hay raíz, es decir, que se cumpla el primer `if`, la función regresa de inmediato pero no hay ninguna señal que indique lo que ocurrió. Además, no hay un límite en la cantidad de iteraciones, en caso de que el algoritmo quede ciclado, que no sería raro tomando en cuenta la aritmética imprecisa de la computadora.

Otra de las primeras cosas que se debe notar es que no hay necesidad de calcular el producto $f(a)f(x)$, sino sólo revisar que los signos sean opuestos. Eliminar un producto en cada iteración (o sustituirlo por una verificación de signo) es una reducción de tiempo significativa, principalmente en el caso de funciones complicadas.

La mejor manera de calcular el punto medio de un intervalo es sumar al límite inferior la mitad de su longitud actual, como $a + t/2$, donde t es la longitud del intervalo. Ambos números pueden almacenarse en variables adecuadas desde la primer iteración. Un mejor intento es el código 5.3.

Se han resuelto algunos de los problemas de la primera versión y las cosas lucen mejor, aunque de 11 instrucciones se pasó a 22 y ya no es tan legible. Se agregó a los criterios de paro un contador para la cantidad de iteraciones, por si acaso algo sale realmente mal y el método se queda en un ciclo infinito. Además, no se repiten evaluaciones, se introdujo el error relativo como medida de paro y la función f se evalúa una sola vez cada ciclo. Al regresar NaN no hay malos entendidos cuando la condición inicial no se cumple.

Pero los dos primeros argumentos de la función son valores flotantes en el formato de IEEE754R, ¿qué pasa si $b = \infty$ o NaN? Los parámetros normalmente vendrán de algún otro procedimiento y pocas veces los escribirá el usuario desde

Código 5.3: Método de bisección para aproximar ceros de ecuaciones (versión no tan simple, pero aún incompleta).

```
1 double Biseccion(double a, double b, int MaxItera)
2 {
3     int i;
4     double fa=f(a), fr, L, Epsi, r;
5     if ( Signo(fa) == Signo(f(b)) )
6         return NaN; /* Para no usar la raiz */
7     if ( b<a ) {
8         double t=a;
9         a=b;
10        b=t;
11    }
12    L = b-a;
13    Epsi = Epsilon((b+a)*.5);
14    for ( i=1 ; i<MaxItera ; i++ ) {
15        L *= .5;
16        if ( L < Epsi ) /* Intervalo muy pequeno */
17            break;
18        r = a + L;
19        fr=f(r);
20        if ( fabs((fr-fa)/fr)<Epsi )
21            break;
22        if ( Signo(fr) != Signo(fa) )
23            b = r;
24        else {
25            a = r;
26            fa = fr;
27        }
28    }
29    return r;
30 }
```

el teclado. Por la norma 754 de IEEE, es posible que la evaluación de una función de estos resultados, por lo que al comparar el signo de $f(a)$ y $f(b)$ pudiera obtenerse una respuesta aparentemente adecuada.

Aún así, al intento anterior se le puede criticar su interfaz. Para efectos de evaluación, pudiera ser útil conocer el estado en que terminaron las evaluaciones, si se llegó al máximo de iteraciones o cuántas fueron, la precisión calculada o si en algún momento la condición de cambio de signo dejó de cumplirse.

Por otra parte, el error relativo es muy útil pues asegura una cantidad fija de cifras significativas correctas independientemente de la magnitud del intervalo, pero si la raíz es cercana al cero, la división se vuelve sumamente inestable y falta un criterio para ese caso. Como puede verse, programar bien un algoritmo numérico no es tan sencillo. Una versión mejorada de la anterior que considerara todas las posibilidades que dicta la norma casi triplica la cantidad de instrucciones y no se incluye en el texto.

La función **Epsilon**, que calcula una tolerancia adecuada para el intervalo inicial, puede ser algo como el código 5.4, donde se asegura una precisión de 16 decimales para el valor A.

Código 5.4: Cálculo de un épsilon adecuado.

```

1  double Epsilon(double A)
2  {
3      double act=A;
4      while ( act/A > 1e-16 )
5          act *= .5;
6      return act;
7  }
```

En caso de trabajar con simple precisión, la constante puede ser 10^{-8} en vez de 10^{-16} . Si se trabaja con cuádruple precisión, se puede emplear 10^{-34} . Si se supone que la computadora respeta la norma de IEEE, podría indicarse el número sin hacer cálculos.

Un buen ejercicio para el lector.

Bisección tendrá problemas en una función aparentemente simple como la de la figura 5.4. La forma parece indicar que hay una raíz cerca del cero con multiplicidad 0 y que el cambio de signo ocurre, por lo que con cierta confianza se puede a comenzar a iterar en el intervalo $[0, 2]$.

Una vez que falle el algoritmo y se analice con mayor cuidado la función, sale a la luz la realidad. Primero en la figura 5.5a, donde se aprecia que son en realidad tres raíces, una en $[.6, .8]$ otra cerca de 1 y otra cerca de 1.5. Ahora pudiera creerse conocer mejor a f e iniciar con un intervalo seguro. No habrá problemas con las raíces de los extremos, pero de nuevo fallará el método en la central. En la figura 5.5b, la gráfica en $[1, 1.0004]$ evidencia otras dos raíces más¹⁴. Es por ello que seleccionar adecuadamente el intervalo inicial es tan crítico.

Aprovéchese para observar en la gráfica 5.5b la repetición de abscisas por redondeo del graficador empleado.

Aunque bisección sea un método robusto, verificar que las condiciones de convergencia se cumplen requiere cierto cuidado.

¹⁴Podría ser un cuento de nunca acabar: el método gráfico ataca de nuevo.

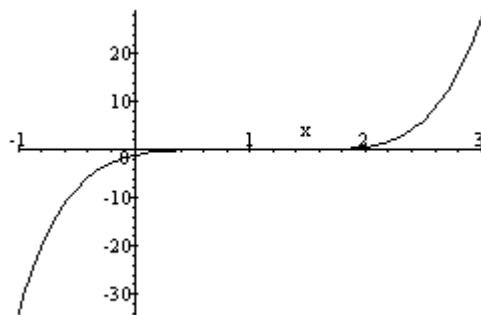


Figura 5.4: Gráfica de una función de apariencia inocente.

Mejorando la eficiencia de bisección

Puede pensarse que la división del intervalo en dos partes es muy limitada y que tal vez sea mejor idea subdividir en tres o más, para luego buscar en qué subintervalo ocurre el cambio de signo. La mejora en la aproximación más veloz de la raíz se pierde por la necesidad de bucar el subintervalo indicado. La única forma aparente de mejorar el desempeño del método de bisección es utilizando paralelismo.

Si se cuenta con $m + 1$ procesadores, se puede subdividir el intervalo en m partes y delegarle a cada uno de los procesos la verificación de cambio de signo. Se espera que uno solo responda. La idea suena sencilla pero en la práctica resulta un tanto inocente, debido a que en la típica arquitectura con muchos procesadores, cada procesador trabaja con su propia memoria local.

Si el intervalo $[a, b]$, se subdivide en m partes (suponiendo un procesador que dirige o controla a los demás), al procesador i le tocará el subintervalo

$$\left[a + (i - 1) \frac{b - a}{m}, a + i \frac{b - a}{m} \right]$$

y deberá evaluar a la función en ambos límites para verificar el cambio de signo. Como cada procesador utiliza su propia memoria local, cada uno repetirá la evaluación considerando como límite inferior el límite superior del anterior, pues es más veloz que transferir el resultado.

El procesador principal debe enviar a cada proceso los límites del intervalo y la función que debe evaluar, por lo que si son muchos, cuando el último procesador esté recibiendo el intervalo $[a + (m - 1) \frac{b - a}{m}, b]$, posiblemente muchos habrán terminado su pequeña tarea de dos evaluaciones y una comprobación de signo.

Una forma más eficiente puede lograrse si los procesadores comparten memoria la memoria principal. Las operaciones pueden organizarse de tal manera

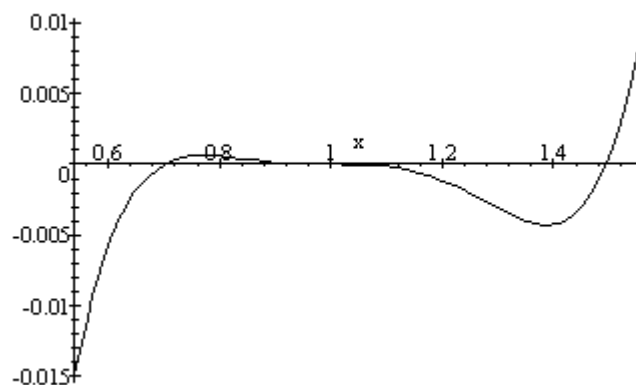
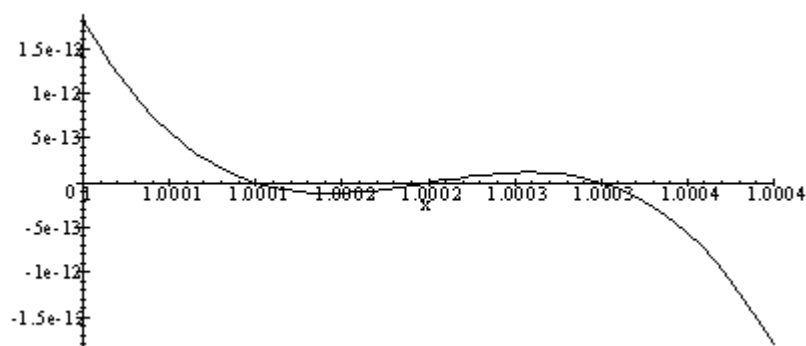
a. En $(0.6, 1.52)$.b. En $(1, 1.0004)$.

Figura 5.5: El acercamiento a las raíces de la función de la figura 5.4 en la página 207 aclara la situación. No se puede confiar en el método gráfico.

que al inicio de las iteraciones, a cada procesador se le asigna su propia i que indica qué subintervalo le corresponde, independientemente del intervalo en que se esté. Cada procesador i evalúa la función sólo en su límite superior y almacena el resultado, señala que evaluó con una bandera y queda en espera de la bandera del procesador $i - 1$. En cuanto se puede comprobar el cambio de signo, se envía un mensaje al procesador principal, que deberá cambiar el intervalo inicial. De esta manera no se repiten evaluaciones.

Es posible que exista más de un cero en el intervalo inicial, ya sea porque la función es engañosa (como la figura 5.4) o por inexactitudes numéricas, por lo que se debe prever estos casos. Se puede aplicar el algoritmo a cada subintervalo donde aparentemente hay una raíz o reportar el evento y detener las iteraciones.

5.3.2. Método de la secante

El método de la secante es una variación del método de Newton, útil cuando no hay derivada cerca de r o que su cálculo es demasiado complejo. La idea básica es sustituir la recta tangente por una secante. Por ello, se requieren dos puntos iniciales. La derivación es muy simple, radica en escribir la ecuación de la recta secante a partir de dos puntos de la función f , por donde pasa la secante, como en la siguiente gráfica.

Según la ecuación punto pendiente y suponiendo que x_{n-1} y x_n son los puntos iniciales, la secante que pasa por los puntos $(x_{n-1}, f(x_{n-1}))$ y $(x_n, f(x_n))$ es

$$y - f(x_n) = \frac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}}(x - x_n).$$

Si ahora se hace x_{n+1} el corte de la recta con el eje x y se escribe

$$f(x_n) + \frac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}}(x_{n+1} - x_n) = 0$$

de donde se despeja x_{n+1} , obteniéndose el esquema iterativo del método:

$$x_{n+1} = x_n - \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}f(x_n) \quad (5.45)$$

que es similar al método de Newton¹⁵. Ya se había utilizado esta idea en la página 231 (ecuación (5.63)).

El mayor problema de este método puede ocurrir si dos aproximaciones sucesivas están en lados contrarios de la raíz y que la función no cambie de signo, con el riesgo potencial de que se cumpla que $f(x_n) = f(x_{n-1})$, produciéndose una división entre cero. Ante este caso desafortunado se puede ajustar alguno de los extremos o ambos, reduciendo el intervalo a la mitad y continuando.

¹⁵Véase más adelante el método de Laguerre, para una extensión de estas ideas.

Convergencia del método de la secante

Si las dos aproximaciones iniciales son suficientemente buenas, se sabe que el método tiene un orden de convergencia de

$$\frac{1 + \sqrt{5}}{2} \approx 1.618 \quad (5.46)$$

y se dice que tiene convergencia superlineal. Esta convergencia es un cálculo empírico a partir del hecho de que la definición de convergencia (5.8) en la página 181 es $|e_{n+1}| \leq \alpha |e_n|^k$. Al despejar k se obtiene

$$k = \frac{\log(e_{n+1})}{\log(\alpha e_n)} \quad (5.47)$$

que converge a (5.46).

Esta técnica (5.47) se puede aplicar a cualquier método iterativo. Es un buen ejercicio comprobar las convergencias teóricas con las reales pues depende de la condición de cada función.

Secante se puede aplicar sin problemas si f es dos veces diferenciable y la raíz r es simple, es decir, no tiene multiplicidad. Al igual que el método de Newton, hay problemas cuando la derivada se acerca a 0 en valor absoluto.

El estudio más detallado sobre la convergencia de secante y su relación con Newton es el de Kahan en [205]. Una vasta variedad de consideraciones teóricas y de gran importancia práctica, que de manera general se pueden resumir con los siguientes puntos. El método de la secante converge siempre que Newton converge, por lo que es más robusto, y debe preferirse cuando

Además del obvio caso de no contar con f' .

1. calcular la derivada agrega más de 44 % al cálculo de f ,
2. hay cambio de signo en la raíz que se busca,
3. se requieren muchas iteraciones para alcanzar la exactitud deseada y
4. los errores de redondeo de la evaluación de f son tolerables (f se evalúa 3 veces contra una sola en Newton).

5.3.3. Método de falsa posición o regla falsa

En el método de la secante las dos aproximaciones en cada iteración están ambas a un mismo lado de la raíz. Una variante es aplicar el teorema del valor intermedio, utilizando la idea del método de bisección y poner un punto a cada lado de la raíz, calculando la secante. El corte de la secante con el eje x es la nueva aproximación, de donde se tiende la nueva secante. Esto asegura que el corte en el eje x de cada secante sucesiva pasa más cerca de r .

Lo único que se requiere es calcular el corte en el eje x de la secante en cada iteración. La ecuación de la secante a f en los extremos del intervalo $[a_n, b_n]$ en la n -ésima iteración es

$$y - f(b_n) = \frac{f(b_n) - f(a_n)}{b_n - a_n}(x - b_n).$$

De hecho es la misma ecuación de la secante pero con la notación del método de bisección. Por lo que la aproximación Se selecciona c_n como el corte en el eje x , por lo que sustituyendo y despejando se obtiene

$$f(b_n) + \frac{f(b_n) - f(a_n)}{b_n - a_n}(c_n - b_n) = 0$$

de donde se despeja c_n :

$$c_n = b_n - \frac{b_n - a_n}{f(b_n) - f(a_n)}f(b_n) \quad (5.48)$$

que es igual que la iteración del método de la secante, pero se debe estar revisando que $f(a)f(b) < 0$ como en el método de bisección. También se acostumbra escribirlo de la siguiente forma

$$c_n = \frac{a_n f(b_n) - b_n f(a_n)}{f(b_n) - f(a_n)}$$

para pensar que se calcula un punto en el intervalo $[a, b]$, como en bisección.

En este método, en ocasiones siempre es el mismo extremo el que se aproxima a r para algunas ecuaciones, por lo que el intervalo no se va reduciendo a la mitad como en el método de bisección, lo que resulta en una convergencia pobremente lineal.

J. A. Ford presentó en 1995 un reporte técnico [119] donde estudia varias maneras de mejorar algoritmos para funciones no lineales, entre ellos el de la regla falsa. Dentro de otras cosas, propone evaluar si el mismo extremo ha cambiado en dos iteraciones consecutivas para forzar un cambio en el otro extremo, con las fórmulas

$$c_n = \frac{\frac{1}{2}a_n f(b_n) - b_n f(a_n)}{\frac{1}{2}f(b_n) - f(a_n)} \quad \text{ó} \quad c_n = \frac{a_n f(b_n) - \frac{1}{2}b_n f(a_n)}{f(b_n) - \frac{1}{2}f(a_n)}$$

dependiendo el extremo repetido. Esto garantiza una convergencia superior a la del método original. Esta variante, llama Método Illinois, asegura una convergencia de 1.442 y es similar a la conocida como Pegasus, que es todavía un poco mejor con 1.642. En esta última, el $1/2$ se sustituye con $y_i/(y_i + y_{i+1})$.

Un refinamiento aún mejor es sustituyendo el $1/2$ por α , definida como

$$\alpha = \begin{cases} \beta, & \beta > 0 \\ \frac{1}{2}, & \beta \leq 0 \end{cases}$$

$$\text{con } \beta = \frac{(x_i - x_{i-1})(f(x_{i+1}) - f(x_i))}{(x_{i+1} - x_i)(f(x_i) - f(x_{i-1}))}$$

que de hecho se obtiene aplicando el concepto de diferencias divididas. El orden de convergencia del refinamiento es de al menos 1.68, apenas superior a secante.

5.3.4. Método de Müller

Como la secante es una burda aproximación a la función con una recta, en 1956 Müller agregó un punto más para utilizar una parábola¹⁶ y obtener una mejor aproximación en [279] que coincidiera con la función en tres puntos. Este método puede requerir aritmética compleja por los motivos que se expondrán. Ahora se parte de x_0 , x_1 y x_2 al inicio del algoritmo.

Aunque Müller lo presenta para el caso de polinomios, el método fue aplicado desde su nacimiento a funciones generales, en particular a funciones analíticas. Las raíces múltiples también son localizadas, aunque con convergencia menor, pero superlineal aún, a diferencia de Newton-Raphson. Una ventaja es que aunque requiere varios cálculos auxiliares, sólo requiere una evaluación nueva de f en cada iteración y no usa la derivada.

Es conveniente introducir el concepto de *diferencias divididas*. Sea el conjunto de n puntos $\{(x_i, f(x_i))\}_{i=0}^{n-1}$. La diferencia divididas de orden k , denotada como $[x_1, \dots, x_k]$, se define recursivamente como

$$[x_k] = f(x_k) \quad (5.49)$$

$$[x_k, \dots, x_{k+j}] = \frac{[x_{k+1}, \dots, x_{k+j}] - [x_k, \dots, x_{k+j-1}]}{x_{k+j} - x_k} \quad (5.50)$$

y son utilizadas para la evaluación de funciones dadas en tablas, en especial para la construcción de los coeficientes del polinomio de interpolación de Newton. Usando esta notación, en el método de falsa posición se podría reescribir la β del último refinamiento como $\beta = [x_{i+1}, x_i]/[x_i, x_{i-1}]$.

Regresando al método de Müller, se utilizan los tres valores iniciales para construir el polinomio interpolante de Newton, que pasa por los puntos iniciales $\{(x_i, y_i)\}_{i=0}^2$ como

$$y = f(x_2) + (x - x_2)f[x_2, x_1] + (x - x_2)(x - x_1)f[x_2, x_1, x_0].$$

Escribiendo $m = f[x_2, x_1] + f[x_2, x_0] - f[x_1, x_0]$ se puede escribir la iteración de Müller como

$$x_3 = x_2 - \frac{2f(x_2)}{m \pm \sqrt{m^2 - 2f(x_2)f[x_2, x_1, x_0]}}$$

y el signo se escoge de tal manera que el denominador sea lo mayor posible para evitar errores. En general se aplican las mismas observaciones que en el caso de la fórmula general.

¹⁶La diferencia con el método de Halley es que allí la parábola se busca tangente a f en cada nueva aproximación.

Como las aproximaciones pueden ser números complejos, es necesario realizar las operaciones con aritmética compleja. Es por ello que este método normalmente se utiliza sólo con polinomios, aunque sí se aplica a funciones analíticas, como se dijo antes.

Otra idea es que si se buscan raíces reales, a cualquier aproximación intermedia se le puede anular la parte imaginaria y continuar los cálculos con números reales. Demerita la convergencia pero facilita la programación.

Aunque la convergencia de (5.3.4) es local, padece de problemas similares a otros métodos, como cuando se aproxima una raíz y se llega a otra. Claro, siempre es posible acotar cada paso que se da en la aproximación e impedir un salto excesivo, principalmente cuando ocurre por una imprecisión numérica.

La convergencia es superlineal, con la ventaja mencionada: converge a raíces reales o complejas a partir de una aproximación real.

Programación

Normalmente se prefiere seguir los siguientes pasos, que han mostrado ser suficientemente estables. Sea la iteración k .

$$h_k = x^{(k)} - x^{(k-1)}$$

$$r_k = h_k / h_{k-1}$$

$$a_k = r_k f(x^{(k)}) - r_k(1 + r_k)f(x^{(k-1)}) + r_k^2 f(x^{(k-2)})$$

$$b_k = (2r_k + 1)f(x^{(k)}) - (1 + r_k)^2 f(x^{(k-1)}) + r_k^2 f(x^{(k-2)})$$

$$c_k = (1 + r_k)f(x^{(k)})$$

$$d_k = \frac{-2c_k}{b_k \pm \sqrt{b_k^2 - 4a_k c_k}} \text{ (raíz de la parábola, debe tomarse la que garantice el mayor denominador en valor absoluto)}$$

$$x^{(k+1)} = x^{(k)} + h_k d_k$$

Y estas operaciones se repiten hasta alcanzar alguno de los criterios de paro convencionales. Claro, la codificación puede optimizarse con unas cuantas operaciones intermedias, como el cálculo de $1 + r_k$ que debe hacerse una vez. El discriminante debe calcularse con el cuidado de costumbre, escalando de ser necesario.

Detalles de implementación

Los criterios normales deben aplicarse: máximo número de iteraciones en caso de que la convergencia sea lenta y detenerse cuando las iteraciones están en la tercera etapa: el error de las aproximaciones no disminuye. Normalmente debe establecerse el paro cuando

$$\left| \frac{z^{(k+1)} - z^{(k)}}{z^{(k+1)}} \right| < \varepsilon.$$

En el cálculo de la raíz de la parábola debe cuidarse que d_k no alcance un valor demasiado grande, pues en ese caso la aproximación puede saltar a otra raíz, ocasionando convergencia lenta o inconsistencias. Esto puede evitarse acotando el incremento relativo de d_k entre una iteración y otra.

Para evitar un problema de overflow, se puede calcular

$$n \log_{10} |z^{(k+1)}|$$

y si el resultado es demasiado grande, aproximar con

$$x^{(k+1)} = x^{(k)} + \frac{h_k d_k}{2}$$

y repetir hasta que no ocurra el overflow. El efecto será acercarse cada vez más al valor anterior $x^{(k)}$. Por supuesto, el criterio $|f(x^{(k+1)})| < \varepsilon$ debe imperar.

5.3.5. Método de Ridder

Una variante del método de falsa posición fue propuesta por C. J. F. Ridder en [319], de 1979. En esta variante se calcula el punto medio c_n del intervalo $[a_n, b_n]$ y con los tres puntos se calcula el cuarto punto

$$d_n = c_n + (c_n - a_n) \frac{\text{signo}(f(a_n) - f(b_n))f(c_n)}{\sqrt{f(c_n)^2 - f(a_n)f(b_n)}} \quad (5.51)$$

que es la raíz de una parábola que aproxima a f como en el caso del método de Müller, salvo que no es tangente a f .

Como d_n está en el intervalo $[a_n, b_n]$, ahora se evalúa dónde ocurre el cambio de signo, para proseguir con el intervalo adecuado:

- $[a_n, d_n]$ si $f(a_n)f(d_n) < 0$ o
- $[d_n, b_n]$ si $f(d_n)f(b_n) < 0$.

Su convergencia es cuadrática, aunque por la evaluación de funciones, el desempeño real es de convergencia $\sqrt{2}$.

Pese a ello, (5.51) tiene cualidades que se esperan en los buenos métodos de aproximación de raíces. Por ejemplo, cada aproximación d_n se mantiene siempre dentro del intervalo $[a, b]$. El discriminante en el denominador jamás es negativo por las restricciones de cambio de signo, por lo que la aritmética siempre es real.

Lo que hace es resolver la ecuación exponencial

$$e^{2t}f(b_n) - 2e^t f(c_n) + f(a_n) = 0$$

que consiste en una función cuadrática para la nueva variable $w = e^t$. Como en el caso del método de Müller, se aplica la fórmula general de segundo grado.

De nuevo, los cuidados comentados al inicio del capítulo en la sección 6.5 deben considerarse, como el escalamiento del discriminante y las banderas de estado.

* * *

Los siguientes tres métodos son ejemplo de que la combinación de estrategias es beneficiosa pues cuando un algoritmo rápido pierde el rumbo la corrección se obtiene de un algoritmo lento pero seguro.

5.3.6. Método de Dekker

El método de Dekker, publicado en 1969, combina bisección con falsa posición o secante y en cada iteración determina con cuál calcular la siguiente aproximación. Dekker llama a la secante interpolación lineal en su publicación. El método de Dekker trabaja de la siguiente manera.

Theodorus Jozef Dekker (1925–), matemático holandés, famoso por un algoritmo que permite compartir un recurso no divisible usando sólo memoria compartida.

Si se busca la raíz de la función continua f , se seleccionando puntos a y b tales que $f(a)f(b) < 0$, como en bisección. La existencia de la raíz está garantizada si se cumplen estas condiciones. Como en bisección y secante, un tercer punto está involucrado, ya sea el punto medio de bisección o la siguiente aproximación por secante. Se espera que el tercer punto esté más cerca de la raíz que las aproximaciones anteriores. Ahora se definen las siguientes cantidades.

Sea b_n el mejor candidato a raíz es decir, que se cumpla que $|f(b_n)| < |f(a_n)|$.

Sea a_n es el punto contrario, en términos del signo, $f(a)f(b) < 0$.

Sea b_{n-1} el candidato anterior. En el caso de la primera iteración se hace $b_{-1} = a_0$.

Se calculan dos valores provisionales¹⁷:

$$\underbrace{m = \frac{a_n + b_n}{2}}_{\text{bisección}} \quad \text{y} \quad \underbrace{t = b_n - \frac{b_n - b_{n-1}}{f(b_n) - f(b_{n-1})} f(b_n)}_{\text{secante}} \quad (5.52)$$

Si $t \in (m, b)$, entonces t se convierte en el nuevo candidato, $b_{n+1} = t$. Si no es así, entonces $b_{n+1} = m$. Claro, debe verificarse que el cambio de signo ocurra. En la figura 5.6 se muestran los dos casos posibles.

Los otros casos se dan por simetría.

Ahora se determinan el *punto contrario*. En caso de que $f(a_n)f(b_{n+1}) < 0$, a_n permanece igual, $a_{n+1} = a_n$. En caso contrario, $a_{n+1} = b_n$.

Si $|f(b_n)| < |f(a_n)|$ entonces los puntos contrarios se mantienen iguales. En otro caso, como a_{n+1} parece mejor candidato a raíz, se intercambia con b_{n+1} .

La iteración del método de Dekker termina cuando se cumple alguna de las condiciones de paro que el programador determine, como $f(b_n) < \epsilon$, $|b_n - a_n| <$

¹⁷Dekker calcula un tercer valor adicional que consiste en acercar el extremo más prometedor del intervalo hacia la raíz pero con un paso tan pequeño como la tolerancia aceptable, es decir, hace $b' = b + \text{signo}(a - b) \times \epsilon$.

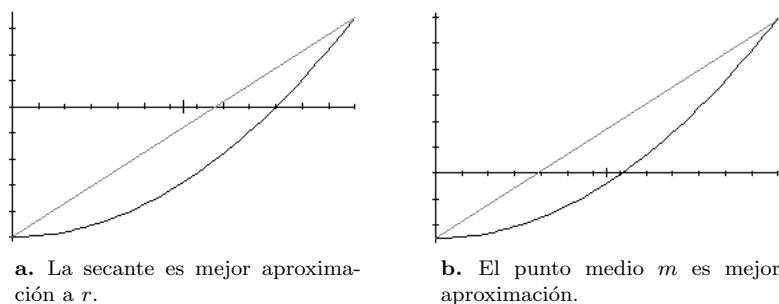


Figura 5.6: Aproximaciones del método de Dekker.

ϵ o haber llegado a una cantidad máxima de iteraciones. Dekker indica que se termina de iterar cuando se cumple la segunda de las condiciones, aunque eso revela la aritmética de punto flotante de la época.

La convergencia teórica es superlineal, heredada más de secante que de bisección. Al mismo tiempo, hereda los problemas con ciertas funciones.

En 1975, Dekker y Bus publican [60] donde diseñan otros dos algoritmos a partir de este, agregando una posibilidad más a 5.52. En vez de solo interpolar con una recta como es secante o falsa posición, además de la seguridad de bisección, utilizan lo que se conoce como interpolación racional, que consiste en

$$\phi(x) = \frac{x - w}{px + q} \quad (5.53)$$

donde los parámetros p , q y w se determinan de tal manera que $\phi(x) = f(x)$ en a y b . Para ello se requiere un tercer punto además de los límites del intervalo, normalmente se toma b_n o a_n , dependiendo de cómo se actualizó en la iteración anterior. Es simplemente una nueva función que se espera se parezca más a f que una simple secante, como lo sería ajustar una parábola a partir de 3 puntos, pero sin necesidad de aplicar radicales para encontrar la aproximación.

Para el segundo método, la diferencia consiste en asegurar que no se apliquen más de dos pasos del mismo algoritmo, que es una idea basada en el comportamiento asintótico de los métodos, con lo que se logra una mejora teórica de una convergencia de orden 1.842.

Ambas variantes no han vuelto a ser referidas con frecuencia y sigue conociéndose al método de Dekker como el explicado al principio.

5.3.7. Método de Brent

Este método es realmente un híbrido más, desarrollado en conjunto por van Wijngaarden y Dekker en Amsterdam en 1969 y posteriormente Brent en 1973, quien le dio su forma final presentada en [43]. Este método se emplea en la NetLib con ligeras modificaciones, además de MatLab. Está basado en

Richard Brent (1946 –),
matemático asustaliano.

los de bisección y falsa posición, agregando la posibilidad de usar *interpolación cuadrática inversa* (ICI), lo que requiere de explicación adicional.

Interpolación cuadrática inversa

Cuando se interpola un conjunto de puntos $\{(x_i, f(x_i))\}_{i=0}^{n-1}$, se busca poder aproximar $f(x)$ para valores de x que no existen en la tabla pero sí dentro de su rango. Por ejemplo, en el caso más simple, si se tienen los dos puntos $\{(2, 8), (3, 10)\}$ se puede interpolar con una recta que pase por ambos. Entonces se puede contestar cuál es el valor de la función interpolada en $x = 2.7$, por decir algo.

En vez de eso, la interpolación inversa supone el valor de $f(x)$ y busca el valor correspondiente x . En el ejemplo anterior, la pregunta sería en qué valor del intervalo $[2, 3]$ se obtiene la ordenada 9.1 y se despejaría para encontrar la abscisa¹⁸. La ICI interpola con un polinomio de Lagrange de grado dos (5.54) y se busca la abscisa para la que el polinomio debiera valer cero, es decir, $f^{-1}(0)$.

La figura 5.7 puede verse en dos orientaciones. En la orientación vertical se ve la gráfica convencional de f y las coordenadas $(x_i, f(x_i))$. Cuando se rota el texto se ve como la gráfica de la función inversa, $\mathcal{F} \equiv f^{-1}$. El polinomio p interpola los tres puntos $(\mathcal{F}(x_i), x_i)$ y la nueva aproximación es $x_3 = p(0)$.

No debe haber confusión: puede ser que la función que interesa no tenga inversa en la raíz r , pues pudiera ser raíz múltiple, pero la parábola interpolante p construida en la ICI siempre tendrá inversa en su cruce con cero por la forma de construcción. Lo peor que pueda pasar es que los puntos son tan distantes de r que la parábola no cruza el eje.

Si los tres puntos iniciales son $\{(x_i, f(x_i))\}_{i=0}^2$, el paso de ICI es

$$x_3 = \frac{f(x_0)f(x_2)x_1}{[f(x_1) - f(x_0)][f(x_1) - f(x_2)]} + \frac{f(x_1)f(x_2)x_0}{[f(x_0) - f(x_1)][f(x_0) - f(x_2)]} + \frac{f(x_0)f(x_1)x_2}{[f(x_2) - f(x_0)][f(x_2) - f(x_1)]} \quad (5.54)$$

que tiene convergencia más rápida que bisección. Ya Ostrowski en 1966 había probado que con pura interpolación lineal se alcanzaba convergencia de orden $\frac{1}{2}(\sqrt{5} + 1) = 1.618\dots$, que es lo mínimo que se espera de este método.

Algoritmo de Brent

La idea de Bren es utilizar bisección, secante y ICI. La pregunta es cuál de los tres métodos utilizar en cada iteración. El más robusto es bisección, aunque más ineficiente. El más eficiente es ICI, pero con riesgos de desorientación. En general

¹⁸Algunos le llaman interpolación en reversa. El método de la secante es una interpolación lineal inversa en el fondo pues aún intercambiando abscisas con ordenadas se obtiene la misma recta.

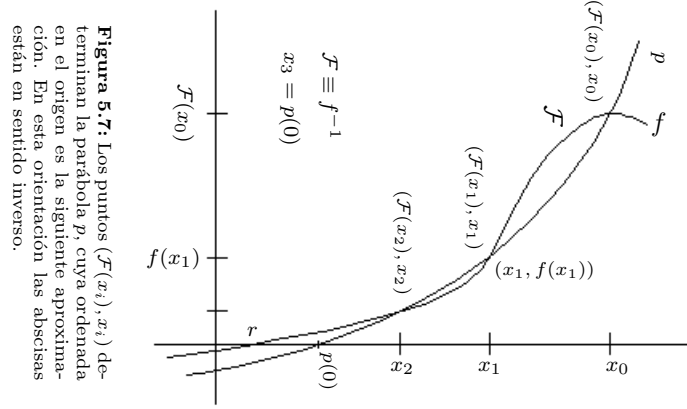


Figura 5.7: Interpolación cuadrática inversa. En esta orientación, la forma de determinar x es con la fórmula general, lo que da dos raíces.

la idea es utilizar ICI cada vez que se pueda y en caso de algún problema utilizar el método de la secante. Si secante no obtiene una aproximación suficientemente buena, se da un paso seguro con bisección.

Los puntos x_0 y x_1 son el intervalo inicial $[a_0, b_0]$ y se generará la sucesión de intervalos $[a_i, b_i]$ donde b_i es el mejor candidato a raíz. Por ello, si al inicio ocurre que $f(a_0) < f(b_0)$ entonces hay que intercambiar los valores.

Puede ocurrir que la aproximación del método de la secante se repita en iteraciones sucesivas, con la desventaja de que para algunas funciones, la diferencia $|b_n - b_{n-1}|$ es demasiado pequeña, por lo que la convergencia es lenta. Para evitar este problema, Dekker propone lo siguiente.

1. Si en la iteración anterior se utilizó el método de la secante, verificar que $|x_{n+1} - b_n| < \frac{1}{2} |b_n - b_{n-1}|$. Si esto no se cumple, la aproximación actual t no se acepta y se prefiere el punto medio (bisección), que reduce con certeza el intervalo a la mitad.
2. Si en el paso previo se utilizó ICI, se verifica que

$$|x_{n+1} - b_n| < \frac{1}{2} |b_{n-1} - b_{n-2}|$$

y si no se cumple también se utiliza bisección. Es decir, se le exige más a la ICI.

Como el paso de ICI es el mejor, se aplica siempre que $f(b_n)$, $f(a_n)$ y $f(b_{n-1})$ son distintos, en un intento de mejorar la eficiencia. Esto ocasiona un cambio ligero en el criterio para aceptar la nueva aproximación x_{n+1} . Se acepta x_{n+1} si está entre $(3a_n + b_n)/4$ y b_n .

Código 5.5: Método de Brent para aproximar ceros de ecuaciones.

```

1  if ( fabs(f(a)) < fabs(f(b)) ) {
2      w=a;    a=b;    b=w;
3  }
4  c=a;
5  B=SI;
6  for ( i=1; i<MaxIteraciones ; i++ ) {
7      if ( f(a)!=f(c) && f(b)!=f(c) ) {
8          x = a*f(b)*f(c)/((f(a)-f(b))*(f(a)-f(c)))+
9              f(a)*b*f(c)/((f(b)-f(a))*(f(b)-f(c))) +
10             f(a)*f(b)*c/((f(c)-f(a))*(f(c)-f(b)));
11      } else
12          x = b-f(b)*(b-a)/(f(b)-f(a));
13      if ( x<min((3*a+b)/4,b) || x>max((3*a+b)/4,b) ||
14          B==SI && fabs(x-b)>=fabs(b-c)/2 ||
15          B==NO && fabs(x-b)>=fabs(c-d)/2 ) {
16          x=(a+b)/2;
17          B=SI;
18      } else
19          B=NO;
20      d=c;
21      c=b;
22      if ( f(a)*f(x)<0 )
23          b=x;
24      else
25          a=x;
26      if ( fabs(f(a)) < fabs(f(b)) ) {
27          w=a;    a=b;    b=w;
28      }
29  }

```

Detalles de programación

Con todo lo anterior, el algoritmo de Brent sin optimizar se programaría como el código 5.5. Sea dado el intervalo $[a, b]$ donde f tiene un cambio de signo. Bisección se agrega para cuando los otros dos no ofrecen una buena aproximación, así que será la última opción por considerar. Primero se asegura de que b sea el límite del intervalo donde la función f es más cercana a cero, lo que da cierta posibilidad de que sea la mejor aproximación.

Este código que reescribe el algoritmo descrito, es bastante ineficiente. Primeramente, al inicio de cada iteración se tienen que calcular los valores $f(a_n)$, $f(b_n)$ y $f(c_n)$ y guardar los valores en tres variables, por ejemplo **fa**, **fb** y **fc**. Sólo debe actualizarse la que cambie.

Por otra parte, como la ICI requiere de una gran cantidad de evaluaciones, hay que optimizarlas. La fórmula 5.54 puede reescribirse como $x = b + \frac{p}{q}$, donde

p y q se definen de la siguiente manera.

$$\begin{aligned} p &= s[t(r-t)(c-b) - (1-r)(b-a)] \\ q &= (t-1)(r-1)(s-1) \end{aligned}$$

y las cantidades restantes se definen como sigue.

$$\begin{aligned} r &= \frac{f(b)}{f(c)} \\ s &= \frac{f(b)}{f(a)} \\ t &= \frac{f(a)}{f(c)} \end{aligned}$$

En cada momento se espera que b sea una buena aproximación a la raíz y que el término p/q sea una corrección no muy grande. De esta forma la instrucción que calcula la aproximación de la iteración de la ICI se cambiaría por los cálculos de r , s , t , p y q , para luego aplicar la fórmula $x = b + \frac{p}{q}$.

Como cada iteración incluye un potencial paso con bisección, se deben utilizar las optimizaciones que ya se discutieron para este método. Una vez que se combinan los tres métodos de manera óptima, el código no queda tan legible como el presentado, pero funciona con mayor eficiencia. Este es un excelente método para una librería de funciones matemáticas.

La ICI no es la única opción que han tomado algunos autores, por ejemplo, en [6] Alefeld combina ICI con interpolación cúbica inversa, lo que acelera un poco más la convergencia, pero sin llegar a cuadrática. En este caso, se requiere un punto más que en la ICI para construir el polinomio de grado tres que pasa por los cuatro puntos $\{(\mathcal{F}(x_i), x_i)\}$.

La librería gsl del proyecto GNU incluye el método de Brent dentro de sus buscadores de ceros, pero usa una variante donde la interpolación inversa es cúbica. El código fuente puede obtenerse en Internet.

Posterior modificación

En 2012, se publicó [389]

5.3.8. Método de Crenshaw

Fué desarrollado al inicio por IBM en la década de 1960 y distribuido dentro de su librería científica, hasta ser descubierto por Jack Crenshaw, físico reconocido en el desarrollo de sistemas empujados. El método de Crenshaw, es conocido ahora como TNIEBRF (The New, Improved, and Even Better Root Finder). En general alterna una iteración de bisección con otra de ICI, en vez de seleccionar la mejor entre ambas, como el de Dekker.

Se esperaría una mejor convergencia, pero son una cantidad considerable de operaciones adicionales.

Modestamente llamado por Jack Crenshaw *The World's Best Root Finder*.

Lo que Crenshaw hizo fue modificar los criterios de convergencia y cambiar la estructura del método, para hacer pruebas a la interpolación antes de usarla. En el caso de los criterios de convergencia, cambió el error absoluto usado por IBM por un criterio relativo.

Para las abscisa lo determina como $\epsilon_x = |x_n - x_{n-1}|$. En el caso de las ordenadas, como no se tienen los valores de $y = f(x_i)$ por adelantado, recurre a un esquema dinámico en el que va actualizando el mínimo y el máximo de f . en cada iteración. Con eso, es posible calcular $\epsilon_y = |y_{\max} - y_{\min}|$.

Según Crenshaw, tras intentar combinar los dos criterios en uno sólo encontró que era suficiente con el segundo de ellos. Además, para mantener estructurado el código y actualizar sin problemas los valores límites de la función, separó la evaluación del código, mediante una función intermedia que administra $y_{\max}$ y $y_{\min}$. El código se muestra a continuación.

Código 5.6: Función que controla la convergencia en el método de Crenshaw.

```

1 double Eval_f(double x, double Epsi, double *ymax,
2             double *ymin, bool *convergencia)
3 {
4     double y = f(x);
5     ymax = max(ymax, y);
6     ymin = min(ymin, y);
7     convergencia = (abs(y) < Epsi*(ymax-ymin));
8     return y;
9 }
```

Como puede verse, esta función no sólo mantiene el mínimo y el máximo, sino verifica si la convergencia ya se alcanzó.

En cuanto a la estructura del método, no se intercambian siempre bisección e interpolación. Más bien se utiliza bisección y si se puede también interpolación. Al esquema básico de IBM, Crenshaw agregó más comprobaciones en la estructura, haciendo en términos generales lo siguiente.

Código 5.7: Esquema general del método de Crenshaw.

```

1 for ( i=0 ; i<MaxItera ; i++ ) {
2     Aplicar biseccion: m=(a+b)/2
3     if ( f(b)-f(m) != 0 )
4         Aplicar interpolacion
5 }
```

Por supuesto, cada iteración tiene muchas más operaciones. Por ejemplo, cada evaluación de la función requiere una llamada a `Eval_f()`, además de una posterior revisión de si se alcanzó la convergencia.

Aunque las bases son las mismas que en los algoritmos anteriores, Crenshaw asegura que el método es superior debido a la estructura general de alternar bisección con interpolación cuadrática inversa y a la mejora de los criterios de paro. Realmente no se alternan bisección e ICI, sino que luego de aplicar bisección, se evalúa la conveniencia de ICI antes de realizarla.

* * *

Los dos siguientes métodos sirven de ejemplo de órdenes de convergencia más altos, que aunque ponen de condición que la primera derivada no se anule, en un intervalo que contenga a la raíz.

5.3.9. Método de Sharma-Goyal

Anonymous, 1980, Problems. BIT, 20, 261–262.

En 2006, Sharma y Goyal presentan un esquema iterativo de orden sexto, donde cada iteración consta de dos pasos. El esquema es

$$\begin{aligned} x_n^* &= x_n - \frac{m[f(x_n)]^2}{f(x_n + mf(x_n)) - f(x_n)} \\ x_{n+1} &= x_n^* - \frac{mf(x_n)f(x_n^*)}{f(x_n + mf(x_n)) - f(x_n)} \frac{f(x + mf(x))[f(x_n) + f(x_n^*)]}{f(x_n^*)f(x)} \end{aligned} \quad (5.55)$$

y parte de los esquemas iterativos derivados Newton donde la derivada se aproxima de dos maneras distintas, como Steffensen en la ecuación (5.73).

El valor de m puede ser ± 1 . El esquema presentado en [344] es algo más complejo pues depende de un parámetro más, una función racional en x . Sin embargo, (5.55) es el esquema iterativo más sencillo, aplicado en los ejemplos de [344].

El método de Sharma y Goyal resulta menos preciso que buscar un polinomio de grado 4 tangente a f en x_n , pero al menos no requiere las derivadas que el polinomio de Taylor necesitaría.

5.3.10. Método de Neta

Beny Neta utilizó el programa de cálculo simbólico MACSYMA¹⁹ para conseguir un método iterativo de orden 6. Sólo hace uso de la primera derivada pues al inicio requiere un paso de Newton Raphson.

El esquema iterativo que Neta presenta en [288] es

$$\begin{aligned} w_n &= x_n - \frac{f(x_n)}{f'(x_n)} \\ z_n &= w_n - \frac{f(w_n)}{f'(x_n)} \frac{f(x_n) - \frac{1}{2}f(w_n)}{f(x_n) - \frac{5}{2}f(w_n)} \\ x_{n+1} &= z_n - \frac{f(z_n)}{f'(x_n)} \frac{f(x_n) - f(w_n)}{f(x_n) - 3f(w_n)} \end{aligned} \quad (5.56)$$

¹⁹El proyecto Mac's SYmbolic MANipulation system, desarrollado hace décadas en lenguaje Lisp y utilizado principalmente para cálculo simbólico, aunque también permite algo de cálculo numérico, pero sin usar las bondades de la aritmética de punto flotante moderna.

que obtuvo a partir de las fórmulas más generales

$$w_n = x_n - \frac{f(x_n)}{f'(x_n)} z_n = w_n - \frac{f(w_n)}{f'(w_n)} \frac{f(x_n) + af(w_n)}{f(x_n) + bf(w_n)}$$

$$x_{n+1} = w_n - \frac{f(z_n)}{f'(z_n)} \frac{f(x_n) + cf(w_n) + df(z_n)}{f(x_n) + ef(w_n) + g(z_n)}$$

y utilizó MACSYMA para obtener las expresiones de error de w_n y z_n . A partir de las expresiones, ajusta los parámetros a, b, c, d, e y g para anular los términos de orden 5 y menores, obteniendo (5.56).

Expresiones bastante grandes que son muy tardadas de obtener si no se dispone de paquetería para cálculo simbólico.

En cuanto a la cantidad de evaluaciones, el método es equivalente a dar un paso de Newton y dos de secante, lo que alcanzaría una convergencia de orden 5.2 como máximo. En el caso de (5.56), en esencia se dan tres pasos en cada iteración, ajustando la aproximación de Newton con funciones racionales.

Dar tres pasos de Newton sería más barato, por lo que el método queda mejor como ejemplo de una técnica que se puede utilizar para deducir esquemas iterativos a partir de fórmulas generales.

5.4. Métodos que usan la geometría de la función

Sin lugar a dudas, el método de Newton-Raphson es el más conocido y estudiado de todos, así como sus extensiones para la optimización de funciones de varias variables. Se agrupa en esta sección muchos temas asociados con esta iteración, cuyas ideas conceptuales además se repiten en otros métodos presentados más adelante.

5.4.1. Método de Newton-Raphson

Este es un método poderoso para la búsqueda de raíces, que se presenta sin considerar que se trate de polinomios. Fue propuesto por Raphson en 1690, en su libro “Analysis aequationum universalis”.

Joseph Raphson, (1648 – 1715), matemático inglés.

Se publicó también como trabajo de Newton en 1736, pero escrito en 1671. Se dice que Newton se limitó a aplicarlo a polinomios²⁰. Posteriormente fue estudiado por otros matemáticos, por ejemplo Simpson, en 1740, quien vio la conexión con las derivadas, según [334], o Mourraille, quien en 1768 se percató de lo sensible y difícil de determinar una buena aproximación inicial.

Isaac Newton, (1648 – 1715), matemático inglés.

Consiste en iniciar con una aproximación x_0 cercana a la raíz r y calcular el cruce en el eje x de la recta tangente a la función en x_0 , como muestra la figura 5.8.

²⁰De hecho, usa la función $x^3 - 2x - 5 = 0$ como ejemplo en su “*Methodus fluxionum et serierum infinitarum*” por 1669, sin utilizar explícitamente el concepto de derivada.

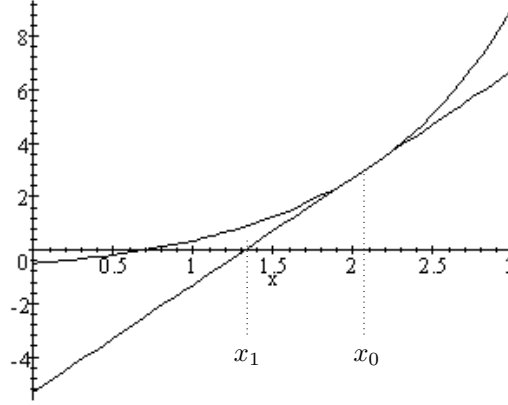


Figura 5.8: Método de Newton: el punto de cruce de la recta tangente, $(x_0 - \frac{f(x_0)}{f'(x_0)}, 0) = (x_1, 0)$ es la siguiente aproximación.

Una forma de deducirlo es a partir del polinomio de Taylor. Si la función f posee una raíz en el valor r y que es continua hasta al menos la segunda derivada en una vecindad de r . La aproximación de $f(x)$ para una x en la vecindad de r viene dada por

$$\begin{aligned} 0 &= f(r) = f(x + h) \\ &= f(x) + hf'(x) + O(h^2). \end{aligned}$$

De aquí se deduce que $h = r - x$. Al eliminar los términos de segundo grado, obtenemos que $h = -f(x)/f'(x)$. Ahora, si x es próximo a r , entonces $x - h$ debiera estar más próximo aún. De esta manera, obtenemos un procedimiento para aproximar la raíz a partir de un primer candidato:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \quad (5.57)$$

obtenida con el supuesto de que la derivada no es cero en la vecindad de la raíz y suponiendo que h es pequeña, es decir, la aproximación inicial es buena. Este es un esquema iterativo de punto fijo.

También puede verse (5.57) como la función $g(x) = x - f(x)/f'(x)$, que es una función de $\mathbb{R}$ en $\mathbb{R}$, es decir, un endomorfismo. En la práctica se debe tomar en cuenta que se trata de $\mathbb{IF}$ y no de $\mathbb{R}$.

Otra manera de deducir la misma fórmula (5.57) es geoméricamente a partir de la figura 5.8.

Para simplificar expresiones, de ahora en adelante se usará también la notación (común en muchos textos, como el de Traub [367])

$$u(x) = \frac{f(x)}{f'(x)} \quad (5.58)$$

o simplemente

$$u = \frac{f}{f'}$$

ya que aparecerá constantemente en muchas expresiones. Este término es conocido como *corrección de Newton*. La iteración de Newton se escribe en forma más simple:

$$x_{n+1} = x_n - u(x_n)$$

Una desventaja del método es que requiere la evaluación de la derivada de f en cada iteración y es muy común que las derivadas sean funciones mucho más complicadas que la original. Una táctica útil en estos casos es actualizar la derivada cada k iteraciones para no realizar tanto cálculo.

Condiciones de convergencia

Las aproximaciones generadas por el esquema iterativo (5.57) son una sucesión de puntos en $\mathbb{R}$ que cumple $|x_{n+1} - r| < |x_n - r|$ si x_0 está suficientemente cerca de r . Retomando la terminología de la sección A.6, si se encuentra un intervalo $[a, b]$ donde la única raíz sea r y una aproximación inicial $x_0 \in [a, b]$, si el esquema iterativo (5.57) es contractivo entonces es estará asegurando la convergencia. Hay diversas maneras de lograr esto.

Una es exigiendo que $|f'(r)| > 0$, que f sea doblemente diferenciable en una vecindad de r y si para algún $\delta > 0$ ocurre que

$$\left| \frac{f''(\bar{x})}{f'(x)} \right| < \frac{2}{\delta}$$

para $\bar{x}$ y x en el intervalo $[r - \delta, r + \delta]$ entonces Newton converge y lo hace cuadráticamente.

Mínimamente habría que exigir que f'' sea continua, r sea un cero simple, con lo que siempre será posible encontrar una aproximación inicial, o en otras palabras, un intervalo $[a, b]$ tal que $r \in [a, b]$ y una constante L tal que

$$|x_{n+1} - r| \leq L|x_n - r|^2$$

comenzando desde cualquier punto inicial

La única circunstancia en la que el método converge desde cualquier punto inicial es cuando la función es creciente, convexa y que tenga un cero, lo que es conocido como condiciones de Cauchy. Es el caso de funciones como $e^x - 2$.

Una prueba más formal y rigurosa de que este método (y otros similares) converge se basa en el *Teorema de Kantorovich*²¹.

Leonid Vitalyevich Kantorovich, (1912-1986), matemático ruso ganador del premio Nobel de Economía en 1975 por sus trabajos en optimización.

²¹Este teorema ha encontrado aplicaciones en muchas ramas de las matemáticas, como control, elemento finito, ecuaciones integrales y muchos más, (consúltese [309]) al proporcionar condiciones para la existencia y unicidad de soluciones de operadores no lineales en espacios de Banach, aunque en esta presentación sólo interesa el caso particular de los números reales.

El teorema enuncia que si f tiene al menos hasta la segunda derivada continua y satisface la condición de Lipschitz

$$\left| \frac{f'(x) - f'(\bar{x})}{f'(r)} \right| \leq \alpha |x - \bar{x}|$$

para toda x y $\bar{x}$, con

$$|x - \bar{x}| + |r - \bar{x}| \leq \frac{1}{\alpha}$$

y además $\beta = |u(r)|$, si $\kappa = \alpha\beta \leq 1/2$ entonces la sucesión x_n generada por $x_{n+1} = x_n - u(x_n)$ (método de Newton Raphson) converge a la solución única r en el intervalo cerrado $[r - \frac{1}{\alpha}, r + \frac{1}{\alpha}]$.

Y todavía más, Kantorovich asegura que si

$$\gamma = \frac{1 - \sqrt{1 - 2\kappa}}{\alpha} \leq \frac{1}{\alpha}$$

entonces la aproximación se alcanza para valores iniciales en el intervalo abierto $(r - \gamma, r + \gamma)$, donde para el método de Newton²², $\kappa < 1/2$. Otra buena referencia es [403].

Este método ha sido muy estudiado y ha permitido que se determinen condiciones menos estrictas que también garanticen la convergencia. Por ejemplo, mientras que el teorema de Kantorovich exige la condición de Lipschitz

$$|f'(x) - f'(y)| \leq L|x - y| \quad (5.59)$$

en [157], Gutiérrez y Hernández muestran que es suficiente con

$$|f'(x) - f'(x_0)| \leq L|x - x_0| \quad (5.60)$$

lo que implica alterar algunas de las otras condiciones, particularmente el tamaño del intervalo donde puede seleccionarse x_0 .

Los interesados en estos temas no deben dejar de buscar los teoremas de Miranda, Moore y Borsuk, que imponen criterios distintos para la existencia de ceros de funciones, generalmente formulados en espacios de Banach. Kantorovich resulta un caso especial de una generalización del de Miranda y este del de Borsuk, según Alefeld, Frommer, Heidl y Mayer. En particular los de Moore y Miranda son útiles como pruebas en toda rutina que busque raíces de funciones de una o más variables. Miranda es particularmente útil en el caso de utilizar aritmética de intervalos.

Velocidad de Convergencia de Newton

Se parte de nuevo de la aproximación de Taylor. Sea e_n al error de la aproximación en el paso n , es decir, $e_n = |x_n - r|$, con r la raíz que se busca, $e_0 < 1$.

²²La prueba puede consultarse en [310].

La aproximación con el polinomio de Taylor es

$$\begin{aligned} 0 &= f(r) = f(x_n - e_n) \\ &= f(x_n) - e_n f'(x_n) + \frac{1}{2!} e_n^2 f''(\delta), \end{aligned}$$

con δ algún número entre x_n y r . De aquí se obtiene la relación

$$e_n f'(x_n) - f(x_n) = \frac{1}{2!} e_n^2 f''(\delta). \quad (5.61)$$

Ahora lo que interesa es ver cómo se comporta el error de un paso a otro para poder determinar el orden de convergencia.

$$\begin{aligned} e_{n+1} &= x_{n+1} - r \\ &= x_n - \frac{f(x_n)}{f'(x_n)} - r \\ &= x_n - r - \frac{f(x_n)}{f'(x_n)} \\ &= e_n - \frac{f(x_n)}{f'(x_n)}. \end{aligned}$$

Esta expresión se puede escribir como $e_{n+1} = \frac{e_n f'(x_n) - f(x_n)}{f'(x_n)}$ y, considerando la ecuación 5.61 y que r es cercano a x_n , ahora se puede escribir como

$$\begin{aligned} e_{n+1} &= \frac{1}{2} \frac{e_n^2 f''(\delta)}{f'(x_n)} \\ &\approx \frac{1}{2} \frac{f''(r)}{f'(x_n)} e_n^2 \\ &= C e_n^2 \end{aligned}$$

para alguna constante C , por lo que se concluye que el método de Newton posee convergencia cuadrática. Esto significa que en cada iteración se duplica la cantidad de cifras significativas correctas.

La C es la constante de Lipschitz (véase la página 367) si se ve la sucesión de errores como los puntos generados por una función. Es suficiente con que

$$\frac{1}{2} \frac{f''(r)}{f'(x_n)} < 1$$

para que sea un mapeo contractivo y sea posible la convergencia.

Además, si f y g tienen como raíz a r , son funciones convexas en una vecindad de r y son dos veces diferenciables con primera derivada no nula en r , entonces la iteración de Newton (5.57) converge más rápido en la función con mayor convexidad logarítmica L_f . Al no tener la raíz desde el inicio de las iteraciones, L_f puede evaluarse con las aproximaciones x_n .

Código 5.8: Método de Newton-Raphson para aproximar ceros de ecuaciones.

```

1  double Newton(double x, int MaxItera, int *Itera)
2  {
3      int i;
4      double Epsi, fx, dfx;
5      Epsi = Epsilon(x);
6      for ( i=1 ; i<MaxItera ; i++ ) {
7          fx = f(x);
8          dfx = df(x); /* la derivada */
9          x = x - fx/dfx;
10         if ( fabs(fx) < Epsi )
11             break;
12     }
13     *Itera = i;
14     return x;
15 }

```

Aspectos de la programación

El código más directo de una función como Newton es algo como el código 5.8, en el que la función recibe la aproximación inicial, la cantidad máxima de iteraciones y la variable donde quedará cuántas iteraciones fueron necesarias.

Es claro que el código aplica el método de Newton tal y como fue presentado, pero generalmente esto no es del todo eficiente o seguro. Por ejemplo, la fórmula de actualización pudiera dar un cociente entre cero no previsto, por lo que hay que tomar precauciones y cambiar la fórmula por al siguiente segmento de código.

Puede considerarse algo arriesgado utilizar *Epsi*, pero el otro remedio es simplemente abortar las iteraciones indicando que no se llegó a la aproximación requerida.

```

if ( fabs(dfx) > Epsi )
    x = x - fx/dfx;
else
    x = x - fx/Epsi;

```

Por otra parte, el único criterio de paro de si la función es casi cero no es suficiente en el caso general, conviene ir revisando si la actualización actual es muy cercana a la anterior para detenerse cuando ya no se está mejorando sensiblemente. Es equivalente al criterio del tamaño del intervalo en bisección, donde si el intervalo es suficientemente pequeño se detiene el proceso. Para esto, es necesario mantener una variable con la aproximación anterior para poder comparar.

Comparando los métodos de Bisección y Newton, la tabla 5.6 muestra las iteraciones para la función $(x - 1)(x - 10)$, buscando aproximar la raíz 10. Bisección comienza con el intervalo $[9, 10.5]$ mientras que Newton parte de la aproximación inicial 10.5. Se muestra la aproximación correspondiente.

Como puede verse, Bisección tardó 32 iteraciones en llegar a la precisión

Iteración	Bisección	Newton
1	9.75	10.0250000000000004
2	10.125	10.0000690607734803
3	9.9375	10.0000000005299245
4	10.03125	10
5	9.984375	-
6	10.0078125	-
7	9.99609375	-
⋮	⋮	-
30	10.000000004656613	-
31	9.9999999976716936	-
32	10.000000001164153	-

Tabla 5.6: Tabla comparativa de las iteraciones de Bisección y Newton-Raphson.

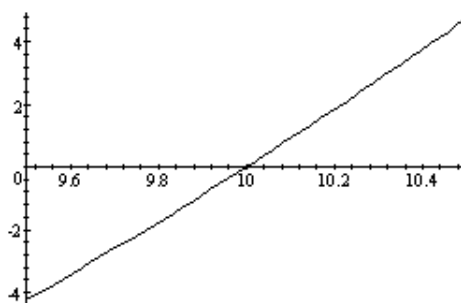
Iteración	Bisección	Newton
1	9.75	10.3362068965517242
2	10.125	10.2254672162094131
3	9.9375	10.1509187900252726
4	10.03125	10.100887568334679
5	9.984375	10.0673821864663324
6	10.0078125	10.0449769574305048
7	9.99609375	10.0300094473814365
8	10.001953125	10.0200173671539492
9	9.9990234375	10.0133498436609738
10	10.00048828125	10.0089020916573741
11	9.999755859375	10.0059357048437469
12	-	10.0039575711505133
⋮	-	⋮
18	-	10.0003475106001272

Tabla 5.7: Comportamiento de Bisección y Newton-Raphson ante una raíz múltiple.

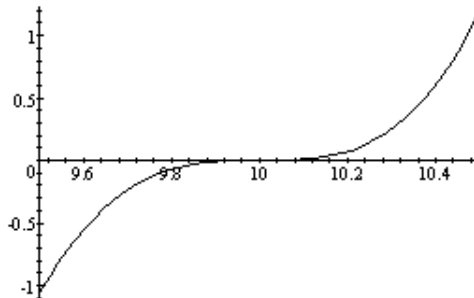
relativa indicada en el programa, mientras que Newton la superó en sólo 4 pasos. En este caso, Bisección ocupó el 96 % del tiempo de ejecución mientras que Newton sólo el 4 %, sin tomar en cuenta el resto del código²³. El método de Newton comienza a tener problemas cuando las raíces tiene multiplicidad. Por ejemplo, si se cambia la función a $(x - 1)(x - 10)^3$ el resultado es el de la tabla 5.7. La gráfica de ambas funciones se muestra en la figura 5.9.

Ahora es Newton el que tarda más iteraciones en llegar a la precisión exigida, aunque los tiempos de ejecución son similares: 48 % de bisección contra 52 % de Newton. Es de esperarse que la precisión sea menor debido a la forma de esta función cerca de la raíz, que ocasiona una reducción de la convergencia de cuadrática a lineal.

²³Los porcentajes hacen referencia a la proporción de tiempo de ejecución utilizado por cada una de las dos rutinas, sin contar el resto del programa.



a. $(x-1)(x-10)$.



b. $(x-1)(x-10)^3$.

Figura 5.9: Comportamiento de una raíz múltiple. El riesgo de que la derivada sea 0 aumenta con la multiplicidad.

Aproximación de la derivada

Uno de los problemas de este veloz método es que requiere de la derivada. Como se sabe, al derivar una función es posible que la expresión resultante sea más complicada. Para evitar este problema, puede utilizarse una aproximación. La forma más simple es aproximar la derivada con el cociente de diferencias

$$f'(x_n) \approx \frac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}}$$

utilizando cada par de aproximaciones sucesivas. Con este esquema, el denominador se va haciendo cada vez más pequeño y puede haber problemas. No es posible fijar un valor h y utilizar la fórmula

Este es el origen del método de la Secante, que se verá a continuación.

$$f'(x_n) \approx \frac{f(x_n + h) - f(x_n)}{h} \quad (5.62)$$

pues el resultado dependerá del punto donde se calcule. Por el teorema de Taylor, la ecuación 5.62 tiene un error de truncamiento de $-\frac{h}{2}f''(\xi)$, con $\xi \in (x, x+h)$.

Aproximando $f(x+h)$ y $f(x-h)$ y calculando su diferencia se obtiene una mejor aproximación de la derivada:

$$f'(x_n) \approx \frac{f(x_n + h) - f(x_n - h)}{2h}$$

y el error de truncamiento es de sólo $-\frac{h^2}{6}f'''(\xi)$. La iteración de Newton queda

$$x_{n+1} = x_n - \frac{2hf(x_n)}{f(x_n + h) - f(x_n - h)} \quad (5.63)$$

y debe ser claro que la constante $2h$ debe calcularse por anticipado. Esto significa que en vez de encontrar una recta tangente se utiliza una secante que pasa por dos puntos muy próximos a x_n .

No es posible determinar el mejor valor de h para cada x , por lo que se utiliza un proceso iterativo, reduciendo el valor de h hasta llegar a la mejor aproximación posible de $f'(x)$.

Al aproximar la derivada, se está cambiando la fórmula utilizada en el análisis de la velocidad de convergencia, por lo que esta se ve perjudicada. Al hacer dos evaluaciones de función por iteración el orden de convergencia baja de 2 a $\sqrt{2}$, aunque esto depende de qué tan buena sea la aproximación. Lo mejor es tener la expresión de la derivada, siempre que sea posible.

Problemas externos a los métodos

La fuente constante de problemas es la aritmética de punto flotante. Por ejemplo, con el polinomio $(x-1)(x-10)^5$, iterando con bisección con el intervalo inicial $[9.1, 11]$ se llega en 10 iteraciones al intervalo $[9.998048875, 9.99990234375]$

y ¡la raíz ya quedó fuera del intervalo! Todo porque la evaluación de un número menor que 10 dio positivo en vez de dar negativo.

Peor aún, los polinomios $p_1(x) = (x-1)(x-10)^3$ y $p_2(x) = (x-1)(x-10)^5$ son negativos en $[9, 10)$ y positivos en $(10, 11]$, por lo que $p(10-h) < 0$ y $p(10+h) > 0$ para toda $0 < h < 1$. ¡Pero ocurre que $p_i(10-h) > 0$ y $p_i(10+h) < 0$ cuando $h = .000003814697265625$!

Analizando el fenómeno anterior con más cuidado, es evidente que no fallan los métodos de aproximación de raíces, sino la aritmética de la computadora. Una pregunta interesante es si valdría la pena agregar código a bisección para verificar si en cada iteración se satisface que $f(a)f(b) < 0$. En caso de no cumplirse se podría abortar la aproximación indicando el motivo y regresar la actual mejor aproximación. Por supuesto, a costa del tiempo extra en cada iteración. Esto sólo si se desea que la rutina sea muy robusta y decida las respuestas de manera autónoma.

Continuando con Newton, a menos que se programe una función muy general, es adecuado tratar de simplificar el cociente $u(x) = f(x)/f'(x)$. Si la derivada se aproxima a 0 cerca de la raíz pueden surgir problemas con la convergencia. De hecho, evidentemente no es posible aplicar el algoritmo si f no tiene derivada en una vecindad de r .

5.4.2. Variantes de Newton

Una variante que presentan algunos libros para cuando la derivada es muy compleja es no actualizar la derivada cada iteración, sino decir cada cuántas iteraciones hacerlo. Así, si se actualiza cada tres, el método evaluará la derivada en las iteraciones 1, 3, 6, ... hasta lograr la convergencia. Esto significa que en la primera iteración, se calcula la x_1 a partir de x_0 con una tangente y x_2 y x_3 se aproximan con una secante como en la gráfica siguiente.

Luego, al actualizar la pendiente en la aproximación que está cerca del 1.1, la nueva recta tangente es

y el nuevo corte ocurre en el intervalo $[.4, .6]$.

Parece una buena idea, pero la convergencia no es cuadrática. Además, si la convergencia original es cuadrática, la cantidad de cifras correctas se duplica, siendo al inicio 1, 2, 4, 8, 16 y 32, lo que significa que 6 iteraciones se podría rebasar la precisión de la computadora. Entonces sólo se actualizará la derivada una vez además del cálculo inicial. Pareciera demasiado detalle de programación para un resultado tan cuestionablemente mejor, debido a la velocidad de las computadoras modernas.

La convergencia en general queda como algún número entre 1.6 y 2, excepto en la presencia de raíces múltiples, que se revisa a continuación.

Raíces múltiples

Este es el caso en el que Newton y la mayoría de los métodos tienen más problemas.

En el caso de raíces múltiples, por Schroeder [337] es sabido²⁴ que

$$\lim_{x \rightarrow r} \frac{u(x)}{x - r} = \frac{1}{m}$$

La iteración de Newton definida como

$$x_{n+1} = x_n - m u(x_n) \quad (5.64)$$

donde u se define como en (5.58) en la página 224, permite una convergencia cuadrática a r . El problema es que puede ser que m no se conozca por anticipado y si el valor estimado no coincide con la multiplicidad real, entonces los resultados no son adecuados y la convergencia puede estar comprometida (típicamente no se alcanza). La idea sería encontrar m en tiempo de ejecución. Utilizando el hecho de que los errores sucesivos de Newton, en el caso de una raíz múltiple, cumplen con

$$e_{n+1} = \frac{m-1}{m} e_n$$

es posible encontrar una aproximación de la multiplicidad como

$$m = \left\lfloor \frac{e_n}{e_n - e_{n+1}} \right\rfloor \quad (5.65)$$

cuando $x_0 > r$. De esta manera, con las dos primeras iteraciones de Newton es posible encontrar una estimación de la multiplicidad de la raíz que se está aproximando.

Aplicando lo anterior a $f(x) = (x-1)(x-10)^3$ como en el ejemplo anterior, se obtiene la tabla 5.8. Se utilizó el método de Newton normal y además se probó 5.64 con $m = 2$, $m = 3$ y el cálculo dinámico 5.65, indicado solo por m . Es claro que $m = 2$ se usó para experimentar una multiplicidad no adecuada. La tabla muestra la cantidad de iteraciones en que se alcanza la convergencia con una tolerancia de 10^{-15} . No se muestra el resultado con $m = 4$ pues ningún método nunca logró la convergencia en 100 iteraciones.

Los porcentajes de la última fila son el tiempo de ejecución empleado por cada método. El cálculo dinámico de m emplea dos pasos “lentos” para estimar la multiplicidad y luego en otras dos iteraciones se acerca cuadráticamente al resultado desde una aproximación bastante buena. De allí la diferencia con el caso de $m = 3$.

También se experimentó con la ecuación a $f(x) = (x-1)(x-10)$ como en el primer ejemplo comparativo de Bisección y Newton. En esta función sin raíz

²⁴Con el mérito correspondiente a la traducción de Stewart [338].

x_0	$m = 1$	$m = 2$	$m = 3$	m
11	29	11	3	4
10.90625	29	11	3	4
10.8125	29	11	3	4
10.71875	28	11	3	4
10.625	28	11	2	4
10.53125	27	10	2	4
10.4375	27	10	2	4
10.34375	26	10	2	4
10.25	25	10	2	4
10.15625	24	9	2	4
10.0625	22	8	2	4
Tiempo invertido:	58 %	23 %	7 %	10 %

Tabla 5.8: Iteraciones necesarias con multiplicidad fija y calculada para la función $f(x) = (x - 1)(x - 10)^3$ partir de diversos valores iniciales.

múltiple, para $m > 1$ no se logra la convergencia, mientras que con la estimación de m en tiempo de ejecución, se iguala el caso de Newton normal pues m se evalúa como 1.

La evaluación dinámica de m es un resultado interesante pues permite una convergencia cuadrática ante los casos de raíces simples y múltiples con una evaluación muy sencilla. En [205], Kahan comenta que si $f(x)$ tiene una raíz múltiple, la multiplicidad es el límite

$$m = \lim_{x \rightarrow r} \frac{[f'(x)]^2}{[f'(x)]^2 - f(x)f''(x)}$$

y para no requerir de $f''(x)$ sugiere utilizar una multiplicidad dinamicamente estimada con

$$m_n = \text{máx}\{1, \text{INT}(\frac{x_n - x_{n-1}}{u(x_n) - u(x_{n-1})})\} \quad (5.66)$$

$$x_{n+1} = x_n - m_n u x_n$$

que es una variante de (5.65). La diferencia mayor es que el cálculo dinámico (5.66) en cada iteración falla en el caso general al igual que (5.65), pudiendo llegar a una raíz distinta. Según Kahan, no se conoce manera de regular la variación de valores de m .

La solución para un caso particular con aceleración modesta se obtiene con (según la notación de Kahan en [205]) las iteraciones

$$N(x) = x - u(x) \quad (5.67)$$

$$M(x) = x - 2u(x) \quad (5.68)$$

y se utiliza (5.68) mientras las aproximaciones cumplan $x_n > r$ y $f'(x_n) > 0$. En cuanto $x_n < r$ (detectado por cambio de signo) se utiliza (5.67). La prueba

original la hizo Kahan para polinomios y posteriormente Werner Greub para todo tipo de funciones. La idea es que $M(x)$ se acerca más rápido que Newton en raíces con multiplicidad desconocida (incluso $m = 1$) y al saltarse la raíz el método de Newton es eficiente pues se está (o se espera estarlo) en una aproximación suficientemente buena.

Existe otra manera de acelerar el método de Newton y consiste en construir una función donde la raíz r tenga multiplicidad 1. Sea $f(x) = (x - r)^m g(x)$, la dicha función y $g(r) \neq 0$. Entonces u tiene una sola raíz ya que

$$\begin{aligned} u(x) &= \frac{f(x)}{f'(x)} \\ &= \frac{(x - r)^m g(x)}{m(x - r)^{m-1} g(x) + (x - r)^m g'(x)} \\ &= \frac{(x - r)g(x)}{mg(x) + (x - r)g'(x)} \end{aligned}$$

y es claro que u tiene una sola raíz en r pues $g(r) \neq 0$. Como

$$\begin{aligned} u'(x) &= \frac{[f'(x)]^2 - f(x)f''(x)}{[f'(x)]^2} \\ &= 1 - \frac{f(x)f''(x)}{[f'(x)]^2}, \end{aligned}$$

la función de iteración que aproxima la única raíz de u es

$$\begin{aligned} x_{n+1} &= x_n - \frac{u(x_n)}{u'(x_n)} \\ &= x_n - \frac{f(x_n)f'(x_n)}{[f'(x_n)]^2 - f(x_n)f''(x_n)} \\ &= x_n - \frac{f(x_n)}{f'(x_n) - L_f(x_n)/f'(x_n)} \end{aligned} \tag{5.69}$$

donde L_f es la aceleración convexa de la función f . Como la multiplicidad de r para la función u es 1, la convergencia será cuadrática, como el caso normal de Newton-Raphson.

El inconveniente de la iteración (5.69) es que se requiere la segunda derivada. Se puede aproximar con

$$f''(x) = \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}$$

pero el esquema iterativo sigue siendo mucho más complicado que el original. De nuevo, para poder aproximar adecuadamente la segunda derivada se requiere un proceso iterativo pues no es posible conocer por anticipado el mejor valor posible de h .

Usando aritmética de intervalos

Si se trata de confiabilidad, un intervalo es una respuesta segura.

A manera de ejemplo, se muestra a continuación la manera de utilizar aritmética de intervalos para aplicar el método de Newton-Raphson.

La raíz cuadrada

Una aplicación inmediata es el algoritmo para calcular la raíz cuadrada. Si se desea obtener la raíz de m , se construye la función $f(x) = x^2 - m$ y se aplica el método de Newton, obteniendo la recurrencia

$$x_{n+1} = \frac{1}{2}\left(x_n + \frac{m}{x_n}\right)$$

que fue empleada ya hace siglos por Herón.

Esta es la base del procedimiento que se emplea en las computadoras para la raíz cuadrada, aunque en la práctica, sólo se utiliza una cantidad fija de iteraciones para el refinamiento final, luego de las etapas de reducción de rango y aproximación polinomial, pues el método es muy estable.

5.4.3. Aceleración de Aitken

Alexander Craig (Alec) Aitken, (1895-1967), matemático inglés.

Como el método original de Newton-Raphson es lento cuando la raíz r es múltiple, Aitken diseñó un método conocido como δ^2 de Aitken con el que acelera cualquier proceso con convergencia lineal. Dada la sucesión de aproximaciones $\{x_n\}$, se define $\delta x_n = x_{n+1} - x_n$. Las potencias de δx_n se definen recursivamente:

$$\delta^k x_n = \delta^{k-1}(\delta x_n), \text{ para } k > 1.$$

Para $k = 2$, la fórmula $\delta^2 x_n = x_{n+2} - 2x_{n+1} + x_n$ es útil. Aitken demostró que si existe un número $|\alpha| < 1$ tal que

$$\lim_{n \rightarrow \infty} \frac{r - x_{n+1}}{r - x_n} = \alpha$$

entonces la sucesión $\{z_n\}$ definida como

$$z_n = x_n - \frac{(x_{n+1} - x_n)^2}{x_{n+2} - 2x_{n+1} + x_n} \quad (5.70)$$

converge más rápido, en el sentido de que $\lim_{n \rightarrow \infty} \left| \frac{r - z_n}{r - x_n} \right| = 0$. La iteración del método de Aitken se construye calculando z_n cada paso de Newton. Otras dos formas de la aceleración matemáticamente equivalentes son

$$\begin{aligned} z_n &= x_{n+1} - \frac{(x_{n+1} - x_n)(x_{n+2} - x_{n+1})}{x_{n+2} - 2x_{n+1} + x_n} & \text{y} \\ z_n &= x_{n+1} + \frac{1}{\frac{1}{x_{n+2} - x_{n+1}} - \frac{1}{x_{n+1} - x_n}} \end{aligned}$$

y la tercera es la más estable desde el punto de vista numérico.

La deducción anterior se debe a Aitken, pero es posible encontrar el mismo resultado considerando dos aproximaciones sucesivas de una sucesión con convergencia lineal. Sea $|c| < 1$. Se cumplen

$$\begin{aligned} |x_{n+1} - r| &= c |x_n - r| \\ |x_{n+2} - r| &= c |x_{n+1} - r| \end{aligned}$$

y despejando c de ambas e igualando las expresiones se obtiene

$$\frac{x_{n+1} - r}{x_n - r} = \frac{x_{n+2} - r}{x_{n+1} - r}.$$

Despejando r se obtiene de donde se obtiene la relación

$$r = x_n - \frac{(x_{n+1} - x_n)^2}{x_{n+2} - 2x_{n+1} + x_n}$$

que es la aceleración presentada.

El proceso de aceleración de Aitken es útil ante raíces múltiples, pero sólo puede emplearse en el caso de convergencias lineales. Si la convergencia es cuadrática, no hay ninguna aceleración.

Basado en este proceso de aceleración, surge el siguiente método de aproximación.

5.4.4. Método de Steffensen

El eficiente método de Newton se ve seriamente afectado ante la presencia de raíces múltiples. La técnica de aceleración de Aitken fue utilizada por el matemático norteamericano Arnold Steffensen para ayudar al método de Newton-Raphson en casos donde la convergencia es más lenta. El método consiste en iterar dos veces con Newton-Raphson (5.57) y luego acelerar con Aitken²⁵. De esta manera, en cada iteración se hacen tres aproximaciones, partiendo de tres iniciales. De la iteración anterior se tienen x_i y ahora se calculan

$$\begin{aligned} x_{i+1} &= x_i - u(x_i) \\ x_{i+2} &= x_{i+1} - u(x_{i+1}) \\ x_{i+3} &= x_i - \frac{(x_{i+1} - x_i)^2}{x_{i+2} - 2x_{i+1} + x_i} \end{aligned} \tag{5.71}$$

y se garantiza convergencia cuadrática aún cuando Newton-Raphson tiene problemas.

²⁵No todos los autores coinciden en atribuirle este método a Steffensen. De hecho, hay otro esquema adicional atribuido al mismo autor que se presenta en la página 241.

Iteración	Newton	Steffensen
1	10.3362068965517242	10.3362068965517242
2	10.2254672162094131	10.2254672162094131
3	10.1509187900252726	9.99431762991743078
4	10.100887568334679	9.99621215224659743
5	10.0673821864663324	9.99747494539873749
6	10.0449769574305048	9.9999992019996391
7	10.0300094473814365	-
8	10.0200173671539492	-
⋮	⋮	
27	10.0000090634125449	-

Tabla 5.9: Comparación de Newton y Steffensen ante una raíz múltiple.

x	$f(x)$	$f'(x)$
10.35	.01718...	.29648...
10.29	.00576...	.119...
10.24	<i>Paso de Aitken</i>	
9.9982...	$-4.945... \times 10^{-12}$	$-4.547... \times 10^{-13}$
10.875...		

Tabla 5.10: Iteraciones del método de Steffensen con $(x - 1)(x - 10)^5$ comenzando en 10.35, con un error relativo de 10^{-15} .

La principal desventaja de Steffensen es la potencial cancelación de cifras significativas en el denominador en cuanto las aproximaciones se van acercando más y más, por lo que hay que tomar precauciones.

Comparando los métodos de Newton y Steffensen, utilizando de nuevo el polinomio $(x - 1)(x - 10)^3$, que fue donde Newton tuvo problemas de convergencia. Utilizando ahora un error relativo más pequeño de 10^{-15} , los resultados de la comparación son los de la tabla 5.9.

En la tabla 5.9 se están contando como uno solo los tres pasos que se da en cada iteración de Stefensen. Considerando sólo estos dos métodos, los porcentajes de tiempo de ejecución son de 72 % para Newton y 28 % para Steffensen. Aún así, es posible encontrar condiciones donde el método de Steffenson se comporte mal. Por ejemplo, con la función $(x - 1)(x - 10)^3$ y el punto inicial 10.55, Newton y Steffenson realizan 32 y 33 iteraciones respectivamente.

En el caso de la función $(x - 1)(x - 10)^5$, ambos métodos agotan 100 iteraciones sin llegar a la aproximación exigida. El problema es esencialmente la poca estabilidad de los polinomios y la raíz múltiple en 10.

No es difícil encontrar condiciones donde la derivada cerca de la raíz es tan pequeña que la tangente corta el eje x demasiado lejos de la raíz. Por ejemplo, con un error relativo de 10^{-15} , el polinomio $(x - 1)(x - 10)^5$ y el método de Steffensen comenzando en 10.35, se tienen iteraciones de la tabla 5.10.

Otro ejemplo de mal comportamiento es el de la figura 5.10, donde se muestra de la iteración 11 a la 64 y se ve con claridad que los defectos numéricos de la

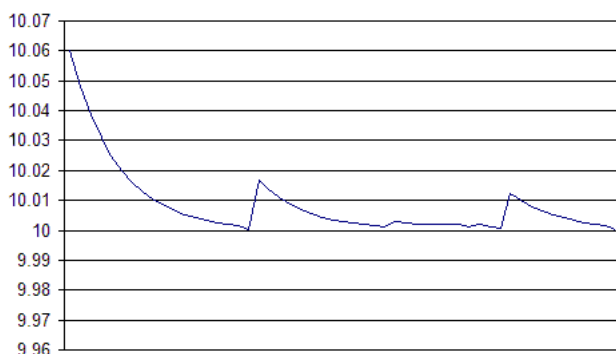


Figura 5.10: Problemas de Newton-Raphson al acercarse a una raíz múltiple. Se presentan las iteraciones de la 11 a la 64 para la función $(x-1)(x-10)^5$.

evaluación del polinomio ocasionan que no se acerque más a la raíz. De hecho, en la iteración 65 la aproximación se pasó a -5.999, lo que ocasionó la convergencia a la raíz más alejada 1.

Este trastabilleo es típico en raíces múltiples.

Ese comportamiento errático también llega a ocurrir en raíces simples y debe detectarse

5.4.5. Steffensen modificado

Steffenson da tres pasos en cada iteración, dos con Newton y uno con Aitken. Con algo de manipulación algebraica se puede reestructurar y mezclar los pasos 2 y 3. La modificación consiste los dos siguientes pasos.

Esta es una modificación del autor.

$$\begin{aligned} x_{i+1} &= x_i - u(x_i) \\ x_{i+2} &= x_i - \frac{(u(x_i))^2}{u(x_i) - u(x_{i+1})} \end{aligned} \quad (5.72)$$

Esto se obtiene de contraer la tercera fórmula del algoritmo original 5.71, sustituyendo x_{i+2} y x_{i+1} por sus definiciones²⁶. Esto significa que se dan los dos pasos finales de Steffensen en uno solo, con menor costo computacional, que tiene como ventaja las siguientes:

1. Ya no se requieren las evaluaciones de x_{i+2} , $f(x_{i+2})$ ni de su derivada. Debe recordarse que la cantidad de evaluaciones de funciones y operaciones elementales es en perjuicio del desempeño de los métodos.

En algunos casos, la convergencia teórica reduce de orden cuando el esquema iterativo es muy complicado.

²⁶Atreverse a presentar una variante de un método conocido, requiere más pruebas que las aquí presentadas. El autor promete un análisis más exhaustivo de la propuesta de modificación.

Iteración	Steffensen	Modificado
1	10.37662338	10.37662338
2	10.2834036	9.995245988
3	9.995245988	9.996434649
4	9.996434655	9.999999452
5	9.99732608	-
6	9.999999466	

Tabla 5.11: Comparación de Steffensen con la modificación del autor. En este caso la mejora es evidente.

x_0	Newton	Steffenson	Modificado
11	39	27	10
10.899	31	6	4
10.798	34	6	4
10.697	31	15	4
10.596	31	6	4
10.495	58	21	4
10.394	33	6	4
10.293	38	6	4
10.192	25	6	4
10.091	23	3	2

Tabla 5.12: Comparación de Newton, Steffensen y la modificación del autor para diversos valores iniciales.

2. La única cantidad nueva es $f(x_{i+1})/f'(x_{i+1}) = u(x_{i+1})$ pues las demás ya se calcularon.

La modificación del algoritmo de Steffensen invierte menos pasos. Por ejemplo, para la función $(x - 1)(x - 10)^3$, con precisión relativa de 10^{-15} , la comparación con el método original se muestra en la tabla 5.11 con el punto inicial 10.5, por poner el mismo caso de la comparación con Newton.

Como puede apreciarse, hay una ganancia en la cantidad de iteraciones, aunque en este caso, las evaluaciones del método modificado hacen que este consuma más tiempo que el original, para las condiciones iniciales establecidas.

Comparando los tres métodos en varios puntos iniciales, la tabla 5.12 muestra la cantidad de iteraciones que invierte cada uno en alcanzar la precisión exigida, para cada aproximación inicial a la raíz 10 de la misma función anterior.

De la tabla 5.12 se ve que la modificación 5.72 obtiene ventaja cuando Steffensen se desvía y tarda demasiadas iteraciones en alcanzar la aproximación. Por otra parte, es claro que entre más cercano esté x_0 de la raíz mejora la convergencia y que la modificación tiene menos problemas que el método original.

No fue difícil encontrar un experimento donde la modificación tuviera problemas. Por ejemplo, para la función $(x - 1)(x - 10)^5$, el algoritmo modificado no fue el único pero sí el que más se desvió a la raíz 1, en vez de localizar la raíz 10. Cuando se redujo la precisión de 10^{-15} a 10^{-12} ningún método se desvió a la otra raíz, lo que deja ver la dependencia de la precisión exigida para el buen

No se puede esperar gran cosa de tan solo una simplificación algebraica

comportamiento de los algoritmos. Queda claro por simple sentido común: no es conveniente exigir más precisión que la realmente necesaria.

En la literatura se encuentra la iteración que también recibe el nombre de Steffensen, que consiste en

$$x_{n+1} = x_n - \frac{[f(x_n)]^2}{f(x_n + f(x_n)) + f(x_n)} \quad (5.73)$$

y aparece en una gran cantidad de referencias de todo tipo.

Falta la variante de Multiple-precision zero-finding methods and the complexity of elementary function evaluation - Brent.

Al aplicar Aitken, el método de Newton o Steffensen se acelera la débil convergencia lineal en los casos de raíces múltiples o muy cercanas, logrando de nuevo una convergencia cuadrática.

Hay métodos de convergencia cúbica casi tan antiguos como el de Newton-Raphson, que algunos autores los vieron como generalizaciones del original.

Si se desea una convergencia superior, se pueden utilizar diversos métodos, donde en general se requiere continuidad hasta derivadas superiores (en proporción al orden). Más adelante se presentan algunos esquemas donde no se requieren derivadas (como el de bisección).

5.4.6. Método de Halley

Uno de los primeros métodos con convergencia cúbica es el método de Halley, publicado en 1694. Según comenta Traub en [367], Halley encontró el método tras generalizar una técnica que le impresionó de Fautet Lagny de 1692 para encontrar raíces (en particular cúbicas) de potencias. Lo que lo impresionó fue que $\sqrt[3]{a^3 + b}$ se encuentra siempre entre

Edmond Halley (1654-1742), famoso por el cálculo de la trayectoria y periodicidad del cometa que lleva su nombre.

$$a + \frac{ab}{3a^3 + b} \quad \frac{a}{2} + \sqrt{\frac{a^2}{4} + \frac{b}{3a}}$$

cuando $a^3 \gg b > 0$.

Hay dos versiones del método, llamadas racional e irracional. Como Newton aproxima la raíz con una recta, utilizando un término más en la expansión de Taylor se está utilizando una parábola tangente en x_0 , que se ajusta mejor a la función de interés.

La *forma irracional* parte de la aproximación polinomial de la función f

$$p(x) = f(x_n) + (x - x_n)f'(x_n) + \frac{1}{2}(x - x_n)^2 f''(x_n) \quad (5.74)$$

e igualando a cero y despejando $x - x_n$ como

$$x - x_n = \frac{-f'(x_n) \pm \sqrt{[f'(x_n)]^2 - 2f''(x_n)f(x_n)}}{f''(x_n)}.$$

Esta fórmula tiene dos problemas: 1- qué signo tomar del radical y 2- que cuando x_n se acerca a la raíz r , $f(x_n)$ es casi cero, por lo que el discriminante es casi $f'(x_n)$, ocasionando la diferencia de dos cantidades casi iguales en el numerador.

El signo del radical debe tomarse de tal manera que el numerador sea lo menor posible, pero depende de $f'(x_n)$, así que se puede dividir arriba y abajo entre $f'(x_n)$, quedando

$$x - x_n = \frac{-1 \pm \sqrt{1 - \frac{2f''(x_n)f(x_n)}{[f'(x_n)]^2}}}{f''(x_n)/f'(x_n)}$$

pero sólo se resolvió el problema del signo y falta la potencial diferencia entre dos cantidades casi iguales a 1. Pero haciendo uso de que $(1 - \sqrt{1 - a})(1 + \sqrt{1 - a}) = a$, que es la transformación usada usada en la página 294 en (6.37), la fórmula queda

$$x_{n+1} = x_n - \frac{2f(x_n)}{f'(x_n) \pm \sqrt{[f'(x_n)]^2 - 2f(x_n)f''(x_n)}} \quad (5.75)$$

que tiene parecido con la iteración de Newton, con unos términos adicionales. Este esquema tiene convergencia cúbica.

En la *forma racional* del método de Halley, se parte de nuevo de la aproximación (5.74)

$$f(x_n) + (x - x_n)f'(x_n) + \frac{1}{2}(x - x_n)^2 f''(x_n) = 0$$

y despejando $x - x_n$ como en la ecuación general de segundo grado:

$$x - x_n = -\frac{f(x_n)}{f'(x_n) + \frac{1}{2}(x - x_n)f''(x_n)}.$$

Ahora, partiendo de la aproximación $x = x_n - u(x_n)$, se sustituye $x - x_n$ por $-u(x_n)$, dando:

$$\begin{aligned} x_{n+1} &= x_n - \frac{f(x_n)}{f'(x_n) - \frac{1}{2}u(x_n)f''(x_n)} \\ &= x_n - \frac{f(x_n)f'(x_n)}{[f'(x_n)]^2 - \frac{1}{2}f(x_n)f''(x_n)} \end{aligned} \quad (5.76)$$

y se tiene la forma racional, que es la más conocida de las dos²⁷, también llamada *método de hipérbola tangente* debido a que cuando $x_{n+1} = 0$ se tiene la intersección con el eje x de una hipérbola que es osculadora a $y = f(x)$ en x_n . En el trabajo [334] de Scavo se da una interpretación geométrica del método de Halley, presumiblemente similar a los razonamientos de finales del siglo XVII.

²⁷Halley prefería la forma irracional argumentando mayor precisión, pero ahora se sabe que no encontró los contraejemplos adecuados.

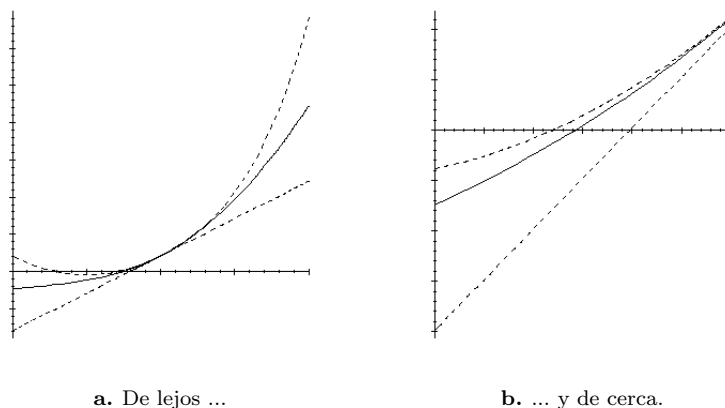


Figura 5.11: Comparación gráfica de los métodos de Newton y Halley. La línea sólida es la función $f(x) = e^x - 8$, la recta es la aproximación de Newton.

Es curioso que Taylor dedujo su teorema (ver página 359) alrededor de 1712 tras identificar las derivadas en la formulación original de Halley, según [110] (véase (A.36) en la página 359).

La convergencia se garantiza si se cumple que

$$2|x_1 - x_0| |g''(x)| \leq |g'(x_0)|$$

y que todas las x_k estarán en el intervalo $[x_0, x_0 + 2h]$ si $h = x_1 - x_0 > 0$ o en $[x_0 + 2h, x_0]$ si $h < 0$. En caso de no cumplirse estas condiciones, la concavidad de la parábola aproximante la curvará antes de tocar el eje x , con lo que no ocurrirá el corte (como el que sí se ve en la figura 5.11a) y por lo tanto no habrá raíz real que sea la siguiente aproximación.

Ejemplo. Sea la función $f(x) = e^x - 8$. Si se comienza a iterar en $x_0 = 3$ se obtienen la recta de Newton y la parábola de Halley que se muestran en la figura 5.11. Es evidente que el corte de la parábola de Halley pasa más cerca de la raíz.

En [5], el matemático alemán Götz Alefeld muestra (entre otras cosas) que el método de Halley (5.76) converge, partiendo de que es posible derivarlo tras aplicar Newton a

$$g(x) = \frac{f(x)}{\sqrt{f'(x)}}.$$

Como la derivada de g es

$$g'(x) = \sqrt{f'(x)} - \frac{1}{2} \frac{f(x) f''(x)}{\sqrt{f'(x)}^3},$$

aplicando la iteración de Newton se obtiene

$$\begin{aligned}
 x &= x - \frac{\frac{f(x)}{\sqrt{f'(x)}}}{\sqrt{f'(x)} - \frac{1}{2} \frac{f(x)f''(x)}{\sqrt{f'(x)}^3}} \\
 &= x - \frac{f(x)}{f'(x) - \frac{1}{2} \frac{f(x)f''(x)}{f'(x)}} \\
 &= x - \frac{f(x)f'(x)}{[f'(x)]^2 - \frac{1}{2} f(x)f''(x)}
 \end{aligned}$$

que es el método de Halley.

Para probar la convergencia del método de Halley, se utiliza una variante del teorema de Kantorovich, debida a Ostrowski, que puede consultarse en el artículo de Dong Cheng, donde además se demuestra su convergencia cúbica.

5.4.7. Variantes con cuadratura

Se han desarrollado diversos esquemas iterativos de punto fijo que evitan el uso de derivadas. Los ejemplos clásicos son bisección, secante y la regla de falsa posición, pero hay muchos otros. Otra forma de no usar derivadas es mediante aproximaciones, por ejemplo, el algoritmo LZ4 de Le, en [237], que utiliza la iteración de Chebyshev (pag. 248) como paso principal sin utilizar explícitamente ni f' ni f'' .

Por otra parte, Amat presenta en [10] un método llamado de dos pasos atribuido a Potra y Ptak, que también tiene convergencia cúbica pero sin usar la segunda derivada. El esquema es

$$x_{n+1} = x_n - \frac{f(x_n) + f(x_n - u(x_n))}{f'(x_n)} \quad (5.77)$$

y durante mucho tiempo fue el único conocido con estas características.

Una interesante manera de deducir el método de Newton se basa en una conocida y sencilla técnica de integración, llamada Newton-Cotes. Como la integral

$$\int_{x_n}^{x_{n+1}} f(x) dx$$

es el área bajo la curva de f en el intervalo $[x_n, x_{n+1}]$, entonces una aproximación muy burda del área es el rectángulo cuya base es $x_{n+1} - x_n$ y su altura es $f(x_n)$ o $f(x_{n+1})$. Con cualquiera de las dos alturas se obtiene una burda aproximación a la integral que interesa.

Aprovechando esto, se puede notar que si se usa el teorema de Newton

$$f(x) = f(x_n) + \int_{x_n}^x f'(t) dt$$

y se aproxima la integral con un rectángulo de altura $f'(x_n)$ y base $x - x_n$, se obtiene

$$\int_{x_n}^x f'(t) dt \approx (x - x_n)f'(x_n).$$

Haciendo $f(x) = 0$ y despejando x se obtiene $x_{n+1} = x_n - u(x_n)$, que es el método de Newton.

Esta manera de deducir se deriva de la aproximación de la integral con un rectángulo, conocido como la regla de Newton-Cotes de orden cero, que es la aproximación con mayor error. Aproximando la integral con un rectángulo de altura $\frac{1}{2}(f'(x) + f'(x_n))$ se obtiene una mejor aproximación que lleva a la iteración de punto fijo

Conocido como la regla del trapecio.

$$x_{n+1} = x_n - \frac{2f(x_n)}{f'(x_n) + f'(x_{n+1})}$$

que es una ecuación implícita en x_{n+1} , es decir, no se puede despejar x_{n+1} . Lo que sí se puede hacer es sustituir en el argumento de f' con $x_{n+1} = x_n - u(x_n)$ y reescribir

$$x_{n+1} = x_n - \frac{2f(x_n)}{f'(x_n) + f'(x_n - u(x_n))}. \quad (5.78)$$

Este método iterativo, presentado por Weerakoon en [385], tiene convergencia cúbica, como el método de Halley, pero sin requerir la segunda derivada. Otra forma que también tiene convergencia cúbica se obtiene si en vez de utilizar un trapecio se aproxima la integral con la regla del punto medio, como hicieron Frontini y Sormani en [125, 126]. En ese caso, se parte de la aproximación

$$\int_{x_n}^x f'(t) dt \approx (x - x_n)f'(x_n/2 + x/2)$$

y la iteración queda

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n - \frac{1}{2}u(x_n))}. \quad (5.79)$$

No es difícil ver que tanto (5.78) como (5.79) tienen cierto parecido, por el argumento $x_n - k u(x_n)$, donde k vale 1 y $\frac{1}{2}$ respectivamente.

Es natural pensar que si se utilizan reglas de Newton Cotes de orden mayor, como Simpson, Simpson 3/8 o la regla de Bule se llegará a métodos iterativos con convergencia mayor, pero ya en 2003, Frontini y Sormani demostraron en [126] que todas las derivaciones de Newton-Cotes de orden 1 en adelante alcanzan convergencia cúbica como máximo.

Eso sin considerar el fenómeno de Runge, donde en algunos casos, los polinomios de grado alto van degradando cada vez más la aproximación de la integral debido a su sensibilidad.

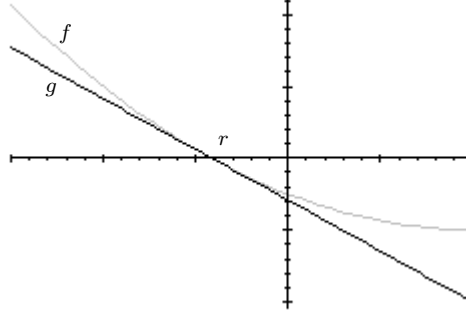


Figura 5.12: Aceleración convexa de Newton.

5.4.8. Aceleración convexa - Super Halley

En el caso de que la función f cumpla con ciertas condiciones de convexidad (similares a las de la página 367) es posible acelerar el método de Newton, como presentan Gutiérrez y Hernández en [158]²⁸. Ellos demuestran que su iteración es una aceleración de Newton si f satisface $f'(x) < 0$ y $f''(x) > 0$ para $a \leq t \leq r$, con $f(r) = 0$. Si la segunda derivada de f es no decreciente en $[a, r]$ entonces la convexidad logarítmica $L_f \leq \frac{1}{2}$ en ese intervalo.

Si x_n es la n -ésima aproximación con Newton, entonces puede obtenerse una sucesión z_n acelerada con el esquema iterativo

$$z_n = x_n - \left[1 + \frac{L_f(x_n)}{2(1 - L_f(x_n))} \right] \frac{f(x_n)}{f'(x_n)} \quad (5.80)$$

que a diferencia de la aceleración de Aitken (5.70) hace uso de la geometría de f . La restricción que debe cumplirse es $f'''(x) \geq 0$ para toda $x \in [a, r]$. A esto se le conoce como *aceleración convexa*.

El esquema iterativo (5.80) lo obtienen de hacer $g(z) = f'(r)(z - r)$, que es la recta tangente a f en r , por lo que cruza el eje x en la raíz, como muestra la figura 5.12. Como en toda recta, $L_g = 0$. De esta manera, $g(z)$ define una sucesión que converge a r más rápido.

Como r no se conoce, se aproxima g con el polinomio de Taylor

$$g(z) = f(z) - \frac{f''(r)}{2}(z - r)^2$$

reescribiendo como

$$g(z_n) = f(z_n) - \frac{f''(z_n)}{2}(z_n - z_{n+1})^2$$

²⁸Gutiérrez y Hernández generalizan sus resultados en espacios de Banach y los utilizan en un ejemplo para resolver un sistema de ecuaciones no lineales.

de donde se deduce la iteración $z_{n+1} = z_n - \frac{g(z_n)}{g'(z_n)}$ que se desarrolla y obtiene (5.80).

Según muestran Gutiérrez y Hernández en [158], la iteración (5.80) es de orden cúbico para raíces simples y degrada a orden lineal en el caso de raíces múltiples. Para polinomios cuadráticos este método alcanza convergencia cuártica en raíces simples, siendo una iteración de (5.80) equivalente a dos iteraciones de (5.57).

5.5. Convergencias superiores

Los métodos descritos, tanto para polinomios como para funciones en general trabajan con órdenes de convergencia bajos, principalmente superlineales, 2 y 3. Algunos logran convergencias altísimas, pero su utilidad es más teórica que práctica.

Es posible encontrar procedimientos con orden cada vez mayor. De hecho, no hay limitante teórica con respecto al orden de convergencia que se puede alcanzar con un proceso iterativo de punto fijo (ver 5.15). Estos descubrimientos no son nada nuevos y se deben a personajes como Chebyshev, Schröder y más recientemente a Householder.

No tienen gran utilidad práctica, pero son buena aportación teórica. El hecho de que actualmente sea complicado calcular los valores involucrados no limita que en el futuro se encuentren maneras de hacerlos de alguna manera que permita su uso en la práctica.

5.5.1. Iteración de Chebyshev

Hay un esquema iterativo que también es conocido como método de Chebyshev o Super Newton:

$$x_{n+1} = x_n - u(x) - u^2(x_n) \frac{f''(x_n)}{2f'(x_n)} \quad (5.81)$$

Pafnuty Lvovich Chebyshev (1821 – 1894), gran matemático ruso, su nombre aparece en muchas áreas de matemáticas.

y puede derivarse del polinomio de Taylor. Su orden de convergencia también es cúbico, pero adolece de lo mismo que (5.69) y otros métodos cúbicos, pues requiere de la segunda derivada.

La iteración (5.81) es un caso particular de una regla más general descubierta por Chebyshev. Se basa en la representación de la inversa de $f(x) = y$, como $x = \mathcal{F}(y)$ mediante la serie de Taylor. Si x_n está suficientemente cerca de una raíz de f y si $c_0 = \beta = f(x_n)$ y $f'(x_n) \neq 0$, entonces

$$x_{n+1} = \mathcal{F}(y) = x_n + \sum_{n \geq 1} d_n (y - b)^n \quad (5.82)$$

y los coeficientes d_n se definen recursivamente de la identidad $x \equiv \mathcal{F}(f(x))$ usando los coeficientes de Taylor c_n de la función $f(x) = \sum_{n \geq 0} d_n(y - \beta)^n$. Haciendo $y = 0$ en (5.82), se obtiene la iteración

$$\begin{aligned} x_{n+1} = & x_n - u(x) - u^2(x_n) \frac{f''(x_n)}{2f'(x_n)} + \\ & - u^3(x_n) \left[\frac{1}{2} \left(\frac{f''(x_n)}{f'(x_n)} \right)^2 - \frac{f^{(3)}(x_n)}{6f'(x_n)} \right] + \\ & - u^4(x_n) \left[\frac{5}{8} \left(\frac{f''(x_n)}{f'(x_n)} \right)^3 - \frac{5f''(x_n)f^{(3)}(x_n)}{12f'(x_n)} + \frac{f^{(4)}(x_n)}{24f'(x_n)} \right] - \dots \end{aligned} \quad (5.83)$$

El orden de convergencia de (5.83) depende de la cantidad de términos que se empleen²⁹. Si se usan dos, se tiene el caso de la iteración de Newton. De usar tres, se obtiene Super Newton (5.81).

5.5.2. Fórmula de Schröder

Ernst Schröder (1841 - 1902), algebrista alemán, primero en usar el término lógica matemática.

Aunque en estas fechas resulta un tanto exagerado, pod'ía decirse.

En 1870, el Schröder publicó [337] uno de los artículos que mayor impacto han causado en la teoría de la solución de ecuaciones no lineales, funciones iterativas. Tanto que Householder decía que si una nueva publicación no hacía referencia al artículo de Schröder, entonces posiblemente había redescubierto algo incluido allí. En su artículo, Schröder clasificó los métodos en dos grandes clases. A la primera pertenecen los métodos iterativos que aplican alguna fórmula de punto fijo como (5.15) de la página 185, de los que el método de Newton es un buen representante. La segunda clase consiste de los métodos que construyen una sucesión de funciones $F_i(x)$ cuyo límite es una raíz r que depende de la selección del valor x . El método de Bernoulli pertenece a esta clase.

El artículo fue traducido en 1992 por G. W. Stewart [338]. La fórmula de Schröder, en su notación original (compacta) es

$$x_{n+1} = x + \sum_{\alpha=1}^{w-1} (-1)^\alpha \frac{f^\alpha}{\alpha!} \left(\frac{1}{f'} \partial \right)^{\alpha-1} \frac{1}{f'} - f^w \times \varphi_w \quad (5.84)$$

y equivale al esquema de Chebyshev (5.83). Al parecer, este resultado se publicó 30 años después del de Chebyshev. La expresión $(\frac{1}{f'} \partial)^\alpha$ es un operador, lo que implica que $\frac{1}{f'}$ debe ser diferenciado sucesivamente α veces.

En [366], Traub comenta que la llamada iteración de Euler, también es equivalente a (5.83) y (5.84). Otro esquema iterativo de orden n lo derivó Traub en [365] pero exclusivamente para las raíces de $x^n - a = 0$.

²⁹Chebyshev derivó esta fórmula a los 17 años, lo que le valió una medalla de plata y ser considerado como el candidato más sobresaliente en 1837 o 38.

5.5.3. Householder

Un conocido método con convergencia cúbica se debe al matemático norteamericano Alston Scott Householder (1904-1993), más conocido por haber puesto orden en 1950 a la entonces caótica área del álgebra lineal numérica. El esquema iterativo es:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \left[1 + \frac{f(x_n)f''(x_n)}{2[f'(x_n)]^2} \right] \quad (5.85)$$

y puede obtenerse de la forma racional del método de Halley, reescribiéndolo como

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \left[1 - \frac{f(x_n)f''(x_n)}{2[f'(x_n)]^2} \right]^{-1}$$

tras reemplazar $(1 - a)^{-1}$ por la aproximación $1 + a + O(a^2)$. Como ya es de esperarse, la tasa de convergencia baja al enfrentarse a raíces múltiples.

Tanto el método de Newton como el de Halley son casos particulares de una regla más general encontrada por Householder y publicada en [174]. Sea $g(x) = 1/f(x)$ con derivadas continuas hasta la $n + 2$. Entonces la iteración definida como

$$x_{n+1} = x_n + (n + 1) \frac{g^{(n)}(x_n)}{g^{(n+1)}(x_n)} \quad (5.86)$$

converge localmente a una raíz con orden de convergencia $n + 2$. Newton se obtiene con $n = 0$ mientras que Halley con $n = 1$.

Si ocurre que las primeras $n+2$ derivadas son continuas y no se anulan en una vecindad de la raíz, se puede obtener un método que, a partir de un valor inicial x_0 adecuadamente aproximado, en un solo paso alcance la exactitud máxima para variables en doble precisión, salvo errores de redondeo. Por ejemplo, para $n = 2$ se tiene la iteración

$$x_{n+1} = x_n + \frac{\frac{1}{2}(f(x_n))^2 f''(x_n) - f(x_n)(f'(x_n))^2}{(f'(x_n))^3 - f(x_n)f'(x_n)f''(x_n) + \frac{1}{6}(f(x_n))^5}$$

que se podría utilizar en un programa verificando que el denominador no es cero. Para $n = 3$ se tendría

$$x_{n+1} = x_n + 4 \frac{6f^2 f' f'' - 6f(f')^3 - f^3 f'''}{8f' f''' + 6(f'')^2 - f^3 f'''' - 36f(f')^2 f'' + 24(f')^4}$$

y se ha omitido el argumento (x_n) por razones de espacio.

De esta manera se pueden generar iteraciones de cualquier orden de convergencia, pero que en la práctica tienen muchos inconvenientes.

Capítulo 6

Raíces de ecuaciones II: polinomios

Aproximar las raíces de un polinomio es un caso particular del problema más general de raíces de ecuaciones, presentado en el capítulo anterior. El problema es muy amplio por lo que se acota en este texto a polinomios de una sólo variable.

Aún si se considera la anterior restricción, presentar los detalles todos los métodos existentes queda fuera de las intenciones de este texto. Se comentan las características y ventajas de métodos importantes, algunos historicamente importantes y otros muy sofisticados que han aparecido recientemente en la literatura especializada (última década en general). Siempre con la perspectiva de la aritmética de punto flotante.

6.1. Sobre los polinomios y sus raíces

Los polinomios son entidades de gran importancia teórica y práctica. Tal relevancia radica no sólo en el hecho de que son sencillos de manipular algebraicamente, sino que además sólo se requieren operaciones aritméticas básicas: suma (o resta) y multiplicación.

Muchos fenómenos naturales, modelos físicos o procesos de ingeniería y ciencias sociales se pueden representar con polinomios o bien aproximarse mediante ellos con gran exactitud. Es común que estos polinomios sean de grado no muy alto, pero hay algunas excepciones.

Una de las aplicaciones más importantes de los polinomios es en el cálculo de valores propios de una matriz, que comunmente representa valores importantes en muchos modelos matemáticos. El problema se plantea de tal manera que las raíces del polinomio son los valores propios. Tales matrices pueden suelen ser

muy grandes, lo que da polinomios de alto grado¹.

Aunque este capítulo hace énfasis en las técnicas para polinomios con coeficientes reales, también se presentan métodos que requieren aritmética compleja, así como algunos métodos que trabajan con coeficientes complejos.

6.1.1. Organización del capítulo

Tal y como el capítulo dedicado a las raíces de funciones en general, en este se presenta el problema con exactitud y se muestran muchos resultados de importancia teórica o práctica en la búsqueda de raíces de polinomios.

6.1.2. Presentación del problema

Las raíces del polinomio

$$p(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0 \quad (6.1)$$

son todos aquellos n valores r_i , no necesariamente distintos, que cumplen con $p(r_i) = 0$, de tal manera que el polinomio p puede escribirse factorizado como

$$p(x) = (x - r_1)^{m_1} (x - r_2)^{m_2} (x - r_3)^{m_3} \cdots (x - r_t)^{m_t} \quad (6.2)$$

donde $m_i \geq 1$ es la multiplicidad de la raíz r_i y se cumple que su suma es el grado de p , es decir, $\sum_{i=1}^t m_i = n$.

Es de notarse que la definición 3 de raíz en $\mathbb{F}$ (página 176), aplica aquí tal y como en el capítulo anterior, al igual que todos los aspectos relacionados. Se reproduce a continuación: $x \in \mathbb{F}$ es raíz del polinomio p si

$$|\text{fl}(p(x))| \leq \epsilon$$

con $\epsilon > 0$ y se dice que x es una buena aproximación de la raíz exacta r .

Ejemplo 48. El polinomio $x^7 - 5x^6 + 2x^5 + 18x^4 - 11x^3 - 25x^2 + 8x + 12$ se factoriza como

$$(x + 1)^3 (x - 2)^2 (x - 3) (x - 1)$$

y la suma de los exponentes (las multiplicidades) es 7, el grado del polinomio.

Una forma de saber si el polinomio p posee raíces múltiples es calcular el máximo común divisor (gcd por sus siglas en inglés) entre p y su derivada p' con el algoritmo de Euclides². Si

$$\text{gcd}(p(x), p'(x)) = K \in \mathbb{R} \quad (6.3)$$

¹Llama la atención que Higham, en su libro [168], un verdadero pilar del análisis de errores y estabilidad, comente que el tema de raíces de polinomios pierde importancia desde que se plantea como la resolución de valores propios con el método QR. El autor ha encontrado muchas investigaciones, tan recientes como se desee, en nuevas brechas de este problema.

²Euclides lo presentó en el libro 7 de sus Elementos, pero se sabe que ya existía mucho antes.

es decir, es una constante, entonces p no posee ninguna raíz múltiple. De otra forma, si el resultado no es una constante sino otro polinomio, existe al menos una raíz múltiple.

La justificación de (6.3) es simple. Si el polinomio p de grado n tiene una raíz r con multiplicidad $m > 1$, entonces se puede reescribir con la forma $p(x) = (x - r)^m q(x)$ y q es forzosamente un polinomio de grado $n - m$ y $q(r) \neq 0$. Es claro que $\gcd(p(x), p'(x))$ no es una constante pues como

$$\begin{aligned} p'(x) &= m(x - r)^{m-1}q(x) + (x - r)^m q'(x) \\ &= (x - r)^{m-1}(mq(x) + (x - r)q'(x)) \end{aligned}$$

entonces $(x - r)^{m-1}$ es factor común a ambos y no es una constante si $m > 1$.

Ejemplo 49. *Considérese los siguientes casos de máximo común divisor entre un polinomio y su derivada.*

$$\gcd(x^4 - 10x^3 + 35x^2 - 50x + 24, 4x^3 - 30x^2 + 70x - 50) = 1$$

por lo que $x^4 - 10x^3 + 35x^2 - 50x + 24$ no posee raíces múltiples, mientras que

$$\gcd(x^4 - 11x^3 + 44x^2 - 76x + 48, 4x^3 - 33x^2 + 88x - 76) = x - 2$$

por lo que hay al menos una raíz múltiple en $x^4 - 11x^3 + 44x^2 - 76x + 48$.

Aún cuando los polinomios son fáciles de manipular algebraicamente, la limitante de la idea anterior es la precisión de las computadoras. Con coeficientes enteros no hay problema mientras la precisión de las mantisas alcance, pero lo común es que los coeficientes sean números reales, que serán representados con sus aproximaciones flotantes.

En la práctica, muchos métodos de aproximación prefieren utilizar *polinomios mónicos*, que cumplen con $a_n = 1$, lo que facilita algunos cálculos. Esta y otras formas son comunes y generalmente se seleccionan para facilitar la deducción del método o su implementación.

6.1.3. Teorema Fundamental del álgebra

En el caso de polinomios, el resultado teórico más importante es, sin duda alguna, el siguiente, que garantiza que las raíces n raíces de p , definido en (6.1), $\mathbb{C}$ es cerradura algebraica de $\mathbb{R}$ existen.

Proposición 15 (Teorema Fundamental del álgebra (TFA)). ³ *Todo polinomio de grado n con p definido como*

$$p(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$$

³Este resultado es conocido desde hace muchos siglos y se utilizaba sin demostración, considerándolo como verdadero. La primera demostración, de hace 2 siglos, se debe a Gauss. El TFA es pilar de gran cantidad de resultados y se considera que pertenece más al área de análisis matemático que al álgebra.

posee n raíces y que pueden ser reales o complejas, es decir, de la forma $a + ib$, donde $i^2 = \sqrt{-1}$.

Es común que también se presente diciendo que todo polinomio en una variable de grado $n = 1$ con coeficientes reales o complejos tiene por lo menos una raíz (real o compleja).

Si un polinomio tiene sólo coeficientes reales, llamado *polinomio real*, y el número complejo $z = a + ib$ es raíz de p , entonces también es raíz de p su *conjugado complejo*, definido como $\bar{z} = a - ib$.

El conjunto de las raíces del polinomio real p es un conjunto que está contenido en el de los números complejos $\mathbb{C}$ y puede definirse matemáticamente como la imagen inversa del cero, es decir, $p^{-1}(0) = \{z \in \mathbb{C}, p(z) = 0\}$, donde de nuevo las raíces no son necesariamente distintas.

La limitada precisión de las computadoras obliga a que todos los coeficientes sean racionales (números en $\mathbb{F}$) y las raíces se obtengan sólo aproximadas, pero toda la teoría se plantea considerando el caso de polinomios reales. Es la manera como fue desarrollada durante siglos, incluso miles de años en el caso de algunos conceptos.

Si los coeficientes del polinomio p son números complejos y no sólo reales, la existencia de una raíz compleja $z = a + bi$ no implica que su conjugado $\bar{z}$ también sea raíz de p .

Se calculará como una raíz con multiplicidad 2.

Una limitante natural para todas las técnicas del capítulo es la cercanía de las raíces. Si hay dos o más raíces entre dos números de punto flotante contiguos, como $(x, \text{nextUp}(x))$ o $(1, 1 + \varepsilon_M)$, no habrá manera de distinguirlas. A lo más las raíces pudieran aproximarse con los límites mismos del intervalo, pero considerando la sensibilidad numérica de los polinomios, esta es una tarea excesiva para la aritmética nativa de los procesadores. Ver sección 5.1.3.

Los polinomios son entidades mal condicionadas numéricamente. El número de condición de un polinomio se definió en (A.10), en la página 348 y se reproduce a continuación.

$$\text{cond}(p, x) = \frac{\sum_{i=0}^n |a_i| |x|^i}{|\sum_{i=0}^n a_i x^i|} = \frac{\bar{p}(x)}{|p(x)|} \quad (6.4)$$

Interesa más el condicionamiento alrededor de las raíces, donde ocurrirá la última etapa de la aproximación.

donde se ve que la condición depende tanto de los signos de los coeficientes como del valor de x , por lo que un polinomio puede ser más estable en unas regiones que en otras. Es claro que el condicionamiento es ∞ en las raíces, por lo que hay problemas cerca de los puntos de interés.

Desde los inicios del cómputo numérico, como [391, 392], se sabe que los problemas de condicionamiento y estabilidad aumentan mucho con el grado del polinomio y la distribución y multiplicidad de sus raíces.

Una referencia inicial obligatoria es el libro clásico de Teoría de Ecuaciones de Uspensky, cuya primera edición data de 1948. Se recomienda al lector interesado

en más detalles revisar la página web “A bibliography on roots of polynomials”, de John Michael McNamee⁴, muy actualizada.

6.1.4. Etapas típicas en la búsqueda de raíces de polinomios

En la práctica, los programas que buscan raíces de polinomios requieren sólo los coeficientes y ninguna información extra, a diferencia de muchos métodos indicados en libros de análisis numérico donde se parte de una solución inicial que supone ser suficientemente cercana.

Un programa profesional debe trabajar como caja negra, recibiendo como entrada sólo los coeficientes y dando como salida las raíces. El usuario no necesita tener idea de lo que ocurre dentro del programa, aunque la documentación debe aclarar hasta cierto punto lo que hace.

Al menos es conveniente saber qué método base se aplica.

No todas las rutinas que aproximan raíces de polinomios trabajan de la misma manera, en particular las más modernas, que aplican técnicas de todo tipo. Sin embargo el camino seguido desde hace décadas es seguro y eficiente en general.

A diferencia de los métodos descritos en el capítulo anterior para ecuaciones generales, los métodos para polinomios no requieren aproximación inicial de las raíces, por lo general.

Las etapas de una rutina moderna típica son las siguientes:

1. acotamiento de todas las raíces,
2. escalamiento de raíces,
3. aislamiento de cada raíces,
4. localizar cada raíz con la exactitud necesaria, refinándola, y
5. reducción de grado,

repetiéndose los pasos hasta encontrar todas las raíces (o llegar a un polinomio para el que haya fórmula directa). Estas etapas generales varían principalmente si se trata de métodos que aproximan todas las raíces al mismo tiempo. Por ejemplo, es claro que estos métodos no requieren de la etapa de reducción de grado.

Cada una de estas etapas será presentada en las siguientes secciones.

Methods for Polynomial with Real coefficients Newton by Madsen Newton by Grant-Hitchins Jenkins-Traub Durand-Kerner Eigenvalue Bairstow Bairstow by Bond

<https://springerlink3.metapress.com/content/?Author=Kaj+Madsen>

Methods for Polynomial with Complex coefficients Newton by Madsen Newton by Grant Hitchins Laguerre Halley Jenkins-Traub Graeffe by Malajovich Durand-Kerner Aberth-Ehrlich

⁴En <http://www1.elsevier.com/homepage/sac/cam/mcnamee/index.html>.

6.1.5. El método gráfico: Lill

En la sección 5.1.4 se presentó el método gráfico para aproximar raíces de ecuaciones en general, mostrando beneficios y problemas. Es claro que el mismo método se puede aplicar para polinomios, pero hay un método menos conocido que es específico para funciones polinomiales.

Más que su utilidad práctica, pues para grado alto no lo es, es interesante la relación geométrica que guardan los coeficientes con las raíces.

M. E. Lill (1867). *Résolution graphique des équations numériques de tous degrés à une seule inconnue, et description d'un instrument inventé dans ce but*. *Nouvelles Annales de Mathématiques*. 2 6: 359–362.

M. E. Lill (1868). *Résolution graphique des équations algébriques qui ont des racines imaginaires*. *Nouvelles Annales de Mathématiques*. 2 7: 363–367.

6.1.6. Métodos polinomiales no abordados en este libro.

Cualquiera que se adentra a todo tipo de métodos para obtener raíces de polinomios, queda sorprendido de la cantidad y variedad de ideas, variantes y detalles. Por lo mismo es imposible. Tan sólo considérense los dos volúmenes del libro

6.2. Evaluación de polinomios y otras operaciones

Evaluar polinomios o funciones en general tiene muchos problemas, aún para el software profesional. Se requiere poder evaluar el polinomio en (a) las raíces encontradas para comprobación, (b) las aproximaciones a las raíces (c) al buscar cotas para las raíces o (d) para cualquier otro valor de interés, por lo que es necesario dedicarle un momento a los detalles. Hay muchas buenas referencias sobre este tema, como [63,133,168,217,311]⁵. Para formas especiales de polinomios, como evaluaciones modulo m , ver [219], dedicado más bien al estudio de la complejidad temporal de los algoritmos.

La forma extendida de un polinomio, como la ecuación (6.1) ocasiona la mayor cantidad de errores posibles. Si el polinomio se conoce como alguna factorización, por ejemplo como producto de polinomios de grado menor, es preferible utilizarlas.

A continuación se presetan técnicas adecuadas para evaluar y hacer algunas operaciones con polinomios, necesarias para la aproximación de raíces. Ejemplos de códigos se presentan con intenciones ilustrativas.

⁵En [219,311] se presentan técnicas curiosas que optimizan aún más, pero que son poco empleadas en la práctica.

La evaluación de la forma extendida debe ser la última opción, pero generalmente es la única.

Código 6.1: Estructura para representar polinomios.

```

1 typedef struct {
2     int n;           /* Grado */
3     double *a;       /* Coeficientes */
4 } Polinomio;

```

6.2.1. Evaluación directa

El primer intento de todo principiante de programación es utilizar la función `pow` en algo parecido a

Esta es una solución MUY mala.

```

S=0;
for ( i=0 ; i<=n ; i++ )
    S += a[i]*pow(x,i);

```

Desgraciadamente, este procedimiento tan obvio falla por muchos motivos. Exceso de multiplicaciones y el uso de una función que se utiliza cuando se eleva un número real a un exponente real y no el caso de exponente entero como en los polinomios. De hecho, la función `pow` típica de cada compilador evalúa $x^y = e^{y \ln x}$ lo que resulta en un error relativo potencialmente alto⁶, así como un exceso de tiempo de cómputo.

x^y es una de las pocas funciones elementales para las que no se conoce los peores casos aún.

Esto es debido a que las funciones exponencial y logaritmo natural se requiere hacer una reducción de rango y otras transformaciones, terminando con algunas iteraciones de Taylor u otro polinomio para mejorar la aproximación. Aún si se programara una función que evaluara ineficientemente x^n , usando $n - 1$ productos, sería mejor que las muchas de operaciones que requiere `pow`.

Sirve recordar de nuevo la observación de la página 132 al evaluar 2^n .

6.2.2. Representación de polinomios en lenguaje C

En lo sucesivo, se considerará que el polinomio p está representado en lenguaje C por el arreglo `a[n+1]` que representan a los coeficientes a_0 hasta a_n .

Como el grado n está ligado al arreglo de coeficientes, es común representar al polinomio como una estructura como la del código 6.1. Se ha definido el nuevo tipo `Polinomio` para hacer declaraciones más adecuadas.

Por ejemplo, para crear un polinomio, se puede usar la función `NewPol` del código 6.2. Una vez disponible la memoria, sólo faltará colocar los coeficientes en el arreglo `P->a`. La función regresa un apuntador al nuevo polinomio.

Lo que hace la función es primero solicitar memoria para la estructura `Polinomio` y luego para el campo `a`, que es el arreglo de coeficientes. Si alguna de las solicitudes de memoria (llamadas a la función `malloc`), la función regresa `NULL`.

⁶Aunque no es lo que recomienda Muller en [281].

Código 6.2: Creando un polinomio.

```

1  Polinomio *New_Pol(int n)
2  {
3      Polinomio *P;

5      P = (Polinomio *) malloc(sizeof(Polinomio));
6      if ( !P )
7          return NULL;
8      P->a = (double *) malloc(sizeof(double)*(n+1));
9      if ( !P->a ) {
10         free(P);
11         return NULL;
12     }
13     P->n = n;

15     return P;
16 }

```

Una posible variante que se utiliza en algunos programas es asumir que se trabajará con polinomios mónicos y solo utilizar un arreglo con n coeficientes, ya que $a_n = 1$ siempre y no se requiere almacenar, sólo desde 0 hasta $n - 1$.

No se considera necesario colocar ceros en los coeficientes, de eso debe encargarse alguna otra rutina que leerá o construirá los coeficientes, dependiendo del polinomio con que se vaya a trabajar. En todo caso, es preferible el valor NaN en los coeficientes, para un polinomio que aún no se ha definido.

6.2.3. Método de Horner y variantes

La evaluación directa de la forma extendida de un polinomio no es la más adecuada pues $ax^2 + bx + c$ requiere tres productos y dos sumas, mientras que $(ax + b)x + c$ requiere sólo dos productos y dos sumas. Este es conocido como el *método de Horner*, que se aplica a polinomios de grado n en general⁷.

¡Y eso que se trata de un polinomio de grado dos!

Evaluando el polinomio (6.25) en Excel con este método, sí se obtiene cero en las raíces y no así con el método que usa la función **pow**. En Maple la misma evaluación da el valor exacto (cero) en ambos casos pues la aritmética se fijó en 60 decimales de precisión.

Para un polinomio $p(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$, la evaluación en x por el método de Horner es

$$(\dots(a_n x + a_{n-1})x + a_{n-2})x + \dots + a_1)x + a_0 \quad (6.5)$$

resultando n sumas y n productos. Lo que está ocurriendo es que se aplica la regla de Ruffini.

⁷Esta técnica era ya utilizada por los chinos antiguamente, e incluso por Newton, pero W. G. Horner la popularizó en 1819.

Código 6.3: Método de Horner para evaluar polinomios.

```

1  double Eval_Pol(Polinomio *P, double x)
2  {
3      int i;
4      double v;

6      v = P->a[P->n];
7      for ( i=P->n-1 ; i>=0 ; i-- )
8          v = v*x + P->a[i];
9      return v;
10 }
```

Por ejemplo

$$2x^3 - 5x^2 + 3x + 2 = ((2x - 5)x + 3)x + 2$$

donde son 3 sumas en ambas notaciones, pero las multiplicaciones pasan de 6 a 3 en el peor caso y de 5 a 3, en el caso de que para calcular x^3 se use x^2 .

Una función que realiza la evaluación se presenta en el código 6.3, donde ya se utiliza la estructura **Polinomio**. El resultado de la evaluación queda en **v**.

Como Horner evalúa la forma extendida de p , se produce la máxima cantidad de errores de redondeo pese a que se realizan la menor cantidad de operaciones aritméticas en general. Por ello desde la década de 1960 se han estudiado con detenimiento, con la intención de medirlos y controlarlos.

En 1961, Allan Gibb presentó una manera de acotar los errores de redondeo usando aritmética de rango. En 1962, Moore presentó técnicas con aritmética de intervalos para encontrar las mismas cotas. En esa época, el costo computacional de ambas ideas era excesivo.

Durante la evaluación (6.5), se van generando las cantidades $v_0 = a_n$, $v_1 = v_0x + a_{n-1}$, $v_2 = v_1x + a_{n-2}$ que son las evaluaciones intermedias al hacer las operaciones anidadas. La sucesión es

$$\begin{aligned} v_0 &= a_n \\ v_i &= v_{i-1}x + a_{n-i} \end{aligned} \quad (6.6)$$

con i desde 1 hasta llegar a v_n que es el valor final de la evaluación. Esta sucesión de valores v_i es de gran importancia para el acotamiento de raíces y para medir el error en la evaluación de p .

Acotar el error de (6.5) significa encontrar una cota para

$$|p(x) - \text{fl}(v_n)| / |p(x)|$$

que equivale a $|p(x) - \text{fl}(p(x))| / |p(x)|$.

A la par de la sucesión (6.6), se va formando otra sucesión, con los errores de redondeo. El error se puede acotar a partir de la ecuación (1.29), página

35. Cada iteración del código 6.3, en particular el cuerpo del ciclo, la línea 8, provoca errores de redondeo por la suma y por el producto.

Lo que está pasando con la sucesión de valores 6.6 es

$$v_i = \text{fl}(\text{fl}(v_{i-1}x) + a_{n-1}).$$

lo que nos indica que cada una de las n iteraciones agrega 2 errores, que van siendo multiplicados por cada siguiente valor a_i . La cota del error la muestra Higham en [168], donde prueba que

$$\frac{|p(x) - \text{fl}(p(x))|}{|p(x)|} \leq \frac{2\mu}{1 - 2\mu} \text{cond}(p, x) \quad (6.7)$$

de donde se ve que, como $\text{cond}(p, x)$ puede ser muy grande, entonces la evaluación puede ser completamente incorrecta. Recuerdese que $\mu = \frac{1}{2}\varepsilon_M$ es la unidad de redondeo, definida y analizada en (1.19), página 27.

Evaluación de Horner con fma

En la línea 8 del código 6.3 es donde se calculan los valores (6.6). Puede observarse que lo que se realiza cada ciclo es un producto y una suma, como una operación **fma** (véase 2.2.5, página 64), lo que sugiere cambiar la línea 8 por la instrucción `v = fma(v,x, P->a[i])`. Entre más alto sea el grado del polinomio, más se notará la diferencia pues los redondeos se reducen a la mitad.

La cota para la nueva evaluación es

$$\frac{|p(x) - \text{fl}(p(x))|}{|p(x)|} \leq \frac{\mu}{1 - \mu} \text{cond}(p, x) \quad (6.8)$$

pues de 2 redondeos en cada iteraciones se reducen a uno solo.

Ojo al compilar.

Lo que parece una buena idea tiene un problema de disponibilidad: no todos los compiladores tienen a **fma** en su biblioteca de funciones. En gcc, esto es admitido si se utiliza la opción de compilación `-ansi` o `-std=c89`, según el manual de la versión 4.3.0, pero hay conflictos con algunas opciones del compilador, por lo que se sugiere probar con cuidado.

Las cotas teóricas son excelente material de estudio y para precaución en el desarrollo de sistemas, pero no suplen la experimentación que permite ver el efecto de los errores de redondeo. Es muy ilustrativo ver estos fenómenos en el ejemplo siguiente.

Ejemplo numérico Sea el conjunto de polinomios definidos como

$$p_n(x) = 2^{52}(x - 1)^n \quad (6.9)$$

evaluados en su forma extendida en los valores $x_k \in \mathbb{F}$ definidos como $x_k = 1 + 2^{-52}k$ con $k \geq 0$. Es claro que, para variables en doble precisión, $x_1 =$

`nextafter(1,2)`, $x_2 = \text{nextafter}(x_1, 2)$, es decir, los números de punto flotante sucesivos desde $x_0 = 1$, por lo que $p_n(x_k) > 0$. Con estas restricciones, se cumple que $p_n(x_k) = k \varepsilon_M$.

Los polinomios p_n están diseñados para evaluarse en valores muy cercanos a la raíz múltiple 1, donde se espera que el redondeo ocasione problemas identificables. El factor 2^{52} magnifica el error ocasionado en una cantidad que debiera ser cero, desplazando $\text{ulp}(p_n(x))$ toda la mantisa. En pocas palabras, son una función cuyo valor es la cantidad de errores de redondeo en su resultado final.

Por ejemplo, si se evalúa $(x-1)^3$ en $x_1 = 1 + \varepsilon_M$, los errores de redondeo ocasionan que el resultado sea exactamente $\varepsilon_M = 2.220\,446\,049\,250\,313 \times 10^{-16}$ al evaluar con el código 6.3, por lo que $p_3(x_1) = 1$ y es más sencillo identificar cuando los resultado son múltiplos de la precisión de la máquina. $2^{-52} = \varepsilon_M$
épsilon de máquina.

Por otra parte, $p_3(x_3) = -1$, lo que significa que el redondeo ocasiona que $p_n(x_i) < 0$, perdiéndose la monotonicidad. El algoritmo del banquero ocasiona esto (consúltese el apartado de redondeo en la sección 2.2).

Si el valor de $p_n(x_k)$ no es un entero, es decir, no es un múltiplo exacto ε_M , entonces hay problemas de condicionamiento, además de los de redondeo que se esperan. Esto ocurre para valores grandes de n o muy grandes de k .

Dadas las limitada precisión de la aritmética en $\mathbb{F}$, es de esperarse que $p_n(x_k) = p_n(x_l)$ para algunos valores de n , k y l . Por ejemplo, $p_2(x_k) = 0$ para muchos valores de k debido a que se trata de un polinomio de grado bajo. Los polinomios (6.9) pueden ser evaluados con la función del código 6.3 o la variante que utiliza `fma` en la línea 8, esperándose resultados distintos. Por ejemplo, para $p_n(x_k)$ la evaluación con `fma` da cero para los primeros valores de k , a diferencia de la evaluación normal de Horner. Para $n > 3$ ambas funciones se ven afectadas por los redondeos.

Los valores fueron generados en este experimento con un programa donde el ciclo importante es

```

1  epsi = nextafter(1,2)-1.0;
2  for ( i=0 ; i<Max; i++ ) {
3      printf("P(%.17g)=\t%.17g\t", x, Eval_Pol(P,x)/epsi);
4      printf("%.17g\n", Eval_Pol_fma(P,x)/epsi);
5      x=nextafter(x,20);
6  }
```

donde en vez de evaluar $2^{52}(x-1)^n$ se calcula $(x-1)^n/2^{-52}$, al dividir entre `epsi`. Nótese que cada cantidad se imprime con 17 decimales de precisión para asegurar que se muestra toda la mantisa disponible y que los valores sean los múltiplos exactos esperados. `Max` es el número de casos que se desean probar. Puede dejarse abierta la condición hasta que se cumpla otro criterio, la cantidad de iteraciones totales puede calcularse con `(x-1)/epsi`.

$p(x) = (x-1)^n$ en el programa.

La salida del programa para $n = 5$ es

```

P(1)= 0 0
P(1.00000000000000002)= -3 -1
```

k	n	3		4		5		6		7		8	
		H	f	H	f	H	f	H	f	H	f	H	f
1		1	0	-1	-2	-3	-1	3	-4	-7	-5	-9	18
2		0	—	0	0	-2	2	6	4	-10	-2	-2	4
3		-1		1	2	-1	1	-3	4	3	-3	17	2
4		0		0	0	0	0	0	0	12	8	8	20
5		—		—	—	1	1	3	-4	-3	3	-17	6
6						2	-2	-6	-4	-6	2	2	-12
7						3	-1	-3	4	-9	5	25	-26
8						0	0	0	0	0	0	16	8
9						—	—	—	—	9	-5	-9	-6
10										6	-2	-18	12
11										3	-3	1	-6
12										-12	-8	8	12
13										-3	3		
14										10	2	.	.
15										7	5	.	.
16										0	0	.	.
17										—	—		
⋮													
⋮													
192												—	—
	máximo	1	0	1	2	3	2	6	4	8	8	25	28

Tabla 6.1: Evaluaciones de $p_n(x_k)$, donde se ven varios efectos del redondeo. Cada n tiene dos columnas, H se refiere a la evaluación con el código 6.3 y f cuando se usa la instrucción fma de la norma. Para $k = 0$ todas las evaluaciones son cero hasta $n = 22$. Para cada columna, el valor mínimo es igual al máximo con signo negativo.

```

P(1.000000000000000004)= -2    -2
P(1.000000000000000007)= -1    1
P(1.000000000000000009)= 0      0
P(1.000000000000000011)= 1     -1
P(1.000000000000000013)= 2      2
P(1.000000000000000016)= 3      1
P(1.000000000000000018)= 0      0
P(1.00000000000000002)= -3     -1

```

donde se ve que los últimos valores $(-3, -1)$ son iguales que los de la segunda línea, con lo que todo lo anterior es un ciclo y es suficiente con identificar el periodo. El periodo se repetirá hasta cambiar para valores altos de k , que ya no interesan.

La primer columna corresponde a la evaluación con el método de Horner normal, que corresponde al código 6.3. El máximo error es 3 (y -3), lo que corresponde a $3\varepsilon_M$ de error al calcular $(x-1)^5$ cerca de 1. La segunda columna corresponde a la evaluación cuando se usa **fma**. El error máximo es ahora 2.

La tabla 6.1 presenta la diversas evaluaciones de $p_n(x_k)$, que permiten inferir varios hechos que ya se habían justificado de manera teórica y que ahora se hacen evidentes. La siguiente es una lista no exhaustiva. En la tabla solo se presentan los valores hasta que se muestra el periodo completo, cuyo final se designa con

—. Cada n tiene dos columnas, H se refiere a la evaluación con el código 6.3 y f cuando se usa **fma**.

1. Los errores de redondeo pueden ser cualquier cosa menos aleatorios.
2. Cada ciclo de errores es antisimétrico: se lee igual comenzando de arriba a abajo o alrevés, pero con signo contrario. El valor central siempre es cero.
3. Los errores se incrementan con el condicionamiento del problema.
4. Si se tratará de los errores de una función cualquiera, todas las evaluaciones debieran considerarse cero. Es claro que se desea que $|f(x)| < \varepsilon_M$ para considerar que es un cero, pero el experimento revela la necesidad de relajar la restricción.
5. No se puede confiar en la monotonicidad: $p_n(x_k)$ es estrictamente creciente a partir de $x_0 = 1$, sin embargo las evaluaciones se hacen negativas una y otra vez.
6. En teoría el uso de **fma** debiera cometer menos errores. En la última fila está la cantidad máxima de errores de redondeo, donde se ve cierta ventaja para la operación **fma**, pero hay casos individuales como para $n = 4$ y $k = 1$, donde ocurre lo contrario.
7. La pendiente con la que p_n cruza el eje es muy pequeña, por lo que crecimiento es muy lento y más conforme n aumenta. Por ejemplo, para $n = 5$, el polinomio central de la tabla 6.1, para que ocurra $p_5(x_k) = \varepsilon_M$ exactamente se requiere que $x_k = \text{fl}(1 + 2^{-10.4})$, lo que ocurre cuando $k = 4194304$, si se considera que el comportamiento de p_5 es lineal en esa vecindad. Eso significa que todos los valores $x_k \in \mathbb{F}$ en el intervalo $[2^{-52}, 2^{-31.2}]$ deben considerarse ceros de p_5 y similarmente con los números de punto flotante inmediatos posteriores de $x_0 = 1$.
8. La observación anterior es de gran significado: refleja la representación de intervalos reales con un mismo número de punto flotante, tras usar redondeo. El problema de fondo queda en evidencia: cómo distinguir que $1 + 2^{-32}$ es un cero y que $1 + 2^{-30}$ no lo es. El autor resolvió analíticamente en este caso, cosa que en la práctica no se puede generalmente.
9. El comportamiento observado no es equivalente al tambaleo mostrado en la figura B.1 (pág. 377), sino a un titubeo numérico por usar aritmética finita, que se ejemplifica también en la figura 5.10 (pág. 239) al aproximar una raíz.

Es posible hacer más estudios estadísticos pero no es necesario para nuestros fines, además el ejemplo es suficientemente elocuente del proceso de redondeo.

La forma de las funciones (6.9) sugiere la forma más general

$$f(x) = 2^{52}g(x) \quad (6.10)$$

Código 6.4: Método de Horner-Adams para evaluar polinomios con cota del error.

```

1  double Eval_Pol_Adams(Polinomio *P, double x, double *Err)
2  {
3      int i;
4      double v, Mu, er, absx;

6      Mu = .5*(nextafter(1,2)-1.0);

8      absx = fabs(x);
9      er = absx/2;
10     v = P->a[P->n];
11     for ( i=P->n-1 ; i>=0 ; i-- ) {
12         v = v*x + P->a[i];
13         er = absx*e+fabs(v);
14     }
15     *Err = Mu*(2*er-fabs(v));

17     return v;
18 }
```

que permite ver la sensibilidad de $g(x)$ en el conjunto $\mathbb{F}$. El único requisito es que se tenga localizado un cero de g para partir de este haciendo evaluaciones con los números de punto flotante sucesivos, en ambas direcciones. La pendiente con la que g cruza el eje x es determinante.

Evaluación de Horner con cota de redondeos

En 1967, Duane Adams⁸ presentó en [4] una variante del algoritmo de Horner, como criterio práctico para la búsqueda de raíces. Además de la $p(x)$, evalúa la cantidad ϵ tal que $|v - p(x)| \leq \epsilon$, donde v es el valor aproximado al final de las iteraciones y $p(x)$ el valor exacto.

El algoritmo de Adams se presenta en el código 6.4. La función recibe adicionalmente la dirección donde quedará almacenada la cota de los redondeos, **Err**.

La variable **Mu** se define como la unidad de redondeo de la computadora o μ , que se calcula como $\varepsilon_M/2$, la diferencia entre 1 y su sucesor en la línea 6, haciendo uso de la norma de IEEE. El valor final ***Err** cumple con $|v - p(x)| < \mathbf{*Err}$.

En polinomios de grado alto, el algoritmo de Adams es útil para detener la evaluación en cuanto ocurra que $p \approx e$. En ese caso se podrá tener confianza de que x es un cero de $p(x)$, dentro de las limitaciones de $\mathbb{F}$.

⁸Técnica atribuida a Kahan y Farkas en [209], presentada sin justificación alguna, para el compilador de Fortran de la computadora IBM 7090, con 27 bits de mantisa, lo que obliga el uso de números mágicos el código. En 1967, Kahan ya había hecho el análisis del error para el método de Horner, con polinomios reales evaluados en valores reales y complejos, así como la derivada del polinomio.

Se usa el error absoluto necesariamente.

Código 6.5: Método de Horner para evaluar la derivada de un polinomio.

```

1  double Eval_DerivPol(Polinomio *p, double x)
2  {
3      int i;
4      double v;

6      i = p->n;
7      v = i*p->a[i];
8      for ( i--; i>0 ; i-- )
9          v = v*x + i*p->a[i];

11     return v;
12 }

```

Si se aplica esta forma de evaluación a los polinomios (6.9), la cota de redondeo resulta mucho mayor. La cantidad de redondeos es fija, salvo el condicionamiento de p_n , por lo que va aumentando, tan lentamente como los polinomios. La cota de redondeos calculada es 2^{n-1} , lo que se puede estimar multiplicando $\mathbf{Err} \times \varepsilon_M$.

La ventaja evidente de la cota obtenida con el código 6.4 es que el valor aumenta proporcionalmente a x_k , debido a que se utiliza el valor de la evaluación v_i para obtener $\mathbf{Err}$ (línea 12).

La magnitud de la cota de redondeos para los polinomios (6.9) no significa ninguna desventaja, pues estos se han definido para calcular el error de redondeo con toda exactitud. La cota de Kahan y Farkas es aplicable a cualquier polinomios, por lo que es tan versátil como útil.

Es útil si se realizará el diagnóstico de algún proceso numérico asociado.

Evaluación de la derivada

El método de Horner puede utilizarse también para evaluar la derivada de un polinomio con pocas modificaciones, debido a que la derivada de p es muy sencilla. La forma más directa se muestra en el código 6.5. La derivada tiene gran utilidad en muchos procesos numéricos pues permite calcular números de condición, convexidad logarítmica o manipulación simbólica diversa.

Es muy frecuente que el polinomio y su derivada tengan que evaluarse al mismo tiempo. Wilkinson presentó en [391] la forma de calcular p y p' simultáneamente, utilizando para ello la siguiente sucesión. Considerando que v_i en la sucesión (6.6) es tale que $v_n = p(x)$, entonces se definen

$$\begin{aligned} v_0 &= a_n \\ w_0 &= 0 \end{aligned}$$

y con i desde 1 hasta n se calculan

$$\begin{aligned}w_i &= w_{i-1}x + v_i \\v_i &= v_{i-1}x + a_{n-1}\end{aligned}$$

que genera la sucesión de valores w_i tales que $v_i = p(x)$ y $w_n = p'(x)$. De esta manera el trabajo lo realizará una sola función. La técnica es generalizable y es posible evaluar todas las derivadas en el mismo ciclo, como se muestra en [219, 311]. Higham en [168] presenta un algoritmo para calcular el valor de las primeras k derivadas.

Es claro que si se necesita tener disponible la polinomio p' para cálculos posteriores, es preferible calcularlo y guardarlo en su propia estructura `Polinomio`.

Evaluación en valores complejos $p(x + iy)$

En caso de que se necesite evaluar el polinomio con coeficientes reales p en un número complejo $z = x + iy$, pueden utilizarse operaciones con números reales de la siguiente manera. Se define la sucesión

$$\begin{aligned}v_0 &= a_n \\w_0 &= a_{n-1} \\t &= x + y \\s &= x^2 + y^2\end{aligned}$$

y con i desde 1 hasta n se calculan

$$\begin{aligned}v_i &= v_{i-1}t + w_{i-1} \\w_i &= a_{n-i} - sv_{i-1}\end{aligned}$$

y $p(z) = zv_n + w_n$. Por eficiencia, es preferible este método que emplear los tipos `complex` disponibles en algunos lenguajes de programación, excepto cuando se usará aritmética compleja de manera intensiva (y solo por efectos de legibilidad), como sería el caso de utilizar polinomios con coeficientes complejos.

Si la evaluación se realiza con números complejos, la última línea del código 6.4 debe cambiarse por

$$Err = \frac{2\mu(2er - |v|)}{1 - 2\mu}$$

que es una cota mayor. También, en vez de la norma L_2 puede usarse la norma L_1 si el tiempo es un recurso crítico. Esta es una de tantas técnicas para acelerar los procesos, cuando la precisión puede relajarse.

Evaluación compensada de polinomios

En abril de 2006, Graillat, Langlois y Louvet, presentaron en [151] una forma de evaluación compensada del método de Horner y analizaron su desempeño de manera detallada. Para ello utilizan las transformaciones introducidas inicialmente en 1971 por Dekker en [89] (ver el código 3.1 en la página 85), llamadas transformaciones libres de errores por Rump, Ogitta y Oishi (ver la sección 4.9).

Lo que hacen es lo siguiente: si se desea evaluar el polinomio p en x , obtienen los polinomios p_h y p_l tales que $p(x) = p_h(x) + p_l(x)$ exactamente en el sentido de que

$$p_l(x) \leq \text{ulp}(p_h(x))$$

o equivalentemente, $p_l(x) \leq \mu p_h(x)$. A esto le llaman *evaluación compensada de Horner*. Utilizando `fma`, la cota del error para este caso es

$$\frac{|p(x) - fl(p(x))|}{|p(x)|} \leq \mu + \left(\frac{2\mu}{1-2\mu} \right)^2 \text{cond}(p, x) \quad (6.11)$$

y claramente es una mejora con respecto a (6.8). Para efectos prácticos considera $\left(\frac{2\mu}{1-2\mu} \right)^2 \approx \mu^2$. El resultado del algoritmo es equivalente a utilizar una mantisa del doble de la precisión normal, es decir, 104 bits.

Para programar este método es necesario tener macros o funciones que apliquen las transformaciones libres de errores y disponer de la instrucción `fma` en el ambiente de programación.

En el mismo trabajo, de Graillat, se contrasta con el desempeño del método de Horner convencional y la versión con `fma`. Vale la pena que el lector interesado repita los experimentos de Graillat en [151].

6.2.4. División de polinomios

Dados los polinomios $a(x) = a_m x^m + \dots + a_1 x + a_0$ y $b(x) = b_n x^n + \dots + b_1 x + b_0$, el problema es encontrar los polinomios c y r tales que

$$a(x) = b(x)c(x) + r(x) \quad (6.12)$$

donde $r(x) = 0$ cuando $c(x)$ es un divisor de $a(x)$.

6.2.5. Máximo común divisor de dos polinomios

Antes de presentar el

Encontrar el máximo común divisor (MCD) de dos polinomios es de interés por varios motivos.

6.3. Conceptos y fundamentos teóricos

Rouché's theorem, named after Eugène Rouché, states that if the complex-valued functions f and g are holomorphic inside and on some closed contour K , with $|g(z)| < |f(z)|$ on K , then f and $f + g$ have the same number of zeros inside K , where each zero is counted as many times as its multiplicity. This theorem assumes that the contour K is simple, that is, without self-intersections.

The theorem is usually used to simplify the problem of locating zeros, as follows. Given an analytic function, we write it as the sum of two parts, one of which is simpler and grows faster than (thus dominates) the other part. We can then locate the zeros by looking at only the dominating part. For example, the polynomial has exactly 5 zeros in the disk since for every z , and z , the dominating part, has five zeros in the disk.

A. G. Khovanski... Fewnomials, volume 88 of Translations of Mathematical Monographs. American Mathematical Society, Providence, RI, 1991. Translated from the Russian by Smilka Zdravkovska.

6.3.1. Definiciones, notación y resultados básicos

Se define como $\mathbb{P}^n$ el conjunto de todos los polinomios de grado n con coeficientes reales, como en la ecuación (6.1), donde $a_n \neq 0$. En la realidad de las computadoras, los coeficientes son elementos de $\mathbb{F}$, no de $\mathbb{R}$.

Para muchos desarrollos teóricos y experimentos numéricos, es conveniente tener una forma de medir la cercanía entre dos polinomios y como consecuencia la cercanía entre sus respectivas raíces. Lo habitual es definir una distancia $d : \mathbb{P}^n \times \mathbb{P}^n \rightarrow \mathbb{R}$ (métrica), que asocie un número real a cada par de polinomios del mismo grado.

Para ello primero se define la norma de un polinomio $p(x) \in \mathbb{P}^n$. Hay dos alternativas habituales:

$$\|p\| = |a_n| + |a_{n-1}| + \cdots + |a_0| \quad (6.13)$$

y

$$\|p\| = \sqrt{a_n^2 + a_{n-1}^2 + \cdots + a_0^2} \quad (6.14)$$

que si bien no son las únicas posibles, en la práctica se prefieren por su simplicidad (véanse por ejemplo Heindel, Collins, Chin, Hitz). En [50], Brisebarre, Muller y Tisserand presentan otras normas útiles para aproximaciones polinomiales.

A partir de cualquier definición de norma que se seleccione, la distancia entre dos polinomios p y q de $\mathbb{P}^n$ se define como

$$d(p, q) = \|p - q\|. \quad (6.15)$$

Con la definición de distancia (6.15), se puede probar que $\mathbb{P}^n$ es un espacio normado completo, es decir, un espacio de Banach.

Como corolario, se sabe que entre dos raíces consecutivas de la derivada de un polinomio existe una única raíz del polinomio y debe ser una raíz simple. Obsérvese que si se localiza la mayor raíz de la derivada de un polinomio, sea r_M , entonces el polinomio tiene una raíz en (r_M, ∞) . Similarmente si se localiza la menor raíz de la derivada.

6.3.2. Acotación y aislamiento de raíces en general

Regla de los signos de Descartes

Un resultado de gran utilidad es la *Regla de los Signos de Descartes*, de **René Descartes** (1595 - 1650), principios del siglo XVII. Se utiliza para tener una cota superior de la cantidad de raíces reales positivas y negativas. filósofo y matemático francés.

Ejemplo 50. Dado el polinomio

$$p(x) = 3x^8 + 2x^7 - x^5 + 5x^4 - 4x^2 - x - 1$$

se cuentan los cambios de signo de los coeficientes, en este caso son 3, de 2 a -1, de -1 a 5 y de 5 a -4. Eso significa que hay un máximo de 3 raíces positivas. Para las raíces negativas se evalúa

$$p(-x) = 3x^8 - 2x^7 + x^5 + 5x^4 - 4x^2 + x - 1$$

donde cambian de signo los coeficientes de las potencias impares. Ahora los cambios de signo son 5, indicando un máximo de 5 raíces negativas.

Este primer resultado permite establecer a grandes rasgos la porción del eje x donde es conveniente buscar. La regla de los signos calcula la cantidad exacta de raíces reales. Si sólo hay un cambio de signo, existe una única raíz real positiva. Si no hay ningún cambio de signo no hay raíces reales.

Una generalización de esto la ofrece el teorema de Routh-Hurwitz, establecido en 1895, donde se contabilizan las raíces que están en cada lado del plano complejo, al dividirlo con el eje imaginario, usando determinantes.

Routh-Hurwitz Theorem, Cauchy index.

Hurwitz, A., "On the Conditions under which an Equation has only Roots with Negative Real Parts", Rpt. in Selected Papers on Mathematical Trends in Control Theory, Ed. R. T. Ballman et al. New York: Dover 1964

Routh, E. J., A Treatise on the Stability of a Given State of Motion. London: Macmillan, 1877. Rpt. in Stability of Motion, Ed. A. T. Fuller. London: Taylor y Francis, 1975

Ver el concepto de polinomios estables de Programming for Mathematicians, pag. 280 y la regla de los signos de Vincent.

En este caso, los polinomios se definen como estables cuando tiene puras raíces con parte real negativa.

El primer problema que se debe resolver es acotar las raíces del polinomio (6.1), es decir, encontrar un intervalo $I_p = [-M, M]$ donde estén contenidas todas las raíces, aunque no necesariamente debe ser simétrico y puede ser $[M_{\inf}, M_{\sup}]$. Equivalentemente, para raíces complejas, M es el radio del disco complejo que contiene las raíces. Debe ser claro que entre menor sea M la localización de raíces se facilitará.

Por otra parte, no necesariamente el intervalo va a estar centrado en cero, pudiendo ser de la forma $I_p = [Min, Max]$ y es posible que el origen (el 0) esté en el intervalo pero no es obligatorio. Pueden ser puras raíces negativas o puras raíces positivas.

Una vez determinado el intervalo I_p , se aplican subdivisiones sucesivas hasta encontrar los intervalos disjuntos $I_i = [a_i, b_i]$, con i desde 1 hasta t , como en la ecuación (6.2). Los intervalos deben cumplir con $a_1 < b_1 \leq a_2 < b_2 \leq \dots < b_t$ y cada uno contener exclusivamente una raíz, con lo que estarán aisladas.

De esta manera, tendremos cada raíz $r_i \in [a_i, b_i]$ contenida en su propio intervalo. Un parámetro adicional que es deseable encontrar al aislar raíces es su multiplicidad m_i . Esto es más sencillo una vez que la raíz se localiza, evaluando las derivadas sucesivas $p', p'', \dots$ hasta encontrar la primera que no se anula en r_i .

Esto es porque si el polinomio p se factoriza como $p(x) = (x - r_i)^m q(x)$, entonces las derivadas consecutivas tendrán un factor $(x - r_i)^k$ con k reduciéndose una unidad cada derivada, hasta la derivada $p^{(m)}(x)$, que ya no será divisible entre $x - r_i$. La cantidad de derivadas determina su multiplicidad.

Teniendo el intervalo donde está cada raíz, el paso siguiente es aproximarlas una a una, partiendo de una aproximación x_0 contenida en el intervalo I_i , que se espera que sea cercana a la raíz r_i . Los métodos para ello son el objetivo de este capítulo.

Without loss of generality let the x_n term of a polynomial with all real roots have coefficient 1 and let the general term be $a_{-j}x^{-j}$. Let $-a_j$ be the set of negative coefficients. An upper bound for the positive real roots is given by the sum of the two largest numbers in the set $-a_j - 1/j$. This is a improvement on Fujiwara's bound which uses twice the maximum value of this set as its upper bound.

It is possible to determine the bounds of the roots of a polynomial using Samuelson's inequality. Laguerre E (1880) Memoire pour obtenir par approximation les racines d'une equation alebrique qui a toutes les racines reelles. Nouv Ann Math 2'eme serie, 19, 161-172, 193-202

En el acotamiento se ubica el menor intervalo posible que contiene a todas las raíces. El escalamiento tiene la intención de separar raíces muy cercanas, en caso de realizarse se requiere la operación inversa tras encontrar todas las raíces. El aislamiento encuentra subintervalos donde esté contenida una sola raíz. La localización es el objetivo principal de los métodos de aproximación. Por último,

la reducción de grado es una etapa de cálculo simbólico que se realiza cada vez que se obtiene una raíz para facilitar el problema.

Otro paso intermedio que puede ocurrir después del aislamiento es la determinación de la multiplicidad de las raíces.

6.3.3. Reducción de grado

La mayor parte de los métodos encuentran las raíces de un polinomio una a la vez. Cada vez que se tiene bien determinada una raíz se aplica una *reducción de grado* de la siguiente manera: si r es una raíz del polinomio p entonces término $x - r$ es un divisor y se cumple que

$$p(x) = q(x)(x - r) \quad (6.16)$$

para algún polinomio q cuyos coeficientes se pueden obtener mediante el algoritmo de *división sintética*, conocido como la Regla de Ruffini.

Paolo Ruffini (1765-1822),
matemático italiano).

Si p se define como (6.1), entonces q es de grado $n - 1$. Supongamos que los coeficientes de q son desde b_0 hasta b_{n-1} . Según la regla de Ruffini, estos se calculan haciendo

$$\begin{aligned} b_{n-1} &= a_n \\ b_{n-2} &= a_{n-1} + rb_{n-1} \\ &\vdots \\ b_0 &= a_1 + rb_1 \end{aligned} \quad (6.17)$$

lo que generalmente se hace en forma de tabla. Cuando r no es raíz, entonces la cantidad

$$s = a_0 + rb_0 \quad (6.18)$$

es el residuo de la división, que es igual a $p(s)$.

Si $p(x)$ posee otra raíz $r_1 \neq r$, entonces $q(r_1) = 0$. Esto es porque, sustituyendo r_1 en la ecuación (6.16), $0 = p(r_1) = q(r_1)(r_1 - r)$, de donde debe cumplirse que $q(r_1) = 0$ pues $r - r_1 \neq 0$.

Aunque teóricamente el residuo de la división (6.16) es cero, en la práctica no siempre es así, por lo que es de esperarse un residuo causado por imprecisiones numéricas. Este error puede reducirse un poco refinando r antes de la división sintética. El refinamiento consiste en aplicar un método muy preciso que parte de r e iterar unas pocas veces para conseguir que los últimos dígitos estén correctos.

Si el polinomio p es de grado n , entonces el polinomio q es de grado $n - 1$. Esto significa que cada vez que se encuentra una raíz, se obtiene otro polinomio con el grado reducido, hasta llegar al caso de polinomios de grado dos o uno, para los cuales existen fórmulas como la ecuación general de segundo grado, o el sencillo caso lineal, respectivamente.

Por ejemplo, si el polinomio es

$$p(x) = x^7 - 5x^6 + 2x^5 + 18x^4 - 11x^3 - 25x^2 + 8x + 12 \quad (6.19)$$

y se localiza la raíz $r = 3$, entonces se obtiene

$$p(x) = (x - 3)(x^6 - 2x^5 - 4x^4 + 6x^3 + 7x^2 - 4x - 4) \quad (6.20)$$

y ahora se prosigue localizando las raíces del polinomio de grado 6 resultado de la división.

Por otra parte, si un algoritmo de aproximación determina 2 de las raíces en la misma etapa, se hace una reducción de grado con el factor cuadrático que resulta. Por ejemplo, si para el polinomio (6.19) se determina que los valores 1 y 3 son raíces, entonces el factor $(x - 1)(x - 3) = x^2 - 4x + 3$ divide a p , resultando

$$p(x) = (x^5 - x^4 - 5x^3 + x^2 + 8x + 4)(x^2 - 4x + 3)$$

y ahora se prosigue a trabajar con un polinomio de grado 5, tras haber reducido el grado $n = 7$ en 2. Encontrar las raíces de $x^2 - 4x + 3$ es sencillo mediante fórmulas, aunque numéricamente hay detalles que no son tan obvios.

Si se localizan las raíces una a una solo es necesario pulirlas para mejorar la exactitud de las aproximaciones. Pero si la reducción es con factores cuadráticos del polinomio, sus dos raíces deben calcularse con fórmulas. Esto es sencillo pero requiere ciertos cuidados, que se analizan en la sección 6.5.

Cuidados en la reducción de grado

La idea de la reducción de grado es conocida desde hace siglos, pero no es sino hasta que se utilizan computadoras cuando la sensibilidad de los polinomios se manifiesta al hacer una reducción de grado sin cuidados.

James Wilkinson fue el primero en darse cuenta de que al usar precisión limitada, el orden en el que se aproximan las raíces es importante para obtener mejor exactitud. Los dos casos opuestos son cuando se obtienen las raíces en orden creciente o decreciente de su magnitud. Algunos de sus estudios quedaron plasmados en un artículo escrito en dos partes en 1959, [391] y [392].

La prueba de Wilkinson fue perturbar uno a uno los coeficientes de un polinomio mónico de grado $n = 20$, definido en la ecuación (6.29) y calcular las raíces de nuevo, en diversos órdenes, es decir, dado $p(x)$, calcular las raíces del polinomio perturbado $p(x)(1 + \delta)$, probando con la alteración del coeficiente a_j

$$\tilde{p}(x) \equiv p(x) + \delta a_j x^j \quad (6.21)$$

para cada j desde 0 hasta $n - 1$. Valores grandes de δ ocasiona que aparezcan raíces complejas⁹. Desde las primeras pruebas con polinomios a mediados del siglo XX ya se operaba con aritmética compleja.

⁹De hecho, $\delta > 2^{-33}$ es suficiente para que surjan raíces complejas, mientras que valores menores simplemente modifican las raíces reales.

En la teoría, según el argumento de Wilkinson, si r_i es una raíz de p con multiplicidad 1, entonces $r_i(1 + \delta)$ debe ser una raíz de $\tilde{p}$ si ocurre que

$$p(r_i(1 + \delta)) + \delta a_j (r_i + \delta r_i)^j = 0$$

de donde, desarrollando con series de Taylor y eliminando los términos de orden cuadrado, se obtiene que

$$\delta r_i = -\frac{\delta a_j r_i^j}{p'(r_i)} \quad (6.22)$$

y $p'(r_i) \neq 0$ pues r_i es una raíz múltiple. La ecuación 6.22 es una aproximación lineal del error que se obtendrá en la raíz r_i al perturbar el coeficiente a_j , multiplicándolo por el factor $1 + \delta$. Esta aproximación lineal es válida sólo para valores suficientemente pequeños de δ .

Wilkinson la llama perturbación lineal.

Como consecuencia de la perturbación, algunas raíces permanecen sin alteraciones mientras que otras tienen grandes variaciones, por lo que el condicionamiento de cada raíz es independiente. Hablar de que un polinomio está mal condicionado significa realmente que algunas de las raíces son las mal condicionadas, no necesariamente todas.

El orden en el que las raíces se utilizan para reducir el grado (el orden en el que son removidas) resulta importante pero depende principalmente de su condicionamiento. Puede ocurrir que se localiza una raíz mal condicionada con poca precisión y en la etapa siguiente se obtiene una raíz bien condicionada con mejor precisión a partir del polinomio reducido.

La conclusión de Wilkinson es que el orden es importante cuando las raíces difieren enormemente en magnitud. Esta sencilla idea es fundamental para muchos métodos que localizan raíces de polinomios. Incluso es posible omitir el refinamiento de las raíces en algunos casos.

Por lo anterior, **si no se tiene una medida de la distribución y condicionamiento de las raíces**, es mejor aplicar la regla general de aproximarlas en orden de magnitud creciente e ir aplicando las reducciones de grado de esta manera.

Una mala distribución de raíces puede ser peor que la existencia de raíces múltiples.

La Regla de Ruffini es sólo una manera de realizar la división $q(x) = p(x)/(x - r)$, calculando los coeficientes de q en orden de la mayor potencia a la menor, como se muestra en 6.17. También es posible realizar la división calculando los coeficientes en sentido contrario, como lo muestra el artículo [28] de Bingham de 1967, donde muestra cómo aplicar al mismo tiempo ambas reglas, calculando los coeficientes partiendo de cada extremo y deteniendo el proceso de manera oportuna. El proceso se detiene al cumplirse cierta condición de error. Bingham presenta métodos equivalentes para factores cuadráticos, además del caso lineal.

La idea de Bingham fue importante mientras no había una norma que garantizara ciertas condiciones mínimas de seguridad. Con el arribo de la norma de IEEE, la Regla de Ruffini es suficiente.

Reducciones de grado más agresivas

Reducciones recientes utilizan técnicas de optimización para separar al polinomio dado como el factor de 2 polinomios de grados $\lfloor n/2 \rfloor$ y $\lceil n/2 \rceil$, en el mejor de los casos. Estas son técnicas tipo divide y vencerás. Normalmente se utiliza optimización por mínimos cuadrados.

También se han reportado trabajos que, luego de una etapa convencional, utilizan precisión múltiple (arbitraria) hasta cierta precisión predeterminada.

Muchos matemáticos (y sabios de hace siglos) que han estudiado el problema de encontrar las raíces de polinomios han obtenido resultados teóricos que en la práctica son de gran utilidad. Cuando se presenta un polinomio el primer problema es determinar cuántas raíces distintas tiene y cómo se encuentran distribuidas en la recta real o en el plano complejo.

Estos son indispensables para el desarrollo de programas que encuentren las raíces dados solo los coeficientes del polinomio. El ser humano puede graficar y ver por donde andan los cruces con el eje de las abscisas, ajustando el rango interactivamente, pero la computadora requiere de formulaciones algorítmicas para proceder, algunas de las cuales se presentan a continuación.

La sencillez de los polinomios permite hacer operaciones simbólicas con ellos, lo que permite y/o facilita la solución de gran cantidad de problemas. Por eso, algunos los llaman algoritmos seminuméricos. En el caso de las funciones en general, no se tiene estas mismas ventajas y todo proceso es en esencia numérico, pero con polinomios se usan operaciones tanto simbólicas como numéricas.

Referencia iniciales para esta sección son los clásicos libros [374] de Uspensky y [219] de Knuth. El segundo, al ser de programación, incluye algoritmos especificados de manera más clara para programadores.

6.3.4. Cota de Cauchy

El polinomio $p(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$ tiene sus raíces en el intervalo $(-M, M)$ donde

$$M = 1 + \frac{\max_{i=0}^{n-1} |a_i|}{|a_n|}.$$

Por ejemplo, si el coeficiente del término de mayor grado es mayor que el resto de los coeficientes en valor absoluto, entonces se sabe que las raíces están en el intervalo $[-1, 1]$, lo que es de gran ayuda. En el caso general, cuando se consideran raíces complejas, todas se encuentran en el círculo abierto de radio M centrado en el origen. Esta es una versión cuantitativa del Teorema Fundamental del álgebra de Cauchy, de 1829.

Por otra parte, para algunos polinomios esta cota es muy mala. Por ejemplo, el coeficiente mayor de $(x-2)^5(x-3)^5$ es 66 600 y el de x^{10} es 1, por lo que el

intervalo de búsqueda es $[-66\,600, 66\,600]$, cuando la raíz mayor es 3 y la menor 2.

Sirva meramente como un burdo intervalo inicial, si es necesario.

6.3.5. Cota de Knuth

Es una mejora de la cota de Cauchy.

Cota general (Mc Laurin): Sean

$$\begin{aligned} A &= \max |a_{n-1}|, \dots, |a_1|, |a_0| \\ M &= |a_n/a_0|, \dots, |a_1/a_0| \\ r &\in \mathbb{R} \text{ con } p(r) = 0 \end{aligned}$$

entonces

$$1/(1+M) < |r| < 1 + (A/|a_n|) \quad (6.23)$$

Regla de Newton: Un número natural k es cota superior de las raíces positivas de $p(x)$ si

$$P(k) \geq 0, P'(k) \geq 0, \dots, P(n-1)(k) \geq 0, P(n)(k) > 0. \quad (6.24)$$

Regla de Laguerre: k es cota superior de las raíces positivas de $p(x)$ si al dividir p por $x - k$ todos los coeficientes del cociente son ≥ 0 y el resto es ≥ 0 .

6.3.6. Teorema de Sturm

Este resultado¹⁰ de 1829, permite ubicar en qué intervalos están las raíces reales de un polinomio real sin raíces múltiples, es decir, sin tomar en cuenta su multiplicidad, solo su ubicación. Como determina la cantidad de raíces en un intervalo dado, es posible subdividir para encontrar intervalos donde esté cada raíz. Para ello se construyen las cadenas o funciones de Sturm¹¹, que son como sigue.

Jacques Charles François Sturm (1803 – 1855), matemático francés de una generación bastante selecta, como Laplace, Poisson, Cauchy, Gay-Lussac, Fourier (de quien fue asistente), Ampere y otros.

Sea $p(x)$ un polinomio real y $p'(x)$ su derivada. Se define la sucesión de polinomios p_i como

$$\begin{aligned} p_0 &= p \\ p_1 &= p' \\ p_2 &= -p_0 \text{ mód } p_1 \\ &\vdots \\ p_i &= -(p_{i-2} \text{ mód } p_{i-1}) \\ 0 &= -(p_{r-1} \text{ mód } p_r). \end{aligned}$$

¹⁰Se dice que realmente las descubrió el gran matemático Jean Baptiste Fourier pero que el artículo fue olvidado por muchos años debido a la revolución francesa.

¹¹En la literatura se utiliza indistintamente cadena, sucesión o funciones de Sturm.

p_i	$-(p_{i-2} \bmod p_{i-1})$
$p_0 = p$	$x^5 - 3x^4 - 5x^3 + 15x^2 + 4x - 12$
$p_1 = p'$	$5x^4 - 12x^3 - 15x^2 + 30x + 4$
p_2	$\frac{86}{25}x^3 - \frac{36}{5}x^2 - \frac{34}{5}x + \frac{288}{25}$
p_3	$\frac{15\,400}{1849}x^2 - \frac{18\,900}{1849}x - \frac{16\,900}{1849}$
p_4	$\frac{282\,897}{42\,350}x - \frac{49\,923}{6050}$
p_5	$\frac{4840\,000}{534\,361}$

Tabla 6.2: Cadena de Sturm de p (ver texto).

Esto significa que se aplica el algoritmo generalizado de Euclides cambiando de signo al residuo. El polinomio p_r es el máximo común divisor entre p y su derivada.

Ahora, sea $\sigma(x)$ la cantidad de cambios de signo en la sucesión $\{p_i(x)\}_{i=0}^r$ en el valor fijo x

El Teorema de Sturm establece que para el intervalo $(a, b]$, la cantidad de raíces es $\sigma(a) - \sigma(b)$. Cuando se revisa con pequeños pasos un intervalo, es posible identificar dónde están las raíces de manera individual, por lo que es un resultado muy importante para buscar condiciones de convergencia.

Por ejemplo, sea el polinomio $p(x) = x^5 - 3x^4 - 5x^3 + 15x^2 + 4x - 12$. Ahora, se forma la sucesión de Sturm que se muestra en la tabla 6.2.

Si se calcula $\sigma(-3) = 5$ y $\sigma(0) = 3$, representan la cantidad de cambios de signo de la cadena de Sturm para $x = -3$ y $x = 0$ respectivamente, entonces la diferencia $|\sigma(-3) - \sigma(0)| = 2$, por lo que el teorema de Sturm identifica la existencia de 2 raíces en el intervalo $(-3, 0]$.

Similarmente, como $\sigma(4) = 0$, entonces $|\sigma(0) - \sigma(4)| = 3$, por lo que hay 3 raíces en el intervalo $(0, 4]$. Estos dos cálculos son confirmados con la gráfica del polinomio p , que se muestra en la figura 6.1. A partir de la gráfica, es claro que el polinomio se generó como $p(x) = (x+2)(x+1)(x-1)(x-2)(x-3)$.

En este caso, las cotas de Cauchy para el intervalo donde se encuentran las raíces y de Sturm para la cantidad de raíces distintas son útiles para establecer los parámetros iniciales de manera automática.

Existe una generalización del teorema de Sturm para el caso de que los coeficientes sean complejos, que permite ubicar cuántas raíces están en cada parte del plano complejo, llamado el teorema de Routh–Hurwitz. Esta basado en una idea publicada en 1877 por Routh en su libro XXX, reimpresso en 1975. La idea fue retomada por Hurwitz en 1964, en YYY.

Hurwitz, A., "On the Conditions under which an Equation has only Roots with Negative Real Parts", Rpt. in Selected Papers on Mathematical Trends in Control Theory, Ed. R. T. Ballman et al. New York: Dover 1964 Routh, E. J., A

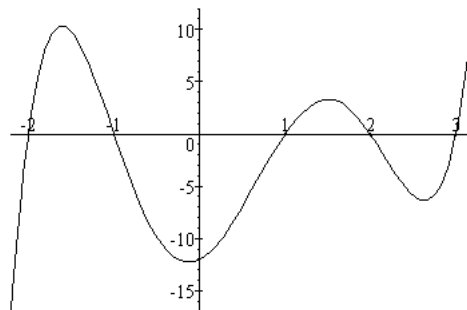


Figura 6.1: Raíces acotadas por la cadena de Sturm para el polinomio $p(x) = x^5 - 3x^4 - 5x^3 + 15x^2 + 4x - 12$.

Treatise on the Stability of a Given State of Motion. London: Macmillan, 1877.
Rpt. in Stability of Motion, Ed. A. T. Fuller. London: Taylor and Francis, 1975

6.3.7. Aislamiento de raíces

Para el caso de que el polinomio tenga raíces enteras, el artículo de Heidel CCC de 1971, trata el problema de aislamiento con bastante detalle.

En 1974, Collins y Loos presentan un algoritmo para aislar raíces basado en diferenciación. Utilizan derivadas sucesivas del polinomio, el teorema de Rolle y construcción de tangentes. Para determinar la multiplicidad de las raíces, se aplica el algoritmo de división de Euclides, como en 6.3. Como resultado adicional, obtiene el aislamiento de raíces de todas las derivadas de p .

Ese mismo año, Collins y Akritas retoman el algoritmo de aislamiento de raíces que Uspensky presenta en su libro CCC, basado en la regla de los signos de Descartes. Como el planteamiento de Uspensky es costoso computacionalmente, modifican el algoritmo para hacerlo más práctico, reduciendo su complejidad computacional.

De hecho le dedican el artículo a Uspensky.

Con los resultados anteriores, determinar los intervalos I_i que contienen raíces individuales resulta más sencillo. Hay muchos trabajos que presentan técnicas diversas para lograrlo.

6.3.8. Estimación de punto

Existen varios resultados teóricos que determinan que la aproximación inicial x_0 es suficientemente buena. Las condiciones para que el proceso de aproximación de raíces converja han sido estudiadas desde hace mucho tiempo, pero fue hasta

Código 6.6: Realizando una división sintética.

```

1  double DivisionSintetica(Polinomio *p, Polinomio *q,
2                                double c)
3  {
4      int i, n;
5      double Residuo;

7      n = P->n;
8      q->a[n-1] = p->a[n];
9      for ( i = n-1 ; i>0 ; i-- )
10         q->a[i-1] = p->a[i] + c*q->a[i];
11     Residuo = p->a[0] + c*q->a[0];

13     return Residuo;
14 }

```

1986, cuando Smale presenta un resultado que termina acuñando este problema como *estimación de punto*¹².

Para los algoritmos que aproximan todas las raíces a la vez, se repite el proceso de estimación en cada intervalo I_i .

6.3.9. Reducción de grado

División sintética

Para aplicar la reducción de grado a un polinomio una vez que se localiza una raíz c , se necesita realizar la división sintética de $p(x)$ entre $x - c$. Se utiliza la Regla de Ruffini, que se ejemplifica en el código 6.6.

En el arreglo **q** quedan los coeficientes, la función regresa el residuo de la división, que debiera ser cero si c es una raíz, o al menos de magnitud insignificante. La función **DivisionSintetica** realiza la recurrencia 6.17.

Como puede verse a partir de las funciones que se han mostrado en este capítulo, los polinomios son entidades sencillas de manipular algebraicamente. Su integral, producto y hasta el cociente de polinomios son fáciles de programar.

División sintética extendida

La regla de Ruffini para factorizar un polinomio puede ser extendida para realizar la división entre dos polinomios, que es de gran utilidad para calcular el máximo común divisor o aplicar reducción de grado cuando se localiza un factor cuadrático.

¹²No debe confundirse con el mismo término que en estadística se emplea para referirse a diversos estadísticos.

6.3.10. Variedad peyorativa

Cuando se realiza el análisis de los errores de un algoritmo que aproxima raíces, se tienen los errores forward y backward. Se dice que como los polinomios son mal condicionados, las soluciones son imprecisas en general. Esta es una forma de ver el problema, pues William Kahan, en 1972, introdujo un nuevo concepto que cambia la perspectiva desde la que se ve el problema de aproximación de raíces.

La idea de Kahan es que la respuesta imprecisa se obtiene porque se hace una pregunta imprecisa. Esta es la esencia de una manera distinta de resolver este y otros problemas.

6.4. Probando que la aproximación de raíces es exacta

¿Cómo se puede probar que el cálculo de las raíces es correcto? Lo que realmente se desea saber es la exactitud de la aproximación obtenida. Por la primera parte de este texto, ya se sabe de los errores de redondeo, así que no se esperan soluciones exactas al hacer los cálculos con una computadora usando aritmética convencional.

Por supuesto, probar unos cuantos casos no es suficiente. Lo primero y obvio es evaluar las raíces encontradas en la ecuación original y comprobar que el resultado es cero.

Por ejemplo, el polinomio

$$p(x) = 987x^2 - 1220x + 377 \quad (6.25)$$

cuya gráfica se muestra en la figura 6.2 tiene dos raíces racionales, $\frac{13}{21}$ y $\frac{29}{47}$, ambas con representación infinita periódica en base 10, indicada con la llave.

$$\begin{aligned} \frac{29}{47} &= . \overbrace{6170212765957446808510638297872340425531914893} \\ \frac{13}{21} &= . \overbrace{619047} \end{aligned}$$

Aún cuando no se cometieran errores de redondeo, es necesario ajustarlas a la mantisa de la precisión seleccionada. En caso de ser de doble precisión y un error de conversión de binario a decimal menor a .5 ulp, los valores serían

$$\begin{aligned} r_1 &= .617021276595744 \\ r_2 &= .619047619047619 \end{aligned}$$

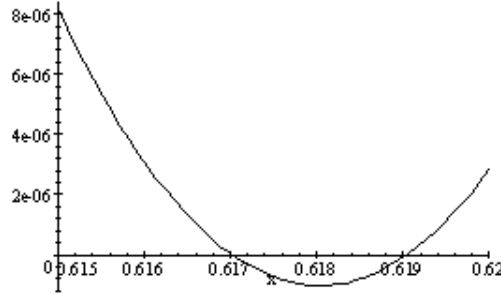


Figura 6.2: Gráfica de $p(x) = 987x^2 - 1220x + 377$. Sus dos raíces requieren una precisión mayor que la que permite la aritmética nativa de una computadora.

aunque esto resultaría sólo si es necesario escribir el resultado. El error relativo de las aproximaciones es

$$\frac{\left| .617021276595744 - \frac{29}{47} \right|}{\frac{29}{47}} \approx 1.103448172413793 \times 10^{-15}$$

$$\frac{\left| .619047619047619 - \frac{13}{21} \right|}{\frac{13}{21}} \approx 7.6923 \times 10^{-17}$$

De evaluarse directamente en el polinomio, se usarán las representaciones internas, sin el error de conversión. Aún al evaluar el polinomio en dichos valores internos que aproximan las raíces no puede esperarse obtener cero pero sería deseable obtener $|p(r_i)| < \varepsilon_M$, el épsilon de máquina.

El ejemplo numérico anterior hace pensar que la diferencia entre la evaluación del polinomio cuadrático exacto $p(x)$ y la evaluación del polinomio con la aproximación numérica $p(x + \varepsilon_M)$ es

$$a(x + \varepsilon_M)^2 + b(x + \varepsilon_M) + c - (ax^2 + bx + c) \approx (2ax + b)\varepsilon_M$$

por lo que no hay garantía de un error como $|p(r_i)| < \varepsilon_M$ si ocurre que $2ax + b > 1$, lo cual puede ocurrir frecuentemente. Esta es una cota máxima y por lo tanto pesimista, pero la cancelación puede ser afortunada en muchos casos y obtener evaluaciones muy precisas.

Con las limitaciones de $\mathbb{F}$ se espera un error mayor, dependiendo del grado del polinomio. Por ello es necesario tener claro cuál es la tolerancia aceptable para poder considerar un candidato x_i como raíz. Utilizar el error relativo es

El caso en que el error absoluto es más adecuado que el relativo.

tentador pero no es adecuado dividir entre una cantidad tan pequeña. Si se es muy exigente, se puede utilizar el número más pequeño que la norma de IEEE permite para precisión que se está empleando, en este caso doble, por lo que $2^{-52} \approx 10^{-16}$ es una tolerancia que requiere un resultado con la máxima cantidad de dígitos significativos correctos.

En general es de gran dificultad probar una rutina. Los experimentos deben ser automatizados, rápidos, orientados a los problemas que se espera encontrar, cuidando de no repetir casos y muy numerosos. En el caso que interesa, para tener la seguridad de su correctitud se tendría que probar todas las posibles situaciones: raíces alejadas, cercanas, raíz con multiplicidad, coeficientes del mismo signo, todas las combinaciones de signos, proporciones diversas, coeficientes y evaluaciones en NaN e Inf, dispersiones aleatorias, direccionar redondeo en búsqueda de anomalías, comparar con librerías comerciales y otras más. Por supuesto, cada experimento debe ser reproducible para poder corregir los errores. El esfuerzo para probar bien una rutina es normalmente mucho mayor que el de programarla.

Probar rutinas es aún un arte que requiere de mucha práctica para dominarlo.

Codificar es un entusiasta acto de buena fe que casi siempre es seguido de largas horas de penosa depuración.

6.4.1. La prueba exhaustiva

Una *prueba exhaustiva* consiste en llamar a la función con todas las posibles combinaciones de entradas. Siendo excesivamente simplistas y buscando una cota superior, si los parámetros son variables en precisión doble, cada uno puede tomar como valor cualquier combinación de los 64 bits de su formato, es es decir, uno de 2^{64} posibles. Siendo 3 coeficientes, el total de entradas posibles es de

$$2^{64} \times 2^{64} \times 2^{64} \approx 6.2771 \times 10^{57}.$$

Además, debe asegurarse de que para cada entrada la salida es correcta, o al menos tan exacta como la APF permite. La única manera es conocer por anticipado todos los posibles resultados, pero si se conocieran no sería necesaria la función `EcGral`.

Es obvio que incluso en un problema tan pequeño como una ecuación de segundo grado, la prueba exhaustiva no es viable, tanto por la cantidad de posibles argumentos distintos como por ignorar su solución. Lo que sí se puede hacer es evaluar la ecuación cuadrática $p(x)$ en las raíces para comprobar que realmente lo sean. Si r es cualquiera de las dos raíces de p , se espera que

$$|p(r)| \leq |p(\text{nextUp}(x))|$$

y

$$|p(r)| \leq |p(\text{NextDown}(x))|$$

por monotonicidad, ya que p es de grado 2. La garantía es mayor si se asegura que la distancia entre las dos raíces es de al menos $2 \text{ ulp}(r)$.

En polinomios de grado n , las posibles raíces no garantizan esta propiedad.

Código 6.7: Intento de prueba exhaustiva para el cálculo de raíces de la ecuación cuadrática.

```

1  while ( el usuario no detenga el proceso ) {
2      a=random();
3      if ( a==0 ) continue;
4      b=random();
5      c=random();
6      if ( EcGral(a,b,c,&x1,&x2) ) { /* Raices reales */
7          r = EvalPol(a,b,c,x1);
8          if ( fabs(r)>Tolerancia )
9              Grabar(a,b,c); /* Para investigar */
10         r = EvalPol(a,b,c,x2);
11         if ( fabs(r)>Tolerancia )
12             Grabar(a,b,c); /* Para investigar */
13     } else {
14         /* Caso complejo omitido */
15     }
16 } /* while */

```

6.4.2. El método de prueba más sencillo

Siempre existe el procedimiento de fuerza bruta, sencillo pero que no garantiza mucho, por ejemplo el código 6.7, que es el esqueleto de un algoritmo así. La función `EvalPol` sirve para evaluar un polinomio.

El método trabaja de la siguiente manera. Se generan polinomios seleccionando sus coeficientes aleatoriamente (líneas 2 a 5). Se calculan sus raíces en la línea 6 y se evalúan en el polinomio (líneas 7 y 10), esperando obtener un valor cercano a cero. Si en algún caso la evaluación se aleja demasiado del 0 (el error absoluto necesariamente), se graban los coeficientes para analizar después. De ser raíces complejas, caso que no se incluye, puede evaluarse el módulo de la evaluación y también grabarse cuando sea mayor de cierto umbral, aunque lo más sencillo es descartarlas y sólo considerar el caso real.

Si luego de dejar corriendo la prueba toda la noche, el programador se levanta, detiene el proceso y el archivo de coeficientes para investigar está vacío, entonces hay dos posibilidades: 1) dudar de que la tolerancia sea suficientemente pequeña o 2) podrá tener confianza en su función. Esta prueba es la solución para quien no quiere pensar o que no tiene tiempo para probar detalladamente, pero sólo ofrece la confianza de que “casi nunca falla”.

Es imposible saberlo con certeza.

Se puede defender esta técnica argumentando que es un método estadístico y que por la ley de los grandes números, entre más dure la prueba más seguridad tenemos del nivel de eficiencia de nuestra rutina. En el fondo, lo que se está haciendo es aplicar una versión empírica del método de *Monte Carlo* y sería preferible hacerlo con más cuidado. Por una parte, está bien que los coeficientes se seleccionen aleatoriamente, pues todas las combinaciones se están distribuyendo uniformemente dentro de todas las posibles. Pero por otra parte, se debe tener cuidado al usar las funciones que generan números aleatorios, pues todas

¡Puede ser tan corto como 232 en el caso de algunos compiladores!

son cíclicas y la longitud del ciclo depende del compilador.

En general, es preferible dejar Monte Carlo para problemas con más parámetros donde probar es más complicado que en la fórmula general. Claro, se puede utilizar como herramienta adicional a las otras pruebas, pero debe pensarse con cuidado la estrategia por seguir. Por ejemplo, generar las distribuciones de coeficientes polinomiales que dan problemas. Para los interesados en esta técnica, dentro de mucha literatura, una buena referencia es el libro de George Fishman [116] o cualquier libro introductorio sobre simulación por computadora. Otra lectura útil es [201] de Kahan sobre el análisis probabilístico de errores, donde se manifiesta en contra.

A continuación se presentan pruebas orientadas de manera más específica.

6.4.3. Polinomios de prueba para grados 2 y n

Hay muchas pruebas posibles para raíces de polinomios cuadráticos que permiten una mayor confianza. En general, lo mejor es construir polinomios cuyas raíces se conozcan por anticipado para poder comparar con las aproximaciones de la fórmula 6.36.

El tipo de polinomio con que se pruebe puede estar determinado por el tipo de problemas que se desean resolver con la función, pero de no ser así deben aplicarse todas las pruebas posibles.

A continuación se presentan varias pruebas que permiten evaluar el desempeño numérico de la rutina que resuelve la ecuación cuadrática y se termina con una colección de polinomios de grado n útiles para rutinas más generales.

Primera técnica

Una manera sencilla y directa sería partir de las raíces para calcular los coeficientes y después aproximar las raíces. Un método es seleccionar a , x_1 y x_2 arbitrarios (o no del todo) y calcular

$$b = (x_1 + x_2)a$$

$$c = x_1 x_2 a.$$

Los valores a , x_1 y x_2 se pueden generar aleatoriamente o de tal manera que comprometan la precisión de los cálculos, como

$$a \rightarrow 0 \tag{6.26}$$

$$x_1 = 1 - a \tag{6.27}$$

$$x_2 = 1 + a \tag{6.28}$$

para causar conflictos adicionales o combinarlas de diversas maneras. El mayor problema de esta técnica es que las operaciones para calcular b y c introducen

Motivo por el cual no es de tanta utilidad al evaluar la precisión de la rutina.

errores no atribuibles a la función que se prueba pero que desafortunadamente no pueden ser aislados.

El mismo problema persiste si se toman como valores arbitrarios iniciales (b, x_1, x_2) o (c, x_1, x_2) . Tendrá que tenerse gran cuidado en la forma de realizar cada operación para no agregar problemas de redondeo innecesarios.

Una variante de esta técnica es seleccionar a y c y luego calcular

$$b = 4ac \pm h$$

y h se escoge como un valor cada vez más pequeño. Este esquema compromete la precisión del discriminante.

Segunda técnica

Un mejor esquema es generar sucesiones de coeficientes para los que las raíces asociadas converjan a puntos fáciles de determinar. Por ejemplo, si F_i son los números de Fibonacci, tomando $a = F_{n+1}$, $b = -2F_n$ y $c = F_{n-1}$. El discriminante es $F_n^2 - 4F_{n+1}F_{n-1} = 4 \times (-1)^n$, por lo que se van alternando raíces reales y complejas pero que convergen a $2/(1 + \sqrt{5})$, pues

$$\begin{aligned} x &= \frac{2F_n + \sqrt{F_n^2 - 4F_{n+1}F_{n-1}}}{2F_{n+1}} \\ &= \frac{2F_n + \sqrt{(-1)^n 4}}{2F_{n+1}} \\ &\approx \frac{F_n}{F_{n+1}} \end{aligned}$$

ya que asintóticamente el 4 no pesa en el cociente y la parte imaginaria de la raíz compleja tiende a cero. Como se sabe, $F_n/F_{n+1} = 1/\tau$, donde τ tiene como límite la razón aurea $\tau = \frac{1+\sqrt{5}}{2}$.

Uno de varios números que aparecen por todas partes.

A la par de resolver la ecuación cuadrática, se debe ir calculando el cociente a partir de los números de Fibonacci, generados a la par. El cociente, aunque tendo un error de redondeo, se sabe correcto en .5 ulp y se puede hacer la comparación.

Tercera técnica

Otro método bastante frecuente es resolver ecuaciones también de la forma $ax^2 - 2bx + c = 0$, seleccionando c y calculando $a = c - 2$ y $b = c - 1$, que son menos operaciones menos propensas a errores que las de la primera técnica.

Las raíces se calculan de manera exacta como

$$\begin{aligned}
 r_{1,2} &= \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \\
 &= \frac{2(c-1) \pm \sqrt{(2(c-1))^2 - 4(c-2)c}}{2(c-2)} \\
 &= \frac{2(c-1) \pm 2}{2(c-2)} \\
 &= \frac{c-1 \pm 1}{c-2} \\
 &= \begin{cases} \frac{c}{c-2} \\ 1 \end{cases}
 \end{aligned}$$

Entre mayor sea el valor de c , más cercanas estarán las raíces, lo que aporta otro eje de análisis. Si se define $c = 2^k$ llegamos a la prueba de precisión sugerida por Kahan en [202], en sus observaciones sobre la norma de 1985.

Otras técnicas para polinomios en general

Las anteriores formas de prueba de la ecuación cuadrática permiten evaluar con facilidad la exactitud de las respuestas de la rutina programada, pero más aún, sugieren ideas para más pruebas de la misma rutina o de otras, para polinomios de grado n en general.

En casi todo artículo que presenta algún método para evaluar polinomios, factorizarlos o aproximar raíces se presenta el resultado de experimentos. Resulta muy ilustrativo analizar lo que han ideado los investigadores para probar sus algoritmos. Para probar su efectividad, normalmente se recurre a un conjunto de polinomios que ofrecen ciertos problemas para los métodos conocidos y que encuentran mejor respuesta con el método propuesto.

El polinomio o la familia de polinomios empleada depende de la característica que se desea probar, ya sea buscar casos complicados para el algoritmo, probar la etapa de reducción de grado, problemas de convergencia o lo robusto en general. A continuación se presentan algunos polinomios que se sugieren, indicando sus características importantes en la prueba.

Polinomio pérfido El primer polinomio importante (y más famoso) fue el utilizado por Wilkinson para probar una rutina que pretendía calcular las raíces, llamado desde entonces *polinomio pérfido*. Sus 20 raíces son los números enteros

del 1 al 20. El polinomio es expandido es

$$\begin{aligned} \prod_{r=1}^{20} (x - r) = & x^{20} - 210x^{19} + 20615x^{18} - 1256850x^{17} + 53\,327\,946x^{16} \\ & - 1672\,280\,820x^{15} + 40\,171\,771\,630x^{14} - 756\,111\,184\,500x^{13} \\ & + 11\,310\,276\,995\,381x^{12} - 135\,585\,182\,899\,530x^{11} \\ & + 1307\,535\,010\,540\,395x^{10} - 10\,142\,299\,865\,511\,450x^9 \\ & + 63\,030\,812\,099\,294\,896x^8 - 311\,333\,643\,161\,390\,640x^7 \\ & + 1206\,647\,803\,780\,373\,360x^6 - 3599\,979\,517\,947\,607\,200x^5 \\ & + 8037\,811\,822\,645\,051\,776x^4 - 12\,870\,931\,245\,150\,988\,800x^3 \\ & + 13\,803\,759\,753\,640\,704\,000x^2 - 8752\,948\,036\,761\,600\,000x \\ & + 2432\,902\,008\,176\,640\,000. \end{aligned} \quad (6.29)$$

– “Una experiencia traumática”, dijo Wilkinson.

Se dice que en una de las pruebas Wilkinson partió de una aproximación inicial cercana a la raíz mayor pretendiendo que el programa localizara $x = 20$, pero el resultado fue inesperado: llegó a otra raíz. Este fue el inicio de una nueva área en análisis numérico.

Desde entonces, el polinomio pérfido es una prueba frecuente para rutinas que localizan raíces de polinomios y puede hacerse uso de la perturbación (6.21) que empleó Wilkinson en sus experimentos. Si bien no posee raíces múltiples, la distribución lineal de las raíces ocasiona un mal condicionamiento.

Otro polinomio que usó Wilkinson fue

$$p(x) = \prod_{r=1}^{20} (x - 2^{-r}) \quad (6.30)$$

Este hecho va en contra de lo que la intuición le puede malamente indicar a un programador.

cuyas raíces resultaron ser más estables que las de (6.29), según prueba Wilkinson en [391].

El polinomio (6.30) es útil para probar casos de underflow, específicamente si un criterio de paro basado en alguna cota de errores de redondeo falla.

Una sola raíz con multiplicidad n Es fácil generar polinomios a partir de la fórmula $p(x) = (x - r)^n$. Los coeficientes se originan a partir de la fórmula del binomio, es decir,

$$(x - r)^n = \binom{n}{0}x^n + \binom{n}{1}x^{n-1}r + \binom{n}{2}x^{n-2}r^2 + \cdots + \binom{n}{n}r^n. \quad (6.31)$$

La ventaja principal es la facilidad con la que los polinomios se pueden generar en un programa. En este caso, la raíz única r tiene multiplicidad n y es altamente mal condicionada, por lo que determinar las raíces es muy propenso a errores de redondeo si no se tienen los cuidados necesarios.

Este esquema lo utilizaron Graillat y otros en [151] con n desde 5 hasta 450 con incrementos de 5.

Se sabe de la dificultad con que algunos métodos aproximan las raíces múltiples. Wilkinson mostró en [395] que un cero de multiplicidad m puede encontrarse con una precisión relativa de $\varepsilon_M^{1/m}$, aunque Dunaway mostró en [96] que esta cota puede mejorarse. En [190], Jenkins y Traub proponen varios polinomios con raíces múltiples que hasta la fecha siguen siendo utilizados.

A partir de n raíces Generalizando las construcciones anteriores, es común partir del conjunto de raíces $r_1, r_2, \dots, r_n$ y construir el polinomio $(x - r_1)(x - r_2) \cdots (x - r_n)$. Las raíces se pueden determinar de muchas maneras, por ejemplo extendiendo las ideas de las ecuaciones (6.26), (6.27) y (6.28), para complicar más el problema.

Los coeficientes se generan fácilmente a partir de las raíces, observando la sucesión

$$\begin{aligned}(x + r_1)(x + r_2) &= x^2 + (r_1 + r_2)x + r_1r_2 \\(x + r_1)(x + r_2)(x + r_3) &= x^3 + (r_1 + r_2 + r_3)x^2 + (r_1r_2 + r_1r_3 + r_2r_3)x + r_1r_2r_3 \\&\vdots\end{aligned}$$

Para el caso de n raíces, el coeficiente del término de grado n es 1, el del grado $n - 1$ se forma por la suma de las raíces, el de grado $n - 2$ con la suma de todos los productos de dos raíces (n tomadas 2), el de grado $n - 3$ por la suma de los productos de cada combinación posible de 3 raíces (n tomadas 3), hasta llegar al término lineal, que se calcula como la suma de los $n - 1$ productos posibles de $n - 1$ raíces (n tomadas $n - 1$) y el término independiente se forma con el producto de todas las raíces.

Es más complicado generar estos coeficientes que el caso de la construcción (6.31), pero de mayor versatilidad, por lo que vale la pena invertir algo de tiempo diseñando, programando y probando una rutina así, para generar polinomios de prueba a partir de cualquier conjunto de raíces, con la distribución que se desee.

El mayor inconveniente de este esquema es la gran cantidad de errores de redondeo que se generan si las raíces son números que usan la mantisa completa (o casi) en su representación. Si las raíces son enteros pequeños o potencias de 2 el problema se deduce. Diseñar bien una rutina no se hace en poco tiempo.

En caso de ocurrir, los errores de redondeo crecen combinatoriamente con la cantidad de raíces, lo que limita la precisión con la que se calculan los coeficientes. Por lo tanto, esta prueba es más adecuada para medir el desempeño temporal de las rutinas.

Un caso particular es el polinomio con raíces enteras consecutivas $p(x) = \prod_{i=1}^n (x - i)$, para valores peque nos de n para que todos los coeficientes tengan representación exacta.

2, 8, 28, 75, ..., es la progresión de redondeos para $n = 2, 3, \dots$

Jenkins y Traub proponen en [190] el polinomio

$$p(x) = b(x^3 - z^2 - a^2z + z^2) = b(x - a)(x + a)(x - 1)$$

utilizado con valores grandes y pequeños de a para comprobar que raíces grandes y pequeñas no hacen faltar el criterio de paro. Valores grandes y pequeños de b prueban lo mismo para el caso de coeficientes grandes y pequeños.

Otro polinomio de esta familia que también se presenta en [190], es de la forma

$$p(x) = (x - a)(x - 1)(x - a^{-1})$$

diseñado para probar qué tan eficiente es la etapa de reducción de grado.

Polinomios de Mignotte En 1981, se publicó [270], donde se presentaron los *polinomios de Mignotte*, que son otra familia de polinomios a los que constantemente se recurre. Se define

$$M_{n,k}(x) = x^n - 2(kx - 1)^2 \quad (6.32)$$

con $k \geq 3$ y $n \geq 3$ enteros¹³.

La forma desarrollada de (6.32) es $x^n - 2k^2x^2 + 4kx - 2$. Mignotte probó en 1981 en [270] que tiene 2 raíces reales a y b que satisfacen

$$\begin{aligned} a &< \frac{1}{k} < b \\ a, b &\in \left(\frac{1}{k} - h, \frac{1}{k} + h\right) \end{aligned}$$

con $h = \frac{1}{\sqrt{k^{n+2}}}$. Esto significa que entre mayor sea k , más cerca estarán las raíces, lo que permite hacer pruebas para raíces muy juntas.

Por ejemplo, para $n = 5$ y $k = 10$ el polinomio es

$$M_{5,10}(x) = x^5 - 200x^2 + 40x - 2$$

que tiene cinco raíces, al menos dos son reales y están en el intervalo

$$\left(\frac{1}{10} - \frac{1}{\sqrt{10^7}}, \frac{1}{10} + \frac{1}{\sqrt{10^7}}\right) \approx (9.968\,377\,223\,398 \times 10^{-2}, .100\,316\,227\,766).$$

y son $9.977\,763\,419\,256\,207 \times 10^{-2}$ y $.100\,224\,865\,957\,4504$ aproximadamente.

En 1995, Mignotte desarrolló ideas similares para raíces complejas en [271].

¹³Aunque Mignotte establece que $n > 2$, es sencillo probar que para $n = 2$ también se cumple. Mignotte utilizó la notación $M_n(x)$, sin agregar la k como subíndice. $M_{n,k}(x)$ se usa aquí por ser más explícito.

Polinomios de Chebyshev Otra familia de polinomios de gran utilidad en las pruebas son los *polinomios de Chebyshev*, definidos recursivamente como

$$T_0(x) = 1 \quad (6.33)$$

$$T_1(x) = x \quad (6.34)$$

$$T_{n+1} = 2xT_n - T_{n-1} \quad (6.35)$$

cuyas raíces son los números reales $\cos((2k+1)\pi/(2n))$, con $k = 0, \dots, n-1$. Claramente, las raíces están en el intervalo $(-1, 1)$, si bien no uniformemente distribuidas, tampoco están aglomeradas en ningún lugar. En ese intervalo, se cumple que $|T_n(x)| < 1$, para $n > 1$.

Como en la práctica los cálculos se realizan en $\mathbb{F}$, es conveniente asegurarse que las raíces se calculan correctamente hasta el último bit. Generar los coeficientes requiere manipulación simbólica.

Raíces primitivas de la unidad Las raíces primitivas de la unidad son las raíces de la ecuación $x^n - 1 = 0$. Las n raíces son los números complejos $e^{2\pi ik/n}$ con k desde 0 hasta $n-1$. Las n raíces están uniformemente distribuidas en el círculo complejo de radio 1.

Conocidos como números de *de Moivre*.

Este polinomio se utiliza para rutinas que trabajan con aritmética compleja. La principal utilidad es de evaluar si el algoritmo de aproximación trabaja adecuadamente con raíces que tienen el mismo módulo. Esto puede representar problemas para métodos que escalan el polinomio en un intento de separar las raíces.

6.4.4. Polinomios aleatorios

Ofrecen cierta comodidad, parece agradable la idea de poner a la computadora a generar polinomios al azar, calcular las raíces y comprobar que su evaluación en el polinomio es 0 (o casi). Sin embargo, no hay mucho control de errores de redondeo y la dispersión de las raíces deja mucho qué desear, por la ley de los grandes números.

Los dos mejores caminos son seleccionar aleatoriamente coeficientes o raíces, pero con cuidado. En ambos casos, mantisa y exponente deben seleccionarse con distribuciones uniformes independientes, para garantizar raíces ampliamente distribuidas.

De cualquier forma, esta prueba debe ser complementaria a otras y no considerarse definitiva.

Otras fuentes de polinomios de prueba

Polinomios como los anteriores son adecuados no sólo para hacer experimentos numéricos y probar rutinas que acoten, aislen o aproximen raíces. Dificilmente se puede ser exhaustivo y pretender mostrar todos los polinomios útiles,

principalmente cuando hay métodos de aproximación especializados para ciertas familias de polinomios, así que es conveniente estar atentos a los avances y revisar la literatura con frecuencia.

Pueden buscarse más polinomios en libros como CITE y similares. El autor recomienda ampliamente leer las baterías de prueba de los artículos de Sekigawa (2006) y Hull (1996).

No se trata de una colección muy basta, pero sí muy selecta.

Desde hace más de 10 años, Bini y Fiorentino han colectado polinomios que tienen diversas características y singularidades, con el objetivo de ilustrar, comparar y evaluar métodos para resolver problemas polinomiales. Es llamada *suite de pruebas para polinomios*¹⁴ (Polynomial test suite) y se compone de polinomios de una variable (incluyendo coeficientes complejos) y sistemas de polinomios de varias variables.

6.5. Polinomios de grado menor

Al aplicar sucesivamente reducción de grado, se llegará hasta polinomios de grado muy bajo, donde ya no es necesario aproximar, sino aplicar fórmulas directamente. Por ello, antes de continuar con raíces de polinomios, se presenta el caso cuadrático.

En los primeros pasos de la enseñanza del álgebra, se ve que el polinomio $x^2 - x(b+a) + ab$ posee las raíces a y b , pues se factoriza como $(x-a)(x-b)$. Pero hay una infinidad de polinomios donde no es tan fácil encontrar los factores y se necesita buscar otros métodos. Utilizar la computadora y graficar es una posibilidad pero no da mucha precisión y es inútil si las raíces se calcularán muchas veces en una rutina intermedia para usar los resultados automáticamente.

6.5.1. Fórmula general de segundo grado

Lo más sencillo y que es enseñado desde educación secundaria, es aplicar la fórmula general. Las raíces de la ecuación cuadrática $ax^2 + bx + c = 0$ son

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}. \quad (6.36)$$

A nivel superior no es correcto decir que no hay raíces cuando el discriminante es negativo.

Si el *discriminante* $b^2 - 4ac < 0$ las raíces son complejas. Por ejemplo, calculando las raíces del polinomio $3x^2 - 4x + 1 = 0$, se obtiene:

$$\begin{aligned} x_1 &= \frac{-(-4) + \sqrt{(-4)^2 - 4(3)(1)}}{2(3)} \\ &= \frac{4 + 2}{6} = 1 \end{aligned}$$

¹⁴Consultar <http://www-sop.inria.fr/saga/POL/BASE/1.unipol/>

y

$$\begin{aligned}x_2 &= \frac{-(-4) - \sqrt{(-4)^2 - 4(3)(1)}}{2(3)} \\&= \frac{4 - 2}{6} = \frac{1}{3} = 0.\hat{3}\end{aligned}$$

Un ejemplo más con un polinomio con raíces complejas: $4x^2 + 2x + 3 = 0$. De la inspección visual se deduce que si se evalúa en cualquier número real positivo jamás se obtendrá cero, por lo que las raíces deben ser complejas. Sus raíces se obtienen reorganizando la fórmula (6.36) como $\frac{-b}{2a} \pm i \frac{\sqrt{-(b^2 - 4ac)}}{2a}$, entonces

$$\begin{aligned}x_1 &= \frac{-2 + \sqrt{2^2 - 4(4)(3)}}{2(4)} \\&= \frac{-2 + \sqrt{4 - 48}}{8} \\&= \frac{-2 + \sqrt{-44}}{8} \\&= -\frac{1}{4} + i \frac{\sqrt{44}}{8} = -\frac{1}{4} + i \frac{\sqrt{11}}{4} \\&= -.25 + .829\,156\,197\,588\,85i\end{aligned}$$

y por tanto la otra raíz es el conjugado complejo de la primera $-\frac{1}{4} - i \frac{\sqrt{11}}{4}$. En este caso, responder con NaN a la raíz de un número negativo no es correcto salvo que sólo se busquen raíces reales.

Todo ha salido muy bien, pero estos polinomios no son como los que aparecen en la práctica. En la vida real los polinomios tienen coeficientes que pueden ser como los siguientes: $-3.0014873x^2 + 328079.345x + 72673562.5673$. Normalmente se resuelven con computadora, por lo que se requiere estudiar con cuidado los problemas potenciales que pueden surgir.

6.5.2. Interfaz para la ecuación general

Pudiera tratar de resolverse aplicando la fórmula general de manera directa. Una posible interfaz para una rutina que trabaje como caja negra¹⁵ y puede ser de la siguiente manera.

Entrada

a,b,c los coeficientes del polinomio
x1, x2 las direcciones de memoria para las raíces

¹⁵De manera autónoma, sin interacción con el usuario ni otras rutinas.

Código 6.8: Calculando las raíces de una ecuación cuadrática.

```
1  int EcGral(double a, double b, double c,  
2           double *x1, double *x2)  
3  {  
4      double Raiz, t, Discriminante = b*b - 4.0*a*c;}  
5      int SonRaicesReales = SI;  
  
6      if ( Discriminante<0 ) {  
7          SonRaicesReales = NO;  
8          Discriminante = -Discriminante;  
9      }  
10     Raiz = sqrt(Discriminante);  
11     t = .5/a;  
12     if ( SonRaicesReales ) {  
13         *x1 = t*(-b+Raiz);  
14         *x2 = t*(-b-Raiz);  
15     } else {  
16         *x1 = -t*b; /* Parte real */  
17         *x2 = fabs(t*Raiz); /* Parte imaginaria */  
18     }  
19     return SonRaicesReales;  
20 }  
21 }
```

Salida	Valor de retorno
	SI
	x1 Raíz menor
	x2 Raíz mayor
	NO
	x1 Parte real
	x2 Parte imaginaria

Un intento tan inocente como lo sencillo de la interfaz. Esta distinción de la parte real y la parte imaginaria puede sustituirse por el tipo de dato `complex`¹⁶ disponible en algunos compiladores, pero esta interfaz es suficientemente sencilla. Un código es el 6.8.

Esto concuerda con la filosofía del lenguaje C: “el programador sabe lo que hace”. Si `a=0` la función `EcGral` no entrega el resultado de la ecuación lineal $bx+c=0$, por lo que debe revisarse antes de la llamada si el polinomio es cuadrático. Si confiamos que el compilador respeta la norma de IEEE, entonces la división entre 0 es algo que no debe preocupar pues simplemente se realizará la propagación del valor especial, como se indicó en 2.2.6, página 69. De esta manera, ambas raíces tendrán como resultado ∞ , en la representación que indica la norma.

La función `EcGral` podría usarse en algo tan sencillo como el código 6.9 que muestra el resultado en la salida salida estándar.

En este caso, la condición es cierta cuando las raíces son reales y la función `EcGral` regresa SI. Se ha usado `%g` para que en tiempo de ejecución se decida si se usará notación científica o no.

¹⁶Normalmente una estructura con dos campos de doble precisión.

Código 6.9: Ejemplo de uso de la función `EcGral`.

```

if ( EcGral(a,b,c,&r1,&r2) )
    printf("r1 = %g\nr2 = %g\n",r1,r2);
else {
    printf("r1 = %g + %g\n",r1,r2);
    printf("r2 = %g - %g\n",r1,r2);
}

```

6.5.3. Problemas potenciales de la interfaz

Aunque es un buen ejemplo para un primer curso de programación, esta aplicación de la fórmula general resulta un tanto inocente desde el punto de vista numérico. Se pueden identificar los siguientes problemas potenciales:

1. Si a es casi cero en valor absoluto, el cociente es propenso a pérdida de precisión.
2. Si b es mucho mayor en comparación con a y c , la raíz del discriminante será cercana a b , por lo que para una de las raíces, el numerador del cociente será $-b + (b + \epsilon)$, con ϵ muy pequeña y tenemos otro problema numérico conocido: la diferencia de cantidades casi iguales o cancelación catastrófica.
3. Si b es aproximadamente igual a $4ac$, se realizará una diferencia de cantidades casi iguales en el discriminante, perdiendo cifras significativas.
4. alguna o varias de las operaciones pueden causar overflow o underflow.
5. Uno o varios de los coeficientes es $\pm\infty$.
6. Uno o varios de los coeficientes es NaN.
7. Los apuntadores `x1` y `x2` pueden ser nulos. Este es un problema no numérico, pero la función debe considerar todos los casos.

Como puede apreciarse por los problemas mencionados, el código anterior debe reescribirse si se desea que funcione correctamente. Los primeros tres problemas merecen un mejor análisis y se dejan para más adelante. Los otros son más simples de resolver.

El problema 4 tiene como solución las mismas técnicas empleadas al calcular la hipotenusa como el escalamiento o el reordenamiento cuidadoso de las operaciones *a la Stewart*. Si es imposible evitarlo, debe regresarse 0 o $\pm\infty$ dependiendo del caso.

Los problemas 5 y 6 pueden ser resueltos suponiendo que el ambiente de programación que se usa cumple con la norma, propagando los valores especiales sin interrumpir el programa. De no ser esto cierto o tener duda de ello, será necesario agregar condiciones que comprueben lo válido de los argumentos.

El problemas 7 no tiene más solución que regresar NaN tanto en **x1** como en **x2**, en un intento de que propiciar que el programador se percate del error. ¿Se debe regresar SI o NO? Con esta interfaz da lo mismo. Si el programador desea defenderse de estos casos conviene cambiarla y tener valores de retorno adicionales, posiblemente incluyendo el caso $a = 0$. Lo más sencillo sería suponer que *el programador sabe lo que hace* y no considerar el problema 7.

6.5.4. Resolviendo los problemas

Los problemas son salvados con algo de manipulación algebraica conveniente para la aritmética de punto flotante y algo más.

Cambiando la fórmula

En el **primer problema**, cuando a se acerca a cero, este puede ser abordado multiplicando numerador y denominador por el conjugado del numerador como se muestra en la ecuación

$$\begin{aligned} \frac{(-b + \sqrt{b^2 - 4ac})}{2a} \frac{(-b - \sqrt{b^2 - 4ac})}{(-b - \sqrt{b^2 - 4ac})} &= \frac{(-b)^2 - (\sqrt{b^2 - 4ac})^2}{2a(-b - \sqrt{b^2 - 4ac})} \quad (6.37) \\ &= \frac{2c}{-b - \sqrt{b^2 - 4ac}} \end{aligned}$$

que es una fórmula muy comunmente utilizada y resuelve el primer problema.

Si a es muy pequeña pero el numerador es similar en magnitud, entonces no hay de qué preocuparse. Por lo tanto, el problema es realmente la diferencia de magnitud entre el numerador y el denominador, por lo que puede evaluarse la diferencia de magnitud entre los exponentes respectivos y seleccionar la fórmula donde la diferencia sea menor.

Para el **segundo problema**, cuando b es mucho mayor que $4ac$, el mejor esquema es calcular la raíz que no da problemas de cancelación en el numerador, que la de mayor magnitud, con $-(b + \text{signo}(b)\sqrt{b^2 - 4ac})$. La otra raíz se calcula despejando de la fórmula $x_1 x_2 = c/a$.

En el caso del **tercer problema**, el único remedio es utilizar precisión extra si está disponible o hacer uso de precisión múltiple, representando cada valor como la suma de dos variables para duplicar la precisión. De esta manera, será posible calcular $b^2 - 4ac$ hasta con el último bit correctamente redondeado.

En caso de que el primer problema se combine nefastamente con alguno de los otros dos, hay que combinar las soluciones planteadas para conseguir una solución mejor.

Un remedio sencillo.

a la Dekker [89]

Utilizando la representación de $\mathbb{F}$

Otra alternativa es hacer uso de que se sabe cuál es la representación flotante de cada coeficiente. Sean

$$a = m_a \times 2^{e_a} \quad (6.38)$$

$$b = m_b \times 2^{e_b} \quad (6.39)$$

$$c = m_c \times 2^{e_c} \quad (6.40)$$

y sin pérdida de generalidad se puede suponer que el signo es positivo. Es posible reducir el exponente del radical de manera segura observando que

$$\begin{aligned} \sqrt{b^2 - 4ac} &= \sqrt{(m_b \times 2^{e_b})^2 - 2^2(m_a \times 2^{e_a})(m_c \times 2^{e_c})} \\ &= \sqrt{m_b^2 \times 2^{2e_b} - m_a \times m_c \times 2^{e_a+e_c+2}} \\ &= \sqrt{2^{2t}(m_b^2 \times 2^{2e_b-2t} - m_a \times m_c \times 2^{e_a+e_c+2-2t})} \\ &= 2^t \sqrt{m_b^2 \times 2^{2e_b-2t} - m_a \times m_c \times 2^{e_a+e_c+2-2t}} \end{aligned} \quad (6.41)$$

y se selecciona t de manera conveniente para evitar problemas numéricos, ya sea overflow o underflow, como se hizo en el capítulo 3 al calcular la hipotenusa. Esta misma idea puede aplicarse al realizar la división entre $2a$.

Las operaciones anteriores se pueden realizar utilizando las llamadas `ldexp` y `frexp` definidas en la norma (ver tabla 3.8, página 123).

Un excelente ejemplo de cómo resolver una ecuación cuadrática lo muestra Kahan, en sus comentarios sobre la norma en [202]. Utiliza de manera eficiente el escalamiento y la precisión extra.

Como puede verse, la versión inicial de la función `EcGra1` debe modificarse bastante para considerar estos casos. En la mayoría de las ocasiones, los programas legibles y sencillos de entender no son tan robustos como aquellos cuyo propósito es la eficiencia.

El costo es la legibilidad.

Por ello no se incluye el código completo de una función que considere todos los casos, pero se espera que el lector encuentre suficientes ideas como para desarrollar el suyo propio.

6.5.5. Polinomios de grado 3 y 4

Tras haber presentado la solución de los polinomios de grado 2, se presentan las soluciones de los de grado 3 y 4, cuyas fórmulas fueron publicadas por Girolamo Cardano (1501-1576) en 1545 en su *Ars Magna*, pero son atribuidas a Ludovico Ferrari (1522-1565), Scipione Ferro (1465-1526) y principalmente a Nicolo Fontana Tartaglia (1500-1557), en una época en la que hasta había torneos donde se calculaban raíces de polinomios.

Cardano y Tartaglia terminaron peleándose por la autoría.

Se llenarían varias hojas más con los detalles de eficiencia numérica debido a las extensas fórmulas, además de los códigos, pero no hay necesidad por dos motivos:

1. en la reducción de grado sólo los casos lineal y cuadrático son importantes y
2. la ecuación cuadrática ha sido suficiente para introducir los problemas numéricos básicos de las raíces de funciones y polinomios.

Sólo se presentan versiones prácticas de las fórmulas. Para su derivación y otros detalles consúltese [40] o el libro clásico de Uspensky [374].

Grado 3

Todo polinomio cúbico puede reescribirse de la forma $x^3 + bx + c$ con la transformación $x = mx + n$. Las fórmulas para esta forma de polinomio cúbico son las siguientes. Sean

$$\begin{aligned}\gamma &= \sqrt{\frac{c^2}{4} + \frac{b^3}{27}} \\ \alpha &= \sqrt[3]{\frac{-c}{2} + \gamma} \\ \beta &= \sqrt[3]{\frac{-c}{2} - \gamma}.\end{aligned}$$

Las raíces de $x^3 + bx + c$ son

$$\begin{aligned}r_1 &= \alpha + \beta \\ r_{2,3} &= -\frac{r_1}{2} \pm i\sqrt{3}\frac{\alpha - \beta}{2}.\end{aligned}$$

Grado 4

Para la ecuación de grado cuatro de la forma $x^4 + ax^3 + bx^2 + cx + d$, sea w una de las raíces del polinomio $w^3 - bw^2 + (ac + 4d)w - a^2d + 4bd - c^2$ y

$$\rho = \sqrt{\frac{a^2}{4} - b + w}.$$

Si $\rho \neq 0$ se calculan

$$\alpha, \beta = \sqrt{\frac{3a^2}{4} - \rho^2 - 2b \pm \frac{4ab - 8c - a^3}{4\rho}}$$

y si $\rho = 0$ se calculan

$$\alpha, \beta = \sqrt{\frac{3a^2}{4} - \rho^2 - 2b \pm 2\sqrt{w^2 - 4d}}.$$

Las raíces de $x^4 + ax^3 + bx^2 + cx + d$ son

$$\begin{aligned} r_{1,2} &= -\frac{\alpha}{4} + \frac{\rho}{2} \pm \frac{\alpha}{2} \\ r_{3,4} &= -\frac{\alpha}{4} + \frac{\rho}{2} \pm \frac{\beta}{2}. \end{aligned}$$

De las observaciones del caso cuadrático se desprende que para los casos de grado 3 y 4 se requiere de mucho cuidado para conseguir una buena rutina que calcule las variables.

6.5.6. Polinomios de grado 5 o mayor

En 1799, Ruffini probó que no había fórmulas posibles para los polinomios de grado mayor a 4, pero no tuvo suficiente difusión.

Paolo Ruffini (1765–1822),
matemático italiano.

Fue hasta 1824 cuando Abel demostró (de nuevo) que no había fórmulas para grados superiores, por lo que los métodos de aproximación cobraron importancia¹⁷.

Niels Henrik Abel (1802–1829),
matemático noruego.

La prueba sólo establece que no existen fórmulas con operaciones elementales y radicales, dejando abierta la posibilidad de otras operaciones. Sí existen fórmulas para grados mayores que 4, por ejemplo utilizando series infinitas, funciones hipergeométricas o en términos de las funciones modulares de Siegel. Tales formulaciones no se presentan en este libro.

Algunos polinomios particulares de grado mayor que 5 poseen fórmula, pero en casos tan específicos que no se consideran en ningún programa de propósito general.

Beyond the Quartic Equation, King, Birkhauser, Boston 1996

Physical applications of a new method of solving the quintic equation by Victor Barsan

6.6. Métodos que aproximan una raíz a la vez

6.6.1. Método de Lin-Bairstow

Este es un método que apareció en un apéndice de un libro de aeronáutica aplicada. Aún cuando los coeficientes de un polinomio sean reales, es normal la

Leonard Bairstow (1880 – 1963),
matemático inglés.

¹⁷Aunque ya Newton había visto las bondades de las pocas operaciones de los métodos de aproximación contra las cada vez mayores fórmulas directas.

existencia de raíces complejas, que siempre se dan en pares, por los conjugados complejos como $(a+ib)(a-ib)$. Por ello, este método busca factores cuadráticos del polinomio

$$p(z) = a_n z^n + a_{n-1} z^{n-1} + \dots + a_1 z + a_0 \quad (6.42)$$

de tal manera que en cada iteración se busca el factor cuadrático $z^2 + pz + q$ y encontrar la descomposición

$$p(z) = (z^2 + pz + q)(b_{n-2} z^{n-2} + b_{n-3} z^{n-3} + \dots + b_0) + Rz + S \quad (6.43)$$

donde $Rz + S$ es el residuo. El factor cuadrático puede considerarse como un polinomio con coeficientes reales, pero con raíces complejas. Igualando los coeficientes del polinomio original 6.42 y de la factorización 6.43, se obtienen las relaciones

$$\begin{aligned} b_k &= a_{k+2} - pb_{k+1} - qb_{k+2}, & k = 0, \dots, n-2 \\ b_{n-1} &= b_n = 0 \\ R &= a_1 - pb_0 - qb_1 \\ S &= a_0 - qb_0. \end{aligned}$$

En caso de que el factor $z^2 + pz + q$ sea divisor de $p(z)$ entonces $R = S = 0$ y el polinomio

$$b_{n-2} z^{n-2} + b_{n-3} z^{n-3} + \dots + b_0$$

es una reducción de p en dos grados y las raíces del factor cuadrático se pueden obtener con la fórmula general de segundo grado.

¿Cómo encontrar el factor $z^2 + pz + q$? Primero hay que observar que R y S pueden verse como funciones de p y q . El problema se traduce a encontrar los valores de p y q en los que $R(p, q) = S(p, q) = 0$, lo que se resuelve aplicando el método de Newton-Raphson al sistema de ecuaciones, lo que significa la iteración

$$\begin{aligned} \mathbf{x}_{i+1} &= \mathbf{x}_i - \mathbf{F}(\mathbf{x}) [\mathbf{F}'(\mathbf{x})]^{-1} \\ \begin{bmatrix} p_{i+1} \\ q_{i+1} \end{bmatrix} &= \begin{bmatrix} p_i \\ q_i \end{bmatrix} - \begin{bmatrix} \frac{\partial R}{\partial p} & \frac{\partial R}{\partial q} \\ \frac{\partial S}{\partial p} & \frac{\partial S}{\partial q} \end{bmatrix}^{-1} \begin{bmatrix} R \\ S \end{bmatrix} \end{aligned}$$

y luego de algunas sustituciones y simplificaciones que el lector puede encontrar en Ralston, se llega a

$$\begin{aligned} p_{i+1} &= p_i - \frac{1}{D} [b_{-1}(d_{-1} + p_i d_0) - (b_{-2} + p_i b_{-1})d_0] \\ q_{i+1} &= q_i - \frac{1}{D} [(b_{-2} + p_i d_{-1})d_{-1} - d_0 b_{-1} q_i] \\ D &= d_{-1}^2 + p_i d_0 d_{-1} + q_i d_0^2 \end{aligned}$$

donde $b_{-1} = a_1$, $b_{-2} = a_0$ y

$$\begin{aligned} d_k &= -b_{k+1} - qd_{k+2}, & k = -1, 0, \dots, n-3 \\ d_{n-2} &= d_{n-1} = 0. \end{aligned}$$

Su convergencia es local, heredando las propiedades de Newton-Raphson, como su convergencia cuadrática pero muy dependiente de una buena aproximación inicial, por lo que se utiliza mas bien para pulir raíces. Ante raíces múltiples su convergencia es lineal.

6.6.2. Metodo de Bernoulli

En 1732, el matemático suizo Daniell Bernoulli (1700 - 1782), famoso entre otras cosas por la distribución de probabilidad que lleva su nombre, diseñó un novedoso método para aproximar iterativamente la mayor raíz (en valor absoluto) de un polinomio, aprovechando la relación entre la ecuación de diferencias lineales y las raíces del polinomio. Genera una sucesión de sumas de potencias de las raíces, donde el límite del cociente de dos sucesivas es la raíz buscada. El método trabaja con raíces reales y es equivalente al método de las potencias para la matriz adjunta, para calcular valores propios, como probó Bauer en 1955. Se define la *raíz dominante* r_m a la raíz mayor en valor absoluto, es decir, $r_m = \max_i \{|r_i|, \text{ con } f(r_i) = 0\}$.

Si el polinomio tiene raíces reales con multiplicidad 1 y es

$$p(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0 = 0$$

con $a_0 a_n \neq 0$, se plantea la *ecuación en diferencias*

$$a_{n-1} y_{n-1} + a_{n-2} y_{n-2} + \dots + a_1 y_1 + a_0 y_0 = 0 \quad (6.44)$$

y se calcula el siguiente término con

$$y_t = -\frac{1}{a_n} (a_{n-1} y_{t-1} + \dots + a_0 y_{t-n}).$$

Los $n - 1$ valores iniciales y_i son arbitrarios, pudiendo ser todos cero al iniciar, excepto $y_{n-1} = 1$ o algún otro valor distinto de cero. Para encontrar la raíz dominante r_m , se calcula el límite del cociente de dos términos sucesivos y_i de tal manera que

$$r_m = \lim_{i \rightarrow \infty} \frac{y_{i+1}}{y_i}. \quad (6.45)$$

Una vez calculada, es necesario hacer una reducción de grado dividiendo p entre $x - r$ o $x + r$. La división $p(x) = q(x)(x \pm r)$ no dejará residuo al ser r una raíz. Ahora se aplica el método de nuevo, para encontrar la raíz dominante del polinomio q .

¿Cómo es que trabaja el método? No es realmente complicada la justificación. Si las n raíces r_i fueran conocidas, podrían calcularse los valores y_t con

$$y_t = w_1 r_1^t + w_2 r_2^t + \dots + w_n r_n^t.$$

Las w_i pueden calcularse, pero realmente no importan. Evaluando el cociente se tiene

$$C_t = \frac{w_1 r_1^{t+1} + w_2 r_2^{t+1} + \cdots + w_n r_n^{t+1}}{w_1 r_1^t + w_2 r_2^t + \cdots + w_n r_n^t}$$

y dividiendo arriba y abajo entre $w_m(r_m)^t$ se obtiene

$$C_t = \frac{\frac{w_1}{w_m} \left(\frac{r_1}{r_m}\right)^{t+1} + \frac{w_2}{w_m} \left(\frac{r_2}{r_m}\right)^{t+1} + \cdots + \frac{w_m}{w_m} r_m + \cdots + \frac{w_n}{w_m} \left(\frac{r_n}{r_m}\right)^{t+1}}{\frac{w_1}{w_m} \left(\frac{r_1}{r_m}\right)^t + \frac{w_2}{w_m} \left(\frac{r_2}{r_m}\right)^t + \cdots + 1 + \cdots + \frac{w_n}{w_m} \left(\frac{r_n}{r_m}\right)^t} \quad (6.46)$$

y el 1 es por el término $w_m r_m^t$ denominador. Si se supone que r_m es la raíz dominante, cada término $\left(\frac{r_i}{r_m}\right)^t$ tiende a 0 cuando $t \rightarrow \infty$, por lo que

$$\lim_{t \rightarrow \infty} C_t = r_m$$

Por ejemplo, sea el polinomio $p(x) = 4x^3 - 20x^2 + 17x - 4$ cuya mayor raíz es 4. Para generar la sucesión y_i , se comienza definiendo $y_0 = y_1 = 0$ y $y_2 = 1$ y se calcula

$$\begin{aligned} y_3 &= -\frac{1}{4}(-20y_2 + 17y_1 - 4y_0) \\ &= -\frac{1}{4}(-20 + 0 + 0) \\ &= 5 \end{aligned}$$

Haciendo avanzar el índice de y se repite el procedimiento:

$$\begin{aligned} y_4 &= -\frac{1}{4}(-20y_3 + 17y_2 - 4y_1) \\ &= -\frac{1}{4}(-20(5) + 17(1) + 0) \\ &= 20.75 \end{aligned}$$

$$\text{Una vez más: } -\frac{1}{4}(-20(20.75) + 17(5) - 4) = 83.5$$

$$\begin{aligned} y_5 &= -\frac{1}{4}(-20y_4 + 17y_3 - 4y_2) \\ &= -\frac{1}{4}(-20(20.75) + 17(5) - 4) \\ &= 83.5 \end{aligned}$$

Continuando con el procedimiento se obtiene la sucesión y_i y los cocientes consecutivos, que se muestran parcialmente en la siguiente tabla:

i	y_i	$\frac{y_{i+1}}{y_i}$
2	1	5
3	5	4.15
4	20.75	4.02409638554217
5	83.5	4.00374251497006
6	334.3125	4.00056085249579
$\vdots$	$\vdots$	$\vdots$
18	5609753202.938 74	4.000000000000020
19	22439012811.755 1	4.000000000000000

En un programa que implemente el método de Bernoulli, no es necesario conservar en memoria la sucesión y_i , sólo los dos términos más recientes, así como el cociente previo, para poder detener la aproximación cuando ocurra que

$$\left| \frac{y_{i+1}}{y_i} - \frac{y_i}{y_{i-1}} \right| < \varepsilon$$

para alguna tolerancia adecuada ε .

Convergencia de Bernoulli

La convergencia depende de la diferencia de magnitud entre la raíz mayor y las restantes. A partir de 6.46 se tiene que entre mayor diferencia en magnitud exista entre la mayor raíz r_m y las restantes, para valores menores de t se llegará a la tolerancia deseada pues los términos $(\frac{r_i}{r_m})^j$ se reducirán más rápido.

Si ocurre lo contrario, es decir, r_m es apenas mayor que las demás raíces, la convergencia será más lenta por el mismo motivo. En estos casos, el riesgo es que la sucesión y_i llegue al mayor valor representable por la computadora, con lo que no se podrá seguir calculando.

En el caso de darse una convergencia lineal, es posible acelerar con Aitken, como en el método de Steffensen.

Raíces múltiples

Si r_m tiene multiplicidad mayor que 1 o es compleja, Bernoulli corre el riesgo de no converger o hacerlo con excesiva lentitud. Por ello se han desarrollado variantes para manejar estos dos casos.

En el caso de que la raíz dominante sea compleja, $r_m = x + iy$, el método no converge. El conjugado complejo tiene la misma magnitud $|r_m|$, conocida como *módulo*. Utilizando la notación $r_m = |r_m| e^{i\theta}$, para describir números complejos, ahora se definen

$$\begin{aligned} d_i &= (y_i)^2 - y_{i-1}y_{i+1} \\ e_i &= y_iy_{i+1} - y_{i-1}y_{i+2} \end{aligned}$$

tiene que

$$|r_m|^2 = \lim_{i \rightarrow \infty} \frac{d_{i+1}}{d_i}$$

y

$$2|r_m| \cos \theta = \lim_{i \rightarrow \infty} \frac{e_i}{d_i}.$$

Una prueba de esto puede encontrarse en *First Course in Numerical Methods* de Jennings, 1964. La idea es similar a la aproximación de la raíz dominante con la fórmula 6.45.

Para determinar la multiplicidad, se utilizan fórmulas distintas si la r_m es real o compleja. Si $(x - r_m)^t$

6.6.3. Método de Duran-Kerner

Duran-Kerner es un método paralelizable que pese a sus excelentes resultados no se recomienda en aplicaciones finales debido a que su convergencia aún es un problema abierto, excepto en el caso de raíces aisladas, donde se sabe que es cuadrática. Cada iteración se aproxima a todas las raíces. Este método aplica el operador de Weierstrass para la función de iteración, por lo que también se conoce con ese nombre. El método requiere que los cálculos se realicen con números complejos, siempre con la máxima precisión. Si las n raíces del polinomio p son x_i y denotamos por $x_i^{(k)}$ la aproximación en la iteración k -ésima, el método se define como

$$x_i^{(k+1)} = x_i^{(k)} - \frac{p(x_i^{(k)})}{\prod_{\substack{j=1 \\ j \neq i}}^n (x_i^{(k)} - x_j^{(k)})}$$

donde $i = 1, \dots, d$. La fórmula se obtiene de aplicar el método de Newton al sistema de ecuaciones de Vieta (ver el método de Graeffe).

Consultar los dos textos (pdf y ps).

6.6.4. Jenkins-Traub

En 1968, Joseph Frederick Traub y su tesista Michael Jenkins presentaron un método en tres etapas que es utilizado por muchas bibliotecas de funciones profesionales. El método reduce el polinomio tras cada raíz encontrada, comenzando con las de menor magnitud, o módulo, pues considera el caso general de raíces complejas.

Existe una versión para polinomios con coeficientes complejos, presentada en CITE. A continuación se describe la versión para polinomios reales, utilizando aritmética real expuesta en CITE. Según los autores, la versión real es

La tesis fue la prueba de la convergencia.

cuatro veces mejor. Las raíces complejas quedan determinadas por los factores cuadráticos encontrados.

Utiliza tres etapas en general. En la primera acentúa las raíces de menor módulo. En teoría esta etapa es innecesaria pero se requiere por las limitaciones numéricas de las computadoras y la ignorancia inicial de la magnitud de las raíces.

En la segunda, el método intenta separar las raíces muy cercanas, es decir, aquellas que tienen casi el mismo módulo, con el fin de no tener confusiones y confundir dos raíces en una sola, sumando su multiplicidad.

En la tercera y última etapa, se aplica un método como el de Newton para terminar de localizar el cero que interesa en este paso.

Pasando a los detalles de CITE, considérese el polinomio mónico con coeficientes reales y k raíces complejas

$$\begin{aligned} p(z) &= z^n + a_{n-1}z^{n-1} + \dots + a_1z + a_0 \\ &= \prod_{i=1}^k (z - \alpha_i)^{m_i} \end{aligned}$$

donde los α_i son las raíces complejas y m_i su multiplicidad. La derivada de p es

$$\begin{aligned} p'(z) &= \sum_{i=1}^k m_i p_i(z) \quad y \\ p_i(z) &= \frac{p(z)}{z - \alpha_i}. \end{aligned}$$

La idea es construir una sucesión de polinomios $q^{(j)}(z) = \sum_{i=1}^k c_i^{(j)} p_i(z)$ con $c_i^{(0)} = m_i$ y si se selecciona una sucesión $q^{(j)}(z) \rightarrow c_1^{(j)} p_1(z)$, es decir,

$$\frac{c_i^{(j)}}{c_1^{(j)}} \rightarrow 0, \quad i = 2, \dots, k$$

entonces ya se tiene una sucesión $\{t_j\}$ que converge a α_1 , definida por

$$t_{j+1} = s_j - \frac{p(s_j)}{\tilde{q}^{(j+1)}(s_j)}$$

donde

$$\tilde{q}^{(j)}(s_j) = \frac{q^{(j+1)}(s_j)}{\sum_{i=1}^k c_i^{(j)}},$$

s_j es una sucesión de números complejos.

6.7. Métodos que aproximan todas las raíces al mismo tiempo

6.7.1. Método de Graeffe

Al parecer en una tremenda coincidencia.

En el siglo XIX, Karl Heinrich Gräeffe (1799-1873), Germinal Pierre Dandelin (1794-1847) y Nikolai Ivanovich Lobachevsky (1792-1856) descubrieron este método (más simbólico que numérico) cada quien por su parte en 1837, 1826 y 1834 respectivamente. El método también puede plantearse como los cuadrados sucesivos de una matriz.

Dado el polinomio $p(x) = (x - r_1)(x - r_2) \cdots (x - r_n)$, este se multiplica por

$$p(-x) = (-1)^n (x + r_1)(x + r_2) \cdots (x + r_n)$$

resultando

$$p(x)p(-x) = (-1)^n (x^2 - r_1^2)(x^2 - r_2^2) \cdots (x^2 - r_n^2).$$

Este puede verse como un polinomio en término de x^2 cuyas raíces son cada r_i^2 . Es claro que se ha elevado al cuadrado cada raíz del polinomio original p , lo que las ha cambiado de distancia relativa entre las raíces pues aquellas con valor absoluto mayor que 1 se han alejado entre sí. Aplicando de nuevo el mismo procedimiento k veces, se obtiene un polinomio entérminos de x^{2^k} . Sea este polinomio $q^k(x) = y^n + b_1 y^{n-1} + \cdots + b_{n-1} y + b_n$ con raíces $y_1, y_2, \dots, y_n$.

Ahora, es posible utilizar las llamadas fórmulas de Vieta para calcular los coeficientes:

$$\begin{aligned} b_1 &= -(y_1 + y_2 + \cdots + y_n) \\ b_2 &= y_1 y_2 + y_1 y_3 + \cdots + y_{n-1} y_n \\ &\vdots \\ b_n &= (-1)^n y_1 y_2 \cdots y_n. \end{aligned}$$

Como el proceso de elevar al cuadrado ha alejado las raíces de los polinomios, y_1 es mucho mayor que el resto, por lo que

$$\begin{aligned} b_1 &\approx -y_1 \\ b_2 &\approx y_1 y_2 \\ &\vdots \\ b_n &= (-1)^n y_1 y_2 \cdots y_n \end{aligned}$$

de donde se puede despejar y obtener

$$\begin{aligned} y_1 &\approx -b_1 \\ y_2 &\approx -\frac{b_2}{y_1} \\ &\vdots \\ y_n &\approx -\frac{b_n}{y_{n-1}} \end{aligned}$$

y las raíces originales se obtienen con las fórmulas

$$\begin{aligned} x_1 &\approx \sqrt[2k]{-b_1} \\ x_i &\approx \sqrt[2k]{-\frac{b_i}{b_{i-1}}}, \text{ para } i \text{ de } 2 \text{ a } n. \end{aligned}$$

Se sabe que este método funciona bien principalmente en el caso de que las raíces sean reales. Su convergencia es cúbica. Tiene similitudes con el método de Bernoulli, que se presenta a continuación.

6.7.2. Método de Aberth-Ehrlich

Derivado de [102] y [1], es llamado Aberth-Ehrlich por los trabajos de estos dos autores, de 1973 y 1967 respectivamente. Aprovechando la notación utilizada en el método anterior, el esquema iterativo de Ehrlich es algo más complicado:

Nótese que a la iteración de Newton se le agrega un divisor en el segundo término.

$$x_i^{(k+1)} = x_i^{(k)} - \frac{\frac{p(x_i^{(k)})}{p'(x_i^{(k)})}}{1 - \frac{p(x_i^{(k)})}{p'(x_i^{(k)})} \sum_{\substack{j=1 \\ j \neq i}}^n \frac{1}{x_i^{(k)} - x_j^{(k)}}} \quad (6.47)$$

o utilizando la notación de la corrección de Newton 5.58,

$$x_i^{(k+1)} = x_i^{(k)} - \frac{u(x_i^{(k)})}{1 - u(x_i^{(k)}) \sum_{\substack{j=1 \\ j \neq i}}^n \frac{1}{x_i^{(k)} - x_j^{(k)}}}.$$

Su convergencia superlineal en general y cúbica para raíces simples bajo ciertas condiciones. No existen resultados teóricos sobre su convergencia global. Ehrlich tomó ideas previas de Borsch-Supan, donde se propone para mejoramiento de candidatos a raíz, y de los trabajos de Wilkinson [395] para eigenvalores, como afirma en [102]. De hecho, 6.47 puede usarse fijando una raíz o aproximando todas a la vez.

Una mejora es utilizar los valores ya actualizados de $x_i^{(k+1)}$ para $x_{i+1}^{(k+1)}$ y las siguientes, como indico Ehrlich, para alcanzar la convergencia cúbica. Este orden no se logra si las raíces son múltiples.

Al estilo del método iterativo Gauss-Seidel.

El método de Aberth-Ehrlich consiste en utilizar la identidad

$$\frac{p'(x_i)}{p(x_i)} = \sum_{j=1, j \neq i}^n \frac{1}{x_i - r_j}$$

para llegar a la iteración de punto fijo

$$r_i = x_i - \frac{1}{\frac{p'(x_i)}{p(x_i)} - \sum_{j=1, j \neq i}^n \frac{1}{x_i - r_j}}.$$

Como muchos de estos métodos, es sensible a la eficiencia en el cálculo de la corrección de Newton (ver 224) y la selección de los candidatos iniciales.

Sustituyendo r_i con la siguiente aproximación x_{i+1} se obtiene el método de Aberth-Ehrlich. Mejoras para raíces múltiples se deben a Anton Iliev, quien modificó el método Ehrlich-Kjurkchiev, así como a Wang and Zheng, que en 1984 publicaron una variante de orden 4.

The iteration may be executed in a simultaneous Jacobi-like iteration where first all new approximations are computed from the old approximations or in a sequential Gauss-Seidel-like iteration that uses each new approximation from the time it is computed.

6.7.3. Método de Laguerre

Edmond Nicolas Laguerre (1834 - 1886), fue un matemático francés más conocido por sus estudios de funciones especiales. Su método es lo que se puede denominar sospechoso con excelentes resultados prácticos.

El método de Laguerre trabaja con una aproximación inicial no necesariamente buena. Sea $p(x)$ un polinomio de grado n y x_0 un candidato inicial a raíz. Se calculan

$$\begin{aligned} a &= \frac{p(x)}{p'(x)} \\ b &= a^2 - \frac{p''(x)}{p(x)} \\ h &= \frac{n}{a \pm \sqrt{(n-1)(nb - a^2)}} \end{aligned}$$

y la actualización de la aproximación se hace con $x_{i+1} = x_i - h$. Al calcular c , el signo del denominador se selecciona de tal forma que se obtenga la mayor magnitud: $a + \text{signo}(a)\sqrt{(n-1)(nb - a^2)}$, que coincide con el signo de $p'(x_i)$. Es interesante notar que si $n = 2$ se obtiene el método de Halley (la forma irracional).

Hay dos cosas que llaman la atención:

1. la selección del signo y
2. la aproximación x_i está a una distancia h de la raíz r y las otras raíces agrupadas un una misma distancia g .

Como no hay justificación formal conocida para ninguna, este método casi no se emplea en librerías comerciales, prefiriéndose otros cuya convergencia sí está demostrada.

El método se deriva de considerar que si $p(x) = (x - r_1)(x - r_2) \cdots (x - r_n)$, entonces

$$\ln p(x) = \ln(x - r_1) + \ln(x - r_2) + \cdots + \ln(x - r_n)$$

y su derivada es

$$\begin{aligned} \frac{d}{dx} \ln p(x) &= \frac{1}{x - r_1} + \frac{1}{x - r_2} + \cdots + \frac{1}{x - r_n} \\ &= \frac{1}{h} + \frac{n-1}{g}. \end{aligned}$$

por el segundo supuesto. Si se repite con la segunda derivada se obtiene

$$\begin{aligned} \frac{d^2}{dx^2} \ln p(x) &= \frac{1}{(x - r_1)^2} + \frac{1}{(x - r_2)^2} + \cdots + \frac{1}{(x - r_n)^2} \\ &= \frac{1}{h^2} + \frac{n-1}{g^2}. \end{aligned}$$

Las dos ecuaciones se resuelven simultáneamente para h , de donde se obtiene el método.

Pese a los supuestos, su convergencia cúbica, el hecho de que sin importar lo cercano de la primera aproximación, el método converge a alguna raíz y su simplicidad comparada con los de otras técnicas lo hacen un candidato para muchos problemas.

El método de Laguerre es más laborioso que otros porque necesita la primera y segunda derivadas del polinomio, pero se compensa por tener convergencia cúbica, lo que reduce la cantidad de iteraciones. Si la raíz es múltiple, la convergencia es lineal, pero está garantizada de cualquier forma, a diferencia de otros métodos como el de Newton. La falta de una prueba formal de su convergencia impide que sea seleccionado en muchas aplicaciones, pero la sencillez y su convergencia cúbica lo hacen buen candidato en otras ocasiones, como en cálculos a mano o incluso para refinamiento de raíces.

En el caso de raíces simples, hay una variante con orden de convergencia 4, debida a Hansen, Patrick y Rusnak, de 1977. BIT NUMERICAL MATHEMATICS Volume 17, Number 4 (1977)

La iteración original se modifica a

$$z_j^{(k+1)} = z_j^{(k)} - \frac{n}{T_1 \pm \sqrt{(n-1)(nT_2 - T_1^2 - n\delta_j^2)}},$$

donde

$$\delta_j^2 = \sum_{\substack{i=1 \\ j \neq i}}^n \left[\frac{1}{z_j^{(k)} - z_i^{(k)}} - \frac{1}{n-1} \sum_{\substack{i=1 \\ j \neq i}}^n \frac{1}{z_j^{(k)} - z_i^{(k)}} \right]^2$$

lo que significa más cálculos, pero cuadruplicando los dígitos exactos cada iteración.

6.7.4. Método de la Matriz Adjunta

Este método trabaja para polinomios mónicos, es decir, aquellos donde el coeficiente del término de mayor grado es 1. Para el polinomio

$$p(x) = x^n + a_{n-1}x^{n-1} + \dots + a_1x + a_0$$

se construye la matriz cuadrada

$$\begin{bmatrix} 0 & 0 & \cdots & 0 & -a_0 \\ 1 & 0 & \cdots & 0 & -a_1 \\ 0 & 1 & \cdots & 0 & -a_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & 1 & -a_{n-1} \end{bmatrix}.$$

El polinomio p es el polinomio característico de esta matriz. En caso de tener n raíces distintas, estas serán los valores propios o eigenvalores de esta matriz.

Existe una gran cantidad de métodos para encontrar los eigenvalores de una matriz. De hecho el cálculo de eigenvectores es toda una gran área de investigación, por lo que no presentaremos los métodos que se han propuesto para su solución. Ante problemas serios, lo más recomendable es utilizar librerías probadas que resuelvan eigensistemas. La mayor parte de las rutinas actuales derivan de los métodos ideados por Wilkinson, así que sus publicaciones son una buena referencia para el tema.

Muy recomendable es la librería BLAS, (Basic Linear Algebra Subroutines), donde varios expertos de primera línea participaron, consiguiendo rutinas de la más alta eficiencia. Véase [243].

Los métodos directos y sencillos como el método de las potencias resultan imprácticos. Normalmente, un buen paquete de solución de eigensistemas trae rutinas separadas para calcular el mayor eigenvalor, el menor, el eigenvector correspondiente a un eigenvalor, para matrices simétricas, Hermitianas, tridiagonales, en fin. El problema es bastante complejo. Dos técnicas relacionadas son los métodos de Bernoulli y Graeffe, que se presentan poco más adelante.

Es por ello, que la búsqueda de raíces se ve beneficiada de los métodos que calculan eigenvalores. El tiempo que se le ha dedicado al problema ha producido

algoritmos que son buenos en general para localizar las raíces del polinomio característico.

Se sugiere al lector interesado remitirse a referencias especializadas.

Consultar High order simultaneous zeros

S. Goedecker, Remark on Algorithms to Find Roots of Polynomials, SIAM J. Sci. Comput. 15(5), 10591063 (September 1994).

Wankere R. Mekwi (2001). Iterative Methods for Roots of Polynomials. Master's thesis, University of Oxford.

V. Y. Pan, Solving a Polynomial Equation: Some History and Recent Progress, SIAM Rev. 39(2), 187220 (June 1997).

Anthony Ralston and Philip Rabinowitz, A First Course in Numerical Analysis, McGraw-Hill, 1978, ISBN 0-07-051158-6.

XXXXXXXXXXXXXXXXXXXXX

Es similar al método de Abert, pero aquí se agrega a cada iteración una etapa de XXXXXXXXXXXX

Zhonggang Zeng utiliza ideas de geometría algebraica para resolver el problema, que le mereció el premio por el artículo distinguido de la ACM en la conferencia ACM ISAAC (International Symposium on Symbolic and Algebraic Computation) de 2003.

Existe una técnica desarrollada por Andrew Sommece que utiliza ideas algebraicas y simbólicas distintas.

1974 simultaneous approximation, Alefeld y Hertzberger.

Bairstow, Newton (Madsen, Grant-Hitchins, Jenkins-Traub), Halley,

Capítulo 7

Programación de funciones:

e^x

elefunt

La programación de funciones elementales es toda una rama del cómputo numérico que desde los inicios de las computadoras cobró importancia. Ya desde siglos atrás, la necesidad de hacer cálculos manuales fomentó el desarrollo de técnicas eficientes que facilitarían la publicación de tablas trigonométricas y logarítmicas.

Los logaritmos son desde entonces un recurso invaluable al momento de hacer cálculos pues reducen problemas complicados a versiones más sencillas debido a sus propiedades. Las tablas con logaritmos se popularizaron hasta la llegada de las calculadoras de bolsillo.

Los logaritmos fueron inventados por John Napier (1550–1617), matemático escocés.

Desde finales de la década de 1940 a la fecha, ha habido grandes avances en la programación de funciones elementales, por los trabajos de excelentes matemáticos como Hirono Kuki¹, James Cody, William Kahan y más recientemente Jean Michel Muller. Al inicio, se tenía que hacer todo el trabajo con software pues no había un estándar como el 754 de IEEE que comprometiera ciertos requisitos mínimos en los procesadores.

En los 70s se discutió la posibilidad de que las funciones elementales estuvieran en el procesador mismo y no en la biblioteca de funciones del compilador, por ejemplo [300]. La carencia de normas que facilitaran las decisiones dio pie a discusiones como la publicada en [30] que abogaban por una norma para funciones elementales². La sugerencia no se cumplió pues al año siguiente se aprobó la norma 754 de IEEE y la historia cambió.

¹Vale la pena la lectura del *In memoriam* del padre de la aritmética de punto flotante tras la muerte de Kuki [196]. Kuki hizo de la mediocre librería matemática de Fortran la mejor del momento, ayudado por Cody, trabajo reportado en [228].

²En ese entonces, un error de 2 ulp era admisible para las funciones elementales.

De cualquier manera, en 2004 se publicó [88], aprovechando la revisión de la norma de punto flotante. Los autores (Defour, Zimmermann y Muller entre otros) proponen las bases de una norma para la programación de funciones matemáticas, considerando gran cantidad de aspectos. Lo central radica en los tres niveles de redondeo en que se agrupan las funciones.

Desde el principio los programadores se dieron cuenta de que las rutinas que aproximaran funciones trascendentes tenían que hacerse con muchos cuidados. La exactitud teórica de polinomios como el de Taylor son un verdadero desastre cuando se realizan mal. Muchas estrategias, como las presentadas en el capítulo 3 fueron empleadas, algunas con gran eficiencia.

Aunque se pensó en la posiblemente alta complejidad de rutinas eficientes, comparadas con las operaciones aritméticas, la experiencia ganada en los primeros años mostró que no era del todo así. En 1984, Alt demostró en [8] que las funciones elementales eran tan complejas como la división, en términos del tamaño del circuito lógico necesario para evaluarlas, para una salida de n bits la complejidad es $O(\log n)$ en la profundidad del circuito. Su resultado es para punto fijo, pero aplica igual para punto flotante.

Cuatro años después, publicó otro artículo [9] donde acota aún más la complejidad para el caso de las funciones algebraicas, mostrando que es equivalente a la multiplicación (la misma profundidad de $O(\log n)$ con un tamaño de $O(n \log n \log \log n)$ simultáneamente³).

Cada función elemental tiene características particulares, que motivaron diversas técnicas de programación. En general pueden agruparse dependiendo de sus propiedades y el resultado ha sido un gran conjunto de cada vez mejores aproximaciones en busca de la cota ideal: el $\text{ulp}(f(x))$ correctamente redondeado.

Este capítulo presenta una variedad de formas de programar rutinas que aproximen la función exponencial e^x y diversas maneras de probarlas, apoyándose en los principios básicos generales.

Pudiera haberse utilizado cualquier otra función, pero esta es sólo una muestra representativa de las técnicas que se pueden aplicar para resolver problemas eficientemente. El lector debe referirse a libros como [76, 281] para mayor información. Trigonométricas, hiperbólicas o logaritmos hubieran ofrecido un bufet similar.

7.1. Fundamentos de programación de funciones

Antes de abordar el caso particular de la función exponencial, es conveniente revisar, al menos superficialmente la teoría general. Aunque las funciones

³La división también tiene ese tamaño, pero aún se desconoce si ocurre simultáneamente con profundidad $O(\log n)$.

elementales se definen sobre el conjunto de los números reales, los únicos resultados posibles están en el intervalo $[-2^{1022}, 2^{1022}]$. Los resultados mas pequeños en magnitud deben ser mayores en valor absoluto que 2^{-1074} o ser cero⁴.

Como son mucho más complejas que las operaciones elementales, la norma de 1985 no exigía que las funciones elementales dieran resultados correctamente redondeados hasta el último bit. Pero en la revisión de 2007, los avances en las investigaciones son tales que ya aparece ese requisito para algunas de estas. Una técnica generalizada es la que presenta Ziv en [405], que se comenta más adelante. La técnica redondeo de Ziv fue revisada y extendida en [278].

Ya son públicos algoritmos que calculan la mayoría de las funciones elementales, con algunas excepciones, por lo que todo programador debe asegurarse de trabajar con un compilador que las incluya en su biblioteca de funciones.

Los pasos típicos de la aproximación de una función elemental son los siguientes.

1. Reducción de rango, que pueden ser por etapas.
2. Aproximación por tabla y algún polinomio o función aproximante.
3. Reconstrucción, que solo revierte de manera segura la reducción.

La programación de funciones matemáticas elementales posee una basta teoría y está llena de detalles. Las siguientes secciones ofrecen una idea básica del camino que se debe seguir. La eficiencia en términos de velocidad y precisión es directamente proporcional a lo sofisticado de los algoritmos. Buenas fuentes de información son [49, 76, 282].

7.1.1. Fórmulas y propiedades

El primer recurso es buscar fórmulas, identidades o propiedades matemáticas. Aún sabiendo que las aproximaciones no pueden ser exactas debido a que se trabaja con $\mathbb{F}$ y no con $\mathbb{R}$, es deseable que se cumplan la mayor parte de las propiedades de las funciones elementales. Es de aquí de donde surge la batería de pruebas con que se estudia su desempeño, exactitud y precisión.

Una de las propiedades más importantes es la monotonicidad. Una función f es *monotónica* en un dominio D (que puede ser un simple intervalo) si se cumple que $x < y$ implica $f(x) < f(y)$.

Idealmente, si $x \in \mathbb{F}$ es un número de punto flotante y $f : \mathbb{R} \rightarrow \mathbb{R}$ es una función estrictamente creciente, entonces se espera que

$$f(x) < f(x + \text{ulp}(x)). \quad (7.1)$$

Por la precisión limitada de las computadoras, el $<$ de (7.1) se relaja a $\leq$. Claro, si una función es estrictamente creciente, se espera que su aproxima- $f : \mathbb{F} \rightarrow \mathbb{F}$

⁴Esto aún es cierto debido a que la nueva norma aún no entra en vigor. El rango se debe ampliar con la llegada de la precisión cuádruple.

mación sea al menos creciente y no ocurra que exista $x \in \mathbb{F}$ tal que $f(x) > f(\text{nextUp}(x))$.

La limitante anterior es clara si se piensa en una función como $\sqrt{x}$, que mapea el intervalo $[0, 4]$ al $[0, 2]$, donde existen la mitad de elementos de $\mathbb{F}$, contrario a la definición de función. Por ello es imposible suponer que no existan $x, y \in \mathbb{F}$, con $x < y$ tales que $\text{fl}(\sqrt{x}) \neq \text{fl}(\sqrt{y})$.

A este respecto, Dunham publicó [99] en 1987, donde muestra las dificultades de probar la monotonidad de funciones en general, especialmente en el caso de polinomios de grado alto. Tres años después, Dunham publicó artículo de dos páginas, bastante pesimista, [100] donde pone en duda la posibilidad de logra la cota perfecta en la evaluación de funciones elementales⁵.

Otra propiedad importante para algunas funciones es la *simetría*, que implica que $f(-x) = f(x)$ o la *antisimetría*, $f(-x) = -f(x)$. La función seno es antisimétrica y coseno es simétrica.

Algunas funciones poseen otra agradable propiedad: son *periódicas*. Eso significa que $f(x) = f(x + k)$ con $k \in \mathbb{R}$ conocido como el periodo. Es el caso de algunas funciones trigonométricas, cuyo periodo es $\pi/2$.

Aunque las anteriores propiedades son generales, no debe olvidarse las propiedades particulares que cada función puede tener. Todo elemento teórico es útil para diseñar rutinas eficiente o para probarlas.

7.1.2. Reducción de rango

Este es el primero de los pasos que debe realizar un programa que aproxime una función elemental y es realmente crítico. Por ello esta sección se extiende más que las otras en esta introducción a la programación de funciones.

La *reducción de rango* consiste en aplicar propiedades y transformaciones algebraicas de tal manera que el rango del argumento se acote a un subconjunto donde se garantice una mejor aproximación. El ejemplo típico es que no se requiere programar la función seno en todo $\mathbb{R}$ pues $\sin 370^\circ = \sin 10^\circ$. De hecho, con ajustar el argumento a $[0, \pi/4]$ es suficiente.

El argumento original x debe reducirse a un nuevo valor más pequeño y . Si el *argumento reducido* y se encuentra en un intervalo $[-t, t]$ se llama *reducción simétrica*. En caso de que el intervalo sea $[0, t]$ se llama *reducción positiva*. El intervalo final es llamado *intervalo de convergencia*. Normalmente no se es tan estricto con los límites y en la práctica hay cierto valor ϵ que se pueden extender en ambos sentidos, dependiendo de la reducción, facilitando aún más las cosas.

Además del tipo de la forma del intervalo, hay dos tipos básicos de reducción de rango, la aditiva y la multiplicativa, conocidas desde hace mucho tiempo. En la *reducción aditiva* se busca un entero k tal que el argumento reducido

⁵Incluso se queja de que Kahan sí garantiza .5 ulp pero sin mostrar las técnicas con que se logra.

$y = x - kC$ quede en el rango deseado y C es una constante que depende de la función. Eso es lo que se hace en el caso de la función seno, donde puede hacerse $C = \pi/2$.

En la *reducción multiplicativa* el argumento reducido adopta la forma $y = x/C^k$, pero ahora C se elige como la base numérica. Por ejemplo, en el caso de $\ln(x)$, se busca k tal que $y = x/2^k \in [.5, 1]$ y evaluar $\ln(x) = \ln(y) + k \ln(2)$, por lo que puede hacerse $C = \ln 2$.

Cualquiera que sea la idea que se aplique, debe entenderse que debido a las operaciones aritméticas realizadas, la equivalencia matemática no está garantizada. Basta pensar que el redondeo de la expresión $x - k\pi/2$ puede tener un error mayor a .5 ulp. En general, hacer $k = \lfloor x/C \rfloor$ y $x - kC$ es desastroso si $x \approx kC$.

Otro problema grave es que posiblemente el argumento x fuera mucho muy grande, molestia de las funciones trigonométricas en particular. Podían requerirse muchos bits adicionales de π para asegurar que la etapa de normalización no fuera perjudicial.

Ante estos problemas, los programadores de rutinas matemáticas comenzaron a desarrollar técnicas seguras de reducción de rango, algunas para casos especiales y otras orientadas a la mayoría de los casos.

El dilema de los constructores de tablas

Utilizar precisión múltiple puede resultar mucho más costoso, por lo que no es una solución adecuada, además de que no es trivial valorar cuánta precisión se necesita. El problema exacto se conoce como el *table maker's dilemma* (TMD) y es el siguiente. Acuñado así por Kahan en 1974.

Se desea garantizar que la aproximación de $f(x)$ está correctamente redondeada, es decir, $\text{fl}(f(x)) = f(x)(1 + \delta)$ con $\delta < \mu$, para todos los modos de redondeo. La mantisa de la aproximación de $f(x)$ antes de la renormalización tendrá q bits extra en total, $q > p$, que son necesarios para redondear correctamente y obtener $\text{fl}(f(x))$ con un error máximo de .5 ulp.

Redondear no es trivial pues pueden ocurrir los siguientes casos. Si el modo de redondeo es al más cercano, el resultado puede estar muy cerca del punto medio de los dos NPF más cercanos a $f(x)$, más de lo que la precisión intermedia permite determinar. Por ejemplo, si la mantisa de $f(x)$ es

$$\overbrace{.d_1 d_2 d_3 \dots d_p 1000 \dots 0 b_1 b_2 b_3 \dots}^{q \text{ bits}} \quad \text{o} \quad \overbrace{.d_1 d_2 d_3 \dots d_p 0111 \dots 1 b_1 b_2 b_3 \dots}^{q \text{ bits}} \quad (7.2)$$

antes de la renormalización. $b_1 = 1$ basta para cambiar el redondeo. De haber usado $q + 1$ bits no habría habido duda. Ese es realmente el TMD, no saber cuántos se requieren. Los modos de redondeo direccionado no se salvan pues

$$\overbrace{.d_1 d_2 d_3 \dots d_p 000 \dots 0 b_1 b_2 b_3 \dots}^{q \text{ bits}} \quad \text{o} \quad \overbrace{.d_1 d_2 d_3 \dots d_p 111 \dots 1 b_1 b_2 b_3 \dots}^{q \text{ bits}} \quad (7.3)$$

dan problemas similares en `roundUp` y `roundDown` respectivamente, o el correspondiente redondeo a cero dependiendo del signo.

El problema persiste: ¿cuántos bits son necesarios para redondear correctamente? Si la aproximación intermedia no es suficientemente exacta, se tiene algunos de los casos anteriores de mantisa. Kahan sugirió un método que usa fracciones continuas para encontrar los peores casos de (7.2) y (7.3), que se ha usado desde entonces, basta revisar el libro [281], de 2006.

Una especie de lista negra de argumentos.

Se lo caliza a así a los peores casos y se puede determinar cuánto cómputo extra se requiere, separando los casos sencillos (la mayoría) y dedicando mayor esfuerzo a los otros.

Si $x = .x_1x_2\dots x_p \times \beta^e$ y se escribe con mantisa entera como $x = M \times \beta^{e+1-p}$, entonces la reducción de rango aditiva equivale a encontrar un entero k y un número real $|r| < 1$ tales que

$$\frac{x}{C} = k + r \quad (7.4)$$

y $y = rC$ es la reducción de rango. La ecuación (7.4) puede reescribirse como

$$\frac{\beta^{e+1-p}}{C} = \frac{k}{M} + \frac{r}{M}. \quad (7.5)$$

Si la reducción de rango y es muy pequeña en magnitud, r también debe serlo, por lo que el lado izquierdo de la ecuación (7.5) debe ser una muy buena aproximación de k/M , dándose una de las situaciones (7.2) o (7.3). Lo que se requiere para encontrar el menor valor posible de y es analizar la sucesión de la mejor aproximación racional posible a $\frac{\beta^{e+1-p}}{C}$, lo que se hace con fracciones continuas.

Los detalles de esta técnica están disponibles en los extensos (y desafortunadamente en código ASCII) comentarios del programa `nearpi.c`, escrito por McDonald a partir de una versión en Basic de Kahan [198]. En su libro [281], Muller incluye los peores casos para algunas funciones elementales en diversos rangos⁶, para precisión doble de IEEE.

El remedio práctico, sugerido inicialmente por Gal y Bachelis en [16] con ajustes finales de Ziv en [405], es aproximar $f(x)$ de nuevo con un valor mayor de q y repetir hasta asegurar el redondeo. La primera aproximación puede hacerse con las operaciones nativas del procesador, pero las siguientes requerirán de precisión múltiple. En la práctica, el redondeo correcto puede hacerse con una o dos iteraciones en la mayoría de los casos.

La importancia de determinar los peores casos para cada función elemental (el valor máximo de q) radica que eso acota el problema pues ya se tendría por adelantado la precisión máxima necesaria por función y por dominio, resolviendo el TMD. Logrado eso ya se puede diseñar una evaluación tal que el resultado final difiera del infinitamente correcto en .5 ulp como máximo.

⁶También [239] (o perso.ens-lyon.fr/jean-michel.muller/TMDworstcases.pdf, que es una versión más actualizada).

Dentro de las funciones elementales donde los peores casos no han sido determinados están x^n y x^y , por lo que aún no existen algoritmos que alcancen la cota perfecta.

Reducciones de rango más eficientes

Para resolver los problemas intrínsecos de la reducción de rango, se han desarrollado varias técnicas. En los años recientes, los procedimientos hacen uso de la norma de punto flotante para mejorar la eficiencia.

En el libro [76], Cody y Waite sugieren encontrar dos valores C_1 y C_2 con A la Dekker. representación exacta en $\mathbb{F}$ tales que $C = C_1 + C_2$, con $C_1 \approx C$ usando pocos bits. De esta manera, la evaluación $x - kC$ se realiza con mayor exactitud con

$$y = (x - kC_1) - kC_2 \quad (7.6)$$

lo que simula una precisión mayor que la nativa del procesador.

En esencia, es un método rápido que sirve para reducir la cancelación de cifras. Puede extenderse a usar 3 constantes con poca complejidad temporal adicional.

Posteriormente, en 1983 Payne y Hanek publicaron en [303] un algoritmo para la reducción de rango aditiva más seguro, que se aplicó de inmediato a la librería matemática de las computadoras VAX. En ese entonces, la norma 754 de IEEE no se había aprobado y como VAX era la compañía que más se negaba a aceptar el underflow gradual, este método de reducción no dependía de la representación de los NPF.

El trabajo fue desarrollado pensando en argumentos muy grandes en las funciones trigonométricas, por lo que $C = \pi/4$. El argumento reducido y debe quedar en $[0, C]$. EL algoritmo tiene tres características importantes:

1. En el caso de argumentos muy grandes, evita los cálculos con aquellos bits que de cualquier manera serán descartados.
2. Utiliza bits adicionales alrededor de las raíces de la función que se evalúa, donde generalmente es crítico el resultado.
3. En los otros casos, la complejidad algorítmica es independiente del tamaño del argumento.

Siguiendo la formulación inicial, sea $r = 2\pi/2^{-i}$ e i toma valores de 0 a 4, dependiendo si se considera el periodo entero, la mitad, un cuadrante o un octante. El objetivo es determinar el entero no negativo j , un entero l y un valor h tales que cumplan con

$$\begin{aligned} 0 &\leq l < 2^i \\ 0 &\leq h < 1 \end{aligned}$$

y de tal forma que

$$x = r(j2^i + l + h) \quad (7.7)$$

con lo que se está separando el cociente x/r en dos partes enteras y una real. Es importante notar que l tiene i ceros y es un múltiplo de 2π , por lo que no contribuye en el caso del seno o coseno.

Con la actual norma aprobada, considerando los p de bits de mantisa, x puede escribirse en términos de un entero M como $M \times 2^{e-p+1}$, donde, en este caso, e no está recorrido como en la norma. M satisface $2^{p-1} \leq M \leq 2^p$.

Como la reducción de rango en general puede escribirse como $y = \pi/4(4x/\pi - k)$, el valor $\alpha = 4/\pi$ toma gran importancia pues la reducción de Payne y Hanek separa la mantisa de αx en tres partes, que juntas forman la cadena de p bits del formato.

La separación queda

$$\alpha x = 8Mp_1(x) + 2^w Mp_2(x) + 2^w Mp_3(x)$$

donde w depende de la precisión que se necesita, permitiendo separar de manera conveniente en tres la mantisa y $p_i(x)$ son las partes de la mantisa ya separada de x . Como $8Mp_1(x)$ es múltiplo de 8, no influye en la evaluación de la función trigonométrica. Por su parte, $2^w Mp_3(x)$ puede hacerse tan pequeño como se desee.

Algunos expertos consideran que el algoritmo de Payne y Hanek es exacto pero adolece de requerir demasiadas operaciones para garantizar tal exactitud, por lo que se utiliza exclusivamente para argumentos grandes ($x > 2^{40}$).

En 1994, fue descrito por Daumas el algoritmo que se conoce como *reducción modular de rango* (RMR), pero su análisis completo se publicó en [86] un año después, por los mismos 4 autores. El método no es exclusivo para funciones trigonométricas.

RRM se basa en reescribir el argumento en punto fijo base 2, quedando

$$x = x_{n-1}x_{n-2} \dots x_0.x_{-1}x_{-2} \dots x_{-p}$$

que representa al número $\sum_{i=-p}^{n-1} x_i 2^i$.

Sea v un entero que cumple con $2^v < C \leq 2^{v+1}$ y se define $m_i \in [-C/2, C/2]$ tal que $(2^i - m_i)/C$ es un entero, con $i \geq v$.

El primer paso de RMR es calcular

$$r = x_{n-1}m_{n-1} + x_{n-2}m_{n-2} + \dots + x_v m_v + x_{v-1} \dots x_{-p}$$

que queda en el intervalo $[-\frac{C(n-v+2)}{2}, \frac{C(n-v+2)}{2}]$.

Para el segundo paso, sean r_i los dígitos binarios de r , $\epsilon > 0$ un número pequeño y s los primeros bits de r descartando los últimos $\lceil -\log(\epsilon) \rceil$.

s tiene $\left\lfloor \log\left(\frac{C(n-v+2)}{2}\right) \right\rfloor + \lceil -\log(\epsilon) \rceil$ bits, que en general se espera que sea un número pequeño.

Ahora se tienen dos opciones, dependiendo de la reducción deseada. Si k se selecciona como el entero más cercano a s/C , entonces $r - kC$ queda en $[-C/2 - \epsilon, C/2 + \epsilon]$.

Si se hace $k = \lfloor s/C \rfloor$, entonces $r - kC \in [-\epsilon, C/2 + \epsilon]$. Esto permite seleccionar reducción simétrica o positiva con facilidad.

El segundo paso puede programarse como una simple búsqueda en tabla, sin hacer ningún cálculo. El mayor costo del algoritmo es la suma de los $n - v + 1$ términos de la primera reducción. La extensión de RMR para números de punto flotante es realmente simple, simplemente reemplazando la suma de los m_i por la suma de m_{e-i} , donde e es el exponente de la representación flotante. Como antes, $m_i = 2^i \bmod C$.

El método de Payne–Hanek es el adecuado para grandes argumentos, mientras que RMR lo es para pequeños ($x < 10$). En 2005 se publicó un nuevo método adecuado para argumentos de mediano tamaño en [46].

Lo complicado del método lo deja más allá de las intenciones de esta introducción a los métodos para reducción de rango. El artículo [46] presenta en su primera parte el algoritmo de Payne–Hanek, así como un estudio estadístico de la distribución de los argumentos reducidos en el intervalo de convergencia y la forma de analizar sus peores casos (el TMD de la sección anterior, que los autores definen como los valores del argumento x para los que se obtienen los menores valores del argumento reducido y , en valor absoluto.).

En la segunda parte, presentan el algoritmo, planteado para una reducción aditiva con $C = \pi/2$, pero se asegura que la extensión a otras constantes es directa. El que se separa el argumento en 8 partes I_i de 7 bits cada una. Aplicar una primera reducción con

$$S(x) = I_0(x) \bmod \pi/2 + 2^8 I_1(x) \bmod \pi/2 + 2^{16} I_2(x) \bmod \pi/2 + \\ + \dots + 2^{56} I_7(x) \bmod \pi/2 + \rho(x)$$

y $\rho(x)$ es exactamente representable en precisión doble. Si $x > 2^{52}$, entonces $\rho(x) = 0$. El valor $x - S(x)$ es un múltiplo de $\pi/2$. A $S(x)$, se le aplica una segunda reducción de rango. Los cálculos indican que para alcanzar el objetivo de redondeo correcto aún en los peores casos, se requieren 153 bits.

Los valores $|2^{8i} I_i(x)| \bmod \pi/2$ deben guardarse en tablas para garantizar la rápida reducción de rango. Cada valor requiere 3 valores en precisión doble para mantener los 153 bits. La reducción también queda almacenada en tres variables, por lo que el resultado final se obtiene con algoritmos suma de precisión múltiple. EL sugerido por los autores es el de Knuth en [219], donde la suma exacta de $a + b$ se almacena en dos variables $z + r$.

El argumento reducido final y se representa con dos variables, de tal forma que $y = y_h + y_l$ con una precisión de 2^{-87} , que para el rango del intervalo de convergencia es mejor que la cota perfecta.

Actualmente, una buena rutina de reducción de rango decidirá la técnica de reducción más conveniente, dependiendo del argumento x . La reducción de rango puede repetirse varias veces, hasta llegar al intervalo donde la aproximación polinomial sea tan confiable como se requiere.

7.1.3. Tipos de aproximaciones polinomiales

En 1885, Karl Weierstrass demostró⁷ que toda función $f : \mathbb{R} \rightarrow \mathbb{R}$ continua en un intervalo $[a, b]$ puede ser aproximada por polinomios que pasan arbitrariamente cerca de la función. Es decir, dado $\epsilon > 0$, existe un polinomio p que cumple con

$$\|f(x) - p(x)\| < \epsilon$$

para toda $x \in [a, b]$.

El teorema de Weierstrass es de gran importancia tanto teórica como práctica pues garantiza que con operaciones elementales se pueden aproximar funciones continuas de cualquier tipo.

Un corolario sencillo es que existe una infinidad de aproximaciones polinomiales, así que el problema se reduce a seleccionar la más adecuada para ser programada. Existen dos criterios básicos de selección, minimizando ya o sea el error medio o la distancia máxima.

Es una aproximación de mínimos cuadrados.

Minimizar el error medio significa encontrar la aproximación p tal que la distancia $d(f, p) = \|f(x) - p(x)\|$ sea mínima en el dominio de interés $[a, b]$. Una forma común de definirla es

$$d(f, p) = \sqrt{\int_a^b (f(x) - p(x))^2 dx}$$

pero no es la única manera. Polinomios importantes de esta familia son los de Chebyshev, Laguerre, Jacobi o Legendere.

También conocida como aproximación *minimax*.

La otra forma es *minimizar el error máximo*. En este caso, se define la distancia como

$$d(f, p) = \max_{a \leq x \leq b} \|f(x) - p(x)\| \quad (7.8)$$

y el polinomio más conocido es el de Taylor. El error lo acota el residuo, que es el último término de la ecuación (A.37) de la página 359.

Es más complicada esta técnica, por lo que se recurre no solo a polinomios. De las más conocidas son las aproximaciones racionales de Padé, que son cocientes de polinomios que se obtienen a partir del de Taylor. La aproximación de Padé se denota como $P_{m,n}$ y consiste en un polinomio de grado m sobre uno de grado n que aproximan a la función tan bien como el polinomio de Taylor de grado $m + n$. Más detalles pueden consultarse en textos como [133, 217].

⁷La prueba original es para funciones analíticas, es decir, definidas sobre el plano complejo $\mathbb{C}$. Existen muchas generalizaciones, dentro de ellas, una muy importante es el teorema de Stone-Weierstrass, en espacios de Banach.

El método estándar para encontrar la mejor aproximación en espacios de Chebyshev (de polinomios) es el algoritmo que Evgeny Y. Remez publicó en 1934, utilizado por paquetes como Matlab, Maple o Mathematica. El polinomio encontrado con el algoritmo de Remez se llama polinomio de Chebyshev.

Detalles de implementación pueden consultarse en el artículo de Fraser [123]. La idea básica es interpolar la función en un conjunto de puntos iniciales (los nodos de Chebyshev) e irlos ajustando aplicando el teorema de alternancia de Chebyshev hasta lograr el mínimo error con el criterio 7.8. Detalles adicionales sobre la selección de los puntos iniciales está en el artículo de Dunham [98].

El método es tan importante que se encuentra fácilmente en textos de deducción superior como [57, 217]. Comunmente está acompañado de la teoría de subespacios de Haar.

La manera de encontrar la mejor función aproximante depende de cuál de las dos estrategias se seleccione.

7.1.4. Aproximación por tablas

Si se dispone de memoria infinita, se podrían evaluar por anticipado las funciones de interés en cada NPF y sólo consultar cuando se necesiten. Aún cuando la memoria es barata, no lo es en ese grado, por lo que este esquema es impráctico.

Sin embargo, tener ciertas evaluaciones exactas en el intervalo de convergencia es indispensable. Tales evaluaciones definen subintervalos no necesariamente del mismo tamaño. Con esas aproximaciones exactas se pueden construir polinomios aproximantes que se ajusten a cada subintervalo, almacenando en una tabla los coeficientes que correspondan. Esta técnica se ha usado desde finales de los años 60.

Con esos polinomios la aproximación alcanza la convergencia en una cantidad fija de iteraciones. Los coeficientes se seleccionan dependiendo del subintervalo al que pertenezca el argumento reducido y . Por ejemplo, si e^x se aproxima en el intervalo $[0, \ln(2)]$ con un polinomio de Taylor truncado de grado 6 centrado en cero

$$p(x) = 1 + x + \frac{x^2}{2} + \frac{x^3}{6} + \frac{x^4}{24} + \frac{x^5}{120} + \frac{x^6}{720}$$

la diferencia $|e^x - p(x)|$ es $O(10^{-5})$ y su gráfica se presenta en la figura 7.1. Es evidente que el error no está distribuido uniformemente, sino en un solo extremo del intervalo en este caso.

Entre más subintervalos se tengan se requerirá de polinomios de menor grado, lo que hace más veloz los procesos al tiempo de necesitar mayor cantidad de memoria. Un buen diseño se basa en el equilibrio adecuado entre la complejidad temporal y espacial, considerando las restricciones del sistema que se desarrolla.

Los puntos que delimitan los subintervalos y la evaluación exacta de la función en los mismos forman la tabla a la que se refiere el título de esta sección.

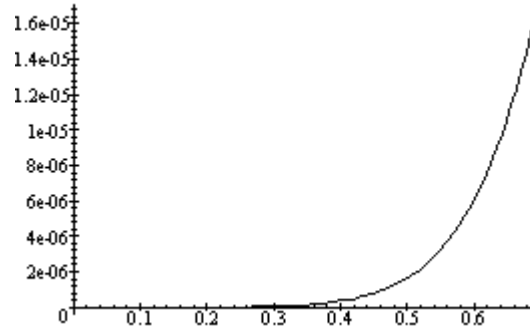


Figura 7.1: p de grado 6 en $[0, \ln(2)]$. Se grafica la diferencia $|e^x - p(x)|$. Es común que el mayor error esté en un extremo del intervalo.

Hay tres formas básicas de diseñarlas.

1. Las *tablas normales* son aquellas en las que el intervalo de convergencia se subdivide en partes iguales, típicamente una potencia de dos. Por ejemplo, si se separa en $2^5 = 32$ partes, entonces los primeros 5 bits del argumento reducido servirán como el índice en la tabla de 32 valores. Son sencillas de programar, con la desventaja de requerir una precisión mayor que la nativa para conseguir una aproximación con un error máximo de .5 ulp. El mejor representante es el algoritmo de Tang [361, 362].
2. Las *tablas exactas* no dividen el intervalo de convergencia en partes iguales, ya que los puntos de división son frecuentemente números reales de forma $iC/2^k$, con i y k enteros. Como normalmente C es trascendente, las subdivisiones deben redondearse, por lo que ya acarrearán un error. La idea de las tablas exactas es subdividir el intervalo de tal manera que cada punto de división x_i sea un número de $\mathbb{F}$. Se espera que las subdivisiones estén distribuidos lo más uniformemente posible. Este método no requiere el uso de precisión extra como en el caso de las tablas normales.

El representante típico es el algoritmo de Gal, descrito en [16] junto con todos los detalles de las tablas exactas (pasando por una reducción de rango similar a las presentadas y la aproximación minimax), evitando el uso de precisión extra, pero calculando el $\text{ulp}(x)$ correctamente redondeado. En dicho artículo, Gal y Bachelis construyen todo sobre la norma de punto flotante de IEEE, consiguiendo que el 99.7 % de los resultados estén correctamente redondeados⁸.

La idea de Gal fue extendida con éxito por Ziv en [405], logrando el 100 %

⁸A partir de una muestra de 300 000 evaluaciones distribuidas uniformemente entre $-174 < x < 174$.

de los resultados correctamente redondeados. Se utilizaron dos reducciones de rango y precisión extra cuando se detectaba algún caso de redondeo problemático.

3. Las *tablas múltiples* se componen de tablas en varios niveles. La idea es usar el argumento reducido para determinar el subintervalo de la primera tabla. Una vez determinado, el argumento se usa de nuevo para seleccionar el siguiente subintervalo. Puede haber tantas tablas como se necesiten. En la práctica estas tablas requieren implementarse en hardware, por lo que no se describirá ningún método.

No se presentan más detalles pues se utilizarán con la función exponencial.

7.1.5. Pruebas de exactitud

Las pruebas de eficiencia son más complicadas que el objeto de la prueba. El principal problema radica en el marco de referencia: ¿contra qué se compara? Si se compara contra las rutinas existentes, no se está suponiendo que sean eficientes, solo son el punto de comparación, por lo que la respuesta será relativa.

Medir la velocidad puede ser más sencillo pues las funciones que miden el tiempo son fáciles de utilizar. Medir la exactitud requiere conocer los resultados correctos para medir la exactitud de las funciones programadas. Cómo obtener esos resultados correctos para comparar es el problema.

Una manera es basarse en las propiedades de las funciones programadas y ver que las identidades se cumplen, salvo los errores de redondeo de las operaciones que implican. La selección de tales propiedades no es sencilla.

Lo mejor es construir una tabla de valores correctamente redondeados utilizando precisión múltiple y luego comparar los resultados de las rutinas programadas. Es la manera más sencilla de verificar la exactitud.

Los valores especiales deben asegurarse, por lo que la evaluación correcta en ∞ , $-\infty$, 0, ceros y asíntotas es importante. Además, toda prueba debiera incluir valores dentro de los peores casos del TMD (ver sección 7.1.2).

La manera más ampliamente utilizada para evaluar la exactitud es comparar las rutinas contra resultados calculados utilizando mayor precisión, como en [362]. La mayor complicación es que el programa de evaluación generalmente no es portable, además de que no puede aplicarse cuando la rutina que se prueba está en la máxima precisión posible. Claro, siempre es posible simular la precisión adicional con técnicas como las de Dekker.

En [75], Cody sugiere cómo usar polinomios de Taylor para evaluar la exactitud de rutinas y evitar el uso de precisión extra.

* * *

Como segunda parte de este capítulo se presentan una diversidad de maneras de programar la función exponencial. Las primeras versiones son tan sencillas como inocentes, pero la idea es ilustrar las distintas técnicas de programación posibles, poniendo en evidencia aquellas que resultan deficientes.

Aunque hay métodos muy sofisticados que alcanzan la cota perfecta de $fl(e^x) = e^x(1 + \delta)$ con $|\delta| \leq \mu$, estos no se muestran con todos los detalles por la extensión que se requiere para explicarlos. Si el lector desea estudiarlos, se sugiere ir a las referencias que aparecen en el texto.

7.2. Propiedades básicas de e^x

La función exponencial es bastante conocida y se sabe de gran cantidad de propiedades. Aparece constantemente en identidades de todo tipo. Para efectos de cómputo interesan las propiedades que involucran pocas operaciones.

En resumen, se puede decir que la función $\exp(x) = e^x$ es real⁹, trascendente, definida positiva, monotónica y estrictamente creciente. Además cumple con propiedades útiles (algunas) al momento de probar si una rutina que la aproxime trabaja bien, como son las siguientes.

$$e^0 = 1 \quad (7.9)$$

$$\begin{aligned} e^1 &= \lim_{n \rightarrow \infty} \left(1 + \frac{1}{n}\right)^n \\ &= 2.71828182845904523536\ 0287471352 \dots \end{aligned} \quad (7.10)$$

$$e^{a+b} = e^a e^b \quad (7.11)$$

$$e^x < e^y \quad \text{siempre que} \quad x < y \quad (7.12)$$

$$(7.13)$$

Otra propiedad importante es que su derivada es igual a sí misma, es decir, la recta tangente en el punto x tiene pendiente e^x .

Numéricamente, si se considera la precisión doble de IEEE, no puede calcularse e^x para todo NPF pues se causa overflow o underflow. La tabla 7.1 muestra los valores que debe regresar la función exponencial en diversos intervalos, incluyendo números subnormales.

Por ser una función tan importante para muchas ciencias, desde el inicio de la computación se han programado aproximaciones cada vez mejores. Se han usado desde polinomios de Taylor, fracciones continuas y todo tipo de esquemas.

⁹La definición para variable compleja $e^z = e^{a+ib} = e^a(\cos b + i \sin b)$ no se analizará.

Valor de x			Resultado de e^x
		$709.7827 < x$	$+\infty$
$-708.396 \leq x \leq 709.7827$			Número normal
$-745.1332 \leq x < -708.396$			Número subnormal
$-2^{-54} \leq x \leq 2^{-53}$			1
$x < -745.1332$			0
NaN			NaN

Tabla 7.1: Los rangos válidos para evaluar la función e^x en $\mathbb{F}$, considerando precisión doble. Los valores son meras aproximaciones.

7.3. Aproximación de e^x con series de potencias

Lo más sencillo es generar el polinomio de Taylor con

$$e^x = 1 + x + \frac{x^2}{2} + \frac{x^3}{6} + \frac{x^4}{24} + \frac{x^5}{120} + \cdots \quad (7.14)$$

y evaluarlo en los puntos donde el usuario indique. Aunque en los libros de cálculo se diga que aumentando la cantidad de términos se reduce el error y se mejora la aproximación lejos de $x = 0$, esta no es una buena idea.

Tan solo para garantizar un error máximo de .5 ulp (un error absoluto máximo de 2^{-53}) en el intervalo $[-1, 1]$, se requieren 18 términos de (7.14), pues con 17 se tiene un error acotado por $1.68 \times 10^{-16} \approx 2^{52.4}$, es decir, casi .8 ulp. Sin embargo, con esos 18 términos, el error en $x = 2$ aumenta a 4.53×10^{12} .

Lo anterior significa que una serie de potencias u otra aproximación que se emplee, no obtendrá buenos resultados a menos que se acompañe de una adecuada reducción de rango. Una versión bastante deficiente se muestra en el código 7.1.

El código 7.1 es el cálculo directo de la fórmula (7.14) en la forma como típicamente lo hace un programador novato. Va calculando cada término como el cociente de la potencia y la actualización del factorial anterior (líneas 9 y 10) y lo agrega a la suma parcial `r` (línea 15). Se detiene al cumplirse una de tres condiciones:

1. El siguiente término ya no modifica la suma parcial (línea 11).
2. El siguiente término es menor que cierta tolerancia aceptable (línea 13).
3. Se alcanza el máximo de iteraciones. `MaxTerm` debe definirse previamente.

Pese a que en algebraicamente los cálculos, esta versión adolece de varias cosas, las más notables son las siguientes.

1. Lo más crítico es el uso de la función `pow`, incluida en las bibliotecas de funciones para calcular la complicada función x^y y no una potencia entera.

Los programadores principiantes abusan de esta función constantemente.

Código 7.1: Versión deficiente de la función exponencial.

```

1  double exp_v0(double x)
2  {
3      int i;
4      double r;      /* Resultado final */
5      double Fact=1.0, Term;

7      r = 1;
8      for ( i=1 ; i<MaxTerm ; i++ ) {
9          Fact *= (double)i;
10         Term = pow(x,i)/Fact;
11         if ( r+Term==r )
12             break;
13         if ( fabs(Term) < Tol )
14             break;
15         r += Term;
16     }
17     return r;
18 }

```

2. El cálculo por separado de x^n y $n!$ tiene riesgo de overflow aún cuando el cociente tenga representación en $\mathbb{F}$. Es más seguro actualizar el término anterior como

$$\frac{x^{n+1}}{(n+1)!} = \frac{x^n}{n!} \frac{x+1}{n+1}$$

3. La tolerancia `Tol` está dada en valor absoluto, lo que no es adecuado fuera del intervalo $[-1, 1]$.
4. En el caso de que el argumento x sea positivo, entonces el número de condición para la suma de la serie es (véase (4.5) en la página 147)

$$\text{cond}(1 + x + \frac{x^2}{2} + \frac{x^3}{6} + \dots) = 1.$$

Pero si $x < 0$ la cosa cambia radicalmente pues cada término con exponente impar será negativo y la cancelación catastrófica impide garantizar un error relativo bajo. En este caso lo mejor es valerse de la propiedad $a^{-x} = 1/a^x$.

5. Debieran verificarse los límites de overflow y underflow de la tabla 7.1.

Con las observaciones anteriores, el código debe modificarse. Una mejor versión, que aún adolece de la falta de reducción de rango y otros detalles se muestra en el código 7.2. La mejora cuando $x < 0$ es substancial, lo que es evidente a partir de experimentos sencillos.

Pocas mejoras adicionales son posibles. Puede utilizarse suma compensada en vez de la línea 17 o algún algoritmo más sofisticado de suma, pero no se

Código 7.2: Versión un poco más eficiente de la función exponencial.

```
1 double exp_v2(double x)
2 {
3     int i;
4     double r;
5     double Term=1.0;
6
7     if ( x<0 )
8         return 1/exp_v1(-x);
9
10    if ( x>709 )
11        return Huge*Huge;    /* overflow */
12    if ( x<-745 )
13        return 1/(Huge*Huge); /* underflow */
14    if ( fabs(x)<1e-16 )
15        return 1;
16
17    r = 1;
18    for ( i=1 ; i<MaxTerm ; i++ ) {
19        Term *= x/(double)i;
20        if ( r+Term == r )
21            break;
22        if ( fabs(Term) < Tol )
23            break;
24        r += Term;
25    }
26    return r;
27 }
```

trata de inexactitud, sino de imprecisión, por lo que se deben buscar mejores soluciones.

Una vez determinado el intervalo de convergencia, se puede dejar fija la cantidad de términos necesarios para alcanzar la precisión necesaria. Entre más pequeño el intervalo menos términos se requerirán.

Las comprobaciones iniciales (overflow, underflow y e^ε , líneas 7–15) no se realizarán en los siguientes códigos con el propósito de ahorrar líneas, pero debe suponerse que siempre son necesarias.

7.4. Reducciones básicas de rango

La reducción de rango para la función exponencial son del tipo multiplicativo, ya que no se trata de una función periódica. Sea $x \in \mathbb{R}$ el argumento de la función.

7.4.1. Reducciones sencillas

Esto no es realmente una reducción de rango, sino una técnica *divide y vencerás*.

Una primera idea es separarlo en dos problemas más sencillos: sus partes entera y fraccional. Si x tiene la forma $n.f$ con n entero y $|f| < 1$, entonces

$$e^x = e^{n+f} = e^n \times e^f \quad (7.15)$$

y ahora el problema se separa en dos partes: elevar e a una potencia entera¹⁰ y calcular e^f en el intervalo de convergencia $(-1, 1)$, donde puede utilizarse la función `exp_v2` con cierta confianza.

La mejor manera de separar x en sus partes entera y fraccionaria es utilizar la función `modf` especificada en la norma. Su prototipo especifica que recibe el valor flotante `x`, la dirección del entero `n` y regresa la parte fraccionaria `f`. La nueva versión de función exponencial se muestra en el código 7.3, donde la constante `M_E` se define como $\text{fl}(e^1)$ en `math.h`.

Lo único que falta es una función que calcule x^n , para n entero, de manera más eficiente que `pow`. La idea detrás del código 7.4 data de hace 22 siglos. Si bien no es óptima, pues para algunos valores de n se puede calcular x^n con menos multiplicaciones, al menos sí lo es asintóticamente. La idea es separar n como la suma de potencias de 2.

Otras técnicas más eficientes para calcular x^n pueden consultarse en [219], donde además se analizan las cotas. De la lectura se desprende que a la fecha no existe ningún método superior a otros para toda n .

Todo entero puede escribirse como la suma de potencias de 2, por lo que si

$$n = 2^{p_1} + 2^{p_2} + \dots + 2^{p_n}$$

¹⁰ e^n parece un problema sencillo, pero e es un número trascendente y por tanto $\text{fl}(e)$ es inexacto.

Código 7.3: Separando la función exponencial en sus partes fraccionaria y entera.

```
1 double exp_v3(double x)
2 {
3     double n, f;

4
5     /* modf se incluye en la biblioteca de todos
6        los compiladores que respetan el C ANSI */
7     f = modf(x, &n);

8
9     return Pot(M_E, n)*exp_v2(f);
10 }
```

Código 7.4: Método sencillo y eficientemente para calcular x^n .

```
1 double Pot(double x, int n)
2 {
3     double r;

4
5     if ( n==0 )
6         return 1;
7     if ( n<0 )
8         return 1.0/Pot(x, -n);

9
10    while ( !(n&1) ) {
11        n>>=1;
12        x *= x;
13    }
14    r = x;
15    for (;;) {
16        n>>=1;
17        if ( n==0 )
18            break;
19        x *= x;
20        if ( n&1 )
21            r *= x;
22    }
23    return r;
24 }
```

entonces

$$x^n = x^{2^{p_1}} x^{2^{p_2}} \dots x^{2^{p_n}}$$

que es lo que hace el código 7.4. Por ejemplo, $7^{11} = 7^8 7^2 7^1$ ya que $11 = 8 + 2 + 1$.

La función `exp_v3` tiene errores de redondeo intrínsecos. La separación con `modf` es exacta, pero el uso de `exp_2` no lo es, tampoco la potencia mediante `Pot` y el último redondeo no controlado es el producto de la línea 9 del código 7.3.

Lo anterior implica que hay que sumar los errores por lo que son varios ulp incorrectos. Sin embargo no deja de ser una versión que depende de la buena aproximación de `exp_v2` en $[-1, 1]$.

Una reducción de rango sencilla es $y = x \ln 2$, pero resulta tímida pues el factor de reducción es apenas .6, por lo que el intervalo de convergencia no cambia gran cosa. Lo que se hace es calcular

$$e^x = 2^y$$

y se cambia el problema a encontrar una buena aproximación de 2^y . La forma de programarla podría ser un simple `return pow(2, x*M_LOG2E);`. Claro, puede aplicarse de nuevo la separación de y en sus partes entera y_n y fraccionaria y_f como en `exp_v3`, haciendo

$$e^x = 2^{y_n} 2^{y_f} \quad (7.16)$$

y aproximando 2^{y_f} con un polinomio de Taylor de orden fijo evaluado con el método de Horner y usando `fma` de ser posible.

Debe ser claro que trabajar con base 2 en vez de e tiene sus ventajas en el caso de los formatos con $\beta = 2$, lo que será más obvio en la sección 7.5. Para los formatos decimales puede usarse el equivalente en base 10, es decir, transformar $e^x = 10^y$ con las operaciones correspondientes.

Esta sí es una reducción de rango, de tipo multiplicativo, con $C = 2$.

Otra idea es determinar el entero k tal que $y = x/k$ puede calcularse con exactitud y $|x/k| < 1$, basta que k sea una potencia de 2 para no perder exactitud pues sólo se modifica el exponente de x . Entonces se puede aproximar e^y en el intervalo de convergencia con el polinomio (7.14) truncado. La reconstrucción se hace con

$$e^x = (e^y)^k \quad (7.17)$$

y es idea de Hull y Abrham¹¹, presentada en [176] y la califican de “razonablemente eficiente”. El valor de k puede calcularse como la menor potencia de 2 mayor que x .

La idea se ejemplifica en el código 7.5, que aún puede optimizarse más si se considera que la división entre k (línea 8) sólo requiere modificar el campo del exponente en la codificación flotante. Las funciones `frexp` y `ldexp` pertenecen a la norma (véase la tabla 3.8). La primera extrae el exponente e y la mantisa m del formato flotante de x y la segunda procede a la inversa, calculando $x = m \times 2^e$.

¹¹Se presenta sólo la idea, pues la versión original utiliza base 10 en una computadora VAX.

Código 7.5: La función exponencial con la técnica de Hull. Se han suprimido las comprobaciones de los límites.

```

1  double exp_Hull(double x)
2  {
3      double m, f, k;
4      int e;

6      m = frexp(x, &e); /* Extrae el exponente */
7      k = ldexp(1.0, e); /* Calcula 1*2^e */
8      y = x/k;

10     return Pot(exp_v2(y), k);
11 }

```

Incluso se puede aumentar el exponente a $e + 1$ y reducir el intervalo de convergencia a la mitad, con lo que la función `exp_v2` utilizará menos términos.

Otra posibilidad es la de Brown y Feldman publicada en [53]¹², donde el argumento reducido y queda en el intervalo $[0, \ln 2] \approx [0, .7]$. Entre más pequeño sea el intervalo de convergencia se gana en exactitud. Lo que se hace es calcular $q = \lfloor x/\ln 2 \rfloor$ y luego $y = x - q \times \ln 2$. La reconstrucción es

$$e^x = 2^q \times e^y. \quad (7.18)$$

Como el exponente de e^x es $q + 1$, puede verificarse que no ocurra overflow antes de calcular (7.18) comparando con el exponente máximo e_{max} . En ese caso, se regresa ∞ . El valor de 2^q debe calcularse con la función `ldexp`.

7.4.2. Reducciones más sofisticadas

En este caso se trata no sólo de reducciones de rango que consiguen un intervalo de convergencia muy pequeño, sino que se apoyan con tablas precalculadas. Los 3 métodos siguientes se publicaron con sólo dos años de diferencia. Debido a la gran cantidad de detalles que acompañan a los algoritmos, solo se presentan de manera general.

En [361], de 1989, Tang reduce x al intervalo $[-\frac{\ln 2}{64}, \frac{\ln 2}{64}] \approx [-.02, .02]$. La idea es encontrar enteros m y j y valores r_1 y r_2 tales que

$$x = (32m + j)\left(\frac{\ln 2}{32}\right) + (r_1 + r_2)$$

donde $|r_1 + r_2| \leq \frac{\ln 2}{64}$. Se aproxima $z = e^{r_1 + r_2}$ mediante un polinomio minimax y se reconstruye con

$$e^x = 2^m(2^{j/32} + 2^{j/32}z).$$

¹²En este artículo realmente proponen parámetros numéricos como exponente y fracción independientes del lenguaje y algunos términos genéricos específicos para Fortran 77. Algunos de estos ya los comentaban desde antes Forsythe, Moler e incluso Wilkinson. La versión de función exponencial es sólo un ejemplo de uso de la propuesta.

Tang también especifica cómo manejar todas las posibilidades de argumento x , considerando las posibles excepciones y explica cómo optimizar el uso de las tablas exactas.

En 1991, Gal y Bachelis publicaron [16] donde presentan los excelentes resultados del centro científico israelí de IBM. Como el anterior, se basa en el uso de tablas exactas con 64 bits de mantisa de precisión y un polinomio minimax de orden 4.

Primero aplican una técnica de reducción de rango similar a (7.16) y construyen el valor $z = y - i/512 - \epsilon_i$, con $-177 \leq i \leq 177$, por lo que ahora

$$e^x = 2^{y_n} \times f_i \times e^z$$

y el último factor, e^z , se aproxima con un polinomio minimax en el intervalo

$$[-2^{-10}, 2^{-10}] \approx [-.001, .001].$$

Las f_i se almacenan en una tabla. Como el polinomio tiene un coeficiente 1, la aproximación es realmente $2^{y_n} \times (f_i + f_i \times pz)$.

Para garantizar la exactitud de la reducción $y = x - y_n \times \ln 2$ se representa $\ln 2 = L_0 + L_1$ como la suma de dos números, donde el sumando mayor L_0 tiene sus últimos 10 bits en cero, para garantizar que el producto $L_0 \times n$ sea exacto para toda $-1024 \leq n \leq 1024$. El cálculo de y queda

$$n = \text{INT}(x / \ln 2)$$

$$y = x - n \times L_0$$

y la corrección $d = n \times L_1$ se mantiene a parte. Con lo anterior, la reducción de rango queda

$$e^x = 2^{y_n} \times (f_i + f_i \times (p(z) - d - d \times p(z))).$$

El método de Gal consigue el último bit correctamente redondeado en el 99.8 % del intervalo $[-170, 170]$.

En ese mismo año, Abraham Ziv publicó una novedosa técnica que logra calcular $\text{fl}(e^x) = e^x(1 + \delta)$ con $\delta \leq \mu$. Utiliza dos etapas, esperando no necesitar de la segunda, que es en la que sucesivamente se aumenta la precisión iterativamente hasta lograr un redondeo seguro. La primera etapa debe ser lo más rápida posible y pretende cubrir la mayoría de los casos.

En la primera etapa utiliza las reducciones de rango sucesivas

$$y_1 = x - n \ln 2$$

$$y_2 = y_1 - u_i^1$$

$$y_3 = y_2 - u_j^2$$

donde el entero n se selecciona de tal forma que $-\frac{1}{2} \ln 2 < y_1 < \frac{1}{2} \ln 2$. Usa un total de 6 tablas, en algunas de ellas los valores se almacenan como dos variables de doble precisión. La evaluación queda

$$e^x = z + z \times (e^{x^3} - 1)$$

A la Dekker.

donde $z = 2^n \times y'_1 \times y'_2$ y las y son valores de las tablas. La función $e^{x^3} - 1$ se aproxima con un polinomio en un subintervalo de $[-\frac{1}{2} \ln 2, \frac{1}{2} \ln 2]$. Esto concluye la primera etapa.

Si la precisión no es suficiente para redondear con confianza, se pasa a la segunda etapa, que utiliza precisión extra hasta conseguir el redondeo correcto. Normalmente esta etapa se apoya en polinomios de Taylor. El peor caso de la función exponencial tiene 104 dígitos 1 juntos, por lo que se sabe que el proceso tiene fin.

Para ello se requiere disponer de rutinas aritméticas de precisión múltiple donde se puede controlar la precisión. La técnica de Ziv puede aplicarse cuando son conocidos los peores casos del TMD para una función en el intervalo de interés.

En 2005, Dinechin, Ershov y Gast publicaron [87] donde realizan modificaciones a la idea de Ziv, que lo hacen más eficiente. Entre otras cosas, la segunda etapa se realiza en un sólo cálculo suficientemente preciso y no iterativamente hasta alcanzar la precisión requerida. Esto es posible ya que son conocidos los peores casos de muchas funciones elementales.

7.5. Uso básico de tablas

La tabla con valores precalculados para la función depende directamente de la reducción de rango seleccionada pues es del intervalo de convergencia de donde deben seleccionarse, en cuanto a cantidad, precisión y distribución.

Por ejemplo, si se utiliza la reducción $e^x = 2^y$ con $y = x \log e$ y se separa $y = y_n + y_f$ en sus partes entera y fraccionaria, 2^{y_n} no tiene problemas de cálculo al ser potencia entera de dos (se usa `ldexp`).

Para calcular 2^{y_f} se puede usar una tabla que contenga los valores $2^{i/32}$ con i desde 0 hasta 31. Entonces $i = \lfloor 32 \times f \rfloor$ es el índice en la tabla al valor $2^{i/32}$ que corresponde a y_f . Se separa el cálculo como

$$2^{y_f} = 2^{i/32} 2^{y-i/32}$$

y el primer factor está precalculado en la tabla. Consiste de las potencias desde $2^{.00000}$, $2^{.00001}$, hasta $2^{.11111}$, y los exponentes se han escrito en base dos. Por ello la forma de calcular i , pues multiplicar por 32 desplaza el punto 5 posiciones a la derecha. Al truncar la parte fraccionaria se obtiene el índice de la tabla.

Esta idea se ejemplifica en el código 7.6. Se han utilizado decimales extra en la tabla para permitir que el compilador haga la conversión a binario con eficiencia. Debe estarse seguro de que el compilador no comete errores al hacer la conversión. Si se desea mayor seguridad, es preferible calcular las representaciones exactas y escribirlas en hexadecimal o como enteros con la técnica de decodificación de la página 132.

Código 7.6: La función exponencial usando una tabla uniformemente distribuida.

```

1  double exp_v8(double x)
2  {
3      int i;
4      double n, f;
5      static const double Tbl[] = { 1, /* = 2^(-0/32) */
6          1.0218971486541166782, 1.0442737824274138403,
7          1.0671404006768236618, 1.0905077326652576592,
8          1.1143867425958925363, 1.1387886347566916537,
9          1.1637248587775775138, 1.1892071150027210667,
10         1.2152473599804688781, 1.2418578120734840486,
11         1.2690509571917332226, 1.2968395546510096659,
12         1.3252366431597412946, 1.3542555469368927283,
13         1.3839098819638319549, 1.4142135623730950488,
14         1.4451808069770466200, 1.4768261459394993114,
15         1.5091644275934227398, 1.5422108254079408236,
16         1.5759808451078864865, 1.6104903319492543082,
17         1.6457554781539648445, 1.6817928305074290861,
18         1.7186192981224779156, 1.7562521603732994831,
19         1.7947090750031071864, 1.8340080864093424635,
20         1.8741676341102999013, 1.9152065613971472939,
21         1.9571441241754002690 /* = 2^(-31/32) */};

23     if ( x<0 )
24         return 1/exp_v8(-x);

26     f = modf(x*M_LOG2E, &n);
27     i = floor(f*32);
28     f -= i/32.0;

30     return ( ldexp(1.0,(int)n) * Tbl[i] ) * p(f);
31 }

```

La última línea del código 7.6 usa al polinomio p definido como

$$\begin{aligned}
 p(f) = 1 + f \ln 2 + \frac{f^2}{2} \ln^2 2 + \frac{f^3}{6} \ln^3 2 + \frac{f^4}{24} \ln^4 2 + \\
 + \frac{f^5}{120} \ln^5 2 + \frac{f^6}{720} \ln^6 2 + \frac{f^7}{5040} \ln^7 2
 \end{aligned} \tag{7.19}$$

que aproxima la función 2^f en el intervalo $[-.03125, .03125]$ con un error máximo de 1.5×10^{-18} .

La constante `M_LOG2E` se define en `math.h`. Si esa tabla de valores se utilizará con otra función elemental, puede dejarse como global en vez de ser un dato local. La razón de declarar `static` al arreglo es mantenerlo siempre en memoria, lo que se puede cambiar dependiendo de las restricciones.

El polinomio (7.19) no es la única posibilidad, también puede utilizarse una

aproximación como la de Padé $P_{4,4}$ definida como

$$P_{4,4}(x) = \frac{x^4 + 20x^3 + 180x^2 + 840x + 1680}{x^4 - 20x^3 + 180x^2 - 840x + 1680} \quad (7.20)$$

que equivale a un polinomio de Taylor de grado 8, con un error máximo en el intervalo $[-2^{-5}, 2^{-5}] = [-.03125, .03125]$ de 1.2×10^{-21} .

La aproximación con $P_{3,3}$ se queda corta pues $|P_{3,3}(x) - e^x| > 2\mu$, por lo que no es útil para doble precisión. Si se insiste en usar $P_{3,3}$, habría que usar una tabla con 64 valores, ya que en el intervalo reducido $[-2^{-6}, 2^{-6}] = [-.015625, .015625]$ sí se logra un error máximo de 2.2×10^{-18} . $P_{3,3}$ se evalúa más rápido que $P_{4,4}$ y con mayor estabilidad.

Cuando las aproximaciones de Padé tienen el mismo exponente en el denominador y numerador, los coeficientes se diferencian sólo por el signo de algunos de ellos, por lo que se puede hacer la evaluación de una manera muy óptima.

* * *

A partir de los anteriores ejemplos de aproximación puede verse que hay un compromiso permanente entre la reducción de rango, el cálculo previo de constantes, el tamaño de la tabla y el grado del polinomio o función aproximante seleccionada.

7.6. Probando la rutina

Como ya se ha dicho, probar que una rutina es correcta cuesta más que programarla. Lo ideal es comparar contra datos exactos para comprobar la exactitud, por lo que habría que obtener valores correctamente redondeados de e^x en cierto intervalo de interés.

Normalmente se puede utilizar software que usa aritmética de precisión arbitraria como Mathematica, Matlab o Maple, o bien utilizar rutinas de alguna librería que maneje arbitrariamente muchos dígitos.

Como este esquema no es práctico y solo garantiza precisión en los valores evaluados. También se recomienda hacer evaluaciones con precisión extra (extendida o cuádruple) y comparar, pero el esquema falla si se está evaluando una rutina en la precisión más alta disponible.

7.6.1. Monotonicidad

Como ya se comentó, la monotonicidad es una propiedad de gran importancia. Según las observaciones de la página 313, es de esperarse que la monotonicidad de no se cumpla considerando que se trabaja con los números de $\mathbb{F}$. La inestabilidad de los polinomios, que son los que dan la aproximación final, debe tenerse bajo control pues se requiere un resultado correctamente redondeado.

Parte III

Apéndices

Apéndice A

Conceptos matemáticos

Este texto es autocontenido en cuanto a la aritmética de punto flotante, pero sí se requiere de algunos conocimientos previos tanto de programación como de matemáticas.

Los siguientes son sólo síntesis de temas que debieran ser conocidos por cursos anteriores y algunos conceptos básicos nuevos, con unas cuantas anotaciones relacionadas con los detalles de su programación. Se incluyen aquí como referencia rápida para estudiantes de cursos de programación y similares.

Se espera que el lector haya cursado asignaturas como álgebra, cálculo, programación y esté familiarizado con temas como números complejos, matrices, apuntadores y otros similares. Más conceptos se definirán cuando se necesiten. El nivel de detalle de algunos conceptos (como el de números imaginarios) ha sido por solicitud explícita recurrente.

Al cliente lo que pida.

A.1. Conceptos numéricos básicos

Al resolver algunos problemas matemáticos, comunmente no hay fórmulas aritméticas que den un resultado exacto, sino que van siendo aproximados poco a poco. Este proceso se detiene cuando ya no hay mejoras evidentes en la aproximación de la solución, aunque no tengamos un resultado exacto. Lo que interesa es un resultado suficientemente bueno, es decir, se tolera cierto error. Por ejemplo si se estuviera calculando $\sqrt{2} = 1.414\,213\,562\,373\,095\dots$ se podrían tener aproximaciones sucesivas como las de la tabla A.1.

En la tabla se ve que cada nueva iteración se tiene una mejor aproximación del resultado. De aquí que es necesario poder medir la disminución paulatina del error en cada etapa y así saber cuándo es aceptable detenerse. El momento en que se decida que la aproximación es suficientemente buena se está dentro de lo que se conoce como tolerancia, es decir, el margen de error permitido.

Iteración	Aproximación
1	1.41
2	1.41421
3	1.41421356
4	1.414213562373
$\vdots$	$\vdots$

Tabla A.1: La precisión va aumentando al mismo tiempo que la exactitud (estos conceptos se definen en esta sección), pero no siempre ocurre así. Se han suprimido los dígitos incorrectos.

En la práctica, la tolerancia cambia de una aplicación a otra. De cualquier manera se trata de que el resultado no es exacto, pues tiene un error. Por otra parte, es deseable que los programas bien diseñados den resultados que estén acompañados de la certeza de los cálculos, así el usuario sabrá cuánto confiar.

A.1.1. Error absoluto y relativo

La manera de medir el error no es única, por lo que es necesario decidir con cuidado cuándo utilizar cada una.

Error absoluto

Si se denota con x^* el número que se quiere aproximar, es decir, la solución exacta de un problema, y si x es la aproximación que se tiene en un momento dado, entonces a la cantidad

$$E_{abs} = |x - x^*| \quad (\text{A.1})$$

se le conoce como *error absoluto* y es simplemente la diferencia numérica entre la aproximación y el número exacto. Esta es la primera magnitud que se calcula al evaluar aproximaciones.

No se puede decir que el error sea el mismo.

Si bien esta definición es de lo más simple e intuitivo, tiene un problema. Si se aproxima el 100 con el 99, el error absoluto es 1. Si se aproxima el 1000 con el 999, el error absoluto sigue siendo 1. Esto significa que el error absoluto no indica la magnitud del problema que se tiene al aproximar y por ello debe ser utilizado con cuidado.

Si bien resulta muy natural medir el error de esta manera, el resultado puede ser poco representativo. Por ejemplo, un error de 1mm puede ser despreciable si se está construyendo una carretera, pero es inaceptable si se está diseñando un circuito integrado. Muchas situaciones similares están al alcance de la mano.

Error relativo

Resulta más representativo o significativo si se calcula la proporción del error, con respecto al resultado deseado, de la siguiente manera. Calculando

$$E_{rel} = \frac{|x - x^*|}{|x|} \quad (A.2)$$

se obtiene una medida de la relación del error considerando la magnitud de las cantidades. Este número se conoce como *error relativo*. Si se multiplica por 100 al error relativo tenemos el concepto de *porcentaje de error*.

Si se aproxima el 100 con el 99, el error relativo es $1/99 = .01$ o 1 %. Si se aproxima el 1000 con el 999, el error relativo es $1/999 = .001$ o .1 %. Entonces, el error relativo va desde 0 para una aproximación exacta a 1 en el caso de una pésima aproximación donde x^* se aproxima con $2x^*$ (error del 100 %).

Aquí ya se puede hablar con mayor claridad.

En un caso más concreto, si se aproxima π con 3.1416, el error relativo es $E_{rel} = \frac{|\pi - 3.1416|}{\pi} \approx 2.338434 \times 10^{-6}$, lo que significa un error aproximado de 0.000002 %. El error absoluto es $E_{abs} \approx 7.346410 \times 10^{-6}$.

Consideraciones de programación

Si estos conceptos van a utilizarse en un programa para computadora, debe notarse que, tanto en las ecuaciones (A.1) y (A.2), se hace uso de una cantidad no conocida que es x^* . Esto no debe confundir: si $x_1, x_2, \dots$ son las aproximaciones sucesivas, el error en cada paso puede calcularse tomando las dos aproximaciones más recientes.

Si en la iteración $i + 1$, las aproximaciones más recientes son los valores x_i y x_{i+1} , los errores absoluto y relativo se pueden calcular como

$$E_{abs} = |x_i - x_{i+1}| \quad \text{y} \quad E_{rel} = \frac{|x_i - x_{i+1}|}{|x_{i+1}|} \quad (A.3)$$

respectivamente. También es posible sustituir por x_i el denominador del error relativo, pero interesa E_{rel} para la variable más reciente, que es la que tiene más posibilidades de usarse como resultado. La evaluación de estas cantidades merece atención.

El mayor inconveniente de cualquiera de los dos errores es que se necesita calcular la diferencia entre dos números que pueden ser muy cercanos. Esto se detallará cuando se revise la pérdida de precisión y la llamada, tal vez exageradamente, cancelación catastrófica en la sección 1.4.2.

Por otra parte, siempre que en una expresión aritmética aparece un cociente, es necesario verificar que el denominador no sea cero o muy pequeño en valor absoluto para evitarse problemas. En ese caso, utilizar el error absoluto es más seguro si la aproximación ocurre en una vecindad de 0.

En ocasiones, es útil el criterio combinado

No es raro que la mayor parte de los casos sólo un criterio se utilice.

$$\varepsilon_1 |x| + \varepsilon_2 < \delta \quad (\text{A.4})$$

donde ε_1 es el error relativo, ε_2 es el absoluto y δ la tolerancia aceptable de fallo, lo que es admisible para aceptar un resultado como suficientemente aproximado. Si se desconoce el orden de magnitud de x , debe tenerse cuidado al determinar ε_1 .

Una observación importante es que en muchos cálculos, donde las magnitudes son enormes o muy pequeñas, el error relativo siempre queda en la misma escala. De hecho, si se calcula el error relativo entre kx_i y kx_{i+1} , el error relativo se mantiene igual, ya que

$$\frac{|kx_i - kx_{i+1}|}{|kx_{i+1}|} = \frac{k|x_i - x_{i+1}|}{k|x_{i+1}|} \quad (\text{A.5})$$

lo que queda en la misma expresión de (A.3).

Además del error relativo, otra manera más intuitiva de medir lo bien que una cantidad x aproxima a x^* es ver cuántas *cifras significativas* tienen en común. De manera intuitiva, se dice que 3.141592 tiene 7 dígitos en común con π . Formalmente se define que x coincide con x^* en p cifras significativas si $|x - x^*|$ es menor que media unidad (.5) de la $(p + 1)$ -ésima cifra de x .

Ejemplo 51. *Por ejemplo, .55733 y .55734 coinciden en 4 cifras pues la diferencia es de 1 en el último dígito, menos de la mitad que la mayor diferencia posible¹, mientras que .55733 y .55739 coinciden en menos de 4 cifras, pues difieren en el último dígito por 6.*

Algunos autores calculan las cifras que tienen en común dos números reales como

$$C_{a,b} = \log_{10} \left| \frac{a+b}{2(a-b)} \right| \quad (\text{A.6})$$

como una aproximación para el orden de magnitud de la diferencia y $C_{a,a} = \infty$.

Ejemplo 52. *Retomando los datos numéricos del ejemplo 51 se aplica la fórmula (A.6) y se obtiene:*

$$\log_{10} \left| \frac{.55733 + .55734}{2(.55733 - .55734)} \right| \approx 4.74$$

mientras que

$$\log_{10} \left| \frac{.55733 + .55739}{2(.55733 - .55739)} \right| \approx 3.96$$

que son resultados que coinciden con la idea intuitiva.

Si los números no están en base 10, sólo se necesita cambiar la base del logaritmo por la del sistema numérico empleado.

¹La mayor diferencia posible es $10 - 1 = 9$: 1 menos que la base.

Una asociación precisa entre el concepto de error relativo y la cantidad de cifras significativas correctas la presenta Higham en [168]. De cualquier forma, el concepto de error relativo es por sí solo suficiente como medida de lo correcta que es una aproximación y sólo se usará el error absoluto cerca del 0.

A.1.2. Precisión y exactitud

Los conceptos de error absoluto y relativo están íntimamente relacionados a otras formas de medir lo eficaz de una aproximación, como la precisión y la exactitud.

Se llama *precisión* al hecho de obtener casi los mismos resultados a partir de condiciones muy similares. Se le llama *exactitud* a la posibilidad de conseguir una buena aproximación de la solución que se desea. Ambos conceptos son fáciles de formalizar.

Sea y^* el valor que interesa aproximar y que es el resultado exacto de aplicar el método (algoritmo, función, programa o como se le llame) f al valor x^* . El método f recibe como entrada un valor inicial cercano a x^* . Se dispone de una colección de tales valores, representados por $\{x_i\}_{i=1}^n = \{x_1, x_2, \dots, x_n\}$.

Cada evaluación $y_i = f(x_i)$ será exacta en la medida que el error $|y^* - f(x_i)|$ es pequeño,² lo que significa que se puede medir para cada valor x_i . En caso de un conjunto de evaluaciones, se dirá que es exacto si su media aritmética $\bar{y}_i$ es exacta, es decir, $|y^* - \bar{y}_i|$ es un valor pequeño.

El conjunto de evaluaciones $\{y_i\}_{i=1}^n$ es preciso si los valores están poco dispersos, es decir, poco alejados entre sí. Otra forma de medir esto es con el estadístico desviación estándar $\sigma = \sqrt{\sum (\bar{y}_i - y_i)^2}$, que es una medida común de evaluación de la dispersión. Consultar algún libro de estadística para más detalles.

Las *cifras significativas* son aquellas que impactan en la precisión de una cantidad.

La figura A.1 se muestran los puntos x_i que son mapeados a los y_i con precisión por f pero no con exactitud. En este sentido, exactitud sí implica precisión, pero no al revés, como en la gráfica. La región sombreada es el mapeo que hace f sobre el conjunto de valores. Nótese que se ha utilizado el error absoluto. Es mas sencillo entender el concepto de esta manera.

Un gráfico vale más que mil palabras.

Otra manera de definir los mismos conceptos es pensar que la precisión es una medida de la cercanía de la representación interna que la computadora hace de un número, mientras que la exactitud mide la cercanía del procedimiento de cálculo. Si los cálculos de conversión son exactos, la precisión se preserva.

Ejemplo 53. *Un usuario, pretende calcular el área de un círculo. Aunque el radio sea exacto (un entero, por ejemplo), se necesita usar la constante π , que*

²Por *pequeño* se hace referencia la idea intuitiva de una cantidad cuya magnitud es mucho menor en proporción a los valores que interesan.

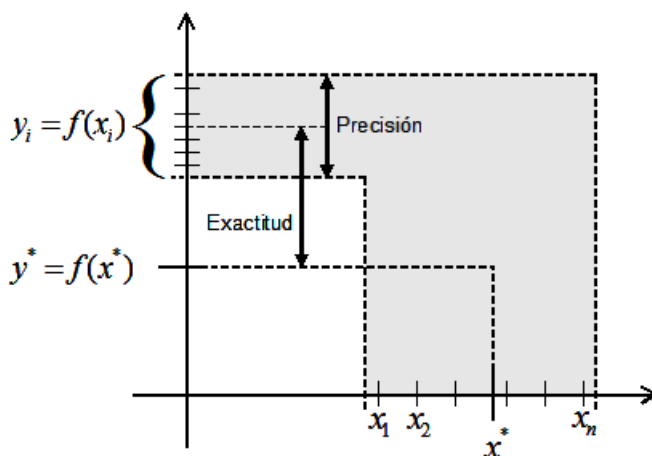


Figura A.1: Precisión y exactitud.

la computadora no puede almacenar con precisión infinita. Por ello, el programador escribe una aproximación truncada como $\pi^* = 3.141592653589793238$, para usar más de 15 cifras significativas. Aquí hay un **error de exactitud** en la representación de π , de la que es conciente el programador.

Cada compilador utiliza su propio algoritmo de conversión, aunque es de esperarse que sean igual de exactos.

Esta aproximación π^* es convertida por la computadora (por el compilador, de hecho) a un formato interno en binario, con lo cual se hace una conversión de base que no siempre puede llevarse a cabo sin pérdida de cifras significativas, por lo que el programa almacena una aproximación de π^* . Ahora tenemos un **error de precisión**, normalmente inadvertido por el programador promedio. Numéricamente ocurre lo siguiente:

$$\begin{aligned}\pi &= 3.141592653589793238462643383279502884197... \\ \pi^* &= 3.141592653589793238 \\ \pi - \pi^* &= 4.62643383279502884197... \times 10^{-19} \quad \text{Error de exactitud con } \pi\end{aligned}$$

En formato interno (en base dos) es

$$\begin{aligned}\pi^* &= 1.1001001000011111101101010100010001000010110100011000 \times 2^1 \\ &= 3.141592653589793 \quad \text{Error de precisión con } \pi^*\end{aligned}$$

Hay ejemplos de este tipo para casi todo problema que se resuelve con computadora. En ocasiones, ambos términos son intercambiables, pero su diferencia es útil en muchas situaciones.

A.1.3. Estabilidad y condicionamiento

Para un problema cualquiera $\mathcal{P}$ es posible determinar varios métodos o algoritmos que lo resuelvan o aproximen su solución. Algunos algoritmos serán

eficientes y otros bastante malos y es necesario poder tener una medida para ello.

Por otra parte, si el problema en sí es un problema de naturaleza muy complicada, aún el mejor algoritmo tendrá problemas y se tendrá una limitación que depende del problema mismo. A continuación se aborda cada uno de estos casos por separado.

Estabilidad

El concepto de *estabilidad* se refiere al posible crecimiento de los errores que se van produciendo durante las operaciones. Un algoritmo es *estable* si los errores no crecen considerablemente y es *inestable* cuando en etapas posteriores, los errores son muy significativos³.

Más formalmente, si en la iteración i se tiene el resultado $x_i + \delta_i$ con δ_i el error en esa iteración y a partir de estos valores se produce el resultado $x_{i+1} + \delta_{i+1}$, el algoritmo será estable si la diferencia de los errores de la primera a la última iteración n es poca, es decir, $|\delta_n - \delta_1|$ es pequeño. Pero si $\delta_n \gg \delta_1$ entonces el algoritmo es inestable.

Dos ejemplos famosos de inestabilidad Los siguientes son dos ejemplos de procesos inestables, que con frecuencia aparecen en libros de análisis numérico.

Ejemplo 54. *Primero considérese la sucesión recursiva (A.7), matemáticamente equivalente a $\frac{1}{3^n}$:*

$$\lim_{n \rightarrow \infty} \frac{1}{3^n} = 0$$

$$\begin{cases} x_0 = 1, & x_1 = \frac{1}{3} \\ x_{n+1} = \frac{13}{3}x_n - \frac{4}{3}x_{n-1}, & \text{para } n \geq 1. \end{cases} \quad (\text{A.7})$$

Aunque sabemos que su límite es cero, esta sucesión diverge y se puede analizar para ver la razón. Lo que ocurre es que el error incluido en el término n -ésimo $x_n + \delta_n$ es multiplicado por $\frac{13}{3}$, y cada iteración los errores se magnifican poco más de cuatro veces.

Resultados parciales calculados con precisión simple se muestran en la tabla A.2, calculados en C, con variables en precisión simple (tipo `float`).

Ejemplo 55. *Otro ejemplo lo tenemos en la sucesión y_n definida como*

$$\begin{aligned} y_n &= \int_0^1 x^n e^x dx \\ &= \frac{e}{n+1} - \int_0^1 e^x \frac{x^{n+1}}{n+1} dx \\ &= \frac{1}{n+1} (e - y_{n+1}). \end{aligned} \quad (\text{A.8})$$

³Qué tan significativos son, depende de lo crítico de las respuestas esperadas.

Iteración	Aproximación	Error absoluto
0	1.0000000	0
1	.33333333	0
2	0.1111111566424	4.47035e-008
3	0.0370372198522	1.82539e-007
4	0.0123464101925	7.31088e-007
5	0.0041181510314	2.92482e-006
6	0.0013834409182	1.16988e-005
7	0.0005040426040	4.67952e-005
8	0.0003395967360	0.000187181
9	0.00079952902160	0.000748724
10	0.0030118301510	0.0029949
11	0.0119852256029	0.0119796
12	0.0479202046990	0.0479183

Tabla A.2: Aproximaciones sucesivas de 3^{-n} con un método inestable y precisión simple. Con estas primeras 12 iteraciones se ve el problema. El resultado comienza a crecer a partir de $n = 9$, pero es incorrecto desde $n = 2$, donde el valor correcto deberían ser puros dígitos 1 después del punto. Para $n = 79$ los valores son incalculables.

Conforme el valor de n aumenta, la integral (el área bajo la curva) debería tender a cero, como dejan ver los gráficos de y_n para $n = 1, 2, 4, 8$ de la figura A.2.

Despejando el término y_{n+1} de la ecuación (A.8) se obtiene la fórmula recursiva

$$y_{n+1} = e - (n + 1)y_n. \quad (\text{A.9})$$

El valor del primer término se obtiene a partir de (A.8) haciendo $n = 0$ y es $y_0 = e - 1$. A partir de la figura A.2 debería ser obvio que la sucesión de valores tiende a cero ($y_n \rightarrow 0$). Sin embargo, esta sucesión también diverge al calcularla en computadora, debido a que el error h_n es multiplicado por n , como se ve en la ecuación (A.9). Es fácil ver que el error inicial es multiplicado por $n!$. Se deja al lector la programación.

Sven Hammarling presenta otra forma de calcular la sucesión (A.8) (página 345) en el libro de Einarsson [103], que corrige la inestabilidad.

Estos dos cálculos se dice que son *inestables* porque los errores producidos en una iteración son aumentados en iteraciones posteriores, degradando seriamente los resultados. Otros dos ejemplos interesantes y que igualmente fallan de manera dramática aparecen en el libro Handbook of Floating-Point Arithmetic ([282]), en la sección 1.3.2.

La forma de determinar si un cálculo es inestable debe ser por medio del error relativo. Es importante mencionar que no es una idea que el programador

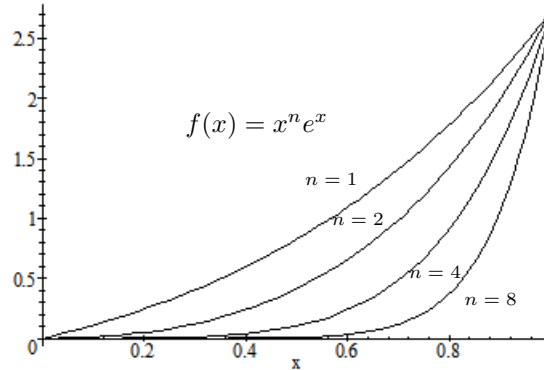


Figura A.2: El área bajo la curva $y_n(x)$ va disminuyendo en $[0, 1]$.

principiante promedio utilice empíricamente, por lo que normalmente se requiere algo de motivación con ejemplos como los anteriores.

Entre los diversos algoritmos para resolver un problema $\mathcal{P}$ pueden existir algunos más veloces o más exactos, así como también más o menos inestables. La estabilidad es una característica intrínseca del algoritmo empleado, no del problema que se está resolviendo.

Condicionamiento

Otro concepto relacionado es el de *condicionamiento*, que indica la proporción esperada de cambio en la salida de un algoritmo cuando se hacen pequeños cambios en la entrada. Si se evalúa $x + \delta$, con δ un valor pequeño, se obtiene $f(x) + \varepsilon$, y si ε también es pequeña (aproximadamente del mismo orden de magnitud que δ) se dice que f está *bien condicionado*, al menos en una vecindad de x . Se dice que está *mal condicionado* si $\delta \ll \varepsilon$.

Con la misma idea relativa de pequeño de la página 343.

A diferencia del caso de la estabilidad, el condicionamiento es una característica del problema, no del algoritmo. Si un problema cualquiera $\mathcal{P}$ está mal condicionado por naturaleza, por más eficiente que sea el algoritmo con que se intente resolverlo, debe tenerse cuidado con la exactitud de las respuestas. Por ejemplo, no se espera gran cosa de ningún algoritmo que se enfrente a la solución de un sistema de ecuaciones con la matriz de Hilbert porque se sabe que es una matriz sumamente mal condicionada, más adelante se presentan.

La variación en la salida de un algoritmo puede escribirse como

$$f(x+h) - f(x) \approx hf'(x)$$

lo que es una aproximación del error absoluto. De nuevo, el error relativo resulta más significativo, por lo que una buena forma de medir el *número de condición*

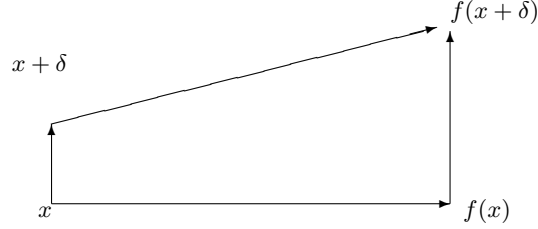


Figura A.3: Backward error (flecha vertical izquierda) y forward error (flecha vertical derecha). La diferencia de tamaño es puramente ilustrativa, en algunos problemas puede ser al revés.

de un problema es

$$\text{cond}(f) = \frac{f(x+h) - f(x)}{f(x)} \approx \frac{hf'(x)}{f(x)}.$$

Obsérvese que el número de condición depende del valor x en el que se evalúa. Esto significa que para ciertos valores el problema puede estar más mal condicionado que para otros.

Ejemplo 56. En el caso de polinomios, se acostumbra calcular el número de condición como

$$\text{cond}(p, x) = \frac{\sum_{i=0}^n |a_i| |x|^i}{|\sum_{i=0}^n a_i x^i|} = \frac{\bar{p}(x)}{|p(x)|} \quad (\text{A.10})$$

donde $p(x)$ es el valor aproximado. Si x es una raíz de p , entonces $\text{cond}(p, x) = \infty$ y se dice que x es una singularidad. Es obvio que si el polinomio no tiene cambios de signo, estará bien condicionado para todos los valores de x .

El número de condición es una forma de medir por adelantado la magnitud de los problemas numéricos que el programador tendrá. Un ejemplo bien conocido es el que presenta Rump en [325], que es retomado en [282] y analizado por diversos autores.

Otras dos definiciones importantes para los analistas numéricos son las siguientes. A δ (o la versión relativa $|\delta/x|$) se le llama *backward error*, mientras que a $|\varepsilon| = |y - f(x + \delta)|$ se le llama *forward error*. El análisis completo de un algoritmo requiere que se den cotas para estos dos errores, lo que significa analizar el error tanto en los datos de entrada como en la salida. La figura A.3 ilustra estas ideas.

En este caso, $f^*(x) = f(x + \delta)$ es procedimiento que se aplica en la realidad, que es el original f perturbado por razones externas (como el modo de redondeo o la precisión del hardware) o internas (como la estabilidad del algoritmo o el condicionamiento del problema).

Es bien conocida la regla que relaciona los dos errores con el número de condición:

$$\text{forward error} = \text{cond} \times \text{backward error} \quad (\text{A.11})$$

de donde se desprende que si el número de condición es alto, se requiere un backward error ínfimo para garantizar buena exactitud en la solución de un problema.

El backward error tiene la ventaja de interpretar los errores de redondeo como perturbaciones en los datos de entrada, es decir, responde la pregunta ¿qué valores próximos a x tienen como solución a $f(x)$? Esto es conocido también como la sensibilidad del problema. Con los polinomios de Taylor se presentará un ejemplo.

¿Qué problema se resolvió realmente?

Backward y forward son combinados en lo que Larson llama análisis-B (una medida alterna de estabilidad) en [235], que permite incluso comparar eficientemente dos algoritmos que resuelven el mismo problema.

En análisis-B requiere resolver un problema minimax.

A.2. Notación asintótica

Al comparar algoritmos, generalmente se mide la cantidad de recursos que utilizan. Los dos recursos básicos son el tiempo (de procesador) y el espacio (la memoria). A esto se le conoce como *complejidad algorítmica* y generalmente se expresa en términos de n , que puede ser la cantidad de datos, el tamaño de la matriz, el grado del polinomio o la cantidad de bits, entre otras.

Siempre es un número natural.

Casi siempre interesa el comportamiento cuando el tamaño del problema aumenta arbitrariamente, por lo que interesa el comportamiento asintótico y en particular el peor caso. Para ello, se define la notación O , llamada notación- O u O grande, que es el conjunto de funciones $g(n)$ que acotan superiormente a una función dada $f(n)$. Por ejemplo, x^2 está acotada superiormente por x^3 , a partir de cierta x_0 que en este caso es 1. Formalmente,

$$\begin{aligned} O(g(n)) = \{f(n) \text{ tales que existen constantes } c_1 \text{ y } n_0 \text{ que cumplen con} \\ 0 \leq f(n) \leq cg(n) \text{ para toda } n \geq n_0\}. \end{aligned} \quad (\text{A.12})$$

Además de determinar una función que acota a otra, se utiliza para agrupar términos. Por ejemplo,

$$f(n) < 4n^2 + O(n)$$

significa que f tiene comportamiento cuadrático más términos de orden lineal cuya expresión es poco relevante. En ese sentido, decir que un algoritmo tiene complejidad temporal $O(n^2)$ significa que el tiempo de ejecución aumenta cuadráticamente conforme n aumenta, que es comunmente la cantidad de datos. Similarmente, si la complejidad espacial es $O(n \log(n))$ significa que para procesar n datos se requiere un espacio equivalente al de $n \log(n)$ datos.

Una operación atómica como una suma o una asignación de un valor a una variable tienen complejidad $O(1)$, que es tiempo constante. Es importante mencionar que la notación asintótica puede usarse de manera general, suponiendo que cada paso de un algoritmo cuenta igual en términos del tiempo, pero también puede especificarse que algunas operaciones son realmente más costosas. Por ejemplo, sumar o restar cuesta mucho menos que multiplicar y dividir es aún más costoso.

El análisis completo de un algoritmo puede consistir en determinar la complejidad mediante una expresión como $O(3n_1 + 5n_2)$, donde cada n pesa distinto. Tal análisis debe incluir el peor caso posible, el mejor caso y el caso medio con su desviación estándar. Más comunmente se simplifica considerando todas las operaciones al mismo costo a menos que se especifique lo contrario.

Por lo anterior, decir que un algoritmo es $O(4n^3 + n^2 + 2n \log n)$ es útil para conocer el detalle del tiempo, en términos de la cantidad de operaciones, pero es equivalente a decir $O(n^3)$ pues el término cúbico domina a los otros. Formalmente, $4n^3 + n^2 + 2n \log n \in O(n^3)$ pues es una función que pertenece a ese conjunto.

En ocasiones, si $|h| \ll 1$ es un término de error, expresiones como

$$f(x) < 4x^3 - \frac{hx^3}{3} + O(h^2)$$

significa que hay términos que incluyen errores de orden cuadrático y superior que no influyen en la cota. Esta es la manera como se emplea en análisis numérico.

Una excelente exposición de este tema se encuentra en el libro *Introduction to Algorithms*⁴ ([79]), donde además de la notación O se presentan o , ω , Ω y Θ , para acotar por debajo o por ambos lados.

A.3. Números complejos

Aunque muchos problemas se resuelven utilizando sólo números reales ($\mathbb{R}$) e incluso enteros ($\mathbb{Z}$), muchos problemas, tanto teóricos y prácticos se resuelven sólo cuando el conjunto de números empleado se extiende. Se muestra a continuación la más empleada.

Una ecuación que tiene la forma

$$x^2 - r = 0 \tag{A.13}$$

donde r es un número real positivo tiene solución muy sencilla. De hecho tiene dos soluciones: $+\sqrt{r}$ y $-\sqrt{r}$. Esto es así de simple en el caso de que el valor r sea positivo, pero si es negativo, el resultado de la raíz cuadrada queda indefinido.

⁴Todo libro serio sobre algoritmos o estructuras de datos define estos conceptos.

La falta de solución de un problema tan simple motiva una idea sencilla y con enormes alcances.

Para poder resolver este problema se define un número especial que cumple con que al multiplicarse por sí mismo se obtiene -1 . A este número se le llama *imaginario* y se le denota por i . Así, $i^2 = -1$.

Con esa definición es posible resolver la ecuación (A.13) para valores de r negativos.

Ejemplo 57. Sea la ecuación $x^2 + 4 = 0$. Como no hay ningún número real tal que $x^2 = -4$, entonces se emplea el número imaginario i de la siguiente forma:

$$\begin{aligned} x &= \sqrt{-4} \\ &= \sqrt{-1 \times 4} \\ &= \sqrt{i^2 4} \\ &= i2 \end{aligned}$$

por lo que la soluciones son $x = +2i$ y $x = -2i$.

De esta manera, ahora se tiene un nuevo tipo de números que no son reales, sino imaginarios. Cuando números reales e imaginarios se suman o restan se forman los *números complejos*, que son de la forma $z = a + bi$, donde a y b son reales y z es el número complejo. A a se lo conoce como parte real y a b como parte imaginaria. El conjunto de los números complejos se denota por $\mathbb{C}$.

De esta manera, los números reales son números complejos con parte imaginaria igual a cero. Comunmente se les denota como par ordenado $z = a + bi = (a, b)$. $\mathbb{R} \subset \mathbb{C}$

De manera natural son representados en un plano cartesiano, llamado ahora *plano complejo*, como pares ordenados (a, b) y los ejes son llamados eje real y eje imaginario. El plano complejo se denota por $\mathbb{C}^2$.

A.3.1. Operaciones básicas

Sean $z_1 = a + bi$, $z_2 = c + di$ y $k \in \mathbb{R}$ un escalar. Las operaciones elementales con números complejos se definen de la siguiente manera:

- Suma (o resta)

$$(a, b) + (c, d) = (a + c, b + d)$$

- Multiplicación

$$(a, b)(c, d) = (ac - bd, ad + bc)$$

- Producto por escalar

$$k(a + b) = (ka, kb)$$

- División

$$\frac{(a, b)}{(c, d)} = \left(\frac{ac + bd}{c^2 + d^2}, \frac{bc - ad}{c^2 + d^2} \right)$$

A.3.2. Otras Propiedades y definiciones

Sean $z = a + bi$. Una cantidad importante de un número complejo es su magnitud, también llamada *módulo* (e incluso valor absoluto) y se define como

$$|z| = \sqrt{a^2 + b^2} \quad (\text{A.14})$$

y debe ser claro que es la distancia euclideana del origen al punto (a, b) .

Todo número complejo puede representarse en lo que se conoce como su *forma polar*. Si $r = |z|$ y $\tan(\theta) = b/a$ entonces

$$z = r(\cos \theta + i \sin \theta) \quad (\text{A.15})$$

Leonhard Euler (1707-1783), fue un notable matemático y físico suizo. Enorme cantidad de variadas contribuciones a la ciencia.

A partir de (A.15) se obtiene la fórmula de Euler

$$e^z = e^r(\cos \theta + i \sin \theta) \quad (\text{A.16})$$

de gran utilidad tanto en álgebra como en trigonometría. Una forma equivalente es $e^z = re^{i\theta}$. De esta se desprenden interesantes propiedades.

Como $e^{i\pi} = -1$.

Si $z_1 = re^{i\theta}$ y $z_2 = se^{i\phi}$, entonces se obtiene otras reglas para operaciones básicas.

$$z_1 z_2 = rse^{i(\theta+\phi)} \quad (\text{A.17})$$

$$\frac{z_1}{z_2} = \frac{r}{s}e^{i(\theta-\phi)} \quad (\text{A.18})$$

Se llama *conjugado* del número complejo $z = a + bi$ al número complejo que resulta de cambiar de signo la parte imaginaria, lo que se denota por $\bar{z} = a - bi$. Viendo el plano complejo $\mathbb{C}^2$, el conjugado complejo es la imagen reflejo del número original, con respecto al eje (de la parte) real.

Los números complejos o más genéricamente: los problemas con variables complejas tiene aplicaciones en muchos problemas de ingeniería y ciencia básica, donde son indispensables. Para más propiedades y ejemplos de aplicaciones se recomienda consultar bibliografía adecuada, ya sea álgebra o variable compleja. En este texto se emplearan en el capítulo de raíces de polinomios.

A.3.3. Consideraciones de programación

Los lenguajes de programación (ni los procesadores) típicos no poseen tipos nativos para variables complejas. El número imaginario i es una abstracción de gran éxito para resolver problemas, pero la computadora sólo reconoce valores reales, por lo que es necesario hacer las operaciones con detalle.

Lo que se puede considerar una deficiencia.

Lenguajes como C y sus derivados no poseen un tipo de datos nativo, pero sí se dispone de funcionalidad para aritmética compleja dentro de su biblioteca de funciones ISO, declaradas en el archivo de cabecera `complex.h`. La librería

Código A.1: Ejemplo de uso de números complejos en C++.

```

1  complex<double> z1;
2  z1 = 8;           // Asigna la parte real
3  z1 += 2.0;        // Altera la parte real
4  z = complex<double>(2,3); // z = 2+3i
5  z = z + z1;       // z = 12+3i

```

incluye todo tipo de funciones y operaciones habituales, todas trabajando en radianes. El tipo de dato `complex` está definido como una estructura con dos campos.

En C++ también se incluye una librería estándar para datos complejos, pero programadas de manera distinta, como clase template: `complex<T>`. Esto significa que las funciones se emplean con diversos tipos de datos, incluyendo los complejos. En C++ se tiene la comodidad de poder sobrecargar operadores, por lo que se obtiene gran ganancia en la abstracción del código. La sobrecarga ya está incluida en el estándar, por lo que códigos como A.1 son posibles.

Fortran sí posee un tipo de dato nativo `complex` que de manera interna funciona como la estructura en C. Otros lenguajes especializados como MatLab o Mathematica tiene facilidades similares que hace su uso transparente para el usuario.

A.4. Vectores y matrices

Se llama vector a una cantidad que agrupa n valores del mismo tipo, de manera ordenada. Se dice que ese vector tiene n dimensiones. Así, $(3, -2, 0, 5)$ es un vector de valores enteros con 4 dimensiones.

De esta manera, los vectores son elementos del producto cartesiano de un conjunto de números fijo. Todos los vectores de 2 valores reales son elementos del conjunto $\mathbb{R}^2$, mejor conocido como el plano cartesiano. Estas son las coordenadas en el plano. Por esto es común dar interpretación geométrica a algunos vectores, como su dirección con respecto al origen o su magnitud o longitud. Así, los vectores $(1, 1)$ y $(2, 2)$ apuntan en la misma dirección, pero tienen magnitudes distintas.

Ejemplo 58. *Los puntos en el espacio, con coordenadas (x, y, z) son vectores de 3 dimensiones cuyos valores tiene representación gráfica en $\mathbb{R}^3$. Cada punto del espacio representa un único vector. El vector $(-x, -y, -z)$ apunta en la misma dirección, pero en sentido contrario.*

En general, el vector $v \in \mathbb{R}^n$ se representa como

$$v = (x_1, x_2, \dots, x_n) \quad (\text{A.19})$$

y el valores x_i es la coordenada i -ésima del vector v . Se puede definir a un vector como fila o columna. Como fila es el caso de la representación anterior (A.19), mientras que como columna se escriben

$$v = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}. \quad (\text{A.20})$$

Es común que los vectores se escriban con tipo negrita como $\mathbf{v}$ o que se les coloque una flecha arriba como $\vec{v}$, lo que puede ser de utilidad para no confundirlos con cantidades escalares. El uso de paréntesis o corchetes es indistinto, más bien preferidos de una u otra forma en distintas áreas científicas. En álgebra lineal se prefieren los corchetes.

Con los vectores es posible describir los diversos valores asociados a una misma entidad (como estatura, peso y edad) o la misma característica de diversas entidades (las estaturas de un conjunto de personas).

Como extensión de la idea de vector, surge la idea de *matriz*, que es un arreglo rectangular de valores, o bien, un arreglo de vectores de la misma dimensión. Así, una matriz A de 2 filas y 3 columnas se representa con

$$\begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,1} & a_{2,2} & a_{2,3} \end{bmatrix} \quad (\text{A.21})$$

y se dice que $A \in \mathbb{R}^{2 \times 3}$, cada $a_{i,j} \in \mathbb{R}$. Otra forma de denotarlas es indicando $A = (a_{ij})_{m \times n}$. El uso más común de las matrices es la representación de sistemas de ecuaciones lineales, pero sus usos son muy variados, al igual que los vectores.

Tanto vectores como matrices pueden agrupar cualquier tipo de valor, incluyendo números complejos. De esta manera, $(3 - 2i, -4, -5i, 9 + i)$ es un vector en $\mathbb{C}^4$.

A.4.1. Operaciones y propiedades básicas

Entre los vectores n -dimensionales $a = (a_1, a_2, \dots, a_n)$ y $b = (b_1, b_2, \dots, b_n)$ se definen las siguientes operaciones:

- Suma

$$a + b = (a_1 + b_1, a_2 + b_2, \dots, a_n + b_n) \quad (\text{A.22})$$

- Producto por un escalar $k \in \mathbb{R}$

$$ka = (ka_1 + ka_2 + \dots + ka_n) \quad (\text{A.23})$$

- Producto punto

$$a \cdot b = (a_1b_1 + a_2b_2 + \dots + a_nb_n) \quad (\text{A.24})$$

El resultado de la suma es un nuevo vector, como el producto por un escalar, pero el producto punto obtiene un escalar.

Dadas las matrices cuadradas $A = (a_{ij})_{n \times n}$ y $B = (b_{ij})_{n \times n}$, se definen las operaciones

- Suma

$$A + B = (a_1 + b_1, a_2 + b_2, \dots, a_n + b_n) \quad (\text{A.25})$$

- Producto entre matrices

$$\begin{aligned} A \cdot B &= (c_{ij})_{n \times n} \\ c_{ij} &= \sum_{r=1}^n a_{ir} b_{rj} \end{aligned} \quad (\text{A.26})$$

El producto de un escalar por una matriz es como el del vector por la matriz: sólo se multiplica el escalar por cada valor a_{ij} . Es importante notar que si bien para vectores se cumple que $a \cdot b = b \cdot a$, la conmutatividad no se cumple para matrices.

Sean el vector $x \in \mathbb{R}^n$ y la matriz cuadrada $A \in \mathbb{R}^{n \times n}$. La norma del vector x se define como

$$\|x\| = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2} \quad (\text{A.27})$$

que es la distancia euclídeana, aunque puede definirse de varias maneras. Si la norma de un vector es 1 entonces se dice que es un vector unitario. Por ejemplo, $(0, 0, 1)$ o $(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}})$ son vectores unitarios.

Sea cual sea el vector x , es posible encontrar un vector unitario que tenga su misma dirección multiplicándolo por el recíproco de su norma: $x / \|x\|$.

De manera equivalente como con vectores, hay muchas formas de definir la *norma de una matriz*, una bastante común es la siguiente.

$$\|A\| = \sqrt{\sum_i^n \sum_j^n |a_{ij}|^2} \quad (\text{A.28})$$

Los elementos de una matriz cuadrada que están en la diagonal que inicia en la posición $(1, 1)$ (y terminan en la (n, n)) se dice que están en la *diagonal principal*. Por la disposición de sus elementos se puede distinguir varias matrices especiales, como las

- diagonales, donde sólo los elementos de la diagonal principal son distintos de cero, es decir, $a_{ij} = 0$ si $i \neq j$ y cualquier valor fuera de la diagonal;
- triangulares, donde hay ceros debajo o arriba de la diagonal principal.

Una matriz cuadrada que sólo tiene números 1 en su diagonal principal se llama matriz identidad y se denota por I . Si A y B son matrices cuadradas y su producto es la matriz identidad, es decir, $AB = I$, entonces se dice que B es inversa de A y (por lo mismo) que A es inversa de B . La inversa de una matriz A se denota por A^{-1} .

Dada una matriz $A = (a_{ij})_{m \times n}$, se define su *matriz transpuesta* A^T como la matriz resultante de intercambiar filas y columnas, es decir, cada a_{ij} se convierte en el a_{ji}^T . La matriz transpuesta de una matriz cuadrada sigue siendo cuadrada, pero si la matriz es rectangular de $m \times n$, la transpuesta es de $n \times m$.

Si una matriz es tal que es igual a su transpuesta, se dice que es una *matriz simétrica*, es decir, $A = A^T$. para una matriz de valores complejos $C = (c_{ij})_{n \times n}$, $c_{ij} \in \mathbb{C}$, su *matriz conjugada* $\bar{C}$ se obtiene sustituyendo cada valor c_{ij} por su conjugado complejo $\bar{c}_{ij}$.

Si una matriz compleja cumple con ser igual a su transpuesta conjugada, se dice que es una *matriz Hermitiana*. Para serlo, necesariamente debe tratarse de una matriz compleja cuadrada y que todos los elementos de la diagonal sean números reales, para que su conjugado sea el mismo número. La matriz

$$\begin{bmatrix} 3 & -2i & -4 + 2i \\ 2i & -2 & 5 + 3i \\ -4 - 2i & 5 - 3i & 6 \end{bmatrix} \quad (\text{A.29})$$

es un ejemplo de Hermitiana.

El producto de una $A = (a_{ij})_{m \times n}$ por un vector $x \in \mathbb{R}^n$ se calcula como

$$Ax = \begin{bmatrix} a_{1,1}x_1 + a_{1,2}x_2 + \cdots + a_{1,n}x_n \\ a_{2,1}x_1 + a_{2,2}x_2 + \cdots + a_{2,n}x_n \\ \vdots \\ a_{m,1}x_1 + a_{m,2}x_2 + \cdots + a_{m,n}x_n \end{bmatrix} \quad (\text{A.30})$$

por lo que el resultado es un vector de dimensión m . Esto obliga que la cantidad de columnas de A coincida con la dimensión de x .

El vector que resulta de multiplicar Ax puede no tener la misma dirección del vector x . En caso de que la tenga, se dice que se trata de un *vector propio* y cumple con

$$Ax = \lambda x \quad (\text{A.31})$$

donde λ es un número real llamado el *valor propio* y no es más que un valor para que Ax tenga la misma magnitud que x .

Ejemplo 59. La matriz

$$A = \begin{bmatrix} 1 & 1 \\ 4 & 1 \end{bmatrix} \quad (\text{A.32})$$

posee los vectores propios $v_1 = [1, -2]^T$ y $v_2 = [1, 2]^T$. Comprobando para v_1 , se tiene

$$\begin{bmatrix} 1 & 1 \\ 4 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ -2 \end{bmatrix} = \begin{bmatrix} -1 \\ 2 \end{bmatrix} \quad (\text{A.33})$$

de donde se ve que $Av_1 = -v_1$ y $\lambda = -1$ es su valor propio.

De manera general, una matriz compleja posee valores propios (y vectores propios) complejos. Sólo en el caso de las matrices Hermitianas, los valores propios siempre son números reales.

Para un estudio más profundo de vectores (y matrices) se recomienda al lector acudir a libros de álgebra lineal, donde se vean las propiedades de espacios vectoriales.

A.4.2. Consideraciones de programación

Las matrices son un buen ejemplo de entidades numéricas donde los problemas mal condicionados abundan. El número de condición para producto de vectores se aproxima con

$$\text{cond}(\mathbf{xy}) = 2 \frac{|\mathbf{x}| |\mathbf{y}|}{|\mathbf{x}^T \mathbf{y}|} \quad (\text{A.34})$$

y el número de condición de una matriz A se aproxima con el producto de las normas de A y su inversa

$$\text{cond}(A) = \|A\| \|A^{-1}\| \quad (\text{A.35})$$

respectivamente. Calcular la inversa de una matriz resulta un trabajo numéricamente muy demandante. Si además la matriz está muy mal condicionada, hay poco que esperar de la exactitud de la inversa. Esto lo estudia con bastante detalle Rump en [324].

Las matrices son entidades que requieren de muchos cuidados, numéricamente hablando. La manera normal de declararlas en un programa debiera ser con un arreglo bidimensional del tipo de dato requerido, pero esta metodología resulta inocente en muchos de los casos. Sólo se practica así para matrices de tamaño pequeño.

Es de esperarse que los problemas comienzan cuando se desea trabajar con matrices de gran tamaño. En lenguajes especializados en cómputo como Fortran, mucho se deja al compilador, aunque en ocasiones el usuario debe estar atento a ciertos detalles del sistema operativo, por ejemplo, para permitir que el cargador de programas prepare la memoria para la aplicación. Instrucciones como

```
setenv KMP_STACKSIZE 2g
```

son frecuentes entre programadores⁵.

Declaraciones estáticas (en tiempo de compilación, ejecutadas al tiempo de que el cargador prepara las condiciones para ceder la ejecución al programa)

⁵Instrucción para Suze 10.0. Suze 10.1 en adelante no requiere, pero trae el Kernel 2.6.16 en vez del 2.6.13 del anterior

Código A.2: Creando una matriz de $m \times n$ en tiempo de ejecución, en len.

```

1  double **MkMatriz(int m, int n)
2  {
3      int i, j;
4      double **M;

6      M = (double **) malloc(sizeof(double *)*m);
7      if ( !M )
8          return NULL;

10     for ( i=0; i<m ; i++ ) {
11         M[i] = (double*) malloc(sizeof(double)*n);
12         if ( !M[i] ) {
13             for ( j=0; j<=i ; j++ )
14                 free(M[j]);
15             return NULL;
16         }
17     }
18     return M;
19 }
```

no son una buena manera en general. Usar memoria dinámica resuelve gran cantidad de problemas. La manera de crear una matriz en tiempo de ejecución en lenguaje C se muestra en el código A.2. La idea es crear un arreglo de apuntadores a cada vector fila de la matriz.

Como es de esperarse, para liberar la memoria de esta matriz, primero se libera (con `free`) cada fila `M[i]` y finalmente el arreglo de apuntadores `M`. La manera de emplearla es simplemente con

```
double **Matriz;
Matriz = MkMatriz(200,200);
```

para una matriz de 200×200 . Solo bastará verificar que `Matriz` no es `NULL` para garantizar que ya se tiene reservada la memoria pedida.

A.5. Polinomios de Taylor

Brook Taylor (de 1685 a 1731), matemático británico. Dio formalidad a este concepto a partir de trabajos incompletos de Newton, Halley, Bernoulli, Leibniz y otros.

Los *polinomios de Taylor* se utilizan para aproximar funciones trascendentes para las que no existe alguna fórmula explícita calculable en una cantidad finita de pasos, como radicales o funciones trigonométricas. Su deducción puede encontrarse en muchos libros de cálculo.

Además de los polinomios de Taylor, existen otros como los de Chebyshev o las cocientes de polinomios de Padé, pero sólo se presentan los de Taylor, que son más ampliamente utilizados.

De manera general, si se conoce el valor de una función f continua en un punto x^* y f es infinitamente diferenciable en una vecindad de x^* , entonces se puede aproximar a f en valores vecinos a x^* con la *Serie de Taylor*:

$$f(x) = f(x^*) + hf'(x^*) + \frac{1}{2}h^2 f''(x^*) + \frac{1}{3!}h^3 f'''(x^*) + \cdots \quad (\text{A.36})$$

donde $h = x - x^*$. Este resultado es conocido como el teorema de Taylor. En general cada función trascendente posee algún valor donde es conocida. Por ejemplo, para la función seno, se conoce con exactitud $\sin(k\frac{\pi}{2})$ para k entera.

Evidentemente, los términos son cada vez más insignificantes por el factorial que aparece en el denominador, por lo que esta serie se trunca cuando se obtiene una aproximación aceptable, quedando el polinomio de Taylor.

Cuando x^* es el 0, obtenemos el polinomio de *McLaurin*. No es difícil suponer que este polinomio es utilizado comunmente en bibliotecas de funciones comerciales, como las que acompañan a muchos compiladores, pero no es así exactamente. Los polinomios de grado alto son mal condicionados, lo que impone algunas restricciones en su uso, presentadas con mayor detalle en el capítulo 7.

Colin Maclaurin (1698-1746).

Ejemplo 60. *Estas son las series de Taylor (McLaurin en estos casos) para algunas funciones.*

$$\begin{aligned} \sin(x) &= x - \frac{1}{6}x^3 + \frac{1}{120}x^5 - \frac{1}{5040}x^7 + \frac{1}{362880}x^9 + \cdots \\ \log(x+1) &= x - \frac{1}{2}x^2 + \frac{1}{3}x^3 - \frac{1}{4}x^4 + \frac{1}{5}x^5 - \frac{1}{6}x^6 + \cdots \\ e^{x^3} &= 1 + x^3 + \frac{1}{2}x^6 + \frac{1}{6}x^9 + \frac{1}{24}x^{12} + \frac{1}{120}x^{15} + \cdots \end{aligned}$$

Si se trunca el polinomio (lo que en la práctica es necesario) es posible agregar un término que represente al error de truncamiento, de la siguiente manera:

$$f(x) = f(x^*) + hf'(x) + \frac{1}{2}h^2 f''(x) + \cdots + \frac{1}{n!}h^n f^{(n)}(\xi), \quad (\text{A.37})$$

donde ξ es algún número en el intervalo $(x^* - h, x^* + h)$. El teorema de Taylor garantiza la existencia de tal número. Este último término es conocido como *residuo* y no es la única forma de expresarlo. Para mayor información debe consultarse algún libro de cálculo.

Determinar el valor máximo del residuo en cierto intervalo permite conocer la exactitud de la aproximación del polinomio de Taylor. También puede usarse la notación asintótica y escribir

$$f(x) = f(x^*) + hf'(x) + \frac{1}{2}h^2 f''(x) + \cdots + O(h^n)$$

que es más compacta. La primera expresión aparece en libros de matemáticas, mientras la segunda es más común en libros de computación y análisis numérico.

Ejemplo 61. Si se considera la serie de Taylor de e^x como el polinomio

$$p(x) = 1 + x + \frac{x^2}{2} + \frac{x^3}{3!} + \frac{x^4}{4!} + \frac{x^5}{5!} + \dots$$

y se trunca a los 5 primeros términos (hasta el término de grado 4), el error forward error analíticamente es

$$|e^x - p(x)| = \frac{x^5}{5!} + \frac{x^6}{6!} + \frac{x^7}{7!} + \frac{x^8}{8!} + \dots$$

El error backward se calcula como el valor h tal que $e^{x+h} = p(x)$. Es inmediato que este caso, $h = |\ln p(x) - x|$. Numéricamente, si se opera con una precisión de 9 decimales, en $x = 2$ se obtiene

En este ejemplo hay precisión pero no exactitud.

$$\begin{aligned} e^2 &\approx 7.389\,056\,098 \\ p(2) &= 7 \end{aligned}$$

con un forward error de 0.389056098 y un backward error de $\ln 7 - 2 \approx -0.540\,898\,509$. Normalmente obtener las expresiones analíticas es lo que interesa y por lo general no resulta tan simple como en este caso.

Nótese que estos errores están calculados como absolutos, que es lo más común en estos casos.

Ejemplo 62. Una función como $\sqrt{x}$ no posee polinomio de McLaurin pues no es continua en cero. Alejándose del cero, se puede desarrollar a

$$\sqrt{x+1} = 1 + \frac{1}{2}x - \frac{1}{8}x^2 + \frac{1}{16}x^3 - \frac{5}{128}x^4 + \frac{7}{256}x^5 - \frac{21}{1024}x^6 + \dots$$

pues esta función sí es continua y diferenciable en 0. De cualquier manera, el método de Newton es mejor en el caso de esta función y es el que se utiliza en las bibliotecas de funciones, luego de la etapa de reducción de rango y búsqueda en tabla⁶.

A.5.1. Interpretación geométrica

Considérese la función $\frac{e^x}{5} - 3 \sin x^2$. Su gráfica en el intervalo $[-2, 2]$ se muestra en la figura A.4. La figura A.4b muestra superpuestas la función f y el polinomio aproximante

$$\begin{aligned} p(x) &= \frac{e^x}{5} - 3 \sin x^2 \\ &= \frac{1}{5} + \frac{x}{5} - \frac{29x^2}{10} + \frac{x^3}{30} + \frac{x^4}{120} + \frac{x^5}{600} + \frac{1801x^6}{3600} + \\ &\quad \frac{x^7}{25\,200} + \frac{x^8}{201\,600} + \frac{x^9}{1814\,400} - \frac{453\,599x^{10}}{18\,144\,000} + \\ &\quad \frac{x^{11}}{199\,584\,000} + \frac{x^{12}}{2395\,008\,000} + \frac{x^{13}}{31\,135\,104\,000} + \dots \end{aligned}$$

⁶Consultar el capítulo de Programación de Funciones en la segunda parte.

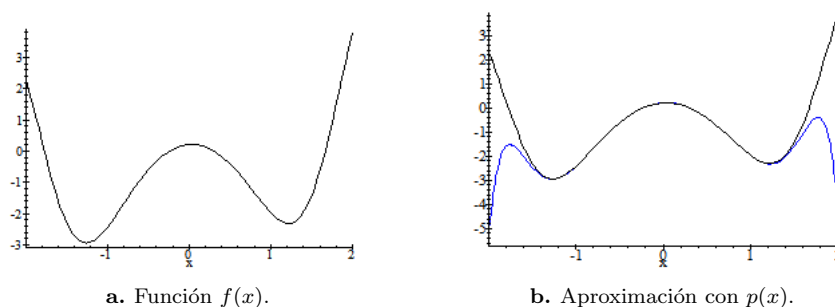


Figura A.4: Gráfica de $f(x) = \frac{e^x}{5} - 3 \sin x^2$ y su aproximación con $p(x)$.

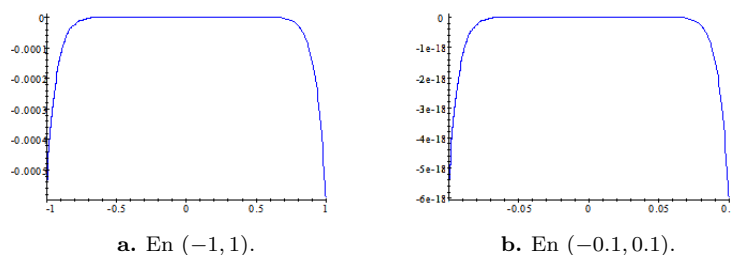


Figura A.5: Diferencia $f - p$ en intervalos distintos. Poseen la misma forma pero la escala del eje de las ordenadas habla por sí solo: el valor máximo de cada gráfica difiere en 14 órdenes de magnitud.

y su valor en cero es $1/5$.

Si se trunca a 14 términos y se grafica junto con la función f se obtiene una aproximación que al alejarse del punto de aproximación toma dirección opuesta a la de la función original, como se muestra en la figura A.4b.

Como puede verse, el polinomio se acerca en una vecindad del cero, aunque se aleja de la función cuando ampliamos el intervalo. Como de -0.5 a 0.5 se confunde por completo con la función original, es más representativo graficar la diferencia entre f y la aproximación polinomial p .

En la gráfica de la diferencia (figura A.5) se aprecia mejor que en la vecindad del 0 la aproximación es mejor, aún teniendo la misma forma, la escala del eje y habla por sí sola: en el intervalo $[-1, 1]$ (figura A.5a) se tiene una precisión de más de 17 decimales.

A.5.2. Consideraciones de programación

Entiéndase exponenciales,
trigonométricas, ...

Los polinomios de Taylor son entidades matemáticas muy prácticas por consistir sólo en operaciones elementales. Por ello no es de extrañar que sean de los primeros candidatos para aproximar funciones más complicadas, como las trascendentes. Utilizadas con cuidado, dan aproximaciones buenas. Lo que se necesita es determinar hasta qué término detener el cálculo de la serie y en qué intervalo se aplicarán. Aunque se darán muchos más detalles en el capítulo de evaluación de funciones, se presentan a continuación algunos lineamientos básicos para comenzar a comprender las ideas fundamentales de la programación numérica.

Cuando se programa, es posible detener la evaluación de los términos utilizando diversos criterios, no necesariamente excluyentes. En todos los casos, decir que algo es suficientemente pequeño significa que su magnitud es menor al error relativo (o absoluto en su caso) que cierta cantidad fija que es la tolerancia aceptable.

- **Cantidad de términos evaluados.** No es recomendable como criterio principal porque en general se ignora el término a partir del cual se corta la aproximación. Este es más bien un criterio de seguridad para evitar que los cálculos se ciclen y es común que se combine con otros criterios.
- **El siguiente término del polinomio es despreciable.** Tiene el inconveniente de que puede ocurrir que antes de ser despreciable, su tamaño ya no modifique el resultado parcial anterior, con lo que se hacen operaciones ociosas. En las siguientes secciones se justificará este hecho.

Por ejemplo, si la suma parcial hasta $n = 20$ es de 4 851 651 95.4 097 y el siguiente término es .000 000 000 797 246, es posible que en la aritmética de la computadora este pequeño valor ya no altere la suma parcial, aún cuando no es despreciable. También es conveniente aplicar la teoría y anticipar cuántos términos se requieren para lograr la precisión requerida en el intervalo de interés⁷.

Entre más se analice mejores resultados se obtendrán.

Para no dejar una idea incorrecta, es necesario aclarar que no se acostumbra programar un polinomio de Taylor general y verificar en tiempo de ejecución cuántos términos se requieren. Se hace un trabajo de análisis previo para determinar cuántos se requerirán para la precisión deseada y se deja fijo en el código.

- **El error absoluto entre las dos aproximaciones más recientes es muy pequeño.** Es sencillo pero no indica la relación entre el error y la magnitud calculada. Un error de .001 puede ser muy grande, pero no si se aproxima un número del orden de 10^{20} . Es equivalente al criterio anterior.
- **El error relativo entre las dos aproximaciones más recientes es muy pequeño.** Un criterio práctico. En este caso debe cuidarse que la

⁷El Teorema del Residuo es un buen inicio.

aproximación no cause un problema de división entre cero. Al dividir el error absoluto entre una de las aproximaciones, puede usarse la mayor en valor absoluto para evitar problemas.

Para evitar que durante la ejecución se tome este tipo de decisiones, es posible analizar el problema y decidir desde el diseño cuántos términos son necesarios. Así, la evaluación del polinomio queda como un conjunto de operaciones fijas que se pueden someter a diversos procesos de optimización. Más detalles sobre la evaluación de polinomios en el capítulo 6.

No en todos los casos los polinomios de Taylor son la única opción. Existen otras familias de polinomios o funciones cuya combinación también puede aproximar a otras funciones, como las trigonométricas en las series de Fourier. En el capítulo 7 de aproximación de funciones se presentan más detalles.

A.6. Distancias, normas y otros conceptos

Cada vez que se resuelva un problema numérico con la computadora, la solución encontrada es una aproximación de la solución matemáticamente exacta. La forma de medir esta diferencia entre la solución y el resultado obtenido es con el término de error o error absoluto, que es una diferencia numérica en el caso del conjunto de los números reales $\mathbb{R}$.

En muchas ocasiones, el conjunto $\mathbb{R}$ es insuficiente, como en el caso de las raíces del polinomio $x^2 + 1 = 0$, que son los números imaginarios $i, -i$, con parte real 0. Algunos métodos para el cálculo de raíces utiliza notación matricial, por lo que también debe ser posible medir el error entre vectores e incluso matrices.

Por lo anterior es necesario extender nuestro concepto de error para poder incluir cualquier entidad matemática: escalares, complejos, vectores, matrices, funciones y cualquier otro que sea necesario.

Esta rama de las matemáticas se profundiza en cualquier texto (y curso) de análisis funcional, aquí sólo se presentan los conceptos básicos que den el vocabulario y notación suficientes como para facilitar el desarrollo de los temas de los capítulos siguientes.

Se acostumbra llamar *espacios* en general a los conjuntos y *puntos* a sus elementos. Así, $\mathbb{R}$ es un espacio compuesto por una infinidad de puntos llamados números reales. El lector encontrará referidos así en casi todo texto de matemáticas y por lo tanto se usarán aquí también.

No debe causar confusión con la idea geométrica de $\mathbb{R}^3$, el espacio más conocido por estudiantes.

A.6.1. Espacios normados

La idea ahora es poder medir espacios cualesquiera donde sea posible medir el tamaño de los objetos y la distancia entre ellos. Aquí se generaliza la definición de norma dada en la sección A.4.

Normas

El término y concepto más básico es el de *norma*. Una norma es una función⁸ denotada por $\|x\|$ que mide el “tamaño” o magnitud del punto x .

Toda norma siempre está asociada a un espacio X en el que debe cumplir las siguientes propiedades. Sean x y y elementos arbitrarios de X y a cualquier escalar.

1. $\|x\| > 0$ si $x \neq 0$
2. $\|ax\| = |a| \|x\|$
3. $\|x + y\| \leq \|x\| + \|y\|$

Las operaciones deben estar bien definidas, como ocurre en el espacio de los números reales $\mathbb{R}$.

La norma es una función que asocia un número real a cada punto del espacio X , es decir, $\|\cdot\| : X \rightarrow \mathbb{R}$.

Ejemplo 63. En el caso de que X sea el conjunto de los números complejos $\mathbb{C}$, los puntos son de la forma $x = a + ib$ donde a es la parte real y b la parte imaginaria. La norma en este caso particular se conoce como módulo y se define como

$$\|x\| = \sqrt{a^2 + b^2} \quad (\text{A.38})$$

y el lector puede comprobar sin problemas que se cumplen las tres condiciones de la norma.

Como de costumbre, $i^2 = -1$.

Equivale también a la hipotenusa. Además, esta definición es útil para el caso de vectores en $\mathbb{R}^2$ pero no es la única norma que se puede definir, sólo la convencional.

Ejemplo 64. Las siguientes normas en $\mathbb{R}^n$, bastante comunes. Dado $x \in \mathbb{R}^n$, $x = (x_1, \dots, x_n)$, la norma de x puede definirse como

$$L_1(x) = \sum_{i=1}^n |x_i| \quad (\text{A.39})$$

$$L_2(x) = \sqrt{\sum_{i=1}^n x_i^2} \quad (\text{A.40})$$

$$L_k(x) = \left(\sum_{i=1}^n x_i^k \right)^{1/k} \quad (\text{A.41})$$

$$L_\infty(x) = \max\{x_i\}_{i=1}^n \quad (\text{A.42})$$

⁸Se acostumbraba usar de manera indistinta las palabras función, aplicación, mapeo y transformación.

y todas cumplen las propiedades descritas. L_1 es conocida como la distancia Manhattan, que es la que usamos cotidianamente para medir las distancias en las ciudades, al decir a cuántas cuadras queda un lugar de otro. L_2 es la distancia Euclidiana en $\mathbb{R}^n$.

Así como se ha definido para vectores, también es común utilizar normas para medir matrices en $\mathbb{R}^{m \times n}$. Este y más ejemplos pueden consultarse en libros especializados en álgebra lineal o análisis funcional.

Espacios normados y distancias

Cuando existe tal norma asociada al espacio X , se dice que X es un *espacio normado* o con norma. Una vez definida la magnitud de cada punto en el espacio, se puede definir la distancia entre cada par de puntos, lo que es sencillo a partir de su norma.

La *distancia* entre dos puntos x y y del espacio normado X es llamada *métrica* y se define como la norma de su diferencia

$$d(x, y) = \|x - y\| \quad (\text{A.43})$$

para lo que se supone que la operación $-$ está bien definida para cada par de puntos en X .

Las propiedades que debe cumplir toda métrica d son las siguientes. Sea z otro punto de X además de x y y .

1. $0 \leq d(x, y) < \infty$
2. $d(x, y) = 0$ si y solo si $x = y$
3. $d(x, y) = d(y, x)$
4. $d(x, y) \leq d(x, z) + d(z, y)$

La cuarta propiedad es la famosa *desigualdad del triángulo*. A X se le llama también *espacio métrico*. Cuando se distingue un espacio métrico de otro por la distancia asociada a cada uno, se acostumbra escribir (X, d_X) , que significa que el espacio métrico X tiene asociada la métrica o distancia d_X .

Así como se define el intervalo cerrado unitario $[0, 1]$ en $\mathbb{R}$, en un espacio métrico en general se define una *bola unitaria cerrada* como el conjunto de todos los puntos con norma menor o igual a 1, es decir, $\overline{B}_1(0) = \{x \in X, \|x\| \leq 1\}$.

Algunos autores escriben $\overline{B}_1(0)$ en vez de $\overline{B}_1(0)$

Igualmente se define la bola unitaria abierta $B_1(0) = \{x \in X, \|x\| < 1\}$, que es como la $\overline{B}_1(0)$ pero sin la corteza o cáscara. La pura corteza serían todos los puntos cuya norma es igual a 1. Esto equivale en $\mathbb{R}$ al intervalo $(0, 1)$, donde la cáscara serían los números 0 y 1.

Se puede definir una bola de cualquier radio, abierta o cerrada, por ejemplo la bola abierta de radio r

$$B_r(0) = \{x \in X, \|x\| < r\} \quad (\text{A.44})$$

y si se desea centrar la bola en el punto z en vez del cero se acostumbra escribir

$$B(z, r) = \{x \in X, \|x - z\| < r\} \quad (\text{A.45})$$

cuya concepción es importante cuando se trata de la convergencia de diversas sucesiones. Quitando al punto z de esa bola, lo que queda es la *vecindad abierta de radio r de z* .

A.6.2. Espacios de Banach

Una extensión muy importante de los espacios normados es el concepto de espacio de Banach, para lo que se requiere definir una *sucesión de Cauchy*. Una sucesión de puntos $\{x_n\}$ es de Cauchy si satisface

$$\lim_{\min(m,n) \rightarrow \infty} d(x_m, x_n) = 0 \quad (\text{A.46})$$

lo que significa que la diferencia entre pares de puntos va disminuyendo conforme se avanza en la sucesión, de tal forma que la sucesión converge. Un espacio es *completo* si toda sucesión que es de Cauchy converge.

Toda sucesión que converge es sucesión de Cauchy pero no al revés, como se muestra en el siguiente ejemplo.

Ejemplo 65. En el conjunto de los números racionales $\mathbb{Q}$, la sucesión a_n definida como

$$a_n = \left(1 + \frac{1}{n}\right)^n \quad (\text{A.47})$$

es de Cauchy pues cumple con (A.46) pero no converge en $\mathbb{Q}$ pues el límite cuando $n \rightarrow \infty$ es el número irracional e .

Ahora ya se puede definir que un *espacio de Banach* es un espacio normado completo. Con definiciones adecuadas de norma, los reales, complejos, $\mathbb{R}^n$ y las matrices son espacios de Banach. Es por esto que el estudio de las propiedades generales de estos espacios es tan importante en análisis numérico: analizando un espacio X cualquiera se analizan todos a la vez.

Lo que se conoce como *endomorfismo*.

Sea $f : X \rightarrow X$ una función de un espacio métrico en sí mismo, es decir, toma un punto de X y su resultado es también un punto de X . Se dice que f es una *función contractiva* si existe una constante k que cumple con $0 \leq k < 1$ y

$$d(f(x), f(y)) \leq k d(x, y) \quad (\text{A.48})$$

para toda x y y puntos de X . Esta definición es de gran importancia para el capítulo 6, donde en particular interesa que una función sea de Lipschitz.

Una función $f : X \rightarrow Y$ de un espacio métrico (X, d_X) a otro (Y, d_Y) , se dice que es de *Lipschitz* cuando

$$d_Y(f(a), f(b)) \leq L d_X(a, b) \quad (\text{A.49})$$

con a y b elementos de X . A L se le llama *constante de Lipschitz*⁹. El caso $X = Y$ es de gran importancia práctica, principalmente cuando $X = \mathbb{R}$, donde la distancia se define como $d(x, y) \equiv |x - y|$.

Es común que para aplicar con éxito algunos algoritmos sea necesario que se cumplan ciertas condiciones. Un caso común es cuando para una función $f : X \rightarrow Y$ se requiere que sea 1-a-1 y que tanto f como su inversa f^{-1} sean continuas. Toda función que cumple con esos requisitos es llamada *homeomorfismo*.

A.6.3. Convexidad

Este es un concepto que se acostumbra aplicar de dos maneras básicas: a conjuntos o a funciones. La mejor forma de comprenderlos es con la idea geométrica. Un subconjunto $C \subset X$ es *convexo* si dados dos puntos cualesquiera x y y , la recta que los une está completamente contenida en C , es decir,

$$\lambda x + (1 - \lambda)y \in C, \quad \text{con } 0 \leq \lambda \leq 1. \quad (\text{A.50})$$

Una función f continua en el intervalo $[a, b]$ con primera y segunda derivadas continuas es *convexa* si para todo $x \in [a, b]$ se cumple que $f''(x) > 0$.

Ejemplo 66. La parábola definida por x^2 es una función convexa pues su segunda derivada es 2 y además define un conjunto convexo, que es la región que queda entre las dos ramas de la parábola que se muestra en la figura A.6. La región externa es un conjunto cóncavo.

Puede verse al conjunto completo de puntos generados por f en cualquier intervalo real como el conjunto de las coordenadas cartesianas $(x, f(x))$ mapeadas dentro de la parábola, como $f([a, b]) \subset \mathbb{R}^2$. Toda la región sombreada en la figura A.6 es $f(\mathbb{R})$.

Queda responder la pregunta ¿Qué tan convexa es la función f ? De manera intuitiva, la convexidad de una función debiera ser mayor donde es *más curva*. La medida que se utiliza es

$$L_f(x) = \frac{f(x)f''(x)}{f'(x)^2} \quad (\text{A.51})$$

para la convexidad de f en el punto x , conocida como *convexidad logarítmica*. Para esto se requiere que $f'(x) \neq 0$ y que f tenga hasta la segunda derivada continua.

⁹Normalmente se toma la menor L que cumpla con la condición (A.49), pero no se necesitará tanto en este texto.

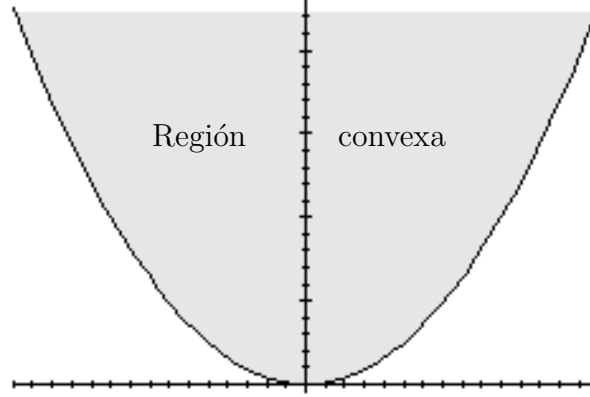


Figura A.6: La función convexa define un conjunto convexo. Cualquier segmento de recta con ambos extremos dentro de la región sombreada queda completamente contenido allí mismo.

$L_f(x)$ representa el grado de convexidad puntual. Se dice que una función f es logarítmicamente convexa si $\log(f)$ también es convexa. La convexidad logarítmica (A.51) es la cantidad de veces que habría que aplicar el logaritmo a una función convexa para que deje de serlo (obtener una función cóncava).

A.7. Cambio de base

El cambio de base es una operación rutinaria en las computadoras, que pasa desapercibida al usuario final (e incluso a la mayoría de los programadores), pero debiera ser bien entendida para un programador, pues es una de las fuentes inevitables de errores numéricos. En las coputadoras, el usuario captura números en base 10, se convierten a base 2, se procesan y el resultado se le da al usuario convirtiéndolo en base 10 de nuevo. Esta sección se refiere no al cambio de base general, donde la base del sistema numérico puede ser cualquier numero real, sino sólo al caso entero. Más detalles (y formalismo) se pueden consultar en [261], donde se presente el teorema de Cambio de Base.

Sean β y λ dos enteros distintos, mayores que cero, las bases de lo que de aquí en adelante se llamará *sistemas numéricos internos*. Para modelar los números que se pueden representar con una computadora, se limita su representación con una cantidad finita de cifras, cualquiera que sea la base. Sea m la cantidad de dígitos permitidos en base β y n la cantidad permitida en base λ . Así, $x_\beta = .d_1d_2 \dots d_m \times \beta^i$ es la forma de los números en base β , con precisión m .

Sea entonces

$$S_\beta^m = \left\{ x \mid x = \sum_{i=1}^m d_i \beta^{j-i} \right\} \quad (\text{A.52})$$

Al hacer la conversión de base, sea T la función que trunca y R la que redondea. Hay dos cuestiones prácticas de interper inicial en ambos casos, independientemente de si se redondea o se trunca.

1. Si la función es inyectiva, entonces números distintos siempre se convierten en números distintos.
2. Si la función es sobreyectiva, entonces todo número representable en el sistema de mantisas representa a al menos un número en el sistema de mantisas inicial.

Habiendo definido lo anterior, se puede enunciar el

Proposición 16 (Teorema del cambio de base (TCB)). *Sean β y λ dos enteros positivos distintos. Si para cualquier par de enteros $i, j > 0$ se cumple que $\beta^i \neq \lambda^j$, entonces, la función de conversión de base por truncamiento (o redondeo) de S_β^m a S_λ^n , es decir, $T : S_\beta^m \rightarrow S_\lambda^n$ (o $R : S_\beta^m \rightarrow S_\lambda^n$ para redondeo) es*

1. *inyectiva (1 a 1) si y sólo si $\lambda^{n-1} \geq \beta^n - 1$ y*
2. *sobreyectiva si y sólo si $\beta^{m-1} \geq \lambda^n - 1$.*

En el ejemplo 68, la conversión de S_{10}^{17} a S_2^{53} falla pues no se cumple la inyectividad ($2^{53-1} \not\geq 10^{17} - 1$).

A.7.1. Implicaciones

El TCB presenta las condiciones para que el cambio de base cumpla con las necesidades del diseñador, ya sea porque se necesita que los números del usuario se conviertan sin ambigüedades en los números internos de la computadora (inyectividad) o para que los resultados se conviertan al sistema numérico del usuario sin pérdida de exactitud. Conseguir que las dos condiciones se cumplan es un problema técnico complicado.

Considerando el orden de los números reales, cada número en S_β^m tiene un vecino inmediato mayor y uno inmediato menor que también pertenecen a S_β^m . De esta forma, se define el sucesor

Esto significa que entre cada par de potencias de la base, se pueden representar β^m o λ^n números distintos y no más.

Ejemplo 69. *Los números en base 10 con 3 dígitos si se pueden convertir a números en base 2 con 10 dígitos sin perder exactitud, pues . En sentido contrario, de base 2 a base 10 no es posible. Esto se debe a que $10^3 = 1000$ y $2^{10} = 1024$, por lo que hay 24 representaciones en base 2 sobrando.*

El ejemplo anterior permite observar que la relación entre las potencias de la base limita las conversiones posibles. Debe ser claro que interesa pasar de una base a otra en ambos sentidos. Deb ser claro que si una base es múltiplo de la otra, siempre hay dos precisiones m y n que permiten una conversión exacta.

En el caso de que una conversión requiera de más dígitos que los permitidos, la decisión es aplicar redondeo, es decir, buscar el número interno más cercano al valor exacto. Los otros modos de redondeo (direccionados) no se considerarán.

A.8. Fracciones continuas

Las fracciones continuas son expresiones de la forma

$$a_0 + \frac{1}{a_1 + \frac{1}{a_2 + \frac{1}{\ddots}}} \quad (\text{A.53})$$

aunque también es posible que se trate de una expresión finita:

$$a_0 + \frac{1}{a_1 + \frac{1}{\ddots + \frac{1}{a_n}}} \quad (\text{A.54})$$

Otros conceptos de utilidad serán definidos al momento de necesitarse. Los anteriores son una base firme tanto conceptual como de vocabulario¹² para poder explicar algunos temas de manera clara y concisa. Debe entenderse que la presentación anterior es sencilla pero suficiente para los fines de este texto. Los conceptos pueden formalizarse más aún, pensando en espacios topológicos, pero no es realmente necesario.

A.9. Ejercicios

1. ¿Cuál es la razón por la que en una aproximación de valores en el intervalo $[-1, 1]$ se prefiere utilizar el error absoluto y no el relativo?
2. Utilice la fórmula (A.6) para calcular las cifras significativas correctas de π truncado con 4, 5 y 6 decimales.
3. En una competencia de tiro con arco, ¿gana quien tiene mucha precisión y poca exactitud o quien tiene mucha exactitud y poca precisión?
4. Reproduzca el experimento numérico del ejemplo 54.

¹²Vocabulario que no debe causar temor, pese a los nombres rimbombantes que las matemáticos han coseguido dar a algunas cosas.

5. Para el ejemplo 55, de la página 345, utilice la fórmula (A.9) para realizar el experimento con un programa de computadora.
6. Programe rutinas para llevar a cabo operaciones básicas con matrices y vectores.
7. Calcule el número de condición de los polinomios $x^3 - x^2 + x - 1$ y $x^3 + x^2 + x + 1$ en 1 y -1 .
8. Calcule los primeros 5 términos del polinomio de Taylor para $\cos x$ centrado en $x = 0$ (polinomio de McLaurin).
9. Con el polinomio de Taylor del ejercicio anterior, haga un programa que compare la exactitud del polinomio encontrado con el de la función $\cos$ de la biblioteca de funciones del compilador del lenguaje utilizado. Solo compare para el intervalo $[0, \pi/4]$.

Apéndice B

Otras aritméticas

También se han propuesto APF que utilizan un sistema numérico distinto a la forma (1.7), página 15. Se han buscado sistemas que signifiquen alguna mejora de alguno de los aspectos de la APF convencional, sin empeorar sus virtudes (al menos no sustancialmente). Mientras algunos solo son nuevos sistemas de cálculo, algunos implican también otro formato. Ejemplo de esto son las fracciones continuas que tienen respuesta numérica excelente. Siendo precisas y veloces son poco utilizadas.

El intento de representar números racionales a/b como la pareja de enteros también fue estudiada pero no se presenta aquí (véase el estudio de Peter Henrici [163]). A continuación se presentan los sistemas más importantes (o mejor conocidos).

B.1. Aritmética de Punto fijo

Hay dos maneras principales de representar números reales con computadora, los que requieren punto decimal: con punto fijo o con punto flotante. Cada representación ofrece ventajas y desventajas, siendo ambas utilizadas en la actualidad, en circunstancias distintas. En los inicios de la computación, por años se debatió la mejor manera y muchos análisis se hacían para los dos tipos de punto, como Wilkinson al analizar el error en la inversión de matrices en [394]. En su investigación, Wilkinson encuentra que la exactitud del punto fijo es mejor, pero afirma que subrutinas que manejen punto flotante se necesitan para la seguridad de los cálculos en matrices.

Los números de *punto fijo* son sencillos de entender a partir de los ejemplos de la tabla B.1, donde el punto separa la parte entera de la fraccionaria.

Los números que se utilizan normalmente para la mayoría de las personas son de este tipo y también se le llama sistema posicional. El punto se mantiene con

550043.4238901
 8123.456
 3.14159265...
 0.0000122
 0.00000000000232

Tabla B.1: Números en notación de punto fijo. Cada posición indica su magnitud u orden (... , decenas, unidades, punto, décimas,...)

su significado convencional de separar la parte fraccionaria de la parte entera¹.

En la práctica, los números tenían que tener una cantidad fija de cifras tanto para la parte entera como la fraccionaria, lo que limitaba considerablemente el rango de los números que se podían representar. Así, un número x es representado como

$$x = d_m d_{m-1} \dots d_1 . f_1 f_2 \dots f_n \quad (\text{B.1})$$

para una cantidad con m cifras antes del punto y n para los decimales. Todos los que diseñaron computadoras digitales, prefirieron a $x \in (-1, 1)$, es decir, $m = 0$, para facilitar las operaciones. Ejemplos de esto son [250, 380, 382]. También se ha pensado en el intervalo semiabierto $[-1, 1)$, como en [172].

Las mismas limitaciones numéricas del punto fijo son las que permiten que sea comunmente implementado utilizando aritmética entera, lo que lo hace muy consistente y veloz. Cuando el rango numérico es suficientemente pequeño, esta aritmética resulta más rápida que la flotante.

Alston Scott Householder, (1904-1993), matemático estadounidense con importantes aportaciones al análisis numérico.

Un tratamiento sistemático de los errores, considerando representación en punto fijo, lo hace Householder² en su artículo [172], donde presenta técnicas de análisis que se pueden usar en muchas situaciones. En su artículo, habla por primera vez de la unidad de redondeo. Retomando la notación de Von Neumann en 1.2, para Householder la unidad de redondeo es β^{-k-1} , para un número con k dígitos en base β .

B.1.1. Pros y contras

Como en punto fijo no se requiere calcular el exponente de cada número, las operaciones aritméticas son más rápidas. Como el el rango numérico queda reducido, se pierde exactitud, aunque se gana precisión para los números sí representados.

¹Las primeras computadoras realizaban todas las operaciones con números enteros y el usuario escalaba resultados posteriormente. Al poco tiempo se comenzó a utilizar un sistema de punto fijo no como B.1, sino uno donde todos los números estaban en el intervalo $(-1, 1)$. Era tan poco flexible que a los pocos años se comenzó a utilizar el sistema de punto flotante, que ya se utilizaba en máquinas calculadoras.

²Householder fue el primero en concentrar la bibliografía existente sobre análisis numérico para el año de 1955 en [173], aunque orientado a las áreas de interés de su libro de 1953, Principios de Análisis Numérico. Incluye 321 referencias entre libros y artículos.

El uso de números de punto fijo requiere un análisis más cuidadoso para que la magnitud de los resultados intermedios no sean mayores que 1. Se requiere el uso constante de factores de escala para lograrlo, factores que deben irse almacenando a la par de los cálculos. Este problema era conocido no solo en usuarios de computadoras, sino en calculadoras que usaban registros de tamaño limitado. En 1952, Codd y Herrick presentaron en [66] la manera de escalar con seguridad, teniendo números en base 10, considerando una calculadora digital en binario.

Por otra parte, la introducción constante de valores de escala interrumpe el flujo natural de un programa. Para no detener el flujo cada ocasión potencial de peligro, se requiere escalamiento más agresivo, lo que introduce otros problemas.

Llevar orden de los escalamientos no es el problema mayor. Como detalla Lotkin en [250], el remedio simple de dividir los cálculos intermedios entre dos cada que se necesita, lleva a problemas de exactitud, que pueden ocasionar inestabilidad. Lotkin muestra una manera cuidadosa de hacer el escalamiento.

En poco tiempo quedó claro que el punto fijo no era el más adecuado para las computadoras digitales de propósito general. Aún siendo potencialmente más preciso que el punto flotante, para un rango determinado, los inconvenientes del punto fijo son suficiente motivo.

B.1.2. Usos del Punto Fijo

En la actualidad, el uso del punto fijo está muy difundido. De hecho nunca dejó de utilizarse, como se entiende bien de [27]. El punto fijo y su aritmética, se emplean en procesadores que no incluyen unidad aritmética, como algunos microcontroladores, procesadores digitales de señales, FPGA (Field-programmable Gate Array) y dispositivos dedicados.

La necesidad de convertir la entrada de los usuarios, típicamente en formato decimal de punto fijo, a sistema binario de punto flotante, hacer cálculos y convertir el resultado en el formato del usuario, obligó a desarrollar técnicas para reducir los errores de conversión. Uno de los primeros trabajos publicados al respecto fue el de Taranto, en 1959 [363], citado por los esfuerzos posteriores, como [260, 262]. No hay duda de que la publicación del Teorema de Cambio de Base [261] fue esencial en esta labor. Actualmente, en aplicaciones como el procesamiento digital de señales, su uso es frecuente, como en [268], con la intención de reducir costos de producción y consumo eléctrico.

En un procesador que no dispone de unidad de punto flotante, con solo disponer de sumas y desplazamientos de bits, es posible simular el resto de las operaciones, con la ayuda de unos pocos resultados precalculados (técnica conocida como table lookup). Esto es útil cuando las cantidades involucradas requieren un rango numérico pequeño y una cantidad fija de decimales.

[258]

Es posible que la razón principal sea el desarrollo de la técnica de cómputo conocida como CORDIC (COordinate Rotation DIgital Computer), introducida por Volder en [379], en 1959. Es una técnica sencilla y eficiente que permite hacer cálculos complejos en hardware que carece de circuitos para multiplicar.

El uso del punto fijo es tan importante en la práctica que una edición posterior de este libro debiera incluir un capítulo completo de sus métodos.

B.2. Sistema logarítmico

Este es el caso del *sistema numérico logarítmico* (SNL), que requiere dos bits adicionales como banderas para señalar excepciones. El rango numérico es similar, así como la precisión, pero es mucho más sencillo realizar algunas operaciones. En general el formato es

$$x = (-1)^s \times n.f$$

donde $n.f$ es la mantisa con su partes entera y fraccionaria, codificadas en complemento a 2. No se requiere conservar exponente pero sí dos bits para excepciones. Este formato no es único, también pudiera ser $x = \pm\beta^i$, con β ligeramente mayor que 1.)

Lo que se necesita es transformar al logaritmo (normalmente base 2) de los números iniciales y realizar todo los procesos numéricos para solo hacer la conversión inversa al final de los cálculos. Por ejemplo, si $x = m_x \times \beta^{e_x}$ y $y = m_y \times \beta^{e_y}$ ya están en sistema logarítmico, el producto xy se obtiene con la suma de mantisas pues $\log(xy) = \log x + \log y$, por lo que hay ahorro de tiempo, espacio en la ALU³ del procesador y energía eléctrica.

Lo mismo ocurre con la división pues $\log(x/y) = \log x - \log y$, por lo que el costo de $\times$ y $\div$ equivale al de la suma y resta respectivamente.

Los problemas son con la representación de enteros pequeños, la suma y la resta en el SNL: si $x > y$

$$\log |x + y| = x + \log |1 + 2^{y-x}|$$

y 2^{y-x} se calcula con interpolación, basándose en una tabla de valores precalculada. El costo es menor aún que el de la multiplicación convencional, así que el SNL sigue siendo más eficiente, pero sólo si no se requiere gran cantidad de decimales.

El hecho de que números como 2 o 3 no puedan representarse con exactitud sí es algo que no es conveniente.

Una buena característica del SNL es que el error relativo entre dos números vecinos se mantiene constante, por lo que no hay wobbling (ver 1.26 en la página

³Arithmetic Logic Unit (Unidad Aritmético Lógica), encargada de las operaciones numéricas.

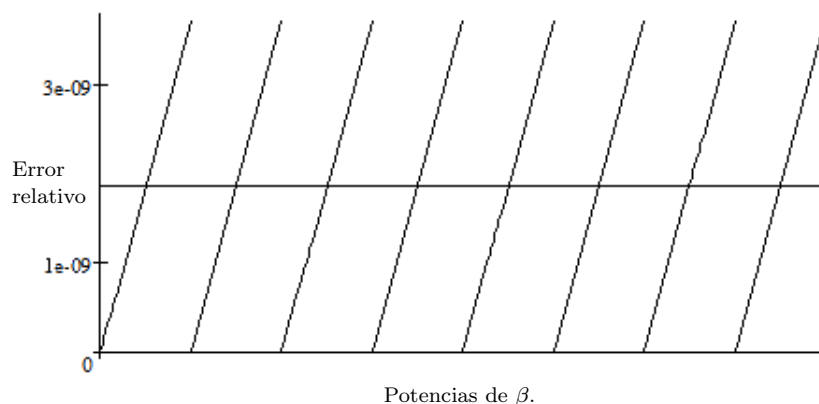


Figura B.1: El error relativo del Wobbling o tambaleo. La línea horizontal representa el error relativo constante, comparado con el wobbling (líneas en diagonal.)

30). La distancia crece de manera continua y no discretizada, como ocurre en $\mathbb{F}$ al pasar de una potencia de β a otra. Y sí hay mucha diferencia entre la constante y el error relativo con wobbling, como se muestra en la figura B.1.

El SNL es utilizado más ampliamente en Europa, aunque poco a poco gana adeptos también en América y Asia. Puede consultarse [359] para mejores referencias. Además, Higham [168] presenta otro modelo logarítmico adicional.

B.3. Sistema exponencial

Otro ejemplo de aritmética es la que propusieron Clenshaw y Olver en [64], con la intención de ampliar el rango de números representables, un poco a costa de la precisión. La idea es representar cada número x como

$$x = e^{e^{\dots e^a}}$$

donde el proceso de exponenciación se realiza l veces. Cuando $l = 0$ se tiene $x = a$. Clenshaw justifica que para fines prácticos no es necesario pasar de $l = 4$ para representar a todos los números reales. Con 3 bits que indiquen los niveles, el overflow estaría olvidado del cómputo convencional.

Este sistema es también llamado *índice de nivel simétrico*. A diferencia del SNL, el exponencial ha prosperado muy poco.

B.4. Aritmética de intervalos

Este es un problema intrínseco de la APF.

Cada $\text{fl}(x)$ representa una aproximación del número real x , cuya exactitud depende de diversos factores. En un proceso de cálculo en general, no se puede estar del todo seguro de qué tan cerca están uno del otro y ni siquiera si $x < \text{fl}(x)$ o $x > \text{fl}(x)$.

Una forma de evitar esta incertidumbre es no trabajar con $\text{fl}(x)$ sino con el intervalo más pequeño que lo contiene, idea surgida antes de la década de los 60s. Así, si dos NPF que rodean a x son $\underline{x}$ y $\bar{x}$, con $\underline{x} < \bar{x}$, entonces $[\underline{x}, \bar{x}]$ es un intervalo donde es seguro que se encuentra x . Típicamente, se representa $[x] = [\underline{x}, \bar{x}]$.

El principio más importante de la aritmética de intervalos es la

Inclusión isotónica Por este principio, se tiene que cumplir que si $x \in [x]$ y $y \in [y]$ son dos valores contenidos en los respectivos intervalos, entonces el resultado de cualquier operación aritmética $\odot$ debe estar en el intervalo que es resultado de la operación, es decir, $x \odot y \in [x] \odot [y]$. Lo mismo se cumple para la raíz cuadrada.

La *longitud* de $[x]$ se define como $|[x]| = \bar{x} - \underline{x}$ y es cero sólo en el caso degenerado, cuando $\bar{x} = \underline{x} = x$. Además de los números $\underline{x}$ y $\bar{x}$, se puede usar una representación alternativa equivalente con la longitud y el punto medio $\text{fl}(x) \approx (\underline{x} + \bar{x})/2$.

En la práctica, si se dispone de un ambiente de programación que cumpla con la norma 754, donde se controle el modo de redondeo, entonces toda operación aritmética puede realizarse primero redondeando hacia $+\infty$ y luego hacia $-\infty$ para obtener el intervalo que contiene el resultado.

En el mejor de los casos, se tiene que $[\underline{x}, \bar{x}] = [\text{fl}_{\nabla}(x), \text{fl}_{\Delta}(x)]$ cuando el intervalo tiene la menor longitud posible: $\text{ulp}(x)$.

Error máximo contra error desconocido.

De esta forma, en vez de representar a x con una cantidad cuya exactitud se ignora, se le representa con un intervalo donde se tiene la seguridad que está contenido x . Con este nuevo concepto $[x]$, se desarrollan propiedades de un nuevo conjunto formado por todos los intervalos reales

$$\mathbb{I} = \{[a, b] \subseteq \mathbb{R}\}.$$

Por ejemplo, las operaciones aritméticas elementales se realizan de la siguiente manera.

$$\begin{aligned} [x] + [y] &= [\underline{x} + \underline{y}, \bar{x} + \bar{y}] \\ [x] - [y] &= [\underline{x} - \bar{y}, \bar{x} - \underline{y}] \\ [x] \times [y] &= [\min\{\underline{x}\underline{y}, \underline{x}\bar{y}, \bar{x}\underline{y}, \bar{x}\bar{y}\}, \max\{\underline{x}\underline{y}, \underline{x}\bar{y}, \bar{x}\underline{y}, \bar{x}\bar{y}\}] \\ [x]^{-1} &= [1/\bar{x}, 1/\underline{x}] \\ [x] \div [y] &= [x] \times [y]^{-1}. \end{aligned}$$

Debe entenderse que los redondeos son direccionados, por ejemplo

$$[x] + [y] = [\text{fl}_{\nabla}(\underline{x} + \underline{y}), \text{fl}_{\Delta}(\bar{x} + \bar{y})]$$

y similarmente con el resto.

Algunas propiedades estructurales de las funciones, como la inyectividad, difíciles de determinar con programas, han probado ser un tanto más sencillas con intervalos. Por ejemplo, el trabajo de Lagrange [231]⁴, donde no sólo se define $f([x])$ sino $[f]([x])$. Incluso extienden el concepto a *inyectividad parcial* para lograr una respuesta correcta siempre.

El principal uso de los resultados de los cálculos que usan aritmética de intervalos es la validación de resultados, que permite encontrar cotas garantizadas de los resultados correctos. La longitud del intervalo que contiene el resultado es una medida de la exactitud del algoritmo. Y la exactitud se puede “ver”.

En pocas palabras: la aritmética de intervalos da resultados matemáticamente correctos. Como la exactitud es ahora tan obvia, hay una entusiasta búsqueda de algoritmos que produzcan intervalos de menor longitud cada vez.

B.4.1. Cuidados al usar aritmética de intervalos

La aritmética de intervalos es una buena idea que mal empleada da resultados que no son útiles por la enorme incertidumbre. Conforme se van haciendo operaciones con los intervalos, la longitud de los resultados intermedios puede ir aumentando hasta tamaños no deseables. Kahan [204] y Rump [323] dan ejemplos de esto, el primero como objetivo único de su artículo y el segundo como parte de su presentación de métodos de validación automática en las pruebas asistidas por computadora. En general se le considera una aritmética pesimista, pero útil en combinación con otras técnicas.

Lo anterior significa que partir de un programa y convertir todas sus variables a intervalos es tan inocente como potencialmente desastroso. Es necesario el diseño cuidadoso de algoritmos que cuiden meticulosamente el crecimiento de la longitud de los intervalos.

La única manera de reducir la longitud de los intervalos es multiplicarlos por constantes pequeñas (o dividirlos entre constantes grandes), lo que debe utilizarse cada vez que sea posible, así como el uso de datos puntuales (NPF) en momentos convenientes. Otra medida muy útil es utilizar datos originales siempre que sea posibles.

Otros problemas surgen al momento de comparar $[x]$ con $[y]$ pues la disposición de los intervalos tiene muchas variantes. Un caso sencillo es cuando $[x] < [y]$ pero decidir cuando $\underline{y} < \bar{x}$ o $[x] \subset [y]$ no es tan claro. Además, parece que el punto medio de los intervalos debe tener su importancia al determinar la relación de orden. La figura B.2 se muestra los posibles casos, faltando sólo los simétricos.

⁴Las pocas referencias sobre el tema, comparadas con la aritmética abitual, hablan de lo novedoso de su uso.

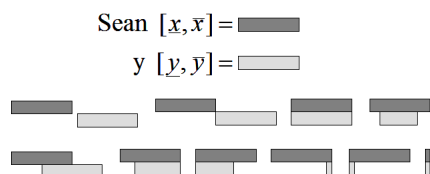


Figura B.2: Casos de comparación de intervalos. Los rectángulos representan intervalos con diversos límites.

Las relaciones pueden ser ahora con resultados más variados, como verdaderas o posiblemente verdaderas. Además se deben agregar las relaciones conjuntistas como la contención o la intersección a los operadores de un lenguaje. Cualquier relación de orden u operación no válida con intervalos debe lanzar una excepción, levantar banderas y posiblemente generar intervalos como $[-\infty, \infty]$ o $[\text{NaN}, \text{NaN}]$. Este es el caso de la división entre un intervalo que contiene al cero o la raíz cuadrada de un intervalo que contiene números negativos.

Las primeras investigaciones serias datan de la década de 1920, por ejemplo [59], con carácter teórico y sin prever la utilidad que tendrían posteriormente. Por no ser un concepto tan nuevo, ya hay gran cantidad de teoría matemática en la que la computación moderna se apoya. Hay versiones para intervalos de gran cantidad de resultados sobre los reales, conjuntos y otras estructuras. Un texto clásico sobre el tema es el de Alefeld y Herzberger [127]. Más actualizado y con el tratamiento correcto de los números especiales de $\mathbb{F}$ es [165].

Otro inconveniente es el aumento en el tiempo de cómputo (debido tanto al cálculo doble como al cambio constante de modo de redondeo del procesador), pero los defensores de la aritmética de intervalos lo justifican alegando

- la velocidad de las computadoras modernas y
- tener la seguridad de en qué intervalo está el resultado y con qué exactitud.

Lo cierto es que la aritmética de intervalos tiene más seguidores en Europa, pero su importancia es suficiente como para que instituciones y organismos la consideren en toda norma y especificación seria de lenguajes científicos.

Funciones como $f(x) = \cos(e^x)$ son más fáciles de graficar usando aritmética de intervalos.

B.4.2. Librerías y otras herramientas

Además de una amplia literatura, ya hay disponibles librerías matemáticas gratuitas para poder programar cómputo sobre intervalos. Por ejemplo C-XSC de la universidad de Karlsruhe⁵, flib++⁶ [240] de la Universidad de Berische

⁵A la aritmética de intervalos también se le llama aritmética de Karlsruhe, donde el Prof. U. Kulish, del Instituto de Matemática Numérica ha sido un gran impulsor. Léanse sus Letters to the IEEE Computer Arithmetic Standards Revision Group, disponibles en muchos sitios de internet.

⁶Originalmente desarrollada en Karlsruhe por Hofschuster y Krämer.

o Gaol, desarrollada en C++ por Frederic Goualard. Ambas poseen tipos de datos predefinidos para dar facilidades al programar con intervalos.

En Fortran 90 está disponible la INTLIB [215]. Más recientemente surgió la CoStLy [287], con gran variedad de funciones elementales para números complejos. En este caso, los intervalos $[x]$ se convierten en un rectángulo sobre los números complejos. Si $z = x + iy$, entonces el rectángulo que lo contiene es el producto cartesiano $[x] \times [y]$. En algunas áreas, se prefiere una bola cerrada $\overline{B}_r(z)$ alrededor de z y no un rectángulo pues desarrollar la teoría puede ser más fácil.

El paquete de cómputo simbólico MATLAB, ampliamente utilizado, no solo en cursos de Análisis Numérico, sino en el desarrollo de aplicaciones profesionales en una amplia gama de áreas de conocimiento, ofrece desde hace una década el toolbox INTLAB. El tipo predefinido `intval` y los operadores sobrecargados, ocasionan que se realicen las operaciones con los modos de redondeo necesarios.

Además, Mathematica, Maple y MuPAD ya incluyen intervalos como tipo de dato. GrafEq es un programa de graficación que permite el uso de intervalos para funciones complicadas.

También ya hay disponibles herramientas diversas, como calculadoras de escritorio que utilizan aritmética de intervalos. Un ejemplo es ItvCalc, originalmente para sistema operativo Unix, pero hay disponible una versión para Windows.

La extensión de esta sección es muestra clara de la importancia de la aritmética de intervalos como alternativa o complemento a la de punto flotante⁷.

B.5. Aritmética de cifras significativas

[22]

La aritmética de cifras significativas (ACS) es similar en algunos aspectos a la de intervalos. La idea es de Nick Metropolis y consiste en ir conservando sólo los dígitos correctos de un cálculo a otro, con una serie de reglas que permiten aproximar el error, haciendo uso de distribuciones de probabilidad. En las computadoras surgió con el propósito de identificar los bits correctos al realizar operaciones numéricas, como apoyo a la aritmética convencional (de números normalizados).

En 1963, Max Goldstein presentó algunas de las primeras experiencias, sobre las máquinas IBM 7090 en [143]. En esas computadoras, el modo de punto flotante estaba separado del de cifras significativas.

El modelo de APF se base en que el error relativo tiene distribución uniforme, es decir, en $\text{fl}(x) = x(1 + \delta)$, con $|\delta| < \mu$, debe ocurrir que δ puede ser cualquier número en $(-\mu, \mu)$ con la misma probabilidad.

⁷El tema da para un capítulo completo, en alguna edición futura.

Según la ACS, la *propagación del error* se puede ir midiendo con reglas que dependen de la naturaleza de las operaciones y su inexactitud particular, conocida por anticipado.

Si dos números x y y tienen errores relativos δ_x y δ_y , entonces el error relativo de la suma $S = x + y$ pudiera calcularse por cuadratura:

$$\delta_S = \sqrt{\delta_x^2 + \delta_y^2}$$

si se supone distribución normal y una contribución pareja al error en la suma. Por ejemplo. $x = 2 \pm .1$ y $y = 3 \pm .2$. Considerando la aritmética convencional, se tiene $x + y = 5 \pm .3$. Pero si se supone una distribución normal del error, se tiene

$$\begin{aligned} x + y &= 5 \pm \sqrt{.1^2 + .2^2} \\ &= 5 \pm .2236... \end{aligned}$$

lo que mejora la aproximación. En esta aritmética, comunmente se habla de *incertidumbre* en vez de error. La ACS es mucho más optimista que la aritmética de intervalos.

Lo mejor (más realista) es suponer que cada cantidad contribuye de manera diferenciada al error final, por lo que debe calcularse la contribución por separado:

$$\delta_{S_x} = \left(\frac{\partial S}{\partial x} \right) \delta_x$$

en el caso continuo, que se discretiza para desarrollar programas de cómputo.

Si se trata del producto xy , el error relativo es

$$\delta = \sqrt{\left(\frac{\delta_x}{x} \right)^2 + \left(\frac{\delta_y}{y} \right)^2}$$

y hay fórmulas para cada operación y función elemental, que acotan mejor el error.

En 1984, Christopher Jones presentó el trabajo [192], en el que calcula el intervalo donde se encuentra el resultado inexacto de cada operación, pero los límites del intervalo se almacenan con precisión limitada por sus cifras significativas. Esto con la idea de reducir el costo de cómputo. Por ejemplo, si el resultado es $9.98328 \pm .0088542$, entonces es mejor escribir $9.98 \pm .009$. Se acostumbra la notación $9.98(9)$ para ahorrar espacio.

Otros trabajos sobre lo que llaman también *aritmética de rango*⁸, son los de Aberth [2] y [3], aunque los trabajos iniciales datan de hace más tiempo, como el de Gibb de 1961 [135].

Otra alternativa para esta aritmética es usar las fórmulas de propagación de errores junto con simulación de Monte Carlo.

⁸También le llaman Crank Three Times.

Suponer cualquier distribución de probabilidad para el error es mucha osadía. Debe sustentarse en algo, incluso para la distribución uniforme

El famoso paquete Mathematica, de Wolfram Technology, utiliza esta dentro de una de sus modalidades de cálculo, que de hecho liga los conceptos de error relativo con el de cifras significativas, para ir midiendo el error y mejorar la exactitud cada operación. En algunas aplicaciones de la física también se utiliza. En ambos casos, debe combinarse con cálculo simbólico para garantizar exactitud y precisión.

En su contra, hay un resultado poco alentador de lo que se puede llamar folklor popular que dice que sin importar cómo se programe la ACS, ocurre una de dos cosas:

1. n operaciones **amplifican** la incertidumbre en $\beta^{\frac{n}{2}}$
2. n operaciones **disminuyen** la incertidumbre en $\beta^{\frac{n}{2}}$.

Por ello tampoco resulta una buena opción, en particular por redondear con diferentes precisiones, en vez de una precisión acotada.

B.6. Aritmética de redundancia

En este sistema numérico cada dígito puede tener su propio signo. En base β se usan los dígitos $0, 1, 2, \dots, \beta - 1$. Según comenta Muller en [281], ya en 1840 Cauchy sugirió utilizar los dígitos del -5 al 5 para facilitar las multiplicaciones. En algunos sistemas digitales se usa -1, 0 y 1 en base 2 para facilitar algunos cálculos. Por ejemplo, una ventaja es que en algunos sistemas es posible realizar sumas sin necesidad de acarreo.

La idea básica es utilizar los dígitos desde $-a$ hasta a , con los que todo número tiene representación si

$$\begin{aligned} a &\leq \beta - 1 \\ 2a &\geq \beta - 1 \end{aligned}$$

se cumplen. Este sistema es redundante pues los números no tienen una única representación, lo que no significa un problema pues puede utilizarse convenientemente. $\beta = 2$ no permite que se cumplan las dos condiciones, pero hay otras alternativas para aritmética sin acarreo para base 2.

Lo más interesante es que en este sistema, la suma no requiere acarreo, por lo que sumar dos números con n dígitos se puede realizar sobre cada dígito al mismo tiempo, lo que lo hace muy veloz. Este descubrimiento se publicó en 1961, por Avizienis en [15] y desde entonces ha sido muy utilizado en sistemas digitales.

Bibliografía

- [1] ABERTH, O. Iteration methods for finding all zeros of a polynomial simultaneously. *Math. of Computations* 27 (1973), 339–344.
- [2] ABERTH, O., AND SCHAEFER, M. J. Precise computation using range arithmetic, via c++. *ACM Transactions on Mathematical Software* 18, 4 (1992), 481–491.
- [3] ABERTH, O., AND SCHAEFER, M. J. Precise matrix eigenvalues using range arithmetic. *SIAM Journal on Matrix Analysis and Applications* 14, 1 (1993), 235–241.
- [4] ADAMS, D. A. A stopping criterion for polynomial root finding. *Communications of the ACM* 10, 10 (octubre 1967), 655–658.
- [5] ALEFELD, G. E. On the convergence of Halley’s method. *American Mathematical Monthly* 88, 7 (septiembre 1981), 530–536.
- [6] ALEFELD, G. E., POTRA, F. A., AND SHI, Y. Algorithm 748: Enclosing zeros of continuous functions. *ACM Transactions on Mathematical Software* 21, 3 (septiembre 1995), 327–344.
- [7] ALLISON, C. Where did all my decimals go? *J. Comput. Small Coll.* 21, 3 (febrero 2006), 47–59.
- [8] ALT, H. Comparison of arithmetic functions with respect to boolean circuit depth. In *STOC ’84: Proceedings of the sixteenth annual ACM symposium on Theory of computing* (New York, NY, USA, 1984), ACM Press, pp. 466–470.
- [9] ALT, H. Comparing the combinational complexities of arithmetic functions. *Journal of the ACM* 35, 2 (abril 1988), 447–460.
- [10] AMAT, S., BUSQUIER, S., AND GUTIÉRREZ, J. M. Geometric constructions of iterative functions to solve nonlinear equations. *Journal of Computational and Applied Mathematics* 157, 1 (2003), 197–205.
- [11] ANDERSON, I. J. A distillation algorithm for floating point summation. *SIAM Journal on Scientific Computing* 20, 5 (1999), 1797–1806.

- [12] ARDEN, B., AND GRAHAM, R. On gat and the construction of translators. *Commun. ACM* 2, 7 (July 1959), 24–26.
- [13] ASHENHURST, R. L. Function evaluation in unnormalized arithmetic. *Journal of the ACM* 11, 2 (1964), 168–187.
- [14] ASHENHURST, R. L., AND METROPOLIS, N. Unnormalized floating point arithmetic. *Journal of the ACM* 6, 3 (julio 1959), 415–428.
- [15] AVIZIENIS, A. Signed-digit number representation for fast parallel arithmetic. *IRE Trans. Electron. Comput. EC-10* (1961), 389–400.
- [16] BACHELIS, S. G. B. An accurate elementary mathematical library for the IEEE floating point standard. *ACM Transactions on Mathematical Software* 17, 1 (marzo 1991), 26–45.
- [17] BACON, D. F., GRAHAM, S. L., AND SHARP, O. J. Compiler transformations for high-performance computing. *ACM Comput. Surv.* 26, 4 (Dec. 1994), 345–420.
- [18] BAILEY, D. H. Algorithm 719: Multiprecision translation and execution of FORTRAN programs. *ACM Trans. Math. Softw.* 19, 3 (Sept. 1993), 288–319.
- [19] BAILEY, D. H. A fortran-90 based multiprecision system. *ACM Transactions on Mathematical Software* 21, 4 (1995), 379–387.
- [20] BAILEY, D. H. Resolving numerical anomalies in scientific computation, 2008.
- [21] BAILEY, D. H., SIMON, H. D., AND BARTON, J. T. Floating point arithmetic in future supercomputers. *The International Journal of Supercomputer Applications* 3, 3 (Fall 1989), 86–90.
- [22] BAJARD, J.-C., DIDIER, L.-S., AND MULLER, J.-M. A new euclidean division algorithm for residue number systems. *J. VLSI Signal Process. Syst.* 19 (July 1998), 167–178.
- [23] BAREISS, E. H., AND BARLOW, J. L. Roundoff error distribution in fixed point multiplication. *BIT Numerical Mathematics* 20 (1980), 247–250. 10.1007/BF01933198.
- [24] BARRETT, G. Formal methods applied to a floating-point number system. *IEEE Trans. Softw. Eng.* 15, 5 (mayo 1989), 611–621.
- [25] BAUER, F. L. Computational graphs and rounding error. *SIAM Journal on Numerical Analysis* 11 (1974), 87–96.
- [26] BENFORD, F. The law of anomalous numbers. *Proceedings Of The American Philosophical Society* 78, 4 (1938), 551–572.

- [27] BERNSTEIN, H. J. Compiling fixed-point multiplications. *Commun. ACM* 15 (August 1972), 772–.
- [28] BINGHAM, J. A. C. An improvement to iterative methods of polynomial factorization. *Commun. ACM* 10, 1 (1967), 57–60.
- [29] BLACK, A. P. Exception handling: the case against. Tech. rep., Dept. Comput. Sci., Univ. Washington, 1983.
- [30] BLACK, C. M., BURTON, R., AND MILLER, T. H. The need for an industry standard of accuracy for elementary-function programs. *ACM Transactions on Mathematical Software* 10, 4 (diciembre 1984), 361–366.
- [31] BLACKFORD, L. S., CLEARY, A., PETITET, A., WHALEY, R. C., DEMMEL, J., DHILLON, I., REN, H., STANLEY, K., DONGARRA, J., AND HAMMARLING, S. Practical experience in the numerical dangers of heterogeneous computing. *ACM Trans. Math. Softw.* 23 (June 1997), 133–147.
- [32] BLUE, J. L. A portable fortran program to find the euclidean norm of a vector. *ACM Transactions on Mathematical Software* 4, 1 (marzo 1978), 15–23.
- [33] BOCHMANN, G. V. Multiple exits from a loop without the goto. *Commun. ACM* 16 (July 1973), 443–444.
- [34] BOHLENDER, G., WALTER, W., KORNERUP, P., AND MATULA, D. W. Semantics for exact floating point operations. In *IEEE Symposium on Computer Arithmetic* (1991), pp. 22–26.
- [35] BOLDO, SYLVIE; MULLER, J.-M. Exact and approximated error of the fina. *IEEE Trans. Comput.* 60 (February 2011), 157–164.
- [36] BOLDO, SYLVIE; DAUMAS, M. Properties of the subtraction valid for any floating point system. Tech. Rep. 4473, INRIA, jun 2002.
- [37] BOLDO, SYLVIE; DAUMAS, M. Representable correcting terms for possibly underflowing floating point operations. In *Proceedings of the 16th IEEE Symposium on Computer Arithmetic (ARITH-16'03)* (Washington, DC, USA, 2003), ARITH '03, IEEE Computer Society, pp. 79–.
- [38] BOLDO, SYLVIE; DAUMAS, M. Properties of two's complement floating point notations. *International Journal on Software Tools for Technology Transfers* 5, 2-3 (2004), 237–246.
- [39] BOLDO, S., DAUMAS, M., MOREAU-FINOT, C., AND THÉRY, L. Computer validated proofs of a toolset for adaptable arithmetic. *CoRR cs.MS/0107025* (2001), 21.
- [40] BORWEIN, P., AND ERDELYI, T. *Polynomials and Polynomial Inequalities*, vol. 161 of *Graduate Texts in Mathematics*. Springer Verlag, 1995. ISBN: 978-0-387-94509-5.

- [41] BRENNAN, P. T. Observations on program-wide ada exception propagation. In *Proceedings of the conference on TRI-Ada '93* (New York, NY, USA, 1993), TRI-Ada '93, ACM, pp. 189–195.
- [42] BRENT, R., PERCIVAL, C., ZIMMERMANN, P., AND BRENT, I. M. O. E. Errors bounds on complex floating-point multiplication. *Mathematics of Computation* 76 (2007), 1469–1481.
- [43] BRENT, R. P. An algorithm with guaranteed convergence for finding a zero of a function. *The Computer Journal* 14, 4 (1971), 422–425.
- [44] BRENT, R. P. On the precision attainable with various floating-point number systems. *IEEE Trans. Comput.* 22 (June 1973), 601–607.
- [45] BRILLOUT, A., KROENING, D., AND WAHL, T. Mixed abstractions for floating-point arithmetic. In *Proceedings of FMCAD 2009* (2009), IEEE, pp. 69–76.
- [46] BRISEBARRE, N., DEFOUR, D., AND REVOL, N. A new range-reduction algorithm. *IEEE Trans. Comput.* 54, 3 (marzo 2005), 331–339. Member-Peter Kornerup and Senior Member-Jean-Michel Muller.
- [47] BRISEBARRE, N., JOLDES, M., KORNERUP, P., MARTIN-DOREL, E., AND MULLER, J.-M. Augmented precision square roots and 2-d norms, and discussion on correctly rounding $\sqrt{x^2 + y^2}$. In *Proceedings of the 2011 IEEE 20th Symposium on Computer Arithmetic* (Washington, DC, USA, 2011), ARITH '11, IEEE Computer Society, pp. 23–30.
- [48] BRISEBARRE, N., MEZZAROBBA, M., MULLER, J.-M., AND LAUTER, C. Comparison between binary64 and decimal64 floating-point numbers. Consultado en 2012, 2012-10.
- [49] BRISEBARRE, N., AND MULLER, J.-M. Correct rounding of algebraic functions, 2007.
- [50] BRISEBARRE, N., MULLER, J.-M., AND TISSERAND, A. Computing machine-efficient polynomial approximations. *ACM Transactions on Mathematical Software* 32, 2 (junio 2006), 236–256.
- [51] BROWN, J. H., III, J. W. C., LARROWE, B., AND MCREYNOLDS, J. R. Prevention of propagation of machine errors in long problems. *Journal of the ACM* 3, 4 (octubre 1956), 348–354.
- [52] BROWN, W. S. A simple but realistic model of floating-point computation. *ACM Transactions on Mathematical Software* 7, 4 (diciembre 1981), 445–480.
- [53] BROWN, W. S., AND FELDMAN, S. I. Environment parameters and basic functions for floating-point computation. *ACM Transactions on Mathematical Software* 6, 4 (diciembre 1980), 510–523.

- [54] BROWN, W. S., AND RICHMAN, P. L. The choice of base. *Communications of the ACM* 12, 10 (octubre 1969), 560–561.
- [55] BUCHHOLZ, W. Fingers or fists? (the choice of decimal or binary representation). *Communications of the ACM* 2, 12 (diciembre 1959), 3–11.
- [56] BUHR, P. A., AND MOK, W. Y. R. Advanced exception handling mechanisms. *IEEE Trans. Softw. Eng.* 26, 9 (Sept. 2000), 820–836.
- [57] BURDEN, R. L., AND FAIRES, J. D. *Análisis Numérico*, 7 ed. International Thomson, México, 1999.
- [58] BURGER, R. G., AND DYBVIG, R. K. Printing floating-point numbers quickly and accurately. In *SIGPLAN Conference on Programming Language Design and Implementation* (1996), pp. 108–116.
- [59] BURKILL, J. C., Ed. *Functions of Intervals* (1924), vol. 2-22, London Mathematical Society, Proceedings of the London Mathematical Society.
- [60] BUS, J., AND DEKKER, T. Two efficient algorithms with guaranteed convergence. *ACM Transactions on Mathematical Software* 1, 4 (diciembre 1975), 330–345.
- [61] CASEY, J. K. Error stability in finite mantissa floating point computers. In *Preprints of papers presented at the 14th national meeting of the Association for Computing Machinery* (New York, NY, USA, 1959), ACM '59, ACM, pp. 1–5.
- [62] CATANZARO, B., AND NELSON, B. Higher radix floating-point representations for fpga-based arithmetic. In *Proceedings of the 13th Annual IEEE Symposium on Field-Programmable Custom Computing Machines* (Washington, DC, USA, 2005), IEEE Computer Society, pp. 161–170.
- [63] CHAPRA, S. C., AND CANALE, R. P. *Métodos Numéricos para Ingenieros*, 4 ed. WCB/McGraw-Hill, Boston, MA, USA, 2003.
- [64] CLENSHAW, C. W., AND OLVER, F. W. J. Beyond floating point. *Journal of the ACM* 31, 2 (abril 1984), 319–328.
- [65] CLINGER, W. D. How to read floating point numbers accurately. In *Proceedings of the ACM SIGPLAN 1990 Conference on Programming Language Design and Implementation* (New York, NY, USA, 1990), ACM Press, pp. 92–101.
- [66] CODD, E. F., AND HERRICK, H. L. Input scaling and output scaling a binary calculator. In *Proceedings of the 1952 ACM national meeting (Toronto)* (New York, NY, USA, 1952), ACM '52, ACM, pp. 21–22.
- [67] CODY, W. J. Double-precision square root for the cdc-3600. *Commun. ACM* 7, 12 (Dec. 1964), 715–718.

- [68] CODY, W. J. The influence of machine design on numerical algorithms. In *Proceedings of the April 18-20, 1967, spring joint computer conference* (New York, NY, USA, 1967), AFIPS '67 (Spring), ACM, pp. 305–309.
- [69] CODY, W. J. Static and dynamic numerical characteristics of floating-point arithmetic. *IEEE Trans. Comput.* 22, 6 (june 1973), 598–601.
- [70] CODY, W. J. The FUNPACK package of special function subroutines. *ACM Transactions on Mathematical Software* 1, 1 (marzo 1975), 13–25.
- [71] CODY, W. J. Software basics for computational mathematics. *SIGNUM Newsl.* 15, 2 (junio 1980), 18–29.
- [72] CODY, W. J. Algorithm 665: Machar: a subroutine to dynamically determined machine parameters. *ACM Transactions on Mathematical Software* 14, 4 (diciembre 1988), 303–311.
- [73] CODY, W. J. Algorithm 714; CELEFUNT: a portable test package for complex elementary functions. *ACM Transactions on Mathematical Software* 19, 1 (marzo 1993), 1–21.
- [74] CODY, W. J., COONEN, J. T., GAY, D. M., HANSON, K., HOUGH, D., KAHAN, W., KARPINSKI, R., PALMER, J., RIS, F. N., AND STEVENSON, D. A proposed radix- and word-length-independent standard for floating-point arithmetic. *SIGNUM Newsl.* 20, 1 (1985), 37–51.
- [75] CODY, W. J., AND STOLTZ, L. The use of Taylor series to test accuracy of function programs. *ACM Transactions on Mathematical Software* 17, 1 (marzo 1991), 55–63.
- [76] CODY, W. J., AND WAITE, W. *Software Manual for the Elementary Functions*. Prentice-Hall series in computational mathematics. Prentice-Hall, Inc., Upper Saddle River, NJ, USA, 1980.
- [77] COMMITTEE, M. S. Standard for floating-point arithmetic. Borrador, 2007.
- [78] COONEN, J. T. Underflow and the denormalized numbers. *Computer* 14, 3 (Mar. 1981), 75–87.
- [79] CORMEN, T. H., LEISERSON, C. E., RIVEST, R. L., AND STEIN, C. *Introduction to Algorithms*, 2 ed. MIT Press (McGraw Hill), Cambridge MA, 2001.
- [80] CORNEA-HASEGAN, M. Proving IEEE correctness of iterative floating-point square root, divide, and remainder algorithms. Tech. rep., Intel Corporation, 1998. Intel Technology Journal, Q2.
- [81] CORNEA-HASEGAN, M. IA-64 floating-point operations and the IEEE standard for binary floating-point arithmetic. Tech. rep., Intel Corporation, 1999. Intel Technology Journal, Q4.

- [82] CORPORATE IFIP WORKING GROUP 2, N. S. Comments on version 3.1 of draft iso/iec 10967: 1991 language compatible arithmetic. *SIGNUM Newsl.* 27, 1 (1992), 2–3.
- [83] COWLISHAW, M. F. Densely packed decimal encoding. *IEE Computers and Digital Techniques* 149, 3 (mayo 2002), 102–104.
- [84] COWLISHAW, M. F. Decimal floating-point: Algorism for computers. In *ARITH '03: Proceedings of the 16th IEEE Symposium on Computer Arithmetic (ARITH-16'03)* (Washington, DC, USA, 2003), IEEE Computer Society, p. 104. ISBN:0-7695-1894-X.
- [85] CUYT, A., AND VERDONK, B. A remarkable example of catastrophic cancellation unraveled. *Computing* 66, 3 (2001), 309–320.
- [86] DAUMAS, M., MAZENC, C., MERRHEIM, X., AND MULLER, J.-M. Modular range reduction. *Journal of Universal Computer Science* 1, 3 (marzo 1995), 162–175.
- [87] DE DINECHIN, F., ERSHOV, A. V., AND GAST, N. Towards the post-ultimate libm. In *ARITH '05: Proceedings of the 17th IEEE Symposium on Computer Arithmetic* (Washington, DC, USA, 2005), IEEE Computer Society, pp. 288–295.
- [88] DEFOUR, D., HANROT, G., LEFEVRE, V., MULLER, J.-M., REVOL, N., AND ZIMMERMANN, P. Proposal for a standardization of mathematical function implementation in floating-point arithmetic. *Numerical Algorithms* 37 (2004), 367–375.
- [89] DEKKER, T. J. A floating-point technique for extending the available precision. *Numerische Mathematik* 18, 3 (junio 1971), 224–242.
- [90] DEMMEL, J. Underflow and the reliability of numerical software. *SIAM J. Scientific Computing* 5, 4 (Dec 1982), 887–919.
- [91] DEMMEL, J., AND HIDA, Y. Accurate floating point summation. Tech. rep., University of California, Berkeley, CA 94720, mayo 2002. UCB//CSD-02-1180.
- [92] DEMMEL, J., AND HIDA, Y. Accurate and efficient floating point summation. *SIAM Journal on Scientific Computing* 25 (2003), 1214–1248.
- [93] DEMMEL, J., AND HIDA, Y. Fast and accurate floating point summation with application to computational geometry. *Numerical Algorithms* 37 (2004), 101–112.
- [94] DEMMEL, J. W., AND LI, X. Faster numerical algorithms via exception handling. *IEEE Trans. Comput.* 43 (August 1994), 983–992.
- [95] DIJKSTRA, E. W. Letters to the editor: go to statement considered harmful. *Commun. ACM* 11 (March 1968), 147–148.

- [96] DUNAWAY, D. K. Calculation of zeros of a real polynomial through factorization using euclid's algorithm. *SIAM Journal on Numerical Analysis* 11, 6 (1974), 1087–1104.
- [97] DUNHAM, C. B. On hardware for scientific computation. *SIGNUM Newsl.* 7 (April 1972), 20–20.
- [98] DUNHAM, C. B. Choice of basis for chebyshev approximation. *ACM Transactions on Mathematical Software* 8, 1 (marzo 1982), 21–25.
- [99] DUNHAM, C. B. Provably monotone approximations. *SIGNUM Newsl.* 22, 2 (abril 1987), 6–11.
- [100] DUNHAM, C. B. Feasibility of "perfect" function evaluation. *SIGNUM Newsl.* 25, 4 (junio 1990), 25–26.
- [101] EGGERS, T. W., LEONARD, J. S., AND PAYNE, M. H. Handling of floating point exceptions. In *Proceedings of the SIGNUM Conference on the Programming Environment for Development of Numerical Software* (New York, NY, USA, 1979), ACM Press, pp. 100–108.
- [102] EHRLICH, L. W. A modified Newton method for polynomials. *Communications of the ACM* 10, 2 (1967), 107–108.
- [103] EINARSSON, B., Ed. *Accuracy and Reliability in Scientific Computing (Software, Environments, Tools) (Software, Environments, Tools)*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, enero 2005.
- [104] ERHEL, J. Experiments with data perturbations to study condition numbers and numerical stability. *Computing* 51 (1993), 29–44. 10.1007/BF02243827.
- [105] EVE, J. Starting approximations for the iterative calculation of square roots. *Comput. J.* 6 (1963), 274–276.
- [106] EVEN, GUY; SEIDEL, P.-M. A comparison of three rounding algorithms for IEEE floating-point multiplication. *IEEE Trans. Comput.* 49 (July 2000), 638–650.
- [107] FATEMAN, R. J. An algorithm for deciding the convergence of the rational iteration $x_{n+1} = f(x_n)$. *ACM Trans. Math. Softw.* 3, 3 (septiembre 1977), 272–278.
- [108] FATEMAN, R. J. High-level language implications of the proposed IEEE floating-point standard. *ACM Trans. Program. Lang. Syst.* 4, 2 (abril 1982), 239–257.
- [109] FEDELE, A. E. G. Accurate floating-point summation: a new approach. *Applied Mathematics and Computation* 189 (2007), 410–424.

- [110] FEIGENBAUM, L. Taylor and the method of increments. *Arch. Hist. Exact Sci.* 34 (1985), 1–140.
- [111] FELDSTEIN, A., AND GOODMAN, R. Convergence estimates for the distribution of trailing digits. *J. ACM* 23 (April 1976), 287–297.
- [112] FERGUSON, JR., W. E. Exact computation of a sum or difference with applications to argument reduction. In *Proceedings of the 12th Symposium on Computer Arithmetic* (Washington, DC, USA, 1995), ARITH '95, IEEE Computer Society, pp. 216–.
- [113] FIELD, J. A. Optimizing floating point arithmetic via post addition shift probabilities. In *Proceedings of the May 14-16, 1969, spring joint computer conference* (New York, NY, USA, 1969), AFIPS '69 (Spring), ACM, pp. 597–603.
- [114] FIGUEROA, S. A. When is double rounding innocuous? *SIGNUM Newsl.* 30, 3 (July 1995), 21–26.
- [115] FISCHER, P. C. Automatic propagated and round-off error analysis. In *Preprints of papers presented at the 13th national meeting of the Association for Computing Machinery* (New York, NY, USA, 1958), ACM '58, ACM, pp. 1–2.
- [116] FISHMAN, G. *Monte Carlo: Concepts, algorithms, and applications*, 4 ed. Springer Series in Operations Research. Springer, 2003.
- [117] FLOYD, R. W. An algorithm for coding efficient arithmetic operations. *Commun. ACM* 4, 1 (Jan. 1961), 42–51.
- [118] FORD, B. Parameterization of the environment for transportable numerical software. *ACM Transactions on Mathematical Software* 4, 2 (junio 1978), 100–103.
- [119] FORD, J. A. Improved algorithms of Illinois-type for the numerical solution of nonlinear equations. Technical report csm-257, University of Essex, 1995.
- [120] FOUSSE, L., HANROT, G., LEFÈVRE, V., PLISSIE, P., AND ZIMMERMANN, P. MPFR: A multiple-precision binary floating-point library with correct rounding. *ACM Transactions on Mathematical Software* 33, 2 (junio 2007), 1.
- [121] FRALEY, B., AND WALTHER, S. Proposal to eliminate denormalized numbers. *SIGNUM Newsl.* 14 (October 1979), 22–23.
- [122] FRANK, W. L. Finding zeros of arbitrary functions. *J. ACM* 5, 2 (Apr. 1958), 154–160.

- [123] FRASER, W. A survey of methods of computing minimax and near-minimax polynomial approximations for functions of a single independent variable. *Journal of the ACM* 12, 3 (julio 1965), 295–314.
- [124] FRIEDLAND, P. Algorithm 312: Absolute value and square root of a complex number. *Commun. ACM* 10, 10 (Oct. 1967), 665–.
- [125] FRONTINI, M., AND SORMANI, E. Modified Newton’s method with third-order convergence and multiple roots. *Journal of Computational and Applied Mathematics* 156, 2 (julio 2003), 345–354.
- [126] FRONTINI, M., AND SORMANI, E. Some variant of Newton’s method with third-order convergence. *Applied Mathematics Letters* 140, 2-3 (agosto 2003), 419–426.
- [127] G. ALEFELD, J. H. *Introduction to Interval Computations*. Academic Press, New York, 1974.
- [128] G. E. FORSYTHE, M. A. M. Y. C. B. M. *Computer Methods for Mathematical Computations*. Prentice-Hall, Englewood Cliffs, NJ, 1977.
- [129] GALAL, SAMEH; HOROWITZ, M. Latency sensitive fma design. In *Proceedings of the 2011 IEEE 20th Symposium on Computer Arithmetic* (Washington, DC, USA, 2011), ARITH ’11, IEEE Computer Society, pp. 129–138.
- [130] GAY, D. M. Correctly rounded binary-decimal and decimal-binary conversions. Tech. Rep. 90-10, AT&T Bell Laboratories, Murray Hill, NJ, USA, noviembre 1990.
- [131] GEHANI, N. H. Exceptional c or c with exceptions. *Softw. Pract. Exper.* 22, 10 (Oct. 1992), 827–848.
- [132] GENTLEMAN, W. M., AND MAROVICH, S. B. More on algorithms that reveal properties of floating point arithmetic units. *Communications of the ACM* 17, 5 (mayo 1974), 276–277.
- [133] GERALD, C. F., AND WHEATLEY, P. O. *Análisis numérico con aplicaciones*. Pearson Educación, 2000.
- [134] GHAZI, K. R., LEFEVRE, V., THEVENY, P., AND ZIMMERMANN, P. Why and how to use arbitrary precision. *IEEE Des. Test* 12, 3 (May 2010), 5–.
- [135] GIBB, A. Algorithm 61: procedures for range arithmetic. *Communications of the ACM* 4, 7 (1961), 319–320.
- [136] GIVENS, W. *Numerical computation of the characteristic values of a real symmetric matrix*. Oak Ridge National Laboratory, 1954.

- [137] GLASER, A., LIU, X., AND ROKHLIN, V. A fast algorithm for the calculation of the roots of special functions. *SIAM Journal on Scientific Computing* 29, 4 (2007), 1420–1438.
- [138] GODEFROID, P., AND KINDER, J. Proving memory safety of floating-point computations by combining static and dynamic program analysis. In *Proceedings of the 19th international symposium on Software testing and analysis* (New York, NY, USA, 2010), ISSTA '10, ACM, pp. 1–12.
- [139] GÖK, MUSTAFA; ÖZBILEN, M. M. Evaluation of sticky-bit generation methods for floating-point multipliers. *J. Signal Process. Syst.* 56 (July 2009), 51–57.
- [140] GOLDBERG, D. What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys* 23, 1 (Marzo 1991), 5–48.
- [141] GOLDBERG, D. The design of floating-point data types. *ACM Lett. Program. Lang. Syst.* 1, 2 (junio 1992), 138–151.
- [142] GOLDBERG, I. B. 27 bits are not enough for 8-digit accuracy. *Commun. ACM* 10 (February 1967), 105–106.
- [143] GOLDSTEIN, M. Significance arithmetic on a digital computer. *Communications of the ACM* 6, 3 (1963), 111–117.
- [144] GOODENOUGH, J. B. Exception handling design issues. *SIGPLAN Not.* 10, 7 (july 1975), 41–45.
- [145] GOODENOUGH, J. B. Exception handling: issues and a proposed notation. *Commun. ACM* 18, 12 (Dec. 1975), 683–696.
- [146] GOODENOUGH, J. B. Structured exception handling. In *Proceedings of the 2nd ACM SIGACT-SIGPLAN symposium on Principles of programming languages* (New York, NY, USA, 1975), POPL '75, ACM, pp. 204–224.
- [147] GOODMAN, R. On round-off error in fixed-point multiplication. *BIT Numerical Mathematics* 16 (1976), 41–51. 10.1007/BF01940776.
- [148] GOODMAN, R., AND FELDSTEIN, A. Round-off error in products. *Computing* 15 (1975), 263–273. 10.1007/BF02242373.
- [149] GOODMAN, R., AND FELDSTEIN, A. Effect of guard digits and normalization options on floating point multiplication. *Computing* 18 (1977), 93–106. 10.1007/BF02243619.
- [150] GRAILLAT, S., LAMOTTE, J.-L., AND HONG, D. N. Numerical validation in current hardware architectures. In *Numerical Validation in Current Hardware Architectures*, A. Cuyt, W. Krämer, W. Luther, and P. Markstein, Eds. Springer-Verlag, Berlin, Heidelberg, 2009, ch. Error-Free Transformation in Rounding Mode toward Zero, pp. 217–229.

- [151] GRAILLAT, S., LANGLOIS, P., AND LOUVET, N. Improving the compensated horner scheme with a fused multiply and add. In *SAC '06: Proceedings of the 2006 ACM symposium on Applied computing* (New York, NY, USA, 2006), ACM, pp. 1323–1327.
- [152] GRAILLAT, S., AND MENISSIER-MORAIN, V. Accurate summation, dot product and polynomial evaluation in complex floating point arithmetic. Tech. rep., Université Pierre et Marie Curie, 2007. In preparation.
- [153] GRAILLAT, S., AND MENISSIER-MORAIN, V. Error-free transformations in real and complex floating point arithmetic. In *Proceedings of the International Symposium on Nonlinear Theory and its Applications* (Vancouver, Canada, 2007), pp. 341–344.
- [154] GRAM, C. On the representation of zero in floating-point arithmetic. *BIT Numerical Mathematics* 4 (1964), 156–161. 10.1007/BF01956026.
- [155] GRAY, H. L., AND HARRISON, JR., C. Normalized floating-point arithmetic with an index of significance. In *Papers presented at the December 1-3, 1959, eastern joint IRE-AIEE-ACM computer conference* (New York, NY, USA, 1959), IRE-AIEE-ACM '59 (Eastern), ACM, pp. 244–248.
- [156] GREGORY, R. T., AND RANEY, J. L. Floating-point arithmetic with 84-bit numbers. *Communications of the ACM* 7, 1 (enero 1964), 10–13.
- [157] GUTIÉRREZ, J. M., AND HERNÁNDEZ, M. A. Newton's method under weak Kantorovich conditions. *Journal of Numerical Analysis* 20 (2000), 521–532.
- [158] GUTIÉRREZ, J. M., AND HERNÁNDEZ, M. A. An acceleration of newton's method: Super-halley method. *Applied Mathematics and Computation* 117, 2-3 (enero 2001), 223–239.
- [159] GUY L. STEELE JR., J. L. W. How to print floating-point numbers accurately. In *Proceedings of the ACM SIGPLAN 1990 Conference on Programming Language Design and Implementation* (New York, NY, USA, 1990), ACM Press, pp. 112–126.
- [160] HARRISON, J. A machine-checked theory of floating point arithmetic. In *Proceedings of the 12th International Conference on Theorem Proving in Higher Order Logics* (London, UK, 1999), TPHOLs '99, Springer-Verlag, pp. 113–130.
- [161] HATTON, L. Embedded system paranoia: a tool for testing embedded system arithmetic. <http://www.leshatton.org/Documents/esp.pdf>, febrero 2004.
- [162] HAUSER, J. R. Handling floating-point exceptions in numeric programs. *ACM Trans. Program. Lang. Syst.* 18, 2 (Marzo 1996), 139–174.

- [163] HENRICI, P. A subroutine for computations with rational numbers. *Journal of the ACM* 3, 1 (enero 1956), 6–9.
- [164] HENRICI, P. Test of probabilistic models for the propagation of roundoff errors. *Commun. ACM* 9 (June 1966), 409–410.
- [165] HICKEY, T., JU, Q., AND EMDEN, M. H. V. Interval arithmetic: From principles to implementation. *Journal of the ACM* 48, 5 (2001), 1038–1068.
- [166] HIDA, Y., LI, X. S., AND BAILEY, D. H. Algorithms for quad-double precision floating point arithmetic. In *Proceedings of the 15th Symposium on Computer Arithmetic* (2001), IEEE Computer Society Press, pp. 155–162.
- [167] HIGHAM, N. J. The accuracy of floating point summation. *SIAM Journal on Scientific Computing* 14, 4 (1993), 783–799.
- [168] HIGHAM, N. J. *Accuracy and Stability of Numerical Algorithms*, 1 ed. ed. SIAM, 1996.
- [169] HILL, I. D. Faults in functions, in algol and fortran. *The Computer Journal* 14, 3 (1971), 315–316.
- [170] HOCKERT, N., AND COMPTON, K. Improving floating-point performance in less area: Fractured floating point units (ffpus). *J. Signal Process. Syst.* 67, 1 (Apr. 2012), 31–46.
- [171] HOTELLING, H. Some new methods in matrix calculation. *Annals of Mathematical Statistics* 14, 1 (1943), 1–34.
- [172] HOUSEHOLDER, A. S. Generation of errors in digital computation. *Bulletin of American Mathematical Society* 60 (1954), 234–247.
- [173] HOUSEHOLDER, A. S. Bibliography on numerical analysis. *J. ACM* 3 (April 1956), 85–100.
- [174] HOUSEHOLDER, A. S. *The Numerical Treatment of a Single Nonlinear Equation*. McGraw-Hill, New York, 1970.
- [175] HULL, D. E. Desirable floating-point arithmetic and elementary functions for numerical computation. *SIGNUM Newsl.* 14 (October 1978), 96–99.
- [176] HULL, T. E., AND ABRHAM, A. Variable precision exponential function. *ACM Transactions on Mathematical Software* 12, 2 (junio 1986), 79–91.
- [177] HULL, T. E., ABRHAM, A., COHEN, M. S., X., CURLEY, A. F., HALL, C. B., PENNY, D. A., AND M., SAWCHUK, J. T. Numerical turing. *SIGNUM Newsl.* 20 (July 1985), 26–34.

- [178] HULL, T. E., COHEN, M. S., SAWSHUK, J. T. M., AND WORTMAN, D. B. Exception handling in scientific computing. *ACM Transactions on Mathematical Software* 14, 3 (septiembre 1988), 201–217.
- [179] HULL, T. E., ENRIGHT, W. H., AND SEDGWICK, A. E. The correctness of numerical algorithms. *SIGPLAN Not.* 7, 1 (Jan. 1972), 66–73.
- [180] HULL, T. E., FAIRGRIEVE, T. F., AND TANG, P.-T. P. Implementing complex elementary functions using exception handling. *ACM Trans. Math. Softw.* 20, 2 (June 1994), 215–244.
- [181] HULL, T. E., FAIRGRIEVE, T. F., AND TANG, P. T. P. Implementing the complex arcsine and arccosine functions using exception handling. *ACM Transactions on Mathematical Software* 23, 3 (septiembre 1997), 299–335.
- [182] HULL, T. E., AND SWENSON, J. R. Tests of probabilistic models for propagation of roundoff errors. *Commun. ACM* 9 (February 1966), 108–113.
- [183] IFIP WORKING GROUP 2.5, C. The enable construct for exception handling in fortran 90. *SIGNUM Newsl.* 28 (October 1993), 7–16.
- [184] IKEBE, Y. Note on triple-precision floating-point arithmetic with 132-bit numbers. *Communications of the ACM* 8, 3 (marzo 1965), 175–177.
- [185] IORDACHE, C., AND MATULA, D. W. On infinitely precise rounding for division, square root, reciprocal and square root reciprocal. In *Proceedings of the 14th IEEE Symposium on Computer Arithmetic* (Washington, DC, USA, 1999), ARITH '99, IEEE Computer Society, pp. 233–.
- [186] ISSARNY, V. An exception handling model for parallel programming and its verification. *SIGSOFT Softw. Eng. Notes* 16 (September 1991), 92–100.
- [187] JABREF DEVELOPMENT TEAM. *JabRef*. JabRef Development Team, 2010.
- [188] JACOBI, C., WEBER, K., PARUTHI, V., AND BAUMGARTNER, J. Automatic formal verification of fused-multiply-add fpus. In *DATE '05: Proceedings of the conference on Design, Automation and Test in Europe* (Washington, DC, USA, 2005), vol. 2, IEEE Computer Society, pp. 1298–1303.
- [189] JEANNEROD, C.-P., LOUVET, N., AND MULLER, J.-M. On the componentwise accuracy of complex floating-point division with an FMA. Consultado en 2012, 09 2012.
- [190] JENKINS, M. A., AND TRAUB, J. F. Principles for testing polynomial zerofinding programs. *ACM Trans. Math. Softw.* 1, 1 (1975), 26–34.

- [191] JOHN W. CARR, I. Error analysis in floating point arithmetic. *Communications of the ACM* 2, 5 (mayo 1957), 10–15.
- [192] JONES, C. B. A significance rule for multiple-precision arithmetic. *ACM Transactions on Mathematical Software* 10, 1 (1984), 97–107.
- [193] K. HILLESLAND, A. L., Ed. *GPU floating-point paranoia* (2004), vol. p. C-8 of *Proceedings of the ACM Workshop on General Purpose Computing on Graphics Processors*.
- [194] KAHAN, W. A logarithm too clever by half. Disponible en la dirección <http://www.eecs.berkeley.edu/~wkahan/LOG10HAF.TXT>.
- [195] KAHAN, W. Further remarks on reducing truncation errors. *Communications of the ACM* 8, 1 (enero 1963), 40.
- [196] KAHAN, W. In memoriam: Hirono kuki: Apr. 25, 1925 - dec. 28, 1971. *SIGNUM Newsl.* 7, 1 (abril 1972), 8–10.
- [197] KAHAN, W. Why we need a floating-point arithmetic standard. Tech. rep., Univ. of California at Berkeley, Univ. of California at Berkeley, Marzo 5 1981.
- [198] KAHAN, W. Minimizing $q \times m - n$. Comentario inicial del programa nearpi.c, en <http://www.cs.berkeley.edu/~wkahan/testpi/nearpi.c>, 1983.
- [199] KAHAN, W. Branch cuts for complex elementary functions. Notas de curso, 1987.
- [200] KAHAN, W. Analysis and refutation of the LCAS. *SIGNUM Newsl.* 26, 3 (1991), 2–15.
- [201] KAHAN, W. The improbability of probabilistic error analyses for numerical computations. Tech. rep., Univ. of Calif. Berkeley, 1011 Evans Hall, Feb. 1996. UCB Statistics Colloquium.
- [202] KAHAN, W. Lecture notes on the status of IEEE standard 754 for binary floating-point arithmetic. Tech. rep., University of California at Berkeley, Octubre 1 1997.
- [203] KAHAN, W. Matlab's loss is nobody's gain. Tech. rep., Univ. of Calif. Berkeley, julio 2002. Creado en 1998.
- [204] KAHAN, W. An ordinary differential equation in interval-arithmetic. Tech. rep., Univ. of Calif. Berkeley, marzo 2002. Notas de curso.
- [205] KAHAN, W. Lecture notes on real root-finding. Tech. rep., University of California at Berkeley, junio 2004.
- [206] KAHAN, W. A demonstration of presubstitution. Lecture notes, Univ. of Calif. Berkeley, julio 2005.

- [207] KAHAN, W., COONEN, J., PALMER, J., PITTMAN, T., AND STEVENSON, D. A proposed standard for binary floating point arithmetic. *SIGNUM Newsl.* 14, si-2 (1979), 4–12.
- [208] KAHAN, W., AND DARCY, J. How Java's floating-point hurts everyone everywhere. Página web, marzo 1998.
- [209] KAHAN, W., AND FARKAS, I. Algorithm 167: calculation of confluent divided differences. *Commun. ACM* 6, 4 (1963), 164–165.
- [210] KAHAN, W., AND PALMER, J. On a proposed floating-point standard. *SIGNUM Newsl.* 14 (October 1979), 13–21.
- [211] KAIVOLA, R., AND NARASIMHAN, N. Formal verification of the Pentium 4 floating-point multiplier. In *DATE '02: Proceedings of the conference on Design, automation and test in Europe* (Washington, DC, USA, 2002), IEEE Computer Society, p. 20.
- [212] KANEKO, T., AND LIU, B. On local roundoff errors in floating-point arithmetic. *Journal of the ACM* 20, 3 (1973), 391–398.
- [213] KANNER, H. An algebraic translator. *Commun. ACM* 2, 10 (Oct. 1959), 19–22.
- [214] KANTOROVICH, L. V. On a mathematical symbolism convenient for performing machine calculations. *Transactions of the Academy of Sciences* 4, 113 (1957), 738–741. en ruso.
- [215] KEARFOTT, R. B., DAWANDE, M., DU, K., AND HU, C. Algorithm 737: Intlib—a portable Fortran 77 interval standard-function library. *ACM Transactions on Mathematical Software* 20, 4 (diciembre 1994), 447–459.
- [216] KERN, C., AND GREENSTREET, M. R. Formal verification in hardware design: a survey. *ACM Trans. Des. Autom. Electron. Syst.* 4, 2 (abril 1999), 123–193.
- [217] KINCAID, D., AND CHENEY, W. *Análisis Numérico*. Addison Wesley Iberoamericana, 1994.
- [218] KNUTH, D. *Structured programming with go to statements*. Yourdon Press, Upper Saddle River, NJ, USA, 1979, pp. 257–321.
- [219] KNUTH, D. E. *The art of computer programming: seminumerical algorithms*, 3 ed., vol. 2. Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA, noviembre 1997.
- [220] KOENING, A., AND STROUSTRUP, B. Exception handling for c++. *J. Object Oriented Program.* 3, 2 (June 1990), 16–33.

- [221] KOOPMAN, P., AND DEVALE, J. The exception handling effectiveness of posix operating systems. *IEEE Trans. Softw. Eng.* 26, 9 (Sept. 2000), 837–848.
- [222] KOREN, I. *Computer Arithmetic Algorithms*, 2 ed. A K Peters, Ltd., 2002. Primera edición de Prentice-Hall.
- [223] KORNERUP, P., LEFEVRE, V., LOUVET, N., AND MULLER, J. M. On the computation of correctly-rounded sums. In *Proceedings of the 2009 19th IEEE Symposium on Computer Arithmetic* (Washington, DC, USA, 2009), ARITH '09, IEEE Computer Society, pp. 155–160.
- [224] KORNERUP, P., LEFÈVRE, V., AND MULLER, J.-M. Computing integer powers in floating-point arithmetic. Tech. Rep. RR2007-23, INRIA, mayo 2007.
- [225] KORNERUP, P., AND MULLER, J.-M. Choosing starting values for certain newton-raphson iterations. *Theor. Comput. Sci.* 351, 1 (Feb. 2006), 101–110.
- [226] KRISCHER, R., AND BUHR, P. A. Asynchronous exception propagation in blocked tasks. In *Proceedings of the 4th international workshop on Exception handling* (New York, NY, USA, 2008), WEH '08, ACM, pp. 8–15.
- [227] KUCK, D., PARKER, S. J., AND SAMEH, A. Analysis of rounding methods in floating-point arithmetic. *IEEE Trans. Comput.* 26, 7 (july 1977), 643–650.
- [228] KUKI, H., AND CODY, W. J. A statistical study of the accuracy of floating point number systems. *Communications of the ACM* 16, 4 (abril 1973), 223–230.
- [229] KULISCH, U. *Advanced Arithmetic for Digital Computer: Design of Arithmetic units*. Springer-Verlag, 2002.
- [230] KULISCH, U. W. A new arithmetic for scientific computation with exact evaluation of expressions. In *Invited Lectures from the European Conference on Computer Algebra-Volume I - Volume I* (London, UK, 1985), Springer-Verlag, pp. 114–123.
- [231] LAGRANGE, S., DELANOUE, N., AND JAULIN, L. On sufficient conditions of the injectivity: Development of a numerical test algorithm via interval analysis. *Reliable computing* 13, 5 (octubre 2007), 409–421. Publicado en internet.
- [232] LAKE, G. T. Hardware conversion of decimal and binary numbers. *Communications of the ACM* 5, 9 (septiembre 1962), 468–469.
- [233] LAM, M. O. Dynamic floating-point cancellation detection, 2010.

- [234] LARSON, J., AND SAMEH, A. Efficient calculation of the effects of roundoff errors. *ACM Trans. Math. Softw.* 4, 3 (Sept. 1978), 228–236.
- [235] LARSON, J., AND SAMEH, A. Algorithms for roundoff error analysis — a relative error approach. *Computing* 24, 4 (1980), 275–297. 10.1007/BF02237815.
- [236] LAUTER, C. Q. Basic building blocks for a triple-double intermediate format. Tech. Rep. RR-5702, INRIA, Sep 2005.
- [237] LE, D. An efficient derivative-free method for solving nonlinear equations. *ACM Transactions on Mathematical Software* 11, 3 (1985), 250–262.
- [238] LEE, S., YANG, B.-S., KIM, S., PARK, S., MOON, S.-M., EBCIOĞLU, K., AND ALTMAN, E. Efficient java exception handling in just-in-time compilation. In *Proceedings of the ACM 2000 conference on Java Grande* (New York, NY, USA, 2000), JAVA '00, ACM, pp. 1–8.
- [239] LEFÈVRE, V., AND MULLER, J.-M. Worst cases for correct rounding of the elementary functions in double precision. In *Proceedings of the 15th Symposium on Computer Arithmetic* (Vail, Colorado, 2001), N. Burgess and L. Ciminiera, Eds., pp. 111–118.
- [240] LERCH, M., TISCHLER, G., GUDENBERG, J. W. V., HOFSCHESTER, W., AND KRÄMER, W. FILIB++, a fast interval library supporting containment computations. *ACM Transactions on Mathematical Software* 32, 2 (junio 2006), 299–324.
- [241] LEVIN, R. *Program Structures for Exceptional Condition Handling*. PhD thesis, Carnegie Mellon University, Pittsburgh, PA, USA, 1977. AAI7804937.
- [242] LI, R.-C., BOLDO, S., AND DAUMAS, M. Theorems on efficient argument reductions. In *Proceedings of the 16th IEEE Symposium on Computer Arithmetic (ARITH-16'03)* (Washington, DC, USA, 2003), ARITH '03, IEEE Computer Society, pp. 129–.
- [243] LI, X. S., DEMMEL, J. W., BAILEY, D. H., HENRY, G., HIDA, Y., ISKANDAR, J., KAHAN, W., KANG, S. Y., KAPUR, A., MARTIN, M. C., THOMPSON, B. J., TUNG, T., AND YOO, D. J. Design, implementation and testing of extended and mixed precision blas. *ACM Transactions on Mathematical Software* 28, 2 (junio 2002), 152–205.
- [244] LINNAINMAA, S. Towards accurate statistical estimation of rounding errors in floating-point computations. *Bit Numerical Mathematics* 15 (1975), 165–173.
- [245] LINNAINMAA, S. Software for doubled-precision floating-point computations. *ACM Transactions on Mathematical Software* 7, 3 (septiembre 1981), 272–283.

- [246] LINZ, P. Accurate floating-point summation. *Communications of the ACM* 13, 6 (junio 1970), 361–362.
- [247] LISKOV, B., AND SNYDER, A. *Structured Except Handling*. Cambridge, Dec 1977.
- [248] LOAN, G. H. G. Y. C. F. V. *Matrix Computations*, segunda edición ed. Johns Hopkins Univ. Press, 1989.
- [249] LOH, E., AND WALSTER, G. W. Rump’s example revisited. *Reliable Computing* 8 (2002), 245–248. 10.1023/A:1015569431383.
- [250] LOTKIN, M., AND RAMAGE, R. Scaling and error analysis for matrix inversion by partitioning. *Annals of Mathematical Statistics* 24, 3 (1953), 428–439.
- [251] LUCKHAM, D. C., AND POLAK, W. Ada exception handling: an axiomatic approach. *ACM Trans. Program. Lang. Syst.* 2, 2 (Apr. 1980), 225–233.
- [252] LYNCH, W. C. On a wired-in binary-to-decimal conversion scheme. *Communications of the ACM* 5, 3 (marzo 1962), 159.
- [253] MAEHLY, H. J. Approximations for the cdc 1604. Tech. rep., Control Data Corp., 1960.
- [254] MALCOLM, M. A. On accurate floating-point summation. *Communications of the ACM* 14, 11 (noviembre 1971), 731–736.
- [255] MALCOLM, M. A. Algorithms to reveal properties of floating-point arithmetic. *Communications of the ACM* 15, 11 (noviembre 1972), 949–951.
- [256] MARKSTEIN, P. *IA-64 and Elementary Functions: Speed and Precision*. Hewlett-Packard Professional Books. Prentice Hall, 2000.
- [257] MARTEL, M. Semantics of roundoff error propagation in finite precision calculations. *Higher Order Symbol. Comput.* 19, 1 (Mar. 2006), 7–30.
- [258] MARTEL, M. Enhancing the implementation of mathematical formulas for fixed-point and floating-point arithmetics. *Form. Methods Syst. Des.* 35, 3 (Dec. 2009), 265–278.
- [259] MARTEL, M. Program transformation for numerical precision. In *Proceedings of the 2009 ACM SIGPLAN workshop on Partial evaluation and program manipulation* (New York, NY, USA, 2009), PEPM ’09, ACM, pp. 101–110.
- [260] MATULA, D. W. Base conversion mappings. In *Proceedings of the April 18-20, 1967, spring joint computer conference* (New York, NY, USA, 1967), AFIPS ’67 (Spring), ACM, pp. 311–318.

- [261] MATULA, D. W. The base conversion theorem. *Proc. Amer. Math. Soc.* 19 (1968), 716–723.
- [262] MATULA, D. W. In-and-out conversions. *Communications of the ACM* 11, 1 (enero 1968), 47–50.
- [263] MATULA, D. W. Towards an abstract mathematical theory of floating-point arithmetic. In *Proceedings of the May 14-16, 1969, spring joint computer conference* (New York, NY, USA, 1969), AFIPS '69 (Spring), ACM, pp. 765–772.
- [264] MATULA, D. W. A formalization of floating-point numeric base conversion. *IEEE Trans. Comput.* 19 (August 1970), 681–692.
- [265] MATULA, D. W., AND MCFEARIN, L. D. A formal model and efficient traversal algorithm for generating testbenches for verification of IEEE standard floating point division. In *DATE '06: Proceedings of the conference on Design, automation and test in Europe* (3001 Leuven, Belgium, Belgium, 2006), European Design and Automation Association, pp. 1134–1138.
- [266] MCCANN, G. D., BARNES, J. L., STEELE, F., RIDENOUR, L., AND VANCE, A. W. An evaluation of analog and digital computers. In *Proceedings of the February 4-6, 1953, western computer conference* (New York, NY, USA, 1953), AIEE-IRE '53 (Western), ACM, pp. 19–48.
- [267] MCNAMEE, J. M. A comparison of methods for accurate summation. *ACM SIGSAM Bulletin* 38, 1 (marzo 2004), 1–1.
- [268] MENARD, D., CHILLET, D., AND SENTIEYS, O. Floating-to-fixed-point conversion for digital signal processors. *EURASIP J. Appl. Signal Process.* 2006 (Jan. 2006), 77–77.
- [269] MIDY, P., AND YAKOVLEV, Y. Computing some elementary functions of a complex variable. *Math. Comput. Simul.* 33, 1 (July 1991), 33–49.
- [270] MIGNOTTE, M. Some inequalities about univariate polynomials. In *SYMSAC '81: Proceedings of the fourth ACM symposium on Symbolic and algebraic computation* (New York, NY, USA, 1981), ACM, pp. 195–199.
- [271] MIGNOTTE, M. On the distance between the roots of a polynomial. *Applicable Algebra in Engineering, Communication and Computing* 6 (1995), 327–332.
- [272] MIKOV, A. I. Largescale addition of machine real numbers: accuracy estimates. *Theor. Comput. Sci.* 162 (1996), 151–170.
- [273] MILLER, W. Software for roundoff analysis. *ACM Transactions on Mathematical Software* 1, 2 (junio 1975), 108–128.

- [274] MILLER, W., AND SPOONER, D. Software for roundoff analysis, ii. *ACM Transactions on Mathematical Software* 4, 4 (diciembre 1978), 369–387.
- [275] MOLER, C., AND MORRISON, D. Replacing square roots by pythagorean sums. *Journal or Research and Development* 28 (1983), 577–582.
- [276] MOLLER, O. Quasi double-precision in floating point addition. *BIT Numerical Mathematics* 5 (1965), 37–50. 10.1007/BF01975722.
- [277] MOURSUND, D. G. Optimal starting values for newton-raphson calculation of x_1^2 . *Commun. ACM* 10, 7 (July 1967), 430–432.
- [278] MULLER, J. M.; LAUTER, C. On ziv’s rounding test. Tech. rep., Université Pierre et Marie Curie Laboratoire d’Informatique de Paris, may 2012.
- [279] MÜLLER, D. E. A method for solving algebraic equations using an automatic computer. *Math. Tables Aids Comput.* 10 (1956), 208–215.
- [280] MULLER, J.-M. On the definition of $ulp(x)$. Tech. Rep. 5504, INRIA, Febrero 2005.
- [281] MULLER, J.-M. *Elementary functions*, 2 ed. Birkhauser, 2006.
- [282] MULLER, J.-M., BRISEBARRE, N., DE DINECHIN, F., JEANNEROD, C.-P., LEFÈVRE, V., MELQUIOND, G., REVOL, N., STEHLÉ, D., AND TORRES, S. *Handbook of Floating-Point Arithmetic*. Birkhäuser Boston, 2010. ACM G.1.0; G.1.2; G.4; B.2.0; B.2.4; F.2.1., ISBN 978-0-8176-4704-9.
- [283] MULLER, J. M. . E. M.-D. Some issues related to double roundings. Tech. rep., Université de Lyon, Dec 2011.
- [284] MULLER, S. B. J.-M. Some functions computable with a fused-mac. In *Proceedings of the 17th IEEE Symposium on Computer Arithmetic* (2005).
- [285] NANNARELLI, ALBERTO; LANG, T. Power-delay tradeoffs for radix-4 and radix-8 dividers. In *Proceedings of the 1998 international symposium on Low power electronics and design* (New York, NY, USA, 1998), ISLPED ’98, ACM, pp. 109–111.
- [286] NEDIALKOV, N. S. Precision control and exception handling in scientific computing, 1994.
- [287] NEHER, M. Complex standard functions and their implementation in the CoStly library. *ACM Transactions on Mathematical Software* 33, 1 (marzo 2007), 2.
- [288] NETA, B. A sixth-order family of methods for nonlinear equations. *International Journal of Computer Mathematics* 7, 2 (1979), 157–161.

- [289] NGUYEN, H. D., AND GRAILLAT, S. Extended precision with a rounding mode toward zero environment. application on the cell processor, 2008.
- [290] NIEVERGELT, Y. Scalar fused multiply-add instructions produce floating-point matrix arithmetic provably accurate to the penultimate digit. *ACM Transactions on Mathematical Software* 29, 1 (marzo 2003), 27–48.
- [291] NIEVERGELT, Y. Analysis and applications of priest’s distillation. *ACM Trans. Math. Softw.* 30 (December 2004), 402–433.
- [292] NIKMEHR, H., PHILLIPS, B., AND LIM, C.-C. A fast radix-4 floating-point divider with quotient digit selection by comparison multiples. *Comput. J.* 50 (January 2007), 81–92.
- [293] NOBLE, J. M. The control of exceptional conditions in pl/1 object programs. In *IFIP Congress (1)* (1968), pp. 565–571.
- [294] OGASAWARA, T., KOMATSU, H., AND NAKATANI, T. A study of exception handling and its dynamic optimization in java. In *Proceedings of the 16th ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications* (New York, NY, USA, 2001), OOPSLA ’01, ACM, pp. 83–95.
- [295] OGITA, T., RUMP, S., AND OISHIN, S. Accurate sum and dot product. *SIAM Journal on Scientific Computing* 26, 6 (2005), 1955–1988.
- [296] O’LEARY, J., ZHAO, X., GERTH, R., , AND SEGER, C.-J. H. Formally verifying IEEE compliance of floating point hardware. *Intel Technology Journal* 3, 1 (1999), 10.
- [297] OLVER, F. W. J. Error bounds for polynomial evaluation and complex arithmetic. *IMA Journal of Numerical Analysis* 6, 3 (1986), 373–379.
- [298] OVERTON, M. *Numerical Computing with IEEE Floating Point Arithmetic*. SIAM, 2001.
- [299] OVERTON, M. *Cómputo numérico con aritmética de punto flotante IEEE*. No. 19 in Aportaciones matemáticas. SIAM - SMM, 2002. Traducción de Alejandro Casares.
- [300] PAUL, G., AND WILSON, M. W. Should the elementary function library be incorporated into computer instruction sets? *ACM Transactions on Mathematical Software* 2, 2 (junio 1976), 132–142.
- [301] PAYNE, M., AND BHANDARKAR, D. VAX floating point: A solid foundation for numerical computation. *ACM SIGARCH Computer Architecture News* 8, 4 (junio 1980), 22–33.
- [302] PAYNE, M., AND STRECKER, W. Draft proposal for a binary normalized floating point standard. *SIGNUM Newsl.* 14, 2 (octubre 1979), 24–28.

- [303] PAYNE, M. H., AND HANEK, R. N. Radian reduction for trigonometric functions. *SIGNUM Newsl.* 18, 1 (enero 1983), 19–24.
- [304] PERRY, C. Conversion between floating point presentation. *Commun. ACM* 3 (June 1960), 352–.
- [305] PICHAT, M. Correction d’une somme en arithmetique a virgule flottante. *Numerische Mathematik* 19 (1972), 400–406.
- [306] PINKHAM, R. S. On the distribution of first significant digits. *Annals or Mathematical Statistics* 32, 4 (1961), 1223–1230.
- [307] PLAGIANAKOS, V. P., NOUSIS, N. K., AND VRAHATIS, M. N. Locating and computing in parallel all the simple roots of special functions using pvm. *Journal of Computational and Applied Mathematics* 133, 1-2 (2002), 545–554.
- [308] POPOVA, E. D. On a formally correct implementation of IEEE computer arithmetic. *Journal of Universal Computer Science* 1, 7 (julio 1995), 560–569.
- [309] POTRA, F. A. The Kantorovich theorem and interior point methods. *Math. Program.* 102, 1 (febrero 2005), 47–70.
- [310] POTRA, F. A., AND WRIGHT, S. J. Interior-point methods. *Journal of Computational and Applied Mathematics* 124, 1-2 (2000), 281–302.
- [311] PRESS, W. H., TEUKOLSKY, S. A., VETTERLING, W. T., AND FLANNERY, B. P. *Numerical Recipes in C: The Art of Scientific Computing*. Cambridge University Press, New York, NY, USA, 1992.
- [312] PRIEST, D. M. *On Properties of Floating-Point Arithmetics: Numerical Stability and the Cost of Accurate Computations*. PhD thesis, University of California, Berkeley, CA, 1992.
- [313] PRIEST, D. M. Efficient scaling for complex division. *ACM Transactions on Mathematical Software* 30, 4 (diciembre 2004), 389–401.
- [314] QUACH, N., TAKAGI, N., AND FLYNN, M. J. On fast ieee rounding. Tech. rep., Stanford University, Stanford, CA, USA, 1991.
- [315] RAIMI, R. A. On the distribution of first significant figures. *The American Mathematical Monthly* 76, 4 (1969), 342–348.
- [316] REDISH, AND WARD. Environment enquiries. *SIGNUM Newsl.* 6, 1 (enero 1971), 10–15.
- [317] REID, J. Functions for manipulating floating-point numbers. *SIGNUM Newsl.* 14, 4 (diciembre 1979), 11–13.

- [318] REINSCH, C. H. Principles and preferences for computer arithmetic. *SIG-
NUM Newsl.* 14 (March 1979), 12–27.
- [319] RIDDERS, C. F. J. A new algorithm for computing a single root of a
real continuous function. *IEEE Transactions on Circuits and Systems* 26
(1979), 979–980.
- [320] ROBERTAZZI, T. G., AND SCHWARTZ, S. C. Best “ordering” for floating-
point addition. *ACM Transactions on Mathematical Software* 14, 1 (marzo
1988), 101–110.
- [321] ROBERTS, E. S. Implementing exceptions in c. Tech. Rep. 40, Digital
Systems Research Center, march 1989.
- [322] ROMANOVSKY, A., AND SANDÉN, B. Except for exception handling
Ada Lett. XXI (September 2001), 19–25.
- [323] RUMP, S. *Computer-assisted Proofs and Self-validating Methods*. SIAM,
2005, ch. 10, pp. 195–239.
- [324] RUMP, S. Inversion of extremely ill-conditioned matrices in floating-point.
Japan Journal of Industrial and Applied Mathematics 26 (2009), 249–277.
- [325] RUMP, S. M. *Algorithms for verified inclusions—theory and practice*.
Academic Press Professional, Inc., San Diego, CA, USA, 1988, pp. 109–
126.
- [326] RUMP, S. M. Ultimately fast accurate summation. *SIAM J. Scientific
Computing* 31, 5 (2009), 3466–3502.
- [327] RUMP, S. M., OGITA, T., AND OISHI, S. Acurate floating-point summa-
tion. Tech. rep., Hamburg University of Technology, noviembre 2005.
- [328] RUMP, S. M., OGITA, T., AND OISHI, S. Accurate floating-point sum-
mation part i: faithful rounding. Enviado para publicación en SISC, 2005,
revisado en abril de 2007, 2007.
- [329] RUMP, S. M., OGITA, T., AND OISHI, S. Accurate floating-point sum-
mation part ii: sign, k-fold faithful, and rounding to nearest. Enviado para
publicación en SISC, 2005, revisado en abril de 2007, 2007.
- [330] RUSSINOFF, D. M. A mechanically checked proof of correctness of the
AMD K5 floating point square root microcode. *Form. Methods Syst. Des.*
14, 1 (enero 1999), 75–125.
- [331] RYDER, B. G., AND SOFFA, M. L. Influences on the design of excep-
tion handling acm sigsoft project on the impact of software engineering
research on programming language design. *SIGSOFT Softw. Eng. Notes*
28, 4 (July 2003), 29–35.

- [332] SAMELSON, K., AND BAUER, F. L. Sequential formula translation. *Commun. ACM* 3, 2 (Feb. 1960), 76–83.
- [333] SARAFYAN, D. A new method of computation of square roots without using division. *Communications of the ACM* 2, 11 (noviembre 1959), 23–24.
- [334] SCAVO, T. R., AND THOO, J. B. On the geometry of Halley’s method. *The American Mathematical Monthly* 102, 5 (mayo 1995), 417–426.
- [335] SCHEIDT, J. K., AND SCHELIN, C. W. Distribution of floating point numbers. *Computing* 38 (November 1987), 315–324.
- [336] SCHILLING, J. L. Optimizing away c++ exception handling. *SIGPLAN Not.* 33, 8 (Aug. 1998), 40–47.
- [337] SCHRÖDER, E. Über unendlich viele algorithmen zur auflösung der gleichungen. *Mathematische Annalen* 23 (1870), 447–449.
- [338] SCHRÖDER, E. On infinitely many algorithms for solving equations. Tech. rep., University of Maryland, College Park, MD, USA, 1993. Traductor G. W. Stewart.
- [339] SCHULTE, M., SWARTZLANDER, E., AND OMAR, J. Optimal initial approximations for the newton-raphson division algorithm. *Comput. J.* 53, 3-4 (1994), 233–242.
- [340] SCHWARZ, E. M., SCHMOOKLER, M., AND DAO TRONG, S. Fpu implementations with denormalized numbers. *IEEE Trans. Comput.* 54 (July 2005), 825–836.
- [341] SETHI, R., AND ULLMAN, J. D. The generation of optimal code for arithmetic expressions. *J. ACM* 17, 4 (Oct. 1970), 715–728.
- [342] SHAH, H., GÖRG, C., AND HARROLD, M. J. Why do developers neglect exception handling? In *Proceedings of the 4th international workshop on Exception handling* (New York, NY, USA, 2008), WEH ’08, ACM, pp. 62–68.
- [343] SHARKOVSKY, O. M. Co-existence of cycles of a continuous mapping of the line into itself. *Ukranian Math. Zh.* 16, 1 (1964), 61–71. En ruso, traducido en 1995.
- [344] SHARMA, J. R., AND GOYAL, R. K. Fourth-order derivative-free methods for solving non-linear equations. *International Journal of Computer Mathematics* 83, 1 (enero 2006), 101–106.
- [345] SHEWCHUK, J. R. Adaptive precision floating-point arithmetic and fast robust geometric predicates. *Discrete & Computational Geometry* 18 (1996), 305–363.

- [346] SIEGEL, S. F., MIRONOVA, A., AVRUNIN, G. S., AND CLARKE, L. A. Using model checking with symbolic execution to verify parallel numerical programs. In *ISSTA '06: Proceedings of the 2006 international symposium on Software testing and analysis* (New York, NY, USA, 2006), ACM Press, pp. 157–168.
- [347] SMALE, S. Algorithms for solving equations. In *Proceedings of the International Congress of Mathematicians* (1986), pp. 172–195.
- [348] SMALE, S. Newton's method estimates from data at one point. In *The Merging of Disciplines: New Directions in Pure, Applied and Computational Mathematics* (August 1986), Springer, pp. 185–196.
- [349] SMITH, D. M. Algorithm 786: Multiple-precision complex arithmetic and functions. *ACM Transactions on Mathematical Software* 24, 4 (diciembre 1998), 359–367.
- [350] SMITH, R. L. Algorithm 116: Complex division. *Communications of the ACM* 5, 8 (agosto 1962), 435.
- [351] STEELE, JR., G. L. Debunking the expensive procedure call myth or, procedure call implementations considered harmful or, lambda: The ultimate goto. In *Proceedings of the 1977 annual conference* (New York, NY, USA, 1977), ACM '77, ACM, pp. 153–162.
- [352] STERBENZ, P. H. *Floating-point Computation*. Prentice-Hall series in automatic computation. Prentice Hall, 1974.
- [353] STEWART, G. W. A note on complex division. *ACM Transactions on Mathematical Software* 11, 3 (septiembre 1985), 238–241.
- [354] STEWART, G. W. *Errors in variables for numerical analysts*. Recent Advances in Total Least Squares Techniques and Errors-in-Variables Modeling. SIAM, 1997.
- [355] STUDIO, S. O. Numerical computation guide, 2003.
- [356] SU, F. E. Borsuk-Ulam implies brouwer: A direct construction. *Notices Amer. Math. Soc* 104 (1997), 855–859.
- [357] SUMNER, W. N., BAO, T., ZHANG, X., AND PRABHAKAR, S. Coalescing executions for fast uncertainty analysis. In *Proceedings of the 33rd International Conference on Software Engineering* (New York, NY, USA, 2011), ICSE '11, ACM, pp. 581–590.
- [358] SWEENEY, D. An analysis of floating-point addition. *IBM Journal of Research and Development* 4, 1 (1965), 31.

- [359] SYLVAIN COLLANGE, J'EREMIE DETREY, F. D. D. Floating point or LNS: Choosing the right arithmetic on an application basis. *Proceedings of the 9th EUROMICRO Conference on Digital System Design 1* (2006), 10–16.
- [360] TANG, E., BARR, E., LI, X., AND SU, Z. Perturbing numerical calculations for statistical analysis of floating-point program (in)stability. In *Proceedings of the 19th international symposium on Software testing and analysis* (New York, NY, USA, 2010), ISSTA '10, ACM, pp. 131–142.
- [361] TANG, P.-T. P. Table-driven implementation of the exponential function in IEEE floating-point arithmetic. *ACM Transactions on Mathematical Software* 15, 2 (junio 1989), 144–157.
- [362] TANG, P.-T. P. Accurate and efficient testing of the exponential and logarithm functions. *ACM Transactions on Mathematical Software* 16, 3 (septiembre 1990), 185–200.
- [363] TARANTO, D. Binary conversion, with fixed decimal precision, of a decimal fraction. *Commun. ACM* 2 (July 1959), 27–.
- [364] TIEN CHI-CHEN, I. T. H. Storage-efficient representation of decimal data. *Communications of the ACM* 18, 1 (enero 1975), 49–52.
- [365] TRAUB, J. F. Comparison of iterative methods for the calculation of nth roots. *Communications of the ACM* 4, 3 (1961), 143–145.
- [366] TRAUB, J. F. On a class of iteration formulas and some historical notes. *Communications of the ACM* 4, 6 (junio 1961), 276–278.
- [367] TRAUB, J. F. *Iterative Methods for the Solution of Equations*, 1982, segunda ed. Chelsea Publishing Company, Englewood Cliffs, New York, N. J., 1964.
- [368] TSAO, N.-K. On the distributions of significant digits and roundoff errors. *Commun. ACM* 17 (May 1974), 269–271.
- [369] TURING, A. Proposal for development in the mathematics division of an automatic computing engine (ace). Tech. rep., NATIONAL PHYSICAL LABORATORY, 1945.
- [370] TURING, A. Rounding-off errors in matrix processes. *Quarterly Journal of Mechanics & Applied Mathematics* 1, 1 (1948), 287–308.
- [371] TURING, A. M. On computable numbers, with an application to the entscheidungsproblem. *Proceedings of the London Mathematical Society* 42 (1936), 230–265.
- [372] UKKONEN, E. On the calculation of the effects of roundoff errors. *ACM Trans. Math. Softw.* 7, 3 (Sept. 1981), 259–271.

- [373] URABE, M. Roundoff error distribution in fixed-point multiplication and a remark about the rounding rule. *SIAM Journal of Numerical Analysis* 5, 2 (june 1968), 202–210.
- [374] USPENSKY. *Teoría de Ecuaciones*, 2005 ed. Limusa, 1948.
- [375] VAN ELLEN, T., AND HASSELBRING, W. Extended exceptions for contingencies. In *Proceedings of the 2008 RISE/EFTS Joint International Workshop on Software Engineering for Resilient Systems* (New York, NY, USA, 2008), SERENE '08, ACM, pp. 107–116.
- [376] VERDONK, B., CUYT, A., AND VERSCHAEREN, D. A precision- and range-independent tool for testing floating-point arithmetic i: basic operations, square root, and remainder. *ACM Transactions on Mathematical Software* 27, 1 (marzo 2001), 92–118.
- [377] VERDONK, B., CUYT, A., AND VERSCHAEREN, D. A precision- and range-independent tool for testing floating-point arithmetic ii: conversions. *ACM Transactions on Mathematical Software* 27, 1 (marzo 2001), 119–140.
- [378] VERSCHAEREN, D., CUYT, A., AND VERDONK, B. On the need for predictable floating-point arithmetic in the programming languages fortran 90 and c/c++. *SIGPLAN Not.* 32, 3 (marzo 1997), 57–64.
- [379] VOLDER, J. The cordic computing technique. In *Papers presented at the the March 3-5, 1959, western joint computer conference* (New York, NY, USA, 1959), IRE-AIEE-ACM '59 (Western), ACM, pp. 257–261.
- [380] VON NEUMANN, AND GOLDSTINE. Numerical inverting of matrices of high order. *Bulletin of American Mathematical Society* 53 (1947), 1021–1099.
- [381] VON NEUMANN, AND GOLDSTINE. Numerical inverting of matrices of high order ii. *Proceedings of American Mathematical Society* 2, 2 (abril 1951), 188–202. 15 páginas.
- [382] VON NEUMANN, J. First draft of a report to the edvac. Tech. rep., Ballistics Research Laboratory, 1945.
- [383] WADEY, W. G. Floating-point arithmetics. *Journal of the ACM* 7, 2 (abril 1960), 129–139. Remington Rand UNIVAC.
- [384] WANG, X., AND HAN, D. On the dominating sequence method in the point estimates and smale's theorem. *Scientia Sinica* 33 (1989), 135–144.
- [385] WEERAKOON, S., AND FEMANDO, T. A variant of newton's method with accelerated third-order convergence. *Applied Mathematics Letters* 13, 8 (noviembre 2000), 87–93.
- [386] WEGSTEIN, J. H. From formulas to computer oriented language. *Commun. ACM* 2, 3 (Mar. 1959), 6–8.

- [387] WEIMER, W. R. *Exceptional Situations And Program Reliability*. PhD thesis, University of California, Berkeley, 2005.
- [388] WG5. Technical report for floating point exception handling. Tech. rep., ISO/IEC JTC1/SC22/WG5, 1996.
- [389] WILKINS, G., AND GU, M. A modified brent's method for finding zeros of functions. *Numerische Mathematik On line first* (jun 2012), 1–12.
- [390] WILKINSON, FOX, L., AND HUSKEY, H. D. Notes on the solution of algebraic linear simultaneous equations. *Quarterly Journal of Mechanics & Applied Mathematics 1* (1948), 149–173.
- [391] WILKINSON, J. H. The evaluation of the zeros of ill-conditioned polynomials. I. *Numerische Mathematik 1* (1959), 150–166.
- [392] WILKINSON, J. H. The evaluation of the zeros of ill-conditioned polynomials. II. *Numerische Mathematik 1* (1959), 167–180.
- [393] WILKINSON, J. H. Error analysis of floating-point computation. *Numerische Mathematik 2* (1960), 319–340. 10.1007/BF01386233.
- [394] WILKINSON, J. H. Error analysis of direct methods of matrix inversion. *J. ACM 8* (July 1961), 281–330.
- [395] WILKINSON, J. H. *Rounding Errors in Algebraic Processes*. Prentice-Hall, Englewood Cliffs, New Jersey, 1964.
- [396] WILKINSON, J. H. *The algebraic eigenvalue problem*. Oxford University Press, Inc., New York, NY, USA, 1988.
- [397] WOLFE, J. M. Reducing truncation errors by programming. *Communications of the ACM 7*, 6 (junio 1964), 355–356.
- [398] WOODCOCK, J., LARSEN, P. G., BICARREGUI, J., AND FITZGERALD, J. Formal methods: Practice and experience. *ACM Comput. Surv. 41*, 4 (Oct. 2009), 19:1–19:36.
- [399] YEMINI, S. An axiomatic treatment of exception handling. In *Proceedings of the 9th ACM SIGPLAN-SIGACT symposium on Principles of programming languages* (New York, NY, USA, 1982), POPL '82, ACM, pp. 281–288.
- [400] YEMINI, S., AND BERRY, D. M. A modular verifiable exception handling mechanism. *ACM Trans. Program. Lang. Syst. 7* (April 1985), 214–243.
- [401] YOHE, J. M. Roundings in floating-point arithmetic. *IEEE Trans. Comput. C-22*, 6 (junio 1973), 577–586.

- [402] ZHANG, L., KRINTZ, C., AND NAGPURKAR, P. Supporting exception handling for futures in java. In *Proceedings of the 5th international symposium on Principles and practice of programming in Java* (New York, NY, USA, 2007), PPPJ '07, ACM, pp. 175–184.
- [403] ZHENGDA, H. A note on the Kantorovich theorem for Newton iteration. *Journal of Computational and Applied Mathematics* 47, 2 (1993), 211–217.
- [404] ZIELKE, G., AND DRYGALLA, V. Genaue lösung linearer gleichungssysteme. *GAMM Mitt. Ges. Angew. Math. Mech.* 26 (2003), 7–107.
- [405] ZIV, A. Fast evaluation of elementary mathematical functions with correctly rounded last bit. *ACM Transactions on Mathematical Software* 17, 3 (septiembre 1991), 410–423.

Índice alfabético

- a-b, 83
- Accurate Mathematical Library, 85
- ACE, 82
- aceleración, 236
- aceleración convexa, 246
- acotamiento de raíces, 255, 270
- Ada, 101
- aislamiento de raíces, 255, 270
- algoritmo de Euclides, 276
- algoritmo del banquero, 56
- AMD, 114
- antisimetría, 314
- aproximación de Taylor, 226
- aproximación inicial, 228
- apuntador a función, 199
- argumento reducido, 314
- aritmética de cifras significativas, 381
- aritmética de intervalos, 29, 164, 226, 259, 378
- aritmética de Karlsruhe, 380
- aritmética de rango, 259
- aritmética decimal, 51
- aritmética pesimista, 379
- aritmética de punto fijo, 11
- aritmética de punto flotante, 10, **35**
- asíntota, 198

- backward error, 128, **348**, 360
- banderas de estado, 74, 214
- banderas de excepción, 59, 137
- base, **15**, 20, 36, 48, 383
- Basic, 101, 116
- bcd, 76
- bcd8421, 52
- BigDecimal class, 51
- BigEndian, 120

- bisección, 215
- bit de guardia, **37**, 38, 41, 57, 150
- bit de redondeo, 57, 150
- bit de signo, 69, 72, 73, 120, 122
- bit implícito, **18**, 31, 49
- bit pegajoso, *véase* sticky bit
- bits, 17
- BLAS, 71
- bola cerrada, 381
- bola unitaria, 365
- Borneo, 116

- C++, 60
- C99, 46, 67, 69, 71, 100
- c99, 137
- cadena de Sturm, 275
- cálculo simbólico, 271
- cambio de base, 22, 54, 368
- campo de bits, 122
- cancelación catastrófica, **38**, 40, 87, 88, 293, 326, 341
- cast, 120
- cero simple, *véase* raíces
- cifras significativas, **17**, 44, 206, 227, **342**
 - pérdida de, 26, 83
- complejidad algorítmica, **349**
- complejidad espacial, 150, 152
- complejidad temporal, 150, 181
- complemento a 2, 52, 376
- condición de Lipschitz, 187, 226
- condicionamiento, 13, 75, 119, 147, 177, 254, 265, **347**, 357
- conjugado, 352
- conjugado complejo, **90**, 254, 291
- conjunto convexo, 225, **367**

- constante asintótica del error, 181
 constante de Lipschitz, 367
 convergencia
 cúbica, 241, 242, 244, 245, 247, 249, 305–307
 cuártica, 247, 306, 307
 cuadrática, 227, 237, 241, 299, 302
 global, 305
 lineal, 203, 236, 241, 247, 301, 307
 local, 213, 299
 n -ésima, 249
 sexta, 222
 superior, 241
 superlineal, 183, 210–214, 216, 217, 305
 convergencia global, 183
 convergencia local, 183
 conversión, 61
 decimal $\longleftrightarrow$ binario, 54
 convexidad logarítmica, 227, **367**
 convexidad logaritmica, 265
 corrección de Newton, 225
 corrección de Newton, 176
 correctamente redondeado, 144
 CoStLy, 381
 Cray, 102
 cuadratura, 244

 Dec, 44
 decimal codificado en binario, 52
 decNumber, 51
 δ^2 de Aitken, 236
 densely packed decimal, 53
 codificación doble, 76
 delet, 53
 depuración, 75, 76
 desborde, *véase* overflow
 Descartes, 269
 desigualdad del triángulo, 365
 desnormalizado, 19
 destilación, 159, **161**, 162
 doble destilación, 161
 diagonal principal, 355
 diferencias divididas, 212
 dígito de guardia, *véase* bit de guardia
 discriminante, 88, 213, 214, **290**, 293
 distancia, 268, **365**
 división entre cero, 73
 división sintética, 271, 278
 doble redondeo, 65, 87
 Dyninst, 39

 ecuación cuadrática, **290**
 ecuaciones de Vieta, 302
 ecuaciones no lineales, 144
 efecto numérico colateral, 74
 eigenvalores, 308
 ELEFUNT, 83, 116
 endomorfismo, 224, **366**
 eps, 27
 épsilon de máquina, **26**
 error
 absoluto, 30, 197, 264, 280, 282, **340**, 341, 343, 347, 360, 362
 de redondeo, 249, 279, **349**
 porcentaje de, 31, **341**
 relativo, 28, 30, 31, 35, 38, 57, 65, 88, 134, 148, 197, 203, 204, 206, 238, 257, 280, **341**, 343, 346, 347, 362, 376, 381
 escalamiento, 90, 91, 94, 100, 134, 163, 214, 293
 escalamiento de raíces, 255
 espacio, 363
 completo, 269, **366**
 de Banach, 269, **366**
 métrico, **365**
 normado, 365
 estabilidad, 119, 160, 177, 238, 254, **345**
 estimación de punto, 278
 eta, 41
 evaluación compensada de Horner, 267
 evaluación de expresiones, 60, 69, 74
 exactitud, 22, 26, 51, 137, **343**, 349, 359, 374, 379, 383
 excepción, **73**, 76, 100, 108
 excepciones, 95
 exponente, **16**, 20, 29, 122, 133
 máximo, 48
 mínimo, 48
 falsa posición, 215

- fenómeno de Runge, 245
- flib++, 380
- fma, 45, 64, 68, 260
 - doble redondeo, 64
- fordward error, 128, 360
- fork, 77
- forma polar, 352
- formato destino, 60, 64
- formato externo, **61**
- formato intermedio, 60, 63
- fórmula general, 212, 283, **290**, 293
- Fortran, 83, 107, 114
- Fortran 2003, 46, 71, 100
- Fortran 90, 381
- forward error, **348**
- Fourier, 363
- fracciones continuas, 22, **371**, 373
- Fraley-Walther, 44
- frexp, 123, 295
- función
 - contractiva, **366**
 - convexa, 225, 227, 367
 - función de Lipschitz, 366
- función exponencial, 144
- funciones analíticas, 213
- funciones elementales, 76
- FunPack, 83
- fused multiply-add, *véase* fma

- gcc, 260
- gsl, 123, 220

- hipotenusa, 90, 128, 293, 364
- homeomorfismo, 367
- Horner., 67

- IBM, 58, 85
- IEC, 100
- IEC 60559, 46
- IEEE 854, 45
- IEEE-754R, 45
- IeeeCC754, 117
- IEEE_GET_FLAGS, 107
- IFIP, 45
- IMSL, 117
- incertidumbre, 382

- inclusión isotónica, 378
- índice de eficiencia, **182**
- índice de nivel simétrico, 377
- inestabilidad, 375
- inexacto, 73, 111, 112
- infinito, 23, 69
- Intel, 58, 62, 68, 114
- interfaz, 148, 199, 291
- interpolación cuadrática inversa, 217
- intervalo de convergencia, 314
- intervalos disjuntos, 270
- INTLAB, 381
- INTLIB, 381
- isinf, 72
- isNaN, 120
- isnan, 72
- ISO/IEC-10967, 46
- Itanium, 67
- ItvCalc, 381

- Java, 51, 116

- K-C-S (Kahan-Coonen-Stone), 44

- LCAS, 45
- ldexp, 123, 295
- lenguaje Z, 113
- ley de los grandes números, 282
- LIA, 100
- LittleEndian, 120
- longitud de intervalo, 378

- máximo común divisor, 276
- metodo de Halley, 241
- método de Laguerre, 307
- método paralelizable, 302
- métrica, 268
- módulo, 93, 282, 301
- modulo, 352
- machar, 116
- MACSYMA, 222
- malloc, 257
- mantisa, **15**, 19, 20, 29, 31, 36, 37, 57, 133, 279
- math.h, 72
- Mathematica, 166, 383

- MATLAB, 381
- MatLab, 116
- Matlab, 166, 216
- matriz, **354**
- matriz adjunta, 308
- matriz conjugada, 356
- matriz Hermitiana, 356
- matriz inversa, 356
- matriz simétrica, 356
- matriz transpuesta, 356
- máximo común divisor, 267
- método de Brent, 216
- método de Dekker, 215
- método de dos pasos, 244
- metodo de Halley
 - forma irracional, 241
 - forma racional, 242
- método de Horner, **258**
- método de la secante, **209**, 218
- método de Laguerre, 306
- método de Müller, 212
- método de Ridder, 214
- método gráfico, **178**
- métodos sin derivadas, 244
- métrica, 365
- modo de redondeo, **348**, 378
- módulo, **364**
- monotonicidad, 195, 261, 281, 313
- Monte Carlo, 119, 282, 382
- MPCHECK, 117
- MPFR, 129
- multihilo, 77
- multiplicidad, **175**, 210, **252**, 270, 301

- NAG, 117
- NaN, **23**, 69, 204, 281, 291, 293
 - qNaN, 72
 - sNaN, 72
- NetLib, 116, 216
- Newton-Cotes, 244
- Newton-Raphson, 68
- nextAfter, 60
- nextDown, 60
- nextUp, 177
- nextUp, 60, 71
- norma, 90, 266, 364
 - de un polinomio, 268
- norma de IEEE, 6, 144
- norma de una matriz, 355
- normalización, 26
- notación científica, 15
 - desnormalizada, 17
 - normalizada, 17, 31
- notación expandida, 15
- número complejo, **90**, 254, 266
- número de condición, *véase* condicio-
namiento
- número imaginario, **351**
- números complejos, 351
- números de Fibonacci, 284
- números subnormales, 20, **31**, 83, 324

- operación inexacta, 69
- operación inválida, **23**, 73
- operaciones elementales, **18**, 145
- operador de Weierstrass, 302
- orden de convergencia, 181, 227
- overflow, **25**, 36, 69, 73, 75, 90, 91, 93,
100, 101, 111, 112, 128, 132,
134, 135, 150, 152, 293, 377

- paranoia, 116
- parte imaginaria, **292**
- parte real, **292**
- Payne-Strecker, 44
- periodicidad, 314
- perturbación, **349**
- pipeline, 67, 165
- plano complejo, 351
- Playstation, 125
- polinomio, **252**
- polinomio de Taylor, 224, **358**, 362
- polinomio pérfido, 285
- polinomio real, **254**
- polinomios de Chebyshev, 289
- polinomios de Mignotte, 288
- polinomios mónicos, **253**, 308
- POSIX, 108
- pow, 257
- PowerPC, 67, 68
- precisión, **16**, 17, 20, 22, 48, 150, 258,
266, **343**, 374, 383

- arbitraria, 6, 112, 129
- cuádruple, 45, 50
- doble, 50, 91
- extendida, 50
- extra, 116, 294
- infinita, 60
- intermedia, 177
- múltiple, 84, 100, 294
- simple, 5, 206
- precisión aumentada, 84, 130
- precisión múltiple, 274
- presubstitución, 70, 76, 100–102
- primitivas de la unidad, 289
- `printf`, 63
- programación estructurada, 96
- promedio, 36
- propagación, 69, 76, 99, 111, 292
- propagación del error, 382
- prueba exhaustiva, 281
- pruebas formales, 113
- punto decimal, 373
- punto fijo, 82, **185**, 224, 244, 248, **373**
- punto flotante, 19, **20**
 - norma de , 45
- raíces aisladas, 302
- raíces múltiples, 306
- raíz de una ecuación, 173
- raíces complejas, 180, 282, **291**
- raíces múltiple, 233
- raíces múltiples, 217, 238, 249, 275
- raíces reales, 180, 284
- raíz, **175**
- razón aurea, 284
- reales extendidos, 33
- redondeo, 23, **27**, 37, 62
 - al más cercano, 56
 - TiesToAway, 56
 - TiesToEven, 56
 - default, 56
 - direccionado, 56
 - TowardNegative, 56, 378
 - TowardPositive, 56, 378
 - TowardZero, 56
 - doble redondeo, 64
- redondeo fiel, 29, 87, 124
- reducción, 160
- reducción de grado, 255, **271**, 278, 288
- reducción de rango, 257, 314
- refinamiento, 271
- refinamiento de raíces, 307
- registro flotante, 64, 123, 154
- registros de estado y control, 59
- regla de los signos, 269
- Regla de Ruffini, 271
- residuo, 359
- resultado intermedio, 60
- RS6000, 67, 120
- `scalb`, 123, 135
- semimorfismo, 27
- semiperímetro, 88
- sensibilidad, 75, 179, 245, 254, 349
- serie de Taylor, **359**
- señal, 74, 108
- SIGFPE, 108
- `signal.h`, 108
- signo, **20**
- simetría, 314
- sistema normalizado, 23
- sistema numérico
 - exponencial, 377
 - logarítmico , 376
- sistema posicional, 373
- sobrecarga de operadores, 60
- Solaris, 104
- Sparc, 58
- SRTEST, 117
- stack overflow, 101
- sticky bit, **58**, 62, 116, 150
- subnormales, 75
- sucesión de Cauchy, 366
- suma, 145
- suma compensada, 24, **145**, 158
- suma parcial, 362
- suma recursiva, 150
- SUN, 130
- Sun Microsystems, 44, 117
- tambaleo, 30, 263
- teorema de Kantorovich, 225

teorema de no intercambio Sharkovsky, 189
teorema de punto fijo de Banach, 187
teorema de punto fijo de Brouwer, 186
teorema del residuo, 362
teorema del valor medio, 202
teorema fundamental del álgebra, 253
TestBase, 117
TestFloat, 117
transformación libre de error, **85**, 161, 164, 267
transformaciones libres de errores, 124, 267
trap, 100
truncamiento, **28**, 37, 62

UCBTEST, 117
ufp, **16**, 18, 58
ulp, **16**, 18, 63, 134
ulp, 267
underflow, **25**, 31, 36, 73, 90, 91, 100, 111, 133–135, 146, 293
underflow gradual, 31, 44, 49, 100, 116, 155
unidad aritmético lógica, 376
unidad de redondeo, **27**, 264, 374
union de C, 121
units in last position, *véase* ulp
uso informacional, 182

valor correctivo, 124
valor propio, 356
valores especiales, 20, 21
valores propios, 251
VAX, 44
vector, **353**
vector propio, 356
vector unitario, 355
velocidad de convergencia, 181
verificación formal, 114

wobbling, 30, 51, 376

x86, 120